

LeetLLM

A Hands-On Companion to Transformer Inference on Apple Silicon

The LeetLLM Project

2026

Contents

Preface	1
How to use this book	1
Build and notation	1
1 Start Here: Build an LLM Inference Engine	2
1.1 What LeetLLM Studio is	2
1.2 Who the course is for	2
1.3 How to use the Studio	3
1.4 Trust and local execution	3
1.5 The inference path in plain language	3
1.5.1 Tokens, vocabulary, and tokenization	3
1.5.2 Vectors, tensors, shapes, and weights	4
1.5.3 Embeddings and the residual stream	4
1.5.4 Queries, keys, values, and attention	4
1.5.5 Feed-forward branch, logits, and sampling	4
1.5.6 Prefill, decode, and the KV cache	5
1.6 CPU, Metal, and evidence	5
1.7 How the course is organized	5
1.8 Completion checklist	7
I Foundations	8
2 Vector Dot Product	9
2.1 Why this exists	9
2.2 Learning outcomes	9
2.3 Prerequisites	9
2.4 Vocabulary	9
2.5 Math from first principles	10
2.6 Shape, layout, and dtype contract	10
2.7 Performance model	12
2.8 CPU reference path	12
2.8.1 Checkpoint	12
2.9 Correctness method and reduction order	13
2.10 Metal mapping	13
2.10.1 Memory spaces in this kernel	13
2.10.2 Why every barrier exists	15
2.11 Implementation checkpoints	15
2.12 Hints	15
2.13 Controlled experiments	16
2.13.1 Benchmark the operator	16
2.13.2 Required size-sweep experiment	16
2.14 Tradeoffs	17
2.15 Engine integration	17
2.16 Completion checklist	17

2.17	Exercise source files	18
2.17.1	Sources/LeetLLMExercises/P001VectorDotExercise.swift	18
2.17.2	Sources/LeetLLMExercises/P001VectorDotMetal.swift	18
2.17.3	Sources/LeetLLMExercises/Metal/P001VectorDot.metal	18
2.18	Worked solution	19
2.18.1	Implementation points	19
2.18.2	Code map	20
2.18.3	Complete canonical listings	20
2.18.4	Sources/LeetLLMSolutions/P001VectorDotMetal.swift	20
2.18.5	Sources/LeetLLMSolutions/P001VectorDotSolution.swift	20
2.18.6	Sources/LeetLLMSolutions/Metal/P001VectorDot.metal	21
II	Tensors and Dense Linear Algebra	22
3	Tensor Storage and Strides	23
3.1	Why this exists	23
3.2	Learning outcomes	23
3.3	Prerequisites	23
3.4	Vocabulary	23
3.5	Math from first principles	24
3.5.1	Worked numbers	24
3.6	Shape, layout, and dtype contract	25
3.7	CPU reference path	25
3.8	Correctness method	26
3.9	Performance model	26
3.10	Metal mapping	26
3.11	Implementation checkpoints	26
3.12	Controlled experiments	27
3.12.1	Experiment 1: offset table	27
3.12.2	Experiment 2: view versus copy	27
3.12.3	Experiment 3: traversal locality	27
3.13	Engine integration	27
3.14	Tradeoffs	27
3.15	Hints	28
3.16	Completion checklist	28
3.17	Exercise source files	29
3.17.1	Sources/LeetLLMExercises/P002TensorStorageExercise.swift	29
3.18	Worked solution	30
3.18.1	Implementation points	30
3.18.2	Code map	31
3.18.3	Complete canonical listings	31
3.18.4	Sources/LeetLLMSolutions/P002TensorStorageSolution.swift	31
4	Transpose and Tiled Copy	32
4.1	Why this exists	32
4.2	Learning outcomes	32
4.3	Prerequisites	32
4.4	Vocabulary	32
4.5	Math from first principles	33
4.6	Shape, layout, and dtype contract	33

4.7	CPU reference path	34
4.8	Correctness method	34
4.9	Performance model	34
4.10	Metal mapping	35
4.10.1	Bounds and barriers	35
4.11	Implementation checkpoints	35
4.12	Controlled experiments	37
4.12.1	Experiment 1: shape sweep	37
4.12.2	Experiment 2: skinny matrices	37
4.12.3	Experiment 3: copy amortization	37
4.13	Engine integration	37
4.14	Tradeoffs	37
4.15	Hints	38
4.16	Completion checklist	38
4.17	Exercise source files	39
4.17.1	Sources/LeetLLMExercises/P003TransposeExercise.swift	39
4.17.2	Sources/LeetLLMExercises/P003TransposeMetal.swift	39
4.17.3	Sources/LeetLLMExercises/Metal/P003Transpose.metal	39
4.18	Worked solution	41
4.18.1	Implementation points	41
4.18.2	Code map	42
4.18.3	Complete canonical listings	42
4.18.4	Sources/LeetLLMSolutions/P003TransposeMetal.swift	42
4.18.5	Sources/LeetLLMSolutions/P003TransposeSolution.swift	43
4.18.6	Sources/LeetLLMSolutions/Metal/P003Transpose.metal	43
5	Matrix-Vector Multiplication	44
5.1	Why this exists	44
5.2	Learning outcomes	44
5.3	Prerequisites	44
5.4	Vocabulary	44
5.5	Math from first principles	45
5.5.1	Worked numbers	45
5.6	Shape, layout, and dtype contract	45
5.7	CPU reference path	46
5.8	Correctness method	46
5.9	Performance model	46
5.10	Metal mapping	47
5.11	Implementation checkpoints	47
5.12	Controlled experiments	48
5.12.1	Experiment 1: vary rows only	48
5.12.2	Experiment 2: cross the group width	48
5.12.3	Experiment 3: same FLOPs, different shape	48
5.13	Engine integration	48
5.14	Tradeoffs	49
5.15	Hints	49
5.16	Completion checklist	49
5.17	Exercise source files	50
5.17.1	Sources/LeetLLMExercises/P004GEMVExercise.swift	50
5.17.2	Sources/LeetLLMExercises/P004GEMVMetal.swift	50

5.17.3	Sources/LeetLLMExercises/Metal/P004GEMV.metal	50
5.18	Worked solution	52
5.18.1	Implementation points	52
5.18.2	Code map	53
5.18.3	Complete canonical listings	53
5.18.4	Sources/LeetLLMSolutions/P004GEMVMetal.swift	53
5.18.5	Sources/LeetLLMSolutions/P004GEMVSolution.swift	54
5.18.6	Sources/LeetLLMSolutions/Metal/P004GEMV.metal	54
6	Matrix-Matrix Multiplication	55
6.1	Why this exists	55
6.2	Learning outcomes	55
6.3	Prerequisites	55
6.4	Vocabulary	55
6.5	Math from first principles	56
6.5.1	Worked numbers	56
6.6	Shape, layout, and dtype contract	56
6.7	CPU reference path	57
6.8	Correctness method	57
6.9	Performance model	57
6.10	Metal mapping	58
6.11	Implementation checkpoints	60
6.12	Controlled experiments	60
6.12.1	Experiment 1: square size sweep	60
6.12.2	Experiment 2: decode-like versus prefill-like	60
6.12.3	Experiment 3: edge-tile efficiency	60
6.13	Engine integration	60
6.14	Tradeoffs	61
6.15	Hints	61
6.16	Completion checklist	61
6.17	Exercise source files	62
6.17.1	Sources/LeetLLMExercises/P005GEMMExercise.swift	62
6.17.2	Sources/LeetLLMExercises/P005GEMMMetal.swift	62
6.17.3	Sources/LeetLLMExercises/Metal/P005GEMM.metal	62
6.18	Worked solution	64
6.18.1	Implementation points	64
6.18.2	Code map	65
6.18.3	Complete canonical listings	65
6.18.4	Sources/LeetLLMSolutions/P005GEMMMetal.swift	65
6.18.5	Sources/LeetLLMSolutions/P005GEMMSolution.swift	66
6.18.6	Sources/LeetLLMSolutions/Metal/P005GEMM.metal	66
7	Build a Roofline Measurement	68
7.1	Why this exists	68
7.2	Learning outcomes	68
7.3	Prerequisites	68
7.4	Vocabulary	68
7.5	Math from first principles	69
7.5.1	Worked numbers	69
7.6	Shape, layout, and dtype contract	70
7.7	CPU reference path	70

7.8	Correctness method	71
7.9	Performance model	71
7.10	Metal mapping	72
7.11	Implementation checkpoints	72
7.12	Controlled experiments	72
7.12.1	Experiment 1: move across the ridge	73
7.12.2	Experiment 2: fixed model, changed boundary	73
7.12.3	Experiment 3: peak sensitivity	73
7.12.4	Required report record	73
7.13	Engine integration	73
7.14	Tradeoffs	73
7.15	Hints	74
7.16	Completion checklist	74
7.17	Exercise source files	75
7.17.1	Sources/LeetLLMExercises/P006RooflineExercise.swift	75
7.18	Worked solution	76
7.18.1	Implementation points	76
7.18.2	Code map	77
7.18.3	Complete canonical listings	77
7.18.4	Sources/LeetLLMSolutions/P006RooflineSolution.swift	77
III	Neural-Network Operators	78
8	ReLU, GELU, and SiLU	79
8.1	Why this exists	79
8.2	Learning outcomes	79
8.3	Prerequisites	79
8.4	Vocabulary	79
8.5	Math from first principles	80
8.5.1	Worked numerical example	80
8.6	Shape, layout, and dtype contract	81
8.7	CPU reference path	81
8.8	Correctness method	81
8.9	Performance model	82
8.10	Metal mapping	82
8.11	Implementation checkpoints	82
8.12	Controlled experiments	82
8.12.1	Experiment A: same bytes, different math	82
8.12.2	Experiment B: launch overhead	83
8.12.3	Experiment C: separate versus fused use	83
8.13	Engine integration	83
8.14	Tradeoffs	83
8.15	Hints	83
8.16	Completion checklist	83
8.17	Exercise source files	84
8.17.1	Sources/LeetLLMExercises/P007ActivationExercise.swift	84
8.17.2	Sources/LeetLLMExercises/P007ActivationMetal.swift	84
8.17.3	Sources/LeetLLMExercises/Metal/P007Activation.metal	84
8.18	Worked solution	85
8.18.1	Implementation points	85

8.18.2	Code map	86
8.18.3	Complete canonical listings	86
8.18.4	Sources/LeetLLMSolutions/P007ActivationMetal.swift	86
8.18.5	Sources/LeetLLMSolutions/P007ActivationSolution.swift	86
8.18.6	Sources/LeetLLMSolutions/Metal/P007Activation.metal	87
9	SwiGLU Feed-Forward Gate	88
9.1	Why this exists	88
9.2	Learning outcomes	88
9.3	Prerequisites	88
9.4	Vocabulary	88
9.5	Math from first principles	89
9.5.1	Worked numerical example	89
9.6	Shape, layout, and dtype contract	90
9.7	CPU reference path	90
9.8	Correctness method	91
9.9	Performance model	91
9.10	Metal mapping	91
9.11	Implementation checkpoints	92
9.12	Controlled experiments	92
9.12.1	Experiment A: hidden-width sweep	92
9.12.2	Experiment B: separate versus fused gate	92
9.12.3	Experiment C: output width	92
9.13	Engine integration	92
9.14	Tradeoffs	92
9.15	Hints	93
9.16	Completion checklist	93
9.17	Exercise source files	94
9.17.1	Sources/LeetLLMExercises/P008SwiGLUExercise.swift	94
9.17.2	Sources/LeetLLMExercises/P008SwiGLUMetal.swift	94
9.17.3	Sources/LeetLLMExercises/Metal/P008SwiGLU.metal	94
9.18	Worked solution	96
9.18.1	Implementation points	96
9.18.2	Code map	97
9.18.3	Complete canonical listings	97
9.18.4	Sources/LeetLLMSolutions/P008SwiGLUMetal.swift	97
9.18.5	Sources/LeetLLMSolutions/P008SwiGLUSolution.swift	97
9.18.6	Sources/LeetLLMSolutions/Metal/P008SwiGLU.metal	98
10	Numerically Stable Softmax	99
10.1	Why this exists	99
10.2	Learning outcomes	99
10.3	Prerequisites	99
10.4	Vocabulary	99
10.5	Math from first principles	100
10.5.1	Worked numerical example	101
10.6	Shape, layout, and dtype contract	101
10.7	CPU reference path	101
10.8	Correctness method	101
10.9	Performance model	102
10.10	Metal mapping	102

10.11	Implementation checkpoints	102
10.12	Controlled experiments	102
10.12.1	Experiment A: translation invariance	103
10.12.2	Experiment B: reduction boundaries	103
10.12.3	Experiment C: row parallelism	103
10.13	Engine integration	103
10.14	Tradeoffs	103
10.15	Hints	103
10.16	Completion checklist	103
10.17	Exercise source files	105
10.17.1	Sources/LeetLLMExercises/P009SoftmaxExercise.swift	105
10.17.2	Sources/LeetLLMExercises/P009SoftmaxMetal.swift	105
10.17.3	Sources/LeetLLMExercises/Metal/P009Softmax.metal	105
10.18	Worked solution	106
10.18.1	Implementation points	106
10.18.2	Code map	107
10.18.3	Complete canonical listings	107
10.18.4	Sources/LeetLLMSolutions/P009SoftmaxMetal.swift	107
10.18.5	Sources/LeetLLMSolutions/P009SoftmaxSolution.swift	107
10.18.6	Sources/LeetLLMSolutions/Metal/P009Softmax.metal	108
11	RMSNorm	109
11.1	Why this exists	109
11.2	Learning outcomes	109
11.3	Prerequisites	109
11.4	Vocabulary	109
11.5	Math from first principles	109
11.5.1	Worked numerical example	110
11.6	Shape, layout, and dtype contract	110
11.7	CPU reference path	112
11.8	Correctness method	112
11.9	Performance model	112
11.10	Metal mapping	112
11.11	Implementation checkpoints	113
11.12	Controlled experiments	113
11.12.1	Experiment A: epsilon sensitivity	113
11.12.2	Experiment B: accumulation precision	113
11.12.3	Experiment C: row width and row count	113
11.13	Engine integration	113
11.14	Tradeoffs	113
11.15	Hints	114
11.16	Completion checklist	114
11.17	Exercise source files	115
11.17.1	Sources/LeetLLMExercises/P010RMSNormExercise.swift	115
11.17.2	Sources/LeetLLMExercises/P010RMSNormMetal.swift	115
11.17.3	Sources/LeetLLMExercises/Metal/P010RMSNorm.metal	115
11.18	Worked solution	117
11.18.1	Implementation points	117
11.18.2	Code map	118
11.18.3	Complete canonical listings	118

11.18.4 Sources/LeetLLMSolutions/P010RMSNormMetal.swift	118
11.18.5 Sources/LeetLLMSolutions/P010RMSNormSolution.swift	118
11.18.6 Sources/LeetLLMSolutions/Metal/P010RMSNorm.metal	119
12 Residual Streams and Precision	120
12.1 Why this exists	120
12.2 Learning outcomes	120
12.3 Prerequisites	120
12.4 Vocabulary	120
12.5 Math from first principles	121
12.5.1 Worked numerical example	122
12.6 Shape, layout, and dtype contract	122
12.7 CPU reference path	122
12.8 Correctness method	122
12.9 Performance model	123
12.10 Metal mapping	123
12.11 Implementation checkpoints	123
12.12 Controlled experiments	123
12.12.1 Experiment A: update magnitude	123
12.12.2 Experiment B: layer count	124
12.12.3 Experiment C: downcast placement	124
12.12.4 Experiment D: order	124
12.13 Engine integration	124
12.14 Tradeoffs	124
12.15 Hints	124
12.16 Completion checklist	124
12.17 Exercise source files	126
12.17.1 Sources/LeetLLMExercises/P011ResidualPrecisionExercise.swift	126
12.18 Worked solution	127
12.18.1 Implementation points	127
12.18.2 Code map	128
12.18.3 Complete canonical listings	128
12.18.4 Sources/LeetLLMSolutions/P011ResidualPrecisionSolution.swift	128
13 Fuse Norm, Scale, and Projection Input	129
13.1 Why this exists	129
13.2 Learning outcomes	129
13.3 Prerequisites	129
13.4 Vocabulary	129
13.5 Math from first principles	130
13.5.1 Worked numerical example	130
13.6 Shape, layout, and dtype contract	130
13.7 CPU reference path	131
13.8 Correctness method	131
13.9 Performance model	131
13.10 Metal mapping	133
13.11 Implementation checkpoints	133
13.12 Controlled experiments	133
13.12.1 Experiment A: output-row sweep	133
13.12.2 Experiment B: feature-width sweep	133
13.12.3 Experiment C: baseline versus fused traffic	134

13.12.4 Experiment D: parity under scale	134
13.13 Engine integration	134
13.14 Tradeoffs	134
13.15 Hints	134
13.16 Completion checklist	134
13.17 Exercise source files	135
13.17.1 Sources/LeetLLMExercises/P012FusedRMSNormGEMVExercise.swift	135
13.17.2 Sources/LeetLLMExercises/P012FusedRMSNormGEMVMetal.swift	135
13.17.3 Sources/LeetLLMExercises/Metal/P012FusedRMSNormGEMV.metal	135
13.18 Worked solution	137
13.18.1 Implementation points	137
13.18.2 Code map	138
13.18.3 Complete canonical listings	138
13.18.4 Sources/LeetLLMSolutions/P012FusedRMSNormGEMVMetal.swift	138
13.18.5 Sources/LeetLLMSolutions/P012FusedRMSNormGEMVSolution.swift	138
13.18.6 Sources/LeetLLMSolutions/Metal/P012FusedRMSNormGEMV.metal	139

IV Positions and Attention 140

14 Embedding Lookup and Tied Output Weights	141
14.1 Why this exists	141
14.2 Learning outcomes	141
14.3 Prerequisites	141
14.4 Vocabulary	141
14.5 Math from first principles	142
14.5.1 Worked numerical example	142
14.6 Shape, layout, and dtype contract	143
14.7 CPU reference path	143
14.8 Independent correctness method	143
14.9 Performance model	143
14.10 Metal mapping	144
14.11 Implementation checkpoints	144
14.12 Controlled experiments	144
14.12.1 Token locality	144
14.12.2 Vocabulary sweep	144
14.12.3 Decode versus prefill	144
14.13 Engine integration	144
14.14 Tradeoffs	145
14.15 Hints	145
14.16 Completion checklist	145
14.17 Exercise source files	146
14.17.1 Sources/LeetLLMExercises/P013EmbeddingExercise.swift	146
14.17.2 Sources/LeetLLMExercises/P013EmbeddingMetal.swift	146
14.17.3 Sources/LeetLLMExercises/Metal/P013Embedding.metal	146
14.18 Worked solution	148
14.18.1 Implementation points	148
14.18.2 Code map	149
14.18.3 Complete canonical listings	149
14.18.4 Sources/LeetLLMSolutions/P013EmbeddingMetal.swift	149
14.18.5 Sources/LeetLLMSolutions/P013EmbeddingSolution.swift	149

14.18.6 Sources/LeetLLMSolutions/Metal/P013Embedding.metal	150
15 Q/K/V Projections and Head Views	151
15.1 Why this exists	151
15.2 What query, key, and value mean	151
15.3 Learning outcomes	151
15.4 Prerequisites	152
15.5 Vocabulary	152
15.6 Math from first principles	152
15.6.1 Worked numerical example	152
15.7 Shape, layout, and dtype contract	153
15.8 CPU reference path	153
15.9 Independent correctness method	153
15.10 Performance model	154
15.11 Metal mapping	154
15.12 Implementation checkpoints	154
15.13 Controlled experiments	154
15.13.1 Head-count sweep	154
15.13.2 View versus copy	154
15.13.3 Decode versus prefill	154
15.14 Engine integration	155
15.15 Tradeoffs	155
15.16 Hints	155
15.17 Completion checklist	155
15.18 Exercise source files	156
15.18.1 Sources/LeetLLMExercises/P014QKVPProjectionExercise.swift	156
15.19 Worked solution	157
15.19.1 Implementation points	157
15.19.2 Code map	158
15.19.3 Complete canonical listings	158
15.19.4 Sources/LeetLLMSolutions/P014QKVPProjectionSolution.swift	158
16 Rotary Position Embeddings	159
16.1 Why this exists	159
16.2 Learning outcomes	159
16.3 Prerequisites	159
16.4 Vocabulary	159
16.5 Math from first principles and exact convention	159
16.5.1 Worked numerical example	160
16.6 Shape, layout, and dtype contract	160
16.7 CPU reference path	160
16.8 Independent correctness method	161
16.9 Performance model	161
16.10 Metal mapping	161
16.11 Implementation checkpoints	161
16.12 Controlled experiments	161
16.12.1 Position sweep	161
16.12.2 Base sweep	162
16.12.3 Precomputed versus computed angles	162
16.12.4 Partial dimension sweep	162
16.13 Engine integration	162

16.14	Tradeoffs	162
16.15	Hints	162
16.16	Completion checklist	162
16.17	Exercise source files	163
16.17.1	Sources/LeetLLMExercises/P015RoPEExercise.swift	163
16.17.2	Sources/LeetLLMExercises/P015RoPEMetal.swift	163
16.17.3	Sources/LeetLLMExercises/Metal/P015RoPE.metal	163
16.18	Worked solution	165
16.18.1	Implementation points	165
16.18.2	Code map	166
16.18.3	Complete canonical listings	166
16.18.4	Sources/LeetLLMSolutions/P015RoPEMetal.swift	166
16.18.5	Sources/LeetLLMSolutions/P015RoPESolution.swift	166
16.18.6	Sources/LeetLLMSolutions/Metal/P015RoPE.metal	167
17	Causal Attention for One Head	168
17.1	Why this exists	168
17.2	Learning outcomes	168
17.3	Prerequisites	168
17.4	Vocabulary	168
17.5	Math from first principles	168
17.5.1	Worked numerical example	169
17.6	Shape, layout, and dtype contract	169
17.7	CPU reference path	169
17.8	Independent correctness method	169
17.9	Performance model	171
17.10	Metal mapping	171
17.11	Implementation checkpoints	171
17.12	Controlled experiments	171
17.12.1	Sequence-length sweep	171
17.12.2	Head-width sweep	171
17.12.3	Offset sweep	171
17.13	Engine integration	172
17.14	Tradeoffs	172
17.15	Hints	172
17.16	Completion checklist	172
17.17	Exercise source files	173
17.17.1	Sources/LeetLLMExercises/P016CausalAttentionExercise.swift	173
17.17.2	Sources/LeetLLMExercises/P016CausalAttentionMetal.swift	173
17.17.3	Sources/LeetLLMExercises/Metal/P016CausalAttention.metal	173
17.18	Worked solution	174
17.18.1	Implementation points	174
17.18.2	Code map	175
17.18.3	Complete canonical listings	175
17.18.4	Sources/LeetLLMSolutions/P016CausalAttentionMetal.swift	175
17.18.5	Sources/LeetLLMSolutions/P016CausalAttentionSolution.swift	175
17.18.6	Sources/LeetLLMSolutions/Metal/P016CausalAttention.metal	176
18	Multi-Head Attention	177
18.1	Why this exists	177
18.2	Learning outcomes	177

18.3	Prerequisites	177
18.4	Vocabulary	177
18.5	Math from first principles	177
18.5.1	Worked numerical example	178
18.6	Shape, layout, and dtype contract	179
18.7	CPU reference path	179
18.8	Independent correctness method	179
18.9	Performance model	179
18.10	Metal mapping	179
18.11	Implementation checkpoints	179
18.12	Controlled experiments	180
18.12.1	Head sweep	180
18.12.2	Fixed dh sweep	180
18.12.3	Head isolation	180
18.13	Engine integration	180
18.14	Tradeoffs	180
18.15	Hints	180
18.16	Completion checklist	180
18.17	Exercise source files	182
18.17.1	Sources/LeetLLMExercises/P017MultiHeadAttentionExercise.swift	182
18.17.2	Sources/LeetLLMExercises/P017MultiHeadAttentionMetal.swift	182
18.17.3	Sources/LeetLLMExercises/Metal/P017MultiHeadAttention.metal	182
18.18	Worked solution	183
18.18.1	Implementation points	183
18.18.2	Code map	184
18.18.3	Complete canonical listings	184
18.18.4	Sources/LeetLLMSolutions/P017MultiHeadAttentionMetal.swift	184
18.18.5	Sources/LeetLLMSolutions/P017MultiHeadAttentionSolution.swift	184
18.18.6	Sources/LeetLLMSolutions/Metal/P017MultiHeadAttention.metal	185
19	MHA, MQA, and GQA	186
19.1	Why this exists	186
19.2	Learning outcomes	186
19.3	Prerequisites	186
19.4	Vocabulary	186
19.5	Math and mapping	186
19.5.1	Worked mapping example	187
19.6	Shape, layout, and dtype contract	187
19.7	CPU reference path	187
19.8	Independent correctness method	188
19.9	Performance and cache model	188
19.10	Metal mapping	188
19.11	Implementation checkpoints	188
19.12	Controlled experiments	188
19.12.1	KV-head sweep	188
19.12.2	Shared-KV reuse	189
19.12.3	Architecture mismatch	189
19.13	Engine integration	189
19.14	Tradeoffs	189
19.15	Hints	189

19.16	Completion checklist	189
19.17	Exercise source files	190
19.17.1	Sources/LeetLLMExercises/P018GroupedQueryAttentionExercise.swift	190
19.17.2	Sources/LeetLLMExercises/P018GroupedQueryAttentionMetal.swift	190
19.17.3	Sources/LeetLLMExercises/Metal/P018GroupedQueryAttention.metal	190
19.18	Worked solution	191
19.18.1	Implementation points	191
19.18.2	Code map	192
19.18.3	Complete canonical listings	192
19.18.4	Sources/LeetLLMSolutions/P018GroupedQueryAttentionMetal.swift	192
19.18.5	Sources/LeetLLMSolutions/P018GroupedQueryAttentionSolution.swift	192
19.18.6	Sources/LeetLLMSolutions/Metal/P018GroupedQueryAttention.metal	193
20	Online Softmax Attention	194
20.1	Why this exists	194
20.2	Learning outcomes	194
20.3	Prerequisites	194
20.4	Vocabulary	194
20.5	Math from first principles	194
20.5.1	Worked numerical example	195
20.6	Shape, layout, and dtype contract	196
20.7	CPU reference path	196
20.8	Independent correctness method	196
20.9	Performance model	196
20.10	Metal mapping	196
20.11	Implementation checkpoints	196
20.12	Controlled experiments	197
20.12.1	Increasing-score sequence	197
20.12.2	Length sweep	197
20.12.3	Head-width sweep	197
20.13	Engine integration	197
20.14	Tradeoffs	197
20.15	Hints	197
20.16	Completion checklist	197
20.17	Exercise source files	199
20.17.1	Sources/LeetLLMExercises/P0190onlineAttentionExercise.swift	199
20.17.2	Sources/LeetLLMExercises/P0190onlineAttentionMetal.swift	199
20.17.3	Sources/LeetLLMExercises/Metal/P0190onlineAttention.metal	199
20.18	Worked solution	200
20.18.1	Implementation points	200
20.18.2	Code map	201
20.18.3	Complete canonical listings	201
20.18.4	Sources/LeetLLMSolutions/P0190onlineAttentionMetal.swift	201
20.18.5	Sources/LeetLLMSolutions/P0190onlineAttentionSolution.swift	201
20.18.6	Sources/LeetLLMSolutions/Metal/P0190onlineAttention.metal	202
21	Tiled Fused Attention	203
21.1	Why this exists	203
21.2	Learning outcomes	203
21.3	Prerequisites	203
21.4	Vocabulary	203

21.5	Math from first principles	203
21.5.1	Worked tiled example	204
21.6	Shape, layout, dtype, and bounds contract	204
21.7	CPU reference path	204
21.8	Independent correctness method	205
21.9	Performance model	205
21.10	Metal grid, memory, and barriers	205
21.11	Implementation checkpoints	205
21.12	Controlled experiments	207
21.12.1	Tile-boundary sweep	207
21.12.2	Head-width sweep	207
21.12.3	Materialized versus fused traffic	207
21.12.4	Query-tile thought experiment	207
21.13	Engine integration	207
21.14	Tradeoffs	207
21.15	Hints	207
21.16	Completion checklist	207
21.17	Exercise source files	209
21.17.1	Sources/LeetLLMExercises/P020TiledAttentionExercise.swift	209
21.17.2	Sources/LeetLLMExercises/P020TiledAttentionMetal.swift	209
21.17.3	Sources/LeetLLMExercises/Metal/P020TiledAttention.metal	209
21.18	Worked solution	210
21.18.1	Implementation points	210
21.18.2	Code map	211
21.18.3	Complete canonical listings	211
21.18.4	Sources/LeetLLMSolutions/P020TiledAttentionMetal.swift	211
21.18.5	Sources/LeetLLMSolutions/P020TiledAttentionSolution.swift	211
21.18.6	Sources/LeetLLMSolutions/Metal/P020TiledAttention.metal	212
22	Sliding-Window Attention	213
22.1	Why this exists	213
22.2	Learning outcomes	213
22.3	Prerequisites	213
22.4	Vocabulary	213
22.5	Math and position semantics	213
22.5.1	Worked numerical example	214
22.6	Shape, layout, and dtype contract	214
22.7	CPU reference path	214
22.8	Independent correctness method	215
22.9	Performance and memory model	215
22.10	Metal mapping	215
22.11	Implementation checkpoints	215
22.12	Controlled experiments	215
22.12.1	Window sweep	216
22.12.2	Offset invariance	216
22.12.3	Cache eviction simulation	216
22.12.4	Layer stacking	216
22.13	Engine integration	216
22.14	Tradeoffs	216
22.15	Hints	216

22.16	Completion checklist	216
22.17	Exercise source files	217
22.17.1	Sources/LeetLLMExercises/P021SlidingWindowAttentionExercise.swift	217
22.17.2	Sources/LeetLLMExercises/P021SlidingWindowAttentionMetal.swift	217
22.17.3	Sources/LeetLLMExercises/Metal/P021SlidingWindowAttention.metal	217
22.18	Worked solution	218
22.18.1	Implementation points	218
22.18.2	Code map	219
22.18.3	Complete canonical listings	219
22.18.4	Sources/LeetLLMSolutions/P021SlidingWindowAttentionMetal.swift	219
22.18.5	Sources/LeetLLMSolutions/P021SlidingWindowAttentionSolution.swift	219
22.18.6	Sources/LeetLLMSolutions/Metal/P021SlidingWindowAttention.metal	220
V	KV-Cache Engineering	221
23	Preallocate and Append K/V	222
23.1	Why this exists	222
23.2	Learning outcomes	222
23.3	Prerequisites	222
23.4	Vocabulary	222
23.5	Math and worked offsets	223
23.6	Shape, layout, and dtype contract	223
23.7	CPU reference path	223
23.8	Independent correctness method	223
23.9	Performance, bandwidth, and allocation model	223
23.10	Metal mapping	225
23.11	Implementation checkpoints	225
23.12	Controlled experiments	225
23.12.1	Capacity sweep	225
23.12.2	Layer interleaving	225
23.12.3	Growth comparison	225
23.13	Engine integration	225
23.14	Tradeoffs	226
23.15	Hints	226
23.16	Completion checklist	226
23.17	Exercise source files	227
23.17.1	Sources/LeetLLMExercises/P022ContiguousKVCacheExercise.swift	227
23.18	Worked solution	228
23.18.1	Implementation points	228
23.18.2	Code map	229
23.18.3	Complete canonical listings	229
23.18.4	Sources/LeetLLMSolutions/P022ContiguousKVCacheSolution.swift	229
24	Cached Single-Token Attention	230
24.1	Why this exists	230
24.2	Learning outcomes	230
24.3	Prerequisites	230
24.4	Vocabulary	230
24.5	Math from first principles	230
24.5.1	Worked decode step	231

24.6	Shape, layout, and dtype contract	231
24.7	CPU reference path	231
24.8	Independent correctness method	231
24.9	Performance, bytes, and allocation model	232
24.10	Metal mapping	232
24.11	Implementation checkpoints	232
24.12	Controlled experiments	232
24.12.1	Cached versus recomputed projections	232
24.12.2	Context sweep	232
24.12.3	Absolute-position shift	233
24.12.4	Kernel ownership	233
24.13	Engine integration	233
24.14	Tradeoffs	233
24.15	Hints	233
24.16	Completion checklist	233
24.17	Exercise source files	234
24.17.1	Sources/LeetLLMExercises/P023CachedAttentionExercise.swift	234
24.17.2	Sources/LeetLLMExercises/P023CachedAttentionMetal.swift	234
24.17.3	Sources/LeetLLMExercises/Metal/P023CachedAttention.metal	234
24.18	Worked solution	236
24.18.1	Implementation points	236
24.18.2	Code map	237
24.18.3	Complete canonical listings	237
24.18.4	Sources/LeetLLMSolutions/P023CachedAttentionMetal.swift	237
24.18.5	Sources/LeetLLMSolutions/P023CachedAttentionSolution.swift	237
24.18.6	Sources/LeetLLMSolutions/Metal/P023CachedAttention.metal	239
25	KV Layout Shootout	240
25.1	Why this exists	240
25.2	Learning outcomes	240
25.3	Prerequisites	240
25.4	Vocabulary	240
25.5	Math from first principles: offset derivation	240
25.6	Shape, layout, and dtype contract	241
25.7	CPU reference path	241
25.8	Independent correctness method	242
25.9	Performance, bandwidth, and allocation model	242
25.10	Metal mapping	242
25.11	Implementation checkpoints	242
25.12	Controlled experiments	242
25.12.1	Token-count sweep	243
25.12.2	Head-count sweep	243
25.12.3	Append versus read	243
25.12.4	Conversion break-even	243
25.13	Engine integration	243
25.14	Tradeoffs	243
25.15	Hints	243
25.16	Completion checklist	243
25.17	Exercise source files	245
25.17.1	Sources/LeetLLMExercises/P024KVLayoutShootoutExercise.swift	245

25.18	Worked solution	246
25.18.1	Implementation points	246
25.18.2	Code map	247
25.18.3	Complete canonical listings	247
25.18.4	Sources/LeetLLMSolutions/P024KVLayoutShootoutSolution.swift	247
26	Shared KV Heads	250
26.1	Why this exists	250
26.2	Learning outcomes	250
26.3	Prerequisites	250
26.4	Vocabulary	250
26.5	Math from first principles: head mapping and bytes	250
26.6	Shape, layout, and dtype contract	251
26.7	CPU reference path	251
26.8	Independent correctness method	252
26.9	Performance, bandwidth, and allocation model	252
26.10	Metal mapping	252
26.11	Implementation checkpoints	252
26.12	Controlled experiments	252
26.12.1	KV-head sweep	252
26.12.2	Mapping mutation	252
26.12.3	Shared-read locality	253
26.13	Engine integration	253
26.14	Tradeoffs	253
26.15	Hints	253
26.16	Completion checklist	253
26.17	Exercise source files	254
26.17.1	Sources/LeetLLMExercises/P025SharedKVHeadsExercise.swift	254
26.18	Worked solution	255
26.18.1	Implementation points	255
26.18.2	Code map	256
26.18.3	Complete canonical listings	256
26.18.4	Sources/LeetLLMSolutions/P025SharedKVHeadsSolution.swift	256
27	Ring-Buffer Sliding Cache	257
27.1	Why this exists	257
27.2	Learning outcomes	257
27.3	Prerequisites	257
27.4	Vocabulary	257
27.5	Wraparound derivation and worked example	257
27.6	Shape, layout, and dtype contract	258
27.7	CPU reference path	258
27.8	Independent correctness method	259
27.9	Performance, bandwidth, and allocation model	259
27.10	Metal mapping	259
27.11	Implementation checkpoints	259
27.12	Controlled experiments	259
27.12.1	Wrap-count sweep	259
27.12.2	Window versus capacity	260
27.12.3	Split-range read	260
27.13	Engine integration	260

27.14	Tradeoffs	260
27.15	Hints	260
27.16	Completion checklist	260
27.17	Exercise source files	261
27.17.1	Sources/LeetLLMExercises/P026RingKVCacheExercise.swift	261
27.18	Worked solution	262
27.18.1	Implementation points	262
27.18.2	Code map	263
27.18.3	Complete canonical listings	263
27.18.4	Sources/LeetLLMSolutions/P026RingKVCacheSolution.swift	263
28	Paged KV Allocation	266
28.1	Why this exists	266
28.2	Learning outcomes	266
28.3	Prerequisites	266
28.4	Vocabulary	266
28.5	Math from first principles: paged addressing	266
28.6	Shape, layout, and dtype contract	267
28.7	CPU reference path	268
28.8	Independent correctness method	268
28.9	Performance, bytes, and fragmentation model	268
28.10	Metal mapping	268
28.11	Implementation checkpoints	269
28.12	Controlled experiments	269
28.12.1	Page-size sweep	269
28.12.2	Completion-order fragmentation	269
28.12.3	Gather versus materialize	269
28.13	Engine integration	269
28.14	Tradeoffs	269
28.15	Hints	269
28.16	Completion checklist	270
28.17	Exercise source files	271
28.17.1	Sources/LeetLLMExercises/P027PagedKVCacheExercise.swift	271
28.18	Worked solution	272
28.18.1	Implementation points	272
28.18.2	Code map	273
28.18.3	Complete canonical listings	273
28.18.4	Sources/LeetLLMSolutions/P027PagedKVCacheSolution.swift	273
29	Quantized KV Cache	277
29.1	Why this exists	277
29.2	Learning outcomes	277
29.3	Prerequisites	277
29.4	Vocabulary	277
29.5	Quantization derivation and worked example	278
29.6	Shape, layout, and dtype contract	278
29.7	CPU reference path	279
29.8	Independent correctness method	279
29.9	Performance model: bytes, bandwidth, and error	279
29.10	Metal mapping	279
29.11	Implementation checkpoints	280

29.12	Controlled experiments	280
29.12.1	Head-dimension sweep	280
29.12.2	Scale granularity	280
29.12.3	Context sweep	280
29.12.4	Convention fault injection	280
29.13	Engine integration	280
29.14	Tradeoffs	280
29.15	Hints	280
29.16	Completion checklist	281
29.17	Exercise source files	282
29.17.1	Sources/LeetLLMExercises/P028QuantizedKVCacheExercise.swift	282
29.17.2	Sources/LeetLLMExercises/P028QuantizedKVCacheMetal.swift	282
29.17.3	Sources/LeetLLMExercises/Metal/P028QuantizedKVCache.metal	283
29.18	Worked solution	284
29.18.1	Implementation points	284
29.18.2	Code map	285
29.18.3	Complete canonical listings	285
29.18.4	Sources/LeetLLMSolutions/P028QuantizedKVCacheMetal.swift	285
29.18.5	Sources/LeetLLMSolutions/P028QuantizedKVCacheSolution.swift	286
29.18.6	Sources/LeetLLMSolutions/Metal/P028QuantizedKVCache.metal	289

VI Weight Quantization 290

30	Symmetric INT8 Quantization	291
30.1	Why this exists	291
30.2	Learning outcomes	291
30.3	Prerequisites	291
30.4	Vocabulary	291
30.5	Derivation and worked numbers	292
30.6	Shape, layout, and dtype contract	292
30.7	CPU reference path	292
30.8	Independent correctness	294
30.9	Performance model: bytes and arithmetic intensity	294
30.10	Metal mapping	294
30.11	Implementation checkpoints	294
30.12	Controlled experiments	294
30.12.1	Range sweep	294
30.12.2	Tie-rule injection	295
30.12.3	Tensor-size sweep	295
30.13	Engine integration	295
30.14	Tradeoffs	295
30.15	Hints	295
30.16	Completion checklist	295
30.17	Exercise source files	296
30.17.1	Sources/LeetLLMExercises/P029SymmetricInt8Exercise.swift	296
30.18	Worked solution	297
30.18.1	Implementation points	297
30.18.2	Code map	298
30.18.3	Complete canonical listings	298
30.18.4	Sources/LeetLLMSolutions/P029SymmetricInt8Solution.swift	298

30.18.5 Sources/LeetLLMSolutions/WeightQuantizationSolutionSupport.swift	299
31 Per-Channel and Groupwise Scales	302
31.1 Why this exists	302
31.2 Learning outcomes	302
31.3 Prerequisites	302
31.4 Vocabulary	302
31.5 Derivation and worked numbers	302
31.6 Shape, layout, and dtype contract	303
31.7 CPU reference path	303
31.8 Independent correctness	304
31.9 Performance model: bytes and arithmetic intensity	304
31.10 Metal mapping	304
31.11 Implementation checkpoints	304
31.12 Controlled experiments	305
31.12.1 Group-size sweep	305
31.12.2 Row-range imbalance	305
31.12.3 Tail placement	305
31.13 Engine integration	305
31.14 Tradeoffs	305
31.15 Hints	305
31.16 Completion checklist	305
31.17 Exercise source files	306
31.17.1 Sources/LeetLLMExercises/P030GroupwiseInt8Exercise.swift	306
31.18 Worked solution	307
31.18.1 Implementation points	307
31.18.2 Code map	308
31.18.3 Complete canonical listings	308
31.18.4 Sources/LeetLLMSolutions/P030GroupwiseInt8Solution.swift	308
32 Pack and Unpack INT4	309
32.1 Why this exists	309
32.2 Learning outcomes	309
32.3 Prerequisites	309
32.4 Vocabulary	309
32.5 Derivation and byte-exact example	309
32.6 Shape, layout, dtype, and format contract	311
32.7 CPU reference path	311
32.8 Independent correctness	311
32.9 Performance model: bytes and arithmetic intensity	311
32.10 Metal mapping	312
32.11 Implementation checkpoints	312
32.12 Controlled experiments	312
32.12.1 Nibble-order fault	312
32.12.2 Odd-width matrix	312
32.12.3 Group-size byte model	312
32.13 Engine integration	312
32.14 Tradeoffs	312
32.15 Hints	313
32.16 Completion checklist	313
32.17 Exercise source files	314

32.17.1 Sources/LeetLLMExercises/P031PackQ4Exercise.swift	314
32.18 Worked solution	315
32.18.1 Implementation points	315
32.18.2 Code map	316
32.18.3 Complete canonical listings	316
32.18.4 Sources/LeetLLMSolutions/P031PackQ4Solution.swift	316
33 Dequantize Then GEMV	317
33.1 Why this exists	317
33.2 Learning outcomes	317
33.3 Prerequisites	317
33.4 Vocabulary	317
33.5 Derivation and worked projection	317
33.6 Shape, layout, and dtype contract	318
33.7 CPU reference path	318
33.8 Independent correctness	319
33.9 Performance model: bytes and arithmetic intensity	319
33.10 Metal mapping	319
33.11 Implementation checkpoints	319
33.12 Controlled experiments	320
33.12.1 Width sweep	320
33.12.2 Reuse sweep	320
33.12.3 Format fault localization	320
33.13 Engine integration	320
33.14 Tradeoffs	320
33.15 Hints	320
33.16 Completion checklist	320
33.17 Exercise source files	321
33.17.1 Sources/LeetLLMExercises/P032DequantizeThenGEMVExercise.swift	321
33.18 Worked solution	322
33.18.1 Implementation points	322
33.18.2 Code map	323
33.18.3 Complete canonical listings	323
33.18.4 Sources/LeetLLMSolutions/P032DequantizeThenGEMVSolution.swift	323
34 Fused Q4 GEMV	324
34.1 Why this exists	324
34.2 Learning outcomes	324
34.3 Prerequisites	324
34.4 Vocabulary	324
34.5 Derivation and worked projection	324
34.6 Shape, layout, dtype, and format contract	326
34.7 CPU fused reference	326
34.8 Independent correctness	326
34.9 Bytes and arithmetic intensity	326
34.10 Metal grid, unpacking, and reduction	327
34.11 Benchmark contract	327
34.12 Implementation checkpoints	327
34.13 Controlled experiments	327
34.13.1 Width sweep	327
34.13.2 Group-size sweep	328

34.13.3 Staged versus fused	328
34.13.4 Nibble-order injection	328
34.14 Engine integration	328
34.15 Tradeoffs	328
34.16 Hints	328
34.17 Completion checklist	328
34.18 Exercise source files	329
34.18.1 Sources/LeetLLMExercises/P033FusedQ4GEMVExercise.swift	329
34.18.2 Sources/LeetLLMExercises/P033FusedQ4GEMVMetal.swift	329
34.18.3 Sources/LeetLLMExercises/Metal/P033FusedQ4GEMV.metal	329
34.19 Worked solution	331
34.19.1 Implementation points	331
34.19.2 Code map	332
34.19.3 Complete canonical listings	332
34.19.4 Sources/LeetLLMSolutions/P033FusedQ4GEMVMetal.swift	332
34.19.5 Sources/LeetLLMSolutions/P033FusedQ4GEMVSolution.swift	332
34.19.6 Sources/LeetLLMSolutions/Metal/P033FusedQ4GEMV.metal	333
35 Quantization Error Propagation	335
35.1 Why this exists	335
35.2 Learning outcomes	335
35.3 Prerequisites	335
35.4 Vocabulary	335
35.5 Pipeline and worked metric example	336
35.6 Shape, layout, and dtype contract	336
35.7 CPU investigation path	336
35.8 Independent correctness	338
35.9 Convention-mismatch diagnostic	338
35.10 Bytes and arithmetic-intensity model	338
35.11 Metal mapping	339
35.12 Implementation checkpoints	339
35.13 Controlled experiments	339
35.13.1 Depth sweep	339
35.13.2 Group-size sweep	339
35.13.3 Convention intervention	339
35.13.4 Near-tie logits	339
35.14 Engine integration	339
35.15 Tradeoffs	340
35.16 Hints	340
35.17 Completion checklist	340
35.18 Exercise source files	341
35.18.1 Sources/LeetLLMExercises/P034QuantizationPropagationExercise.swift	341
35.19 Worked solution	342
35.19.1 Implementation points	342
35.19.2 Code map	343
35.19.3 Complete canonical listings	343
35.19.4 Sources/LeetLLMSolutions/P034QuantizationPropagationSolution.swift	343

VII Assemble Inference	346
36 One Decoder Transformer Block	347
36.1 Why this exists	347
36.2 Learning outcomes	347
36.3 Prerequisites	347
36.4 Vocabulary	347
36.5 Math and exact order	348
36.5.1 Worked values	348
36.6 Shape, layout, and dtype contract	349
36.7 CPU reference path	349
36.8 Independent correctness	349
36.9 Performance model: work, bytes, and allocation	351
36.10 Metal mapping	351
36.11 Implementation checkpoints	351
36.12 Controlled experiments	352
36.12.1 Position-offset intervention	352
36.12.2 GQA head-count sweep	352
36.12.3 Sequence-length sweep	352
36.12.4 Capture removal	352
36.13 Engine integration	352
36.14 Tradeoffs	352
36.15 Hints	352
36.16 Completion checklist	353
36.17 Exercise source files	354
36.17.1 Sources/LeetLLMExercises/P035DecoderBlockExercise.swift	354
36.18 Worked solution	355
36.18.1 Implementation points	355
36.18.2 Code map	356
36.18.3 Complete canonical listings	356
36.18.4 Sources/LeetLLMSolutions/P035DecoderBlockSolution.swift	356
37 Parse a Model Weight Format	360
37.1 Why this exists	360
37.2 Learning outcomes	360
37.3 Prerequisites	360
37.4 Vocabulary	360
37.5 Math and binary contract with worked bytes	361
37.6 JSON metadata contract	361
37.7 Decoder configuration and required names	362
37.8 CPU parser path	362
37.9 Independent correctness	362
37.10 Performance model: bytes, work, and allocation	364
37.11 Metal mapping	364
37.12 Implementation checkpoints	364
37.13 Controlled experiments	364
37.13.1 One-field corruption matrix	364
37.13.2 Descriptor-order permutation	365
37.13.3 Header-size sweep	365
37.13.4 Truncation sweep	365
37.14 Engine integration	365

37.15	Tradeoffs and limitations	365
37.16	Hints	365
37.17	Completion checklist	365
37.18	Exercise source files	367
37.18.1	Sources/LeetLLMExercises/P036WeightContainerExercise.swift	367
37.19	Worked solution	368
37.19.1	Implementation points	368
37.19.2	Code map	369
37.19.3	Complete canonical listings	369
37.19.4	Sources/LeetLLMSolutions/P036WeightContainerSolution.swift	369
38	Byte-Level BPE Tokenization and Detokenization	372
38.1	Why this exists	372
38.2	Learning outcomes	372
38.3	Prerequisites	372
38.4	Vocabulary	372
38.5	Math and algorithm from first principles	373
38.6	Worked merge sequence	373
38.7	Vocabulary and API contract	374
38.8	CPU reference path	374
38.9	Independent correctness	374
38.10	Performance model: work, bytes, and allocation	374
38.11	Metal mapping	376
38.12	Implementation checkpoints	376
38.13	Controlled experiments	376
38.13.1	Merge-density sweep	376
38.13.2	Rank intervention	376
38.13.3	Unicode boundary set	376
38.13.4	Special-token policy matrix	377
38.14	Engine integration	377
38.15	Tradeoffs and limitations	377
38.16	Hints	377
38.17	Completion checklist	377
38.18	Exercise source files	378
38.18.1	Sources/LeetLLMExercises/P037ByteBPExercise.swift	378
38.19	Worked solution	379
38.19.1	Implementation points	379
38.19.2	Code map	380
38.19.3	Complete canonical listings	380
38.19.4	Sources/LeetLLMSolutions/P037ByteBPESolution.swift	380
39	Logits and Sampling	382
39.1	Why this exists	382
39.2	Learning outcomes	382
39.3	Prerequisites	382
39.4	Vocabulary	382
39.5	Math and probability derivation	383
39.6	Worked filtering example	383
39.7	Shape, dtype, and error contract	383
39.8	Deterministic PRNG contract	384
39.9	CPU reference path	384

39.10	Independent correctness	384
39.11	Performance model: work, bytes, and allocation	386
39.12	Metal mapping	386
39.13	Implementation checkpoints	386
39.14	Controlled experiments	386
39.14.1	Temperature sweep	386
39.14.2	Top-k boundary sweep	386
39.14.3	Top-p boundary sweep	386
39.14.4	Seed and call-order experiment	387
39.14.5	Near-tie intervention	387
39.15	Engine integration	387
39.16	Tradeoffs and limitations	387
39.17	Hints	387
39.18	Completion checklist	387
39.19	Exercise source files	388
39.19.1	Sources/LeetLLMExercises/P038LogitsSamplingExercise.swift	388
39.20	Worked solution	389
39.20.1	Implementation points	389
39.20.2	Code map	390
39.20.3	Complete canonical listings	390
39.20.4	Sources/LeetLLMSolutions/P038LogitsSamplingSolution.swift	390
40	Prompt Prefill	392
40.1	Why this exists	392
40.2	Learning outcomes	392
40.3	Prerequisites	392
40.4	Vocabulary	392
40.5	Math, model, and equations	393
40.6	Worked prompt trace	393
40.7	Shape, state, and error contract	393
40.8	CPU reference algorithm	394
40.9	Independent correctness	394
40.10	Performance model: FLOPs, bytes, cache, and allocation	396
40.11	Honest Metal mapping	396
40.12	Implementation checkpoints	396
40.13	Controlled experiments	397
40.13.1	Prompt-length sweep	397
40.13.2	Position-offset intervention	397
40.13.3	Layer-order intervention	397
40.13.4	Tied versus independent output	397
40.14	Engine integration	397
40.15	Tradeoffs	397
40.16	Hints	397
40.17	Completion checklist	397
40.18	Exercise source files	399
40.18.1	Sources/LeetLLMExercises/P039PromptPrefillExercise.swift	399
40.19	Worked solution	400
40.19.1	Implementation points	400
40.19.2	Code map	401
40.19.3	Complete canonical listings	401

40.19.4 Sources/LeetLLMSolutions/P039PromptPrefillSolution.swift	401
40.19.5 Sources/LeetLLMSolutions/MiniDecoderCPUEngine.swift	401
41 Autoregressive Decode	407
41.1 Why this exists	407
41.2 Learning outcomes	407
41.3 Prerequisites	407
41.4 Vocabulary	407
41.5 One-token equations	408
41.6 Worked state transition	408
41.7 Shape and state contract	408
41.8 CPU reference algorithm	409
41.9 Stateful generation helper	409
41.10Independent correctness	409
41.11Performance model: FLOPs, bytes, and cache	411
41.12Honest Metal mapping	411
41.13Implementation checkpoints	411
41.14Controlled experiments	411
41.14.1 Context-length sweep	411
41.14.2 Recompute intervention	412
41.14.3 GQA sweep	412
41.14.4 Seed call-order intervention	412
41.15Engine integration	412
41.16Tradeoffs	412
41.17Hints	412
41.18Completion checklist	412
41.19Exercise source files	413
41.19.1 Sources/LeetLLMExercises/P040AutoregressiveDecodeExercise.swift	413
41.20Worked solution	414
41.20.1 Implementation points	414
41.20.2 Code map	415
41.20.3 Complete canonical listings	415
41.20.4 Sources/LeetLLMSolutions/P040AutoregressiveDecodeSolution.swift	415
42 Buffer Reuse and Memory Planning	418
42.1 Why this exists	418
42.2 Learning outcomes	418
42.3 Prerequisites	418
42.4 Vocabulary	418
42.5 Lifetime and aliasing math	419
42.6 Worked placement example	419
42.7 Input and output contract	420
42.8 Deterministic planning algorithm	420
42.9 Decoder-derived schedules	420
42.10Independent correctness	422
42.11Bytes, bounds, and complexity	422
42.12Honest Metal mapping	422
42.13Implementation checkpoints	422
42.14Controlled experiments	423
42.14.1 Sequence-length sweep	423
42.14.2 Alignment sweep	423

42.14.3 Lifetime extension	423
42.14.4 Strategy comparison	423
42.15 Engine integration	423
42.16 Tradeoffs and limitations	423
42.17 Hints	423
42.18 Completion checklist	424
42.19 Exercise source files	425
42.19.1 Sources/LeetLLMExercises/P041BufferPlanningExercise.swift	425
42.20 Worked solution	426
42.20.1 Implementation points	426
42.20.2 Code map	427
42.20.3 Complete canonical listings	427
42.20.4 Sources/LeetLLMSolutions/P041BufferPlanningSolution.swift	427
43 Checkpoint Parity and First Divergence	430
43.1 Why this exists	430
43.2 Learning outcomes	430
43.3 Prerequisites	430
43.4 Vocabulary	430
43.5 Capture schema and order	431
43.6 Math: error metrics and tolerance	431
43.7 Worked fault localization	432
43.8 Identity and rejection contract	432
43.9 Correctness method: independent educational reference	432
43.10 External reference replacement protocol	434
43.11 Performance model: work, bytes, and diagnostic cost	434
43.12 Honest Metal mapping	434
43.13 Implementation checkpoints	435
43.14 Controlled experiments	435
43.14.1 Tolerance sweep	435
43.14.2 Position identity intervention	435
43.14.3 Capture removal	435
43.14.4 Near-tie logits	435
43.15 Engine integration	435
43.16 Tradeoffs and limitations	435
43.17 Hints	436
43.18 Completion checklist	436
43.19 Exercise source files	437
43.19.1 Sources/LeetLLMExercises/P042CheckpointParityExercise.swift	437
43.20 Worked solution	438
43.20.1 Implementation points	438
43.20.2 Code map	439
43.20.3 Complete canonical listings	439
43.20.4 Sources/LeetLLMSolutions/P042CheckpointParitySolution.swift	439
VIII Tradeoffs from Evidence	444
44 Fuse RMSNorm and Q/K/V Projections	445
44.1 Why this exists	445
44.2 Learning outcomes	445

44.3	Prerequisites	445
44.4	Vocabulary	445
44.5	Math from first principles	446
44.6	Shape, layout, and dtype contract	446
44.7	CPU reference path	446
44.8	Correctness method	447
44.9	Performance model	447
44.10	Metal mapping	447
44.11	Implementation checkpoints	448
44.12	Controlled experiments	448
44.12.1	Sequence sweep	448
44.12.2	Head-sharing sweep	448
44.12.3	Boundary intervention	448
44.13	Engine integration	448
44.14	Tradeoffs and limitations	448
44.15	Hints	448
44.16	Completion checklist	449
44.17	Exercise source files	450
44.17.1	Sources/LeetLLMExercises/P043FusedQKVExercise.swift	450
44.17.2	Sources/LeetLLMExercises/P043FusedQKVMetal.swift	450
44.17.3	Sources/LeetLLMExercises/Metal/P043FusedQKV.metal	450
44.18	Worked solution	452
44.18.1	Implementation points	452
44.18.2	Code map	453
44.18.3	Complete canonical listings	453
44.18.4	Sources/LeetLLMSolutions/P043FusedQKVMetal.swift	453
44.18.5	Sources/LeetLLMSolutions/P043FusedQKVSolution.swift	454
44.18.6	Sources/LeetLLMSolutions/Metal/P043FusedQKV.metal	455
45	Profile Prefill and Decode Separately	457
45.1	Why this exists	457
45.2	Learning outcomes	457
45.3	Prerequisites	457
45.4	Vocabulary	457
45.5	Math from first principles	458
45.6	Shape, layout, and dtype contract	458
45.7	CPU reference path	458
45.8	Correctness method	460
45.9	Performance model	460
45.10	Metal mapping	460
45.11	Implementation checkpoints	460
45.12	Controlled experiments	461
45.12.1	Context-length sweep	461
45.12.2	Prompt-length sweep	461
45.12.3	Allocation boundary	461
45.12.4	Distribution stability	461
45.13	Engine integration	461
45.14	Tradeoffs and limitations	461
45.15	Hints	461
45.16	Completion checklist	462

45.17	Exercise source files	463
45.17.1	Sources/LeetLLMExercises/P044PrefillDecodeProfilingExercise.swift	463
45.18	Worked solution	464
45.18.1	Implementation points	464
45.18.2	Code map	465
45.18.3	Complete canonical listings	465
45.18.4	Sources/LeetLLMSolutions/P044PrefillDecodeProfilingSolution.swift	465
46	Static and Continuous Batching	468
46.1	Why this exists	468
46.2	Learning outcomes	468
46.3	Prerequisites	468
46.4	Vocabulary	468
46.5	Math from first principles	469
46.6	Shape, layout, and dtype contract	469
46.7	CPU reference path	469
46.8	Correctness method	471
46.9	Performance model	471
46.10	Metal mapping	471
46.11	Implementation checkpoints	471
46.12	Controlled experiments	472
46.12.1	Slot-count sweep	472
46.12.2	Arrival burst	472
46.12.3	Length variance	472
46.12.4	Prefill cost	472
46.13	Engine integration	472
46.14	Tradeoffs and limitations	472
46.15	Hints	472
46.16	Completion checklist	473
46.17	Exercise source files	474
46.17.1	Sources/LeetLLMExercises/P045BatchSchedulingExercise.swift	474
46.18	Worked solution	475
46.18.1	Implementation points	475
46.18.2	Code map	476
46.18.3	Complete canonical listings	476
46.18.4	Sources/LeetLLMSolutions/P045BatchSchedulingSolution.swift	476
47	Speculative Decoding	480
47.1	Why this exists	480
47.2	Learning outcomes	480
47.3	Prerequisites	480
47.4	Vocabulary	480
47.5	Math from first principles	481
47.6	Shape, layout, and dtype contract	481
47.7	CPU reference path	481
47.8	Correctness method	482
47.9	Performance model	483
47.10	Metal mapping	483
47.11	Implementation checkpoints	483
47.12	Controlled experiments	484
47.12.1	Acceptance sweep	484

47.12.2 Draft-length sweep	484
47.12.3 Distribution mismatch	484
47.12.4 Target verification cost	484
47.13 Engine integration	484
47.14 Tradeoffs and limitations	484
47.15 Hints	484
47.16 Completion checklist	484
47.17 Exercise source files	486
47.17.1 Sources/LeetLLMExercises/P046SpeculativeDecodingExercise.swift	486
47.18 Worked solution	487
47.18.1 Implementation points	487
47.18.2 Code map	488
47.18.3 Complete canonical listings	488
47.18.4 Sources/LeetLLMSolutions/P046SpeculativeDecodingSolution.swift	488
48 Capstone Inference Engine	491
48.1 Why this exists	491
48.2 Learning outcomes	491
48.3 Prerequisites	491
48.4 Vocabulary	491
48.5 Math from first principles	492
48.6 Shape, layout, and dtype contract	492
48.7 CPU reference path	492
48.8 Correctness method	492
48.9 Performance model	493
48.10 Metal mapping	493
48.11 Implementation checkpoints	493
48.12 Controlled experiments	495
48.12.1 Prompt and seed repeatability	495
48.12.2 Decode-length sweep	495
48.12.3 Metal boundary	495
48.12.4 Arena comparison	495
48.12.5 Unsupported input	495
48.13 Engine integration	495
48.14 Tradeoffs and limitations	495
48.15 Hints	496
48.16 Completion checklist	496
48.17 Exercise source files	497
48.17.1 Sources/LeetLLMExercises/P047CapstoneExercise.swift	497
48.18 Worked solution	498
48.18.1 Implementation points	498
48.18.2 Code map	499
48.18.3 Complete canonical listings	499
48.18.4 Sources/LeetLLMSolutions/P047CapstoneSolution.swift	499
IX Reference	504
A Anatomy of One Generated Token	505
A.1 Symbols	505
A.2 Inference has two distinct stages	505

A.2.1	Prefill	505
A.2.2	Decode	506
A.3	One decoder layer	506
A.4	Step-by-step decode trace	507
A.4.1	1. Token embedding	507
A.4.2	2. Attention normalization	507
A.4.3	3. Q/K/V projection	507
A.4.4	4. Position encoding with RoPE	507
A.4.5	5. KV-cache append	507
A.4.6	6. Scaled dot-product attention	507
A.4.7	7. Attention output and residual	508
A.4.8	8. MLP normalization and SwiGLU	508
A.4.9	9. Repeat layers	508
A.4.10	10. Final normalization and logits	508
A.4.11	11. Sampling	508
A.4.12	12. Detokenization and repeat	508
A.5	The two dominant memory budgets	508
A.5.1	Weights	508
A.5.2	KV cache	508
A.6	Questions to carry through the course	509
B	Math Primer	510
B.1	Notation and shapes	510
B.2	Dot product	510
B.3	Matrix-vector multiplication	510
B.4	Matrix-matrix multiplication	511
B.5	Linear layers	511
B.6	Elementwise activations and gates	511
B.7	Stable softmax	511
B.8	RMS normalization	512
B.9	Rotary position embeddings	512
B.10	Scaled dot-product attention	512
B.11	Quantization	512
B.12	FLOPs, bytes, and arithmetic intensity	513
B.13	Latency and throughput	513
B.14	Floating-point reductions	513

Preface

LeetLLM builds one small decoder-only transformer inference engine from first principles. The chapters begin with a scalar reduction and end with a measured, inspectable generation pipeline. Swift provides the readable semantic path; Metal exposes the memory, synchronization, and execution decisions that make inference a systems problem.

This companion is generated from the same lesson and source files as the native LeetLLM Studio application. It is not a parallel curriculum. Equations, exercises, diagrams, starter files, experiments, and canonical implementations remain tied to the repository that executes them.

How to use this book

Each chapter follows one loop:

1. Work through the derivation and the small numerical example.
2. Write down the requested prediction before running code.
3. Implement the Swift CPU path and make it pass its focused judge.
4. Implement the Metal path where the chapter defines one.
5. Run the controlled experiments in a release build.
6. Explain the result in terms of shapes, bytes, operations, and execution boundaries.
7. Only then read the worked solution at the end of the chapter.

The worked solutions are intentionally more than source dumps. Each ending states the strategy, identifies the invariants and failure modes, maps the important code regions to the ideas in the chapter, and then includes the full chapter-owned canonical files. Infrastructure established in earlier chapters is referenced rather than repeatedly relisted.

Two generated appendices make the volume self-contained: a one-token decode walkthrough names the complete inference path, and a just-in-time math primer collects the notation and derivations used across the exercises.

Chapter focus

A green judge is evidence of contract compliance, not the end of the exercise. The durable artifact is the explanation: why the implementation is correct, which costs it pays, and under which shapes or machine conditions another design would be preferable.

Build and notation

Commands assume an Apple Silicon Mac with the Swift and Metal command-line tools installed. Correctness runs use debug builds unless stated otherwise; measurements use release builds. Tensor shapes are written in logical order, and every performance claim should identify its units and measurement boundary.

The PDF is vector-first. Equations are typeset by LaTeX and curriculum flowcharts are translated to TikZ, so both remain sharp under magnification.

Start Here: Build an LLM Inference Engine

1.1 What LeetLLM Studio is

A **large language model (LLM)** is a learned numerical model that receives a sequence of tokens and produces scores for what token should come next. A **decoder-only** model predicts using only the tokens at or before the current position. **Inference** means running an already trained model to produce those predictions; unlike training, it does not update the model's learned weights.

LeetLLM Studio is a hands-on course for learning how that inference works from the arithmetic upward. It combines a lesson reader, editable Swift and Metal source files, and correctness checks in one macOS app. The repository also provides the same material as a companion book and command-line project.

This is not a chatbot application and it does not train a language model. You will implement individual inference operators, measure their behavior, and then connect them into a small deterministic educational engine. The capstone uses a seven-token synthetic model: its purpose is to expose the complete execution path and make every result independently checkable, not to generate useful prose. Generation in that capstone runs on the readable CPU reference backend; Metal executes a named verification slice rather than a complete GPU inference backend.

There are 47 runnable problems after this orientation. They progress from one dot product through tensors, neural-network operators, attention, KV-cache engineering, quantization, complete inference, and performance decisions.

1.2 Who the course is for

The course assumes that you can read basic Swift, write loops and functions, and follow scalar arithmetic. It does not assume prior Metal experience or prior knowledge of transformer internals. Every required model, math, GPU, and systems concept is introduced in the course before you are expected to use it. The math primer is available for extra derivations, but it is part of the course rather than an external prerequisite.

You are expected to do more than make checks turn green. For each operator, you should be able to:

1. state what the inputs and outputs mean;
2. predict a small result or performance trend before running code;
3. implement the simplest correct CPU behavior;
4. map that behavior to Metal when the lesson includes a GPU stage;
5. compare results using the lesson's stated numerical tolerance; and
6. explain the tradeoff shown by the evidence you collected.

1.3 How to use the Studio

Select a lesson in the catalog. Runnable lessons show the lesson reader and a workbench containing the chapter-owned starter files. This orientation is reader-only, so it deliberately has no editor or Run button.

Use the same loop for each runnable problem:

1. **Read.** Learn the operator's purpose, vocabulary, shape contract, and correctness method.
2. **Predict.** Work through the small example and write down what should happen before executing anything.
3. **Implement.** Replace the incomplete bodies in the learner source files.
4. **Run.** Use the CPU and Metal checks shown by the workbench. A passing check means the implementation met the tested contract; a failure is evidence to inspect, not a reason to copy the canonical code.
5. **Explain.** Complete the experiments and state why the implementation is correct, where time and memory go, and which tradeoffs the measurements support.
6. **Mark progress.** Check the chapter's completion items. Checkboxes persist locally, but they are a learning record, not a substitute for executable checks or your explanation.

The Text Size menu in the toolbar scales lesson text, controls, source code, and results from 80% through 200%. The keyboard commands are Command-Plus to increase, Command-Minus to decrease, and Command-Zero to return to 100%.

If you prefer a terminal, each lesson also gives focused `swift run leetllm` commands. The app and command line use the same course sources and judges.

1.4 Trust and local execution

Before the first runnable lesson, the packaged Studio asks you to choose a dedicated build folder through the macOS folder picker. It stores editable sources, compiler output, and generated learner executables under that folder. The Studio, separate runner, and learner executable are signed to use App Sandbox. The Studio host has the client entitlement required by its bundled WebKit views, and the runner currently inherits the Studio sandbox. Built-in checks do not send learner code to a remote model or service.

The selected folder remains writable to learner code, and a bad implementation can consume CPU, memory, or GPU time until cancellation or a timeout takes effect. Use a dedicated folder and review third-party code before running it. Command-line checks are not App Sandbox constrained; they use the permissions of the terminal process that launches them.

1.5 The inference path in plain language

An inference engine takes text that already exists and repeatedly predicts one next token. After choosing a token, it adds that token to the context and predicts again. The following concepts name the data and operations along that path.

1.5.1 Tokens, vocabulary, and tokenization

A **token** is an integer ID used by the model as one unit of input or output. A token can represent a word, part of a word, punctuation, a byte, or a special marker. Text and tokens are not interchangeable: a **tokenizer** applies fixed rules that encode text into token IDs and decode IDs back into text or bytes.

The **vocabulary** is the complete finite set of token IDs the model supports. If its size is v , valid IDs are $0 \leq \text{ID} < v$. The tokenizer and model must agree on what every ID means.

1.5.2 Vectors, tensors, shapes, and weights

A **vector** is an ordered row of numbers. A **matrix** is a rectangular grid of numbers. A **tensor** generalizes those ideas to any number of dimensions. Its **shape** lists the size of each dimension, such as $[S, D]$ for S token positions whose representations each contain D numbers.

Model **weights** are numerical parameters established during training. Inference treats them as fixed input data. A **projection** is a matrix multiplication that uses weights to remix an input vector into a new vector; it may change the vector's width or give the same numbers a new role.

1.5.3 Embeddings and the residual stream

The model cannot calculate directly with a token ID. An **embedding table** stores one learned vector per vocabulary item. Looking up a token ID selects its initial vector representation.

That vector enters the **residual stream**, the main representation carried from layer to layer. A **layer** is one repeated block of model operations. Its attention branch and feed-forward branch each compute a change and add that change back to the stream. This additive route preserves earlier information while allowing every layer to refine it. **Normalization** rescales a vector before a branch so values remain in a controlled numerical range.

1.5.4 Queries, keys, values, and attention

Attention lets the representation at one token position gather relevant information from token positions in its context. It does not consume the residual stream directly. Three learned projections create a query, key, and value for each attention head:

- A **query (Q)** represents what the current position is trying to find.
- A **key (K)** represents how one context position can be matched.
- A **value (V)** contains the information that position can contribute.

These descriptions explain the roles of the three vectors; they are not literal text questions or database keys. The engine takes a **dot product** between a query and each key. A dot product multiplies corresponding vector elements and adds the products, producing one similarity score. **Softmax** turns all visible scores into nonnegative weights that sum to one. The weighted sum of the corresponding value vectors is the attention output.

A **causal mask** hides positions to the right of the current position, so a prediction cannot use future tokens. Several **heads** perform this operation in separate learned subspaces, allowing different matching patterns. Their outputs are placed side by side and projected back into the residual stream.

Token order also matters. **RoPE**, or rotary position embeddings, rotates pairs of query and key features according to their positions. Those rotations make relative position affect query-key scores without adding a separate position vector to the residual stream.

1.5.5 Feed-forward branch, logits, and sampling

After attention, a layer's **feed-forward network**, also called an MLP, transforms each token position independently through learned projections and a nonlinear activation. Its output is added to the residual stream. Attention mixes information across positions; the feed-forward branch transforms the features within each position.

After the final layer, an output projection produces one **logit** for every vocabulary item. A logit is an unnormalized score: larger means the model ranks that token more highly, but logits are not probabilities. **Sampling** converts or adjusts the scores and selects the next token according to a stated rule, such as taking the largest score or drawing from a probability distribution.

1.5.6 Prefill, decode, and the KV cache

Generation has two execution stages:

- **Prefill** processes all tokens in the input prompt. Their work can be arranged as larger matrix-matrix operations and many positions can be evaluated in parallel.
- **Decode** processes one newly generated token at a time. Each step depends on the token selected by the previous step, so generation is sequential across steps.

Every layer saves the keys and values produced for prior positions in a **KV cache**. During decode, the new query reads those saved keys and values instead of recomputing all earlier layers for every token. The cache makes that reuse possible, but it consumes memory that grows with context length.

1.6 CPU, Metal, and evidence

The **CPU** is the general-purpose processor that runs the Swift reference implementations. The **GPU** performs large amounts of parallel arithmetic. On Apple platforms, **Metal** is the programming API used here to submit GPU work. A **kernel** is a function executed by many GPU threads; each thread owns part of the data, and some algorithms require threads to synchronize while combining partial results.

GPU execution is not automatically faster. Launch overhead, memory traffic, temporary storage, synchronization, and problem size all matter. The course therefore asks for predictions and measurements with explicit boundaries. A benchmark supports only the claim it actually measured, on the configuration it actually ran.

1.7 How the course is organized

The progression is cumulative:

1. Problems 001-006 establish reductions, tensors, matrix operations, and GPU execution.
2. Problems 007-012 build normalization, activations, softmax, residual precision, and fused neural-network operators.
3. Problems 013-021 construct embeddings, Q/K/V projections, positional encoding, causal attention, head mappings, and optimized attention.
4. Problems 022-028 make the KV cache correct, bounded, paged, and measurable.
5. Problems 029-034 introduce quantized weights and kernels while preserving independently checked numerical behavior.
6. Problems 035-042 assemble model layers, tokenization, sampling, prefill, decode, memory plans, and parity boundaries.
7. Problems 043-047 use evidence to evaluate fusion, profiling, batching, speculative decoding, and the connected capstone.

When a later chapter names an earlier concept, use the catalog to revisit its defining lesson. The appendices provide a one-token execution map and a math primer, but the runnable sequence remains the source of truth for contracts and checks.

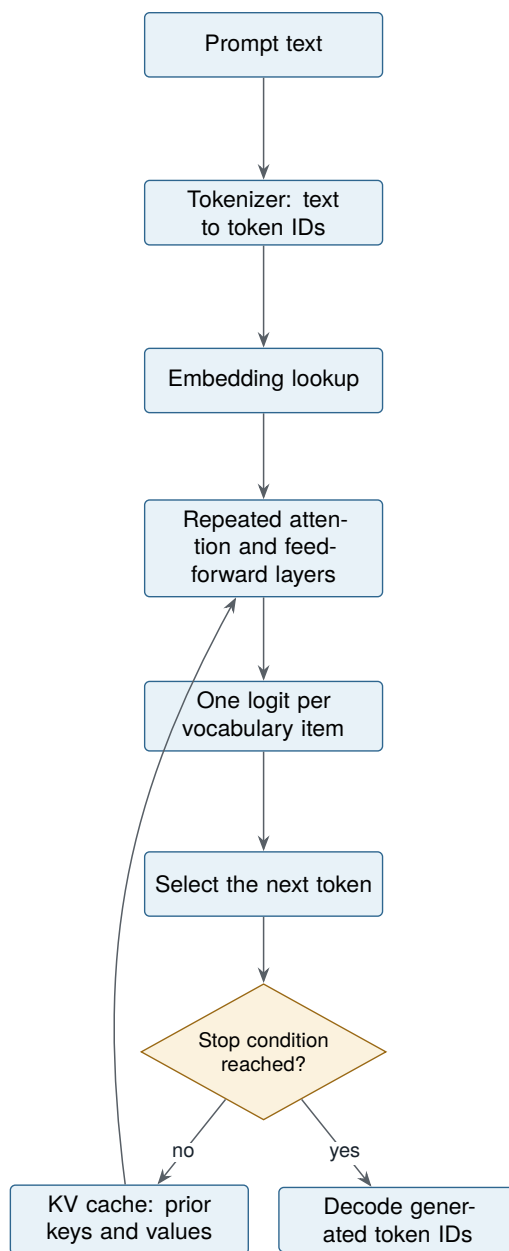


Figure 1.1: Token generation loop.

1.8 Completion checklist

- ☐ I can describe what the Studio, course, and capstone do and do not do.
- ☐ I know the read, predict, implement, run, explain, and mark-progress loop.
- ☐ I have set the text size to a comfortable reading level.
- ☐ I understand the packaged Studio's selected-folder sandbox and the command line's broader permissions.
- ☐ I can trace prompt text through tokens, embeddings, layers, logits, and a selected next token.
- ☐ I can explain the roles of Q, K, V, attention, the causal mask, and the KV cache without outside references.
- ☐ I am ready to begin Problem 001.

Part I

Foundations

Vector Dot Product

2.1 Why this exists

A dot product is not included because it is a convenient beginner loop. It is the inner operation of two important inference paths:

- One output of matrix-vector multiplication is a row of weights dotted with an activation vector. Decode is full of these projections.
- One attention score is a query vector dotted with a key vector before scaling and softmax.

The equation is tiny, but its GPU implementation immediately exposes the core systems questions of this course: bounds, work assignment, memory traffic, parallel reduction, synchronization, floating-point order, launch overhead, and when the CPU can beat the GPU.

2.2 Learning outcomes

After this problem, you should be able to:

1. Derive a dot product and compute a small case by hand.
2. State its shape contract, operation count, and approximate input traffic.
3. Implement a readable Float32 CPU oracle.
4. Map elements to Metal threads and reduce 256 products per threadgroup.
5. Explain every required threadgroup barrier.
6. Explain why CPU and GPU results need not be bit-identical.
7. Measure size-dependent CPU and GPU behavior without calling one universally faster.
8. Connect the reduction to GEMV and attention.

2.3 Prerequisites

- Swift loops, arrays, and thrown errors
- Scalar multiplication and addition
- No prior Metal experience is assumed

Read the dot-product, arithmetic-intensity, and floating-point sections in [the math primer](#).

2.4 Vocabulary

Reduction An operation that combines many input values into fewer output values, often one. Sum, maximum, and dot product are reductions.

Grid All threads dispatched for one Metal compute command.

Threadgroup A group of threads that can synchronize and share fast threadgroup memory.

Lane or thread index The current thread's position in a grid or threadgroup.

Barrier A synchronization point that prevents participating threads from continuing until earlier accesses in the named memory space are visible as required.

Partial sum One threadgroup's contribution to the final result.

2.5 Math from first principles

For vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^N$,

$$s = \mathbf{x} \cdot \mathbf{y} = \sum_{i=0}^{N-1} x_i y_i.$$

For

$$\mathbf{x} = [1, -2, 3, 4], \quad \mathbf{y} = [0.5, 2, -1, 0.25],$$

the element products are

$$[0.5, -4, -3, 1]$$

and the result is -5.5 .

Before coding, answer:

1. What should two empty vectors return under the usual empty-sum convention?
2. What should happen if the lengths differ?
3. What is the output shape?

This course uses 0 for the empty dot product, throws on unequal lengths, and returns one Float32 scalar.

2.6 Shape, layout, and dtype contract

Item	Contract
Left input	Contiguous Float32 vector [N]
Right input	Contiguous Float32 vector [N]
Output	One Float32 scalar
Accumulation	Float32 in learner and canonical implementations
Empty vectors	Return the empty sum, 0
Unequal lengths	Throw <code>VectorDotError.lengthMismatch</code>
CPU/Metal comparison	Scale-aware relative tolerance

The arrays are contiguous Swift storage on the CPU and shared Metal buffers on the GPU. Later tensor problems make shape and stride metadata explicit; this first problem deliberately begins with the simplest one-dimensional layout.

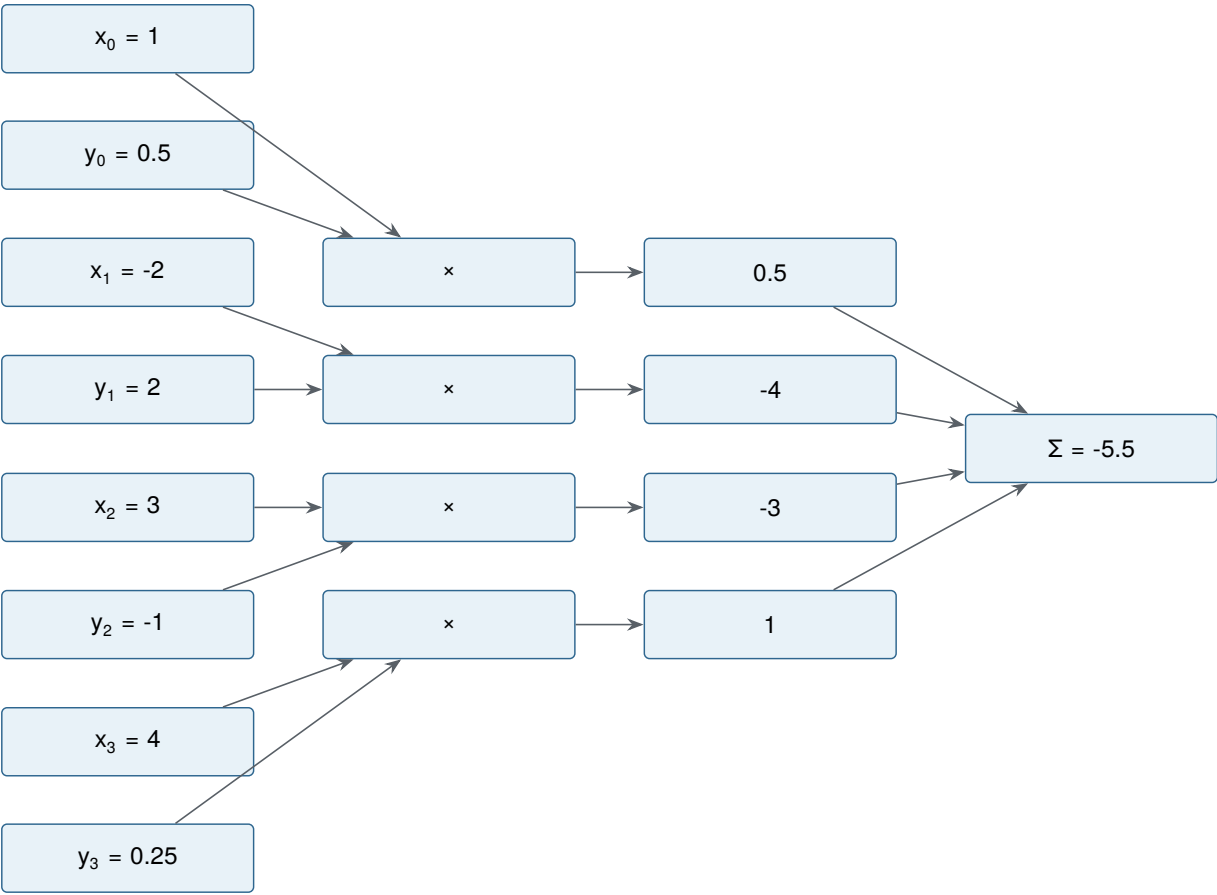


Figure 2.1: Elementwise dot.

2.7 Performance model

For length N :

- N multiplications
- approximately N additions
- about $2N$ FLOPs by the convention used in this course
- two Float32 input reads per element, or about $8N$ input bytes

Ignoring the scalar output and cache effects,

$$I \approx \frac{2N}{8N} = 0.25 \text{ FLOP/byte.}$$

That low arithmetic intensity predicts that a sufficiently large dot product will usually be constrained by data movement rather than multiply-add capacity. It does not predict the winner for small N , where Metal compilation, command submission, allocation, and synchronization can dominate.

Write down this prediction before benchmarking:

Below size _____, I expect _____ to win because _____. Above that size, I expect _____ to dominate because _____.

2.8 CPU reference path

Open [P001VectorDotExercise.swift](#).

Implement this algorithm without using an acceleration framework:

```
1 if lengths differ, throw
2 sum = 0
3 for each valid index:
4     sum += lhs[index] * rhs[index]
5 return sum
```

The point is to create the clearest executable meaning of the operator. Later implementations are checked against this behavior.

Run only the CPU stage:

```
1 swift run leetllm check 001 --cpu
```

The judge includes:

- empty inputs;
- one element;
- mixed signs;
- 1,025 elements, which later crosses several 256-thread groups;
- unequal lengths.

2.8.1 Checkpoint

Do not start the kernel until the CPU check reports 5/5.

2.9 Correctness method and reduction order

The CPU loop accumulates from left to right:

$$(((p_0 + p_1) + p_2) + \dots) + p_{N-1}.$$

The Metal kernel will use a tree. For eight products, its grouping resembles

```
1 level 0: p0 p1 p2 p3 p4 p5 p6 p7
2 level 1: p0+p4 p1+p5 p2+p6 p3+p7
3 level 2: (p0+p4)+(p2+p6) (p1+p5)+(p3+p7)
4 level 3: one partial sum
```

Floating-point rounding happens after each addition, so different groupings can produce slightly different low bits. The judge uses relative tolerance rather than bitwise equality. That tolerance is permission for expected rounding, not permission to ignore large or systematic differences.

2.10 Metal mapping

The host pipeline dispatches threadgroups of 256 threads. For N elements,

$$G = \left\lceil \frac{N}{256} \right\rceil$$

threadgroups are launched. The grid is padded to $256G$ threads, so threads in the final group may not correspond to an input element.

Each thread performs these stages:

1. Compute its global input index.
2. If the index is valid, multiply `lhs[index] * rhs[index]`; otherwise use zero.
3. Store the product in its threadgroup's 256-element scratch array.
4. Synchronize the group.
5. Repeatedly add the upper half of the remaining values into the lower half.
6. Synchronize after every reduction level.
7. Let local thread zero write one partial sum for the group.

The host waits for completion and adds the G partial sums. This two-stage design is intentionally simple. Later problems can compare a second GPU pass, SIMD-group primitives, or atomics.

2.10.1 Memory spaces in this kernel

MSL address space	Contents	Visibility and purpose
device	Input vectors and output partial sums	Large buffers visible to the grid and host
constant	Element count	Small read-only dispatch parameter
threadgroup	256 products/partial sums	Fast shared storage for one group
Per-thread value	Current product and indices	Private temporary state

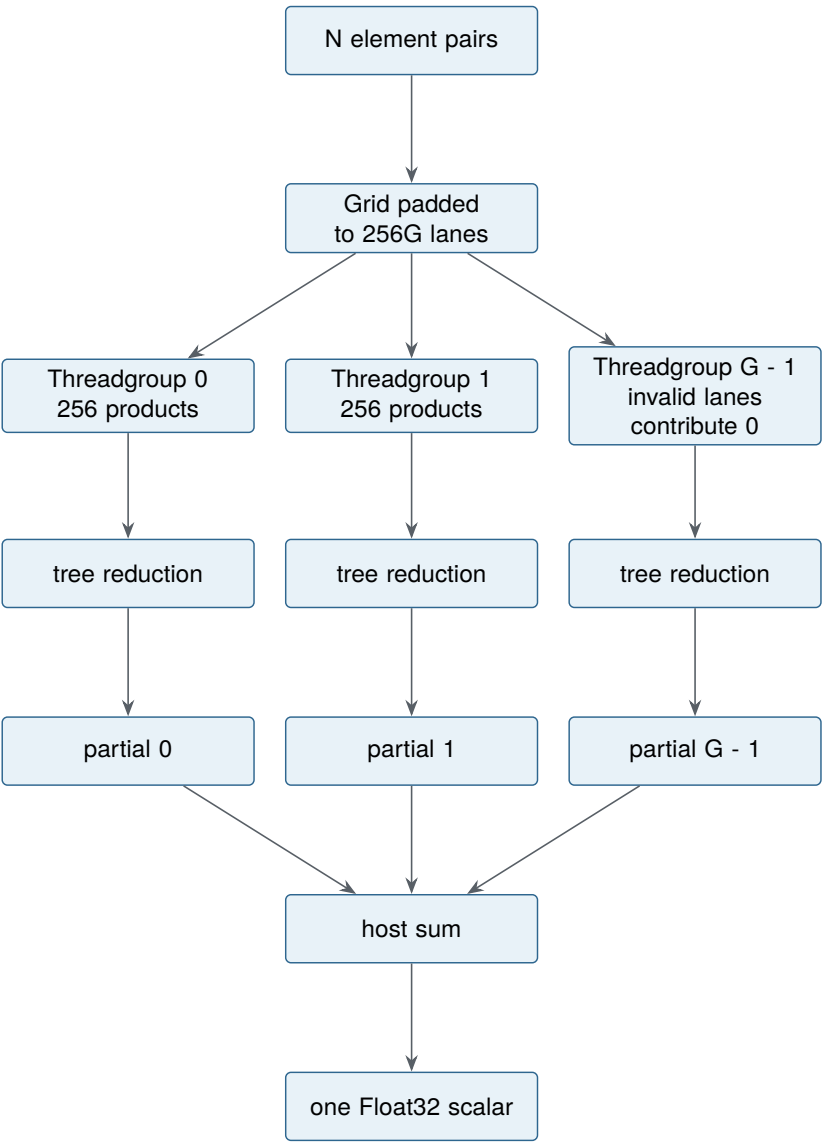


Figure 2.2: Metal reduction.

2.10.2 Why every barrier exists

After products are written to scratch, no thread may read a neighbor's slot until every thread has written. That requires the first barrier.

At reduction stride 128, threads 0 through 127 update scratch. The next stride reads values produced by that level. Without a barrier between levels, some threads could observe old values while others observe new ones.

A barrier outside the conditional is essential. If only the threads performing an addition reach it, the threadgroup does not converge at the synchronization point and behavior is invalid.

2.11 Implementation checkpoints

Open [P001VectorDot.metal](#).

The host-side command setup is already provided in [MetalVectorDotPipeline.swift](#). Read it before filling in the kernel. Identify where each buffer index, the element count, the number of groups, and the 256-thread width are encoded.

Your kernel needs these additional inputs:

```
1 uint globalIndex [[thread_position_in_grid]]
```

and one threadgroup array:

```
1 threadgroup float scratch[256];
```

Use a stride sequence of 128, 64, 32, ..., 1. Keep every barrier reachable by all threads in the group.

Run only the Metal stage:

```
1 swift run leetllm check 001 --metal
```

Then run both stages:

```
1 swift run leetllm check 001
```

2.12 Hints

Bounds hint

The padded threads should contribute the additive identity, '0.0f', to scratch. They must still participate in every barrier.

Reduction-loop hint

At each stride, only local indices smaller than the stride add one partner at 'localIndex + stride'. Synchronize after the conditional addition.

Harness check

To prove the judge and host path independently of your edits, run:

```
""sh swift run leetllm check 001 -solution ""
```

The canonical source is in the ‘LeetLLMSolutions’ module.

2.13 Controlled experiments

2.13.1 Benchmark the operator

Use a release build:

```
1 swift run -c release leetllm benchmark 001
```

The current benchmark measures the canonical implementations so that exercise edits cannot accidentally change the measurement harness. It reports the median of repeated synchronous calls.

The Metal end-to-end line includes:

- shared-buffer allocation;
- copying Swift array contents into those buffers;
- command encoding and submission;
- GPU execution;
- CPU/GPU synchronization;
- CPU reduction of group partials.

The reported effective GB/s counts the two logical input reads, $8N$ bytes. It is not a hardware memory-bus counter and does not include every host copy or internal transaction. Its purpose is comparison across controlled runs of this operator.

Record:

```
1 Machine:
2 OS and Swift version:
3 Build configuration:
4 N:
5 Iterations:
6 CPU median:
7 Metal end-to-end median:
8 Prediction matched? Why or why not?
9 Largest cost not isolated by this benchmark:
```

2.13.2 Required size-sweep experiment

Run at least these sizes with enough iterations for stable medians:

```
1 swift run -c release leetllm benchmark 001 --size 64 --iterations 100
2 swift run -c release leetllm benchmark 001 --size 4096 --iterations 100
3 swift run -c release leetllm benchmark 001 --size 1048576 --iterations 20
```

Before running, predict the curve. After running, answer:

1. At which tested size does Metal first win, if any?
2. Does CPU time grow approximately linearly once the input is large?
3. Does Metal show a fixed-cost floor at small sizes?
4. Why is the measured Metal path not yet a fair kernel-only comparison?
5. Which next instrument would separate allocation, command submission, and GPU execution time?

Do not infer a machine-wide peak bandwidth from this benchmark. It measures one specific end-to-end path.

2.14 Tradeoffs

1. Why might one Metal thread that loops over all elements be slower for large vectors but faster for very small vectors?
2. What extra work and synchronization would a second GPU reduction pass add?
3. When could an atomic add be simpler, and what contention might it create?
4. What changes if inputs are Float16 but accumulation remains Float32?
5. In GEMV, can a loaded activation value be reused across several output rows?
6. In attention decode, how does cache layout affect the key-vector dot products?

2.15 Engine integration

Problem 004 turns this reduction into GEMV:

$$y_m = W_{m,:} \cdot \mathbf{x}.$$

Instead of producing one scalar, a grid produces one value per weight row. That changes the useful parallelism and creates opportunities to reuse parts of the activation vector.

Problem 016 reuses the same idea for one attention head:

$$\text{score}_t = \frac{\mathbf{q} \cdot \mathbf{k}_t}{\sqrt{d_h}}.$$

There, many key vectors come from the KV cache, and the next operation is a stable softmax reduction. The simple dot product is therefore both a complete first kernel and a component that survives into the final engine.

2.16 Completion checklist

- ☐ I computed the four-element example by hand.
- ☐ The CPU exercise reports 5/5.
- ☐ I can draw the 256-to-1 reduction tree.
- ☐ I can justify the bounds mask and each barrier.
- ☐ The Metal exercise reports 5/5.
- ☐ I wrote a size-sweep prediction before measuring.
- ☐ I recorded release-build measurements and interpreted the crossover.
- ☐ I can derive the approximate 0.25 FLOP/byte arithmetic intensity.
- ☐ I can identify the exact future GEMV and attention uses.

2.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

2.17.1 Sources/LeetLLMExercises/P001VectorDotExercise.swift

```
1 import LeetLLMCore
2
3 public enum P001VectorDotExercise {
4     public static func dot(_ lhs: [Float], _ rhs: [Float]) throws -> Float {
5         guard lhs.count == rhs.count else {
6             throw VectorDotError.lengthMismatch(lhs: lhs.count, rhs: rhs.count)
7         }
8
9         // TODO: Replace this placeholder with the CPU reference implementation.
10        return 0
11    }
12 }
```

2.17.2 Sources/LeetLLMExercises/P001VectorDotMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P001VectorDotExercise {
5     public static func makeMetalPipeline() throws -> MetalVectorDotPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P001VectorDot",
8             withExtension: "metal",
9             subdirectory: "Metal"
10        ) else {
11            throw MetalVectorDotError.kernelResourceMissing("P001VectorDot.metal")
12        }
13
14        let source = try String(contentsOf: url, encoding: .utf8)
15        return try MetalVectorDotPipeline(source: source)
16    }
17 }
```

2.17.3 Sources/LeetLLMExercises/Metal/P001VectorDot.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void vector_dot_partial(
5     device const float *lhs [[buffer(0)]],
6     device const float *rhs [[buffer(1)]],
7     device float *partialSums [[buffer(2)]],
8     constant uint &elementCount [[buffer(3)]],
9     uint localIndex [[thread_index_in_threadgroup]],
10    uint groupIndex [[threadgroup_position_in_grid]]
11 ) {
12     if (localIndex == 0) {
13         // TODO: Replace this placeholder with a parallel threadgroup reduction.
14         partialSums[groupIndex] = 0.0f;
15     }
16 }
```

2.18 Worked solution

Solution strategy

Treat the CPU function as the executable definition of the operator, then preserve that contract while changing only the reduction schedule on the GPU. The CPU path validates equal lengths and performs one left-to-right Float32 accumulation. The Metal path assigns one product to each lane, substitutes the additive identity for padded lanes, reduces within each 256-lane threadgroup, and returns one partial sum per group for the host to finish.

Correctness invariant

Every valid pair contributes exactly one product. Every padded lane contributes exactly zero. At reduction stride s , scratch slots 0 through $s-1$ contain the sums represented by the two child slots from the preceding level.

Common failure mode

Do not put a threadgroup barrier inside the $\text{localIndex} < \text{stride}$ branch. Lanes that perform no addition still have to reach every barrier, or the threadgroup does not converge at the synchronization point.

2.18.1 Implementation points

Validate the shape contract first

The length guard runs before indexing either array. It makes unequal inputs a typed failure and naturally leaves the equal empty-input case with the additive identity, zero.

Keep the CPU oracle deliberately plain

A single indexed loop exposes the exact Float32 accumulation order. That clarity matters when the tree-shaped Metal result differs in low bits but remains within the judge's scale-aware tolerance.

Mask work, not participation

A padded Metal lane stores 0.0 into threadgroup scratch instead of returning early. It therefore contributes no value while still participating in each synchronization point.

Reduce by halving the active range

Each stride adds $\text{scratch}[\text{localIndex} + \text{stride}]$ into the lower half. The following barrier makes the completed level visible before any lane reads it as input to the next level.

Separate GPU and host responsibilities

Only local lane zero writes the threadgroup result. The host pipeline later combines those group partials, avoiding an additional global GPU reduction in this first implementation.

2.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P001VectorDotMetal.swift

- Line 4: extension P001VectorDotSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalVectorDotPipeline {

Sources/LeetLLMSolutions/P001VectorDotSolution.swift

- Line 3: public enum P001VectorDotSolution {
- Line 4: public static func dot(_ lhs: [Float], _ rhs: [Float]) throws -> Float {

Sources/LeetLLMSolutions/Metal/P001VectorDot.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

2.18.3 Complete canonical listings

2.18.4 Sources/LeetLLMSolutions/P001VectorDotMetal.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P001VectorDotSolution {
5     public static func makeMetalPipeline() throws -> MetalVectorDotPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P001VectorDot",
8             withExtension: "metal",
9             subdirectory: "Metal"
10        ) else {
11            throw MetalVectorDotError.kernelResourceMissing("P001VectorDot.metal")
12        }
13
14        let source = try String(contentsOf: url, encoding: .utf8)
15        return try MetalVectorDotPipeline(source: source)
16    }
17 }

```

2.18.5 Sources/LeetLLMSolutions/P001VectorDotSolution.swift

```

1 import LeetLLMCore
2
3 public enum P001VectorDotSolution {
4     public static func dot(_ lhs: [Float], _ rhs: [Float]) throws -> Float {
5         guard lhs.count == rhs.count else {
6             throw VectorDotError.lengthMismatch(lhs: lhs.count, rhs: rhs.count)
7         }
8
9         var result: Float = 0
10        for index in lhs.indices {
11            result += lhs[index] * rhs[index]
12        }
13        return result
14    }
15 }

```

2.18.6 Sources/LeetLLMSolutions/Metal/P001VectorDot.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint threadgroupWidth = 256;
5
6 kernel void vector_dot_partial(
7     device const float *lhs [[buffer(0)]],
8     device const float *rhs [[buffer(1)]],
9     device float *partialSums [[buffer(2)]],
10    constant uint &elementCount [[buffer(3)]],
11    uint globalIndex [[thread_position_in_grid]],
12    uint localIndex [[thread_index_in_threadgroup]],
13    uint groupIndex [[threadgroup_position_in_grid]]
14 ) {
15     threadgroup float scratch[threadgroupWidth];
16     scratch[localIndex] = globalIndex < elementCount
17         ? lhs[globalIndex] * rhs[globalIndex]
18         : 0.0f;
19     threadgroup_barrier(mem_flags::mem_threadgroup);
20
21     for (uint stride = threadgroupWidth / 2; stride > 0; stride >>= 1) {
22         if (localIndex < stride) {
23             scratch[localIndex] += scratch[localIndex + stride];
24         }
25         threadgroup_barrier(mem_flags::mem_threadgroup);
26     }
27
28     if (localIndex == 0) {
29         partialSums[groupIndex] = scratch[0];
30     }
31 }
```


Part II

Tensors and Dense Linear Algebra

Tensor Storage and Strides

3.1 Why this exists

An inference equation talks about vectors, matrices, tokens, heads, and channels. Memory contains a one-dimensional sequence of bytes. A tensor layout is the contract that connects those two descriptions.

Every later operator depends on this contract. A projection must know which weights form one row. Attention must distinguish token, head, and feature axes. A KV cache can have the right element count and still return the wrong values if its strides use a different axis order. This problem therefore builds checked row-major tensor storage before adding more arithmetic.

The artifact is `FloatTensor` plus `FloatTensorView` in `LeetLLMCore`. The exercise uses those types to gather values at logical indices. The canonical and learner implementations run through the same judge.

3.2 Learning outcomes

After this problem, you should be able to:

1. Distinguish rank, shape, element count, storage, offset, and stride.
2. Derive row-major strides for any shape.
3. Convert a logical index into an element and byte offset.
4. Explain why a transpose can be a view while a reshape requires contiguity.
5. State the behavior of scalar and zero-sized tensors.
6. Reject invalid dimensions, storage counts, ranks, and indices before access.
7. Identify which layout metadata a later Metal kernel needs.

3.3 Prerequisites

- Problem 001's use of contiguous vectors
- Integer multiplication and array indexing
- Swift thrown errors

No linear algebra beyond rows and columns is needed.

3.4 Vocabulary

Rank The number of logical axes. Shape `[2, 3, 4]` has rank three.

Shape The length of every logical axis.

Storage The flat `[Float]` that owns physical values.

Stride The number of storage elements skipped when one logical index on an axis increases by one.

Offset The first storage element addressed by a view, or the result of mapping a complete logical index into storage.

Contiguous A layout whose strides match packed row-major order for its shape.

View Shape, strides, and a base offset interpreting existing storage without rearranging its values.

3.5 Math from first principles

For rank r , shape

$$\mathbf{d} = [d_0, d_1, \dots, d_{r-1}],$$

has element count

$$N = \prod_{a=0}^{r-1} d_a.$$

Packed row-major order makes the last axis contiguous. Its strides satisfy

$$s_{r-1} = 1, \quad s_a = d_{a+1} s_{a+1}.$$

For logical index $\mathbf{i}$ and view base offset b , the element offset is

$$o(\mathbf{i}) = b + \sum_{a=0}^{r-1} i_a s_a.$$

Every index must obey $0 \leq i_a < d_a$ before this sum is used.

3.5.1 Worked numbers

For shape $[2, 3, 4]$, work from right to left:

```
1 axis 2: stride 1
2 axis 1: 4 * 1 = 4
3 axis 0: 3 * 4 = 12
```

The strides are $[12, 4, 1]$. Index $[1, 2, 3]$ maps to

$$1(12) + 2(4) + 3(1) = 23.$$

For Float32, element 23 starts at byte offset

$$23 \times 4 = 92 \text{ bytes.}$$

A $[2, 3]$ matrix with storage $[0, 1, 2, 3, 4, 5]$ has strides $[3, 1]$. The element at row 1, column 2 is offset $1(3) + 2(1) = 5$ and value 5.

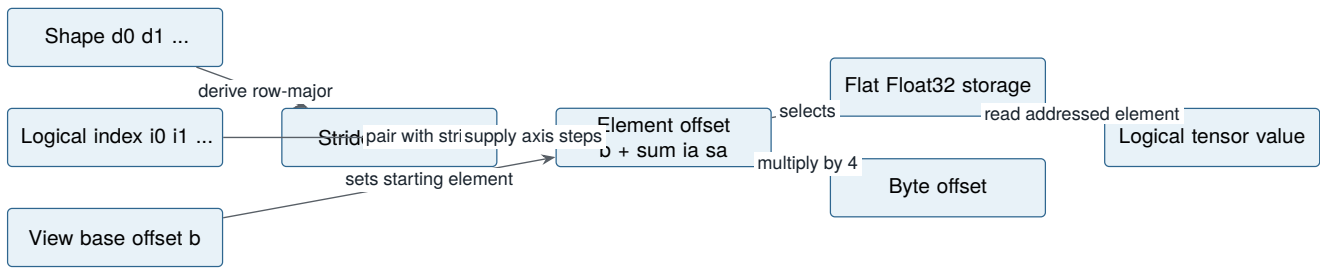


Figure 3.1: Logical index to storage.

3.6 Shape, layout, and dtype contract

This problem uses Float32 storage only.

Property	Contract
Shape	Every dimension is nonnegative
Element count	Product of dimensions, checked for Int overflow
Owned tensor	Storage count equals element count exactly
Row-major strides	Derived from right to left; final stride is one
Index	Rank matches and each coordinate is in bounds
View	Base offset is nonnegative and maximum addressed offset is in storage
Reshape	Element count is unchanged and source layout is contiguous
Transpose view	Rank two; swaps shape and strides without moving values
Scalar	Shape [], one value, index []
Empty tensor	Any zero dimension gives zero elements

FloatTensor is always owned and row-major. FloatTensorView can be non-contiguous. Swift arrays use copy-on-write, so constructing a view does not eagerly duplicate the element buffer, but mutating a copied array would create independent storage. These course views are read-only.

3.7 CPU reference path

Open [P002TensorStorageExercise.swift](#).

The starter already constructs a checked tensor and validates every requested index. It deliberately returns zeros. Replace the placeholder with the readable semantic path:

```

1 tensor = checked FloatTensor(storage, shape)
2 result = empty array
3 for logical index in indices:
4     append tensor.value(at: logical index)
5 return result

```

Run:

```
1 swift run leetllm check 002 --cpu
```

The expected completed result is CPU: 6/6 cases.

3.8 Correctness method

The judge uses deterministic cases that expose different assumptions:

- corners of a [2, 3] matrix;
- rank-three offsets in [2, 3, 4];
- a scalar tensor with shape [1];
- a gather from a tensor containing a zero dimension;
- storage-count mismatch;
- an index equal to its dimension, which is out of bounds.

Direct tests also verify [12, 4, 1], offset 23, contiguous reshape, transpose strides [1, 3], and rejection of reshaping a transpose view. Integer offset math is checked for overflow before storage access.

The independent oracle for a row-major matrix is ordinary nested-loop order: row zero occupies the first columns values, row one the next columns, and so on. Do not validate an offset function only by calling the same function twice.

3.9 Performance model

Indexing performs roughly one integer multiply and add per axis. It then reads four bytes for one Float32 value. For a low-rank tensor this metadata arithmetic is small, but repeated generic indexing inside an inner numerical loop can be material.

A contiguous loop can increment one pointer. A generic rank- r loop may load strides, multiply coordinates, check bounds, and branch for every element. Later kernels therefore validate shape/layout once on the host and use specialized address expressions in the hot loop.

A view allocates only small metadata plus the Swift array value. A materialized layout conversion reads and writes all N values, approximately $8N$ bytes for Float32. That difference is why views are preferred when consumers support the strides.

3.10 Metal mapping

Problem 002 has no Metal exercise because layout is a host-side contract, not a standalone arithmetic kernel. Its consequences on Metal are immediate:

- contiguous vectors need only an element count;
- a generic strided kernel also needs shape, strides, and base offset;
- a rank-two contiguous kernel can specialize offset as $\text{row} * \text{columns} + \text{col}$;
- buffers are byte-addressed by the API, while typed Metal pointers index elements;
- bounds validation belongs on the host, while padded-grid bounds checks remain in the kernel.

Problem 003 will compare passing a transpose view to a stride-aware consumer with paying for a materialized, contiguous transpose.

3.11 Implementation checkpoints

1. Derive [12, 4, 1] for [2, 3, 4] without running code.
2. Inspect `TensorLayout.rowMajor` and identify its overflow check.
3. Make matrix-corner and rank-three gather cases pass.

4. Keep scalar and empty-tensor behavior passing.
5. Confirm invalid storage and invalid indices still throw.
6. Explain why `transposed2D()` changes strides but not storage.
7. Explain why that transpose view cannot be reshaped without copying.

3.12 Controlled experiments

3.12.1 Experiment 1: offset table

Write down all logical indices and offsets for shape `[2, 2, 3]`. Predict that the final coordinate changes every element, the middle every three elements, and the first every six. Verify with `TensorLayout.offset(for:)`.

3.12.2 Experiment 2: view versus copy

Create a `[2, 3]` tensor, inspect its transpose view, then materialize the same values in row-major order. Predict:

- both expose shape `[3, 2]` and equal logical values;
- the view has strides `[1, 3]`;
- the copy has strides `[2, 1]`;
- only the copy can reshape directly to `[6]`.

3.12.3 Experiment 3: traversal locality

For a large row-major matrix, compare summing in row-major order with summing in column-major logical order. Predict the direction of the result before timing. The number of values and FLOPs is identical; only access locality changes.

3.13 Engine integration

Problem 003 uses a transpose view to define the logical result and a tiled copy to materialize it. Problems 004 and 005 require contiguous rank-two tensors and derive every matrix address from this row-major contract. Later, Q/K/V views add token and head axes, while KV-cache problems deliberately compare alternative stride orders.

The checked host types remain the boundary that prevents a correct kernel from being dispatched with an incorrect interpretation of memory.

3.14 Tradeoffs

1. When is a non-contiguous view cheaper than a contiguous copy?
2. When can the slower downstream access pattern repay the copy cost?
3. Why is shape alone insufficient to describe a transpose view?
4. Should a zero stride be accepted for broadcast views? What would mutation mean if many logical indices refer to one value?
5. What metadata would a slice need in addition to shape and strides?
6. Why should overflow be checked before calculating a buffer byte count?
7. When is specializing a kernel for a fixed rank worth the extra code?

3.15 Hints

Gather hint

Construct 'FloatTensor' once, then map each requested index through 'tensor.value(at:)' . Do not recompute a layout for every query.

Reshape hint

A reshape is metadata-only only when linear storage order already matches the new row-major order. Equal element counts are necessary but not sufficient for a non-contiguous view.

3.16 Completion checklist

- ☐ I derived row-major strides for rank two and rank three.
- ☐ I computed offset 23 and byte offset 92 by hand.
- ☐ The learner judge reports 6/6.
- ☐ Invalid storage, rank, and coordinates still throw.
- ☐ I can distinguish scalar shape [] from empty shape [0].
- ☐ I can explain why transpose can be a view.
- ☐ I can explain why only contiguous views reshape without a copy.
- ☐ I recorded predictions and results for the three experiments.
- ☐ I can name the layout metadata a generic Metal kernel would require.

3.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

3.17.1 Sources/LeetLLMExercises/P002TensorStorageExercise.swift

```
1 import LeetLLMCore
2
3 public enum P002TensorStorageExercise {
4     public static func gather(
5         storage: [Float],
6         shape: [Int],
7         indices: [[Int]]
8     ) throws -> [Float] {
9         _ = try FloatTensor(storage, shape: shape)
10        for index in indices {
11            _ = try TensorLayout.rowMajor(shape: shape).offset(for: index)
12        }
13
14        // TODO: Read each logical index through the checked tensor view.
15        return Array(repeating: 0, count: indices.count)
16    }
17 }
```


3.18 Worked solution

Solution strategy

Implements a tensor abstraction that maps logical multi-dimensional indices to physical linear-storage offsets using a stride vector. Decomposes index access into: (1) rank and dimension validation, (2) per-axis bounds checking ($0 \leq i_a < d_a$), (3) offset computation via the formula $o(i) = \text{base_offset} + \sum(i_a * s_a)$, and (4) element retrieval. Provides `FloatTensorView` for arbitrary strides and shapes, and `FloatTensor` for contiguous row-major storage. All arithmetic uses Swift's `multipliedReportingOverflow` and `addingReportingOverflow` to detect `Int` overflow during stride computation and offset accumulation.

Correctness invariant

For rank- r tensor with shape $[d_0, \dots, d_{r-1}]$, row-major strides satisfy $s_{r-1}=1$ and $s_a = d_{a+1} * s_{a+1}$. Element count $= \prod d_a$ must not overflow `Int.max`. Every logical index must obey $0 \leq i_a < d_a$. For non-empty tensors, the maximum addressable offset $\text{base_offset} + \sum((d_a-1)*s_a)$ must be strictly less than `storage.count`. Empty tensors (element count = 0) are valid with offset $\leq \text{storage.count}$.

Common failure mode

Overflow in element count or offset computation is silent without explicit checks. A shape whose dimension product exceeds `Int.max`, such as `[Int.max, 2]`, must throw instead of wrapping. A view `[2,3]` with strides `[1,2]` instead of `[3,1]` changes the row/column mapping and produces incorrect values. Forgetting to validate `base_offset ≥ 0` before adding relative offsets allows an invalid negative base. Transposing a view (swapping shape and strides) is free, but reshaping a non-contiguous view throws—confusion here breaks reshape-based view caching.

3.18.1 Implementation points

Row-Major Stride Derivation

Strides are computed right-to-left, working backward from the last axis. The last axis always has stride 1. For axis a (from $r-2$ down to 0), $\text{stride}[a] = \text{shape}[a+1] * \text{stride}[a+1]$. For shape `[2,3,4]`, strides are: `[12, 4, 1]`. This ensures the last dimension is contiguous in storage—neighboring elements differ by 1 storage position.

Logical vs. Physical Distinction

A view is a shape + strides + offset that reinterprets existing storage. A transpose swaps shape and strides without moving data; a reshape with a transposed view throws `TensorError.reshapeRequiresContiguousLayout`. Materialization (copying to new contiguous storage) is necessary when the consumer requires a different stride order or when tiling a GPU kernel needs packed access patterns.

Overflow Detection at Every Arithmetic Step

The stride derivation loop multiplies dimensions right-to-left, checking multipliedReportingOverflow after each product. The offset computation adds $i_a * s_a$ into a running sum, checking both operations for overflow. For example, `[Int.max, 2]` cannot have a representable element count, so `rowMajor` must throw `TensorError.elementCountOverflow` rather than construct wrapped strides.

Bounds Guarantee for Non-Empty Views

For a view with non-zero element count, the implementation computes the maximum offset reached by any valid index: $(\text{shape}[0]-1)*\text{strides}[0] + \dots + (\text{shape}[r-1]-1)*\text{strides}[r-1]$, adds `base_offset`, and asserts this is strictly less than `storage.count`. This guarantee ensures that `layout.offset(for: validIndex)` never accesses beyond the buffer.

Zero-Sized and Scalar Tensors

A tensor with element count 0 (e.g., shape `[2,0,3]`) is valid and poses no overflow risk because the maximum offset calculation short-circuits (if `layout.elementCount == 0`, only check `offset ≤ storage.count`). A scalar tensor has shape `[]` and rank 0; it accesses `storage[base_offset]` and is valid as long as `base_offset < storage.count`.

3.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P002TensorStorageSolution.swift

- Line 3: `public enum P002TensorStorageSolution {`
- Line 4: `public static func gather(`

3.18.3 Complete canonical listings

3.18.4 Sources/LeetLLMSolutions/P002TensorStorageSolution.swift

```
1 import LeetLLMCore
2
3 public enum P002TensorStorageSolution {
4     public static func gather(
5         storage: [Float],
6         shape: [Int],
7         indices: [[Int]]
8     ) throws -> [Float] {
9         let tensor = try FloatTensor(storage, shape: shape)
10        return try indices.map { try tensor.value(at: $0) }
11    }
12 }
```

Transpose and Tiled Copy

4.1 Why this exists

Changing a tensor's logical axes is free when the consumer understands strides. Changing physical order is not free: every element must be read and written. Inference systems nevertheless materialize transposes when the new order makes many later accesses coalesced, enables a specialized matrix kernel, or matches a model's stored weight format.

This problem makes that tradeoff concrete. The CPU implementation is a readable layout conversion. The Metal implementation is a real tiled transpose whose loads and stores are both contiguous at the threadgroup level. Dimensions do not need to be multiples of the tile width.

4.2 Learning outcomes

After this problem, you should be able to:

1. Derive the index mapping between $[R, C]$ and $[C, R]$.
2. Distinguish a transpose view from a materialized transpose.
3. Implement a CPU row-major transpose.
4. Explain why a naive GPU transpose has a strided side.
5. Map a 16×16 tile through threadgroup memory.
6. Justify the 16×17 scratch layout and both barriers.
7. Handle partial tiles without out-of-bounds reads or writes.
8. Decide when repeated downstream reuse can repay the copy.

4.3 Prerequisites

- Problem 002: shape, row-major strides, views, and offsets
- Problem 001: grids, threadgroups, bounds checks, and barriers
- No matrix multiplication is required yet

4.4 Vocabulary

Transpose A rank-two axis swap. Output element (c, r) equals input element (r, c) .

Materialization Allocating storage and writing values in the logical order of a view.

Coalesced access Neighboring GPU threads access neighboring addresses so memory transactions can be combined efficiently.

Tile A small rectangular region cooperatively processed by one threadgroup.

Bank conflict Serialization caused when threads access shared-memory addresses that map to the same bank. Padding a square tile's row length changes that mapping.

Edge tile A tile that overlaps a matrix boundary and contains invalid padded lanes.

4.5 Math from first principles

Let $A \in \mathbb{R}^{R \times C}$. Its transpose $B = A^T \in \mathbb{R}^{C \times R}$ satisfies

$$B_{c,r} = A_{r,c}.$$

In row-major storage:

$$\text{offset}_A(r, c) = rC + c,$$

$$\text{offset}_B(c, r) = cR + r.$$

For

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix},$$

the input storage is [1, 2, 3, 4, 5, 6] and

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

has storage [1, 4, 2, 5, 3, 6].

The element 6 moves from input offset $1(3) + 2 = 5$ to output offset $2(2) + 1 = 5$. Equal offsets here are accidental; most elements move.

4.6 Shape, layout, and dtype contract

Item	Contract
Input	Contiguous Float32 FloatTensor, rank two, shape [R, C]
Output	Contiguous Float32 tensor, shape [C, R]
Value rule	<code>output[c, r] == input[r, c]</code>
Empty axes	<code>[0, C] -> [C, 0]</code> and <code>[R, 0] -> [0, R]</code>
Invalid input	Any rank other than two throws <code>TensorError.rankMismatch</code>
Metal dimensions	R and C must fit UInt32
Numerical tolerance	1e-6 relative; transpose performs no arithmetic

A transpose view from Problem 002 has shape [C, R] and strides [1, C]. This problem's output instead has row-major strides [R, 1] because it owns a new physical ordering.

4.7 CPU reference path

Open [P003TransposeExercise.swift](#).

The readable algorithm is:

```
1 validate rank two
2 allocate R*C output values
3 for row in 0..

```

Run:

```
1 swift run leetllm check 003 --cpu
```

Keeping the input loop row-major gives contiguous reads and strided writes. If the loops are reversed, writes become contiguous and reads become strided. The CPU oracle prioritizes clarity; the Metal path fixes the one-strided-side problem with tiling.

4.8 Correctness method

The shared judge covers:

- the worked 2x3 matrix;
- a single row;
- an empty row dimension;
- 17x19, which crosses both dimensions of a 16x16 tile;
- rejection of a rank-one input.

Because transpose does no floating-point arithmetic, exact equality would be reasonable. The shared judge still uses a tiny relative tolerance to retain the same reporting style as later GPU operators. The independent oracle computes each destination with the scalar equation $B_{c,r} = A_{r,c}$.

Two useful properties are:

$$(A^T)^T = A$$

and preservation of element multiset. Neither property alone proves the layout is correct: returning the original matrix passes the multiset property, and two identical wrong transformations can pass the involution property.

4.9 Performance model

Transpose performs no floating-point operations. It reads and writes each Float32 once, for minimum traffic

$$4RC + 4RC = 8RC \text{ bytes.}$$

Its arithmetic intensity is effectively zero. Performance is governed by memory traffic, access coalescing, allocation, dispatch, and synchronization.

A materialized transpose also allocates $4RC$ output bytes. A view allocates no element buffer. If one downstream kernel consumes the result once and supports strides efficiently, copying is unlikely to win. If many downstream kernels would otherwise make strided accesses, one copy may amortize its cost.

4.10 Metal mapping

Open [P003Transpose.metal](#).

The canonical dispatch uses:

- one 16x16 threadgroup per input tile;
- grid groups $\text{ceil}(C/16)$ by $\text{ceil}(R/16)$;
- one input load and one output store per valid thread;
- a threadgroup tile declared as `float tile[16][17]`.

For local position (x, y) in group (g_x, g_y) , the load coordinate is

$$r = 16g_y + y, \quad c = 16g_x + x.$$

Threads load neighboring columns, so loads are contiguous. After a barrier, thread positions are exchanged when reading scratch. The output coordinate is

$$r_{out} = 16g_x + y, \quad c_{out} = 16g_y + x.$$

and the value comes from `tile[x][y]`. Neighboring threads now write neighboring output columns, so stores are contiguous too.

The extra scratch column changes the address stride from 16 to 17. When a wave reads the tile down columns, this padding avoids the repeated bank mapping that a square power-of-two row length can create.

4.10.1 Bounds and barriers

The load checks `inputRow < R` && `inputColumn < C`. The store independently checks `outputRow < C` && `outputColumn < R`. These conditions are not interchangeable on rectangular edge tiles.

All 256 threads reach the barrier, including threads whose load was out of bounds. A thread only reads a scratch cell that was written by the corresponding valid transposed input coordinate before storing a valid output coordinate.

4.11 Implementation checkpoints

1. Compute the 2x3 result by hand.
2. Make the CPU rectangular and empty cases pass.
3. Draw one 4x4 tile and exchange local x/y positions.
4. Add threadgroup `float tile[16][17]` to the starter kernel.
5. Implement guarded contiguous loads.
6. Place one unconditional threadgroup barrier after loads.
7. Implement exchanged scratch reads and guarded stores.
8. Pass the 17x19 Metal fixture.

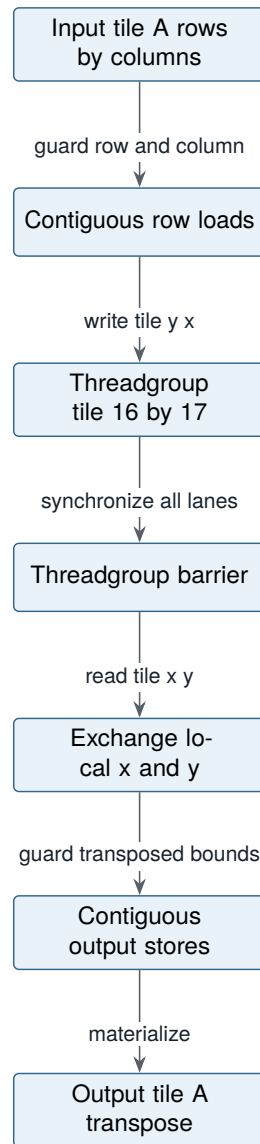


Figure 4.1: Tiled transpose copy.

Run each path independently:

```
1 swift run leetllm check 003 --cpu
2 swift run leetllm check 003 --metal
3 swift run leetllm check 003
```

4.12 Controlled experiments

4.12.1 Experiment 1: shape sweep

Compare square multiples (16x16, 256x256) with partial tiles (17x19, 257x263). Predict that partial tiles do extra padded-thread and bounds work, but the fraction falls as matrices grow.

4.12.2 Experiment 2: skinny matrices

Compare 1x4096, 16x256, and 64x64, all containing 4,096 values. Predict which shapes waste more lanes and which pay the same total byte traffic but use different numbers of partially occupied threadgroups.

4.12.3 Experiment 3: copy amortization

Time a stride-aware consumer over a transpose view, then time materialization plus one, ten, and one hundred contiguous consumer passes. Predict the reuse count at which the up-front 8RC bytes become worthwhile before measuring.

Record build configuration, dimensions, iteration count, and whether timing is kernel-only or end-to-end. Do not report a universal transpose speed from one shape.

4.13 Engine integration

Problem 005's GEMM expects row-major contiguous operands. A model loader may materialize weights once into the preferred orientation rather than transpose on every token. Attention later changes between token-major and head-major views; a tiled copy is justified only when repeated kernels benefit from the new order.

This kernel also establishes the reusable GPU pattern for tiled GEMM: load a bounded tile cooperatively, synchronize, reuse threadgroup memory, and handle edge lanes with identity values.

4.14 Tradeoffs

1. When should a consumer accept strides instead of requiring a copy?
2. Why can a transpose be bandwidth-bound even though the GPU has many ALUs?
3. What goes wrong if only valid edge threads reach the barrier?
4. Why are separate load and store bounds needed for rectangular matrices?
5. What does 16x17 cost in threadgroup memory, and what conflict can it avoid?
6. Could 32x8 outperform 16x16 on some hardware or shapes?
7. When should model weights be transposed at conversion time instead of load time or inference time?

4.15 Hints

CPU hint

Write the destination offset first: output row is the old column, output column is the old row, and the output row length is the old row count.

Metal coordinate hint

Load `'tile[localY][localX]'`. After the barrier, store `'tile[localX][localY]'` using group axes exchanged in the global coordinate.

4.16 Completion checklist

- ☐ I derived both row-major offset equations.
- ☐ I computed the 2x3 example by hand.
- ☐ CPU reports 5/5.
- ☐ I can draw the 16x16 load and exchanged store coordinates.
- ☐ I can justify the tile padding and barrier.
- ☐ Metal reports 5/5, including 17x19.
- ☐ I recorded predictions before the three experiments.
- ☐ I distinguished transpose view cost from materialization cost.
- ☐ I can name a later engine use that repays a layout conversion.

4.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

4.17.1 Sources/LeetLLMExercises/P003TransposeExercise.swift

```
1 import LeetLLMCore
2
3 public enum P003TransposeExercise {
4     public static func transpose(_ input: FloatTensor) throws -> FloatTensor {
5         guard input.rank == 2 else {
6             throw TensorError.rankMismatch(expected: 2, actual: input.rank)
7         }
8
9         // TODO: Materialize the transpose in row-major output order.
10        return try FloatTensor(
11            Array(repeating: 0, count: input.elementCount),
12            shape: [input.shape[1], input.shape[0]]
13        )
14    }
15 }
```

4.17.2 Sources/LeetLLMExercises/P003TransposeMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P003TransposeExercise {
5     public static func makeMetalPipeline() throws -> MetalTransposePipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P003Transpose",
8             withExtension: "metal",
9             subdirectory: "Metal"
10        ) else {
11            throw MetalTransposeError.kernelResourceMissing("P003Transpose.metal")
12        }
13        return try MetalTransposePipeline(source: String(contentsOf: url, encoding: .utf8))
14    }
15 }
```

4.17.3 Sources/LeetLLMExercises/Metal/P003Transpose.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint tileWidth = 16;
5
6 kernel void tiled_transpose(
7     device const float *input [[buffer(0)]],
8     device float *output [[buffer(1)]],
9     constant uint2 &shape [[buffer(2)]],
10    uint2 localPosition [[thread_position_in_threadgroup]],
11    uint2 groupPosition [[threadgroup_position_in_grid]]
12 ) {
13     uint inputRow = groupPosition.y * tileWidth + localPosition.y;
14     uint inputColumn = groupPosition.x * tileWidth + localPosition.x;
15     if (inputRow < shape.x && inputColumn < shape.y) {
16         // TODO: Stage a tile in threadgroup memory and store it transposed.
17     }
18 }
```

```
17     output[inputColumn * shape.x + inputRow] = 0.0f;  
18 }  
19 }
```

4.18 Worked solution

Solution strategy

Materializes a rank-two transpose by creating a new storage array and reindexing each element from $[r,c] \rightarrow [c,r]$. CPU implementation iterates rows then columns of input, computing both source and destination offsets ($\text{src} = r \cdot C + c$, $\text{dst} = c \cdot R + r$), and copies directly. GPU implementation decomposes the matrix into 16×16 tiles. Each threadgroup cooperatively loads one tile into shared memory (with 16×17 padding to avoid bank conflicts), then stores it transposed. Handles partial tiles at edges with guarded reads and writes.

Correctness invariant

Output element at $[c,r]$ equals input element at $[r,c]$ in value. For row-major layout, input offset is $r \cdot C + c$ and output offset is $c \cdot R + r$. The GPU shared-memory transposition requires a thread-group_barrier after loads (before any store) to ensure all 256 lanes have finished reading before column stores begin. Padded lanes (outside matrix bounds) skip both load and store; barrier participation is unconditional.

Common failure mode

During the transposed scratch read, neighboring threads access different rows at the same column. A power-of-two row stride can repeat the same bank mapping; declaring the tile as $[16][17]$ changes that stride and avoids the repeated mapping described by the lesson. Omitting the transpose barrier allows threads to read data before a peer has written it. Forgetting the bounds guard on edge tiles (for example, not checking $\text{inputRow} < \text{shape.x}$ before loading) can read outside the input or write outside the output.

4.18.1 Implementation points

Index Mapping and Offset Computation

The transpose swaps logical axes: $B[c,r] = A[r,c]$. Row-major storage maps $[r,c]$ to offset $r \cdot C + c$ (input) and $[c,r]$ to offset $c \cdot R + r$ (output). The CPU path loops over rows r and columns c of the input, computes both offsets, and copies $\text{storage}[r \cdot C + c]$ to $\text{output}[c \cdot R + r]$. Example: $[1,2,3,4,5,6]$ shaped $[2,3]$ transposes to $[1,4,2,5,3,6]$ shaped $[3,2]$.

View vs. Materialization Trade-off

A transposed view (FloatTensorView with swapped shape and strides) is zero-copy but changes stride interpretation for downstream kernels. A materialized transpose copies all elements to a new contiguous layout, enabling coalesced GPU access and simplifying consumer code. The choice depends on whether downstream kernels expect row-major strides or can handle arbitrary strides.

Shared-Memory Bank Padding

A square 16×16 Float tile has a power-of-two row stride of 16 elements. The transposed access $\text{tile}[x][y]$ walks down rows at a fixed column, so that stride can repeatedly select the same bank

mapping. Declaring `tile[16][17]` changes the scratch row stride from 16 to 17 elements and avoids that repeated pattern without relying on a particular bank count or threadgroup-memory capacity.

Barrier and Synchronization Points

After all 256 threads load their `[localPosition.y][localPosition.x]` values into the shared tile, `threadgroup_barrier(mem_flags::mem_threadgroup)` ensures visibility. Threads then read `tile[localPosition.x][localPosition.y]` (transposed indices). Without the barrier, a store in one thread might happen before a read in another, causing data races. The 16×17 padding doesn't eliminate the need for the barrier—it only eliminates bank conflicts.

Partial-Tile Edge Handling

When `inputRow >= shape.x` or `inputColumn >= shape.y`, the load is skipped (the conditional guards the store to tile). Similarly, on output, if `outputRow >= shape.y` or `outputColumn >= shape.x`, the store is skipped. This allows matrices with dimensions not divisible by 16 (e.g., 17×19) to transpose correctly: the incomplete tile is padded with uninitialized values, which are never written to output.

4.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P003TransposeMetal.swift

- Line 4: `extension P003TransposeSolution {`
- Line 5: `public static func makeMetalPipeline() throws -> MetalTransposePipeline {`

Sources/LeetLLMSolutions/P003TransposeSolution.swift

- Line 3: `public enum P003TransposeSolution {`
- Line 4: `public static func transpose(_ input: FloatTensor) throws -> FloatTensor {`

Sources/LeetLLMSolutions/Metal/P003Transpose.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

4.18.3 Complete canonical listings

4.18.4 Sources/LeetLLMSolutions/P003TransposeMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P003TransposeSolution {
5     public static func makeMetalPipeline() throws -> MetalTransposePipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P003Transpose",
8             withExtension: "metal",
9             subdirectory: "Metal"
10        ) else {
11            throw MetalTransposeError.kernelResourceMissing("P003Transpose.metal")
12        }
13        return try MetalTransposePipeline(source: String(contentsOf: url, encoding: .utf8))
14    }
15 }
```

4.18.5 Sources/LeetLLMSolutions/P003TransposeSolution.swift

```

1 import LeetLLMCore
2
3 public enum P003TransposeSolution {
4     public static func transpose(_ input: FloatTensor) throws -> FloatTensor {
5         guard input.rank == 2 else {
6             throw TensorError.rankMismatch(expected: 2, actual: input.rank)
7         }
8         let rows = input.shape[0]
9         let columns = input.shape[1]
10        var output = Array(repeating: Float.zero, count: input.elementCount)
11        for row in 0..

```

4.18.6 Sources/LeetLLMSolutions/Metal/P003Transpose.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint tileWidth = 16;
5
6 kernel void tiled_transpose(
7     device const float *input [[buffer(0)]],
8     device float *output [[buffer(1)]],
9     constant uint2 &shape [[buffer(2)]],
10    uint2 localPosition [[thread_position_in_threadgroup]],
11    uint2 groupPosition [[threadgroup_position_in_grid]]
12 ) {
13     threadgroup float tile[tileWidth][tileWidth + 1];
14
15     uint inputRow = groupPosition.y * tileWidth + localPosition.y;
16     uint inputColumn = groupPosition.x * tileWidth + localPosition.x;
17     if (inputRow < shape.x && inputColumn < shape.y) {
18         tile[localPosition.y][localPosition.x] = input[inputRow * shape.y + inputColumn];
19     }
20     threadgroup_barrier(mem_flags::mem_threadgroup);
21
22     uint outputRow = groupPosition.x * tileWidth + localPosition.y;
23     uint outputColumn = groupPosition.y * tileWidth + localPosition.x;
24     if (outputRow < shape.y && outputColumn < shape.x) {
25         output[outputRow * shape.x + outputColumn] = tile[localPosition.x][localPosition.y];
26     }
27 }

```

Matrix-Vector Multiplication

5.1 Why this exists

During one-token decode, each linear layer multiplies a large weight matrix by one activation vector. Query, key, value, output, gate, up, and down projections all contain this operation. The arithmetic is a row of dot products; the systems behavior is dominated by rereading weights for every generated token.

This problem turns Problem 001's scalar reduction into a tensor-shaped GEMV operator and keeps Problem 002's shape contract at the API boundary. The CPU path is the semantic oracle. The Metal path assigns one cooperative threadgroup to each output row and reduces columns in Float32.

5.2 Learning outcomes

After this problem, you should be able to:

1. Derive GEMV from row-wise dot products.
2. State the $[M, K] \times [K] \rightarrow [M]$ shape contract.
3. Implement a scalar Float32 CPU oracle.
4. Count GEMV FLOPs and minimum algorithmic bytes.
5. Explain why large decode projections often have low arithmetic intensity.
6. Map one output row to a Metal threadgroup reduction.
7. Handle K smaller than, equal to, or larger than the threadgroup width.
8. Separate kernel arithmetic from allocation, copies, launch, and waiting.

5.3 Prerequisites

- Problem 001: dot products and parallel reductions
- Problem 002: contiguous row-major tensors and rank checking
- Problem 003: memory access and threadgroup coordination

5.4 Vocabulary

GEMV General matrix-vector multiplication, $y = Wx$.

Output row One row of the matrix and therefore one dot product producing one output.

Projection A learned linear map represented by matrix multiplication, usually followed by bias, reshaping, or another neural-network operation.

Weight streaming Reading matrix values that receive little or no reuse during one invocation.

Effective bandwidth Logical bytes in the chosen traffic model divided by elapsed time. It is not a hardware memory-bus counter.

5.5 Math from first principles

For $W \in \mathbb{R}^{M \times K}$ and $\mathbf{x} \in \mathbb{R}^K$,

$$\mathbf{y} = W\mathbf{x}, \quad y_m = \sum_{k=0}^{K-1} W_{m,k}x_k.$$

Every output is exactly Problem 001 applied to one matrix row and the shared vector.

5.5.1 Worked numbers

Let

$$W = \begin{bmatrix} 1 & 2 & 3 \\ -1 & 0.5 & 4 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 2 \\ -1 \\ 0.5 \end{bmatrix}.$$

Then

$$y_0 = 1(2) + 2(-1) + 3(0.5) = 1.5,$$

$$y_1 = -1(2) + 0.5(-1) + 4(0.5) = -0.5.$$

The result is [1.5, -0.5] with shape [2].

5.6 Shape, layout, and dtype contract

Item	Contract
Matrix	Contiguous Float32 tensor, rank two, shape [M, K]
Vector	Contiguous Float32 tensor, rank one, shape [K]
Output	Contiguous Float32 tensor, shape [M]
Accumulation	Float32 in learner and canonical paths
Shape error	Matrix rank, vector rank, or unequal inner dimensions throws
M == 0	Empty output with shape [0]
K == 0	Length-M output filled with the empty-sum value zero
Metal dimensions	M and K fit UInt32
Tolerance	2e-5 relative to an independent Double-accumulation oracle

Requiring contiguous inputs keeps the first GEMV kernel readable. A later operator could accept FloatTensorView and use explicit strides, but that would change both address arithmetic and performance.

5.7 CPU reference path

Open [P004GEMVExercise.swift](#).

The starter preserves all validation and returns the correct output shape with wrong zero values. Replace only the value computation:

```
1 validate ranks and K
2 allocate M zeros
3 for row in 0..

```

Run:

```
1 swift run leetllm check 004 --cpu
```

This loop is intentionally direct. Faster CPU libraries may vectorize, parallelize rows, block caches, or use specialized matrix routines, but the oracle should remain easy to audit.

5.8 Correctness method

The judge covers:

- the worked 2x3 projection;
- $K == 0$ with three output rows;
- $M == 0$;
- three rows with $K == 257$, crossing a 256-thread reduction boundary;
- bad matrix rank;
- inner-dimension mismatch.

Expected values for the wide fixture use Double multiplication and accumulation, then convert once to Float. CPU and Metal implementations accumulate Float in different orders, so the relative tolerance scales with result magnitude.

Shape correctness is checked separately from values. Returning the right count with the wrong tensor rank is not accepted.

5.9 Performance model

GEMV performs approximately

$$2MK \text{ FLOPs.}$$

A minimum one-invocation Float32 traffic model reads MK matrix values, reads K vector values, and writes M outputs:

$$B_{min} = 4(MK + K + M) \text{ bytes.}$$

Therefore

$$I_{GEMV} = \frac{2MK}{4(MK + K + M)}.$$

For large M and K , matrix traffic dominates and intensity approaches 0.5 FLOP/byte. The vector can be reused across rows in cache or threadgroup memory, but each weight is normally used once for one token. That low reuse is why decode projections often behave like bandwidth workloads.

This model omits allocation, Swift-to-buffer copies, cache-line effects, command submission, and synchronization. Those costs matter especially for small matrices.

5.10 Metal mapping

Open [P004GEMV.metal](#).

The host dispatches M threadgroups of 256 threads. Group position identifies the output row. Local thread t accumulates columns

$$t, t + 256, t + 512, \dots < K.$$

Each lane writes one partial sum to a 256-element threadgroup array. The group then uses the same halving reduction as Problem 001:

```
1 128, 64, 32, 16, 8, 4, 2, 1
```

Every reduction level ends with a threadgroup barrier. Local thread zero writes the row result.

When $K < 256$, inactive lanes contribute zero but still participate in every barrier. When $K > 256$, each lane loops over multiple columns before the shared reduction. The host handles $K == 0$ without dispatching or allocating zero-byte input buffers.

The vector is read by every row group. This simple kernel relies on device caches rather than explicitly staging vector tiles. That choice is inspectable, not universally optimal.

5.11 Implementation checkpoints

1. Compute the worked two-row output by hand.
2. Keep rank and inner-dimension failures passing.
3. Make the CPU small and zero-dimension cases pass.
4. Identify row, column, and matrix offset in the Metal kernel.
5. Add a per-lane strided column loop.
6. Store lane sums in threadgroup scratch and synchronize.
7. Add the 256-to-1 reduction with unconditional barriers.
8. Pass the $K == 257$ Metal fixture.

Commands:

```
1 swift run leetllm check 004 --cpu
2 swift run leetllm check 004 --metal
3 swift run leetllm check 004
```

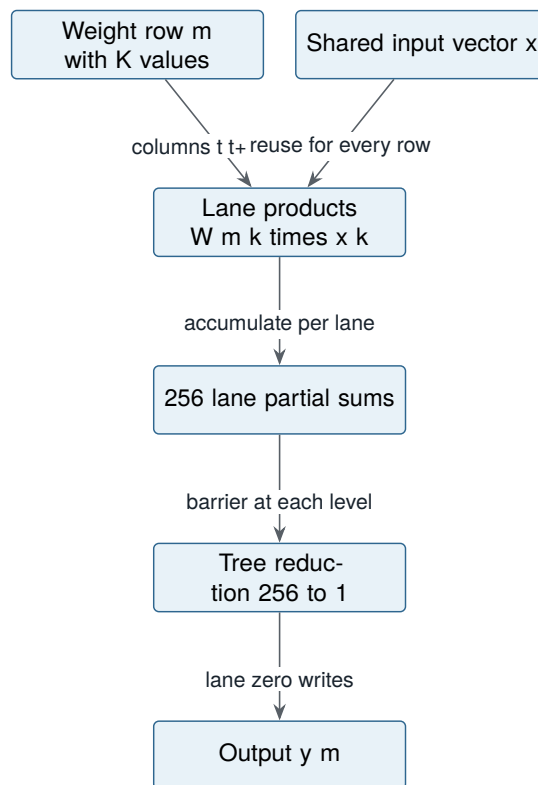


Figure 5.1: Row wise gemv reduction.

5.12 Controlled experiments

5.12.1 Experiment 1: vary rows only

Hold K fixed and sweep M . Predict approximately linear matrix bytes and work. Also predict a fixed launch/allocation floor at small M .

5.12.2 Experiment 2: cross the group width

Compare $K = 255, 256, 257, 512$. Predict that correctness is smooth while work per lane changes discretely. Explain why one extra column at 257 activates a second loop iteration in one lane of every row group.

5.12.3 Experiment 3: same FLOPs, different shape

Compare a tall/narrow and short/wide matrix with similar MK . Predict how the number of thread-groups, reduction occupancy, and vector reuse differ even when the simple FLOP and byte counts are close.

Before measuring, write which path you expect to win and whether you expect bandwidth, reduction work, or fixed overhead to dominate. Use release builds.

5.13 Engine integration

For one decode token, a linear layer uses this exact form:

$$\mathbf{q} = W_q \mathbf{x}, \quad \mathbf{k} = W_k \mathbf{x}, \quad \mathbf{v} = W_v \mathbf{x}.$$

MLP gate/up/down projections are also GEMV. Later quantization problems reduce weight traffic by storing fewer bits and fuse unpacking into this projection. Problem 005 changes the activation from one vector to many token rows, enabling more weight reuse and a different performance regime.

5.14 Tradeoffs

1. Why can the vector be reused while each matrix weight is used only once?
2. When would one thread per row beat one threadgroup per row?
3. What happens to occupancy when M is very small?
4. Would staging vector chunks in threadgroup memory reduce device traffic, or duplicate data already served by cache?
5. What numerical behavior changes with Float16 inputs and Float32 accumulation?
6. Why might fusing bias or activation help a bandwidth-bound projection?
7. Which costs in the host pipeline are absent from the minimum-byte model?

5.15 Hints

CPU offset hint

Row 'm' begins at 'm * K'. Its column 'k' is 'matrix.storage[m * K + k]'.

Metal loop hint

Initialize a private 'float sum = 0'. Start 'column' at 'localIndex' and add the threadgroup width until 'column >= K'.

5.16 Completion checklist

- ☐ I derived GEMV as M row-wise dot products.
- ☐ I computed $[1.5, -0.5]$ by hand.
- ☐ CPU reports 6/6.
- ☐ Metal reports 6/6, including $K == 257$.
- ☐ I can derive the FLOP, byte, and intensity formulas.
- ☐ I can justify every barrier and inactive lane.
- ☐ I recorded predictions before the three experiments.
- ☐ I can explain why decode GEMV often has low weight reuse.
- ☐ I can identify the Q/K/V and MLP integration points.

5.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

5.17.1 Sources/LeetLLMExercises/P004GEMVExercise.swift

```

1 import LeetLLMCore
2
3 public enum P004GEMVExercise {
4     public static func multiply(matrix: FloatTensor, vector: FloatTensor) throws -> FloatTensor {
5         guard matrix.rank == 2 else {
6             throw TensorError.rankMismatch(expected: 2, actual: matrix.rank)
7         }
8         guard vector.rank == 1 else {
9             throw TensorError.rankMismatch(expected: 1, actual: vector.rank)
10        }
11        guard matrix.shape[1] == vector.shape[0] else {
12            throw DenseLinearAlgebraError.innerDimensionMismatch(
13                operation: "GEMV",
14                lhs: matrix.shape[1],
15                rhs: vector.shape[0]
16            )
17        }
18
19        // TODO: Compute one matrix-row/vector dot product per output value.
20        return try FloatTensor(Array(repeating: 0, count: matrix.shape[0]), shape: [matrix.shape[0]])
21    }
22 }

```

5.17.2 Sources/LeetLLMExercises/P004GEMVMetal.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P004GEMVExercise {
5     public static func makeMetalPipeline() throws -> MetalGEMVPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P004GEMV",
8             withExtension: "metal",
9             subdirectory: "Metal"
10        ) else {
11            throw MetalGEMVError.kernelResourceMissing("P004GEMV.metal")
12        }
13        return try MetalGEMVPipeline(source: String(contentsOf: url, encoding: .utf8))
14    }
15 }

```

5.17.3 Sources/LeetLLMExercises/Metal/P004GEMV.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void gemv_rows(
5     device const float *matrix [[buffer(0)]],
6     device const float *vector [[buffer(1)]],
7     device float *output [[buffer(2)]],
8     constant uint2 &shape [[buffer(3)]],
9     uint localIndex [[thread_index_in_threadgroup]],

```

```
10     uint row [[threadgroup_position_in_grid]]
11 ) {
12     if (localIndex == 0 && row < shape.x) {
13         // TODO: Cooperatively reduce this row against the vector.
14         output[row] = 0.0f;
15     }
16 }
```

5.18 Worked solution

Solution strategy

Implements matrix-vector multiplication $y = W \cdot x$ as M independent dot products, one per output element. CPU oracle iterates rows m , then columns k , accumulating $y_m = \sum_k W[m,k] \cdot x[k]$ in Float32. GPU dispatches one threadgroup per output row. Threads within the group cooperatively reduce over K : thread i computes partial sum $\sum_j W[\text{row}, i + j \cdot \text{threadgroupWidth}] \cdot x[i + j \cdot \text{threadgroupWidth}]$, storing in shared memory. After barrier, a tree reduction (stride $128 \rightarrow 64 \rightarrow 32 \rightarrow \dots \rightarrow 1$) produces the final sum in `partials[0]`.

Correctness invariant

Each output y_m is exactly the dot product of matrix row m and the input vector, computed as $\sum_k W[m,k] \cdot x[k]$. Shape contract: $[M,K] \times [K] \rightarrow [M]$. If $K=0$, all outputs are zero (empty sum). Accumulation is Float32 throughout. For thread i in the reduction tree, `partials[i]` must contain the sum of all products computed by thread i before the barrier.

Common failure mode

Decode GEMV has low arithmetic intensity under the course's minimum Float32 traffic model: $\text{FLOPs} = 2 \cdot M \cdot K$, $\text{bytes_moved} = 4 \cdot (M \cdot K + K + M)$, and intensity approaches 0.5 FLOP/byte when matrix traffic dominates. With illustrative peaks of 100 GB/s and 1000 GFLOP/s, the bandwidth roof is therefore about 50 GFLOP/s. Also, when $K > \text{threadgroupWidth}$ (256), the lane loop must advance by 256 or it under-sums the row. Starting the tree reduction at stride 64 instead of 128 likewise omits half of a 256-lane group's partials.

5.18.1 Implementation points

Row-Wise Reduction Decomposition

Each output y_m is independent: $y_m = \sum_k W[m,k] \cdot x[k]$. The CPU oracle uses two loops—rows outside and columns inside—and accumulates one Float per row. The GPU preserves the same row-wise dot-product definition while distributing columns through a lane-strided loop and combining lane partials with a threadgroup reduction.

Threadgroup Collective Reduction

One threadgroup processes one row. All 256 threads cooperatively sum the K elements of that row. Thread i sums $x[i]$, $x[i+256]$, $x[i+512]$, ..., storing partial sums in `partials[i]`. After `threadgroup_barrier`, a binary tree reduces: stride 128 sums `partials[i] += partials[i+128]` (if $i < 128$), then stride 64, ..., down to stride 1. Thread 0 then writes `partials[0]` to `output[row]`.

Strided Loop for $K > \text{Threadgroup Width}$

When $K = 300 > 256$, every lane processes its first column i , and lanes 0 through 43 also process $i+256$. Advancing by `threadgroupWidth` assigns every column to exactly one lane. When $K < 256$, lanes with $i \geq K$ execute no product and store 0 in `partials[i]`, so they contribute the additive

identity to the reduction.

Tree-Reduction Barrier Discipline

Between each stride halving ($\text{stride}/=2$), a `threadgroup_barrier` ensures all additions are visible. For a 256-thread group, the tree has 8 levels: stride 128, 64, 32, 16, 8, 4, 2, 1. Each level has a barrier inside the stride loop. At stride 1, only thread 0 holds the full sum; thread 0 (and only thread 0) writes `output[row]`.

Weight Streaming and Arithmetic Intensity

A $[4096, 4096]$ one-token projection performs $2 \times 4096 \times 4096 = 33,554,432$ FLOPs. The minimum Float32 model reads 64 MiB of weights and 16 KiB of vector data, then writes 16 KiB of output, giving about 0.5 FLOP/byte. With illustrative peaks of 100 GB/s and 1000 GFLOP/s, its bandwidth roof is about 50 GFLOP/s, so the roofline model classifies it as memory-limited. GEMM can raise intensity by reusing weights across token rows during prefill.

5.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P004GEMVMetal.swift

- Line 4: extension `P004GEMVSolution` {
- Line 5: public static func `makeMetalPipeline()` throws -> `MetalGEMVPipeline` {

Sources/LeetLLMSolutions/P004GEMVSolution.swift

- Line 3: public enum `P004GEMVSolution` {
- Line 4: public static func `multiply(matrix: FloatTensor, vector: FloatTensor)` throws -> `FloatTensor` {

Sources/LeetLLMSolutions/Metal/P004GEMV.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

5.18.3 Complete canonical listings

5.18.4 Sources/LeetLLMSolutions/P004GEMVMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P004GEMVSolution {
5     public static func makeMetalPipeline() throws -> MetalGEMVPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P004GEMV",
8             withExtension: "metal",
9             subdirectory: "Metal"
10        ) else {
11            throw MetalGEMVError.kernelResourceMissing("P004GEMV.metal")
12        }
13        return try MetalGEMVPipeline(source: String(contentsOf: url, encoding: .utf8))
14    }
15 }
```


5.18.5 Sources/LeetLLMSolutions/P004GEMVSolution.swift

```

1 import LeetLLMCore
2
3 public enum P004GEMVSolution {
4     public static func multiply(matrix: FloatTensor, vector: FloatTensor) throws -> FloatTensor {
5         guard matrix.rank == 2 else {
6             throw TensorError.rankMismatch(expected: 2, actual: matrix.rank)
7         }
8         guard vector.rank == 1 else {
9             throw TensorError.rankMismatch(expected: 1, actual: vector.rank)
10        }
11        let rows = matrix.shape[0]
12        let columns = matrix.shape[1]
13        guard columns == vector.shape[0] else {
14            throw DenseLinearAlgebraError.innerDimensionMismatch(
15                operation: "GEMV",
16                lhs: columns,
17                rhs: vector.shape[0]
18            )
19        }
20
21        var output = Array(repeating: Float.zero, count: rows)
22        for row in 0..

```

5.18.6 Sources/LeetLLMSolutions/Metal/P004GEMV.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint threadgroupWidth = 256;
5
6 kernel void gemv_rows(
7     device const float *matrix [[buffer(0)]],
8     device const float *vector [[buffer(1)]],
9     device float *output [[buffer(2)]],
10    constant uint2 &shape [[buffer(3)]],
11    uint localIndex [[thread_index_in_threadgroup]],
12    uint row [[threadgroup_position_in_grid]]
13 ) {
14     threadgroup float partials[threadgroupWidth];
15     float sum = 0.0f;
16     for (uint column = localIndex; column < shape.y; column += threadgroupWidth) {
17         sum += matrix[row * shape.y + column] * vector[column];
18     }
19     partials[localIndex] = sum;
20     threadgroup_barrier(mem_flags::mem_threadgroup);
21
22     for (uint stride = threadgroupWidth / 2; stride > 0; stride >= 1) {
23         if (localIndex < stride) {
24             partials[localIndex] += partials[localIndex + stride];
25         }
26         threadgroup_barrier(mem_flags::mem_threadgroup);
27     }
28     if (localIndex == 0) {
29         output[row] = partials[0];
30     }
31 }

```

Matrix-Matrix Multiplication

6.1 Why this exists

Prompt prefill processes many tokens at once. Instead of multiplying one weight matrix by one activation vector, it can multiply matrices. The same weights are then reused across token rows, and activation tiles are reused across output columns. This reuse gives GEMM much higher arithmetic intensity than decode GEMV and lets the GPU spend more time computing relative to moving data.

This problem implements the readable triple-loop definition and a real tiled Metal kernel. The edge fixture deliberately makes all three dimensions partial tiles so correctness cannot depend on convenient multiples.

6.2 Learning outcomes

After this problem, you should be able to:

1. Derive one GEMM output cell as a dot product.
2. State the $[M,K] \times [K,N] \rightarrow [M,N]$ shape contract.
3. Implement and validate a scalar CPU GEMM oracle.
4. Count GEMM FLOPs and minimum algorithmic traffic.
5. Explain why tiling reuses A and B values.
6. Map one output tile to a Metal threadgroup.
7. Zero-fill partial K tiles and guard partial M/N stores.
8. Explain why prefill and decode can favor different kernels.

6.3 Prerequisites

- Problem 002: row-major tensor storage
- Problem 003: tiled loads, barriers, and edge tiles
- Problem 004: row-wise dot products and projection shapes

6.4 Vocabulary

GEMM General matrix-matrix multiplication, $C = AB$.

Inner dimension The shared K dimension reduced to form every output cell.

Data reuse Using a loaded value in more than one multiply before it must be fetched again.

Output tile A rectangular region of C produced by one threadgroup.

K tile One chunk of the reduction dimension loaded into shared storage.

Operational or arithmetic intensity Floating-point work divided by bytes moved under an explicit traffic model.

6.5 Math from first principles

For $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$,

$$C = AB \in \mathbb{R}^{M \times N},$$

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

Each output cell is a dot product between row m of A and column n of B .

6.5.1 Worked numbers

Let

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 3 & 0 \end{bmatrix}.$$

Then

$$C_{0,0} = 1(1) + 2(0) + 3(3) = 10,$$

$$C_{0,1} = 1(2) + 2(-1) + 3(0) = 0,$$

$$C_{1,0} = 4(1) + 5(0) + 6(3) = 22,$$

$$C_{1,1} = 4(2) + 5(-1) + 6(0) = 3.$$

Output storage is $[10, 0, 22, 3]$ with shape $[2, 2]$.

6.6 Shape, layout, and dtype contract

Item	Contract
Left operand	Contiguous Float32, rank two, $[M, K]$
Right operand	Contiguous Float32, rank two, $[K, N]$
Output	Contiguous Float32, $[M, N]$
Accumulation	Float32
Shape errors	Non-matrix rank or unequal inner dimensions throws
$M == 0$ or $N == 0$	Empty storage with shape $[M, N]$
$K == 0$	$M \times N$ zeros by the empty-sum convention
Matlab dimensions	M, K , and N fit UInt32
Tolerance	$4e-5$ relative to a Double-accumulation oracle

The first implementation accepts only row-major FloatTensor. A production GEMM API might support transposed flags or explicit leading dimensions to avoid materialization, but those options enlarge the contract and kernel set.

6.7 CPU reference path

Open [P005GEMMExercise.swift](#).

Implement the direct definition:

```
1 validate ranks and K
2 allocate M*N zeros
3 for row in 0..

```

Run:

```
1 swift run leetllm check 005 --cpu
```

This loop repeatedly walks columns of row-major B with stride N. It is a semantic oracle, not an optimized CPU GEMM. Loop interchange, blocking, packed panels, SIMD instructions, and Accelerate are valid later comparisons.

6.8 Correctness method

The judge includes:

- the worked 2x3 by 3x2 product;
- $K == 0$ with a nonempty 2x3 output;
- $M == 0$;
- 17x19 by 19x18, creating partial M, K, and N tiles;
- bad operand rank;
- unequal inner dimensions.

The independent oracle accumulates each cell in Double and converts once. The canonical implementations accumulate Float, and tiled reduction order differs from a scalar loop, so comparison uses scale-aware tolerance.

Useful properties such as identity multiplication and distributivity supplement fixed fixtures but do not replace them. Shape errors must occur before buffer allocation or dispatch.

6.9 Performance model

The operation performs approximately

$$2MKN \text{ FLOPs.}$$

If every input is read once and output written once, the minimum Float32 traffic is

$$B_{min} = 4(MK + KN + MN) \text{ bytes.}$$

The idealized intensity is

$$I_{GEMM} = \frac{2MKN}{4(MK + KN + MN)}.$$

For square $M = K = N = D$,

$$I_{GEMM} = \frac{2D^3}{12D^2} = \frac{D}{6} \text{ FLOP/byte.}$$

Unlike GEMV's asymptotic intensity near 0.5, GEMM intensity grows with square dimension if caches or threadgroup tiles realize reuse. An untiled implementation may move far more than the minimum because A and B values are reloaded for many output cells.

6.10 Metal mapping

Open [P005GEMM.metal](#).

One 16x16 threadgroup produces one 16x16 output tile. Local thread (x, y) owns output

$$m = 16g_y + y, \quad n = 16g_x + x.$$

For K-tile index q , each thread cooperatively loads:

$$A[m, 16q + x]$$

into `lhsTile[y][x]`, and

$$B[16q + y, n]$$

into `rhsTile[y][x]`. Out-of-bounds positions load zero, the additive identity.

After the first barrier, every thread computes 16 multiply-adds from its tile row and tile column. A second barrier prevents any thread from overwriting scratch for the next K tile while another thread still reads the current tile.

After all $\lceil K/16 \rceil$ tiles, valid M/N threads store one output. The 17x19 by 19x18 judge case checks:

- a second row tile with only one valid row;
- a second column tile with only two valid columns;
- a second K tile with only three valid inner values.

Each 16-value A row in scratch is reused by 16 output columns. Each B column is reused by 16 output rows. That is the central benefit of tiling.

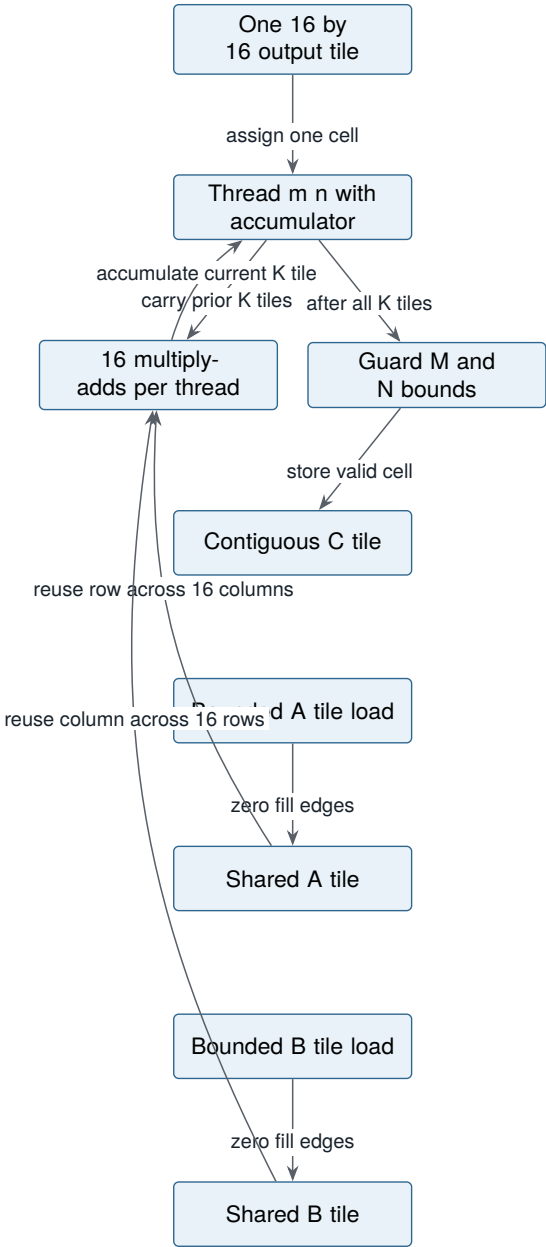


Figure 6.1: Cooperative gemm tiles.

6.11 Implementation checkpoints

1. Compute all four worked output cells by hand.
2. Pass small, empty-output, and zero-K CPU cases.
3. Draw one 4x4 output tile and its A/B input tiles.
4. Add two 16x16 threadgroup arrays.
5. Implement bounded A/B loads with zero fill.
6. Add the barrier before tile computation.
7. Accumulate 16 products per K tile.
8. Add the barrier before the next tile load.
9. Guard final M/N stores and pass the edge fixture.

Commands:

```
1 swift run leetllm check 005 --cpu
2 swift run leetllm check 005 --metal
3 swift run leetllm check 005
```

6.12 Controlled experiments

6.12.1 Experiment 1: square size sweep

Sweep $D = 8, 16, 32, 64, 128$ for $[D, D] \times [D, D]$. Predict a fixed-cost floor at small sizes and rising arithmetic intensity as D grows. Record whether the measured implementation approaches its modeled trend.

6.12.2 Experiment 2: decode-like versus prefill-like

Compare $[1, K] \times [K, N]$ with $[S, K] \times [K, N]$ for increasing token count S . Predict that larger S reuses each B weight across more output rows and fills more output tiles. Connect the first shape to GEMV-like decode.

6.12.3 Experiment 3: edge-tile efficiency

Compare dimensions 63, 64, and 65. Predict that 64 fills every 16x16 group, while 65 launches an additional mostly empty tile along each expanded axis. Explain why the numerical work model and actual dispatched lanes diverge.

Use a release build, warm up the pipeline, and label end-to-end versus kernel timing. Write predictions before collecting times.

6.13 Engine integration

With S prompt tokens arranged as rows, a projection becomes

$$X_{[S, K]} W_{[K, N]} = Y_{[S, N]}.$$

Q/K/V and MLP prefill projections use this form. The operation also supplies the dense components of attention score and weighted-value products before later attention kernels fuse or stream them.

Problem 006 uses this canonical GEMM as an executable roofline workload and compares modeled ceilings with measured CPU and Metal throughput.

6.14 Tradeoffs

1. Why does increasing token rows improve weight reuse?
2. What threadgroup-memory budget does a larger tile consume?
3. Why is the second barrier needed even though each thread owns one output?
4. When could a rectangular tile fit a skinny matrix better than 16×16 ?
5. What changes if B is stored transposed?
6. Why can the idealized byte count understate actual traffic?
7. When should a production engine call MPS/Accelerate instead of maintaining a custom kernel?
8. Which epilogue operations could be fused into the final store?

6.15 Hints

CPU hint

For fixed '(row, column)', only 'inner' changes. A uses 'row*K + inner'; B uses 'inner*N + column'.

Edge-tile hint

Never skip the barriers. Invalid load lanes write zero to scratch and still participate. Only the final output store can be skipped by invalid M/N lanes.

6.16 Completion checklist

- ☐ I derived one output cell as a dot product.
- ☐ I computed [10, 0, 22, 3] by hand.
- ☐ CPU reports 6/6.
- ☐ Metal reports 6/6 with partial M/K/N tiles.
- ☐ I can derive $2MKN$ and the minimum-byte formula.
- ☐ I can explain A/B reuse within one output tile.
- ☐ I can justify both barriers and zero-filled lanes.
- ☐ I recorded predictions before the three experiments.
- ☐ I can explain why prefill GEMM differs from decode GEMV.

6.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

6.17.1 Sources/LeetLLMExercises/P005GEMMExercise.swift

```

1 import LeetLLMCore
2
3 public enum P005GEMMExercise {
4     public static func multiply(_ lhs: FloatTensor, _ rhs: FloatTensor) throws -> FloatTensor {
5         guard lhs.rank == 2 else {
6             throw TensorError.rankMismatch(expected: 2, actual: lhs.rank)
7         }
8         guard rhs.rank == 2 else {
9             throw TensorError.rankMismatch(expected: 2, actual: rhs.rank)
10        }
11        guard lhs.shape[1] == rhs.shape[0] else {
12            throw DenseLinearAlgebraError.innerDimensionMismatch(
13                operation: "GEMM",
14                lhs: lhs.shape[1],
15                rhs: rhs.shape[0]
16            )
17        }
18
19        // TODO: Accumulate each output cell over the shared inner dimension.
20        let shape = [lhs.shape[0], rhs.shape[1]]
21        return try FloatTensor(Array(repeating: 0, count: shape[0] * shape[1]), shape: shape)
22    }
23 }

```

6.17.2 Sources/LeetLLMExercises/P005GEMMMetal.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P005GEMMExercise {
5     public static func makeMetalPipeline() throws -> MetalGEMMPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P005GEMM",
8             withExtension: "metal",
9             subdirectory: "Metal"
10        ) else {
11            throw MetalGEMMError.kernelResourceMissing("P005GEMM.metal")
12        }
13        return try MetalGEMMPipeline(source: String(contentsOf: url, encoding: .utf8))
14    }
15 }

```

6.17.3 Sources/LeetLLMExercises/Metal/P005GEMM.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint tileWidth = 16;
5
6 kernel void tiled_gemm(
7     device const float *lhs [[buffer(0)]],
8     device const float *rhs [[buffer(1)]],

```

```
9     device float *output [[buffer(2)]],
10     constant uint4 &shape [[buffer(3)]],
11     uint2 localPosition [[thread_position_in_threadgroup]],
12     uint2 groupPosition [[threadgroup_position_in_grid]]
13 ) {
14     uint row = groupPosition.y * tileWidth + localPosition.y;
15     uint column = groupPosition.x * tileWidth + localPosition.x;
16     if (row < shape.x && column < shape.z) {
17         // TODO: Cooperatively load A/B tiles and accumulate this output cell.
18         output[row * shape.z + column] = 0.0f;
19     }
20 }
```

6.18 Worked solution

Solution strategy

Implements $C = A * B$ where each output cell $C[m,n] = \sum_k A[m,k] * B[k,n]$. CPU oracle uses a triply-nested loop: rows (m), columns (n), inner dimension (k). GPU algorithm tiles the output: each threadgroup computes a 16×16 block of C. The K dimension is split into chunks of width 16. For each chunk, the group cooperatively loads a $16 \times k_chunk$ slice of A and a $k_chunk \times 16$ slice of B into shared memory (zero-filled at out-of-bounds positions), synchronizes, accumulates the partial outer products, then moves to the next K chunk. After all chunks, threads write their output cells (guarded against out-of-bounds writes).

Correctness invariant

Each output cell $C[m,n] = \sum_k A[m,k] * B[k,n]$. Shape contract: $[M,K] \times [K,N] \rightarrow [M,N]$. Partial K chunks are zero-filled, so out-of-bounds reads produce 0, and $0 * 0$ products contribute 0 to the sum (numerically correct). All accumulation is Float32. The tile accumulation loop iterates 16 inner products per K chunk; after all chunks, the sum is complete.

Common failure mode

Incorrect K-chunk count: $tileCount = (K + 15) / 16$, not $K / 16$. For $K=19$, correct is $\text{ceil}(19/16)=2$; incorrect $tileCount=1$ produces incomplete sums. Confusing shape components: $shape.x$ is M, $shape.y$ is K, $shape.z$ is N; code using $shape.z$ for inner dimension fails. Forgetting the second `threadgroup_barrier` (after accumulating one K chunk) allows a thread to overwrite shared memory before peers finish reading it. Out-of-bounds writes to output (when $row \geq M$ or $column \geq N$) corrupt adjacent memory; the guard `if (row < shape.x && column < shape.z)` prevents this. Zero-filling is crucial: if shared memory is uninitialized, garbage products corrupt the sum.

6.18.1 Implementation points

Output Tile Decomposition

GEMM decomposes $[M,N]$ output into 16×16 tiles, one per threadgroup. Dispatching $(M+15)/16 \times (N+15)/16$ groups ensures complete coverage. Each group computes all 16×16 cells in its tile by iterating over K in chunks; one group is responsible for `output[groupY*16:(groupY+1)*16, groupX*16:(groupX+1)*16]`.

Data Reuse via K Tiling

For each K chunk, the group loads a 16×16 slice of A and a 16×16 slice of B into scratch. Every loaded A value is consumed by 16 output columns, and every loaded B value is consumed by 16 output rows before the next chunk overwrites scratch. This is 16-way reuse within a full output tile. Under the course's ideal minimum-traffic model, a square $[D,D]$ GEMM has intensity $2 * D^3 / (12 * D^2) = D/6$ FLOP/byte; actual traffic depends on whether the implementation realizes that reuse across the full matrix.

K-Chunk Iteration and Count

Let $\text{tileCount} = (K + 15) / 16$. For each tileIndex from 0 to $\text{tileCount}-1$, the group loads $\text{lhsColumn} = \text{tileIndex} * 16 + \text{localPosition.x}$ from A and $\text{rhsRow} = \text{tileIndex} * 16 + \text{localPosition.y}$ from B. The loop index inner goes 0..15, and $\text{sum} += \text{lhsTile}[\text{localPosition.y}][\text{inner}] * \text{rhsTile}[\text{inner}][\text{localPosition.x}]$. For $K=19$, $\text{tileCount}=2$ (ceiling division); the second chunk loads indices [16,17,18] with zero-fill for unused lanes.

Partial Tile Zero Filling

When loading $\text{lhsColumn} \geq K$ or $\text{rhsRow} \geq K$, the element is set to 0.0. When writing output, if $\text{row} \geq M$ or $\text{column} \geq N$, the store is skipped. Zero-filling is numerically safe: $0 * \text{anything} = 0$ contributes 0 to the sum. Example: for $K=19$ in the second chunk, $\text{lhsColumn} \in \{16,17,18\}$, and loads for $\text{lhsColumn} \in \{19..31\}$ are zero-filled. Without zero-filling, uninitialized shared memory would corrupt the result.

Barrier Synchronization at Chunk Boundaries

A `threadgroup_barrier` follows each inner-product loop (stride-16 inner products per K chunk). This ensures all threads have read `lhsTile` and `rhsTile` before the group writes new data for the next chunk. A second barrier follows the chunk load. Without barriers, racing writes and reads corrupt the tile. Correct placement: (1) load both tiles, (2) barrier, (3) loop inner products, (4) barrier, (5) next chunk. Incorrect placement (barrier inside the inner loop) causes false waiting.

6.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P005GEMMMetal.swift

- Line 4: extension `P005GEMMSolution` {
- Line 5: public static func `makeMetalPipeline()` throws -> `MetalGEMMPipeline` {

Sources/LeetLLMSolutions/P005GEMMSolution.swift

- Line 3: public enum `P005GEMMSolution` {
- Line 4: public static func `multiply(_ lhs: FloatTensor, _ rhs: FloatTensor)` throws -> `FloatTensor` {

Sources/LeetLLMSolutions/Metal/P005GEMM.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

6.18.3 Complete canonical listings

6.18.4 Sources/LeetLLMSolutions/P005GEMMMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P005GEMMSolution {
5     public static func makeMetalPipeline() throws -> MetalGEMMPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P005GEMM",
8             withExtension: "metal",
9             subdirectory: "Metal"
10        ) else {
```

```

11         throw MetalGEMMErrors.kernelResourceMissing("P005GEMM.metal")
12     }
13     return try MetalGEMMPipeline(source: String(contentsOf: url, encoding: .utf8))
14 }
15 }

```

6.18.5 Sources/LeetLLMSolutions/P005GEMMSolution.swift

```

1 import LeetLLMCore
2
3 public enum P005GEMMSolution {
4     public static func multiply(_ lhs: FloatTensor, _ rhs: FloatTensor) throws -> FloatTensor {
5         guard lhs.rank == 2 else {
6             throw TensorError.rankMismatch(expected: 2, actual: lhs.rank)
7         }
8         guard rhs.rank == 2 else {
9             throw TensorError.rankMismatch(expected: 2, actual: rhs.rank)
10        }
11        let m = lhs.shape[0]
12        let k = lhs.shape[1]
13        let n = rhs.shape[1]
14        guard k == rhs.shape[0] else {
15            throw DenseLinearAlgebraError.innerDimensionMismatch(
16                operation: "GEMM",
17                lhs: k,
18                rhs: rhs.shape[0]
19            )
20        }
21
22        var output = Array(repeating: Float.zero, count: m * n)
23        for row in 0..

```

6.18.6 Sources/LeetLLMSolutions/Metal/P005GEMM.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint tileWidth = 16;
5
6 kernel void tiled_gemm(
7     device const float *lhs [[buffer(0)]],
8     device const float *rhs [[buffer(1)]],
9     device float *output [[buffer(2)]],
10    constant uint4 &shape [[buffer(3)]],
11    uint2 localPosition [[thread_position_in_threadgroup]],
12    uint2 groupPosition [[threadgroup_position_in_grid]]
13 ) {
14     threadgroup float lhsTile[tileWidth][tileWidth];
15     threadgroup float rhsTile[tileWidth][tileWidth];
16
17     uint row = groupPosition.y * tileWidth + localPosition.y;
18     uint column = groupPosition.x * tileWidth + localPosition.x;
19     float sum = 0.0f;
20     uint tileCount = (shape.y + tileWidth - 1) / tileWidth;
21
22     for (uint tileIndex = 0; tileIndex < tileCount; ++tileIndex) {
23         uint lhsColumn = tileIndex * tileWidth + localPosition.x;

```

```
24     uint rhsRow = tileIndex * tileWidth + localPosition.y;
25     lhsTile[localPosition.y][localPosition.x] = row < shape.x && lhsColumn < shape.y
26         ? lhs[row * shape.y + lhsColumn]
27         : 0.0f;
28     rhsTile[localPosition.y][localPosition.x] = rhsRow < shape.y && column < shape.z
29         ? rhs[rhsRow * shape.z + column]
30         : 0.0f;
31     threadgroup_barrier(mem_flags::mem_threadgroup);
32
33     for (uint inner = 0; inner < tileWidth; ++inner) {
34         sum += lhsTile[localPosition.y][inner] * rhsTile[inner][localPosition.x];
35     }
36     threadgroup_barrier(mem_flags::mem_threadgroup);
37 }
38
39 if (row < shape.x && column < shape.z) {
40     output[row * shape.z + column] = sum;
41 }
42 }
```

Build a Roofline Measurement

7.1 Why this exists

An elapsed time says which implementation was faster in one run. It does not say why, whether the result is close to a plausible machine limit, or which optimization is justified. The roofline model connects arithmetic intensity to two assumed ceilings: peak arithmetic throughput and peak memory bandwidth.

This problem makes the model executable and keeps it separate from observations. `RooflinePrediction` contains only quantities derived from workload counts and assumed machine peaks. `RooflineMeasurement` contains only quantities derived from an elapsed duration. `RooflineReport` displays both and explicitly warns that the model ceiling is not a measurement.

7.2 Learning outcomes

After this problem, you should be able to:

1. Compute arithmetic intensity in FLOP/byte.
2. Derive the bandwidth roof and ridge point.
3. Classify a workload as memory-, compute-, or ridge-limited in the model.
4. Convert an elapsed duration into achieved GFLOP/s and effective GB/s.
5. Distinguish assumed peaks, modeled ceilings, and measured results.
6. Build a reproducible median benchmark report.
7. Explain why algorithmic minimum bytes are not hardware traffic counters.
8. Use a roofline result to choose the next experiment rather than declare a universal bottleneck.

7.3 Prerequisites

- Problem 004: GEMV work and byte counts
- Problem 005: GEMM reuse and tiled execution
- Units: seconds, nanoseconds, bytes, FLOP/s, and byte/s

7.4 Vocabulary

Arithmetic intensity Floating-point operations divided by bytes moved under a stated traffic model.

Compute peak An assumed maximum arithmetic rate for a machine and precision.

Bandwidth peak An assumed maximum memory-transfer rate for a machine and memory path.

Bandwidth roof IP_B , the compute rate that available bandwidth could sustain at intensity I .

Ridge point The intensity P_C/P_B where compute and bandwidth roofs meet.

Modeled ceiling The lower of compute peak and bandwidth roof. It is a prediction under model assumptions, not an observation.

Measured throughput Work divided by observed elapsed time for one implementation and benchmark boundary.

7.5 Math from first principles

Let workload FLOPs be F , modeled bytes be B , compute peak be P_C in GFLOP/s, and bandwidth peak be P_B in GB/s.

Arithmetic intensity is

$$I = \frac{F}{B} \text{ FLOP/byte.}$$

Because GFLOP and GB both use 10^9 , the bandwidth roof is directly

$$P_{BW} = IP_B \text{ GFLOP/s.}$$

The roofline prediction is

$$P_{model} = \min(P_C, P_{BW}).$$

The ridge point is

$$I_{ridge} = \frac{P_C}{P_B}.$$

Below the ridge, the model calls the workload memory-limited. Above it, the compute roof is lower. Equality is classified as balanced.

7.5.1 Worked numbers

Suppose $P_C = 1000$ GFLOP/s and $P_B = 100$ GB/s. The ridge is

$$I_{ridge} = 1000/100 = 10 \text{ FLOP/byte.}$$

A streaming workload with $F = 2000$ FLOPs and $B = 8000$ bytes has

$$I = 0.25, \quad P_{BW} = 0.25(100) = 25 \text{ GFLOP/s.}$$

The modeled ceiling is 25 GFLOP/s and the model classifies it as memory-limited.

If that workload takes 2 microseconds, the measured rates are

$$P_{measured} = \frac{2000}{2 \times 10^{-6}} / 10^9 = 1 \text{ GFLOP/s,}$$

$$BW_{effective} = \frac{8000}{2 \times 10^{-6}} / 10^9 = 4 \text{ GB/s.}$$

The 25 GFLOP/s ceiling remains a model result; the 1 GFLOP/s value is the observation. The gap could include launch overhead, poor access, insufficient parallelism, cache behavior, or incorrect peak assumptions.

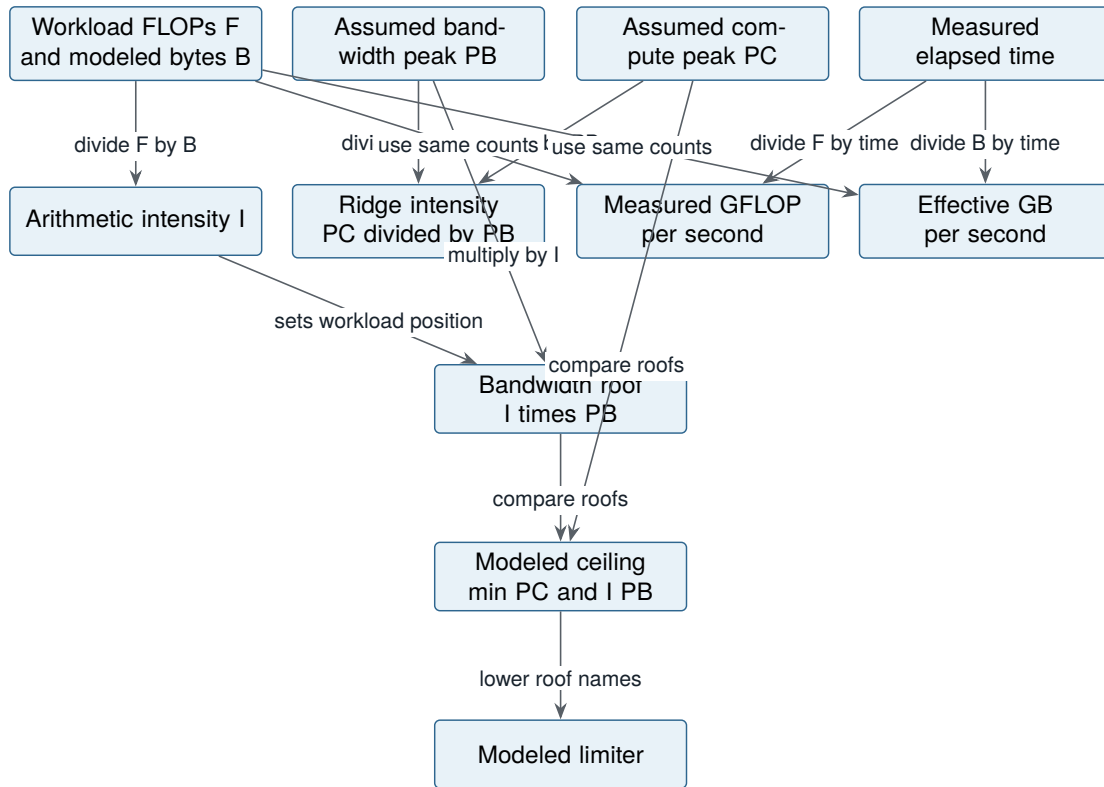


Figure 7.1: Roofline model and measurement.

7.6 Shape, layout, and dtype contract

Problem 006 models metadata rather than returning a tensor, but its inputs still have a strict contract.

Item	Contract
Workload FLOPs	Nonnegative Double
Workload bytes	Positive Double, with the traffic model named
GEMV/GEMM dimensions	Nonnegative integers; products are formed as Double
Machine compute peak	Positive GFLOP/s
Machine bandwidth peak	Positive GB/s
Measured duration	Positive UInt64 nanoseconds
Benchmark iterations	Positive integer; median selected
GEMM workload	Float32 minimum bytes 4(MK + KN + MN)
Report	Model always present; measurement explicitly optional

The canonical 006 benchmark uses Problem 005 Float32 GEMM. It reports algorithmic minimum traffic, not cache-line transactions, host copies, or a hardware counter. CPU and Metal measurements use the same workload counts but different implementation boundaries.

7.7 CPU reference path

Open [P006RooflineExercise.swift](#).

Implement these four steps:

```
1 intensity = FLOPs / bytes
2 bandwidthCeiling = intensity * peakBandwidth
3 predictedCeiling = min(peakCompute, bandwidthCeiling)
4 classify by comparing bandwidthCeiling with peakCompute
```

Run:

```
1 swift run leetllm check 006 --cpu
```

The judge uses memory-bound, compute-bound, and exact-ridge fixtures. Input validation is enforced by the throwing initializers for `RooflineWorkload` and `RooflineMachine`, so the prediction function receives valid values.

7.8 Correctness method

Deterministic tests check:

- intensity, bandwidth roof, lower ceiling, and limiter;
- all three limiter classifications;
- rejection of negative FLOPs and nonpositive bytes/peaks;
- exact GEMM FLOP and minimum-byte counts;
- conversion of a fixed 2 ms duration into measured rates;
- report text containing distinct `MODEL` and `MEASURED` labels.

Benchmark timing itself is nondeterministic and is not used as a unit-test oracle. Tests construct `RooflineMeasurement` from a known duration. This tests the report arithmetic without pretending elapsed wall time is deterministic.

7.9 Performance model

The roofline model is deliberately simple. It assumes one dominant memory roof, one compute roof, enough parallelism to approach them, and a known byte count. Real execution may add:

- host-to-shared-buffer copies;
- cache hits that reduce external traffic;
- cache misses and write allocation that increase traffic;
- command submission and synchronization;
- threadgroup occupancy and instruction dependencies;
- scalar setup and allocation;
- thermal and frequency variation.

For Problem 005 GEMM,

$$F = 2MKN,$$

$$B_{min} = 4(MK + KN + MN).$$

For Problem 004 GEMV,

$$F = 2MK,$$

$$B_{min} = 4(MK + K + M).$$

The formulas predict why increasing token rows can move projection work toward the compute side of the roofline, provided the implementation realizes reuse.

7.10 Metal mapping

Problem 006 introduces no new kernel. Its Metal measurement runs the canonical tiled GEMM from Problem 005. This is important: GPU checking still executes real Metal work rather than a CPU stand-in.

The CLI constructs the Metal pipeline once, outside timed iterations. Each timed call currently includes:

- shared-buffer allocation;
- copying both Swift tensors;
- command encoding and submission;
- GPU kernel execution;
- CPU/GPU synchronization;
- copying the output buffer into a Swift array.

The report labels this Metal canonical GEMM (end-to-end). It must not be compared with a kernel-only GPU peak as though the benchmark boundary contained only arithmetic. A later profiler can isolate GPU timestamps and counters.

7.11 Implementation checkpoints

1. Compute the worked intensity and 25 GFLOP/s roof by hand.
2. Make the memory-limited judge fixture pass.
3. Add compute and balanced classification.
4. Inspect RooflineMeasurement and verify unit conversions.
5. Run the canonical solution check.
6. Choose machine peak assumptions and record their source.
7. Run the release benchmark and retain both CPU and Metal reports.
8. Explain at least one reason each measured value is below its model ceiling.

Commands:

```
1 swift run leetllm check 006
2 swift run leetllm check 006 --solution
3 swift run -c release leetllm benchmark 006 \
4   --m 64 --k 64 --n 64 --iterations 20 \
5   --peak-gflops 1000 --bandwidth-gbps 100
```

The default peak values are examples, not detected properties of the machine. Replace them with values whose source and precision assumptions you can defend.

7.12 Controlled experiments

7.12.1 Experiment 1: move across the ridge

Hold K and N fixed while increasing M from 1 to 128. Before running, predict how minimum-byte intensity changes as B weights are reused across more rows. Record model classification and measured CPU/Metal rates separately.

7.12.2 Experiment 2: fixed model, changed boundary

Measure end-to-end Metal as provided, then use GPU timestamps or a profiler to isolate kernel duration. Predict that removing allocation, copy, and wait costs matters most for small matrices. Do not change workload counts between the two reports.

7.12.3 Experiment 3: peak sensitivity

Keep one measured sample and evaluate it with two defensible peak pairs, such as conservative sustained values and published theoretical values. Predict which classifications stay stable. Explain why the measurement does not change when the assumed roof changes.

7.12.4 Required report record

```

1 Machine and OS:
2 Swift build configuration:
3 Operator and shape:
4 Traffic model:
5 Peak compute assumption and source:
6 Peak bandwidth assumption and source:
7 Predicted intensity and limiter:
8 CPU measured boundary and median:
9 Metal measured boundary and median:
10 Prediction matched the observed trend? Why?
11 Next measurement that would discriminate between explanations:

```

7.13 Engine integration

Every later operator can carry a `RooflineWorkload` estimate beside its measured sample. Activations add low-intensity memory passes. Fused kernels attempt to remove those passes. Attention changes intensity through tiling and online softmax. Quantized GEMV reduces weight bytes while adding unpacking arithmetic.

The model does not choose optimizations automatically. It narrows the next question: reduce bytes for a memory-side operator, improve arithmetic execution for a compute-side operator, or isolate fixed overhead when neither roof explains the gap.

7.14 Tradeoffs

1. Which memory level does the bandwidth peak represent?
2. When is algorithmic minimum traffic a poor estimate of physical traffic?
3. Can a workload modeled as compute-limited still run slowly from low occupancy?
4. Why does changing an assumed peak not change a measured sample?
5. How would Float16 storage with Float32 accumulation change FLOPs and bytes?
6. Should command submission be included in an operator benchmark used for an end-to-end decode decision?
7. What counter or timing boundary would test whether allocation dominates?
8. Why is one roofline point insufficient to characterize an implementation?

7.15 Hints

Unit hint

FLOP/byte times GB/s produces GFLOP/s because both prefixes contribute the same 10^9 factor.

Classification hint

Compare the bandwidth-derived ceiling with compute peak. The smaller roof is the modeled limiter; exact equality is the ridge case.

7.16 Completion checklist

- ☐ I computed intensity, bandwidth roof, and ridge point by hand.
- ☐ The learner judge reports 3/3.
- ☐ I can distinguish assumed, modeled, and measured values.
- ☐ I can derive GEMV and GEMM workload counts.
- ☐ I ran the 006 benchmark in release mode.
- ☐ I recorded the source and precision of peak assumptions.
- ☐ I labeled CPU and Metal benchmark boundaries.
- ☐ I completed all three experiments with predictions written first.
- ☐ I chose one next measurement that discriminates between explanations.
- ☐ I did not describe an effective byte rate as a hardware traffic counter.

7.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

7.17.1 Sources/LeetLLMExercises/P006RooflineExercise.swift

```
1 import LeetLLMCore
2
3 public enum P006RooflineExercise {
4     public static func predict(
5         workload: RooflineWorkload,
6         machine: RooflineMachine
7     ) -> RooflinePrediction {
8         // TODO: Compute intensity, the bandwidth roof, the lower ceiling, and its limiter.
9         RooflinePrediction(
10             arithmeticIntensity: 0,
11             bandwidthCeilingGFLOPS: 0,
12             predictedCeilingGFLOPS: 0,
13             bottleneck: .balanced
14         )
15     }
16 }
```

7.18 Worked solution

Solution strategy

Decouples the roofline model (a prediction based on algorithm properties and assumed machine peaks) from measurement (observed elapsed time). `RooflineModel.predict` computes arithmetic intensity $I = \text{FLOPs} / \text{bytes}$, bandwidth ceiling $= I * \text{peakBandwidth}$ (GB/s), and model ceiling $= \min(\text{peakCompute}, \text{bandwidthCeiling})$. Classification: if $\text{bandwidthCeiling} < \text{peakCompute}$, memory-limited; if $\text{bandwidthCeiling} > \text{peakCompute}$, compute-limited; if equal, balanced. `RooflineMeasurement` converts elapsed nanoseconds into achieved GFLOP/s and effective GB/s. `RooflineBenchmark` collects samples across iterations, sorts them, and returns the median to suppress tail noise from OS preemption.

Correctness invariant

Arithmetic intensity is a property of the algorithm (chosen FLOPs and bytes), not the system. The roofline model is a prediction under stated assumptions (peak values, traffic model); it is not a measurement. Measured throughput is an observation of one implementation on one run, subject to noise and system variance. The model ceiling is valid only if peak assumptions and traffic counts match the actual system. A measured result above the model ceiling indicates an incorrect assumption, not a violation.

Common failure mode

Confusing algorithmic minimum bytes with traffic measured at a particular memory hierarchy. Cache hits can reduce external traffic, while cache-line fetches, write allocation, copies, and coherency can add traffic; the model is meaningful only when its byte definition is stated. For GEMV, this course uses $4 * (M * K + K + M)$ logical bytes. The benchmark sorts elapsed samples and selects `samples[count/2]`, which reduces the influence of large outliers but does not remove all timing variance. Peak compute and bandwidth are explicit assumptions, not properties detected by the implementation.

7.18.1 Implementation points

Arithmetic Intensity Computation

Intensity $I = \text{FLOPs} / \text{bytes_moved}$ (FLOP/byte). For GEMV $[M, K] \times [K]$, $\text{FLOPs} = 2 * M * K$ and minimum Float32 bytes $= 4 * (M * K + K + M)$, so $I = 2 * M * K / (4 * (M * K + K + M))$ and approaches 0.5 when matrix traffic dominates. For GEMM $[M, K] \times [K, N]$, $\text{FLOPs} = 2 * M * K * N$ and minimum bytes $= 4 * (M * K + K * N + M * N)$. For square $M = K = N = D$, $I = 2 * D^3 / (12 * D^2) = D/6$.

Roofline Prediction

Given intensity I (FLOP/byte), peakCompute P_C (GFLOP/s), and peakBandwidth P_B (GB/s): $\text{bandwidthCeiling} = I * P_B$, $\text{predictedCeiling} = \min(P_C, \text{bandwidthCeiling})$. The ridge point is P_C / P_B ; below this, bandwidth is limiting; above, compute is limiting. Example: $P_C = 1000$, $P_B = 100$, ridge = 10 FLOP/byte. A workload with $I = 5$ has $\text{bandwidthCeiling} = 5 * 100 = 500$ GFLOP/s < 1000 , so it is memory-limited.

Bottleneck Classification

Compare bandwidthCeiling to peakCompute: (1) if bandwidthCeiling < peakCompute, the model is memory-limited; (2) if bandwidthCeiling > peakCompute, the model is compute-limited; (3) if equal, it is balanced. This names the lower modeled roof under the stated intensity and peak assumptions. It does not prove what limits a particular implementation; measurements and a clearly defined timing boundary are still required.

Measurement vs. Prediction

A model predicts a ceiling under stated assumptions; a measured result records one implementation and timing boundary. If measured GFLOP/s exceeds the model ceiling, recheck the workload FLOPs, byte model and memory level, peak assumptions, and unit conversions. Launches, copies, and synchronization can explain a result below the ceiling, but they cannot explain one above it. An above-ceiling observation therefore exposes a mismatch between the model assumptions and the measurement.

Median Aggregation for Robust Benchmarking

RooflineBenchmark measures each operation with DispatchTime.now().uptimeNanoseconds, sorts the K durations, and selects samples[K/2]. This is the middle sample for odd K and the upper middle sample for even K. In the illustrative set [100, 101, 105, 500, 501] ns, that rule returns 105 ns while the mean is 241.4 ns, demonstrating the median's lower sensitivity to extreme tail samples.

7.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P006RooflineSolution.swift

- Line 3: public enum P006RooflineSolution {
- Line 4: public static func predict(

7.18.3 Complete canonical listings

7.18.4 Sources/LeetLLMSolutions/P006RooflineSolution.swift

```

1 import LeetLLMCore
2
3 public enum P006RooflineSolution {
4     public static func predict(
5         workload: RooflineWorkload,
6         machine: RooflineMachine
7     ) -> RooflinePrediction {
8         RooflineModel.predict(workload: workload, machine: machine)
9     }
10 }
```


Part III

Neural-Network Operators

ReLU, GELU, and SiLU

8.1 Why this exists

A linear layer can rotate, scale, and combine features, but a stack of linear layers without a nonlinearity is still one linear transformation. Decoder MLPs therefore place an elementwise activation between projections. This lesson implements three common choices and separates their arithmetic cost from the cost of reading and writing another full tensor.

The reusable artifact is `Activation`: later code can name the convention it expects instead of passing an unlabelled closure. In particular, this lesson does not use the phrase “GELU” to hide a formula choice.

8.2 Learning outcomes

After completing the problem, you can:

- distinguish exact GELU from the tanh approximation used by this course;
- evaluate ReLU, approximate GELU, and SiLU for positive and negative inputs;
- preserve a `FloatTensor`’s shape while transforming its contiguous storage;
- map an elementwise operation to one Metal thread per element;
- explain when activation latency is math-bound and when it is another memory pass;
- verify a `Float32` implementation against an independent `Double` oracle.

8.3 Prerequisites

- Problem 002 for `FloatTensor`, row-major storage, and shape validation.
- Problem 006 for bytes moved, arithmetic intensity, and launch overhead.
- Exponentials and hyperbolic tangent from the [Math Primer](#).

8.4 Vocabulary

- **Activation**: an elementwise nonlinear function applied after a linear projection.
- **ReLU**: rectified linear unit, $\max(0, x)$.
- **CDF**: cumulative distribution function. Exact GELU scales x by the probability that a standard normal variable is no greater than x .
- **Exact GELU**: the error-function definition below.
- **Tanh-approximate GELU**: a commonly used approximation with only tanh and polynomial arithmetic. It is the GELU convention implemented here.
- **SiLU**: sigmoid linear unit, also called swish with beta equal to one.
- **Elementwise kernel**: a kernel whose output index depends only on the same input index.

8.5 Math from first principles

For scalar input x , ReLU is

$$\text{ReLU}(x) = \max(0, x).$$

The exact Gaussian error linear unit is

$$\text{GELU}_{\text{exact}}(x) = \frac{x}{2} \left(1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right).$$

This problem deliberately chooses the tanh approximation:

$$\text{GELU}_{\text{tanh}}(x) = \frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right).$$

Exact and approximate GELU are close, but they are not interchangeable test or weight-format conventions. `Activation.geluTanhApproximation` makes that choice visible at the call site.

SiLU multiplies a value by its sigmoid gate:

$$\text{SiLU}(x) = x\sigma(x) = \frac{x}{1 + e^{-x}}.$$

Unlike ReLU, GELU and SiLU retain small negative outputs. They suppress rather than hard-zero every negative feature.

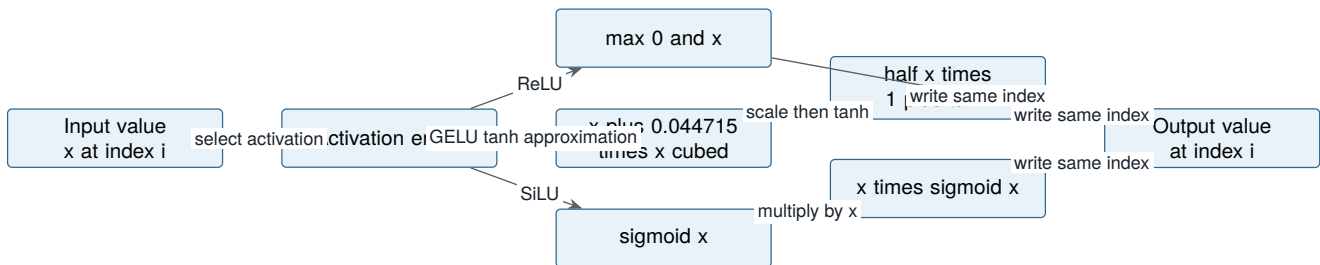


Figure 8.1: Elementwise activation paths.

8.5.1 Worked numerical example

For $x = [-1, 0, 2]$:

x	ReLU	GELU tanh approximation	SiLU
-1	0	about -0.158808	about -0.268941
0	0	0	0
2	2	about 1.954598	about 1.761594

For the GELU value at $x = -1$, compute the cubic term first: $x + 0.044715x^3 = -1.044715$. Multiplying by $\sqrt{2/\pi}$ gives about -0.833 , and the final half-scaled gate gives the negative output above.

8.6 Shape, layout, and dtype contract

ActivationImplementation receives a contiguous row-major FloatTensor of any rank and an Activation. It returns:

- the same shape, rank, strides, and element count;
- newly computed Float32 storage;
- an empty tensor unchanged when any dimension makes the element count zero.

There is no broadcasting and no shape-dependent formula. The CPU solution uses Float32 arithmetic. The judge computes expected values with Double and then rounds once to Float, using absolute tolerance $2e-6$ plus relative tolerance $2e-5$.

8.7 CPU reference path

Start with a storage map, not nested shape loops:

```
1 let output = input.storage.map { value in
2   switch activation {
3     case .relu:
4       return max(0, value)
5     case .geluTanhApproximation:
6       let cubic = value * value * value
7       let inner = sqrt(2 / Float.pi) * (value + 0.044715 * cubic)
8       return 0.5 * value * (1 + tanh(inner))
9     case .silu:
10      return value / (1 + exp(-value))
11   }
12 }
```

Construct the result with `input.shape`; do not flatten the logical contract to `[N]` merely because the physical storage is flat.

8.8 Correctness method

The shared judge checks signs and zero for ReLU, representative GELU values, wide SiLU inputs, matrix shape preservation, and an empty tensor. Expected values come from separate Double formulas in the core target, not from the solution function.

Pay attention to these failure modes:

- implementing exact GELU while the API names the tanh approximation;
- omitting the cubic term or the factor $\sqrt{2/\pi}$;
- returning sigmoid instead of $x * \text{sigmoid}(x)$ for SiLU;
- producing the right flat values with the wrong shape;
- branching on signs in a way that turns negative zero into a numerical issue.

Run:

```
1 swift run leetllm check 007 --cpu
2 swift run leetllm check 007 --metal
3 swift run leetllm check 007 --solution
```

8.9 Performance model

For N Float32 values, every implementation reads $4N$ bytes and writes $4N$ bytes, so compulsory traffic is approximately $8N$ bytes. ReLU adds one comparison. GELU adds a cubic polynomial and a transcendental $\tanh$; SiLU adds an exponential and division.

Arithmetic intensity is therefore activation-dependent, but all three launch the same number of threads and move the same tensor bytes. For small tensors, dispatch and synchronization can dominate. For large ReLU tensors, traffic is likely to dominate. GELU and SiLU give the GPU's math units more work per byte.

A separate activation also writes an intermediate that a later fused projection or gate could consume in registers. Problem 008 removes one such pass for SiLU plus multiplication.

8.10 Metal mapping

The Metal grid is one-dimensional with N threads. Thread i reads `input[i]`, evaluates the selected formula, and writes `output[i]`. There is:

- no threadgroup memory;
- no barrier;
- one bounds check for a partial final threadgroup;
- no dependency between elements.

The host passes the element count and the `Activation` raw value as constant buffers. The canonical shader lives at [P007Activation.metal](#). The starter uses the same bindings and compiles, but currently copies input.

8.11 Implementation checkpoints

1. Implement ReLU and make only the ReLU judge case pass.
2. Add SiLU and verify negative values are not all zero.
3. Add the named $\tanh$ GELU approximation and compare the worked fixture.
4. Preserve matrix and empty-tensor shapes.
5. Port the same switch to MSL with one thread per element.
6. Run the CPU and Metal paths through the same judge.

8.12 Controlled experiments

8.12.1 Experiment A: same bytes, different math

Measure ReLU, GELU, and SiLU for the same large N . Before running, write a prediction. A defensible prediction is: ReLU has the lowest arithmetic cost, while GELU and SiLU take longer because transcendental instructions increase work without changing bytes moved.

Control tensor size, allocation policy, iteration count, and synchronization. Report end-to-end and kernel-only timing separately if you add command-buffer timing.

8.12.2 Experiment B: launch overhead

Sweep N through 32, 256, 4096, and 1_048_576. Prediction: CPU can win at the smallest sizes because one Metal dispatch costs more than the arithmetic; the GPU becomes useful only after enough independent elements amortize launch cost.

8.12.3 Experiment C: separate versus fused use

Compare standalone SiLU followed by multiplication with Problem 008's fused $\text{SiLU}(\text{gate}) * \text{up}$ kernel. Prediction: fusion saves one intermediate read and write and one dispatch; the benefit should grow with hidden width.

8.13 Engine integration

Problem 008 reuses SiLU directly in SwiGLU. Later model-loading code must map a model's declared activation to this enum and reject unsupported conventions. The elementwise kernel is also a baseline for deciding whether an activation should remain separate or be fused into a producer or consumer.

8.14 Tradeoffs

- ReLU is cheap and sparse, but hard-zeroing negatives changes model behavior.
- Exact GELU is a clearer mathematical definition; tanh GELU may map to the available runtime operations more directly. The model convention decides.
- A standalone kernel is simple and reusable; fusion can remove traffic but duplicates formulas and increases kernel specialization.
- Fast-math approximations may improve speed but require a newly measured tolerance.

8.15 Hints

- Compute $\text{value} * \text{value} * \text{value}$; do not call a generic power routine for a cubic.
- Keep all constants as Float in the canonical CPU path to model its actual dtype.
- In MSL, use the global thread index and return when it is outside count.
- If only positive cases pass, inspect whether the function should preserve a small negative value rather than clamp it.

8.16 Completion checklist

- ☐ CPU ReLU, tanh-approximate GELU, and SiLU pass the judge.
- ☐ The output preserves arbitrary input shapes and empty storage.
- ☐ The Metal implementation executes actual MSL and passes the same cases.
- ☐ You can state both exact and chosen approximate GELU formulas.
- ☐ You recorded a prediction before one controlled size or activation sweep.
- ☐ You explained the result using math cost, bytes, or launch overhead.
- ☐ You can identify where Problem 008 removes an activation memory pass.

8.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

8.17.1 Sources/LeetLLMExercises/P007ActivationExercise.swift

```
1 import LeetLLMCore
2
3 public enum P007ActivationExercise {
4     public static func apply(
5         _ input: FloatTensor,
6         activation: Activation
7     ) throws -> FloatTensor {
8         // TODO: Apply the selected activation independently to every value.
9         try FloatTensor(input.storage, shape: input.shape)
10    }
11 }
```

8.17.2 Sources/LeetLLMExercises/P007ActivationMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P007ActivationExercise {
5     public static func makeMetalPipeline() throws -> MetalActivationPipeline {
6         guard let url = Bundle.module.url(forResource: "P007Activation", withExtension: "metal", subdirectory: "Metal")
7             ↪ else {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P007Activation.metal")
9         }
10        return try MetalActivationPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }
```

8.17.3 Sources/LeetLLMExercises/Metal/P007Activation.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void activation_elementwise(
5     device const float *input [[buffer(0)]],
6     device float *output [[buffer(1)]],
7     constant uint &count [[buffer(2)]],
8     constant uint &activation [[buffer(3)]],
9     uint index [[thread_position_in_grid]]
10 ) {
11     if (index < count) {
12         // TODO: Implement ReLU, tanh-approximate GELU, and SiLU from activation.
13         output[index] = input[index] + float(activation) * 0.0f;
14     }
15 }
```

8.18 Worked solution

Solution strategy

Maps three elementwise activation functions to individual GPU threads, one thread per element, leaving all tensor shape and contiguous row-major layout unchanged. ReLU computes $\max(0, x)$; GELU applies the tanh approximation with cubic term and $\sqrt{2/\pi}$ scale factor; SiLU multiplies x by its sigmoid gate. The 'Activation' enum makes the formula choice visible at call sites, preventing GELU variants from hiding inside an unlabeled closure.

Correctness invariant

Output shape and rank match input exactly. Each element is computed independently by its corresponding formula. Negative values are handled distinctly per activation: ReLU hard-zeros negatives; GELU and SiLU suppress rather than zero them.

Common failure mode

Implementing exact GELU erf-based formula instead of the tanh approximation (test explicitly names `geluTanhApproximation`). Omitting the 0.044715 cubic coefficient or the $\sqrt{2/\pi}$ factor. Returning `sigmoid(x)` instead of `x*sigmoid(x)` for SiLU. Producing correct flat values but with wrong output shape.

8.18.1 Implementation points

Storage map, not nested shape loops

The CPU reference maps over storage directly, then wraps the result with `input.shape`. This avoids flattening the logical contract to `[N]` even though physical layout is flat, preserving shape invariants.

GELU tanh approximation: $x/2 * (1 + \tanh(\sqrt{2/\pi}) * (x + 0.044715x^3))$

The cubic term must be present. The constant is $\sqrt{2/\pi} \approx 0.7978846$. The half-scale factor multiplies the entire $(1 + \tanh(...))$ result. This approximation matches common LLM implementations and differs from exact GELU.

SiLU retains suppressed negatives

$\text{SiLU}(x) = x / (1 + \exp(-x))$ does not zero negative x . The sigmoid gate lies between 0 and 1, suppressing magnitude but preserving sign. This is architecturally different from ReLU's hard cutoff.

Metal dispatch: one thread per element with no barriers

Metal kernel has one thread per element, `thread_position_in_grid = index`. No threadgroup memory, no barriers, no dependencies. Bounds check for partial final threadgroup only.

Double oracle for Float32 validation

Judge computes expected values in Double using the same formulas, then rounds once to Float. Tolerance is absolute 2e-6 plus relative 2e-5, catching precision issues without over-fitting to implementation order.

8.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P007ActivationMetal.swift

- Line 4: extension P007ActivationSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalActivationPipeline {

Sources/LeetLLMSolutions/P007ActivationSolution.swift

- Line 4: public enum P007ActivationSolution {
- Line 5: public static func apply(

Sources/LeetLLMSolutions/Metal/P007Activation.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

8.18.3 Complete canonical listings

8.18.4 Sources/LeetLLMSolutions/P007ActivationMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P007ActivationSolution {
5     public static func makeMetalPipeline() throws -> MetalActivationPipeline {
6         guard let url = Bundle.module.url(forResource: "P007Activation", withExtension: "metal", subdirectory: "Metal")
7             <- else {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P007Activation.metal")
9         }
10        return try MetalActivationPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }
```

8.18.5 Sources/LeetLLMSolutions/P007ActivationSolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P007ActivationSolution {
5     public static func apply(
6         _ input: FloatTensor,
7         activation: Activation
8     ) throws -> FloatTensor {
9         let output = input.storage.map { value in
10             switch activation {
11             case .relu:
12                 return max(0, value)
13             case .geluTanhApproximation:
14                 let cubic = value * value * value
15                 let inner = sqrt(2 / Float.pi) * (value + 0.044715 * cubic)
16                 return 0.5 * value * (1 + tanh(inner))
17             case .silu:
18                 return value / (1 + exp(-value))
19             }
20         }
21     }
22 }
```

```

21     return try FloatTensor(output, shape: input.shape)
22   }
23 }

```

8.18.6 Sources/LeetLLMSolutions/Metal/P007Activation.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void activation_elementwise(
5     device const float *input [[buffer(0)]],
6     device float *output [[buffer(1)]],
7     constant uint &count [[buffer(2)]],
8     constant uint &activation [[buffer(3)]],
9     uint index [[thread_position_in_grid]]
10 ) {
11     if (index >= count) {
12         return;
13     }
14     float value = input[index];
15     if (activation == 0) {
16         output[index] = max(0.0f, value);
17     } else if (activation == 1) {
18         float cubic = value * value * value;
19         output[index] = 0.5f * value * (1.0f + tanh(0.7978845608f * (value + 0.044715f * cubic)));
20     } else {
21         output[index] = value / (1.0f + exp(-value));
22     }
23 }

```

SwiGLU Feed-Forward Gate

9.1 Why this exists

The feed-forward sublayer in many decoder models is not simply `down(activation(up(x)))`. SwiGLU computes two independent expansions of the residual vector: one decides how strongly each hidden feature is gated, while the other supplies the values being gated. A third projection returns the expanded hidden representation to the residual width.

This lesson implements the complete readable CPU flow and a useful Metal fusion for the elementwise middle stage. The Metal path is not a CPU fallback: actual MSL computes `SiLU(gate) * up` without materializing SiLU separately.

9.2 Learning outcomes

After completing the problem, you can:

- write the gate, up, elementwise product, and down equations with dimensions;
- explain why SwiGLU has three projection matrices;
- validate each inner dimension before indexing flat storage;
- implement the full operator in readable Float32 CPU code;
- fuse SiLU with its elementwise multiplication in Metal;
- identify which intermediates remain and which one the fused stage removes.

9.3 Prerequisites

- Problem 004 for matrix-vector multiplication and row reductions.
- Problem 007 for the SiLU formula and elementwise Metal mapping.
- Problem 002 for contiguous row-major matrix offsets.

9.4 Vocabulary

- **Residual width D** : the feature count entering and usually leaving a decoder sublayer.
- **Hidden width H** : the expanded MLP feature count.
- **Gate projection**: $W_g x$, whose SiLU output modulates hidden features.
- **Up projection**: $W_u x$, which supplies the values being modulated.
- **Down projection**: $W_d h$, which maps hidden features to output width O .
- **Hadamard product**: elementwise multiplication, written $\odot$.
- **Fusion**: computing adjacent operations in one dispatch without writing an intermediate.

9.5 Math from first principles

Let

- $x \in \mathbb{R}^D$,
- $W_g \in \mathbb{R}^{H \times D}$,
- $W_u \in \mathbb{R}^{H \times D}$,
- $W_d \in \mathbb{R}^{O \times H}$.

The operator is

$$g = W_g x, \quad u = W_u x,$$

$$h = \text{SiLU}(g) \odot u,$$

$$y = W_d h.$$

The gate and up matrices cannot generally be merged: their rows learn different functions, and a nonlinear SiLU lies between the gate projection and multiplication. The down matrix is also necessary because h lives at hidden width H , while the residual stream expects width O (commonly $O = D$).

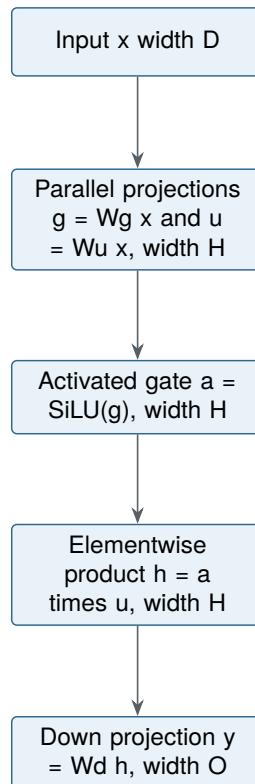


Figure 9.1: Swiglu gate up down flow.

9.5.1 Worked numerical example

Choose $D = 2, H = 2, O = 2$:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad W_g = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad W_u = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Then

$$g = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad u = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$

Using $\text{SiLU}(1) = 0.731059$ and $\text{SiLU}(-2) = -0.238406$:

$$h \approx \begin{bmatrix} 1.462118 \\ -0.715218 \end{bmatrix}.$$

For

$$W_d = \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix},$$

the output is approximately

$$y = \begin{bmatrix} 0.746900 \\ 3.639454 \end{bmatrix}.$$

Notice that negative gate values are suppressed but not necessarily zeroed.

9.6 Shape, layout, and dtype contract

`P008SwiGLUExercise.apply` accepts contiguous `Float32` tensors:

Tensor	Shape	Storage offset
input	[D]	column
gateWeights	[H, D]	row * D + column
upWeights	[H, D]	row * D + column
downWeights	[0, H]	row * H + column
result	[0]	row

All accumulation and intermediates in the canonical path are `Float32`. Gate and up shapes must match; their second dimension must equal `D`; the down matrix's second dimension must equal `H`. A zero hidden width is valid and produces an all-zero `[0]` output. Rank and inner-dimension errors are explicit.

The reusable Metal gate API accepts gate and up tensors of any identical shape and returns that shape.

9.7 CPU reference path

Implement in three visible stages:

1. For each hidden row, compute one gate dot product and one up dot product.
2. Store $(\text{gate} / (1 + \exp(-\text{gate}))) * \text{up}$ in a hidden array.

3. For each output row, dot the down weights with the hidden array.

Keeping these stages visible makes shape mistakes diagnosable. Do not call the exercise implementation from the canonical solution, and do not call a CPU helper from the Metal path.

9.8 Correctness method

The CPU judge includes a hand-sized case with $D = 2$, $H = 3$, and $O = 2$, a zero input, and zero hidden width. It independently accumulates in Double and rounds the final values to Float, with relative tolerance $4e-5$. It also rejects input rank, mismatched gate/up shapes, and an invalid down hidden width.

The Metal gate judge uses negative, zero, and large gate values and checks shape mismatch. This catches returning up, returning sigmoid without the factor gate, and running SiLU without multiplication.

Run:

```
1 swift run leetllm check 008 --cpu
2 swift run leetllm check 008 --metal
3 swift run leetllm check 008 --solution
```

9.9 Performance model

Ignoring bias, the three matrix-vector products cost approximately

$$2HD + 2HD + 2OH$$

floating-point operations. The middle gate adds H SiLU evaluations and H multiplications. Weights dominate bytes for one-token decode: gate and up each read $4HD$ bytes, and down reads $4OH$ bytes.

An unfused middle stage writes $\text{SiLU}(g)$ ($4H$ bytes), then reads it and u before writing h . The fused kernel reads g and u and writes h , removing the SiLU intermediate and one dispatch. It does not fuse any projection; those weight reads and reductions remain.

For prefill with many tokens, GEMM can reuse weights across token rows. For single-token decode, the three weight matrices are streamed for every token, so bandwidth usually matters more than the elementwise gate arithmetic.

9.10 Metal mapping

The implemented Metal stage dispatches one thread per hidden element. Thread i reads $\text{gate}[i]$ and $\text{up}[i]$, computes SiLU and the product, and writes $\text{hidden}[i]$. There is no shared memory or barrier because indices are independent.

This is deliberately a partial fusion with a clear boundary:

```
1 gate GEMV ----\
2             fused SiLU * up ----> hidden ----> down GEMV
3 up GEMV -----/
```

The starter and solution bind the same three device buffers and one element count. The canonical MSL is [P008SwiGLU.metal](#).

9.11 Implementation checkpoints

1. Validate all four ranks and dimensions.
2. Compute gate projection values and compare them by hand.
3. Compute up values separately; do not accidentally share accumulators.
4. Apply the Problem 007 SiLU convention and multiply elementwise.
5. Implement the down projection and preserve [0] shape.
6. Implement the fused Metal gate using one thread per hidden element.
7. Verify CPU full-flow and Metal middle-stage judges independently.

9.12 Controlled experiments

9.12.1 Experiment A: hidden-width sweep

Keep $D = O$ fixed and sweep H through D , $2D$, and $4D$. Prediction: total time and weight traffic grow roughly linearly with H ; decode remains dominated by projection matrix reads rather than the gate kernel.

9.12.2 Experiment B: separate versus fused gate

Compare two Metal dispatches (`SiLU(g)` then multiply by `u`) against the fused kernel. Prediction: fusion wins once H is large enough because it removes a launch and an H -element intermediate round trip. At tiny H , timing noise and command overhead can hide the traffic difference.

9.12.3 Experiment C: output width

Hold D and H fixed, vary O . Prediction: only the down projection scales with O ; gate and up timings should stay constant. If they change, the benchmark is measuring allocation or unrelated end-to-end effects.

Record dimensions, release configuration, warm-up, synchronization point, and whether timing includes buffer creation.

9.13 Engine integration

A decoder MLP will consume an RMS-normalized residual vector, run this SwiGLU flow, and add the result back to the residual stream. Problems 010 and 011 provide those surrounding operations. Problem 012 investigates fusing the norm with a projection input, which can be applied to gate and up projections later.

9.14 Tradeoffs

- Materializing `g` and `u` keeps projection and activation kernels reusable; fusing projections with gating can remove more traffic but creates a larger specialized kernel.
- Larger H increases model capacity and weight traffic together.
- Float32 intermediates are easy to validate; lower precision can save traffic but changes the residual error budget explored in Problem 011.
- Computing gate and up in one matrix kernel may improve weight scheduling, but their logical matrices and outputs remain distinct.

9.15 Hints

- Name D , H , and 0 before writing any index expression.
- Reinitialize both projection accumulators for every hidden row.
- Reuse the SiLU formula, not the Problem 007 solution target.
- The Metal gate accepts tensors of any rank as long as shapes match; use flat contiguous storage because the operation is elementwise.

9.16 Completion checklist

- ☐ The full CPU gate/up/down flow passes all value and error cases.
- ☐ You can annotate every matrix with `[rows, columns]`.
- ☐ The Metal stage executes MSL for `SiLU(gate) * up` and passes its judge.
- ☐ You can explain why all three learned projections are needed.
- ☐ You identified which intermediate the fused stage removes.
- ☐ You ran one controlled hidden-width or fusion experiment in release mode.
- ☐ You interpreted the result using weight bytes, intermediate bytes, and dispatch cost.

9.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

9.17.1 Sources/LeetLLMExercises/P008SwiGLUExercise.swift

```
1 import LeetLLMCore
2
3 public enum P008SwiGLUExercise {
4     public static func apply(_ input: FloatTensor, gateWeights: FloatTensor, upWeights: FloatTensor, downWeights:
5         FloatTensor) throws -> FloatTensor {
6         ↪ try validate(input, gateWeights: gateWeights, upWeights: upWeights, downWeights: downWeights)
7         // TODO: Compute gate/up projections, SiLU(gate) * up, then the down projection.
8         return try FloatTensor(Array(repeating: 0, count: downWeights.shape[0]), shape: [downWeights.shape[0]])
9     }
10
11     private static func validate(_ input: FloatTensor, gateWeights: FloatTensor, upWeights: FloatTensor, downWeights:
12         FloatTensor) throws {
13         ↪ guard input.rank == 1 else { throw TensorError.rankMismatch(expected: 1, actual: input.rank) }
14         ↪ guard gateWeights.rank == 2 else { throw TensorError.rankMismatch(expected: 2, actual: gateWeights.rank) }
15         ↪ guard upWeights.rank == 2 else { throw TensorError.rankMismatch(expected: 2, actual: upWeights.rank) }
16         ↪ guard downWeights.rank == 2 else { throw TensorError.rankMismatch(expected: 2, actual: downWeights.rank) }
17         ↪ guard gateWeights.shape == upWeights.shape else { throw SwiGLUError.hiddenProjectionShapeMismatch(gate:
18             gateWeights.shape, up: upWeights.shape) }
19         ↪ guard gateWeights.shape[1] == input.shape[0] else { throw SwiGLUError.inputWidthMismatch(expected:
20             gateWeights.shape[1], actual: input.shape[0]) }
21         ↪ guard downWeights.shape[1] == gateWeights.shape[0] else { throw SwiGLUError.downWidthMismatch(expected:
22             gateWeights.shape[0], actual: downWeights.shape[1]) }
23     }
24 }
```

9.17.2 Sources/LeetLLMExercises/P008SwiGLUMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P008SwiGLUExercise {
5     public static func makeMetalGatePipeline() throws -> MetalSwiGLUGatePipeline {
6         ↪ guard let url = Bundle.module.url(forResource: "P008SwiGLU", withExtension: "metal", subdirectory: "Metal") else
7             {
8             ↪ throw MetalNeuralOperatorError.kernelResourceMissing("P008SwiGLU.metal")
9             }
10         return try MetalSwiGLUGatePipeline(source: String(contentsOf: url, encoding: .utf8))
11     }
12 }
```

9.17.3 Sources/LeetLLMExercises/Metal/P008SwiGLU.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void swiglu_gate(
5     device const float *gate [[buffer(0)]],
6     device const float *up [[buffer(1)]],
7     device float *output [[buffer(2)]],
8     constant uint &count [[buffer(3)]],
9     uint index [[thread_position_in_grid]]
10 ) {
```

```
11     if (index < count) {  
12         // TODO: Fuse SiLU(gate) with multiplication by up.  
13         output[index] = up[index] + gate[index] * 0.0f;  
14     }  
15 }
```

9.18 Worked solution

Solution strategy

Decomposes the gated feed-forward into three explicit projection stages: gate projection ($W_g \cdot x$) and up projection ($W_u \cdot x$) both to hidden width H , elementwise SiLU gate activation multiplied by up values (Hadamard product), then down projection ($W_d \cdot h$) back to output width O . CPU path evaluates sequentially in three visible loops. Metal fusion merges SiLU activation and multiplication into one elementwise kernel, removing the materialized SiLU intermediate while preserving composed semantics.

Correctness invariant

Gate and up projections have identical shapes $[H, D]$. Output shape is $[O] = [\text{down.shape}[0]]$. Each gated hidden element depends on both the corresponding gate activation and its paired up value. Zero hidden width produces all-zero output vector of correct shape.

Common failure mode

Omitting the down projection (tests reject failure mode when only returning gated hidden). Implementing as simple MLP (linear $\rightarrow$ nonlinear $\rightarrow$ linear) without the gate-up decomposition. Implementing gate as linear without SiLU activation. Returning $\text{sigmoid}(\text{gate})$ instead of $(\text{gate} / (1 + \exp(-\text{gate}))) * \text{up}$ for the gate operation.

9.18.1 Implementation points

Gate and up learn different features

The two $[H, D]$ matrices are not merged. Gate rows learn to produce modulation signals; up rows learn to supply values being modulated. SiLU sits between them, making the structure non-commutative.

SiLU suppresses but does not zero negatives

Large negative gate values produce small but non-zero outputs (e.g., $\text{SiLU}(-2) \approx -0.238$). This soft gating preserves information for backprop compared to ReLU's hard zero.

Metal gate fusion removes one intermediate

The fused Metal kernel reads gate and up vectors once, computes SiLU and multiplication in registers, and writes hidden without materializing SiLU separately. This saves one buffer write, read, and dispatch.

Validation catches shape mismatches early

Guards check input rank, gate/up shape equality, down hidden width matching, and inner dimension coherence before any indexing. Tests verify rejection of input rank, gate/up mismatch, and down width mismatch.

Relative tolerance 4e-5 across all three projections

Judge accumulates gate and up in Double, applies SiLU formula in Double, multiplies in Double, then applies down in Double. Final values are rounded to Float once, tolerating accumulated rounding across all three stages.

9.18.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P008SwiGLUMetal.swift

- Line 4: extension P008SwiGLUSolution {
- Line 5: public static func makeMetalGatePipeline() throws -> MetalSwiGLUGatePipeline {

Sources/LeetLLMSolutions/P008SwiGLUSolution.swift

- Line 4: public enum P008SwiGLUSolution {
- Line 5: public static func apply(_ input: FloatTensor, gateWeights: FloatTensor, upWeights: FloatTensor, downWeights: FloatTensor) throws -> FloatTensor {
- Line 28: public static func gate(_ gate: FloatTensor, up: FloatTensor) throws -> FloatTensor {
- Line 36: private static func validate(_ input: FloatTensor, gateWeights: FloatTensor, upWeights: FloatTensor, downWeights: FloatTensor) throws -> FloatTensor {

Sources/LeetLLMSolutions/Metal/P008SwiGLU.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

9.18.3 Complete canonical listings**9.18.4 Sources/LeetLLMSolutions/P008SwiGLUMetal.swift**

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P008SwiGLUSolution {
5     public static func makeMetalGatePipeline() throws -> MetalSwiGLUGatePipeline {
6         guard let url = Bundle.module.url(forResource: "P008SwiGLU", withExtension: "metal", subdirectory: "Metal") else
7             ↪ {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P008SwiGLU.metal")
9         }
10        return try MetalSwiGLUGatePipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }

```

9.18.5 Sources/LeetLLMSolutions/P008SwiGLUSolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P008SwiGLUSolution {
5     public static func apply(_ input: FloatTensor, gateWeights: FloatTensor, upWeights: FloatTensor, downWeights:
6         ↪ FloatTensor) throws -> FloatTensor {
7         try validate(input, gateWeights: gateWeights, upWeights: upWeights, downWeights: downWeights)
8         let hidden = gateWeights.shape[0]
9         let width = input.shape[0]
10        var gated = Array(repeating: Float.zero, count: hidden)
11        for row in 0..

```

```

14         gate += gateWeights.storage[row * width + column] * input.storage[column]
15         up += upWeights.storage[row * width + column] * input.storage[column]
16     }
17     gated[row] = (gate / (1 + exp(-gate))) * up
18 }
19 var output = Array(repeating: Float.zero, count: downWeights.shape[0])
20 for row in output.indices {
21     for column in 0..

```

9.18.6 Sources/LeetLLMSolutions/Metal/P008SwiGLU.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void swiglu_gate(
5     device const float *gate [[buffer(0)]],
6     device const float *up [[buffer(1)]],
7     device float *output [[buffer(2)]],
8     constant uint &count [[buffer(3)]],
9     uint index [[thread_position_in_grid]]
10 ) {
11     if (index < count) {
12         float gateValue = gate[index];
13         output[index] = (gateValue / (1.0f + exp(-gateValue))) * up[index];
14     }
15 }

```

Numerically Stable Softmax

10.1 Why this exists

Attention produces scores, not probabilities. Softmax converts each score row into nonnegative values that sum to one, but a literal implementation of $\exp(\text{logit})$ can overflow for large positive logits and underflow every term for large negative logits. Both failures can occur even when the correct probabilities are ordinary numbers.

This lesson makes stability part of the operator contract and maps the two required reductions, maximum and exponential sum, to a real Metal threadgroup.

10.2 Learning outcomes

After completing the problem, you can:

- derive why subtracting one constant from every logit leaves softmax unchanged;
- implement stable row-wise softmax with max subtraction;
- test probability sums as an invariant without using them as the only oracle;
- distinguish a zero-row tensor from a row with zero columns;
- implement max and sum reductions with threadgroup memory and barriers;
- explain and test the Metal lesson's explicit 1024-column limit.

10.3 Prerequisites

- Problem 001 for parallel reductions and synchronization.
- Problem 002 for a contiguous [rows, columns] tensor.
- Problem 006 for reduction traffic and dispatch overhead.
- Exponentials from the [Math Primer](#).

10.4 Vocabulary

- **Logit**: an unnormalized score before conversion to probabilities.
- **Normalization constant**: the denominator that makes a row sum to one.
- **Shift invariance**: softmax is unchanged when every row element receives the same additive constant.
- **Overflow**: a finite mathematical result exceeds the representable floating-point range.
- **Underflow**: a very small magnitude rounds to zero.
- **Row reduction**: combining all columns of one row into one maximum or sum.
- **Probability simplex**: nonnegative vectors whose elements sum to one.

10.5 Math from first principles

For logits $z_0, \dots, z_{C-1}$, softmax is

$$p_i = \frac{e^{z_i}}{\sum_{j=0}^{C-1} e^{z_j}}.$$

For any constant c , multiply numerator and denominator by e^{-c} :

$$\frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_i} e^{-c}}{\sum_j e^{z_j} e^{-c}} = \frac{e^{z_i - c}}{\sum_j e^{z_j - c}}.$$

Choose $c = m = \max_j z_j$. Every shifted logit is at most zero, so every exponential lies in $(0, 1]$, and at least one is exactly one:

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}.$$

The denominator cannot underflow to zero for finite, nonempty input because it contains the maximum term $e^0 = 1$.

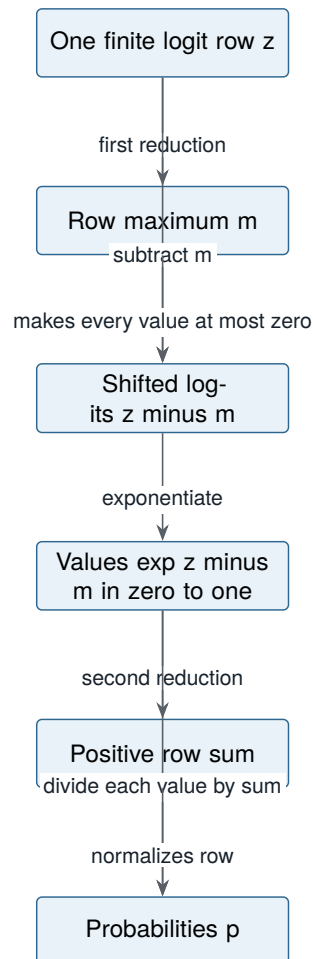


Figure 10.1: Stable row softmax.

10.5.1 Worked numerical example

For logits $[1000, 1001, 999]$, the naive exponentials overflow in Float32. Subtract the maximum 1001 to get $[-1, 0, -2]$:

$$e^{-1} \approx 0.367879, \quad e^0 = 1, \quad e^{-2} \approx 0.135335.$$

Their sum is about 1.503214, producing

$$p \approx [0.244728, 0.665241, 0.090031].$$

The same probabilities result for $[-1000, -999, -1001]$; only relative differences matter.

10.6 Shape, layout, and dtype contract

Input and output are contiguous Float32 tensors with shape $[R, C]$.

- Rank must be exactly two.
- C must be positive because a probability distribution over zero choices is undefined.
- R may be zero; $[0, C]$ returns an empty tensor of the same shape.
- Every logit must be finite. Infinity and NaN are rejected with row and column.
- Output shape and storage order match input.
- Every nonempty output row must be nonnegative and sum to one within $3e-5$.

The CPU API supports any C representable by the tensor. The teaching Metal kernel supports $C \leq \text{MetalSoftmaxPipeline.maximumRowWidth}$, currently 1024. The host rejects wider rows before dispatch.

10.7 CPU reference path

For each row:

1. Scan all columns to find the maximum.
2. Compute and store $\exp(\text{logit} - \text{maximum})$, accumulating their sum.
3. Divide each stored exponential by the sum.

The canonical path uses Float32 so it models the actual operator. The judge's oracle performs the same derivation in Double and rounds each final probability to Float.

Do not test correctness only by checking that the row sums to one. A uniform distribution also sums to one and is wrong for nonuniform logits.

10.8 Correctness method

The shared judge covers ordinary rows, logits near positive 10,000, logits near negative 10,000, single-value rows, a width of 257 that crosses the Metal threadgroup size, and zero rows. It separately checks exact expected values and the probability-sum invariant.

Error cases cover rank one, $[R, 0]$, and non-finite input. The Metal test also constructs width 1025 and verifies the public `rowWidthExceedsMaximum` error.

Run:


```
1 swift run leetllm check 009 --cpu
2 swift run leetllm check 009 --metal
3 swift run leetllm check 009 --solution
```

10.9 Performance model

For R rows and C columns, the CPU path performs two reductions and a normalization pass. It computes RC exponentials. Its explicit array writes the shifted exponentials once and then reads them for division.

The Metal kernel scans input for the maximum, scans it again for the exponential sum, and scans it a third time to recompute exponentials for output. Algorithmic global traffic is therefore roughly three input reads plus one output write, or about $16RC$ bytes, before considering cache. Recomputing the exponential avoids a separate temporary buffer and dispatch.

Each row is one threadgroup. Small c leaves many of 256 threads idle; very few rows expose little group-level parallelism. Large batches provide parallelism across rows, while wide rows provide work within each group.

10.10 Metal mapping

The canonical kernel uses 256 threads per row and a 256-Float scratch array.

1. Each thread scans columns `localIndex`, `localIndex + 256`, and so on.
2. A tree reduction finds the row maximum in threadgroup memory.
3. Threads compute shifted exponential partial sums.
4. A second tree reduction finds the denominator.
5. Threads write normalized probabilities for their strided columns.

Every reduction stage requires a threadgroup barrier before another thread reads a partial value. No thread may return early based on column bounds, because all 256 threads must reach every barrier; out-of-range threads instead contribute `-infinity` to max and zero to sum.

The 1024-column cap is pedagogical, not a claim about Metal hardware. It bounds this first materialized implementation to at most four values per thread and makes unsupported attention rows fail visibly. Problem 019's online softmax is the later artifact for unbounded streaming rows.

See [P009Softmax.metal](#).

10.11 Implementation checkpoints

1. Validate rank, positive column count, and finite values.
2. Implement one stable CPU row and compare the worked example.
3. Extend to multiple and zero rows while preserving shape.
4. Implement only the Metal maximum reduction and inspect it temporarily.
5. Add the shifted exponential sum reduction.
6. Normalize output without allowing any thread to skip a barrier.
7. Test widths 1, 257, 1024, and rejected 1025.

10.12 Controlled experiments

10.12.1 Experiment A: translation invariance

Generate one row, then add constants -10000, 0, and 10000 to every element. Prediction: stable outputs agree within tolerance. A naive implementation will underflow or overflow in the shifted cases.

10.12.2 Experiment B: reduction boundaries

Measure widths 255, 256, 257, 512, and 1024 with fixed row count. Prediction: 257 adds a second loop iteration for one thread but does not double all work; step changes reflect striding and reduction overhead, not a new grid.

10.12.3 Experiment C: row parallelism

Fix $C=128$ and sweep R from 1 to 1024. Prediction: one row underuses the GPU, while more independent row threadgroups improve occupancy until memory or math throughput becomes limiting.

Write predictions before measuring and record whether buffer allocation and CPU/GPU synchronization are included.

10.13 Engine integration

Attention will apply this operator to one score row per query and head after masking. A mask represented as negative infinity would conflict with this lesson's finite-input contract; the later attention operator must either use a finite sentinel before this API or define a mask-aware softmax contract. Problem 019 replaces materialized score rows with online softmax state.

10.14 Tradeoffs

- Storing exponentials saves recomputation but creates an intermediate write and read.
- Recomputing exponentials saves storage traffic at additional math cost.
- One threadgroup per row is simple; SIMD-group reductions can reduce barriers but add hardware-aware structure.
- A fixed width cap gives explicit behavior; streaming or tiled softmax is required for larger contexts rather than silently truncating rows.

10.15 Hints

- Initialize a maximum accumulator to negative infinity, not zero; all logits may be negative.
- Subtract the maximum before every exponential, including the final output pass.
- Keep all threads participating in barriers.
- If sums equal one but values are wrong, compare against the independent oracle.

10.16 Completion checklist

- ☐ Large positive and all-negative logits produce finite correct probabilities.
- ☐ Every nonempty row sums to one within tolerance.
- ☐ Empty rows, invalid rank, empty columns, and non-finite values follow the contract.
- ☐ Metal executes both reductions and handles width 257.
- ☐ Width 1025 is rejected explicitly rather than truncated.
- ☐ You recorded and ran one translation, width, or row-count experiment.

- ☐ You can explain every barrier and neutral reduction value.

10.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

10.17.1 Sources/LeetLLMExercises/P009SoftmaxExercise.swift

```
1 import LeetLLMCore
2
3 public enum P009SoftmaxExercise {
4     public static func apply(_ logits: FloatTensor) throws -> FloatTensor {
5         guard logits.rank == 2 else { throw TensorError.rankMismatch(expected: 2, actual: logits.rank) }
6         guard logits.shape[1] > 0 else { throw SoftmaxError.emptyRow }
7         for row in 0..

```

10.17.2 Sources/LeetLLMExercises/P009SoftmaxMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P009SoftmaxExercise {
5     public static func makeMetalPipeline() throws -> MetalSoftmaxPipeline {
6         guard let url = Bundle.module.url(forResource: "P009Softmax", withExtension: "metal", subdirectory: "Metal") else
7             ↪ {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P009Softmax.metal")
9         }
10        return try MetalSoftmaxPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }
```

10.17.3 Sources/LeetLLMExercises/Metal/P009Softmax.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void softmax_rows(
5     device const float *input [[buffer(0)]],
6     device float *output [[buffer(1)]],
7     constant uint2 &shape [[buffer(2)]],
8     uint localIndex [[thread_index_in_threadgroup]],
9     uint row [[threadgroup_position_in_grid]]
10 ) {
11     // TODO: Reduce the row maximum, then the shifted exponential sum.
12     for (uint column = localIndex; column < shape.y; column += 256) {
13         output[row * shape.y + column] = input[row * shape.y + column] * 0.0f;
14     }
15 }
```

10.18 Worked solution

Solution strategy

Implements row-wise softmax by subtracting the row maximum from every logit (preventing overflow/underflow), then computing normalized exponentials. Metal kernel performs two thread-group reductions: first finds the row maximum via binary tree, second accumulates the exponential sum. Both reductions use threadgroup memory barriers to synchronize partial results. Final normalization divides shifted exponentials by the summed denominator.

Correctness invariant

Each output row is a probability distribution: nonnegative and sums to exactly one (within $3e-5$ tolerance). Shift invariance holds: logits - m and logits_shifted - m_shifted produce identical probabilities for the same shift. At least one logit per row maps to zero after shift (since maximum logit - maximum = 0).

Common failure mode

Naive $\exp(\text{logit})$ overflow for large positive logits or underflow for large negative logits. Omitting max subtraction or applying it per-column instead of per-row. Returning uniform probabilities (tests verify this fails). Allowing threads to skip barriers based on out-of-bounds columns (all 256 threads must synchronize).

10.18.1 Implementation points

Max subtraction is numerically required, not optional

Every shifted logit satisfies $z_i - m \leq 0$, so $\exp(z_i - m) \in (0, 1]$. The denominator sum includes at least the term $\exp(0) = 1$, preventing underflow to zero even for finite, nonempty input.

Two-phase threadgroup reduction with barriers

First phase: parallel threads compute partial maxima, tree reduction finds global max with barrier synchronization at each level. Second phase: threads compute shifted exponentials and partial sums, another tree reduction finds total denominator. Every thread participates in every barrier.

Threads must not skip barriers based on bounds

Threads whose column range is out-of-bounds contribute -infinity to the max reduction and 0 to the sum reduction. They still reach every barrier to keep the threadgroup synchronized. Early return would deadlock.

Probability simplex tested independently of summation

Judge checks both: expected probability values via Double oracle, AND that row sums equal 1. Uniform probabilities would pass the sum check but fail the value check, exposing missing nonlinearity.

1024-column Metal limit is pedagogical

Threadgroup width 256 divides into 4 values per thread at 1024 columns. This bound is explicit in the API (rowWidthExceedsMaximum error) and reflects this first materialized implementation. Problem 019 addresses unbounded online softmax.

10.18.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P009SoftmaxMetal.swift

- Line 4: extension P009SoftmaxSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalSoftmaxPipeline {

Sources/LeetLLMSolutions/P009SoftmaxSolution.swift

- Line 4: public enum P009SoftmaxSolution {
- Line 5: public static func apply(_ logits: FloatTensor) throws -> FloatTensor {
- Line 24: private static func validate(_ logits: FloatTensor) throws {

Sources/LeetLLMSolutions/Metal/P009Softmax.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

10.18.3 Complete canonical listings**10.18.4 Sources/LeetLLMSolutions/P009SoftmaxMetal.swift**

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P009SoftmaxSolution {
5     public static func makeMetalPipeline() throws -> MetalSoftmaxPipeline {
6         guard let url = Bundle.module.url(forResource: "P009Softmax", withExtension: "metal", subdirectory: "Metal") else
7             ↪ {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P009Softmax.metal")
9         }
10        return try MetalSoftmaxPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }

```

10.18.5 Sources/LeetLLMSolutions/P009SoftmaxSolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P009SoftmaxSolution {
5     public static func apply(_ logits: FloatTensor) throws -> FloatTensor {
6         try validate(logits)
7         let rows = logits.shape[0]
8         let columns = logits.shape[1]
9         var output = Array(repeating: Float.zero, count: logits.elementCount)
10        for row in 0..

```

```

19         for column in 0..

```

10.18.6 Sources/LeetLLMSolutions/Metal/P009Softmax.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint threadgroupWidth = 256;
5
6 kernel void softmax_rows(
7     device const float *input [[buffer(0)]],
8     device float *output [[buffer(1)]],
9     constant uint2 &shape [[buffer(2)]],
10    uint localIndex [[thread_index_in_threadgroup]],
11    uint row [[threadgroup_position_in_grid]]
12 ) {
13     threadgroup float scratch[threadgroupWidth];
14     float localMaximum = -INFINITY;
15     for (uint column = localIndex; column < shape.y; column += threadgroupWidth) {
16         localMaximum = max(localMaximum, input[row * shape.y + column]);
17     }
18     scratch[localIndex] = localMaximum;
19     threadgroup_barrier(mem_flags::mem_threadgroup);
20     for (uint stride = threadgroupWidth / 2; stride > 0; stride >>= 1) {
21         if (localIndex < stride) {
22             scratch[localIndex] = max(scratch[localIndex], scratch[localIndex + stride]);
23         }
24         threadgroup_barrier(mem_flags::mem_threadgroup);
25     }
26     float maximum = scratch[0];
27     float localSum = 0.0f;
28     for (uint column = localIndex; column < shape.y; column += threadgroupWidth) {
29         localSum += exp(input[row * shape.y + column] - maximum);
30     }
31     scratch[localIndex] = localSum;
32     threadgroup_barrier(mem_flags::mem_threadgroup);
33     for (uint stride = threadgroupWidth / 2; stride > 0; stride >>= 1) {
34         if (localIndex < stride) {
35             scratch[localIndex] += scratch[localIndex + stride];
36         }
37         threadgroup_barrier(mem_flags::mem_threadgroup);
38     }
39     float denominator = scratch[0];
40     for (uint column = localIndex; column < shape.y; column += threadgroupWidth) {
41         output[row * shape.y + column] = exp(input[row * shape.y + column] - maximum) / denominator;
42     }
43 }

```

RMSNorm

11.1 Why this exists

Transformer sublayers repeatedly consume a residual stream whose magnitude can change across layers. RMSNorm scales each row by its root mean square before a learned per-feature scale. It does not subtract the mean, so it is cheaper than LayerNorm and preserves a different invariant.

Normalization conventions are part of model compatibility. This course uses exactly $x / \text{rms} * \text{gamma}$. gamma is the stored multiplicative scale. There is no implicit $1 + \text{gamma}$ transformation.

11.2 Learning outcomes

After completing the problem, you can:

- derive root mean square from a feature row;
- distinguish RMSNorm from LayerNorm and from residual addition;
- state the exact placement of epsilon and gamma;
- use a higher-precision independent oracle to judge Float32 code;
- parallelize sum-of-squares reduction and scaling in one Metal dispatch;
- identify overflow, cancellation, and epsilon-sensitive fixtures.

11.3 Prerequisites

- Problem 001 for reductions.
- Problem 002 for row-major `[rows, width]` tensors.
- Problem 006 for reduction performance.
- Problem 009 for a two-phase row kernel with barriers.

11.4 Vocabulary

- **Root mean square (RMS)**: square root of the average squared values.
- **Gamma**: a learned multiplicative scale with one value per feature.
- **Epsilon**: a positive stabilizer added inside the square root.
- **Centering**: subtracting a row mean. RMSNorm does not center.
- **Oracle**: an independent implementation used to compute expected values.
- **Reduction precision**: the dtype used while accumulating sum of squares.

11.5 Math from first principles

For a row $x \in \mathbb{R}^D$ and scale $\gamma \in \mathbb{R}^D$, define

$$\text{rms}(x) = \sqrt{\frac{1}{D} \sum_{j=0}^{D-1} x_j^2 + \epsilon}.$$

The output is

$$y_i = \frac{x_i}{\text{rms}(x)} \gamma_i.$$

Epsilon is added to the mean square, before square root. The operator does not compute a mean, subtract a mean, add a bias, or replace gamma with $1 + \gamma$.

Ignoring epsilon, dividing by RMS gives the normalized row mean square one:

$$\frac{1}{D} \sum_i \left(\frac{x_i}{\text{rms}(x)} \right)^2 = 1.$$

Gamma then deliberately changes each feature's scale.

11.5.1 Worked numerical example

Let $x = [3, 4]$, $\gamma = [2, 0.5]$, and $\epsilon = 10^{-6}$.

$$\frac{3^2 + 4^2}{2} = 12.5, \quad \text{rms}(x) = \sqrt{12.500001} \approx 3.535534.$$

Before gamma, the normalized row is approximately $[0.848528, 1.131371]$. After direct multiplication by gamma:

$$y \approx [1.697056, 0.565685].$$

Using $1 + \text{gamma}$ would instead produce about $[2.545584, 1.697057]$; that is a different parameter convention and must fail this lesson's judge.

11.6 Shape, layout, and dtype contract

RMSNormImplementation accepts:

- input: contiguous Float32 [R, D];
- gamma: contiguous Float32 [D];
- epsilon: finite and strictly positive;
- result: contiguous Float32 [R, D].

D must be positive. R may be zero and returns empty storage with shape $[0, D]$. Gamma rank and width must match exactly. Non-finite input is rejected with its row and column.

The canonical operator accumulates Float32 to expose the production numerical path. The judge accumulates squares, divides, square-roots, and scales in Double, then rounds final values to Float. Tolerance is $4e-5$ absolute plus $5e-5$ relative.

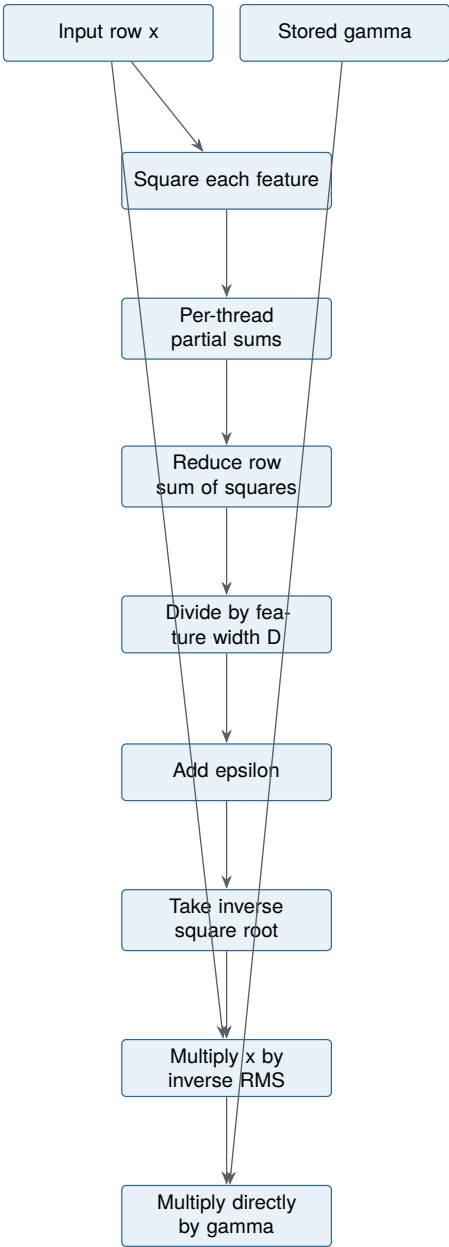


Figure 11.1: Rmsnorm reduction.

11.7 CPU reference path

For each row:

1. Initialize `sumSquares` to Float32 zero.
2. Add `value * value` for every feature.
3. Compute `inverseRMS = 1 / sqrt(sumSquares / Float(D) + epsilon)`.
4. Write `input * inverseRMS * gamma` for every feature.

The second input pass is intentional: the scale is unknown until the whole row has been reduced. A separate normalized array is unnecessary for RMSNorm alone.

11.8 Correctness method

The judge covers mixed signs and nonuniform gamma, a constant row, a zero row whose denominator is controlled by epsilon, large finite values, and zero logical rows. The Double oracle avoids sharing the canonical Float32 reduction.

Errors cover input rank, gamma width, nonpositive epsilon, and zero feature width. The mixed gamma fixture specifically catches an accidental `1 + gamma` convention.

Run:

```
1 swift run leetllm check 010 --cpu
2 swift run leetllm check 010 --metal
3 swift run leetllm check 010 --solution
```

11.9 Performance model

For R rows of width D , RMSNorm performs roughly RD squares and additions, R reciprocal square roots, and $2RD$ final multiplies. It reads input once for reduction and again for output, reads RD gamma values logically, and writes RD outputs.

Ignoring cache, that is approximately three Float32 reads and one write per element, or $16RD$ bytes. Gamma is shared across rows and may remain cached. The arithmetic intensity is low enough that large rows are commonly sensitive to memory traffic, while short rows are sensitive to reduction barriers and launch cost.

Large Float32 values can overflow when squared even if the final normalized result is modest. This lesson tests large but square-representable values; production kernels may use scaled sum-of-squares or wider accumulation when the model's range requires it.

11.10 Metal mapping

One 256-thread threadgroup owns one row. Each thread accumulates squares for columns separated by 256. A tree reduction in threadgroup memory produces the row sum. Every thread then reads the shared result, computes the same `inverseRMS`, and writes its strided output columns.

There is no fixed row-width cap: each thread loops until `column >= D`. All threads must reach every reduction barrier, including threads with no columns. Those threads contribute zero.

The kernel combines reduction and scale in one dispatch, but it still reads input twice. Problem 012 fuses the scaled values directly into a projection so the normalized output is never written to global memory.

See [P010RMSNorm.metal](#).

11.11 Implementation checkpoints

1. Validate rank, width, gamma, epsilon, and finite input.
2. Compute one row's Float32 sum of squares.
3. Check epsilon placement with an all-zero row.
4. Apply gamma directly and compare the worked example.
5. Port the sum-of-squares tree reduction to Metal.
6. Add the post-reduction scale pass in the same dispatch.
7. Test constant, large, zero-row, and width-over-256 cases.

11.12 Controlled experiments

11.12.1 Experiment A: epsilon sensitivity

Use rows whose magnitudes are 0, $1e-4$, 1, and $1e4$, comparing epsilon $1e-6$ and $1e-3$. Prediction: epsilon materially changes only rows whose mean square is near or below epsilon.

11.12.2 Experiment B: accumulation precision

Build a separate Double-accumulating CPU variant and compare it with Float32 for alternating large and small magnitudes. Prediction: output differences grow when small squared terms are added to a much larger running sum, even though there is no sign cancellation after squaring.

11.12.3 Experiment C: row width and row count

Measure widths 64, 256, 257, and 4096 for one row and many rows. Prediction: one short row underuses the GPU; crossing 256 adds loop iterations, while many rows expose more independent threadgroups.

Record tolerance as well as time. A faster path that violates the model's normalization convention is not an optimization.

11.13 Engine integration

Pre-norm decoder blocks apply RMSNorm to the residual stream before attention and before the SwiGLU MLP. Gamma values come directly from model weights and must follow the loader's stored convention. Problem 011 studies the dtype of the residual input; Problem 012 consumes RMS-normalized values inside a GEMV.

11.14 Tradeoffs

- Float32 accumulation is simple and fast; Double or compensated accumulation can improve an oracle but may not map efficiently to the GPU.
- Materializing normalized output makes it reusable by several consumers; fusion saves traffic when one projection consumes it once.
- One threadgroup per row is readable; SIMD-group reductions can use fewer barriers at the cost of more specialized code.
- A larger epsilon improves denominator robustness but changes model outputs.

11.15 Hints

- Divide by D , not by $D - 1$; this is a population mean square.
- Add epsilon before square root.
- Gamma is direct multiplication, not an offset from one.
- In Metal, give inactive threads a zero partial rather than returning before barriers.

11.16 Completion checklist

- ☐ CPU output matches the Double oracle for constant, zero, mixed, and large rows.
- ☐ Gamma is applied as stored, with no implicit +1.
- ☐ Epsilon is finite, positive, and inside the square root.
- ☐ Metal performs a real sum-of-squares reduction and scale pass.
- ☐ Zero rows are accepted and zero feature width is rejected.
- ☐ You ran one epsilon, precision, or shape experiment with a written prediction.
- ☐ You can explain which input round trip Problem 012 removes.

11.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

11.17.1 Sources/LeetLLMExercises/P010RMSNormExercise.swift

```
1 import LeetLLMCore
2
3 public enum P010RMSNormExercise {
4     public static func apply(_ input: FloatTensor, gamma: FloatTensor, epsilon: Float) throws -> FloatTensor {
5         try validateRMSNorm(input, gamma: gamma, epsilon: epsilon)
6         // TODO: Compute  $x / \sqrt{\text{mean}(x^2) + \text{epsilon}}$  * gamma for every row.
7         return try FloatTensor(Array(repeating: 0, count: input.elementCount), shape: input.shape)
8     }
9
10    private static func validateRMSNorm(_ input: FloatTensor, gamma: FloatTensor, epsilon: Float) throws {
11        guard input.rank == 2 else { throw TensorError.rankMismatch(expected: 2, actual: input.rank) }
12        guard gamma.rank == 1 else { throw TensorError.rankMismatch(expected: 1, actual: gamma.rank) }
13        guard input.shape[1] > 0 else { throw RMSNormError.emptyFeatureWidth }
14        guard gamma.shape[0] == input.shape[1] else { throw RMSNormError.gammaWidthMismatch(expected: input.shape[1],
15            ↪ actual: gamma.shape[0]) }
16        guard epsilon.isFinite, epsilon > 0 else { throw RMSNormError.invalidEpsilon(epsilon) }
17        for row in 0..

```

11.17.2 Sources/LeetLLMExercises/P010RMSNormMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P010RMSNormExercise {
5     public static func makeMetalPipeline() throws -> MetalRMSNormPipeline {
6         guard let url = Bundle.module.url(forResource: "P010RMSNorm", withExtension: "metal", subdirectory: "Metal") else
7             ↪ {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P010RMSNorm.metal")
9         }
10        return try MetalRMSNormPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
```

11.17.3 Sources/LeetLLMExercises/Metal/P010RMSNorm.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void rmsnorm_rows(
5     device const float *input [[buffer(0)]],
6     device const float *gamma [[buffer(1)]],
7     device float *output [[buffer(2)]],
8     constant uint2 &shape [[buffer(3)]],
9     constant float &epsilon [[buffer(4)]],
10    uint localIndex [[thread_index_in_threadgroup]],
11    uint row [[threadgroup_position_in_grid]]
```

```
12 ) {  
13     // TODO: Reduce mean square, add epsilon, then scale by gamma.  
14     for (uint column = localIndex; column < shape.y; column += 256) {  
15         output[row * shape.y + column] = input[row * shape.y + column] * gamma[column] * epsilon * 0.0f;  
16     }  
17 }
```

11.18 Worked solution

Solution strategy

Reduces sum of squares across each row in parallel via threadgroup tree reduction, computes inverse RMS as $\text{rsqrt}(\text{mean_square} + \text{epsilon})$, then scales input by inverse RMS and multiplies by per-feature learned scale gamma. Metal kernel assigns one 256-thread group per row; threads accumulate squares for strided columns, reduce the sum, compute inverse RMS once, and write scaled output for their columns.

Correctness invariant

Output shape matches input $[R, D]$. Each row's normalized mean square (before gamma) is exactly one. Gamma is applied with direct multiplication, not $(1 + \text{gamma})$. Epsilon sits inside the square root, before the reciprocal. All rows with valid inputs produce finite outputs.

Common failure mode

Using $(1 + \text{gamma})$ instead of direct gamma multiplication (different model convention, test explicitly rejects this via mixed-gamma fixture). Placing epsilon before or after square root. Dividing by sqrt instead of using rsqrt (inverse square root). Using LayerNorm's centering logic. Computing $\text{gamma} * (1 + \text{gamma})$ or other compound scales.

11.18.1 Implementation points

RMSNorm omits centering, lowering cost vs LayerNorm

No mean computation or subtraction. Only sum-of-squares reduction + square root + scale. This preserves non-zero mean across the residual stream, a different invariant than LayerNorm.

Epsilon must be added inside sqrt to prevent zero-division edge case

For all-zero input, $\text{mean_square} = 0$. $\text{rsqrt}(0 + \text{epsilon}) = \text{rsqrt}(\text{epsilon})$, which is finite. Tests include all-zero row with $\text{epsilon} = 0.25$ to verify epsilon placement.

Gamma is direct per-feature multiplication

$\text{output}[i] = (\text{input}[i] / \text{rms}(\text{input})) * \text{gamma}[i]$. No bias term, no $(1 + \text{gamma})$ transform. This convention is part of model weight format compatibility and must be visible at the call site.

Threadgroup reduction with tree structure and barriers

One 256-thread group per row. Each thread computes partial sum of squares for its strided columns. Tree reduction halves active range at each barrier level. All threads read the final shared result and compute the same inverseRMS.

Double oracle avoids sharing Float32 reduction precision

Expected values accumulated in Double, then rounded to Float once. Tolerance: absolute 4e-5 plus relative 5e-5. Catches both overflow in squared values (tested with 1e18 magnitudes) and precision loss from Float32 accumulation.

11.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P010RMSNormMetal.swift

- Line 4: extension P010RMSNormSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalRMSNormPipeline {

Sources/LeetLLMSolutions/P010RMSNormSolution.swift

- Line 4: public enum P010RMSNormSolution {
- Line 5: public static func apply(_ input: FloatTensor, gamma: FloatTensor, epsilon: Float) throws -> FloatTensor {
- Line 24: private static func validate(_ input: FloatTensor, gamma: FloatTensor, epsilon: Float) throws {

Sources/LeetLLMSolutions/Metal/P010RMSNorm.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

11.18.3 Complete canonical listings

11.18.4 Sources/LeetLLMSolutions/P010RMSNormMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P010RMSNormSolution {
5     public static func makeMetalPipeline() throws -> MetalRMSNormPipeline {
6         guard let url = Bundle.module.url(forResource: "P010RMSNorm", withExtension: "metal", subdirectory: "Metal") else
7             ↪ {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P010RMSNorm.metal")
9         }
10        return try MetalRMSNormPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }
```

11.18.5 Sources/LeetLLMSolutions/P010RMSNormSolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P010RMSNormSolution {
5     public static func apply(_ input: FloatTensor, gamma: FloatTensor, epsilon: Float) throws -> FloatTensor {
6         try validate(input, gamma: gamma, epsilon: epsilon)
7         let rows = input.shape[0]
8         let width = input.shape[1]
9         var output = Array(repeating: Float.zero, count: input.elementCount)
10        for row in 0..

```

```

17     for column in 0..

```

11.18.6 Sources/LeetLLMSolutions/Metal/P010RMSNorm.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint threadgroupWidth = 256;
5
6 kernel void rmsnorm_rows(
7     device const float *input [[buffer(0)]],
8     device const float *gamma [[buffer(1)]],
9     device float *output [[buffer(2)]],
10    constant uint2 &shape [[buffer(3)]],
11    constant float &epsilon [[buffer(4)]],
12    uint localIndex [[thread_index_in_threadgroup]],
13    uint row [[threadgroup_position_in_grid]]
14 ) {
15     threadgroup float scratch[threadgroupWidth];
16     float localSumSquares = 0.0f;
17     for (uint column = localIndex; column < shape.y; column += threadgroupWidth) {
18         float value = input[row * shape.y + column];
19         localSumSquares += value * value;
20     }
21     scratch[localIndex] = localSumSquares;
22     threadgroup_barrier(mem_flags::mem_threadgroup);
23     for (uint stride = threadgroupWidth / 2; stride > 0; stride >= 1) {
24         if (localIndex < stride) {
25             scratch[localIndex] += scratch[localIndex + stride];
26         }
27         threadgroup_barrier(mem_flags::mem_threadgroup);
28     }
29     float inverseRMS = rsqrt(scratch[0] / float(shape.y) + epsilon);
30     for (uint column = localIndex; column < shape.y; column += threadgroupWidth) {
31         output[row * shape.y + column] = input[row * shape.y + column] * inverseRMS * gamma[column];
32     }
33 }

```

Residual Streams and Precision

12.1 Why this exists

The residual stream is the long-lived state that carries information through every decoder block. Attention and MLP outputs are added into it repeatedly. A rounding decision at this boundary is therefore not local: an update lost after one layer is absent from every later layer.

This systems lab compares two explicit policies. One keeps each add in Float32. The other simulates storing the residual as Float16 after every add by round-tripping the new sum through Float16. The point is not that one policy is universally correct; it is to make downcast placement measurable and reviewable.

12.2 Learning outcomes

After completing the problem, you can:

- describe the residual stream's role across decoder sublayers;
- distinguish Float32 arithmetic from Float16 storage round trips;
- predict when a small update falls below a Float16 unit in the last place;
- implement an explicit, deterministic downcast policy;
- measure maximum absolute divergence between two policies;
- explain why this lesson is CPU-only rather than disguising CPU work as Metal.

12.3 Prerequisites

- Problem 002 for tensor shape contracts.
- Problem 006 for separating numerical and performance questions.
- Problem 010 for the normalized values that consume residual state.

12.4 Vocabulary

- **Residual stream:** the feature tensor updated by adding each sublayer's output.
- **Residual update:** an attention or MLP result added to the stream.
- **Downcast:** conversion from a wider or more precise dtype to a narrower one.
- **Round trip:** convert Float32 to Float16 and back to Float32 so storage loss is observable.
- **ULP:** unit in the last place, the spacing between adjacent representable values near a magnitude.
- **Error propagation:** later computation operating on an already rounded state.
- **Policy boundary:** the precise point at which conversion is required by the API.

12.5 Math from first principles

For initial residual $r^{(0)}$ and updates $u^{(1)}, \dots, u^{(L)}$, idealized real arithmetic gives

$$r^{(\ell)} = r^{(\ell-1)} + u^{(\ell)}.$$

The Float32 policy models

$$r_{32}^{(\ell)} = \text{round}_{32} \left(r_{32}^{(\ell-1)} + u^{(\ell)} \right).$$

The per-add Float16 storage policy models

$$r_{16}^{(\ell)} = \text{Float32} \left(\text{round}_{16} \left(r_{16}^{(\ell-1)} + u^{(\ell)} \right) \right).$$

The conversion occurs after every addition. Converting only once after all updates is a different policy and usually loses different information.

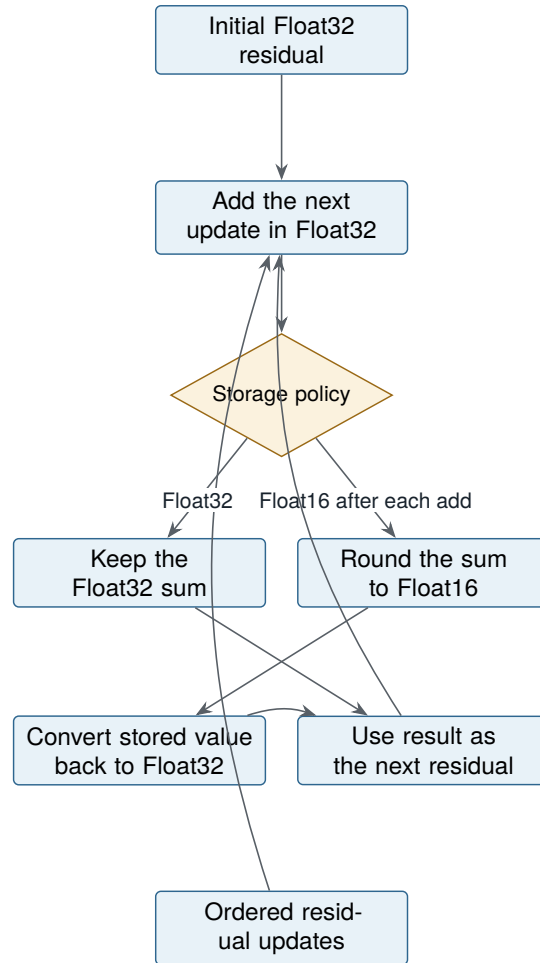


Figure 12.1: Residual downcast boundary.

12.5.1 Worked numerical example

Near 4096, Float16 has an exponent large enough that adjacent representable values are 4 apart. Start with 4096 and add 0.5 four times.

Float32 retains the updates:

$$4096 + 0.5 + 0.5 + 0.5 + 0.5 = 4098.$$

With a Float16 round trip after each addition, each intermediate 4096.5 rounds back to 4096, so the final value remains 4096. The measurable absolute difference is 2.

This is not random noise. It follows deterministically from the chosen storage format and the position of each conversion.

12.6 Shape, layout, and dtype contract

ResidualAccumulationImplementation accepts:

- an initial contiguous Float32 tensor of any shape;
- zero or more update tensors;
- one ResidualPrecisionPolicy.

Every update must have exactly the initial shape. No broadcasting is allowed. The result has the same shape and Float32 Swift storage in both policies; for float16AfterEachAdd, each scalar sum is converted to Float16 and back after every update. With no updates, the initial tensor is returned unchanged.

ResidualPrecisionComparison contains both outputs and their maximum absolute difference. It is a lab report type, not a claim that maximum error alone captures model quality.

12.7 CPU reference path

Copy the initial storage, then process updates in order. For each scalar:

```
1 let sum = output[index] + update.storage[index]
2 output[index] = policy == .float32 ? sum : Float(Float16(sum))
```

Validation happens before mutation so a mismatched update cannot return a partially accumulated tensor. Do not batch all updates into one sum before the Float16 conversion; that moves the policy boundary.

12.8 Correctness method

The judge includes the 4096 fixture under both policies, a matrix-shaped residual, and no updates. Expected values are generated by a separate policy loop in the core target. It also rejects an update whose shape differs despite having the same element count.

The dedicated comparison test asserts:

- Float32 result: 4098;
- per-add Float16 result: 4096;
- maximum absolute difference: 2.

Run:

```
1 swift run leetllm check 011 --cpu
2 swift run leetllm check 011 --solution
```

check 011 --metal is rejected because the problem has no Metal stage.

12.9 Performance model

For L updates and N elements, accumulation performs LN additions. A straight loop reads the current residual and one update and writes the new residual each time. The per-add Float16 policy also performs two conversions per element per update in this simulation.

The simulation still stores Swift arrays as Float32, so it does not claim Float16 memory-bandwidth savings. A real packed Float16 residual would halve residual bytes but require kernels whose loads, arithmetic promotion, and stores exactly match that policy. This problem isolates numerical behavior before making such a performance claim.

Update order can matter in finite precision. Adding a very small update to a large residual may lose it, while combining small updates first can retain part of their sum. Decoder execution order is fixed by the model and must not be rearranged merely to improve a fixture.

12.10 Metal mapping

This lesson is intentionally CPU-only. A one-thread-per-element Metal add would be easy to write, but it would add dispatch and buffer mechanics without teaching the central question: exactly where the residual is downcast across a sequence of layers.

A later integrated decoder can implement either policy on Metal by choosing the residual buffer dtype and store boundary. Its parity test should reuse the deterministic fixtures here. Until that integrated buffer exists, a Metal kernel would be an isolated copy operation rather than evidence about actual residual storage.

12.11 Implementation checkpoints

1. Validate every update shape before adding values.
2. Implement ordered Float32 accumulation.
3. Add an explicit Float16 round trip after each scalar add.
4. Verify no-update behavior for both policies.
5. Reproduce the 4096 worked example.
6. Build a ResidualPrecisionComparison and report maximum difference.
7. Explain why converting only at the end is not equivalent.

12.12 Controlled experiments

12.12.1 Experiment A: update magnitude

Start at 4096 and repeat updates of 0.25, 0.5, 1, 2, and 4. Prediction: per-add Float16 loses the smallest updates; updates at or above the local spacing become visible, subject to round-to-nearest behavior.

12.12.2 Experiment B: layer count

Repeat one small update for 1, 4, 32, and 128 layers. Prediction: Float32 accumulates it until its own precision limit, while per-add Float16 may lose every update, so divergence grows with layer count.

12.12.3 Experiment C: downcast placement

Compare downcast after every add with downcast once after all adds. Prediction: the final-only conversion preserves the accumulated 2 in the worked example, demonstrating that dtype name alone does not specify numerical behavior.

12.12.4 Experiment D: order

Use updates with mixed magnitudes in original and magnitude-sorted order. Prediction: Float16 results can differ. Treat this as a diagnostic, not permission to reorder model operations.

12.13 Engine integration

A pre-norm decoder reads the residual for RMSNorm, computes attention or MLP output, and writes `residual + update`. The eventual engine must choose a buffer dtype and conversion point that match its parity target. Layer-by-layer comparisons should inspect the residual immediately after each add to locate the first policy divergence.

Problem 012 consumes a residual-like vector through RMSNorm and projection but does not change this addition policy.

12.14 Tradeoffs

- Float32 residual storage uses more bytes but retains more small updates.
- Float16 storage can reduce traffic and capacity, but conversion at every layer can erase information before later normalization.
- Maximum absolute error is easy to interpret; relative error, cosine similarity, and downstream logits may expose different consequences.
- A deterministic simulation is portable; an integrated GPU benchmark is necessary before making speed claims.

12.15 Hints

- Validate shape arrays, not only element counts.
- Convert the sum, not each operand independently, unless defining another policy.
- Keep update order unchanged.
- Use values around a known Float16 spacing to make the effect visible.

12.16 Completion checklist

- ☐ Both precision policies pass deterministic judge cases.
- ☐ Shape mismatch is rejected before accumulation.
- ☐ The 4096 fixture produces a measured difference of 2.
- ☐ You can state exactly when the Float16 round trip occurs.
- ☐ You ran one magnitude, layer-count, placement, or order experiment.

- ☐ You did not claim memory savings from the Float32-backed simulation.
- ☐ You can identify the residual store boundary in a future decoder block.

12.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

12.17.1 Sources/LeetLLMExercises/P011ResidualPrecisionExercise.swift

```
1 import LeetLLMCore
2
3 public enum P011ResidualPrecisionExercise {
4     public static func accumulate(_ initial: FloatTensor, updates: [FloatTensor], policy: ResidualPrecisionPolicy)
5         ↪ throws -> FloatTensor {
6         for (index, update) in updates.enumerated() where update.shape != initial.shape {
7             throw ResidualStreamError.updateShapeMismatch(index: index, expected: initial.shape, actual: update.shape)
8         }
9         // TODO: Add every update and apply the selected storage policy at the stated boundary.
10        return initial
11    }
```

12.18 Worked solution

Solution strategy

CPU-only policy comparison: two explicit precision modes for residual stream updates. Float32 policy keeps each sum in Float32. Float16AfterEachAdd policy round-trips each sum through Float16 (converts to Float16 then back to Float32) after every addition, making storage loss observable and deterministic. Updates are applied in order; downcast timing is the only difference between policies.

Correctness invariant

All updates are incorporated into the residual in the declared order. Downcast occurs at the policy boundary (immediately after each addition), not hidden in intermediate operations. Maximum absolute difference between policies is measurable and deterministic given the sequence.

Common failure mode

Applying downcast only once at the end instead of after each addition (wrong timing changes the numerical result fundamentally). Ignoring updates or skipping additions. Using non-deterministic rounding or stochastic conversions. Computing the difference before conversion instead of after.

12.18.1 Implementation points

Policy boundary placement changes error propagation

Float32 policy: errors accumulate at full precision within the residual stream, then interact with later layers' computations. Float16 policy: each update's precision loss is baked into the residual immediately, affecting all downstream layers. Neither is universally 'correct'—the choice is architectural.

Float16 ULP spacing grows with magnitude

Near 4096, adjacent Float16 representable values are 4 apart. Adding 0.5 four times in Float32 gives 4098; with Float16 round-trip after each +0.5, the intermediate 4096.5 rounds back to 4096, so the final value stays 4096. Measurable loss is exactly 2.

Round-trip Float32 → Float16 → Float32 makes loss observable

No per-value rounding logic needed. Storage loss appears automatically because Float16(Float32) truncates to Float16 representation, then converting back to Float32 reproduces the stored precision loss.

ResidualPrecisionComparison measures maximum absolute divergence

Not statistical error or average divergence. The maximum tells you the worst-case precision loss across the entire tensor, making the cost of the policy choice reviewable.

Shape must match across all updates

Each update must have the same shape as the initial residual. Test verifies rejection when update shapes differ. This constraint ensures residual addition is element-wise without broadcasting.

12.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P011ResidualPrecisionSolution.swift

- Line 3: public enum P011ResidualPrecisionSolution {
- Line 4: public static func accumulate(_ initial: FloatTensor, updates: [FloatTensor], policy: ResidualPrecisionPolicy) throws ->
- Line 18: public static func compare(_ initial: FloatTensor, updates: [FloatTensor]) throws -> ResidualPrecisionComparison {

12.18.3 Complete canonical listings

12.18.4 Sources/LeetLLMSolutions/P011ResidualPrecisionSolution.swift

```

1 import LeetLLMCore
2
3 public enum P011ResidualPrecisionSolution {
4     public static func accumulate(_ initial: FloatTensor, updates: [FloatTensor], policy: ResidualPrecisionPolicy)
5         throws -> FloatTensor {
6         for (index, update) in updates.enumerated() where update.shape != initial.shape {
7             throw ResidualStreamError.updateShapeMismatch(index: index, expected: initial.shape, actual: update.shape)
8         }
9         var output = initial.storage
10        for update in updates {
11            for index in output.indices {
12                let sum = output[index] + update.storage[index]
13                output[index] = policy == .float32 ? sum : Float(Float16(sum))
14            }
15        }
16        return try FloatTensor(output, shape: initial.shape)
17    }
18
19    public static func compare(_ initial: FloatTensor, updates: [FloatTensor]) throws -> ResidualPrecisionComparison {
20        ResidualPrecisionComparison(
21            float32: try accumulate(initial, updates: updates, policy: .float32),
22            float16AfterEachAdd: try accumulate(initial, updates: updates, policy: .float16AfterEachAdd)
23        )
24    }
25 }
```

Fuse Norm, Scale, and Projection Input

13.1 Why this exists

A pre-norm decoder commonly computes RMSNorm and immediately feeds the result to one or more linear projections. A separate implementation writes the whole normalized vector to global memory, launches another kernel, then reads that vector for GEMV. Fusion can remove the intermediate and one dispatch.

This lesson keeps a readable separate CPU baseline and implements an actual fused Metal RMSNorm+GEMV kernel. It does not call two CPU functions. The kernel computes the RMS reduction and consumes normalized values directly in each projection row's dot product.

13.2 Learning outcomes

After completing the problem, you can:

- expand a composed RMSNorm and GEMV equation without changing its semantics;
- compare a materialized baseline against a fused implementation;
- account for the normalized intermediate's bytes and dispatch;
- implement two threadgroup reductions in one Metal kernel;
- identify redundant work introduced by the chosen dispatch mapping;
- decide when saved memory traffic justifies a more specialized kernel.

13.3 Prerequisites

- Problem 004 for GEMV shapes and row reductions.
- Problem 006 for arithmetic intensity and end-to-end timing.
- Problem 010 for the exact $x / rms * gamma$ convention.
- Problem 011 for why input dtype and conversion boundaries remain separate decisions.

13.4 Vocabulary

- **Materialization**: writing an intermediate tensor to memory so another operator can read it.
- **Producer**: the operation creating an intermediate, here RMSNorm.
- **Consumer**: the operation reading it, here GEMV.
- **Fused kernel**: one dispatch that computes both while preserving the composed result.
- **Projection row**: one weight row that produces one output scalar.
- **Redundant reduction**: recomputing the same RMS statistic in multiple threadgroups.
- **Dispatch tradeoff**: choosing grid ownership that changes parallelism, synchronization, and repeated work.

13.5 Math from first principles

Let input $x \in \mathbb{R}^D$, scale $\gamma \in \mathbb{R}^D$, and projection weights $W \in \mathbb{R}^{O \times D}$. Define

$$s = \frac{1}{D} \sum_{j=0}^{D-1} x_j^2, \quad n_j = \frac{x_j}{\sqrt{s + \epsilon}} \gamma_j.$$

The separate baseline materializes n and computes

$$y_i = \sum_{j=0}^{D-1} W_{ij} n_j.$$

Substitute the normalized value directly:

$$y_i = \sum_{j=0}^{D-1} W_{ij} \left(\frac{x_j}{\sqrt{\frac{1}{D} \sum_k x_k^2 + \epsilon}} \gamma_j \right).$$

Fusion changes storage and scheduling, not this equation. Gamma is still direct multiplication, and epsilon stays inside the RMS square root.

13.5.1 Worked numerical example

Use $x = [3, 4]$, $\gamma = [2, 0.5]$, $\epsilon = 10^{-6}$, and

$$W = \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix}.$$

From Problem 010, normalized-and-scaled input is approximately

$$n = [1.697056, 0.565685].$$

The baseline gives

$$y_0 = 1(1.697056) + 2(0.565685) \approx 2.828426,$$

$$y_1 = -1(1.697056) + 1(0.565685) \approx -1.131371.$$

The fused kernel must produce these values without allocating storage for n .

13.6 Shape, layout, and dtype contract

FusedRMSNORMGEMVImplementation accepts:

Value	Shape	Dtype
input	[D]	Float32

Value	Shape	Dtype
gamma	[D]	Float32
weights	[0, D] row-major	Float32
output	[0]	Float32

D must be positive. 0 may be zero and returns [0]. Input, gamma, and weights must have ranks one, one, and two. Inner widths must match and epsilon must be finite and positive.

The CPU baseline materializes a Float32 normalized array. The Metal path does not. The judge computes an independent Double reference and accepts relative error $8e-5$, including a width-257 fixture that crosses the threadgroup boundary.

13.7 CPU reference path

The baseline intentionally exposes the avoided intermediate:

1. Accumulate Float32 sum of squares over input.
2. Compute the inverse RMS.
3. Allocate [D] and write $\text{input}[j] * \text{inverseRMS} * \text{gamma}[j]$.
4. Run one readable dot product per weight row.

This is the semantic comparison path, not an attempted CPU fusion. Keeping it separate makes the memory difference visible and gives the Metal output a canonical result.

13.8 Correctness method

The judge compares small hand-sized values, width 257, and zero output rows against a Double oracle. Error cases cover wrong input rank, projection inner width, gamma width, and epsilon.

The wide case distinguishes a proper strided reduction from a kernel that processes only the first 256 features. The wrong-path test performs an unnormalized projection and must fail even though its output shape is correct.

Run:

```
1 swift run leetllm check 012 --cpu
2 swift run leetllm check 012 --metal
3 swift run leetllm check 012 --solution
```

13.9 Performance model

The separate baseline writes and later reads the normalized vector, adding at least $8D$ bytes of intermediate traffic for one consumer, plus a kernel dispatch. GEMV still reads $4OD$ bytes of weights and performs about $2OD$ floating-point operations.

The implemented fused mapping launches one threadgroup per output row. Every group first reduces all D input squares, then reduces one projection dot product. It removes the D -element normalized write/read and a dispatch, but it recomputes the same RMS reduction O times.

Approximate fused work is therefore

$$O \cdot D \text{ squares/adds} + 2OD \text{ projection FLOPs,}$$

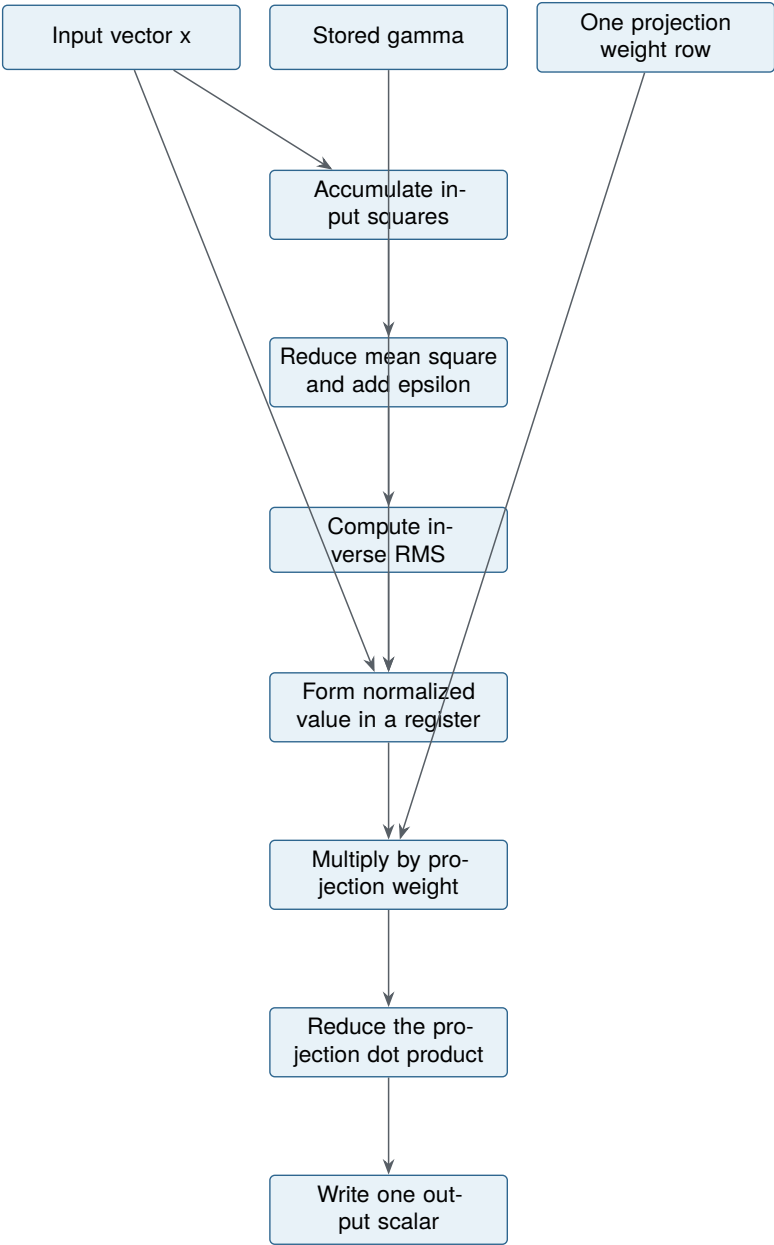


Figure 13.1: Fused rmsnorm gemv dataflow.

instead of one D -element norm reduction plus $2OD$ projection FLOPs. For large O , projection weight traffic may still dominate, but repeated input reduction is real overhead. This kernel is an inspectable first fusion, not a universal optimum.

13.10 Metal mapping

One 256-thread threadgroup owns one output row.

1. Threads scan input in strides of 256 and accumulate squares.
2. A tree reduction computes the shared sum of squares.
3. Every thread derives the same inverse RMS.
4. Threads scan their projection row, form normalized values in registers, and multiply by weights.
5. A second tree reduction produces one output scalar.

No global normalized buffer exists. Threadgroup memory holds only 256 partial Floats and is reused between reductions. All threads reach all barriers.

Threadgroups cannot directly share the computed inverse RMS. A more advanced design could dedicate one group to a tile of output rows so it computes RMS once and reuses it, at the cost of more threadgroup storage and a 2D ownership scheme. That is the next optimization justified if profiling shows repeated norm work matters.

See [P012FusedRMSNormGEMV.metal](#).

13.11 Implementation checkpoints

1. Validate $[D]$, $[D]$, and $[0, D]$ contracts.
2. Implement the separate CPU baseline and compare the worked fixture.
3. Write only the fused Metal sum-of-squares reduction.
4. Compute inverse RMS after the first barrier sequence.
5. Form normalized values in registers during the weight dot product.
6. Reduce the projection and write one scalar per threadgroup.
7. Verify width 257 and zero output rows.
8. Confirm no normalized Metal buffer is allocated by the host pipeline.

13.12 Controlled experiments

13.12.1 Experiment A: output-row sweep

Fix $D = 4096$ and sweep O through 1, 8, 256, and 4096. Prediction: the implemented fusion is attractive at small O because it avoids an intermediate and launch with little repeated RMS work. As O grows, repeated norm reduction becomes increasingly wasteful, though weight traffic may mask it.

13.12.2 Experiment B: feature-width sweep

Fix O and test $D=64, 256, 257$, and 4096. Prediction: reduction and weight work scale with D ; width 257 exercises a second strided element without requiring another threadgroup.

13.12.3 Experiment C: baseline versus fused traffic

Instrument a separate Metal RMSNorm followed by Metal GEMV and compare it with the fused kernel. Prediction: fusion removes one dispatch and $8D$ explicit intermediate bytes. Report kernel-only and end-to-end times because buffer allocation can overwhelm small shapes.

13.12.4 Experiment D: parity under scale

Scale input by powers of two while holding weights and gamma fixed. Prediction: ignoring epsilon, RMSNorm makes projection output approximately scale-invariant; epsilon causes deviations near zero.

13.13 Engine integration

Pre-norm attention and MLP blocks feed one normalized residual into several projections. The current kernel models one projection. Gate and up projections could each use it, but doing so repeats RMS twice as well as once per output row. A multi-projection fused tile could compute the norm once and feed Q/K/V or gate/up rows together.

The engine should keep the separate baseline available for parity and fallback. Fusion is selected by measured shape and reuse, not because fewer source lines or dispatches are automatically faster.

13.14 Tradeoffs

- Fusion removes an intermediate buffer and dispatch but specializes the kernel.
- One output row per group exposes output parallelism but repeats RMS work.
- A tile of output rows can reuse RMS but needs more complex synchronization and may increase register or threadgroup pressure.
- Materialization can win when several consumers reuse the same normalized vector.
- Float32 is the implemented path; changing input or weight dtype requires new parity evidence.

13.15 Hints

- Reuse one scratch array only after the first reduction is completely consumed.
- Do not write normalized values to a device buffer, even temporarily.
- Index weights as $\text{outputRow} * D + \text{column}$.
- Every thread must participate in both reduction barrier sequences.
- If only the first 256 features affect output, inspect the strided loops.

13.16 Completion checklist

- ☐ The separate CPU baseline passes the independent judge.
- ☐ The fused Metal path passes the same value and error cases.
- ☐ No normalized input tensor is allocated by the fused host path.
- ☐ Width 257 contributes every feature.
- ☐ You can quantify the removed intermediate as approximately $8D$ bytes.
- ☐ You can explain the repeated RMS reduction limitation.
- ☐ You ran one output-row, width, traffic, or scale experiment with a prediction.
- ☐ You can name a later multi-row or multi-projection reuse strategy.

13.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

13.17.1 Sources/LeetLLMExercises/P012FusedRMSNormGEMVExercise.swift

```
1 import LeetLLMCore
2
3 public enum P012FusedRMSNormGEMVExercise {
4     public static func project(_ input: FloatTensor, gamma: FloatTensor, weights: FloatTensor, epsilon: Float) throws ->
        FloatTensor {
5         ↪ try validate(input, gamma: gamma, weights: weights, epsilon: epsilon)
6         // TODO: Compute the baseline RMSNorm and projection without changing the public contract.
7         return try FloatTensor(Array(repeating: 0, count: weights.shape[0]), shape: [weights.shape[0]])
8     }
9
10    private static func validate(_ input: FloatTensor, gamma: FloatTensor, weights: FloatTensor, epsilon: Float) throws {
11        guard input.rank == 1 else { throw TensorError.rankMismatch(expected: 1, actual: input.rank) }
12        guard gamma.rank == 1 else { throw TensorError.rankMismatch(expected: 1, actual: gamma.rank) }
13        guard weights.rank == 2 else { throw TensorError.rankMismatch(expected: 2, actual: weights.rank) }
14        guard !input.storage.isEmpty else { throw FusedRMSNormGEMVError.emptyInput }
15        guard weights.shape[1] == input.shape[0] else { throw FusedRMSNormGEMVError.inputWidthMismatch(expected:
16            ↪ weights.shape[1], actual: input.shape[0]) }
17        guard gamma.shape[0] == input.shape[0] else { throw FusedRMSNormGEMVError.gammaWidthMismatch(expected:
18            ↪ input.shape[0], actual: gamma.shape[0]) }
19        guard epsilon.isFinite, epsilon > 0 else { throw FusedRMSNormGEMVError.invalidEpsilon(epsilon) }
20    }
21 }
```

13.17.2 Sources/LeetLLMExercises/P012FusedRMSNormGEMVMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P012FusedRMSNormGEMVExercise {
5     public static func makeFusedMetalPipeline() throws -> MetalFusedRMSNormGEMVPipeline {
6         guard let url = Bundle.module.url(forResource: "P012FusedRMSNormGEMV", withExtension: "metal", subdirectory:
7             ↪ "Metal") else {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P012FusedRMSNormGEMV.metal")
9         }
10        return try MetalFusedRMSNormGEMVPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }
```

13.17.3 Sources/LeetLLMExercises/Metal/P012FusedRMSNormGEMV.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void fused_rmsnorm_gemv(
5     device const float *input [[buffer(0)]],
6     device const float *gamma [[buffer(1)]],
7     device const float *weights [[buffer(2)]],
8     device float *output [[buffer(3)]],
9     constant uint2 &shape [[buffer(4)]],
10    constant float &epsilon [[buffer(5)]],
11    uint localIndex [[thread_index_in_threadgroup]],
12    uint outputRow [[threadgroup_position_in_grid]]
```

```
13 ) {  
14     // TODO: Reduce RMS and the projection in this dispatch without an intermediate tensor.  
15     if (localIndex == 0) {  
16         output[outputRow] = input[0] * gamma[0] * weights[outputRow * shape.y] * epsilon * 0.0f;  
17     }  
18 }
```

13.18 Worked solution

Solution strategy

Fuses RMSNorm and GEMV into a single Metal kernel: computes row sum of squares, reduces to inverse RMS via threadgroup reduction, then accumulates dot products with weight rows using the normalized input directly (without materializing the normalized vector). Two reductions per kernel dispatch: first for RMS, second for dot product. Each threadgroup computes one output element.

Correctness invariant

Output shape is [num_output_rows]. Composed equation is preserved: normalize input then project via weights. Normalized intermediate never materializes to global memory. Each threadgroup redundantly recomputes the same RMS for all output rows it processes. Epsilon remains inside the RMS square root.

Common failure mode

Forgetting to normalize before projection (test rejects unnormalized projection that just multiplies input by weights). Syncing only the RMS reduction, not the dot product reduction. Computing RMS per-threadgroup instead of globally (leads to per-row redundancy, which is acceptable but different). Placing epsilon outside the square root.

13.18.1 Implementation points

Fusion removes write and read of normalized vector

Separate pipeline: RMSNorm writes [D] normalized values, GEMV kernel reads them. Fused pipeline: one kernel normalizes in registers and immediately uses normalized values for dot products. Saves one buffer allocation, one write, one read, and one dispatch.

Composed semantics are preserved exactly

$y_i = \sum_j W[i,j] * (\text{input}[j] / \text{rms}(\text{input})) * \text{gamma}[j]$. Fusion changes storage and scheduling, not the equation. Epsilon is still inside rms sqrt, gamma is still direct multiplication.

Redundant per-threadgroup RMS computation

Dispatch is one kernel per output row. Each threadgroup recomputes the same RMS from the entire input vector. This is redundant (RMS is shared across all output rows) but simpler than shared RMS state. Trade-off is documented and acceptable.

Two threadgroup reductions in one dispatch

First reduction: threads accumulate input squares, tree-reduce to sum, compute inverseRMS. Barrier synchronizes. Second reduction: threads compute normalized * weight products, tree-reduce dot product sum. Only localIndex == 0 writes the final output.

Judge tests wide inputs crossing reduction boundary

Input width 257 is tested with 256-thread threadgroup, forcing threads to loop multiple times. Judge also includes zero output rows (empty weights) to verify shape contract. Tolerance is relative 8e-5 accounting for both RMS reduction and projection accumulation.

13.18.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P012FusedRMSNormGEMVMetal.swift

- Line 4: extension P012FusedRMSNormGEMVSolution {
- Line 5: public static func makeFusedMetalPipeline() throws -> MetalFusedRMSNormGEMVPipeline {

Sources/LeetLLMSolutions/P012FusedRMSNormGEMVSolution.swift

- Line 4: public enum P012FusedRMSNormGEMVSolution {
- Line 5: public static func baseline(_ input: FloatTensor, gamma: FloatTensor, weights: FloatTensor, epsilon: Float) throws -> Fl
- Line 19: private static func validate(_ input: FloatTensor, gamma: FloatTensor, weights: FloatTensor, epsilon: Float) throws {

Sources/LeetLLMSolutions/Metal/P012FusedRMSNormGEMV.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

13.18.3 Complete canonical listings**13.18.4 Sources/LeetLLMSolutions/P012FusedRMSNormGEMVMetal.swift**

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P012FusedRMSNormGEMVSolution {
5     public static func makeFusedMetalPipeline() throws -> MetalFusedRMSNormGEMVPipeline {
6         guard let url = Bundle.module.url(forResource: "P012FusedRMSNormGEMV", withExtension: "metal", subdirectory:
7             ↪ "Metal") else {
8             throw MetalNeuralOperatorError.kernelResourceMissing("P012FusedRMSNormGEMV.metal")
9         }
10        return try MetalFusedRMSNormGEMVPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }

```

13.18.5 Sources/LeetLLMSolutions/P012FusedRMSNormGEMVSolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P012FusedRMSNormGEMVSolution {
5     public static func baseline(_ input: FloatTensor, gamma: FloatTensor, weights: FloatTensor, epsilon: Float) throws ->
6         ↪ FloatTensor {
7         try validate(input, gamma: gamma, weights: weights, epsilon: epsilon)
8         let width = input.shape[0]
9         var sumSquares: Float = 0
10        for value in input.storage { sumSquares += value * value }
11        let inverseRMS = 1 / sqrt(sumSquares / Float(width) + epsilon)
12        let normalized = (0..

```

```

15     }
16     return try FloatTensor(output, shape: [output.count])
17 }
18
19 private static func validate(_ input: FloatTensor, gamma: FloatTensor, weights: FloatTensor, epsilon: Float) throws {
20     guard input.rank == 1 else { throw TensorError.rankMismatch(expected: 1, actual: input.rank) }
21     guard gamma.rank == 1 else { throw TensorError.rankMismatch(expected: 1, actual: gamma.rank) }
22     guard weights.rank == 2 else { throw TensorError.rankMismatch(expected: 2, actual: weights.rank) }
23     guard !input.storage.isEmpty else { throw FusedRMSNormGEMVError.emptyInput }
24     guard weights.shape[1] == input.shape[0] else { throw FusedRMSNormGEMVError.inputWidthMismatch(expected:
25         ↪ weights.shape[1], actual: input.shape[0]) }
26     guard gamma.shape[0] == input.shape[0] else { throw FusedRMSNormGEMVError.gammaWidthMismatch(expected:
27         ↪ input.shape[0], actual: gamma.shape[0]) }
28     guard epsilon.isFinite, epsilon > 0 else { throw FusedRMSNormGEMVError.invalidEpsilon(epsilon) }
29 }

```

13.18.6 Sources/LeetLLMSolutions/Metal/P012FusedRMSNormGEMV.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint threadgroupWidth = 256;
5
6 kernel void fused_rmsnorm_gemv(
7     device const float *input [[buffer(0)]],
8     device const float *gamma [[buffer(1)]],
9     device const float *weights [[buffer(2)]],
10    device float *output [[buffer(3)]],
11    constant uint2 &shape [[buffer(4)]],
12    constant float &epsilon [[buffer(5)]],
13    uint localIndex [[thread_index_in_threadgroup]],
14    uint outputRow [[threadgroup_position_in_grid]]
15 ) {
16     threadgroup float scratch[threadgroupWidth];
17     float localSumSquares = 0.0f;
18     for (uint column = localIndex; column < shape.y; column += threadgroupWidth) {
19         float value = input[column];
20         localSumSquares += value * value;
21     }
22     scratch[localIndex] = localSumSquares;
23     threadgroup_barrier(mem_flags::mem_threadgroup);
24     for (uint stride = threadgroupWidth / 2; stride > 0; stride >= 1) {
25         if (localIndex < stride) {
26             scratch[localIndex] += scratch[localIndex + stride];
27         }
28         threadgroup_barrier(mem_flags::mem_threadgroup);
29     }
30     float inverseRMS = rsqrt(scratch[0] / float(shape.y) + epsilon);
31     float localProjection = 0.0f;
32     for (uint column = localIndex; column < shape.y; column += threadgroupWidth) {
33         float normalized = input[column] * inverseRMS * gamma[column];
34         localProjection += weights[outputRow * shape.y + column] * normalized;
35     }
36     scratch[localIndex] = localProjection;
37     threadgroup_barrier(mem_flags::mem_threadgroup);
38     for (uint stride = threadgroupWidth / 2; stride > 0; stride >= 1) {
39         if (localIndex < stride) {
40             scratch[localIndex] += scratch[localIndex + stride];
41         }
42         threadgroup_barrier(mem_flags::mem_threadgroup);
43     }
44     if (localIndex == 0) {
45         output[outputRow] = scratch[0];
46     }
47 }

```

Part IV

Positions and Attention

Embedding Lookup and Tied Output Weights

14.1 Why this exists

A decoder begins with integer token IDs, but every transformer block consumes vectors. The embedding table supplies that first conversion. At the other end, the final hidden vector must become one logit per vocabulary item. Many models tie those two operations to the same parameter table.

The two directions have different execution shapes. Lookup is an indexed gather of selected rows. Unembedding is a dense projection against every row. Calling both “an embedding layer” hides the difference that matters to an inference engine.

14.2 Learning outcomes

After this problem, you can:

- state the $[V, D]$ table contract and reject invalid token IDs;
- distinguish a gather from GEMV or GEMM;
- derive tied logits from the same rows used for lookup;
- account separately for lookup traffic and unembedding FLOPs;
- map both operations to real Metal kernels; and
- explain when tying saves parameters without saving output computation.

14.3 Prerequisites

- Problem 002 for row-major offsets and checked shapes.
- Problem 004 for recognizing a dot product over a weight row.
- Problem 005 for the multi-token projection interpretation.

14.4 Vocabulary

- **Token**: integer ID representing one model input or output unit, such as a word piece, punctuation mark, byte, or special marker.
- **Vocabulary size V** : number of token IDs accepted by the model.
- **Embedding dimension D** : width of each token vector.
- **Gather**: reading selected rows by integer index rather than combining all rows.
- **Unembedding**: projecting hidden vectors to vocabulary logits.
- **Weight tying**: using one $[V, D]$ table for both input lookup and output projection.
- **Logit**: an unnormalized score for one vocabulary item.

14.5 Math from first principles

Let the table be $E \in \mathbb{R}^{V \times D}$ and token IDs be $t_0, \dots, t_{S-1}$. Lookup is

$$X_{s,d} = E_{t_s,d}.$$

Nothing is multiplied or added. The token controls an address. Tied unembedding uses the gathered row as a hidden vector and computes

$$L_{s,v} = \sum_{d=0}^{D-1} X_{s,d} E_{v,d}.$$

This is $L = XE^T$, even though the table remains physically $[V,D]$.

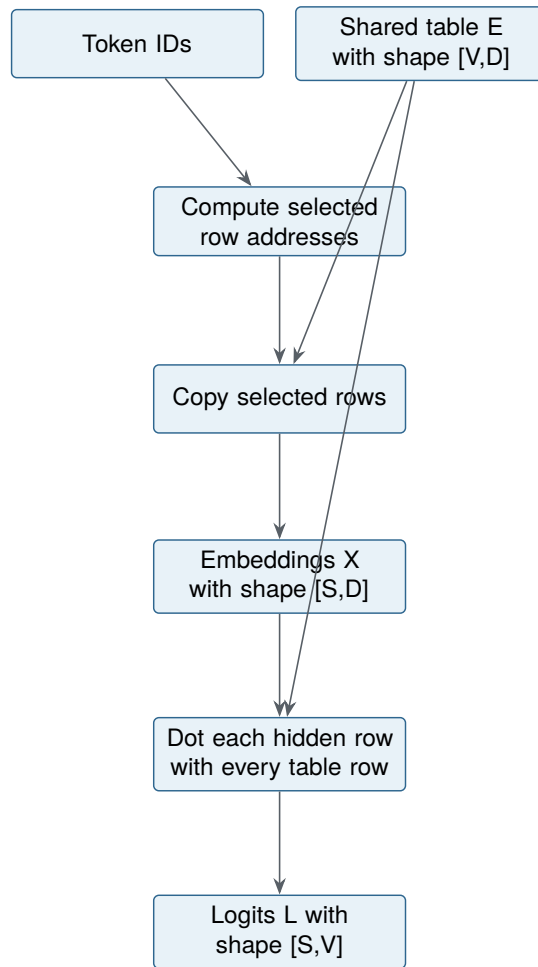


Figure 14.1: Tied embedding dataflow.

14.5.1 Worked numerical example

Take

$$E = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}, \quad t = [2, 0].$$

Lookup gathers

$$X = \begin{bmatrix} 5 & 6 \\ 1 & 2 \end{bmatrix}.$$

The first logit row is $[5(1) + 6(2), 5(3) + 6(4), 5(5) + 6(6)] = [17, 39, 61]$. The second is $[5, 11, 17]$. A solution that gathers correctly but returns zero logits is incomplete.

14.6 Shape, layout, and dtype contract

Value	Shape	Layout	Dtype
table	$[V, D]$	contiguous row-major	Float32
token IDs	$[S]$	Swift array	Int, converted to UInt32 for Metal
embeddings	$[S, D]$	contiguous row-major	Float32
logits	$[S, V]$	contiguous row-major	Float32

V and D must be positive. S may be zero. Every token must satisfy $0 \leq t_s < V$. The API rejects a non-matrix table, empty width or vocabulary, negative IDs, and IDs equal to V .

14.7 CPU reference path

For each token, append exactly one contiguous table row. Then, for every (sequence, vocabulary) pair, run a readable length- D dot product between the gathered vector and table row. The canonical CPU path deliberately keeps these as two loops so gather is not disguised as matrix multiplication.

14.8 Independent correctness method

The judge builds embeddings by row slicing and accumulates tied logits in Double. It checks gather order, duplicate IDs, mixed signs, empty sequences, and invalid IDs. Absolute tolerance is $2e-5$; relative tolerance is $4e-5$. The judge does not call the canonical implementation.

Run:

```
1 swift run leetllm check 013 --cpu
2 swift run leetllm check 013 --metal
3 swift run leetllm check 013 --solution
```

14.9 Performance model

Lookup writes and reads approximately $4SD$ bytes each, plus token IDs. It performs no model FLOPs and has low arithmetic intensity. Random token rows can defeat locality when the table exceeds cache.

Unembedding performs about $2SD$ FLOPs and writes $4SV$ bytes. The table is $4VD$ bytes and is the dominant parameter read. The gathered $[S,D]$ tensor is an explicit intermediate in this lesson. Weight tying removes a second $4VD$ -byte parameter table; it does not remove the SD projection.

14.10 Metal mapping

`embedding_lookup` dispatches $S \cdot D$ threads. A thread derives (s,d) , reads `tokenIDs[s]`, and copies `table[token * D + d]`. There are no barriers.

`tied_unembedding` dispatches $S \cdot V$ threads. Each thread owns one logit and loops over D . The host ends the lookup encoder before starting unembedding, so the second dispatch observes the gathered buffer. This baseline favors a visible mapping over a tiled GEMM; Problem 005 supplies the optimization path.

See [P013Embedding.metal](#).

14.11 Implementation checkpoints

1. Reject invalid table ranks and token IDs before allocation.
2. Gather one known token and inspect its exact row.
3. Preserve input token order and duplicates.
4. Compute one tied logit as a row dot product.
5. Extend to $[S,V]$, including $S=0$.
6. Implement the Metal gather grid.
7. Implement the independent unembedding grid and pass the shared judge.

14.12 Controlled experiments

14.12.1 Token locality

Compare repeated IDs, adjacent IDs, and pseudorandom IDs at fixed S and D . Prediction: repeated and nearby rows should reuse more cache; arithmetic work is unchanged.

14.12.2 Vocabulary sweep

Fix $S=1, D=1024$ and sweep V . Prediction: lookup time changes little, while unembedding grows linearly with V and its table traffic dominates.

14.12.3 Decode versus prefill

Compare $S=1$ and larger S at fixed V, D . Prediction: the simple Metal unembedding leaves reuse on the table for larger S ; a tiled GEMM becomes more attractive.

Write the prediction before recording release-build timings.

14.13 Engine integration

The gathered $[S,D]$ tensor is the initial residual stream. After all decoder layers and final normalization, the same table can project hidden states to $[S,V]$ logits. A production engine may compute only the final row during decode, but the equation and tied storage remain the same.

14.14 Tradeoffs

- Tying reduces parameters but constrains input and output spaces to share a table.
- A direct gather is clearer than a one-hot GEMM and avoids useless zeros.
- The scalar unembedding kernel is inspectable; tiled GEMM has better reuse for prefill.
- Materializing embeddings aids composition; fusion may save traffic in specialized paths.

14.15 Hints

- Row start is token * D.
- Do not transpose the stored table just to express E^T .
- Validate Int IDs before converting them to UInt32.
- Empty S returns [0,D] and [0,V], not an error.

14.16 Completion checklist

- ☐ Lookup and tied logits pass the independent judge.
- ☐ Invalid tokens fail before indexing or dispatch.
- ☐ The Metal check executes both MSL kernels.
- ☐ You can distinguish gather bytes from unembedding FLOPs.
- ☐ You ran a locality, vocabulary, or sequence experiment with a prediction.
- ☐ You can name where both outputs enter the decoder engine.

14.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

14.17.1 Sources/LeetLLMExercises/P013EmbeddingExercise.swift

```
1 import LeetLLMCore
2
3 public enum P013EmbeddingExercise {
4     public static func apply(
5         tokenIDs: [Int],
6         table: FloatTensor
7     ) throws -> EmbeddingLookupResult {
8         try EmbeddingLookupContract.validate(tokenIDs: tokenIDs, table: table)
9         let embeddings = try FloatTensor(
10             Array(repeating: 0, count: tokenIDs.count * table.shape[1]),
11             shape: [tokenIDs.count, table.shape[1]]
12         )
13         let logits = try FloatTensor(
14             Array(repeating: 0, count: tokenIDs.count * table.shape[0]),
15             shape: [tokenIDs.count, table.shape[0]]
16         )
17         return EmbeddingLookupResult(embeddings: embeddings, logits: logits)
18     }
19 }
```

14.17.2 Sources/LeetLLMExercises/P013EmbeddingMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P013EmbeddingExercise {
5     public static func makeMetalPipeline() throws -> MetalEmbeddingPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P013Embedding",
9                 withExtension: "metal",
10                 subdirectory: "Metal"
11             )
12         else {
13             throw MetalNeuralOperatorError.kernelResourceMissing("P013Embedding.metal")
14         }
15         return try MetalEmbeddingPipeline(source: String(contentsOf: url, encoding: .utf8))
16     }
17 }
```

14.17.3 Sources/LeetLLMExercises/Metal/P013Embedding.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void embedding_lookup(
5     device const float *table [[buffer(0)]],
6     device const uint *tokenIDs [[buffer(1)]],
7     device float *embeddings [[buffer(2)]],
8     constant uint4 &shape [[buffer(3)]],
9     uint index [[thread_position_in_grid]]
10 ) {
```

```
11     uint count = shape.y * shape.z;
12     if (index >= count) {
13         return;
14     }
15     embeddings[index] = 0.0f;
16 }
17
18 kernel void tied_unembedding(
19     device const float *embeddings [[buffer(0)]],
20     device const float *table [[buffer(1)]],
21     device float *logits [[buffer(2)]],
22     constant uint4 &shape [[buffer(3)]],
23     uint index [[thread_position_in_grid]]
24 ) {
25     uint count = shape.x * shape.z;
26     if (index >= count) {
27         return;
28     }
29     logits[index] = 0.0f;
30 }
```

14.18 Worked solution

Solution strategy

Embedding lookup is an indexed gather of selected rows from a $[V,D]$ table driven by token IDs. Tied unembedding projects the gathered embeddings against every row of the same table via dot product, materially a row-major GEMV. Two separate access patterns—gather for lookup efficiency, dense projection for unembedding—share one parameter table to reduce model size. The implementation first reserves and fills embeddings by scattering token-indexed rows, then fills logits $[S,V]$ by nested loops: for each sequence, for each vocabulary word, dot the gathered embedding row with the vocabulary word row from the table.

Correctness invariant

Every token ID must satisfy $0 \leq t < V$. Embeddings are rows from the table selected by these IDs; logits equal embedded vectors multiplied by table rows transposed. Tying reduces parameters without eliminating the dense projection work; unembedding is not a lookup.

Common failure mode

Returning zero logits when embeddings are correctly gathered. The gather succeeds but the unembedding dot products are omitted or incorrectly computed, producing incorrect final logits.

14.18.1 Implementation points

Token ID validation and row-major offset mapping

Each token ID t maps to byte offset $t * D$ within the $[V,D]$ table stored row-major. Metal kernels extract tensor dimensions from $\text{shape}[0]=V, \text{shape}[1]=D$; CPU code uses $\text{table.shape}[0]$ and $\text{table.shape}[1]$. Contract validation rejects $t < 0, t \geq V$, or non-matrix tables before any arithmetic.

Gather as indexed copy, not projection

Embedding lookup copies entire rows from the table into a contiguous $[S,D]$ buffer using token indices as addresses. No matrix multiplication occurs; this is a memory read pattern, not a linear combination.

Tied unembedding as GEMV with table rows

Logits $[S,V] = \text{embeddings } [S,D] @ \text{table.T } [D,V]$, even though table is stored $[V,D]$. For each (sequence s , vocabulary v), $\text{logits}[s,v] = \text{sum over } d \text{ of } \text{embeddings}[s,d] * \text{table}[v,d]$. This is a complete dense projection.

Two kernels or one fused operation

Metal provides `embedding_lookup` and `tied_unembedding` as separate kernels: one gathers embeddings via tokenID-indexed reads, one computes logits via nested loops over vocabulary. Both must run; neither can be omitted.

14.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P013EmbeddingMetal.swift

- Line 4: extension P013EmbeddingSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalEmbeddingPipeline {

Sources/LeetLLMSolutions/P013EmbeddingSolution.swift

- Line 3: public enum P013EmbeddingSolution {
- Line 4: public static func apply(

Sources/LeetLLMSolutions/Metal/P013Embedding.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

14.18.3 Complete canonical listings

14.18.4 Sources/LeetLLMSolutions/P013EmbeddingMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P013EmbeddingSolution {
5     public static func makeMetalPipeline() throws -> MetalEmbeddingPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P013Embedding",
9                 withExtension: "metal",
10                 subdirectory: "Metal"
11             )
12         else {
13             throw MetalNeuralOperatorError.kernelResourceMissing("P013Embedding.metal")
14         }
15         return try MetalEmbeddingPipeline(source: String(contentsOf: url, encoding: .utf8))
16     }
17 }
```

14.18.5 Sources/LeetLLMSolutions/P013EmbeddingSolution.swift

```
1 import LeetLLMCore
2
3 public enum P013EmbeddingSolution {
4     public static func apply(
5         tokenIDs: [Int],
6         table: FloatTensor
7     ) throws -> EmbeddingLookupResult {
8         try EmbeddingLookupContract.validate(tokenIDs: tokenIDs, table: table)
9         let vocabularySize = table.shape[0]
10        let embeddingDimension = table.shape[1]
11        var embeddings: [Float] = []
12        embeddings.reserveCapacity(tokenIDs.count * embeddingDimension)
13        for token in tokenIDs {
14            let rowStart = token * embeddingDimension
15            embeddings.append(contentsOf: table.storage[rowStart.. $(rowStart + embeddingDimension)$ ])
16        }
17
18        var logits = Array(repeating: Float.zero, count: tokenIDs.count * vocabularySize)
19        for sequence in tokenIDs.indices {
20            for vocabulary in 0.. $vocabularySize$  {
21                var sum: Float = 0
22                for feature in 0.. $embeddingDimension$  {
23                    sum +=
24                        embeddings[sequence * embeddingDimension + feature]
```



```

25         * table.storage[vocabulary * embeddingDimension + feature]
26     }
27     logits[sequence * vocabularySize + vocabulary] = sum
28 }
29 }
30 return EmbeddingLookupResult(
31     embeddings: try FloatTensor(
32         embeddings,
33         shape: [tokenIDs.count, embeddingDimension]
34     ),
35     logits: try FloatTensor(logits, shape: [tokenIDs.count, vocabularySize])
36 )
37 }
38 }

```

14.18.6 Sources/LeetLLMSolutions/Metal/P013Embedding.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void embedding_lookup(
5     device const float *table [[buffer(0)]],
6     device const uint *tokenIDs [[buffer(1)]],
7     device float *embeddings [[buffer(2)]],
8     constant uint4 &shape [[buffer(3)]],
9     uint index [[thread_position_in_grid]]
10 ) {
11     uint count = shape.y * shape.z;
12     if (index >= count) {
13         return;
14     }
15     uint sequence = index / shape.y;
16     uint feature = index % shape.y;
17     embeddings[index] = table[tokenIDs[sequence] * shape.y + feature];
18 }
19
20 kernel void tied_unembedding(
21     device const float *embeddings [[buffer(0)]],
22     device const float *table [[buffer(1)]],
23     device float *logits [[buffer(2)]],
24     constant uint4 &shape [[buffer(3)]],
25     uint index [[thread_position_in_grid]]
26 ) {
27     uint count = shape.x * shape.z;
28     if (index >= count) {
29         return;
30     }
31     uint sequence = index / shape.x;
32     uint vocabulary = index % shape.x;
33     float sum = 0.0f;
34     for (uint feature = 0; feature < shape.y; ++feature) {
35         sum += embeddings[sequence * shape.y + feature]
36             * table[vocabulary * shape.y + feature];
37     }
38     logits[index] = sum;
39 }

```

Q/K/V Projections and Head Views

15.1 Why this exists

Attention does not consume the residual stream directly. It projects each token into queries, keys, and values, then interprets projection columns as heads. Model architecture is encoded in these dimensions: query-head count can differ from key/value-head count, while every head shares one width.

This problem keeps the transformations visible. It does not hide projection, split, and reshape behind an “attention layer” wrapper.

15.2 What query, key, and value mean

Attention lets each token position gather information from positions in its visible context. It assigns three different roles to learned projections of the same residual-stream input:

- A **query (Q)** represents what the receiving position is looking for.
- A **key (K)** represents how one context position can be matched.
- A **value (V)** contains the information that position can contribute.

For one attention head, a query-key dot product produces a match score. Softmax normalizes the visible scores into weights, and the weighted sum of the value vectors becomes the gathered context. The Q, K, and V labels describe these roles in that calculation; they are numerical vectors, not literal questions, database keys, or stored text.

The next lessons add position information to Q and K, prevent access to future positions, and perform the score-and-value calculation. This lesson owns the learned projections and the exact tensor shapes they must hand to those stages.

15.3 Learning outcomes

You will be able to:

- derive Q, K, and V projection matrix dimensions;
- represent separate H_q and H_{kv} counts;
- map a flat projection column to (head, feature);
- decide when $[S, H, dh]$ is a view and when a copy is required;
- validate all shape contracts before arithmetic; and
- connect decode GEMV and prefill GEMM to the same projection equation.

15.4 Prerequisites

- Problem 002 for shapes, strides, reshape, and contiguity.
- Problems 004 and 005 for matrix-vector and matrix-matrix projection.
- Problem 011 for keeping the residual stream's precision policy separate.

15.5 Vocabulary

- **Model dimension D** : residual-stream width.
- **Head dimension d_h** : feature width assigned to one attention head.
- **Query vector**: projected features used to score relevant context positions.
- **Key vector**: projected features against which a query is scored.
- **Value vector**: projected features mixed according to the query-key scores.
- **Query head H_q** : an independently projected query subspace.
- **KV head H_{kv}** : a key/value subspace, possibly shared by query heads later.
- **Head view**: interpreting contiguous flat columns as $[H, d_h]$ without moving data.
- **Materialization**: copying values to satisfy a new physical layout.

15.6 Math from first principles

For hidden states $X \in \mathbb{R}^{S \times D}$, choose

$$W_Q \in \mathbb{R}^{D \times (H_q d_h)}, \quad W_K, W_V \in \mathbb{R}^{D \times (H_{kv} d_h)}.$$

The flat projections are

$$Q_f = XW_Q, \quad K_f = XW_K, \quad V_f = XW_V.$$

Column c becomes

$$h = \lfloor c/d_h \rfloor, \quad f = c \bmod d_h.$$

No data moves if columns for each head are contiguous. The same storage can be viewed as $Q [S, H_q, d_h]$ and $K, V [S, H_{kv}, d_h]$.

15.6.1 Worked numerical example

Let $X = [1, 2]$, $H_q = 2$, $H_{kv} = 1$, and $d_h = 1$. Use

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad W_V = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

Then $Q_f = [1, 4]$, $K_f = [3]$, and $V_f = [3]$. The head views are $Q [1, 2, 1]$, $K [1, 1, 1]$, and $V [1, 1, 1]$. The values are unchanged by reshape; only the indexing interpretation changes.

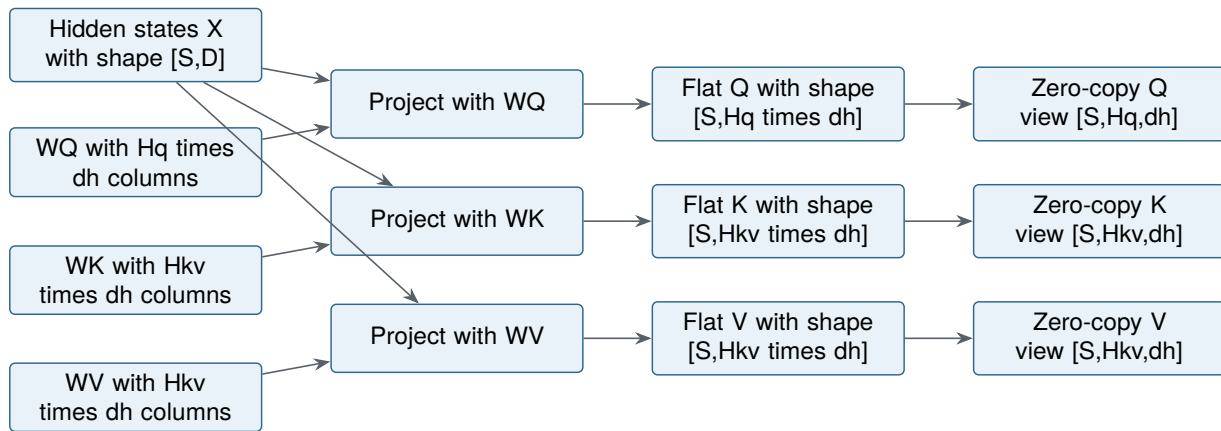


Figure 15.1: Qkv head views.

15.7 Shape, layout, and dtype contract

Value	Shape	Layout	Dtype
hidden	[S,D]	contiguous row-major	Float32
query weights	[D,Hq*dh]	contiguous row-major	Float32
key/value weights	[D,Hkv*dh]	contiguous row-major	Float32
queries	[S,Hq,dh]	strides [Hq*dh,dh,1]	Float32
keys/values	[S,Hkv,dh]	strides [Hkv*dh,dh,1]	Float32

Hq, Hkv, and dh are positive, and Hq must be divisible by Hkv so the same configuration remains valid for Problem 018. S may be zero. All inputs must be finite. Weight inner widths are exact contracts, not inferred from storage count.

15.8 CPU reference path

For each of the three weights:

1. loop over sequence rows;
2. loop over head then feature;
3. map to flat column head*dh+feature;
4. accumulate over D; and
5. construct the final rank-three tensor directly.

The explicit loop order demonstrates that the head split is not another mathematical operation.

15.9 Independent correctness method

The judge projects with Double accumulation and independently constructs [S,H,dh]. It includes Hq != Hkv, an empty sequence, wrong projection width, and non-finite hidden states. Tolerance is $2e-5 + 5e-5 * \text{abs}(\text{expected})$. A starter that returns correctly shaped zeros fails value cases.

```

1 swift run leetllm check 014 --cpu
2 swift run leetllm check 014 --solution

```

15.10 Performance model

The three projections perform approximately

$$2SD(H_q d_h + 2H_{kv} d_h)$$

FLOPs. Weight bytes are $4D(H_q d_h + 2H_{kv} d_h)$. Outputs occupy $4S(H_q d_h + 2H_{kv} d_h)$ bytes. A contiguous reshape allocates zero bytes; materializing a different head-major layout would read and write every output.

For $S=1$, each projection has GEMV's low reuse. For larger S , GEMM can reuse weights across tokens. Reducing H_{kv} lowers K/V projection work and storage but does not lower Q projection work.

15.11 Metal mapping

There is no new Metal stage in this problem. Problems 004 and 005 already own the canonical projection kernels; this lesson owns the shape transformation. On GPU, run three GEMV/GEMM projections and preserve flat column order so the output buffers are already valid $[S, H, d_h]$ views. A fused Q/K/V projection is a later optimization and must preserve the same contract.

No barriers are required for a view. A copy kernel would only be justified for a consumer demanding another physical order.

15.12 Implementation checkpoints

1. Construct and validate an `AttentionConfiguration`.
2. Validate all three weight matrices.
3. Compute one flat query projection row.
4. Map columns to query heads and features.
5. Repeat with the smaller KV width.
6. Verify $[S, H, d_h]$ strides and $S=0$.
7. Reject a shape-correct but value-zero implementation with the judge.

15.13 Controlled experiments

15.13.1 Head-count sweep

Fix D , d_h , and S ; vary H_q and H_{kv} . Prediction: Q work follows H_q , while K/V work and bytes follow H_{kv} .

15.13.2 View versus copy

Compare a contiguous reshape with an explicit reorder to $[H, S, d_h]$. Prediction: reshape is constant-time; copy scales with all projected elements.

15.13.3 Decode versus prefill

Compare $S=1$ against increasing S . Prediction: GEMM gains weight reuse and throughput while GEMV remains bandwidth-oriented.

15.14 Engine integration

Problem 015 rotates Q and K in place conceptually. Problems 016-021 consume the rank-three tensors directly. Later KV-cache problems store K and V using Hkv , not Hq ; preserving that distinction here is part of model correctness.

15.15 Tradeoffs

- Separate projections are transparent; fused QKV can reduce dispatches.
- Token-major $[S, H, dh]$ matches the current kernels; head-major may help other schedules.
- Views avoid copies but constrain producer column order.
- Fewer KV heads reduce work and storage but change the model architecture.

15.16 Hints

- Query columns are $Hq*dh$; key and value columns are $Hkv*dh$.
- Do not divide the model dimension by a head count unless the architecture says so.
- A contiguous $[S, H*dh]$ tensor reshapes to $[S, H, dh]$ with no copy.
- Keep query and KV head counts named separately in every loop.

15.17 Completion checklist

- ☐ Q, K, and V values and shapes pass the judge.
- ☐ $Hq \neq Hkv$ remains explicit.
- ☐ Invalid weights fail before projection.
- ☐ You can derive the rank-three strides.
- ☐ You measured or reasoned about view versus copy.
- ☐ You can identify decode GEMV and prefill GEMM mappings.

15.18 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

15.18.1 Sources/LeetLLMExercises/P014QKVProjectionExercise.swift

```

1 import LeetLLMCore
2
3 public enum P014QKVProjectionExercise {
4     public static func project(
5         hidden: FloatTensor,
6         queryWeights: FloatTensor,
7         keyWeights: FloatTensor,
8         valueWeights: FloatTensor,
9         configuration: AttentionConfiguration
10    ) throws -> QKVProjectionResult {
11        try QKVProjectionContract.validate(
12            hidden: hidden,
13            queryWeights: queryWeights,
14            keyWeights: keyWeights,
15            valueWeights: valueWeights,
16            configuration: configuration
17        )
18        let sequenceLength = hidden.shape[0]
19        return QKVProjectionResult(
20            queries: try FloatTensor(
21                Array(
22                    repeating: 0,
23                    count: sequenceLength * configuration.queryHeadCount * configuration.headDimension),
24                    shape: [sequenceLength, configuration.queryHeadCount, configuration.headDimension]
25                ),
26            keys: try FloatTensor(
27                Array(
28                    repeating: 0,
29                    count: sequenceLength * configuration.keyValueHeadCount * configuration.headDimension),
30                    shape: [sequenceLength, configuration.keyValueHeadCount, configuration.headDimension]
31                ),
32            values: try FloatTensor(
33                Array(
34                    repeating: 0,
35                    count: sequenceLength * configuration.keyValueHeadCount * configuration.headDimension),
36                    shape: [sequenceLength, configuration.keyValueHeadCount, configuration.headDimension]
37                )
38            )
39    }
40 }

```

15.19 Worked solution

Solution strategy

Linear projections of hidden states $[S, D]$ into Q, K, V subspaces are factored by head and feature dimension. Each projection multiplies by a weight matrix $[D, Hq*dh]$ or $[D, Hkv*dh]$, producing flat projections $[S, Hq*dh]$ or $[S, Hkv*dh]$. These flat results are reinterpreted as $[S, H, dh]$ tensors without copying, since contiguous columns map to contiguous head elements via column-index arithmetic: column $c \rightarrow \text{head} = c // dh$, feature $= c \% dh$. Head views enforce strides $[H*dh, dh, 1]$ when underlying data is row-major.

Correctness invariant

Head views require zero-copy interpretation only when projection columns are contiguous by head. Column c must map exactly to $(\text{head}=c//dh, \text{feature}=c\%dh)$ for the view to be valid. Weight matrix inner dimensions are exact contracts: W_Q has $Hq*dh$ columns, W_K and W_V have $Hkv*dh$ columns.

Common failure mode

Materializing a $[S, H, dh]$ copy of a flat $[S, H*dh]$ projection when contiguity allows a zero-copy view. Incorrect head-to-column mapping causes wrong values to appear in wrong head positions.

15.19.1 Implementation points

Projection matrix dimensions and head factorization

W_Q has shape $[D, Hq*dh]$; W_K and W_V have $[D, Hkv*dh]$. Each weight matrix inner dimension must be exact; the API validates these widths, not inferred column counts.

Head-to-column mapping via integer arithmetic

Flat projection column c maps to $\text{head} = \text{floor}(c / dh)$ and $\text{feature} = c \bmod dh$. For sequence s , $\text{flat}[s, c]$ contains the feature value for head $(c // dh)$. Iterating head first, then feature, produces the correct nested loop structure.

Strides and zero-copy head views

A $[S, D] @ [D, H*dh]$ multiplication produces flat $[S, H*dh]$ with strides $[H*dh, 1]$. Reinterpreting as $[S, H, dh]$ without copying requires strides $[H*dh, dh, 1]$, which hold when columns are contiguous by head. This is automatic in row-major layout if columns $0..dh-1$ belong to head 0, $dh..2dh-1$ to head 1, and so on.

Decode vs. prefill shapes and the same projection equation

Decode uses $Q [1, Hq, dh] @ \text{weights} [D, Hq*dh]$, a GEMV. Prefill uses $Q [S, Hq, dh] @ \text{weights} [D, Hq*dh]$, a GEMM. Both invoke the same projection logic; only sequence length changes. Head shapes $[S, Hq, dh]$ result from the same projection in both cases.

15.19.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P014QKVProjectionSolution.swift

- Line 3: public enum P014QKVProjectionSolution {
- Line 4: public static func project(
- Line 18: func projectOne(_ weights: FloatTensor, heads: Int) throws -> FloatTensor {

15.19.3 Complete canonical listings

15.19.4 Sources/LeetLLMSolutions/P014QKVProjectionSolution.swift

```

1 import LeetLLMCore
2
3 public enum P014QKVProjectionSolution {
4     public static func project(
5         hidden: FloatTensor,
6         queryWeights: FloatTensor,
7         keyWeights: FloatTensor,
8         valueWeights: FloatTensor,
9         configuration: AttentionConfiguration
10    ) throws -> QKVProjectionResult {
11        try QKVProjectionContract.validate(
12            hidden: hidden,
13            queryWeights: queryWeights,
14            keyWeights: keyWeights,
15            valueWeights: valueWeights,
16            configuration: configuration
17        )
18        func projectOne(_ weights: FloatTensor, heads: Int) throws -> FloatTensor {
19            let sequenceLength = hidden.shape[0]
20            let modelDimension = hidden.shape[1]
21            let columns = heads * configuration.headDimension
22            var output = Array(repeating: Float.zero, count: sequenceLength * columns)
23            for sequence in 0..

```

Rotary Position Embeddings

16.1 Why this exists

Attention scores otherwise depend on token content but not token order. Rotary position embeddings (RoPE) rotate query and key feature pairs by angles derived from absolute positions. Their dot products then encode relative displacement.

RoPE has several incompatible conventions in real models. This lesson chooses one explicitly and makes offsets and partial rotation part of the API.

16.2 Learning outcomes

After completing the lesson, you can:

- apply a two-dimensional rotation to query and key pairs;
- compute frequency-dependent angles from absolute positions;
- validate even and partial rotary dimensions;
- preserve the unrotated suffix exactly;
- use different query and key position offsets; and
- implement the convention as a real Metal elementwise kernel.

16.3 Prerequisites

- Problem 001 for pairwise arithmetic and numerical parity.
- Problem 002 for [S,H,dh] row-major indexing.
- Problem 014 for the Q and K tensors being rotated.

16.4 Vocabulary

- **Rotary dimension R**: even prefix width that participates in rotation.
- **Adjacent pairing**: pairs (0,1), (2,3), and so on.
- **Frequency pair**: one pair with its own inverse-frequency exponent.
- **Position offset**: absolute position assigned to local sequence index zero.
- **Partial RoPE**: $R < dh$, leaving features $R..<dh$ unchanged.
- **Base B**: frequency scale, commonly but not universally 10,000.

16.5 Math from first principles and exact convention

This repository uses adjacent pairs. For pair index $i = 0, \dots, R/2 - 1$ and absolute position p ,

$$\theta_{p,i} = \frac{p}{B^{2i/R}}.$$

For input pair $(a, b) = (x_{2i}, x_{2i+1})$,

$$x'_{2i} = a \cos \theta - b \sin \theta,$$

$$x'_{2i+1} = a \sin \theta + b \cos \theta.$$

The same formula applies to Q and K, but their absolute offsets can differ. Features $R..<dh$ are copied bit-for-bit. This is not the split-half convention.

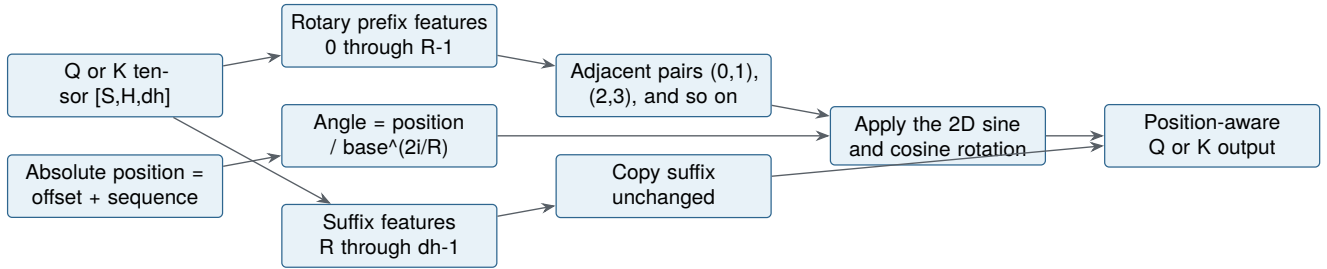


Figure 16.1: Rope adjacent pairs.

16.5.1 Worked numerical example

Let $R=4$, $B=100$, position $p=1$, and $x = [1, 0, 0, 2, 9, 10]$. Pair zero has angle 1; pair one has angle $1/100^{1/2} = 0.1$. Therefore

$$x' \approx [0.5403, 0.8415, -0.1997, 1.9900, 9, 10].$$

At position zero every angle is zero, so all rotated pairs are unchanged. A nonzero offset must change local row zero.

16.6 Shape, layout, and dtype contract

Queries are $[Sq, Hq, dh]$; keys are $[Skv, Hkv, dh]$. Sequence and head counts may differ, but dh must match. Tensors are contiguous row-major Float32. R must be positive, even, and no greater than dh . B must be finite and greater than one. Both offsets are nonnegative integers.

The output shapes equal the input shapes. Partial suffixes are preserved. Empty sequence dimensions are valid, although head and feature dimensions remain defined by the tensor shape.

16.7 CPU reference path

Copy each input tensor, then loop over sequence, head, and pair. Compute the absolute position as offset + sequence, derive the angle, and overwrite only the two prefix features. Keeping the suffix copy visible makes partial RoPE testable.

16.8 Independent correctness method

The judge computes angles and rotations in Double, then casts outputs to Float. It checks positions zero and one, different Q/K offsets, partial suffixes, odd R , $R > d_h$, and negative offsets. Tolerance is $3e-5 + 5e-5 * \text{abs}(\text{expected})$. A no-op implementation passes position zero but fails the offset cases.

```
1 swift run leetllm check 015 --cpu
2 swift run leetllm check 015 --metal
3 swift run leetllm check 015 --solution
```

16.9 Performance model

There are $SHR/2$ pair rotations per tensor. Each pair evaluates one angle, sine, and cosine, then performs four multiplies and two adds/subtracts. The kernel reads and writes the full tensor because the suffix must survive, for about $8SHd_h$ bytes per Q or K tensor.

Precomputing sine/cosine tables trades transcendental work for table storage and reads. Fusing RoPE into Q/K projection or attention can remove a global round trip but couples conventions and offsets to that kernel.

16.10 Metal mapping

The host allocates each output as a copy of its input. The kernel dispatches one thread per (sequence, head, pair). A thread computes two feature indices and overwrites the rotated pair; untouched suffix values remain from the copy.

Q and K use separate dispatches because their sequence lengths, head counts, and offsets can differ. No thread shares a pair, so no barrier or threadgroup memory is needed. Bounds are checked against $S * H * (R/2)$.

See [P015RoPE.metal](#).

16.11 Implementation checkpoints

1. Validate rank, matching d_h , base, offsets, and even R .
2. Rotate one pair at position zero.
3. Rotate one pair at a nonzero position.
4. Add frequency variation across pairs.
5. Preserve a partial suffix.
6. Apply independent Q and K offsets.
7. Match the CPU path with the Metal grid.

16.12 Controlled experiments

16.12.1 Position sweep

Fix one vector and sweep positions. Prediction: pair norms remain constant, while components trace circles at different angular rates.

16.12.2 Base sweep

Fix position and compare bases. Prediction: pair zero is unchanged by the base; higher pair frequencies rotate more slowly as the base grows.

16.12.3 Precomputed versus computed angles

Compare on-the-fly $\sin/\cos$ with a table at fixed shapes. Prediction: a table helps when angles are reused enough to offset extra memory traffic.

16.12.4 Partial dimension sweep

Sweep R while holding dh . Prediction: arithmetic grows with R , but the current host still copies all dh features.

16.13 Engine integration

RoPE runs after Q/K projection and before any query-key dot product. During decode, the new query uses its absolute decode position; cached keys must have been rotated at their own original positions. Position offsets let chunked prefill and cached decode preserve those semantics.

16.14 Tradeoffs

- Adjacent and split-half pairing are not interchangeable model conventions.
- On-the-fly angles save tables; precomputation saves transcendental work.
- Partial RoPE preserves unrotated capacity but is architecture-specific.
- A standalone kernel is inspectable; fusion can remove traffic.

16.15 Hints

- Pair exponent is $\text{feature} / R$, where $\text{feature}=2*i$.
- Read both original pair values before writing either output.
- Use absolute $\text{offset} + \text{sequence}$, not local sequence alone.
- Never rotate the suffix when $R < dh$.

16.16 Completion checklist

- ☐ CPU and Metal pass the same independent judge.
- ☐ The adjacent-pair convention is stated and implemented.
- ☐ Odd or oversized rotary dimensions fail.
- ☐ Nonzero Q and K offsets are tested.
- ☐ The suffix is exactly preserved for partial RoPE.
- ☐ You ran a position, base, table, or dimension experiment with a prediction.

16.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

16.17.1 Sources/LeetLLMExercises/P015RoPEExercise.swift

```
1 import LeetLLMCore
2
3 public enum P015RoPEExercise {
4     public static func apply(
5         queries: FloatTensor,
6         keys: FloatTensor,
7         rotaryDimension: Int,
8         base: Float,
9         queryPositionOffset: Int,
10        keyPositionOffset: Int
11    ) throws -> RoPEResult {
12        try RoPEContract.validate(
13            queries: queries, keys: keys, rotaryDimension: rotaryDimension, base: base,
14            queryPositionOffset: queryPositionOffset, keyPositionOffset: keyPositionOffset)
15        return RoPEResult(queries: queries, keys: keys)
16    }
17 }
```

16.17.2 Sources/LeetLLMExercises/P015RoPEMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P015RoPEExercise {
5     public static func makeMetalPipeline() throws -> MetalRoPEPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P015RoPE", withExtension: "metal", subdirectory: "Metal")
9         else {
10            throw MetalNeuralOperatorError.kernelResourceMissing("P015RoPE.metal")
11        }
12        return try MetalRoPEPipeline(source: String(contentsOf: url, encoding: .utf8))
13    }
14 }
```

16.17.3 Sources/LeetLLMExercises/Metal/P015RoPE.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void rope_adjacent_pairs(
5     device const float *input [[buffer(0)]],
6     device float *output [[buffer(1)]],
7     constant uint4 &shape [[buffer(2)]],
8     constant uint &positionOffset [[buffer(3)]],
9     constant float &ropeBase [[buffer(4)]],
10    uint index [[thread_position_in_grid]]
11 ) {
12     uint count = shape.x * shape.y * shape.w;
13     if (index >= count) {
14         return;
15     }
```

16 }

16.18 Worked solution

Solution strategy

Rotary position embeddings apply 2D rotations to adjacent feature pairs within a rotary prefix $[0, R)$. Each pair $(2i, 2i+1)$ rotates by angle $\theta = \text{position} / B^{(2i/R)}$. For adjacent pair (a, b) , the rotation is $(a \cos \theta - b \sin \theta, a \sin \theta + b \cos \theta)$. Features $[R, dh)$ copy unchanged. The same rotation formula applies to both Q and K, but position offsets can differ: $\text{absolute position} = \text{offset} + \text{sequence index}$.

Correctness invariant

Unrotated pairs must be bit-for-bit unchanged; rotation applies only to $[0, R)$ features in adjacent pairs. The angle formula uses exactly $2i$ in the exponent, not $2i+1$ or other variants. $\text{Absolute position} = \text{offset} + \text{local sequence index}$; comparing positions for relative logic requires using offsets consistently for both Q and K.

Common failure mode

Rotating the entire dh dimension instead of only $[0, R)$. Using local sequence index as position instead of $\text{offset} + \text{sequence}$. Applying different conventions to Q and K so relative position encoding is broken. Forgetting to copy the suffix $[R, dh)$ exactly.

16.18.1 Implementation points

Adjacent pair rotation formula and angle computation

For pair index i (0-indexed), angle $\theta_i = \text{position} / \text{base}^{(2*i/R)}$. The pair $(\text{feature}[2i], \text{feature}[2i+1])$ rotates via $x' = x \cos \theta - y \sin \theta$, $y' = x \sin \theta + y \cos \theta$. This formula is applied identically to Q and K.

Rotary dimension R as even, strict prefix

R must be positive, even, and satisfy $R \leq dh$. Features $[0, R)$ participate; features $[R, dh)$ are copied bit-for-bit without rotation. When $R = dh$, the entire feature dimension rotates in adjacent pairs.

Offset-based absolute position semantics

Absolute position for Q at local sequence q is $\text{queryPositionOffset} + q$. For K at local key k , it is $\text{keyPositionOffset} + k$. The relative position (difference) between Q and K positions encodes their separation, enabling relative position awareness.

Metal kernel pair iteration and feature layout

The Metal kernel stride (from: 0, to: rotaryDimension , by: 2) pairs features. For pair starting at feature pairStart , the angle uses pair index $\text{pairStart}/2$. RoPE is applied per-head, per-sequence; strides index $(\text{sequence} * \text{heads} + \text{head}) * dh + \text{pairStart}$ to access the pair start address.

16.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P015RoPEMetal.swift

- Line 4: extension P015RoPESolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalRoPEPipeline {

Sources/LeetLLMSolutions/P015RoPESolution.swift

- Line 4: public enum P015RoPESolution {
- Line 5: public static func apply(
- Line 16: func rotate(_ tensor: FloatTensor, offset: Int) throws -> FloatTensor {

Sources/LeetLLMSolutions/Metal/P015RoPE.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

16.18.3 Complete canonical listings

16.18.4 Sources/LeetLLMSolutions/P015RoPEMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P015RoPESolution {
5     public static func makeMetalPipeline() throws -> MetalRoPEPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P015RoPE", withExtension: "metal", subdirectory: "Metal")
9         else {
10             throw MetalNeuralOperatorError.kernelResourceMissing("P015RoPE.metal")
11         }
12         return try MetalRoPEPipeline(source: String(contentsOf: url, encoding: .utf8))
13     }
14 }
```

16.18.5 Sources/LeetLLMSolutions/P015RoPESolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P015RoPESolution {
5     public static func apply(
6         queries: FloatTensor,
7         keys: FloatTensor,
8         rotaryDimension: Int,
9         base: Float,
10        queryPositionOffset: Int,
11        keyPositionOffset: Int
12    ) throws -> RoPEResult {
13        try RoPEContract.validate(
14            queries: queries, keys: keys, rotaryDimension: rotaryDimension, base: base,
15            queryPositionOffset: queryPositionOffset, keyPositionOffset: keyPositionOffset)
16        func rotate(_ tensor: FloatTensor, offset: Int) throws -> FloatTensor {
17            var output = tensor.storage
18            let heads = tensor.shape[1]
19            let headDimension = tensor.shape[2]
20            for sequence in 0..

```

```

27     let cosine = cos(angle)
28     let sine = sin(angle)
29     let first = tensor.storage[start + pairStart]
30     let second = tensor.storage[start + pairStart + 1]
31     output[start + pairStart] = first * cosine - second * sine
32     output[start + pairStart + 1] = first * sine + second * cosine
33 }
34 }
35 }
36 return try FloatTensor(output, shape: tensor.shape)
37 }
38 return RoPEResult(
39   queries: try rotate(queries, offset: queryPositionOffset),
40   keys: try rotate(keys, offset: keyPositionOffset)
41 )
42 }
43 }

```

16.18.6 Sources/LeetLLMSolutions/Metal/P015RoPE.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void rope_adjacent_pairs(
5   device const float *input [[buffer(0)]],
6   device float *output [[buffer(1)]],
7   constant uint4 &shape [[buffer(2)]],
8   constant uint &positionOffset [[buffer(3)]],
9   constant float &ropeBase [[buffer(4)]],
10  uint index [[thread_position_in_grid]]
11 ) {
12   uint count = shape.x * shape.y * shape.w;
13   if (index >= count) {
14     return;
15   }
16   uint pair = index % shape.w;
17   uint head = (index / shape.w) % shape.y;
18   uint sequence = index / (shape.w * shape.y);
19   uint feature = pair * 2;
20   uint tensorIndex = (sequence * shape.y + head) * shape.z + feature;
21   float angle = float(positionOffset + sequence)
22     / pow(ropeBase, float(feature) / float(shape.w * 2));
23   float cosine = cos(angle);
24   float sine = sin(angle);
25   float first = input[tensorIndex];
26   float second = input[tensorIndex + 1];
27   output[tensorIndex] = first * cosine - second * sine;
28   output[tensorIndex + 1] = first * sine + second * cosine;
29 }

```

Causal Attention for One Head

17.1 Why this exists

This is the first complete decoder attention operator. It combines earlier dot products, stable softmax, and matrix-weighted values with one new rule: a query may not read a key from the future. The materialized implementation is deliberately readable and stores the score matrix. Problems 019 and 020 will optimize against this semantic oracle.

17.2 Learning outcomes

You can:

- derive scaled query-key scores and weighted values;
- place the causal mask before softmax;
- compute mask visibility from absolute positions;
- implement stable row softmax with future entries excluded;
- quantify the quadratic score intermediate; and
- execute a two-stage materialized Metal implementation.

17.3 Prerequisites

- Problem 001 for dot products.
- Problem 009 for stable softmax.
- Problems 014 and 015 for head-shaped, position-aware Q and K.

17.4 Vocabulary

- **Query position:** absolute position requesting context.
- **Key position:** absolute position represented by one K/V row.
- **Causal mask:** rule that permits a key only when `keyPosition <= queryPosition`.
- **Score matrix:** all scaled query-key dot products before softmax.
- **Attention weight:** normalized probability assigned to a visible value.
- **Materialized attention:** an implementation that writes the score matrix.

17.5 Math from first principles

For one head, $Q \in \mathbb{R}^{S_q \times d_h}$ and $K, V \in \mathbb{R}^{S_{kv} \times d_h}$. The raw score is

$$s_{qk} = \frac{Q_q \cdot K_k}{\sqrt{d_h}}.$$

Set $s_{qk} = -\infty$ when the absolute key position exceeds the query position. For each query row,

$$p_{qk} = \frac{e^{s_{qk} - m_q}}{\sum_j e^{s_{qj} - m_q}}, \quad m_q = \max_j s_{qj},$$

and

$$O_q = \sum_k p_{qk} V_k.$$

17.5.1 Worked numerical example

Let $d_h = 1$, $Q = [1, 1]$, $K = [1, 2]$, and $V = [3, 9]$. Query zero can see only key zero, so its output is exactly 3. Query one has scores $[1, 2]$; stable weights are approximately $[0.2689, 0.7311]$, giving $0.2689(3) + 0.7311(9) \approx 7.3866$. If query zero changes when $v[1]$ changes, the causal mask is wrong.

17.6 Shape, layout, and dtype contract

Problem 016 requires Q $[Sq, 1, dh]$ and K, V $[Skv, 1, dh]$, contiguous row-major Float32. The output is $[Sq, 1, dh]$. dh is positive; K and V lengths match. Offsets assign absolute positions: local query q is $queryOffset + q$, and local key k is $keyOffset + k$.

Every query must have at least one visible key. Non-finite inputs, wrong ranks, head counts other than one, mismatched dimensions, and no-visible-key rows are errors before computation or dispatch.

17.7 CPU reference path

1. Allocate $[Sq, Skv]$ scores initialized to negative infinity.
2. Fill visible entries with scaled dot products.
3. For each row, subtract its finite maximum and normalize exponentials.
4. Multiply probabilities by V and sum into $[Sq, 1, dh]$.

Keeping scores materialized is the point of this baseline.

17.8 Independent correctness method

The judge owns a Double materialized implementation. Cases cover three causal rows, large logits that overflow naive exponentiation, and decode with a longer KV context. It tests wrong query rank and missing visible keys. Tolerance is $3e-5 + 8e-5 \cdot \text{abs}(\text{expected})$.

```
1 swift run leetllm check 016 --cpu
2 swift run leetllm check 016 --metal
3 swift run leetllm check 016 --solution
```

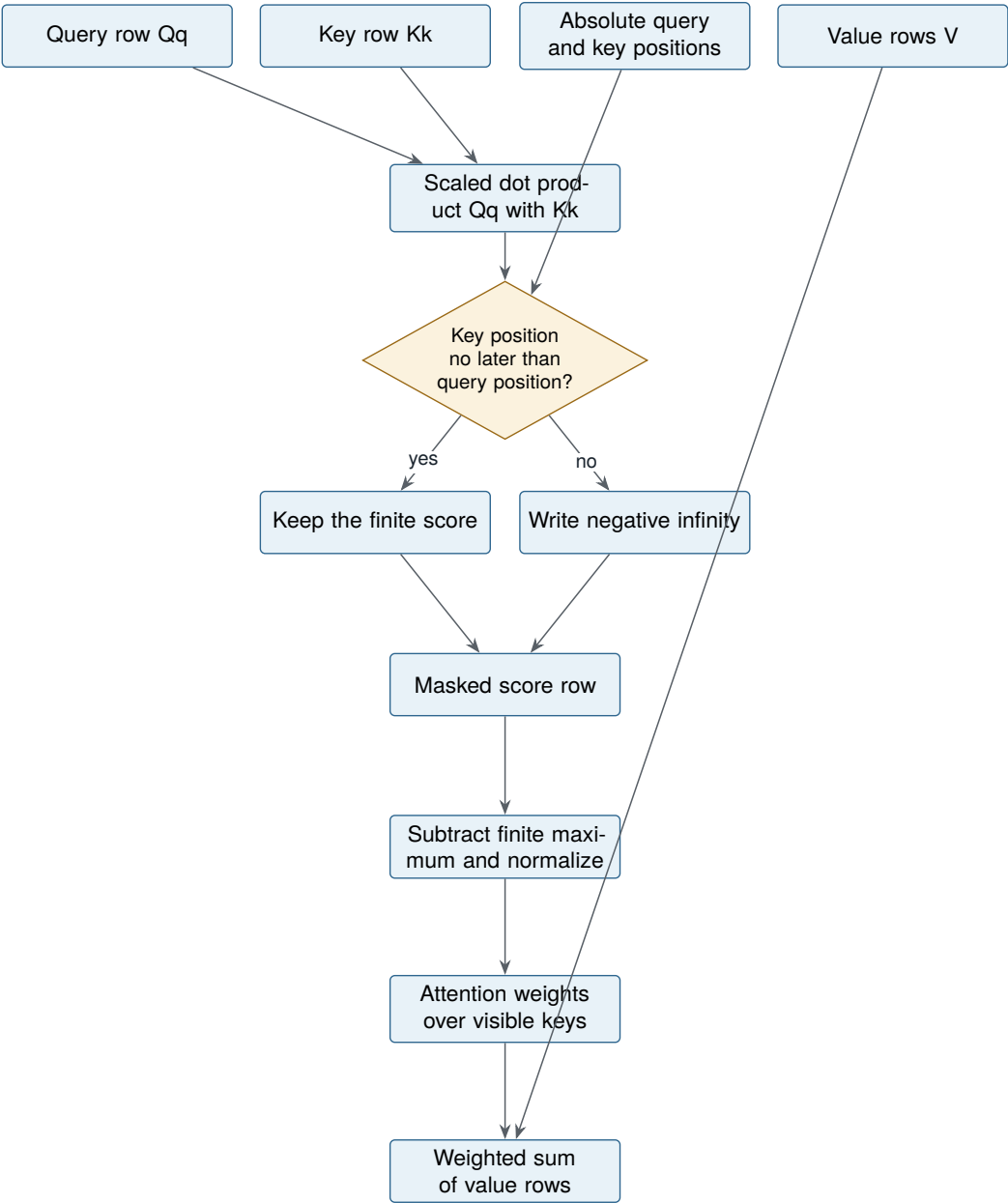


Figure 17.1: Causal mask before softmax.

17.9 Performance model

Score computation performs about $2S_q S_{kv} d_h$ FLOPs. The weighted-value stage performs another $2S_q S_{kv} d_h$ FLOPs. Softmax adds max, exponential, sum, and division work per visible score.

The score buffer occupies $4S_q S_{kv}$ bytes and is written then reread. For self-attention with length S , that is quadratic memory independent of d_h . Q, K, V, and output occupy linear storage. Causal masking reduces useful arithmetic to roughly half a square for full prefill, but this baseline still allocates the whole square.

17.10 Metal mapping

The first kernel dispatches $S_q \times S_{kv}$ threads. One thread writes one score or negative infinity. The second dispatch uses one thread per query row, performs stable softmax over the materialized row, then writes all output features.

Two compute encoders enforce score production before consumption. There is no threadgroup barrier because rows are processed serially inside the applying thread. This is correct and inspectable, not yet a throughput-optimized mapping.

See [P016CausalAttention.metal](#).

17.11 Implementation checkpoints

1. Validate one-head shapes and absolute offsets.
2. Compute one scaled dot product.
3. Write future scores as negative infinity.
4. Normalize one row stably.
5. Compute one weighted output feature.
6. Extend across query rows and d_h .
7. Match the materialized CPU oracle from Metal.

17.12 Controlled experiments

17.12.1 Sequence-length sweep

Sweep equal Q/KV lengths. Prediction: score storage grows quadratically and eventually dominates allocations and traffic.

17.12.2 Head-width sweep

Fix lengths and vary d_h . Prediction: dot and weighted-value work grows with d_h , while score-buffer bytes stay fixed.

17.12.3 Offset sweep

Hold local shapes fixed and change query offset. Prediction: later queries see more cached keys and produce different outputs without changing allocation.

17.13 Engine integration

This function is the correctness oracle for multi-head, grouped, online, tiled, and local attention. The output $[Sq, 1, dh]$ is one head's contribution before head concatenation and output projection.

17.14 Tradeoffs

- Materialization is easy to inspect but has quadratic intermediate memory.
- Negative infinity is a clear mask representation; bounded sentinels can leak probability.
- Serial row application is simple; parallel reductions require synchronization.
- Absolute offsets support cache use but add contract complexity.

17.15 Hints

- Scale scores before softmax by $1/\sqrt{dh}$.
- Mask before finding the row maximum.
- Never include a future value with probability zero “after” a softmax that already normalized it.
- Use absolute, not local, positions for decode visibility.

17.16 Completion checklist

- ☐ CPU and Metal pass the materialized oracle.
- ☐ The first row cannot observe future values.
- ☐ Large logits remain finite.
- ☐ Decode offsets use absolute positions.
- ☐ You can state the exact score-buffer size.
- ☐ You ran a length, width, or offset experiment with a prediction.

17.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

17.17.1 Sources/LeetLLMExercises/P016CausalAttentionExercise.swift

```
1 import LeetLLMCore
2
3 public enum P016CausalAttentionExercise {
4     public static func apply(
5         _ queries: FloatTensor, _ keys: FloatTensor, _ values: FloatTensor,
6         _ configuration: AttentionConfiguration
7     ) throws -> FloatTensor {
8         _ = try P016CausalAttentionContract.validate(
9             queries: queries, keys: keys, values: values, configuration: configuration)
10        return try FloatTensor(Array(repeating: 0, count: queries.elementCount), shape: queries.shape)
11    }
12 }
```

17.17.2 Sources/LeetLLMExercises/P016CausalAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P016CausalAttentionExercise {
5     public static func makeMetalPipeline() throws -> MetalMaterializedAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P016CausalAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P016CausalAttention.metal") }
10        return try MetalMaterializedAttentionPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }
```

17.17.3 Sources/LeetLLMExercises/Metal/P016CausalAttention.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void causal_attention_scores(device const float *queries [[buffer(0)]], device const float *keys [[buffer(1)]],
5     ↪ device float *scores [[buffer(2)]], constant uint4 &shape [[buffer(3)]], constant uint2 &offsets [[buffer(4)]], uint
6     ↪ index [[thread_position_in_grid]]) {
7     if (index < shape.x * shape.y) { scores[index] = 0.0f; }
8 }
9
10 kernel void causal_attention_apply(device const float *scores [[buffer(0)]], device const float *values [[buffer(1)]],
11     ↪ device float *output [[buffer(2)]], constant uint4 &shape [[buffer(3)]], uint query [[thread_position_in_grid]]) {
12     if (query >= shape.x) { return; }
13     for (uint feature = 0; feature < shape.z; ++feature) { output[query * shape.z + feature] = 0.0f; }
14 }
```


17.18 Worked solution

Solution strategy

Causal attention for one head computes scaled dot-product scores $[S_q, S_{kv}]$ between query and key, applies a causal mask (setting future scores to $-\infty$ before softmax), then computes numerically stable row-wise softmax and applies attention weights to values. The implementation materializes the score matrix, computes the finite maximum per row, rescales all exponentials, normalizes, and sums weighted values in a second pass.

Correctness invariant

Scores are initialized to $-\infty$ and filled only for keys at positions $\leq$ query position. Softmax subtracts the row maximum before exponentiating to ensure numerical stability. The sum of attention weights over visible keys equals 1. Output is a weighted average of value rows, corresponding exactly to the normalized softmax probabilities.

Common failure mode

Applying causal mask after softmax, allowing future logits to influence the normalization denominator. Computing softmax without subtracting the maximum, causing overflow when scores are large. Materializing and returning the score matrix instead of the weighted output. Omitting the causality check or using wrong position comparison ($<$ instead of $\leq$).

17.18.1 Implementation points

Causal mask and position semantics

Query at absolute position $\text{queryOffset} + \text{query}$; key at $\text{keyOffset} + \text{key}$. A key is visible only if $\text{keyOffset} + \text{key} \leq \text{queryOffset} + \text{query}$. Set $\text{score}[\text{query}, \text{key}] = -\infty$ for invisible keys before softmax. The causal predicate is applied per-element, not per-row.

Score computation and scaling

$\text{Score} = (Q[q] \cdot K[k]) / \sqrt{d_h}$. Compute all visible dot products, scale by reciprocal square root of head dimension. Initialize invisible entries to $-\infty$, preserving them through softmax.

Numerically stable softmax with finite maximum

For each row of scores, find the maximum among finite entries. Subtract this maximum before exponentiating: $\text{weight} = \exp(\text{score} - \text{maximum})$. Sum the weights (denominator), then divide each weight by the sum. This prevents overflow and underflow while preserving the normalized probability.

Two-pass implementation with materialized scores

Metal provides `causal_attention_scores` and `causal_attention_apply` kernels. First kernel fills $[S_q, S_{kv}]$ score buffer, applying the causal mask and scale. Second kernel reads the score ma-

trix, computes per-row softmax maximum and normalization, and produces [Sq,dh] output by weighted sum.

17.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P016CausalAttentionMetal.swift

- Line 4: extension P016CausalAttentionSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalMaterializedAttentionPipeline {

Sources/LeetLLMSolutions/P016CausalAttentionSolution.swift

- Line 4: public enum P016CausalAttentionSolution {
- Line 5: public static func apply(

Sources/LeetLLMSolutions/Metal/P016CausalAttention.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

17.18.3 Complete canonical listings

17.18.4 Sources/LeetLLMSolutions/P016CausalAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P016CausalAttentionSolution {
5     public static func makeMetalPipeline() throws -> MetalMaterializedAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P016CausalAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P016CausalAttention.metal") }
10        return try MetalMaterializedAttentionPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }
```

17.18.5 Sources/LeetLLMSolutions/P016CausalAttentionSolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P016CausalAttentionSolution {
5     public static func apply(
6         _ queries: FloatTensor, _ keys: FloatTensor, _ values: FloatTensor,
7         _ configuration: AttentionConfiguration
8     ) throws -> FloatTensor {
9         let input = try P016CausalAttentionContract.validate(
10             queries: queries, keys: keys, values: values, configuration: configuration)
11        var scores = Array(repeating: -Float.infinity, count: input.queryLength * input.keyValueLength)
12        let scale = 1 / sqrt(Float(configuration.headDimension))
13        for query in 0..

```

```

24     }
25 }
26 var output = Array(repeating: Float.zero, count: queries.elementCount)
27 for query in 0..(rowStart + input.keyValueLength)]
30     let maximum = row.max()!
31     let exponentials = row.map { exp($0 - maximum) }
32     let denominator = exponentials.reduce(0, +)
33     for feature in 0.. $configuration.headDimension$  {
34         var sum: Float = 0
35         for key in 0.. $input.keyValueLength$  where scores[rowStart + key].isFinite {
36             sum +=
37                 exponentials[key] / denominator
38                 * values.storage[input.keyValueOffset(sequence: key, head: 0, feature: feature)]
39         }
40         output[input.queryOffset(sequence: query, head: 0, feature: feature)] = sum
41     }
42 }
43 return try FloatTensor(output, shape: queries.shape)
44 }
45 }

```

17.18.6 Sources/LeetLLMSolutions/Metal/P016CausalAttention.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void causal_attention_scores(device const float *queries [[buffer(0)]], device const float *keys [[buffer(1)]],
   ↳ device float *scores [[buffer(2)]], constant uint4 &shape [[buffer(3)]], constant uint2 &offsets [[buffer(4)]], uint
   ↳ index [[thread_position_in_grid]]) {
5     if (index >= shape.x * shape.y) { return; }
6     uint query = index / shape.y;
7     uint key = index % shape.y;
8     if (offsets.y + key > offsets.x + query) { scores[index] = -INFINITY; return; }
9     float sum = 0.0f;
10    for (uint feature = 0; feature < shape.z; ++feature) { sum += queries[query * shape.z + feature] * keys[key * shape.z
   ↳ + feature]; }
11    scores[index] = sum * rsqrt(float(shape.z));
12 }
13
14 kernel void causal_attention_apply(device const float *scores [[buffer(0)]], device const float *values [[buffer(1)]],
   ↳ device float *output [[buffer(2)]], constant uint4 &shape [[buffer(3)]], uint query [[thread_position_in_grid]]) {
15    if (query >= shape.x) { return; }
16    uint rowStart = query * shape.y;
17    float maximum = -INFINITY;
18    for (uint key = 0; key < shape.y; ++key) { maximum = max(maximum, scores[rowStart + key]); }
19    float denominator = 0.0f;
20    for (uint key = 0; key < shape.y; ++key) { denominator += exp(scores[rowStart + key] - maximum); }
21    for (uint feature = 0; feature < shape.z; ++feature) {
22        float sum = 0.0f;
23        for (uint key = 0; key < shape.y; ++key) { sum += exp(scores[rowStart + key] - maximum) / denominator *
   ↳ values[key * shape.z + feature]; }
24        output[query * shape.z + feature] = sum;
25    }
26 }

```

Multi-Head Attention

18.1 Why this exists

A transformer uses several attention heads so different learned projections can form different context mixtures. Heads do not attend to one another. Each head runs the one-head equation independently, and their output features are laid next to each other for a later output projection.

18.2 Learning outcomes

You can:

- extend causal attention from one head to H independent heads;
- index token-major $[S, H, d_h]$ tensors correctly;
- explain concatenation as layout rather than summation;
- map (query, head) pairs to parallel Metal work;
- distinguish head parallelism from feature reduction; and
- detect implementations that accidentally reuse one head's K/V .

18.3 Prerequisites

- Problem 014 for head views.
- Problem 016 for the complete one-head equation and mask.
- Problem 006 for separating parallelism from total work.

18.4 Vocabulary

- **Head**: one independently projected $Q/K/V$ attention subspace.
- **Head concatenation**: interpreting $[S, H, d_h]$ as $[S, H * d_h]$ in head-major feature order.
- **Head parallelism**: evaluating separate (query, head) rows concurrently.
- **Output projection**: later matrix multiplication that mixes concatenated heads.
- **Head leakage**: reading K or V from the wrong head.

18.5 Math from first principles

For each head h ,

$$s_{qkh} = \frac{Q_{qh} \cdot K_{kh}}{\sqrt{d_h}}, \quad p_{qkh} = \text{softmax}_{k \leq q}(s_{qkh}),$$

$$O_{qh} = \sum_{k \leq q} p_{qkh} V_{kh}.$$

There is no reduction over h . Concatenation is

$$O_{q,hd_h+f}^{flat} = O_{q,h,f}.$$

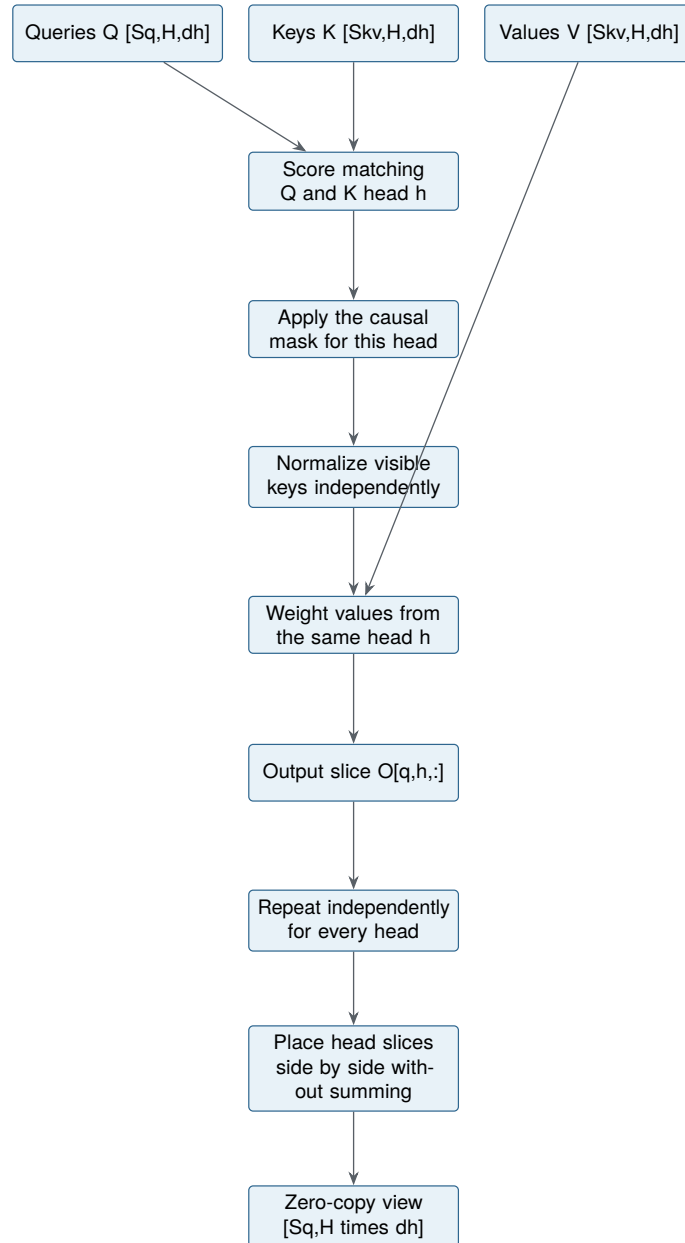


Figure 18.1: Independent head dataflow.

18.5.1 Worked numerical example

Use two heads with $dh=1$ at one query position. Suppose head zero attends to values $[2, 8]$ with weights $[0.75, 0.25]$, while head one attends to $[10, -2]$ with weights $[0.1, 0.9]$. Outputs are 3.5 and -0.8 . The concatenated row is $[3.5, -0.8]$, not 2.7 and not two copies of one head.

18.6 Shape, layout, and dtype contract

Q, K, and V are contiguous Float32 tensors $[S_q, H, d_h]$, $[S_{kv}, H, d_h]$, and $[S_{kv}, H, d_h]$. Problem 017 requires equal query and KV head counts. Output is $[S_q, H, d_h]$ with strides $[H*d_h, d_h, 1]$. Sequence lengths may differ when offsets describe cached K/V context.

The shared contract rejects wrong ranks, head counts, dimensions, K/V lengths, non-finite values, and query rows without visible keys.

18.7 CPU reference path

Nest loops as query, head, visible key, and feature. Each head builds and normalizes its own score list. Write output at $(\text{query} * H + \text{head}) * d_h + \text{feature}$. Keeping the head loop explicit exposes accidental cross-head indexing.

18.8 Independent correctness method

The Double judge generates deterministic but different values per head. It tests three heads, decode offsets, and an invalid KV shape. A correctly shaped zero output and a head-zero broadcast both fail. The judge uses the same absolute/relative tolerance as the materialized one-head oracle.

```
1 swift run leetllm check 017 --cpu
2 swift run leetllm check 017 --metal
3 swift run leetllm check 017 --solution
```

18.9 Performance model

Total score and value work is approximately $4HS_qS_{kv}d_h$ FLOPs. Q/K/V/output storage grows linearly with H ; a fully materialized score tensor would require $4HS_qS_{kv}$ bytes.

Independent heads expose H times more parallel rows. That does not reduce total work. For small S , head parallelism can provide enough work to occupy the GPU; for large S , recomputing scores per output feature in this simple kernel becomes expensive.

18.10 Metal mapping

The solution dispatches one work item per $(\text{query}, \text{head})$. Each work item:

1. scans visible keys for the maximum score;
2. scans again for the exponential denominator; and
3. writes each output feature using weighted values.

Heads execute independently on the GPU; no CPU attention function is called. The baseline recomputes scores while writing features instead of allocating an $H*S*S$ matrix. It uses no barriers or threadgroup memory. Problems 019 and 020 replace that repeated work with streaming state and tiling.

See [P017MultiHeadAttention.metal](#).

18.11 Implementation checkpoints

1. Pass Problem 016 logic for head zero.

2. Add the head stride to Q, K, and V indexing.
3. Keep one independent maximum and denominator per head.
4. Write rank-three output without summing heads.
5. Test head-specific values.
6. Dispatch all (query, head) pairs on Metal.

18.12 Controlled experiments

18.12.1 Head sweep

Fix total width $H*dh$ and vary its decomposition. Prediction: total scalar math stays similar, but the number and width of independent work items change.

18.12.2 Fixed dh sweep

Increase H at fixed dh . Prediction: work and storage grow linearly while parallel occupancy may improve until another resource becomes limiting.

18.12.3 Head isolation

Perturb only one head's V tensor. Prediction: only that output head changes before the output projection.

18.13 Engine integration

The output can be viewed as $[S_q, H*dh]$ and fed to the attention output projection. Problem 018 changes only how query heads select KV heads; it must preserve this query-head output layout.

18.14 Tradeoffs

- More heads provide more learned subspaces but increase Q and output state.
- One work item per head is simple; cooperative rows can use more GPU lanes.
- Recomputing scores saves storage but increases arithmetic.
- Head-major storage can help some kernels but would require layout conversion here.

18.15 Hints

- Compute all offsets with the relevant head count.
- Concatenation is already represented by contiguous $[H, dh]$ features.
- Reset maximum, denominator, and output accumulator for every head.
- Use the same causal absolute-position rule for all heads.

18.16 Completion checklist

- ☐ Every head matches its independent oracle output.
- ☐ Outputs remain $[S, H, dh]$, not head-summed.
- ☐ Metal executes head-parallel MSL.
- ☐ Decode offsets preserve causality.
- ☐ You can derive work and storage as functions of H .

- ☐ You ran a head-count or isolation experiment with a prediction.

18.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

18.17.1 Sources/LeetLLMExercises/P017MultiHeadAttentionExercise.swift

```
1 import LeetLLMCore
2
3 public enum P017MultiHeadAttentionExercise {
4     public static func apply(
5         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration
6     ) throws -> FloatTensor {
7         _ = try P017MultiHeadAttentionContract.validate(q, k, v, c)
8         return try FloatTensor(Array(repeating: 0, count: q.elementCount), shape: q.shape)
9     }
10 }
```

18.17.2 Sources/LeetLLMExercises/P017MultiHeadAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P017MultiHeadAttentionExercise {
5     public static func makeMetalPipeline() throws -> MetalParallelAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P017MultiHeadAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P017MultiHeadAttention.metal") }
10        return try MetalParallelAttentionPipeline(
11            source: String(contentsOf: url, encoding: .utf8), functionName: "multi_head_attention",
12            operation: "multi-head attention")
13    }
14 }
```

18.17.3 Sources/LeetLLMExercises/Metal/P017MultiHeadAttention.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3 kernel void multi_head_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device const
4     float *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4 &dims
5     [[buffer(5)]], uint index [[thread_position_in_grid]]) { if (index >= shape.x * shape.z) return; uint start = index
6     * dims.x; for (uint d = 0; d < dims.x; ++d) o[start + d] = 0.0f; }
```

18.18 Worked solution

Solution strategy

Multi-head attention extends one-head attention to H independent heads by adding an explicit head loop. Each (query, head) pair computes its own score vector, applies its own causal mask, normalizes with its own softmax maximum, and produces its own output slice. Heads are concatenated by layout, not summed: output $[S_q, H, d_h]$ places head slices side-by-side with strides $[H * d_h, d_h, 1]$. The implementation materializes per-head score rows and normalizes per head independently.

Correctness invariant

Heads do not attend to one another. For head h , only $Q[:, h, :]$, $K[:, h, :]$, $V[:, h, :]$ are used; no cross-head reads occur. Outputs are concatenated by position, not linearly combined. Query and KV head counts must be equal in Problem 017 (MHA constraint; Problem 018 relaxes this).

Common failure mode

Reusing one head's K/V for all queries or all query heads. Summing head outputs instead of concatenating them. Incorrect index computation that mixes head indices across different tensors. Using query position from wrong index calculation.

18.18.1 Implementation points

Independent per-head computation and score rows

For each query position and head, build a full visible-key score vector, find its maximum, compute softmax weights, and sum weighted values. Each head's softmax state (maximum, denominator) is private and must not be shared across heads.

Head loop nested explicitly in algorithm

The loop structure is: for query, for head, build visible-key list, compute scores, softmax, output. Keeping the head loop explicit in source code exposes accidental cross-head indexing bugs that would silently corrupt results.

Concatenation as layout, not reduction

Output $[S_q, H, d_h]$ with strides $[H * d_h, d_h, 1]$ places each head's output contiguously in memory but in head-major column order. Downstream processing (e.g., output projection) reads this as $[S_q, H * d_h]$ flat, treating each head's features as separate columns, not as separate tensors to be summed.

Metal kernel dispatch over (query, head) pairs

The Metal kernel indexes as $(\text{query} * H + \text{head})$, assigning one (query, head) pair per thread. Each thread computes score visibility, applies its own softmax loop, and writes output. Barriers and shared state are per-thread-group, not per-head; head-local state (maximum, denominator) resides in registers.

18.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P017MultiHeadAttentionMetal.swift

- Line 4: extension P017MultiHeadAttentionSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalParallelAttentionPipeline {

Sources/LeetLLMSolutions/P017MultiHeadAttentionSolution.swift

- Line 4: public enum P017MultiHeadAttentionSolution {
- Line 5: public static func apply(

Sources/LeetLLMSolutions/Metal/P017MultiHeadAttention.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

18.18.3 Complete canonical listings

18.18.4 Sources/LeetLLMSolutions/P017MultiHeadAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P017MultiHeadAttentionSolution {
5     public static func makeMetalPipeline() throws -> MetalParallelAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P017MultiHeadAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P017MultiHeadAttention.metal") }
10        return try MetalParallelAttentionPipeline(
11            source: String(contentsOf: url, encoding: .utf8), functionName: "multi_head_attention",
12            operation: "multi-head attention")
13    }
14 }
```

18.18.5 Sources/LeetLLMSolutions/P017MultiHeadAttentionSolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P017MultiHeadAttentionSolution {
5     public static func apply(
6         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration
7     ) throws -> FloatTensor {
8         let input = try P017MultiHeadAttentionContract.validate(q, k, v, c)
9         var output = Array(repeating: Float.zero, count: q.elementCount)
10        let scale = 1 / sqrt(Float(c.headDimension))
11        for query in 0..

```

```

28     let denominator = weights.reduce(0, +)
29     for feature in 0..

```

18.18.6 Sources/LeetLLMSolutions/Metal/P017MultiHeadAttention.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3 kernel void multi_head_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device const
  ↪ float *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4 &dims
  ↪ [[buffer(5)]], uint index [[thread_position_in_grid]]) {
4     if (index >= shape.x * shape.z) return; uint query = index / shape.z; uint head = index % shape.z; uint queryPosition
  ↪ = dims.z + query; float maximum = -INFINITY;
5     for (uint key = 0; key < shape.y; ++key) { if (dims.w + key > queryPosition) continue; float score = 0.0f; for (uint
  ↪ d = 0; d < dims.x; ++d) score += q[(query * shape.z + head) * dims.x + d] * k[(key * shape.w + head) * dims.x +
  ↪ d]; maximum = max(maximum, score * rsqrt(float(dims.x))); }
6     float denominator = 0.0f; for (uint key = 0; key < shape.y; ++key) { if (dims.w + key > queryPosition) continue;
  ↪ float score = 0.0f; for (uint d = 0; d < dims.x; ++d) score += q[(query * shape.z + head) * dims.x + d] * k[(key
  ↪ * shape.w + head) * dims.x + d]; denominator += exp(score * rsqrt(float(dims.x)) - maximum); }
7     for (uint d = 0; d < dims.x; ++d) { float sum = 0.0f; for (uint key = 0; key < shape.y; ++key) { if (dims.w + key >
  ↪ queryPosition) continue; float score = 0.0f; for (uint f = 0; f < dims.x; ++f) score += q[(query * shape.z +
  ↪ head) * dims.x + f] * k[(key * shape.w + head) * dims.x + f]; sum += exp(score * rsqrt(float(dims.x)) - maximum)
  ↪ / denominator * v[(key * shape.w + head) * dims.x + d]; } o[(query * shape.z + head) * dims.x + d] = sum; }
8 }

```

MHA, MQA, and GQA

19.1 Why this exists

The number of key/value heads is a model architecture choice, not a kernel detail. Multi-head attention (MHA) uses one KV head per query head. Multi-query attention (MQA) shares one KV head across all query heads. Grouped-query attention (GQA) lies between them. Fewer KV heads reduce projection work, cache size, and decode traffic while preserving the number of query outputs.

19.2 Learning outcomes

You can:

- represent H_q and H_{kv} independently;
- enforce $H_q \% H_{kv} == 0$;
- implement $kvHead = queryHead / groupSize$ exactly;
- identify MHA and MQA as endpoints of one mapping;
- derive K/V cache bytes; and
- preserve a checkpoint's architecture semantics on CPU and Metal.

19.3 Prerequisites

- Problem 014 for separate Q and KV projection widths.
- Problem 017 for query-head outputs.
- Problem 002 for head-dependent row-major offsets.

19.4 Vocabulary

- **MHA**: $H_q == H_{kv}$; each query head owns a KV head.
- **MQA**: $H_{kv} == 1$; all query heads share one KV head.
- **GQA**: $1 < H_{kv} < H_q$; contiguous groups share KV heads.
- **Group size**: H_q/H_{kv} query heads per KV head.
- **KV cache**: stored keys and values from previous positions.
- **Architecture semantics**: dimensions and mappings fixed by trained weights.

19.5 Math and mapping

Let

$$g = \frac{H_q}{H_{kv}}.$$

For query head h_q ,

$$h_{kv} = \left\lfloor \frac{h_q}{g} \right\rfloor.$$

The attention equation is then

$$s_{qkh_q} = \frac{Q_{q,h_q} \cdot K_{k,h_{kv}}}{\sqrt{d_h}}, \quad O_{q,h_q} = \sum_k p_{qkh_q} V_{k,h_{kv}}.$$

19.5.1 Worked mapping example

For $H_q=8, H_{kv}=2, g=4$:

Query heads	KV head
0, 1, 2, 3	0
4, 5, 6, 7	1

For MHA, $g=1$, so the mapping is identity. For MQA, $g=H_q$, so every query head maps to zero. Modulo mapping is wrong for this contiguous-group convention.

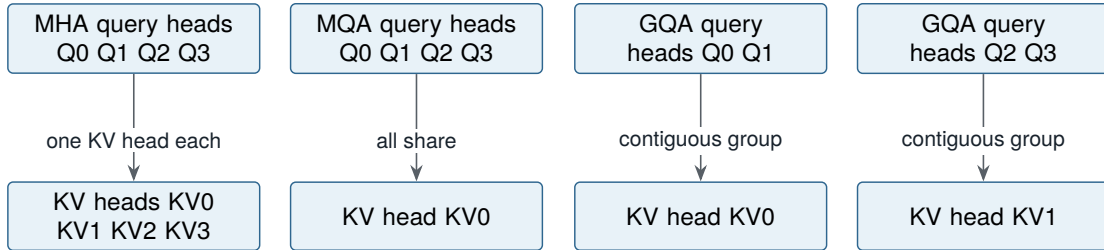


Figure 19.1: Query to kv ownership.

19.6 Shape, layout, and dtype contract

Q is $[S_q, H_q, d_h]$; K and V are $[S_{kv}, H_{kv}, d_h]$; output is $[S_q, H_q, d_h]$. All are contiguous row-major Float32. Counts and d_h are positive, K/V lengths match, H_q is divisible by H_{kv} , and all values are finite. Batch size remains one.

Offsets and causal visibility have the same absolute-position semantics as Problem 016. Reducing H_{kv} never changes the number of output query heads.

19.7 CPU reference path

Loop over $(query, queryHead)$, compute $kvHead$ once, and use it for every K/V read in that row. Softmax state remains private to the query head even when several query heads share K/V data.

19.8 Independent correctness method

The Double judge covers GQA 4:2, MQA 4:1, and MHA 2:2, using distinct KV values so wrong mappings fail. `AttentionConfiguration` independently rejects non-divisible counts. `KVCacheByteModel` tests exact cache-size ratios.

```
1 swift run leetllm check 018 --cpu
2 swift run leetllm check 018 --metal
3 swift run leetllm check 018 --solution
```

19.9 Performance and cache model

For sequence length S , Float32 K and V cache bytes are

$$B_{KV} = 2SH_{kv}d_h \cdot 4.$$

With another element format, replace 4 by its bytes per element. Relative to MHA, ideal cache-size reduction is $H_q/H_{kv} = g$. K/V projection output and decode cache-read traffic shrink by the same head-count factor. Q projection and output size still depend on H_q .

Attention arithmetic for each query head remains: fewer KV heads enable reuse but do not automatically remove query-head dot products.

19.10 Metal mapping

One GPU work item owns $(\text{query}, \text{queryHead})$. It computes $\text{kvHead} = \text{queryHead} / \text{groupSize}$, then performs stable causal attention with that K/V slice. This is actual MSL execution and preserves contiguous grouping.

The simple kernel rereads shared K/V for each query head. Hardware caches may capture some reuse; a cooperative grouped kernel could make it explicit. There are no barriers in the current mapping.

See [P018GroupedQueryAttention.metal](#).

19.11 Implementation checkpoints

1. Validate divisibility and rank-three shapes.
2. Print or test the query-to-KV mapping table.
3. Pass MHA with `groupSize=1`.
4. Pass MQA with one KV head.
5. Pass an intermediate GQA ratio.
6. Calculate cache bytes for all three.
7. Match CPU and Metal outputs.

19.12 Controlled experiments

19.12.1 KV-head sweep

Fix H_q, S, d_h and sweep divisors for H_{kv} . Prediction: cache bytes and K/V projection size shrink linearly with H_{kv} ; Q/output storage does not.

19.12.2 Shared-KV reuse

Compare MHA and GQA with equal query-head work. Prediction: GQA can improve effective K/V cache locality, although the baseline still executes one row per query head.

19.12.3 Architecture mismatch

Deliberately replace division mapping with modulo in a local experiment. Prediction: shapes remain valid but outputs differ, demonstrating why this is model correctness rather than an interchangeable optimization.

19.13 Engine integration

KV-cache allocation in Problems 022 onward must use H_{kv} . Checkpoint loading must verify Q projection width against $H_q * d_h$ and K/V widths against $H_{kv} * d_h$. The decoder output remains one vector per query head.

19.14 Tradeoffs

- Fewer KV heads reduce memory and traffic but can affect model quality when training choices differ.
- MQA maximizes sharing; GQA retains more KV diversity.
- Per-query-head work is simple; grouped cooperative work can expose reuse.
- The mapping must match trained weights exactly.

19.15 Hints

- Compute `groupSize` once from validated counts.
- Use division, not modulo, for contiguous groups.
- Index Q with H_q and K/V with H_{kv} .
- Include both K and V in cache-byte formulas.

19.16 Completion checklist

- ☐ MHA, MQA, and GQA all pass the judge.
- ☐ Non-divisible head counts are rejected.
- ☐ The exact division mapping is tested.
- ☐ Metal uses H_{kv} for K/V offsets.
- ☐ You can derive cache bytes for any dtype.
- ☐ You ran a KV-head or mapping experiment with a written prediction.

19.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

19.17.1 Sources/LeetLLMExercises/P018GroupedQueryAttentionExercise.swift

```
1 import LeetLLMCore
2
3 public enum P018GroupedQueryAttentionExercise {
4     public static func apply(
5         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration
6     ) throws -> FloatTensor {
7         _ = try P018GroupedQueryAttentionContract.validate(q, k, v, c)
8         return try FloatTensor(Array(repeating: 0, count: q.elementCount), shape: q.shape)
9     }
10 }
```

19.17.2 Sources/LeetLLMExercises/P018GroupedQueryAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P018GroupedQueryAttentionExercise {
5     public static func makeMetalPipeline() throws -> MetalParallelAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P018GroupedQueryAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P018GroupedQueryAttention.metal") }
10        return try MetalParallelAttentionPipeline(
11            source: String(contentsOf: url, encoding: .utf8), functionName: "grouped_query_attention",
12            operation: "grouped-query attention")
13    }
14 }
```

19.17.3 Sources/LeetLLMExercises/Metal/P018GroupedQueryAttention.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3 kernel void grouped_query_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device
4     ↪ const float *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4
5     ↪ &dims [[buffer(5)]], uint index [[thread_position_in_grid]]) { if (index >= shape.x * shape.z) return; for (uint d =
6     ↪ 0; d < dims.x; ++d) o[index * dims.x + d] = 0.0f; }
```

19.18 Worked solution

Solution strategy

Grouped query attention maps query heads to KV heads via contiguous grouping: $\text{kvHead} = \text{queryHead} / \text{groupSize}$, where $\text{groupSize} = \text{Hq} / \text{Hkv}$. Hq must be divisible by Hkv . Each query head attends via its assigned KV head; multiple query heads can share one KV head. The attention equation is identical to Problem 017, but K/V lookups use the mapped kvHead instead of the original queryHead .

Correctness invariant

Hq divisible by Hkv ; $\text{groupSize} = \text{Hq} / \text{Hkv}$ computed once. Query heads $[0, g)$ map to KV head 0, $[g, 2g)$ map to KV head 1, etc. MHA ($\text{Hq} == \text{Hkv}$, $g=1$) is the identity mapping; MQA ($\text{Hkv} == 1$, $g=\text{Hq}$) maps all query heads to KV head 0. The mapping must be contiguous by group, not modulo or other permutations.

Common failure mode

Using modulo mapping ($\text{queryHead} \% \text{Hkv}$) instead of integer division $\text{floor}(\text{queryHead} / \text{groupSize})$. Failing to validate Hq divisibility by Hkv . Confusing MQA and MHA in code, or implementing only one variant. Using wrong KV head indices due to off-by-one errors.

19.18.1 Implementation points

Group size and contiguous KV head mapping

Group size = Hq / Hkv . Query heads are partitioned into Hkv contiguous groups of size g . Query heads $[0, g) \rightarrow \text{KV head } 0$, $[g, 2g) \rightarrow \text{KV head } 1$, and so on. This contiguity is essential for KV cache layout efficiency; modulo mapping breaks it.

MHA and MQA as mapping endpoints

MHA: $\text{Hq} == \text{Hkv}$, $g=1$, identity mapping (each query head has its own KV head). MQA: $\text{Hkv} == 1$, $g=\text{Hq}$, all query heads map to KV head 0. GQA spans the middle: $1 < \text{Hkv} < \text{Hq}$.

KV cache byte accounting per head configuration

For one layer, storing both Float32 keys and values requires $2 * \text{sequenceLength} * \text{Hkv} * \text{dh} * 4$ bytes. Equivalently, each KV head contributes $2 * \text{sequenceLength} * \text{dh} * 4$ bytes. Query head count does not appear in this storage formula, so reducing Hkv reduces the K/V cache footprint when sequence length and head dimension stay fixed.

Metal kernel KV head mapping and grouped attention computation

Metal kernel receives $\text{dims.y} = \text{groupSize}$. For queryHead , $\text{kvHead} = \text{queryHead} / \text{dims.y}$. K/V are indexed by $(\text{key} * \text{Hkv} + \text{kvHead}) * \text{dh} + \text{feature}$. This mapping is applied once per (query, queryHead) pair and held constant for all K/V reads in that row.

19.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P018GroupedQueryAttentionMetal.swift

- Line 4: extension P018GroupedQueryAttentionSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalParallelAttentionPipeline {

Sources/LeetLLMSolutions/P018GroupedQueryAttentionSolution.swift

- Line 4: public enum P018GroupedQueryAttentionSolution {
- Line 5: public static func apply(

Sources/LeetLLMSolutions/Metal/P018GroupedQueryAttention.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

19.18.3 Complete canonical listings

19.18.4 Sources/LeetLLMSolutions/P018GroupedQueryAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P018GroupedQueryAttentionSolution {
5     public static func makeMetalPipeline() throws -> MetalParallelAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P018GroupedQueryAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P018GroupedQueryAttention.metal") }
10        return try MetalParallelAttentionPipeline(
11            source: String(contentsOf: url, encoding: .utf8), functionName: "grouped_query_attention",
12            operation: "grouped-query attention")
13    }
14 }
```

19.18.5 Sources/LeetLLMSolutions/P018GroupedQueryAttentionSolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P018GroupedQueryAttentionSolution {
5     public static func apply(
6         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration
7     ) throws -> FloatTensor {
8         let input = try P018GroupedQueryAttentionContract.validate(q, k, v, c)
9         var output = Array(repeating: Float.zero, count: q.elementCount)
10        let scale = 1 / sqrt(Float(c.headDimension))
11        for query in 0..

```

```

28     let weights = scores.map { exp($0 - maximum) }
29     let denominator = weights.reduce(0, +)
30     for feature in 0..

```

19.18.6 Sources/LeetLLMSolutions/Metal/P018GroupedQueryAttention.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3 kernel void grouped_query_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device
  ↪ const float *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4
  ↪ &dims [[buffer(5)]], uint index [[thread_position_in_grid]]) {
4     if (index >= shape.x * shape.z) return; uint query = index / shape.z; uint queryHead = index % shape.z; uint kvHead
  ↪ = queryHead / dims.y; uint queryPosition = dims.z + query; float maximum = -INFINITY;
5     for (uint key = 0; key < shape.y; ++key) { if (dims.w + key > queryPosition) continue; float score = 0.0f; for (uint
  ↪ d = 0; d < dims.x; ++d) score += q[(query * shape.z + queryHead) * dims.x + d] * k[(key * shape.w + kvHead) *
  ↪ dims.x + d]; maximum = max(maximum, score * rsqrt(float(dims.x))); }
6     float denominator = 0.0f; for (uint key = 0; key < shape.y; ++key) { if (dims.w + key > queryPosition) continue;
  ↪ float score = 0.0f; for (uint d = 0; d < dims.x; ++d) score += q[(query * shape.z + queryHead) * dims.x + d] *
  ↪ k[(key * shape.w + kvHead) * dims.x + d]; denominator += exp(score * rsqrt(float(dims.x)) - maximum); }
7     for (uint d = 0; d < dims.x; ++d) { float sum = 0.0f; for (uint key = 0; key < shape.y; ++key) { if (dims.w + key >
  ↪ queryPosition) continue; float score = 0.0f; for (uint f = 0; f < dims.x; ++f) score += q[(query * shape.z +
  ↪ queryHead) * dims.x + f] * k[(key * shape.w + kvHead) * dims.x + f]; sum += exp(score * rsqrt(float(dims.x)) -
  ↪ maximum) / denominator * v[(key * shape.w + kvHead) * dims.x + d]; } o[(query * shape.z + queryHead) * dims.x +
  ↪ d] = sum; }
8 }

```

Online Softmax Attention

20.1 Why this exists

Problem 016 needs an entire score row before stable softmax can normalize it. Online softmax carries enough state to incorporate one score at a time, including scores larger than every previous score. Attention can therefore update its weighted value output while streaming K and V, without storing a score row.

20.2 Learning outcomes

You can:

- derive the running maximum, normalizer, and output recurrence;
- explain why old terms must be rescaled when the maximum changes;
- compute stable causal attention without score storage;
- compare streaming output against the materialized oracle;
- distinguish lower memory from lower arithmetic; and
- implement a real Metal streaming kernel with register/private state.

20.3 Prerequisites

- Problem 009 for max-shifted softmax.
- Problem 016 for the materialized attention oracle.
- Problem 018 for query-to-KV head mapping.

20.4 Vocabulary

- **Online recurrence:** state updated from one input at a time.
- **Running maximum m :** largest score processed so far.
- **Running normalizer l :** sum of exponentials expressed relative to m .
- **Unnormalized output o :** weighted value sum expressed relative to m .
- **Rescale factor α :** correction applied to old state when m increases.
- **Streaming:** consuming keys and values without retaining all scores.

20.5 Math from first principles

Assume current state (m, l, o) represents processed scores. For new score s and value vector v , define

$$m' = \max(m, s), \quad \alpha = e^{m-m'}, \quad \beta = e^{s-m'}.$$

For the empty state, treat $m = -\infty$ and $\alpha = 0$. Update

$$l' = \alpha l + \beta,$$

$$o' = \alpha o + \beta v.$$

After all visible keys, normalized output is o/l . The rescale is required because old terms were represented relative to the old maximum.

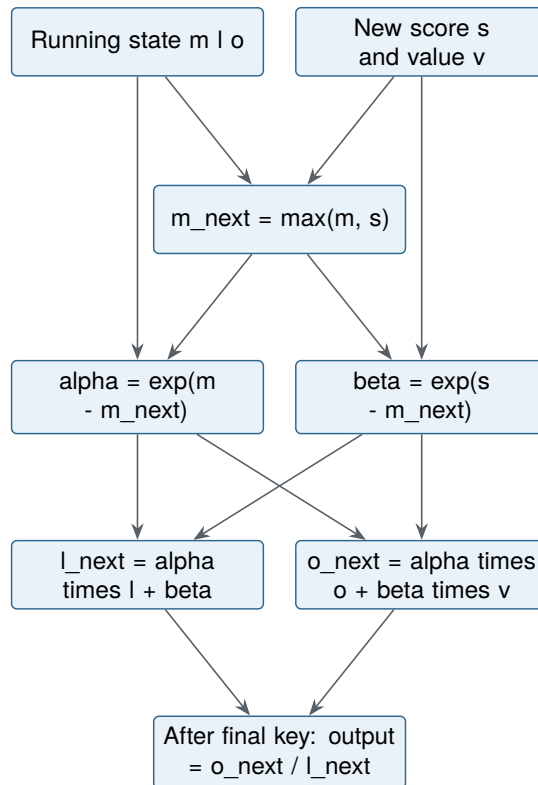


Figure 20.1: Online softmax recurrence.

20.5.1 Worked numerical example

Process scores $[1, 3]$ with scalar values $[2, 10]$.

First score: $m=1$, $l=1$, $o=2$. For score 3, new maximum is 3, $\alpha = e^{-2} \approx 0.1353$, and $\beta = 1$. Thus

$$l' = 0.1353 + 1 = 1.1353,$$

$$o' = 0.1353(2) + 10 = 10.2707.$$

The output is $10.2707/1.1353 \approx 9.0464$, equal to materialized softmax. Omitting alpha gives the wrong answer.

20.6 Shape, layout, and dtype contract

The shared batch-one contract accepts $Q [S_q, H_q, d_h]$, $K/V [S_{kv}, H_{kv}, d_h]$, and output $[S_q, H_q, d_h]$, contiguous Float32. Head counts obey GQA divisibility. Positions and causality use absolute offsets.

The canonical CPU path supports any positive d_h . The Metal kernel is pedagogically bounded to $d_h \leq 128$, because its output accumulator is a fixed private array. Invalid shapes, non-finite inputs, no-visible-key rows, and larger Metal head dimensions are explicit errors.

20.7 CPU reference path

For each $(query, queryHead)$, initialize $m = -\infty$, $\iota = 0$, and a zero length- d_h accumulator. Visit visible keys once. Compute one score, update all three state components, then divide the accumulator by ι . No score array is allocated.

20.8 Independent correctness method

The judge compares to Problem 016's Double materialized oracle, not another online implementation. Fixtures include five keys, 33 keys, large logits, and decode offsets. It checks a mismatched value shape and rejects a last-value-only algorithm. Tolerance is $5e-5 + 1e-4 * \text{abs}(\text{expected})$.

```
1 swift run leetllm check 019 --cpu
2 swift run leetllm check 019 --metal
3 swift run leetllm check 019 --solution
```

20.9 Performance model

Online attention retains the same leading arithmetic as fused attention: roughly $4H_q S_q S_{kv} d_h$ score/value FLOPs plus exponentials. It removes the $4H_q S_q S_{kv}$ -byte score intermediate and its write/read traffic.

Per active row, state is one maximum, one normalizer, and d_h output values: $O(d_h)$ rather than $O(S_{kv})$ memory. K and V are still streamed for every query row, so this recurrence alone does not guarantee cache reuse or high GPU occupancy.

20.10 Metal mapping

One work item owns $(query, queryHead)$. It stores m , ι , and a private `float[128]` accumulator, streams visible K/V rows, and writes one output head. There is no score buffer in the host pipeline and no CPU fallback inside Metal checks.

The kernel uses no barriers because all recurrence state belongs to one thread. That choice is easy to verify but underuses SIMD lanes. Problem 020 distributes score-tile and output work across a threadgroup while preserving the recurrence.

See [P019OnlineAttention.metal](#).

20.11 Implementation checkpoints

1. Reproduce materialized output for two scalar scores.
2. Handle the empty-state `alpha=0` case.
3. Rescale both ι and every component of o .

4. Add causal visibility and absolute offsets.
5. Extend to multiple heads and GQA mapping.
6. Remove every score-row allocation.
7. Pass large-logit and 33-key CPU/Metal cases.

20.12 Controlled experiments

20.12.1 Increasing-score sequence

Feed monotonically increasing scores. Prediction: the maximum changes at every step, so omitting old-state rescaling produces a growing error.

20.12.2 Length sweep

Compare materialized and online paths as skv grows. Prediction: outputs remain within tolerance; online intermediate memory stays constant per row.

20.12.3 Head-width sweep

Increase dh through 128. Prediction: private accumulator pressure grows, and the Metal path rejects 129 rather than silently spilling beyond its contract.

20.13 Engine integration

Online state is the numerical foundation for tiled attention. During decode it also provides a natural one-query streaming pass over the KV cache. The output contract remains identical to materialized attention, allowing direct parity checks and substitution.

20.14 Tradeoffs

- Online recurrence removes score storage but adds state-update arithmetic.
- A single-thread row is transparent but not a high-throughput GPU schedule.
- Fixed private arrays make constraints explicit but limit dh .
- Materialized scores remain useful for debugging and visualization.

20.15 Hints

- Rescale the old output accumulator before adding the new value.
- Keep o unnormalized until the end.
- Compute the score scale exactly as Problem 016.
- Search host code for an $Sq*Skv$ allocation; there should be none.

20.16 Completion checklist

- ☐ CPU and Metal match the materialized oracle.
- ☐ Difficult logits remain finite.
- ☐ No score row is allocated.
- ☐ Both l and o are rescaled when m changes.

- ☐ The Metal $dh \leq 128$ limit is understood.
- ☐ You ran a score, length, or width experiment with a prediction.

20.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

20.17.1 Sources/LeetLLMExercises/P0190onlineAttentionExercise.swift

```
1 import LeetLLMCore
2
3 public enum P0190onlineAttentionExercise {
4     public static func apply(
5         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration
6     ) throws -> FloatTensor {
7         let input = try AttentionInput(queries: q, keys: k, values: v, configuration: c)
8         try validateVisibleKeys(input)
9         return try FloatTensor(Array(repeating: 0, count: q.elementCount), shape: q.shape)
10    }
11 }
```

20.17.2 Sources/LeetLLMExercises/P0190onlineAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P0190onlineAttentionExercise {
5     public static func makeMetalPipeline() throws -> MetalStreamingAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P0190onlineAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P0190onlineAttention.metal") }
10        return try MetalStreamingAttentionPipeline(
11            source: String(contentsOf: url, encoding: .utf8), functionName: "online_attention",
12            operation: "online_attention")
13    }
14 }
```

20.17.3 Sources/LeetLLMExercises/Metal/P0190onlineAttention.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3 kernel void online_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device const float
  ↳ *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4 &dims
  ↳ [[buffer(5)]], constant uint &window [[buffer(6)]], uint index [[thread_position_in_grid]] { if (index >= shape.x *
  ↳ shape.z) return; for (uint d = 0; d < dims.x; ++d) o[index * dims.x + d] = 0.0f; }
```

20.18 Worked solution

Solution strategy

Online softmax eliminates score matrix materialization by maintaining running state (m, l, o) per head: m is the maximum score processed so far, l is the normalized sum of exponentials relative to m , and o is the weighted output sum also relative to m . For each new key score s and value v , update $m' = \max(m, s)$, then rescale old state by $\alpha = \exp(m - m')$, add new contribution $\beta = \exp(s - m')$: $l' = \alpha * l + \beta$, $o' = \alpha * o + \beta * v$. After all visible keys, output = o / l . No score buffer exists; state resides in registers or threadgroup memory.

Correctness invariant

When m increases, all prior terms must be rescaled by $\exp(\text{old_}m - \text{new_}m)$. If m was $-\infty$ (empty state), $\alpha = 0$ (old state contributes nothing). Denominator l and accumulator o scale by the same factor α . Final output equals true softmax within floating-point rounding of materialized softmax.

Common failure mode

Forgetting to rescale old accumulator when maximum increases. Applying α multiplicatively to l and denominator but additively to o , or vice versa. Treating $m = -\infty$ as a normal maximum instead of a sentinel (must set $\alpha = 0$). Computing β incorrectly or failing to initialize state.

20.18.1 Implementation points

Running state (m, l, o) and the rescale factor α

Maximum m starts at $-\infty$. For each new score s , $m' = \max(m, s)$. Rescale factor $\alpha = \exp(m - m')$. If m was $-\infty$, $\alpha = 0$ (old state is discarded). If m is finite and $s > m$, then $m' > m$, so $\alpha < 1$ (old state decays). $\alpha = 1$ only when $s \leq m$ (no update).

Synchronized updates to l and o under rescaling

Both denominator and accumulator must scale by α when m changes. $l' = \alpha * l + \beta$, $o' = \alpha * o + \beta * v$. This preserves the invariant that l and o remain consistent with the current m .

Streaming without score buffer

Keys and values are processed sequentially. No $[S_q, S_{kv}]$ score matrix is allocated; state (m, l, o) is $O(d_h)$ registers per (query, head). This reduces memory bandwidth and enables fusing multiple attention components in a single kernel.

Register-resident state and Metal private memory

In Metal, register-resident accumulators (floats for m, l ; array of floats for o) are declared in kernel scope. For $d_h \leq 128$, a private array accumulator[128] is standard. State persists across loop iterations without threadgroup memory pressure.

20.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P0190OnlineAttentionMetal.swift

- Line 4: extension P0190OnlineAttentionSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalStreamingAttentionPipeline {

Sources/LeetLLMSolutions/P0190OnlineAttentionSolution.swift

- Line 4: public enum P0190OnlineAttentionSolution {
- Line 5: public static func apply(

Sources/LeetLLMSolutions/Metal/P0190OnlineAttention.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

20.18.3 Complete canonical listings

20.18.4 Sources/LeetLLMSolutions/P0190OnlineAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P0190OnlineAttentionSolution {
5     public static func makeMetalPipeline() throws -> MetalStreamingAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P0190OnlineAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P0190OnlineAttention.metal") }
10        return try MetalStreamingAttentionPipeline(
11            source: String(contentsOf: url, encoding: .utf8), functionName: "online_attention",
12            operation: "online attention")
13    }
14 }
```

20.18.5 Sources/LeetLLMSolutions/P0190OnlineAttentionSolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P0190OnlineAttentionSolution {
5     public static func apply(
6         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration
7     ) throws -> FloatTensor {
8         let input = try AttentionInput(queries: q, keys: k, values: v, configuration: c)
9         try validateVisibleKeys(input)
10        var output = Array(repeating: Float.zero, count: q.elementCount)
11        let scale = 1 / sqrt(Float(c.headDimension))
12        for query in 0..

```

```

28     let previousScale = maximum.isFinite ? exp(maximum - newMaximum) : 0
29     let weight = exp(score - newMaximum)
30     denominator = denominator * previousScale + weight
31     for feature in 0..

```

20.18.6 Sources/LeetLLMSolutions/Metal/P0190OnlineAttention.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3 constant uint maxHeadDimension = 128;
4 kernel void online_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device const float
  ↪ *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4 &dims
  ↪ [[buffer(5)]], constant uint &window [[buffer(6)]], uint index [[thread_position_in_grid]]) {
5     if (index >= shape.x * shape.z) return; uint query = index / shape.z; uint qHead = index % shape.z; uint kvHead =
  ↪ qHead / dims.y; uint queryPosition = dims.z + query; float maximum = -INFINITY; float denominator = 0.0f; float
  ↪ accumulator[maxHeadDimension]; for (uint d = 0; d < dims.x; ++d) accumulator[d] = 0.0f;
6     for (uint key = 0; key < shape.y; ++key) { if (dims.w + key > queryPosition) continue; float score = 0.0f; for (uint
  ↪ d = 0; d < dims.x; ++d) score += q[(query * shape.z + qHead) * dims.x + d] * k[(key * shape.w + kvHead) * dims.x
  ↪ + d]; score *= rsqrt(float(dims.x)); float newMaximum = max(maximum, score); float alpha = isfinite(maximum) ?
  ↪ exp(maximum - newMaximum) : 0.0f; float beta = exp(score - newMaximum); denominator = denominator * alpha + beta;
  ↪ for (uint d = 0; d < dims.x; ++d) accumulator[d] = accumulator[d] * alpha + beta * v[(key * shape.w + kvHead) *
  ↪ dims.x + d]; maximum = newMaximum; }
7     for (uint d = 0; d < dims.x; ++d) o[(query * shape.z + qHead) * dims.x + d] = accumulator[d] / denominator;
8 }

```

Tiled Fused Attention

21.1 Why this exists

Online softmax removes score storage, but a one-thread row leaves most GPU lanes idle. This problem maps the same recurrence to a threadgroup: threads compute a tile of key scores, one lane updates tile statistics, and threads cooperatively update the output vector in on-chip memory. No $S \times S$ score buffer exists.

This is a Flash-style attention lesson because score computation, online normalization, and value accumulation are fused around on-chip tiles. It does not dispatch separate score and softmax kernels.

21.2 Learning outcomes

You can:

- combine tile-local and running softmax statistics;
- keep score tiles and output state in threadgroup memory;
- prove that no quadratic intermediate is allocated;
- handle partial key tiles and causal entries safely;
- reason about barriers in a multi-stage threadgroup loop; and
- state the implemented head-width and tile-size limits.

21.3 Prerequisites

- Problem 003 for threadgroup tiles and barriers.
- Problem 009 for stable reduction identities.
- Problem 019 for the (m, l, o) recurrence.

21.4 Vocabulary

- **Key tile**: bounded block of K/V rows processed together.
- **Tile maximum**: largest valid score inside the current key tile.
- **Running state**: maximum, normalizer, and output accumulated across tiles.
- **Threadgroup memory**: on-chip storage shared by one Metal threadgroup.
- **Fusion boundary**: operations executed in one dispatch without global intermediates.
- **Partial tile**: final block with fewer valid keys than the tile width.

21.5 Math from first principles

For a tile T , compute scores and

$$m_T = \max_{k \in T} s_k, \quad m' = \max(m, m_T).$$

Define old-state scale $\alpha = e^{m-m'}$ and tile weights $w_k = e^{s_k-m'}$. Then

$$l' = \alpha l + \sum_{k \in T} w_k,$$

$$o' = \alpha o + \sum_{k \in T} w_k V_k.$$

This is the online recurrence applied to a block instead of one key. Masked or out-of-range tile entries have score $-\infty$ and weight zero.

21.5.1 Worked tiled example

Suppose prior state is $m=2, l=1, o=[4]$. A new two-key tile has scores $[1, 3]$ and values $[10, 20]$. New maximum is 3, so $\alpha = e^{-1} \approx 0.3679$, while tile weights are $[e^{-2}, 1] \approx [0.1353, 1]$.

$$l' = 0.3679 + 0.1353 + 1 = 1.5032,$$

$$o' = 0.3679(4) + 0.1353(10) + 1(20) = 22.8246.$$

Output is approximately 15.184. Scaling the old accumulator once per tile is essential; scaling it once per key is wrong.

21.6 Shape, layout, dtype, and bounds contract

Q $[Sq, Hq, dh]$, K/V $[Skv, Hkv, dh]$, and output $[Sq, Hq, dh]$ use the shared contiguous Float32 batch-one contract and GQA mapping. Absolute offsets control causality.

The implemented Metal kernel has:

- key tile size 16;
- threadgroup width 128;
- maximum $dh=128$; and
- arbitrary sequence length representable by UInt32 and host allocation.

Partial tiles and skv not divisible by 16 are valid. Head dimension 129 is an explicit Metal error. The CPU comparison path is not restricted to 128.

21.7 CPU reference path

The CPU solution mirrors tile semantics: loop over 16-key blocks, build only a small tile score list, combine tile statistics with running state, and normalize at the end. The independent judge still uses the materialized Double oracle.

21.8 Independent correctness method

Cases include a five-key partial tile and 17 keys crossing a tile boundary, with multiple heads in the first case. The judge checks K/V length validation and compares against materialized attention with $6e-5 + 1.2e-4 * \text{abs}(\text{expected})$ tolerance. Metal tests execute MSL and assert the head-width limit.

```
1 swift run leetllm check 020 --cpu
2 swift run leetllm check 020 --metal
3 swift run leetllm check 020 --solution
```

21.9 Performance model

Leading arithmetic remains roughly $4H_q S_q S_{kv} d_h$. Global Q/K/V/output traffic is linear in tensor size per pass; there is no global $4H_q S_q S_{kv}$ score intermediate.

Per threadgroup, on-chip state is approximately:

- 16 score Floats;
- 16 weight Floats;
- 128 accumulator Floats; and
- 3 state Floats.

That is 652 bytes, excluding compiler-managed registers. This row-tiled kernel still rereads K/V for different queries. A larger query tile could reuse each K/V tile across query rows but would require more score and accumulator state.

21.10 Metal grid, memory, and barriers

One 128-thread group owns $(\text{query}, \text{queryHead})$. For every 16-key tile:

1. lanes $0 \dots < 16$ compute one score each into threadgroup memory;
2. all lanes cross a barrier;
3. lane zero computes tile max, weights, new m , new l , and α ;
4. all lanes cross a barrier;
5. lanes cooperatively rescale and update output features;
6. all lanes cross a barrier before score storage is reused.

All 128 lanes reach every barrier, including lanes without a feature. The host allocates only Q, K, V, and output buffers. This is one fused MSL dispatch.

See [P020TiledAttention.metal](#).

21.11 Implementation checkpoints

1. Pass the CPU online oracle without tiling.
2. Combine a two-key tile by hand.
3. Implement score loading with invalid entries as negative infinity.
4. Synchronize before lane-zero statistics.
5. Rescale the threadgroup output once per tile.
6. Synchronize before reusing score/weight arrays.
7. Test lengths 5, 16, and 17.
8. Confirm host code never allocates scores.

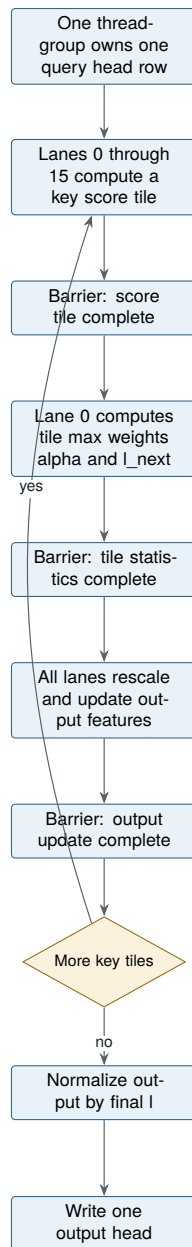


Figure 21.1: Fused key tile loop.

21.12 Controlled experiments

21.12.1 Tile-boundary sweep

Test lengths 15, 16, 17, 31, 32, 33. Prediction: correctness is continuous; runtime changes in steps as another key tile is required.

21.12.2 Head-width sweep

Sweep dh from 16 to 128. Prediction: more lanes perform useful accumulator work; on-chip state and arithmetic grow. $dh=129$ must fail explicitly.

21.12.3 Materialized versus fused traffic

Compare Problem 016 and 020 at increasing S . Prediction: the fused path's advantage grows as avoided quadratic score traffic grows, though this simple row schedule may not win at tiny shapes.

21.12.4 Query-tile thought experiment

Estimate K/V reads if one group owned several query rows. Prediction: K/V reuse improves, but score and output state multiply by query-tile size.

21.13 Engine integration

The output contract is interchangeable with Problems 016 and 019. A prefill engine can select the tiled path when its explicit constraints hold and retain materialized attention for parity. Decode with one query may prefer a different schedule over cached K/V.

21.14 Tradeoffs

- Fusion removes global intermediates but specializes the kernel.
- One query per group limits K/V reuse across queries.
- Fixed tiles simplify storage bounds but create partial-tile work.
- Threadgroup state reduces traffic but constrains dh and occupancy.

21.15 Hints

- Rescale the old accumulator exactly once per tile.
- Give masked lanes score negative infinity and weight zero.
- Place barriers outside lane conditionals.
- Check the host pipeline: separate score or softmax dispatches violate this lesson.

21.16 Completion checklist

- ☐ CPU and Metal match the materialized oracle.
- ☐ Lengths crossing 16 pass.
- ☐ No $S \times S$ buffer or separate score/softmax dispatch exists.
- ☐ Every lane reaches every tile barrier.
- ☐ $dh \leq 128$ is enforced and tested.

- ☐ You ran a boundary, width, or traffic experiment with a prediction.

21.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

21.17.1 Sources/LeetLLMExercises/P020TiledAttentionExercise.swift

```

1 import LeetLLMCore
2
3 public enum P020TiledAttentionExercise {
4     public static func apply(
5         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration
6     ) throws -> FloatTensor {
7         let input = try AttentionInput(queries: q, keys: k, values: v, configuration: c)
8         try validateVisibleKeys(input)
9         return try FloatTensor(Array(repeating: 0, count: q.elementCount), shape: q.shape)
10    }
11 }

```

21.17.2 Sources/LeetLLMExercises/P020TiledAttentionMetal.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P020TiledAttentionExercise {
5     public static func makeMetalPipeline() throws -> MetalTiledAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P020TiledAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P020TiledAttention.metal") }
10        return try MetalTiledAttentionPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }

```

21.17.3 Sources/LeetLLMExercises/Metal/P020TiledAttention.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3 kernel void tiled_fused_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device const
  ↪ float *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4 &dims
  ↪ [[buffer(5)]], uint local [[thread_index_in_threadgroup]], uint group [[threadgroup_position_in_grid]] { if (group
  ↪ >= shape.x * shape.z) return; for (uint d = local; d < dims.x; d += 128) o[group * dims.x + d] = 0.0f; }

```

21.18 Worked solution

Solution strategy

Tiled fused attention processes keys in 16-key tiles within a fused kernel. A threadgroup of 128 threads cooperatively processes one (query, head) pair. Threads compute score tiles in parallel, a single thread (local ID 0) updates running state (m, l) once per tile, then all threads cooperatively update the output accumulator using threadgroup memory for scores and weights. Three threadgroup barriers synchronize: after score computation, after state update, and after output update. No global score buffer exists; fusion boundary is one dispatch.

Correctness invariant

Running state (m, l, o) updated once per 16-key tile, not per key. Tile maximum and tile sum of exponentials are computed and applied together via the rescale factor. All 128 threads participate in each barrier; partial-tile and non-causal entries have score $-\infty$ and weight 0. Output written at final output location is guaranteed numerically equal to online softmax of the same sequence.

Common failure mode

Rescaling accumulator once per key instead of once per tile. Omitting or misplacing barriers, causing data races between compute and read. Assuming tile boundaries align with causal or valid keys; partial final tile must be handled explicitly. Computing tile sum of weights but forgetting to include it in l update.

21.18.1 Implementation points

Key tile size and threadgroup memory layout

Key tile size is 16. Threadgroup memory holds scores[16] and weights[16] shared by the 128 threads. Accumulator[128] for the head dimension features. State array [3] for (m, l, o 's numerator). One thread group processes one (query, head) pair; dimensions are known at compile time.

Tile-level state update and rescaling

After computing a tile's scores, thread 0 finds the tile maximum, computes $m' = \max(\text{old } m, \text{tile max})$, rescale factor $\alpha = \exp(\text{old } m - m')$, and tile weight sum. Updates $l' = \alpha * l + \text{tile sum}$, $m' = \text{new } m$. All 128 threads then rescale their feature accumulators by α and add tile contributions.

Three-tier barrier synchronization

`threadgroup_barrier(mem_flags::mem_threadgroup)` after score computation ensures all scores visible to thread 0. Second barrier after state update ensures new state visible to all threads. Third barrier after output update ensures next tile sees consistent state.

Fusion and no quadratic intermediate

Scores are computed, reduced to one [3] state, and output updated within a single dispatch with no global intermediate. Full attention ([Sq,Skv] scores) would require $O(Sq * Skv)$ bytes; tiling eliminates this, retaining only [16] per tile.

21.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P020TiledAttentionMetal.swift

- Line 4: extension P020TiledAttentionSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalTiledAttentionPipeline {

Sources/LeetLLMSolutions/P020TiledAttentionSolution.swift

- Line 4: public enum P020TiledAttentionSolution {
- Line 5: public static func apply(

Sources/LeetLLMSolutions/Metal/P020TiledAttention.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

21.18.3 Complete canonical listings

21.18.4 Sources/LeetLLMSolutions/P020TiledAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P020TiledAttentionSolution {
5     public static func makeMetalPipeline() throws -> MetalTiledAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P020TiledAttention", withExtension: "metal", subdirectory: "Metal")
9         else { throw MetalNeuralOperatorError.kernelResourceMissing("P020TiledAttention.metal") }
10        return try MetalTiledAttentionPipeline(source: String(contentsOf: url, encoding: .utf8))
11    }
12 }
```

21.18.5 Sources/LeetLLMSolutions/P020TiledAttentionSolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P020TiledAttentionSolution {
5     public static func apply(
6         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration
7     ) throws -> FloatTensor {
8         let input = try AttentionInput(queries: q, keys: k, values: v, configuration: c)
9         try validateVisibleKeys(input)
10        var output = Array(repeating: Float.zero, count: q.elementCount)
11        let scale = 1 / sqrt(Float(c.headDimension))
12        for query in 0..

```

```

20     from: 0, to: input.keyValueLength, by: MetalTiledAttentionPipeline.keyTileSize)
21     {
22         let tileEnd = min(
23             tileStart + MetalTiledAttentionPipeline.keyTileSize, input.keyValueLength)
24         var scores: [(Int, Float)] = []
25         for key in tileStart..

```

21.18.6 Sources/LeetLLMSolutions/Metal/P020TiledAttention.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3 constant uint keyTileSize = 16; constant uint maxHead = 128; constant uint groupWidth = 128;
4 kernel void tiled_fused_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device const
5     float *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4 &dims
6     [[buffer(5)]], uint local [[thread_index_in_threadgroup]], uint group [[threadgroup_position_in_grid]]) {
7     if (group >= shape.x * shape.z) return; uint query = group / shape.z, qHead = group % shape.z, kvHead = qHead /
8     dims.y, queryPosition = dims.z + query; threadgroup float scores[keyTileSize], weights[keyTileSize],
9     accumulator[maxHead], state[3]; for (uint d = local; d < dims.x; d += groupWidth) accumulator[d] = 0.0f; if
10     (local == 0) { state[0] = -INFINITY; state[1] = 0.0f; state[2] = 0.0f; }
11     threadgroup_barrier(mem_flags::mem_threadgroup);
12     for (uint tileStart = 0; tileStart < shape.y; tileStart += keyTileSize) { if (local < keyTileSize) { uint key =
13     tileStart + local; float score = -INFINITY; if (key < shape.y && dims.w + key <= queryPosition) { score = 0.0f;
14     for (uint d = 0; d < dims.x; ++d) score += q[(query * shape.z + qHead) * dims.x + d] * k[(key * shape.w + kvHead)
15     * dims.x + d]; score *= rsqrt(float(dims.x)); } scores[local] = score; }
16     threadgroup_barrier(mem_flags::mem_threadgroup); if (local == 0) { float tileMax = -INFINITY; for (uint j = 0; j
17     < keyTileSize; ++j) tileMax = max(tileMax, scores[j]); float newMax = max(state[0], tileMax); float alpha =
18     isfinite(state[0]) ? exp(state[0] - newMax) : 0.0f; float tileSum = 0.0f; for (uint j = 0; j < keyTileSize; ++j)
19     { weights[j] = isfinite(scores[j]) ? exp(scores[j] - newMax) : 0.0f; tileSum += weights[j]; } state[1] = state[1]
20     * alpha + tileSum; state[0] = newMax; state[2] = alpha; } threadgroup_barrier(mem_flags::mem_threadgroup); for
21     (uint d = local; d < dims.x; d += groupWidth) { float sum = accumulator[d] * state[2]; for (uint j = 0; j <
22     keyTileSize; ++j) { uint key = tileStart + j; if (key < shape.y) sum += weights[j] * v[(key * shape.w + kvHead)
23     * dims.x + d]; } accumulator[d] = sum; } threadgroup_barrier(mem_flags::mem_threadgroup); }
24     for (uint d = local; d < dims.x; d += groupWidth) o[(query * shape.z + qHead) * dims.x + d] = accumulator[d] /
25     state[1];
26 }

```

Sliding-Window Attention

22.1 Why this exists

Full causal attention lets every query read all earlier positions. Sliding-window attention limits each query to recent context. That bounds computation and KV reads for long sequences, but it also removes information. The window is a model policy with quality consequences, not merely a faster implementation of full attention.

22.2 Learning outcomes

You can:

- define inclusive causal window bounds precisely;
- apply bounds using absolute query and key positions;
- handle window one and windows larger than context;
- implement online local attention on CPU and Metal;
- derive full versus windowed work and visibility; and
- separate quality, cache capacity, and per-query compute effects.

22.3 Prerequisites

- Problem 016 for causal visibility.
- Problem 019 for streaming stable attention.
- Problem 018 for KV-cache dimensions and grouped heads.

22.4 Vocabulary

- **Window w** : maximum number of absolute key positions visible to one query.
- **Inclusive bound**: both oldest retained position and current position are visible.
- **Local attention**: attention restricted to a neighborhood rather than all context.
- **Receptive field**: positions that can influence an output directly or through layers.
- **Eviction**: removing KV entries no future layer policy will read.
- **Quality tradeoff**: changed model information flow, not numerical approximation.

22.5 Math and position semantics

For absolute query position p_q , a key at p_k is visible exactly when

$$\max(0, p_q - W + 1) \leq p_k \leq p_q.$$

The implementation uses offsets, so

$$p_q = q_{offset} + q, \quad p_k = k_{offset} + k.$$

The score, stable softmax, and weighted-value equations are unchanged; only the set of keys in their reductions changes.

22.5.1 Worked numerical example

At query position 5 with $w=3$, visible absolute positions are $[3, 4, 5]$. If the local K/V buffer begins at key offset 2, those correspond to local indices $[1, 2, 3]$. Using local query index instead of absolute position would select the wrong rows.

For $w=1$, only position 5 is visible, so output equals $V[5]$ exactly. If w is at least the available causal context, local attention equals full causal attention.

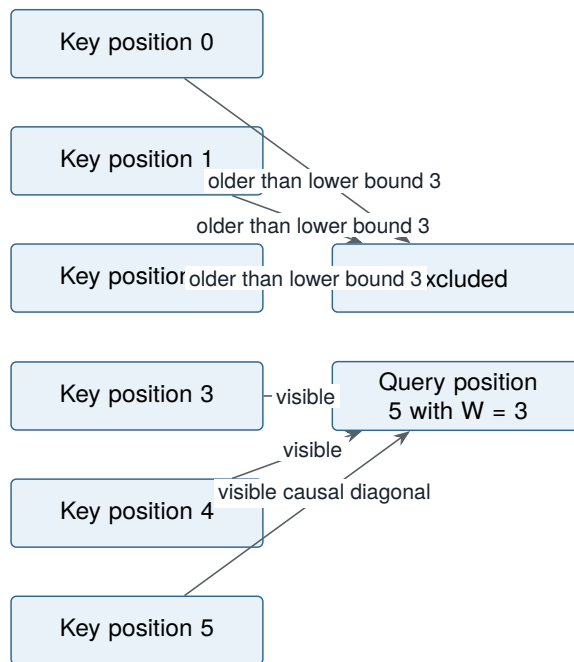


Figure 22.1: Absolute window visibility.

22.6 Shape, layout, and dtype contract

Q $[S_q, H_q, d_h]$, K/V $[S_{kv}, H_{kv}, d_h]$, and output $[S_q, H_q, d_h]$ are contiguous Float32 under the shared batch-one GQA contract. w is a positive integer. Offsets are nonnegative absolute positions. Every query must have at least one key inside both its causal and lower window bounds.

The Metal path shares Problem 019's explicit $d_h \leq 128$ private-accumulator limit. Window size may exceed context; it is not clamped because the visibility predicate naturally selects all available causal keys.

22.7 CPU reference path

For each query, compute absolute lower and upper bounds. Stream only keys whose absolute positions fall inside them and update online (m, l, o) state. Normalize once at the end. Do not slice by local indices

before considering offsets.

22.8 Independent correctness method

The Double materialized oracle applies the same absolute predicate. Tests cover $w=1$, w larger than context, different nonzero Q/K offsets, and $w=0$. A full-causal implementation fails the window-one and offset cases even though its shapes and numerical softmax are otherwise valid.

```
1 swift run leetllm check 021 --cpu
2 swift run leetllm check 021 --metal
3 swift run leetllm check 021 --solution
```

22.9 Performance and memory model

For long steady-state sequences, each query visits at most w keys, so score and value work is approximately

$$4H_q S_q W d_h$$

instead of $4H_q S_q S_{kv} d_h$. Per-query K/V read traffic is bounded by $2W H_{kv} d_h$ elements when all KV heads are needed.

Windowed computation does not by itself shrink allocated cache. Capacity drops only if the engine evicts entries that no layer will need. Models can mix local and global layers, making eviction policy more complex. Lower work, lower live cache, and changed quality are three separate claims.

22.10 Metal mapping

One work item owns (query, queryHead) and keeps online state in private memory. It computes an absolute lower bound without unsigned underflow, skips keys outside [lower, queryPosition], and applies the same GQA mapping as Problem 018.

There is no score buffer, threadgroup memory, or barrier. The MSL kernel is real and independent of the CPU solution. A future tiled local kernel could skip entire K/V tiles outside the window rather than testing every local key.

See [P021SlidingWindowAttention.metal](#).

22.11 Implementation checkpoints

1. Validate $w > 0$.
2. Derive bounds for query positions zero and five.
3. Make $w=1$ return the current value.
4. Make a large window equal full causal attention.
5. Add independent Q and K offsets.
6. Stream only visible keys with online softmax.
7. Match the same cases on Metal.

22.12 Controlled experiments

22.12.1 Window sweep

Fix a long context and sweep w . Prediction: work grows linearly until the window covers available context; outputs may continue changing as older information becomes visible.

22.12.2 Offset invariance

Shift Q and K offsets by the same constant. Prediction: outputs stay unchanged because all relative visibility relationships remain the same.

22.12.3 Cache eviction simulation

Compare retaining full K/V with retaining only the maximum needed local window. Prediction: computation is identical for purely local layers, but mixed global layers prevent blanket eviction.

22.12.4 Layer stacking

Trace which original positions can influence a token after several local layers. Prediction: indirect receptive field can grow beyond one layer's window, but it is not equivalent to direct full attention.

22.13 Engine integration

The decoder scheduler must know each layer's window before choosing cache read ranges or eviction. The kernel accepts explicit offsets so a retained cache slice can begin after absolute position zero. Output remains compatible with the same concatenation and projection path as full attention.

22.14 Tradeoffs

- Smaller windows bound work and reads but remove direct long-range context.
- Retaining full cache simplifies mixed policies; eviction saves capacity.
- Per-key predicates are simple; tile-range dispatch can avoid skipped work.
- Window policy must match model training and layer configuration.

22.15 Hints

- The oldest visible position is $qPosition - w + 1$, not $qPosition - w$.
- Apply lower and upper bounds to absolute key positions.
- Do not clamp w to local K/V length before considering offsets.
- Treat cache allocation and keys visited as separate measurements.

22.16 Completion checklist

- ☐ Window one, large windows, and nonzero offsets pass.
- ☐ Bounds are inclusive and absolute.
- ☐ CPU and actual Metal agree with the independent oracle.
- ☐ $w=0$ is rejected.
- ☐ You can distinguish compute, cache capacity, and quality effects.
- ☐ You ran a window, offset, eviction, or layer experiment with a prediction.

22.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

22.17.1 Sources/LeetLLMExercises/P021SlidingWindowAttentionExercise.swift

```
1 import LeetLLMCore
2
3 public enum P021SlidingWindowAttentionExercise {
4     public static func apply(
5         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration, _ window: Int
6     ) throws -> FloatTensor {
7         let input = try AttentionInput(queries: q, keys: k, values: v, configuration: c)
8         try validateVisibleKeys(input, window: window)
9         return try FloatTensor(Array(repeating: 0, count: q.elementCount), shape: q.shape)
10    }
11 }
```

22.17.2 Sources/LeetLLMExercises/P021SlidingWindowAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P021SlidingWindowAttentionExercise {
5     public static func makeMetalPipeline() throws -> MetalStreamingAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P021SlidingWindowAttention", withExtension: "metal", subdirectory: "Metal")
9         else {
10             throw MetalNeuralOperatorError.kernelResourceMissing("P021SlidingWindowAttention.metal")
11         }
12         return try MetalStreamingAttentionPipeline(
13             source: String(contentsOf: url, encoding: .utf8), functionName: "sliding_window_attention",
14             operation: "sliding-window attention")
15     }
16 }
```

22.17.3 Sources/LeetLLMExercises/Metal/P021SlidingWindowAttention.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3 kernel void sliding_window_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device
4     ↪ const float *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4
5     ↪ &dims [[buffer(5)]], constant uint &window [[buffer(6)]], uint index [[thread_position_in_grid]] { if (index >=
6     ↪ shape.x * shape.z) return; for (uint d = 0; d < dims.x; ++d) o[index * dims.x + d] = 0.0f; }
```

22.18 Worked solution

Solution strategy

Sliding window attention restricts visibility to keys within an inclusive absolute window $[\max(0, \text{qPosition} - W + 1), \text{qPosition}]$. For each query, compute lower bound and upper bound; stream only keys whose absolute positions fall in range and update online (m,l,o) state. The visibility predicate replaces the causal mask; w must be ≥ 1 . Window of 1 means position qPosition only; window $\geq$ context length equals full causal attention.

Correctness invariant

Window lower bound = $\max(0, \text{qPosition} - W + 1)$, upper bound = qPosition , both inclusive. Only keys with $\text{keyPosition} \in [\text{lower}, \text{upper}]$ participate in softmax and output. Window is a model policy (controls information flow), not a numerical approximation. Lowering W removes recent context permanently.

Common failure mode

Using local sequence indices instead of absolute positions to compute bounds. Clamping the window to context length or other values, or applying position offsets incorrectly. Confusing windowed attention (information loss) with numerical approximation. Computing lower bound as $\text{qPosition} - W$ instead of $\text{qPosition} - W + 1$.

22.18.1 Implementation points

Absolute position bounds and inclusive window range

Query absolute position = $\text{queryOffset} + \text{query}$. Key absolute position = $\text{keyOffset} + \text{key}$. Window visibility: lower = $\max(0, \text{qPosition} - W + 1)$, upper = qPosition . A key is visible iff $\text{keyPosition} \geq \text{lower}$ AND $\text{keyPosition} \leq \text{upper}$ (both inclusive).

Window of 1 as position-only attention

$W=1$: lower = $\text{qPosition} - 1 + 1 = \text{qPosition}$, upper = qPosition . Only the current position is visible. Output equals $V[\text{qPosition}]$ exactly (assuming position-aligned indices). This is a special case, not an error.

Window larger than context equals full causal attention

If $W \geq$ available context, the lower bound naturally filters to the earliest available position and the predicate reduces to full causal ($\text{keyPosition} \leq \text{qPosition}$). The window parameter is not clamped; the predicate naturally selects all available causal keys.

Online streaming with window predicate

Metal and CPU implementations iterate all keys and check both causal and window predicates: $\text{keyPosition} \geq \text{lower}$ AND $\text{keyPosition} \leq \text{upper}$. Keys outside this range contribute score = $-\infty$ and

weight = 0. Streaming updates (m,l,o) only for valid keys, automatically handling partial windows.

22.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P021SlidingWindowAttentionMetal.swift

- Line 4: extension P021SlidingWindowAttentionSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalStreamingAttentionPipeline {

Sources/LeetLLMSolutions/P021SlidingWindowAttentionSolution.swift

- Line 4: public enum P021SlidingWindowAttentionSolution {
- Line 5: public static func apply(

Sources/LeetLLMSolutions/Metal/P021SlidingWindowAttention.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

22.18.3 Complete canonical listings

22.18.4 Sources/LeetLLMSolutions/P021SlidingWindowAttentionMetal.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P021SlidingWindowAttentionSolution {
5     public static func makeMetalPipeline() throws -> MetalStreamingAttentionPipeline {
6         guard
7             let url = Bundle.module.url(
8                 forResource: "P021SlidingWindowAttention", withExtension: "metal", subdirectory: "Metal")
9         else {
10             throw MetalNeuralOperatorError.kernelResourceMissing("P021SlidingWindowAttention.metal")
11         }
12         return try MetalStreamingAttentionPipeline(
13             source: String(contentsOf: url, encoding: .utf8), functionName: "sliding_window_attention",
14             operation: "sliding-window attention")
15     }
16 }

```

22.18.5 Sources/LeetLLMSolutions/P021SlidingWindowAttentionSolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P021SlidingWindowAttentionSolution {
5     public static func apply(
6         _ q: FloatTensor, _ k: FloatTensor, _ v: FloatTensor, _ c: AttentionConfiguration, _ window: Int
7     ) throws -> FloatTensor {
8         let input = try AttentionInput(queries: q, keys: k, values: v, configuration: c)
9         try validateVisibleKeys(input, window: window)
10         var output = Array(repeating: Float.zero, count: q.elementCount)
11         let scale = 1 / sqrt(Float(c.headDimension))
12         for query in 0..

```

```

21     let keyPosition = c.keyPositionOffset + key
22     guard keyPosition >= lower, keyPosition <= qPosition else { continue }
23     var score: Float = 0
24     for d in 0..

```

22.18.6 Sources/LeetLLMSolutions/Metal/P021SlidingWindowAttention.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3 constant uint maxHeadDimension = 128;
4 kernel void sliding_window_attention(device const float *q [[buffer(0)]], device const float *k [[buffer(1)]], device
    ↪ const float *v [[buffer(2)]], device float *o [[buffer(3)]], constant uint4 &shape [[buffer(4)]], constant uint4
    ↪ &dims [[buffer(5)]], constant uint &window [[buffer(6)]], uint index [[thread_position_in_grid]]) {
5     if (index >= shape.x * shape.z) return; uint query = index / shape.z, qHead = index % shape.z, kvHead = qHead /
    ↪ dims.y, qPosition = dims.z + query, lower = qPosition + 1 > window ? qPosition - window + 1 : 0; float m =
    ↪ -INFINITY, l = 0.0f, accumulator[maxHeadDimension]; for (uint d = 0; d < dims.x; ++d) accumulator[d] = 0.0f;
6     for (uint key = 0; key < shape.y; ++key) { uint keyPosition = dims.w + key; if (keyPosition < lower || keyPosition >
    ↪ qPosition) continue; float score = 0.0f; for (uint d = 0; d < dims.x; ++d) score += q[(query * shape.z + qHead)
    ↪ * dims.x + d] * k[(key * shape.w + kvHead) * dims.x + d]; score *= rsqrt(float(dims.x)); float newM = max(m,
    ↪ score), alpha = isfinite(m) ? exp(m - newM) : 0.0f, beta = exp(score - newM); l = l * alpha + beta; for (uint d =
    ↪ 0; d < dims.x; ++d) accumulator[d] = accumulator[d] * alpha + beta * v[(key * shape.w + kvHead) * dims.x + d]; m
    ↪ = newM; }
7     for (uint d = 0; d < dims.x; ++d) o[(query * shape.z + qHead) * dims.x + d] = accumulator[d] / l;
8 }

```

Part V

KV-Cache Engineering

Preallocate and Append K/V

23.1 Why this exists

Autoregressive decode produces one key vector and one value vector per layer for each new token. Rebuilding or growing arrays on every step adds allocation, copying, and pointer instability to the serial decode loop. A decoder instead reserves its cache budget before generation and writes each token into a known slot.

This lesson builds that first reusable stateful component. The cache has a fixed batch size of one, fixed layer and shape metadata, independent progress per layer, and absolute logical positions that are not confused with storage slots.

23.2 Learning outcomes

You can:

- preallocate separate contiguous K and V stores without append-driven growth;
- derive [layer, token-slot, KV-head, feature] offsets;
- enforce capacity, shape, layer, and sequential-position invariants;
- read a token/head vector and materialize a layer for attention;
- distinguish logical token position from physical cache slot; and
- calculate allocated bytes before decode starts.

23.3 Prerequisites

- Problem 002 for row-major offsets and checked shapes.
- Problem 014 for [token, Hkv, dh] K/V projections.
- Problem 018 for why cache allocation uses Hkv, not Hq.

23.4 Vocabulary

- **Capacity c**: maximum token slots reserved per layer.
- **Logical position**: absolute token index used by RoPE and attention policy.
- **Physical slot**: zero-based address within one layer's reserved storage.
- **Logical count**: number of initialized slots in one layer.
- **Layer isolation**: appending layer ℓ changes no count or value in another layer.
- **Storage identity**: the backing allocation address, which remains stable here.

23.5 Math and worked offsets

K and V each use logical shape $[L, C, H_{kv}, d_h]$. For token-major row-major storage, the element offset is

$$o(l, t, h, d) = (((lC + t)H_{kv} + h)d_h + d).$$

With $L=2, C=3, H_{kv}=2, d_h=2$, element $(1, 2, 1, 1)$ has offset

$$(((1 \cdot 3 + 2) \cdot 2 + 1) \cdot 2 + 1) = 23.$$

Each K or V allocation therefore has $2 \cdot 3 \cdot 2 \cdot 2 = 24$ Float32 values. Both together occupy $2 \cdot 24 \cdot 4 = 192$ bytes.

Logical positions need not begin at zero. If layer zero first appends absolute position 7, it writes physical slot 0; position 8 writes slot 1. Position 10 is rejected because this contiguous lesson requires no gaps.

23.6 Shape, layout, and dtype contract

KVCacheConfiguration fixes batch size one, L, H_{kv}, d_h , and C ; all four dimensions are positive and checked for integer overflow. Append receives K and V tensors of contiguous Float32 shape $[H_{kv}, d_h]$ plus a layer and nonnegative logical position.

The first position in each layer may be any nonnegative integer. Later appends must increase by exactly one. Every layer has its own count and position table. Appending beyond C , using an invalid layer/head, passing the wrong vector shape, or reading an absent position throws an explicit KVCacheError.

23.7 CPU reference path

Allocate both arrays at $L \cdot C \cdot H_{kv} \cdot d_h$ elements. For an append, validate first, derive $\text{slot} = \text{count}[\text{layer}]$, copy exactly $H_{kv} \cdot d_h$ K elements and V elements into their fixed ranges, record the logical position, then increment that layer's count. Never call `Array.append` on K or V storage.

Reads translate a logical position to its initialized slot and return one d_h -element head vector. The shared `KVCacheReadable.materialized` helper walks logical positions and heads to produce $[T, H_{kv}, d_h]$ tensors for later oracles.

23.8 Independent correctness method

The judge constructs its expected transcript independently from literal values. It interleaves appends to two layers, checks layer isolation and readable values, checks two fixed 24-float allocations and their stable addresses, and requires errors for capacity overflow, a logical-position gap, and a shape mismatch.

```
1 swift run leetllm check 022 --cpu
2 swift run leetllm check 022 --solution
```

23.9 Performance, bandwidth, and allocation model

The allocation budget is

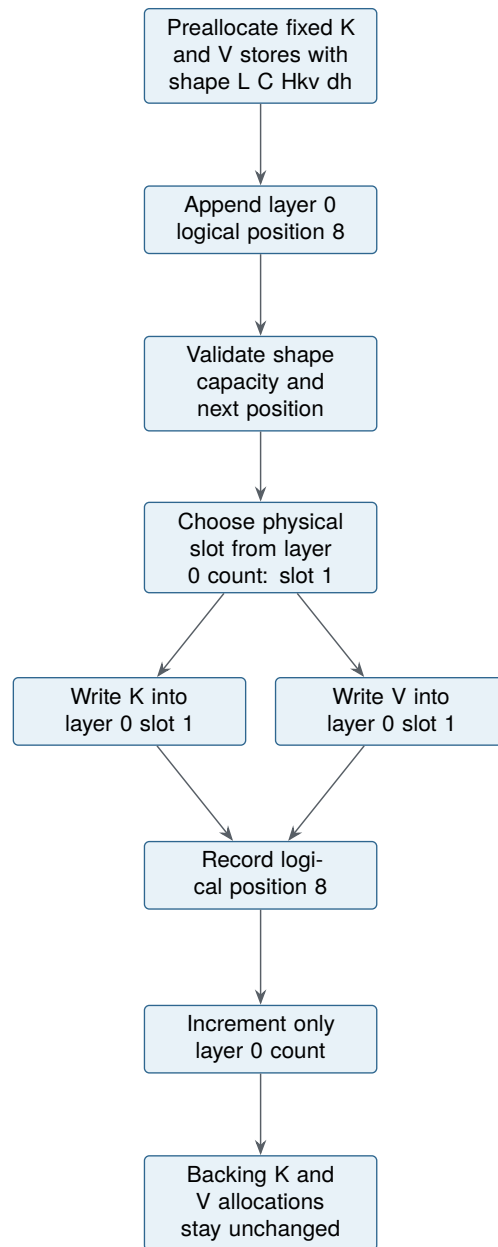


Figure 23.1: Preallocated kv append.

$$B = 2LCH_{kv}d_h \cdot 4 \text{ bytes.}$$

One append writes $2 \cdot H_{kv} \cdot d_h \cdot 4$ data bytes plus small position/count metadata. It does not copy earlier tokens and performs no K/V allocation. The fixed budget can waste unused capacity; that is the explicit tradeoff explored by paging in Problem 027.

Address stability is an invariant test, not a timing claim. Swift’s array implementation and this machine’s allocator are not benchmarked in this lesson.

23.10 Metal mapping

There is no Metal stage. The new behavior is host-side ownership, validation, and lifetime management, not parallel arithmetic. Copying a tiny new-token vector with a GPU dispatch would obscure the allocation invariant. Problem 023 passes this storage to an actual MSL decode-attention kernel.

23.11 Implementation checkpoints

1. Validate configuration dimensions and byte-count overflow.
2. Allocate K, V, positions, and per-layer counts once.
3. Append the first nonzero logical position.
4. Append a second token without changing storage addresses.
5. Interleave two layers and verify isolation.
6. Read one head vector and a materialized layer.
7. Reject overflow, gaps, wrong shapes, and absent reads.

23.12 Controlled experiments

23.12.1 Capacity sweep

Hold L, H_{kv}, d_h fixed and sweep C . Prediction: allocated bytes grow linearly, while append work remains constant until capacity is reached.

23.12.2 Layer interleaving

Append the same positions in layer-major and token-major call order. Prediction: each layer’s final transcript is identical because counts are independent.

23.12.3 Growth comparison

Compare this fixed store with a local implementation that repeatedly appends to empty arrays. Prediction: both produce the same values, but only the fixed store has a pre-decode byte budget and invariant storage identity.

23.13 Engine integration

`KVCacheReadable` and `KVCacheWritable` are the shared batch-one boundary used by cached attention in Problem 023 and by ring, paged, and quantized alternatives. The canonical `ContiguousKVCache` is the simplest decoder cache when maximum context is known and reserving the full budget is acceptable.

23.14 Tradeoffs

- Full preallocation gives simple offsets and stable storage but reserves peak memory.
- Per-layer counts support decoder execution order but add small metadata.
- Sequential positions simplify lookup; arbitrary insertion would require an index.
- Token-major layout makes one append contiguous; other reads may prefer another layout.

23.15 Hints

- Allocate array *count*, not only array capacity.
- Validate both K and V before mutating either store.
- Keep logical positions in their own table; do not infer absolute position from slot.
- Include both K and V in every byte calculation.

23.16 Completion checklist

- ☐ Storage is allocated once at the exact derived size.
- ☐ Nonzero logical positions append and read correctly.
- ☐ Layer counts and values remain isolated.
- ☐ Capacity, shape, position, layer, and read errors are explicit.
- ☐ K/V addresses and element counts remain stable across appends.
- ☐ You ran one capacity, interleaving, or growth experiment with a prediction.

23.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

23.17.1 Sources/LeetLLMExercises/P022ContiguousKVCacheExercise.swift

```

1 import LeetLLMCore
2
3 public enum P022ContiguousKVCacheExercise {
4     public static func run(
5         configuration: KVCacheConfiguration,
6         appends: [KVCacheAppend]
7     ) throws -> ContiguousKVCacheTranscript {
8         var counts = Array(repeating: 0, count: configuration.layerCount)
9         var lastPositions = Array<Int?>(repeating: nil, count: configuration.layerCount)
10        for append in appends {
11            try P022ContiguousKVCacheContract.validate(
12                append, configuration: configuration, counts: counts, lastPositions: lastPositions)
13            counts[append.layer] += 1
14            lastPositions[append.layer] = append.logicalPosition
15        }
16
17        let emptyLayers = try counts.map { _ in
18            try KVCacheLayerSnapshot(
19                logicalPositions: [],
20                keys: FloatTensor([], shape: [0, configuration.keyValueHeadCount, configuration.headDimension]),
21                values: FloatTensor([], shape: [0, configuration.keyValueHeadCount, configuration.headDimension]))
22        }
23        return ContiguousKVCacheTranscript(
24            layers: emptyLayers,
25            allocatedBytes: 0,
26            keyStorageCount: 0,
27            valueStorageCount: 0,
28            storageAddressesStable: false)
29    }
30 }

```

23.18 Worked solution

Solution strategy

Allocate fixed-size contiguous storage for K and V tensors with shape $[L, C, Hkv, dh]$ where C is the maximum token capacity. Appends write each new K/V token to sequential physical slots while tracking absolute logical positions independently per layer. Layer-level count and position metadata enforce sequential position requirements (position N must be followed by $N+1$), and storage base addresses remain stable across all operations—no dynamic reallocation.

Correctness invariant

Physical slots within each layer fill contiguously from zero without gaps. Each layer maintains its own logical position sequence, and appends must respect strict monotonicity: if the last position was N , the next append must be $N+1$. Capacity never overflows. Base addresses of `keyStorage` and `valueStorage` remain constant throughout the cache lifetime.

Common failure mode

Confusing logical position with physical slot index. Position 10 appended first goes to slot 0, not slot 10. Later position gaps (e.g., appending 12 after 10) violate the sequential contract and throw `positionSequenceMismatch`, preventing non-contiguous histories that would break RoPE and causal attention semantics.

23.18.1 Implementation points

Token-major offset formula

Element offset = $((\text{layer} * C + \text{slot}) * Hkv + \text{head}) * dh + \text{feature}$. This packing ensures all features of one head are contiguous, all heads of one token are contiguous, and all tokens in one layer are contiguous, enabling efficient sequential appends.

Layer isolation via per-layer metadata

Each layer has independent `counts[layer]` and `firstPositions[layer]`. Appending to layer 0 increments only `counts[0]` and updates only `firstPositions[0]`, leaving layer 1's state untouched. This isolation is essential for multi-layer inference where each layer progresses independently.

First position can be any nonnegative integer

Unlike slot indices (which start at 0), logical positions are absolute model positions. `firstPositions[layer]` records where the sequence started (e.g., position 7). Subsequent positions must increment by exactly 1, enabling position-relative RoPE calculations.

Storage stability verification

`storageAddresses()` returns the base pointers before and after append sequences. The test verifies `before == after`, proving no `Array.append()` calls on `keyStorage` or `valueStorage`—all writes use

fixed offset arithmetic within pre-allocated arrays.

23.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P022ContiguousKVCacheSolution.swift

- Line 3: public enum P022ContiguousKVCacheSolution {
- Line 4: public static func run(

23.18.3 Complete canonical listings

23.18.4 Sources/LeetLLMSolutions/P022ContiguousKVCacheSolution.swift

```

1 import LeetLLMCore
2
3 public enum P022ContiguousKVCacheSolution {
4     public static func run(
5         configuration: KVCacheConfiguration,
6         appends: [KVCacheAppend]
7     ) throws -> ContiguousKVCacheTranscript {
8         let cache = ContiguousKVCache(configuration: configuration)
9         let before = cache.storageAddresses()
10        for append in appends {
11            try cache.append(
12                layer: append.layer,
13                logicalPosition: append.logicalPosition,
14                key: append.key,
15                value: append.value)
16        }
17        let after = cache.storageAddresses()
18        var layers: [KVCacheLayerSnapshot] = []
19        layers.reserveCapacity(configuration.layerCount)
20        for layer in 0..

```


Cached Single-Token Attention

24.1 Why this exists

During decode, the model receives one new token. Earlier tokens' projected keys and values have not changed, so recomputing all prior Q/K/V projections would be wasted work. The decoder projects only the current token, appends its K/V, and attends the current query over cached history.

This lesson makes that state transition executable on CPU and Metal. The Metal kernel reads the preallocated cache arrays directly; no CPU attention fallback is used by the Metal check.

24.2 Learning outcomes

You can:

- order one decode step as append K/V, then read attention history;
- preserve absolute positions while cache slots begin at zero;
- compare cached decode with a materialized attention oracle;
- map query heads to cache heads in a real MSL kernel;
- identify the computation reused exactly from earlier tokens; and
- separate cache allocation, append traffic, and attention-read traffic.

24.3 Prerequisites

- Problem 016 for scaled causal attention and the materialized oracle.
- Problem 019 for stable online softmax.
- Problem 022 for fixed storage and readable cache vectors.

24.4 Vocabulary

- **Decode step**: processing one newly generated or sampled token.
- **Current K/V**: projections for the token whose query is being evaluated.
- **Cached history**: earlier K/V vectors retained without recomputation.
- **Append-then-attend**: ordering that makes the current token visible to itself.
- **Materialized oracle**: independent Double attention over explicit $[T, H_{kv}, d_h]$ tensors.
- **Absolute position**: model position used by RoPE and causal policy, not a slot index.

24.5 Math from first principles

For current absolute position p , query head h_q , and cached positions $r \leq p$,

$$s_r = \frac{Q_{p,h_q} \cdot K_{r,h_{kv}}}{\sqrt{d_h}}, \quad O_{p,h_q} = \sum_{r \leq p} \text{softmax}(s)_r V_{r,h_{kv}}.$$

Only Q_p, K_p, V_p are newly projected. Every K_r, V_r for $r < p$ is reused bit-for-bit from cache storage.

24.5.1 Worked decode step

Suppose cache positions are [11,12] and the new token is position 13. Append its K/V into slot 2, producing logical positions [11,12,13], then compute one query over all three rows. The materialized oracle uses query offset 13 and key offset 11; using local query index zero would incorrectly mask positions 11 and 12.

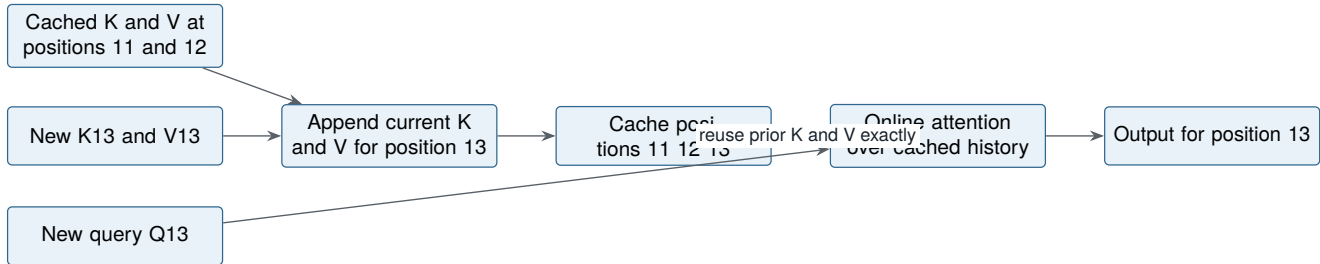


Figure 24.1: Append then attend.

24.6 Shape, layout, and dtype contract

The request is batch one. Query is contiguous Float32 [Hq,dh]. Input K/V are the deterministic append sequence [T,Hkv,dh], including the current token as the last row. Cache storage is separate contiguous Float32 K and V arrays with layout [L,C,Hkv,dh], where T<=C.

Hq must be divisible by Hkv; all dimensions are positive and all values are finite. queryLogicalPosition must equal firstLogicalPosition+T-1. Layer and positions are explicit. Output is [Hq,dh] Float32.

24.7 CPU reference path

Create the fixed cache, slice each token's [Hkv,dh] K/V vectors, and append them in logical order. For each query head, map its KV head, then stream cached positions using online (maximum,denominator,weightedValue) state. Normalize once after the last visible position.

The reusable attend function accepts any KVCacheReadable, which lets later ring, paged, and quantized caches share attention semantics without sharing their physical address calculations.

24.8 Independent correctness method

The judge reshapes the one query to [1,Hq,dh] and invokes the independent Double materialized oracle over the original K/V tensor. It covers one-token self-attention, multiple tokens at nonzero absolute positions and a nonzero layer, allocation bytes, position transcripts, bad query shape, and a query position that is not the appended current token.

```

1 swift run leetllm check 023 --cpu
2 swift run leetllm check 023 --metal
3 swift run leetllm check 023 --solution
  
```

24.9 Performance, bytes, and allocation model

The step writes current K/V once:

$$B_{append} = 2H_{kv}d_h \cdot 4.$$

The semantic minimum for reading all cached K/V is approximately $2TH_{kv}d_h \cdot 4$ bytes when all KV heads are used, plus Q and output. Dot and weighted-value work is about $4TH_qd_h$ FLOPs. There is no score-matrix allocation and no cache growth.

The baseline MSL kernel favors clarity: one work item owns one output feature of one query head and therefore recomputes that head's scores for each output feature. Its traffic and FLOPs exceed the semantic minimum. This is a correct decode baseline, not a claim of optimal throughput.

24.10 Metal mapping

The grid is [dh,Hq]. Each work item reads the full query head and all cached K vectors, maintains a stable softmax recurrence, and accumulates one output feature from V. Cache offsets use [L,C,Hkv,dh]; kvHead=qHead/groupSize.

All buffers use shared storage for this teaching path. There is no threadgroup memory or barrier. The command is compiled from MSL at runtime, dispatched, and checked for completion. The starter kernel also executes MSL but writes an intentionally wrong zero result.

See [P023CachedAttention.metal](#).

24.11 Implementation checkpoints

1. Validate query/K/V/cache shapes and current absolute position.
2. Append a one-token request and recover its value exactly.
3. Append history plus current K/V at a nonzero starting position.
4. Match the materialized Double oracle on CPU.
5. Bind raw cache arrays, layer, capacity, and count to Metal.
6. Implement stable score and value accumulation in MSL.
7. Match the same judge with actual Metal execution.

24.12 Controlled experiments

24.12.1 Cached versus recomputed projections

Count projected tokens for one step at context τ . Prediction: cached decode projects one token; a recomputing path projects τ , although attention still reads τ K/V rows.

24.12.2 Context sweep

Sweep τ with fixed heads and width. Prediction: append traffic stays constant while attention work and cache reads grow linearly.

24.12.3 Absolute-position shift

Shift first and query positions by the same constant. Prediction: this causal decode output is unchanged because ordering is unchanged; RoPE would already have encoded the shifted positions in Q/K.

24.12.4 Kernel ownership

Compare the feature-owned baseline with a local head-owned kernel. Prediction: head ownership avoids repeated score work but needs a bounded accumulator or another output strategy.

24.13 Engine integration

This is the attention operation used inside a one-token decoder layer after Q/K/V projection and RoPE. The new K/V are appended once per layer; the output continues to head concatenation and the output projection. Problem 040 can call the same cache-readable CPU boundary while selecting a production kernel.

24.14 Tradeoffs

- Cached decode removes prior projection work, not the need to read prior K/V.
- Feature-owned MSL is simple and unbounded in d_h , but repeats score computation.
- Shared buffers simplify teaching; private GPU storage and persistent buffers reduce setup cost.
- Contiguous cache offsets are cheap; alternative allocation policies need a gather step.

24.15 Hints

- Append current K/V before attention so the diagonal token is visible.
- Use cache logical positions for policy and slots only for addresses.
- Subtract or stream the maximum before exponentiation.
- A Metal stage that calls CPU attention is not a Metal implementation.

24.16 Completion checklist

- ☐ Current K/V are appended before the query reads cache.
- ☐ Prior K/V are read, not recomputed.
- ☐ Nonzero absolute positions and layers pass.
- ☐ CPU and actual Metal match the materialized Double oracle.
- ☐ Allocation and per-step byte costs are derived separately.
- ☐ You ran one context, position, projection, or kernel-ownership experiment.

24.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

24.17.1 Sources/LeetLLMExercises/P023CachedAttentionExercise.swift

```
1 import LeetLLMCore
2
3 public enum P023CachedAttentionExercise {
4     public static func run(_ request: CachedAttentionRequest) throws -> CachedAttentionResult {
5         CachedAttentionResult(
6             output: try FloatTensor(
7                 Array(repeating: 0, count: request.query.elementCount), shape: request.query.shape),
8             cachedLogicalPositions: Array(
9                 request.firstLogicalPosition...request.queryLogicalPosition),
10            allocatedBytes: request.cacheConfiguration.allocatedFloat32Bytes)
11     }
12 }
```

24.17.2 Sources/LeetLLMExercises/P023CachedAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P023CachedAttentionExercise {
5     public static func makeMetalPipeline() throws -> MetalCachedAttentionPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P023CachedAttention", withExtension: "metal", subdirectory: "Metal")
8         else { throw MetalNeuralOperatorError.kernelResourceMissing("P023CachedAttention.metal") }
9         return try MetalCachedAttentionPipeline(source: String(contentsOf: url, encoding: .utf8))
10    }
11
12    public static func runMetal(
13        _ request: CachedAttentionRequest,
14        pipeline: MetalCachedAttentionPipeline
15    ) throws -> CachedAttentionResult {
16        let zeroStorage = Array(repeating: Float.zero, count: request.cacheConfiguration.elementsPerTensor)
17        return CachedAttentionResult(
18            output: try pipeline.apply(
19                query: request.query,
20                keyStorage: zeroStorage,
21                valueStorage: zeroStorage,
22                configuration: request.cacheConfiguration,
23                queryHeadCount: request.attentionConfiguration.queryHeadCount,
24                layer: request.layer,
25                tokenCount: request.tokenCount),
26            cachedLogicalPositions: Array(request.firstLogicalPosition...request.queryLogicalPosition),
27            allocatedBytes: request.cacheConfiguration.allocatedFloat32Bytes)
28    }
29 }
```

24.17.3 Sources/LeetLLMExercises/Metal/P023CachedAttention.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void cached_decode_attention(
5     device const float *query [[buffer(0)]],
```

```
6  device const float *keys [[buffer(1)]],
7  device const float *values [[buffer(2)]],
8  device float *output [[buffer(3)]],
9  constant uint4 &shape [[buffer(4)]],
10 constant uint4 &dimensions [[buffer(5)]],
11 uint2 index [[thread_position_in_grid]] {
12     if (index.x >= dimensions.y || index.y >= shape.w) return;
13     output[index.y * dimensions.y + index.x] = 0.0f;
14 }
```

24.18 Worked solution

Solution strategy

Append K/V for the current token to the contiguous cache, then compute attention over all cached positions using online softmax. The query (at the appended token's absolute position) is evaluated against cached K/V reused bit-for-bit from prior appends. Causal masking uses absolute logical positions, not slot indices, to correctly mask future tokens and ensure positions that are not yet appended remain invisible.

Correctness invariant

Query position must equal `firstLogicalPosition + tokenCount - 1` (the appended token's own position). All cached K/V vectors are reused exactly as written; no recomputation. Online softmax accumulates numerator and denominator across visible positions in chronological order. Query head `h_q` maps to KV head `floor(h_q / groupSize)` deterministically.

Common failure mode

Using local token index or slot index for causal masking instead of absolute position. Cached positions `[11,12,13]` masked on local index would incorrectly block positions 11 and 12 from a query at local slot 2. Causal correctness requires `logicalPosition <= queryLogicalPosition`, enforced via absolute positions from `logicalPositions()`.

24.18.1 Implementation points

Append-then-attend ordering

K/V for the current token are appended first, making them available for that token's own attention (self-attention). Attention then reads chronologically from the cache, applying causal masking based on absolute positions, not physical slot order.

Online softmax with three-state tracking

Online softmax maintains (maximum, denominator, weighted_accumulator) across the position loop. maximum tracks the largest score seen so far to avoid overflow in `exp()` calculations. After all visible positions, the accumulator is divided by denominator to produce the final normalized output without materializing the full attention matrix.

Head-to-head mapping via groupSize

$\text{kvHead} = \text{floor}(\text{queryHead} / \text{groupSize})$ where $\text{groupSize} = \text{queryHeadCount} / \text{keyValueHeadCount}$. For GQA with $H_q=4$, $H_{kv}=2$, this yields `[0,0,1,1]`, correctly mapping contiguous groups. The same KVCacheReadable protocol abstracts Float and quantized caches, with dequantization transparent to attention logic.

Immutable configuration validation

KVCacheConfiguration is validated at cache construction. Append and read operations reuse the immutable configuration, avoiding repeated validation and enabling Metal kernels to assume fixed layer count, head count, and feature dimension without dynamic branching.

24.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P023CachedAttentionMetal.swift

- Line 4: extension P023CachedAttentionSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalCachedAttentionPipeline {

Sources/LeetLLMSolutions/P023CachedAttentionSolution.swift

- Line 4: public enum P023CachedAttentionSolution {
- Line 5: public static func run(_ request: CachedAttentionRequest) throws -> CachedAttentionResult {
- Line 18: public static func runMetal(
- Line 36: public static func attend(
- Line 95: private static func populatedCache(_ request: CachedAttentionRequest) throws -> ContiguousKVCache {

Sources/LeetLLMSolutions/Metal/P023CachedAttention.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

24.18.3 Complete canonical listings

24.18.4 Sources/LeetLLMSolutions/P023CachedAttentionMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P023CachedAttentionSolution {
5     public static func makeMetalPipeline() throws -> MetalCachedAttentionPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P023CachedAttention", withExtension: "metal", subdirectory: "Metal")
8         else { throw MetalNeuralOperatorError.kernelResourceMissing("P023CachedAttention.metal") }
9         return try MetalCachedAttentionPipeline(source: String(contentsOf: url, encoding: .utf8))
10    }
11 }
```

24.18.5 Sources/LeetLLMSolutions/P023CachedAttentionSolution.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum P023CachedAttentionSolution {
5     public static func run(_ request: CachedAttentionRequest) throws -> CachedAttentionResult {
6         let cache = try populatedCache(request)
7         return CachedAttentionResult(
8             output: try attend(
9                 query: request.query,
10                cache: cache,
11                layer: request.layer,
12                queryLogicalPosition: request.queryLogicalPosition,
13                queryHeadCount: request.attentionConfiguration.queryHeadCount),
14            cachedLogicalPositions: try cache.logicalPositions(layer: request.layer),
15            allocatedBytes: cache.allocatedBytes)
```



```

16 }
17
18 public static func runMetal(
19     _ request: CachedAttentionRequest,
20     pipeline: MetalCachedAttentionPipeline
21 ) throws -> CachedAttentionResult {
22     let cache = try populatedCache(request)
23     return CachedAttentionResult(
24         output: try pipeline.apply(
25             query: request.query,
26             keyStorage: cache.rawKeyStorage(),
27             valueStorage: cache.rawValueStorage(),
28             configuration: cache.configuration,
29             queryHeadCount: request.attentionConfiguration.queryHeadCount,
30             layer: request.layer,
31             tokenCount: try cache.count(layer: request.layer)),
32         cachedLogicalPositions: try cache.logicalPositions(layer: request.layer),
33         allocatedBytes: cache.allocatedBytes)
34 }
35
36 public static func attend(
37     query: FloatTensor,
38     cache: any KVCacheReadable,
39     layer: Int,
40     queryLogicalPosition: Int,
41     queryHeadCount: Int,
42     window: Int? = nil
43 ) throws -> FloatTensor {
44     try cache.configuration.validate(layer: layer)
45     let configuration = try AttentionConfiguration(
46         queryHeadCount: queryHeadCount,
47         keyValueHeadCount: cache.configuration.keyValueHeadCount,
48         headDimension: cache.configuration.headDimension)
49     let expected = [queryHeadCount, cache.configuration.headDimension]
50     guard query.shape == expected else {
51         throw KVCacheError.vectorShapeMismatch(name: "Query", expected: expected, actual: query.shape)
52     }
53     if let window, window <= 0 { throw AttentionError.invalidWindow(window) }
54     let positions = try cache.logicalPositions(layer: layer).filter { position in
55         position <= queryLogicalPosition
56         && (window == nil || position >= queryLogicalPosition - window! + 1)
57     }
58     guard !positions.isEmpty else {
59         throw AttentionError.noVisibleKeys(queryPosition: queryLogicalPosition)
60     }
61     var output = Array(repeating: Float.zero, count: query.elementCount)
62     let scale = 1 / sqrt(Float(configuration.headDimension))
63     for queryHead in 0..

```

```

89         = accumulator[feature] / denominator
90     }
91 }
92 return try FloatTensor(output, shape: query.shape)
93 }
94
95 private static func populatedCache(_ request: CachedAttentionRequest) throws -> ContiguousKVCache {
96     let cache = ContiguousKVCache(configuration: request.cacheConfiguration)
97     let elements = request.cacheConfiguration.elementsPerToken
98     for token in 0..

```

24.18.6 Sources/LeetLLMSolutions/Metal/P023CachedAttention.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void cached_decode_attention(
5     device const float *query [[buffer(0)]],
6     device const float *keys [[buffer(1)]],
7     device const float *values [[buffer(2)]],
8     device float *output [[buffer(3)]],
9     constant uint4 &shape [[buffer(4)]],
10    constant uint4 &dimensions [[buffer(5)]],
11    uint2 index [[thread_position_in_grid]]) {
12    uint feature = index.x;
13    uint queryHead = index.y;
14    if (feature >= dimensions.y || queryHead >= shape.w) return;
15
16    uint kvHead = queryHead / dimensions.z;
17    float maximum = -INFINITY;
18    float denominator = 0.0f;
19    float weightedValue = 0.0f;
20    for (uint token = 0; token < shape.z; ++token) {
21        uint tokenBase = (((shape.x * shape.y) + token) * dimensions.x + kvHead) * dimensions.y;
22        float score = 0.0f;
23        for (uint d = 0; d < dimensions.y; ++d) {
24            score += query[queryHead * dimensions.y + d] * keys[tokenBase + d];
25        }
26        score *= rsqrt(float(dimensions.y));
27        float nextMaximum = max(maximum, score);
28        float alpha = isfinite(maximum) ? exp(maximum - nextMaximum) : 0.0f;
29        float beta = exp(score - nextMaximum);
30        denominator = denominator * alpha + beta;
31        weightedValue = weightedValue * alpha + beta * values[tokenBase + feature];
32        maximum = nextMaximum;
33    }
34    output[queryHead * dimensions.y + feature] = weightedValue / denominator;
35 }

```

KV Layout Shootout

25.1 Why this exists

A tensor's logical values do not choose their physical order. Decode appends a whole token, while attention often reads one KV head across many tokens. Those two access patterns favor different contiguous dimensions. This systems lab implements both layouts, proves they round-trip, derives deterministic access traces, and provides a release benchmark whose result is deliberately local to its shape and machine.

25.2 Learning outcomes

You can:

- implement token-major [L,T,H,D] and head-major [L,H,T,D] offsets;
- copy logical values into either physical order and reconstruct them;
- count bytes and contiguous spans for a defined read pattern;
- benchmark one controlled layout variable with a checksum;
- explain why an access trace is not a hardware timing result; and
- choose a layout only for a stated workload and machine.

25.3 Prerequisites

- Problem 002 for logical versus physical tensor layout.
- Problem 003 for the cost and purpose of layout conversion.
- Problems 022-023 for cache append and decode-read patterns.

25.4 Vocabulary

- **Token-major**: physical order [layer, token, head, feature].
- **Head-major**: physical order [layer, head, token, feature].
- **Round-trip**: logical values survive physical copy and inverse gather.
- **Access trace**: deterministic sequence of element offsets touched by a loop.
- **Contiguous span**: maximal run of offsets increasing by one.
- **Benchmark boundary**: exact loop, allocation policy, build, shape, and machine measured.

25.5 Math from first principles: offset derivation

For token-major layout,

$$o_T = (((lT + t)H + h)D + d).$$

For head-major layout,

$$o_H = (((lH + h)T + t)D + d).$$

Use $L=2, T=4, H=2, D=3$ and element $(l=1, t=2, h=1, d=2)$:

$$o_T = 41, \quad o_H = 44.$$

Reading all features for layer 1, head 1, tokens 0...3 yields token-major offsets

```
1 27 28 29 | 33 34 35 | 39 40 41 | 45 46 47
```

or four contiguous spans. Head-major yields 36...47, one span. Both read $4*3*4 = 48$ Float32 bytes. Span count describes address continuity; it does not predict a universal latency winner.

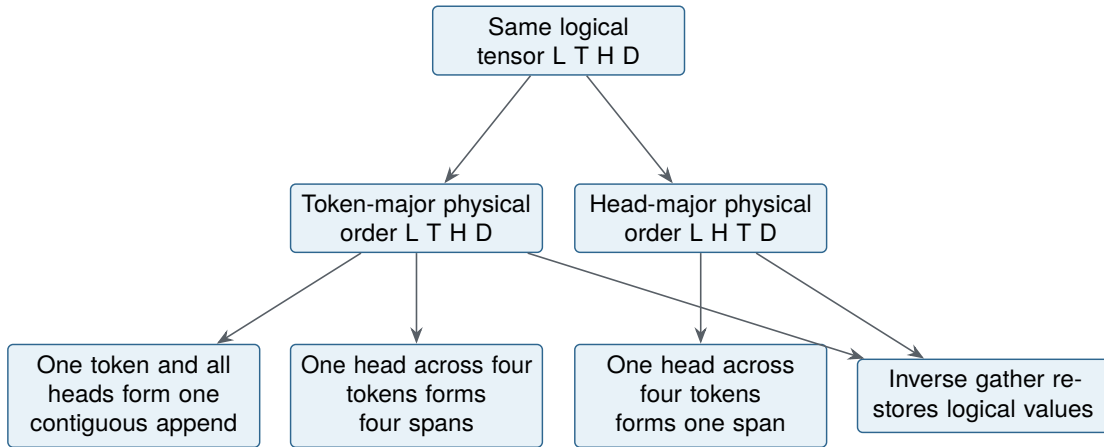


Figure 25.1: Kv physical layout comparison.

25.6 Shape, layout, and dtype contract

Logical input is contiguous Float32 $[L, T, Hk_v, d_h]$, with T equal to the configuration capacity for this complete-layout experiment. Both physical stores contain exactly $L*T*Hk_v*d_h$ Float32 values. Layer, slot, head, and feature indices are bounds checked.

The output contains both reconstructed logical tensors and traces for one selected $(layer, head)$ read. Batch size is one. K and V use the same layout formula; the fixture uses one generic value tensor so layout behavior is not confounded with attention arithmetic.

25.7 CPU reference path

For every logical (l, t, h, d) , compute its standard logical offset and the selected physical offset, then copy. The inverse loop computes the same physical offset and writes back to logical token-major order. Build the trace by holding layer/head fixed and looping token then feature.

Count a new span at the first offset or whenever $current \neq previous+1$.

25.8 Independent correctness method

The judge uses sequential values so every incorrect permutation is visible. It requires exact round-trips for both layouts, literal worked offsets 41 and 44, exact trace sequences, four versus one contiguous spans, and equal 48-byte reads. A fake implementation that labels token-major data as head-major fails even though its values appear unchanged.

```
1 swift run leetllm check 024 --cpu
2 swift run leetllm check 024 --solution
3 swift run -c release leetllm benchmark 024 --layers 16 --tokens 2048 --heads 8 --dimension 64 --iterations 7
```

25.9 Performance, bandwidth, and allocation model

Each layout stores the same bytes:

$$B = LTH_{kv}d_h \cdot 4$$

per K or V tensor. Conversion reads and writes every element, so converting one tensor moves at least 2B algorithmic bytes. A one-head decode scan reads $\tau \cdot d_h \cdot 4$ data bytes in either layout but presents different address runs.

The CLI benchmark prebuilds both stores, changes only the descriptor used by a layer/head/token/feature read loop, reports median nanoseconds and a checksum, and states the shape. Its outcome is shape-, machine-, compiler-, cache-, and loop-specific. Do not turn one result into a hardware-independent claim.

25.10 Metal mapping

This investigation is CPU-only because its required artifact is address and allocation reasoning plus a controlled host benchmark. A useful future Metal experiment would bind both layouts to the same cached-attention arithmetic and measure kernel-only time with persistent buffers. That is not fabricated here.

On GPU, adjacent feature threads naturally favor contiguous D; cooperative heads or token tiles determine whether the τ/H ordering matters. The grid must be specified before interpreting coalescing.

25.11 Implementation checkpoints

1. Derive both formulas on paper.
2. Check offsets for first, interior, and last elements.
3. Copy and round-trip token-major storage.
4. Copy and round-trip head-major storage.
5. Emit exact one-head traces and span counts.
6. Run the debug judge.
7. Write a prediction, then run the release benchmark with recorded shape.

25.12 Controlled experiments

25.12.1 Token-count sweep

Fix L, H, D and sweep T . Prediction: bytes grow linearly; head-major retains one logical span for the defined head scan while token-major retains T spans.

25.12.2 Head-count sweep

Fix total elements by trading H against D . Prediction: token-major gaps between one head's token vectors change with $H \cdot D$; timing need not follow span count monotonically because cache lines and loop overhead also change.

25.12.3 Append versus read

Benchmark copying one new $[H, D]$ token and scanning one head over T tokens separately. Prediction: token-major favors contiguous append, head-major favors the defined scan. The engine's weighting of those operations decides the choice.

25.12.4 Conversion break-even

Measure one conversion plus N reads versus retaining the original layout. Prediction: conversion wins only after enough affected reads repay its full pass.

25.13 Engine integration

`KVLayoutDescriptor` makes the physical formula explicit at the boundary. The canonical contiguous cache remains token-major because append is simple and the 023 kernel already consumes it. A production engine may select another layout only together with matching append, gather, and attention kernels.

25.14 Tradeoffs

- Token-major makes one token's all-head append contiguous.
- Head-major makes one head's full-token scan contiguous.
- Duplicating layouts avoids conversion but doubles cache storage.
- Conversion can help repeated reads but adds a full read/write pass.
- Span count is deterministic evidence; measured latency still needs a benchmark.

25.15 Hints

- Write axis order into the formula before substituting numbers.
- Test values that identify every logical coordinate.
- Keep allocation and conversion outside the timed read loop.
- Record build configuration, shape, iterations, machine, and checksum.

25.16 Completion checklist

- ☐ Both formulas produce the worked offsets.
- ☐ Both physical layouts round-trip exactly.
- ☐ Trace bytes and contiguous spans match the defined loop.
- ☐ The release benchmark reports shape, iterations, and checksum.

- ☐ Your conclusion is limited to the measured machine and workload.
- ☐ You wrote a prediction before one layout experiment.

25.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

25.17.1 Sources/LeetLLMExercises/P024KVLayoutShootoutExercise.swift

```

1 import LeetLLMCore
2
3 public enum P024KVLayoutShootoutExercise {
4     public static func run(
5         logicalValues: FloatTensor,
6         configuration: KVCacheConfiguration,
7         layer: Int,
8         head: Int
9     ) throws -> KVLayoutShootoutResult {
10         let token = KVLayoutDescriptor(kind: .tokenMajor, configuration: configuration)
11         var offsets: [Int] = []
12         for slot in 0..

```


25.18 Worked solution

Solution strategy

Copy logical $[L, T, H, k, v, d, h]$ tensor into both token-major $[L, T, H, k, v, d, h]$ and head-major $[L, H, k, v, T, d, h]$ physical memory layouts. Verify both layouts round-trip (copy back recovers the original logical values). Generate access traces for a selected (layer, head) by looping token-then-feature, counting contiguous memory spans. Both layouts store the same logical values; layout choice depends on workload, not universal superiority.

Correctness invariant

Round-trip logical values must survive copy-to-physical and inverse-gather-to-logical for both layouts. Token-major and head-major offset formulas must produce correct physical addresses for every (layer, slot, head, feature) tuple. Trace offsets must match the computed offset formula exactly; out-of-order memory access or incorrect formula produces wrong trace span counts.

Common failure mode

Assuming one layout universally outperforms the other without measuring on the actual machine and workload. Token-major excels for append (one token is contiguous) but reads one head across T tokens as T disjoint spans. Head-major reverses this: append scatters, one head read is contiguous. Cache locality, SIMD behavior, and prefetch effectiveness vary by processor; benchmark boundary (loop, allocation, build, shape, machine) determines winner.

25.18.1 Implementation points

Token-major vs. head-major offset formulas

Token-major: $((l * T + h) * H + h) * D + d$ groups features, then heads, then tokens. Head-major: $((l * H + h) * T + t) * D + d$ groups features, then tokens, then heads. For $(l=1, t=2, h=1, d=2)$ with $L=2, T=4, H=2, D=3$: token-major yields offset 41, head-major yields 44.

Access span counting for cache behavior

Reading one head across four tokens in token-major yields four 12-byte spans (one per token) due to strided head access. Head-major yields one 48-byte span (all tokens contiguous for that head). Span count quantifies memory fragmentation; fewer spans indicate better cache locality for that access pattern.

Round-trip proves invertibility

Copy to physical, then copy back to logical must recover original values exactly. Test uses sequential values $(0, 1, 2, \dots)$ so any wrong permutation creates visible errors. Both layouts must pass round-trip; a fake implementation that labels one layout but stores the other fails immediately.

Benchmark measurements are machine and workload specific

Access trace is deterministic (same offsets every run), but nanoseconds are not. Benchmark loops over iterations, reads in layer-head-token-feature order (the critical memory access pattern), and records median time plus checksum to prevent dead-code elimination. Results are local to this machine, this build, this shape—not universal.

25.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P024KVLayoutShootoutSolution.swift

- Line 4: public enum P024KVLayoutShootoutSolution {
- Line 5: public static func run(
- Line 32: public static func benchmark(
- Line 57: private static func copy(
- Line 77: private static func roundTrip(
- Line 99: private static func trace(
- Line 118: private static func measure(

25.18.3 Complete canonical listings

25.18.4 Sources/LeetLLMSolutions/P024KVLayoutShootoutSolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P024KVLayoutShootoutSolution {
5     public static func run(
6         logicalValues: FloatTensor,
7         configuration: KVCacheConfiguration,
8         layer: Int,
9         head: Int
10    ) throws -> KVLayoutShootoutResult {
11        let expectedShape = [
12            configuration.layerCount, configuration.capacity,
13            configuration.keyValueHeadCount, configuration.headDimension,
14        ]
15        guard logicalValues.shape == expectedShape else {
16            throw KVCacheError.vectorShapeMismatch(
17                name: "Logical KV values", expected: expectedShape, actual: logicalValues.shape)
18        }
19        try configuration.validate(layer: layer)
20        try configuration.validate(head: head)
21        let tokenDescriptor = KVLayoutDescriptor(kind: .tokenMajor, configuration: configuration)
22        let headDescriptor = KVLayoutDescriptor(kind: .headMajor, configuration: configuration)
23        let tokenStorage = try copy(logicalValues, descriptor: tokenDescriptor)
24        let headStorage = try copy(logicalValues, descriptor: headDescriptor)
25        return KVLayoutShootoutResult(
26            tokenMajorRoundTrip: try roundTrip(tokenStorage, descriptor: tokenDescriptor),
27            headMajorRoundTrip: try roundTrip(headStorage, descriptor: headDescriptor),
28            tokenMajorTrace: try trace(descriptor: tokenDescriptor, layer: layer, head: head),
29            headMajorTrace: try trace(descriptor: headDescriptor, layer: layer, head: head))
30    }
31
32    public static func benchmark(
33        configuration: KVCacheConfiguration,
34        iterations: Int
35    ) throws -> KVLayoutBenchmarkReport {
36        guard iterations > 0 else { throw KVCacheError.invalidTokenCount(iterations) }

```

```

37 let logical = try FloatTensor(
38   (0..

```

```

110     item.offset == 0 || item.element != offsets[item.offset - 1] + 1 ? count + 1 : count
111 }
112 return KVAccessTrace(
113     offsets: offsets,
114     contiguousReadSpans: spans,
115     bytesRead: offsets.count * MemoryLayout<Float>.stride)
116 }
117
118 private static func measure(
119     storage: [Float],
120     descriptor: KVLayoutDescriptor,
121     iterations: Int
122 ) throws -> (nanoseconds: UInt64, checksum: Float) {
123     var samples: [UInt64] = []
124     var checksum: Float = 0
125     for _ in 0..

```

Shared KV Heads

26.1 Why this exists

Problem 018 introduced MHA, MQA, and GQA as attention shapes. A stateful decoder must carry that architecture choice into allocation and cached reads. Reserving one KV head per query head for a GQA checkpoint wastes memory; mapping query heads with modulo silently reads the wrong learned state.

This lesson applies the exact contiguous-group mapping to a real cache-backed decode step and compares complete K-plus-V byte budgets for MHA, MQA, and GQA.

26.2 Learning outcomes

You can:

- allocate cache dimensions from H_{kv} while producing H_q outputs;
- derive and implement $kvHead = qHead / groupSize$;
- explain why modulo mapping is a semantic bug;
- reject head counts for which $H_q \% H_{kv} \neq 0$;
- calculate MHA, MQA, and GQA bytes with both K and V; and
- use the same contiguous cache for all three architecture endpoints.

26.3 Prerequisites

- Problem 018 for grouped-query attention semantics.
- Problem 022 for the cache's $[L, C, H_{kv}, dh]$ allocation.
- Problem 023 for cached single-token attention.

26.4 Vocabulary

- **Query head H_q** : independently normalized attention output channel.
- **KV head H_{kv}** : stored key/value channel, possibly shared.
- **Group size g** : H_q / H_{kv} contiguous query heads per KV head.
- **Division mapping**: $\text{floor}(qHead / g)$, the checkpoint convention used here.
- **Modulo mapping**: $qHead \% H_{kv}$; valid arithmetic, wrong grouping semantics.
- **Cache reduction factor**: H_q / H_{kv} relative to an equal-width MHA cache.

26.5 Math from first principles: head mapping and bytes

After validating divisibility,

$$g = \frac{H_q}{H_{kv}}, \quad h_{kv} = \left\lfloor \frac{h_q}{g} \right\rfloor.$$

For $H_q=4, H_{kv}=2, g=2$, so the map is $[0, 0, 1, 1]$. Modulo produces $[0, 1, 0, 1]$; shapes still pass, but heads 1 and 2 read different learned K/V vectors from the required architecture.

For Float32 cache capacity τ ,

$$B_{KV} = 2LTH_{kv}d_h \cdot 4.$$

With $L=2, T=3, d_h=2, H_q=4$, MHA ($H_{kv}=4$) uses 384 bytes, MQA ($H_{kv}=1$) uses 96, and GQA ($H_{kv}=2$) uses 192.

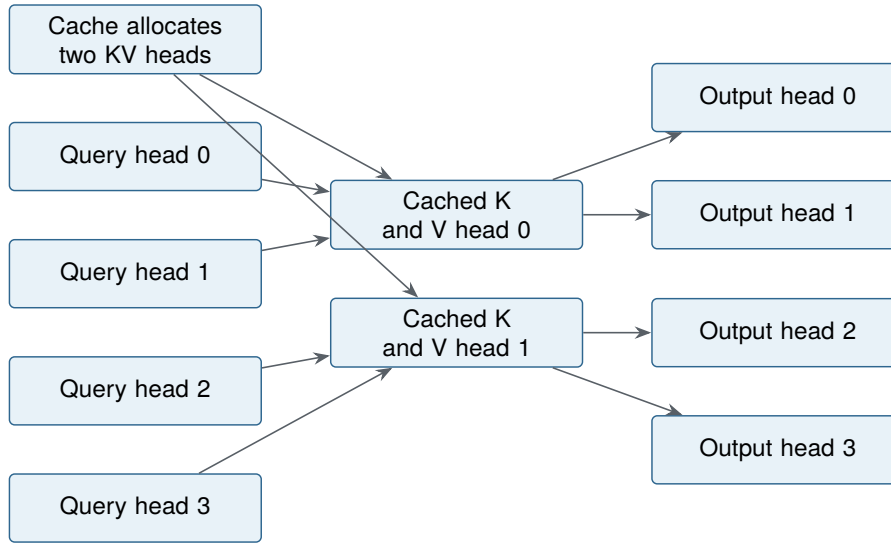


Figure 26.1: Cache backed shared kv heads.

26.6 Shape, layout, and dtype contract

Batch size is one. Query is contiguous Float32 $[H_q, d_h]$; cache K/V are Float32 $[L, C, H_{kv}, d_h]$; output is $[H_q, d_h]$. Counts and width are positive, and H_q must be divisible by H_{kv} before group size or offsets are used.

The cache stores only H_{kv} vectors per token. Query heads remain separate through score normalization and output. Logical positions and layer selection follow Problem 023's append-then-attend contract.

26.7 CPU reference path

Build the same ContiguousKVCache using its configured H_{kv} . For each query head, calculate its KV head once by integer division, then use that head for every cached K and V read. Do not share online softmax state between query heads; sharing stored K/V does not make their queries or probabilities identical.

The canonical solution deliberately reuses the generic cached-attention path so the architecture is represented by validated configuration, not a separate cache implementation.

26.8 Independent correctness method

The judge uses $H_q=4, H_{kv}=2$ with strongly distinct value ranges for the two KV heads. Its Double materialized oracle requires $[0, 0, 1, 1]$; the focused test passes a modulo implementation and verifies rejection. The judge also checks the exact 384/96/192 byte fixture and independently verifies that 3:2 head counts throw.

```
1 swift run leetllm check 025 --cpu
2 swift run leetllm check 025 --solution
```

26.9 Performance, bandwidth, and allocation model

Cache allocation, current-token append bytes, and semantic K/V read bytes all scale linearly with H_{kv} . MQA therefore uses H_q times fewer cache bytes than MHA at equal L, T, d_h ; GQA lies between them.

The number of query-head score rows still scales with H_q . Sharing creates a reuse opportunity but does not automatically remove dot products. Whether a kernel or hardware cache realizes that reuse is a measured implementation question, separate from the exact storage formula.

26.10 Metal mapping

This problem adds no separate MSL file because Problem 023's actual cached decode kernel already takes H_q, H_{kv} , and `groupSize` and uses division mapping on raw cache storage. The new artifact is the allocation/mapping policy and its rejection tests, not a duplicate kernel.

A cooperative future kernel could assign a threadgroup to one KV head and several query heads, staging K/V once. That mapping would need a benchmark against the simple feature-owned kernel before claiming lower traffic.

26.11 Implementation checkpoints

1. Reject nonpositive and non-divisible head counts.
2. Print mapping tables for MHA, MQA, and one GQA ratio.
3. Allocate using H_{kv} , never H_q .
4. Pass MHA and MQA endpoint reasoning.
5. Make the 4:2 fixture use $[0, 0, 1, 1]$.
6. Make a modulo implementation fail the judge.
7. Derive all three byte totals including K and V.

26.12 Controlled experiments

26.12.1 KV-head sweep

Fix H_q, L, T, d_h and sweep divisors of H_q . Prediction: allocated bytes and semantic K/V reads scale with H_{kv} ; output shape stays fixed.

26.12.2 Mapping mutation

Replace division with modulo locally. Prediction: validation and dimensions still pass, but outputs differ on the judge's distinct KV-head fixture.

26.12.3 Shared-read locality

Compare simple per-query-head and cooperative grouped kernels at one GQA shape. Prediction: cooperative staging can reduce repeated K/V reads, but extra synchronization may dominate at small τ .

26.13 Engine integration

Checkpoint metadata must provide both head counts. Q projection width is $H_q * d_h$; K and V projection widths and all cache variants use $H_{kv} * d_h$. The decoder passes the validated group size into cached attention while preserving one output vector per query head.

26.14 Tradeoffs

- MQA minimizes cache bytes; GQA retains more KV diversity.
- Division mapping is fixed by trained architecture, not selected for speed.
- Shared K/V can improve locality, but query-head arithmetic remains.
- One generic cache reduces code duplication; optimized kernels may specialize by ratio.

26.15 Hints

- Validate divisibility before dividing.
- Map contiguous groups with division, not alternating heads with modulo.
- Keep K/V offsets based on H_{kv} and output offsets based on H_q .
- Count K and V and include scalar byte width.

26.16 Completion checklist

- ☐ $H_q \% H_{kv} == 0$ is enforced.
- ☐ 4:2 maps query heads as $[0, 0, 1, 1]$.
- ☐ A modulo implementation fails a focused test.
- ☐ MHA, MQA, and GQA byte totals are exact.
- ☐ Cache allocation uses H_{kv} ; output uses H_q .
- ☐ You ran a KV-head, mapping, or locality experiment with a prediction.

26.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

26.17.1 Sources/LeetLLMExercises/P025SharedKVHeadsExercise.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P025SharedKVHeadsExercise {
5     public static func run(_ request: CachedAttentionRequest) throws -> SharedKVHeadsResult {
6         let c = request.attentionConfiguration
7         var output = Array(repeating: Float.zero, count: request.query.elementCount)
8         let scale = 1 / sqrt(Float(c.headDimension))
9         for queryHead in 0..

```

26.18 Worked solution

Solution strategy

Allocate cache with Hkv heads (not Hq). Map each query head to its KV head using floor division: $\text{kvHead} = \text{floor}(\text{queryHead} / \text{groupSize})$ where $\text{groupSize} = \text{Hq} / \text{Hkv}$. Compute attention separately for each query head, reusing the same K/V vector for groupSize consecutive query heads. Byte comparison shows MHA uses 4x the cache of MQA and demonstrates GQA as a memory-efficient middle ground.

Correctness invariant

Hq must be exactly divisible by Hkv to form equal-sized groups. Each group of groupSize consecutive query heads reads the same cached KV vector, preserving learned shared state. Attention output has Hq channels (one per query head) even though K/V storage has only Hkv channels. No silent aliases from incorrect head mapping.

Common failure mode

Using $\text{qHead} \% \text{Hkv}$ (modulo) instead of $\text{qHead} / \text{groupSize}$ (floor division) for head mapping. For $\text{Hq}=4$, $\text{Hkv}=2$, modulo produces $[0,1,0,1]$ instead of the required $[0,0,1,1]$, causing heads 0 and 2 to share one KV vector and heads 1 and 3 to share another—inverting the contiguous-group architecture and breaking the learned model behavior.

26.18.1 Implementation points

Group size and byte scaling

$\text{groupSize} = \text{Hq} / \text{Hkv}$. MHA ($\text{Hq}=\text{Hkv}$) has $\text{groupSize}=1$ and stores 1 KV per query head. MQA ($\text{Hkv}=1$) has $\text{groupSize}=\text{Hq}$ and stores 1 KV shared by all query heads. GQA intermediate (e.g., $\text{Hq}=8$, $\text{Hkv}=2$) has $\text{groupSize}=4$: four query heads per KV head, reducing cache by 4x vs. MHA while retaining per-head computation.

Exact byte comparison: MHA, MQA, GQA

For $L=2$, $T=3$, $\text{dh}=2$: MHA ($\text{Hkv}=4$) uses 384 bytes, MQA ($\text{Hkv}=1$) uses 96 bytes, GQA ($\text{Hkv}=2$) uses 192 bytes. Byte formula: $2 * L * T * \text{Hkv} * \text{dh} * 4$. GQA is a tunable point between memory-hungry MHA and compute-inefficient MQA.

Online softmax state per query head, not shared

Even if two query heads read the same KV vector, each head accumulates its own (maximum, denominator, value_accumulator) independently. Reuse is only K/V storage; compute state remains separate, avoiding spurious correlations between output channels.

Configuration validation at construction

KVHeadMemoryModel.compare() and AttentionConfiguration constructors validate $H_q \% H_{kv} == 0$ before computing bytes or offsets. Invalid divisibility throws queryHeadsNotDivisible, preventing silent misalignment. All downstream code assumes valid grouping.

26.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P025SharedKVHeadsSolution.swift

- Line 3: public enum P025SharedKVHeadsSolution {
- Line 4: public static func run(_ request: CachedAttentionRequest) throws -> SharedKVHeadsResult {

26.18.3 Complete canonical listings

26.18.4 Sources/LeetLLMSolutions/P025SharedKVHeadsSolution.swift

```

1 import LeetLLMCore
2
3 public enum P025SharedKVHeadsSolution {
4     public static func run(_ request: CachedAttentionRequest) throws -> SharedKVHeadsResult {
5         SharedKVHeadsResult(
6             attention: try P023CachedAttentionSolution.run(request),
7             bytes: try KVHeadMemoryModel.compare(
8                 layerCount: request.cacheConfiguration.layerCount,
9                 tokenCount: request.cacheConfiguration.capacity,
10                 queryHeadCount: request.attentionConfiguration.queryHeadCount,
11                 gqaHeadCount: request.cacheConfiguration.keyValueHeadCount,
12                 headDimension: request.cacheConfiguration.headDimension))
13     }
14 }
```

Ring-Buffer Sliding Cache

27.1 Why this exists

A layer with fixed sliding-window attention never reads arbitrarily old K/V. Keeping the full history still grows memory with generation length. A ring cache reserves exactly C token slots and overwrites the oldest physical slot while logical positions continue increasing forever.

The difficulty is not modulo arithmetic alone. Reads must remain chronological, overwritten positions must become inaccessible, and attention must apply its window to absolute positions rather than physical slot order.

27.2 Learning outcomes

You can:

- append monotonically increasing logical positions into fixed physical storage;
- derive wraparound and oldest-slot rules;
- return chronological positions after several complete wraps;
- reject reads of overwritten entries and position gaps;
- feed a chronological ring view into windowed cached attention; and
- separate bounded allocation from bounded attention work.

27.3 Prerequisites

- Problem 021 for inclusive sliding-window policy.
- Problem 022 for fixed cache storage and append validation.
- Problem 023 for cache-readable attention.

27.4 Vocabulary

- **Ring capacity C :** fixed number of physical token slots per layer.
- **Write cursor:** slot that receives the next append.
- **Oldest slot:** after the ring is full, the write cursor before its next write.
- **Wraparound:** moving the cursor from $C-1$ back to zero.
- **Overwrite:** replacing the oldest K/V and logical-position metadata.
- **Chronological view:** logical order reconstructed independently of physical order.

27.5 Wraparound derivation and worked example

For each append,

$$slot = nextSlot, \quad nextSlot = (slot + 1) \bmod C.$$

Before full capacity, chronological physical slots are $0 \dots \text{count}$. Once full, the oldest slot is `nextSlot`, and chronological slot i is

$$(nextSlot + i) \bmod C.$$

With $C=3$, append logical positions $10 \dots 17$. The final physical position array is $[16, 17, 15]$ and the cursor is slot 2. Reading from the cursor gives $[15, 16, 17]$. Physical sorting or reading slots $0, 1, 2$ would be wrong.

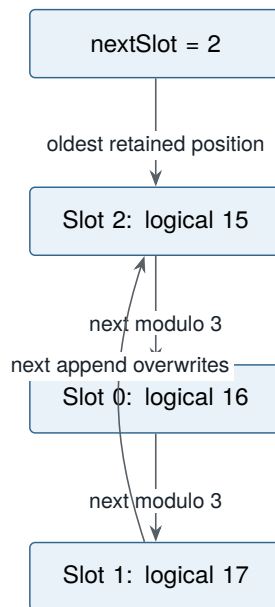


Figure 27.1: Ring chronology.

27.6 Shape, layout, and dtype contract

Batch size is one. K and V remain separate fixed `Float32` stores with physical shape $[L, C, H_{kv}, d_h]$. Append vectors are $[H_{kv}, d_h]$. Each layer has independent count, cursor, last logical position, and fixed position metadata.

The first logical position may be nonzero; each later append must equal `last+1`. Capacity is not an append error: once full, append overwrites. Reads address logical position and throw when it has been evicted. Output attention is `Float32 [H_q, d_h]` and uses a positive window.

27.7 CPU reference path

Validate K and V before mutation. Write both vectors and the absolute position to the cursor slot, cap count at C , advance modulo C , and update the last position. To enumerate, choose oldest slot from count/cursor state and walk exactly count entries modulo capacity.

Logical vector reads search the small fixed position table in this teaching implementation. Windowed attention consumes `KVCacheReadable`, filters logical positions, and never sees physical order.

27.8 Independent correctness method

The judge appends eight tokens into capacity three, beginning at absolute position ten. It checks the position history after every append, final K/V values, stable storage addresses and byte count, and window-two attention against the materialized Double oracle. Focused tests also reject a position gap and a read of overwritten position five.

```
1 swift run leetllm check 026 --cpu
2 swift run leetllm check 026 --solution
```

27.9 Performance, bandwidth, and allocation model

Allocation is still

$$B = 2LCH_{kv}d_h \cdot 4,$$

but C is the retained window capacity rather than maximum generation length. Append writes constant bytes and allocates nothing. Chronological enumeration is $O(C)$; the baseline's logical read search can add $O(C)$ lookup per vector.

A production ring can derive a direct slot from the first retained position or maintain an index. That optimization must preserve absolute position checks so stale overwritten data is never returned.

27.10 Metal mapping

This storage-policy lesson is CPU-only. Problem 021 already executes actual windowed MSL, and Problem 023 executes actual cache-backed MSL. A ring-aware kernel would pass cursor/count and derive each chronological physical slot with modulo before the same attention math.

No Metal speed claim is made here. Modulo cost, split wrapped ranges, and dispatch strategy require a persistent-buffer kernel benchmark.

27.11 Implementation checkpoints

1. Append fewer than C tokens without wrap.
2. Fill exactly C and identify the oldest slot.
3. Append once more and reject the overwritten logical position.
4. Cross several wraps and record every chronological view.
5. Start from a nonzero absolute position.
6. Verify another layer remains empty.
7. Match windowed attention to the materialized oracle.

27.12 Controlled experiments

27.12.1 Wrap-count sweep

Run one, two, and ten full wraps. Prediction: allocated bytes and logical count stay fixed; newest chronological values remain correct regardless of wrap count.

27.12.2 Window versus capacity

Use attention window $w < c$, $w = c$, and $w > c$. Prediction: work is bounded by available visible entries; $w > c$ cannot recover overwritten context.

27.12.3 Split-range read

Compare a physical loop with modulo each token against two contiguous ranges around the wrap. Prediction: two ranges avoid per-token modulo, but benefit is shape- and compiler-specific.

27.13 Engine integration

Use the ring only for layers whose model policy guarantees no future read older than capacity. The decoder still supplies monotonically increasing absolute positions for RoPE and masking. Mixed local/global models need ring storage for local layers and a different retention policy for global layers.

27.14 Tradeoffs

- Fixed memory enables unbounded generation for local layers but discards history.
- Searching position metadata is simple; direct indexing is faster but easier to get stale.
- One ring per layer preserves isolation; shared cursors would couple execution order.
- Wrapped reads can use modulo or two physical ranges.

27.15 Hints

- Store the logical position beside every physical slot.
- Once full, the next write cursor is also the oldest slot.
- Enumerate exactly count entries, not all capacity before the ring fills.
- Apply attention windows to logical positions after reconstructing chronology.

27.16 Completion checklist

- ☐ Storage size and addresses remain fixed across wraps.
- ☐ Logical positions increase while physical slots wrap.
- ☐ Chronological reads are correct after several wraps.
- ☐ Overwritten positions and position gaps are rejected.
- ☐ Windowed attention matches the absolute-position oracle.
- ☐ You ran a wrap, capacity, or split-range experiment with a prediction.

27.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

27.17.1 Sources/LeetLLMExercises/P026RingKVCacheExercise.swift

```
1 import LeetLLMCore
2
3 public enum P026RingKVCacheExercise {
4     public static func run(_ request: RingKVCacheRequest) throws -> RingKVCacheResult {
5         let positions = Array(request.firstLogicalPosition...request.queryLogicalPosition)
6         let history = positions.indices.map { Array(positions[0...$0]) }
7         return RingKVCacheResult(
8             chronologicalHistory: history,
9             finalSnapshot: KVCacheLayerSnapshot(
10                 logicalPositions: positions,
11                 keys: request.keys,
12                 values: request.values),
13             attentionOutput: try FloatTensor(
14                 Array(repeating: 0, count: request.query.elementCount), shape: request.query.shape),
15             allocatedBytes: request.configuration.allocatedFloat32Bytes,
16             storageAddressesStable: true)
17     }
18 }
```


27.18 Worked solution

Solution strategy

Reserve fixed C slots per layer with a `nextSlot` cursor starting at 0. On append, write K/V and logical position to `nextSlot`, then $\text{nextSlot} = (\text{nextSlot} + 1) \% C$. Once count reaches C , the cache is full; further appends overwrite the oldest slot (at `nextSlot`). Chronological view reconstructs logical order via $(\text{nextSlot} + \text{index}) \% C$ without exposing wraparound to attention. Sliding-window attention filters to window bounds and applies causal masking on absolute positions.

Correctness invariant

Logical positions are monotonically increasing (position N followed by $N+1$). Physical slots wrap via modulo, hiding wraparound from external code. Overwritten positions become permanently inaccessible—reads reject `logicalPosition` if the token was evicted. Count caps at C . Window size must be positive.

Common failure mode

Reading chronological positions via simple slot order $[0, 1, 2]$ instead of $[(\text{oldestSlot} + i) \% C]$. For a full cache at slot 2 holding positions $[15, 16, 17]$ (oldest at slot 2), reading slots in order yields $[16, 17, 15]$ instead of $[15, 16, 17]$. Attention then uses wrong absolute positions for RoPE and causal masking, breaking output correctness.

27.18.1 Implementation points

Wraparound and chronological enumeration

After appending positions 10..17 into capacity 3, the final state is $\text{positions}[\text{layer} * C + \text{slot}] = [16, 17, 15]$ at slots $[0, 1, 2]$ with `nextSlot`=2. Enumeration uses $\text{chronologicalSlot} = (\text{nextSlot} + \text{index}) \% C$ for index in $[0, \text{count})$, yielding slots $[2, 0, 1] \rightarrow$ positions $[15, 16, 17]$.

Position metadata storage and lookup

Position array has fixed capacity C , indexed by $[\text{layer} * C + \text{slot}]$. `logicalPositions(layer)` rebuilds chronological order by walking $(\text{nextSlot} + i) \% C$ and reading $\text{positions}[\text{layer} * C + \text{slot}]$, avoiding repeated binary search and making enumeration $O(\text{count})$ linear.

Sliding-window filtering via `KVCacheReadable`

Ring cache implements `KVCacheReadable`, returning chronological positions. Windowed attention receives `logicalPositions()`, filters to window bounds, applies causal policy. Attention code never sees physical slot order or wraparound; the cache abstraction hides implementation.

Overwrite and inaccessibility

Once full, every append increments `nextSlot` and overwrites the oldest token. Overwritten positions are no longer in the position metadata table. Reads that request an evicted position throw `position-`

NotCached, preventing stale K/V reads and forcing window size enforcement at the application layer.

27.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P026RingKVCacheSolution.swift

- Line 3: public final class RingKVCache: KVCacheWritable {
- Line 14: public init(configuration: KVCacheConfiguration) {
- Line 24: public func storageAddresses() -> (key: UInt, value: UInt) {
- Line 31: public func count(layer: Int) throws -> Int {
- Line 36: public func logicalPositions(layer: Int) throws -> [Int] {
- Line 45: public func append(
- Line 71: public func keyVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
- Line 75: public func valueVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
- Line 79: private func vector(
- Line 93: private func offset(layer: Int, slot: Int, head: Int, feature: Int) -> Int {
- Line 99: public enum P026RingKVCacheSolution {
- Line 100: public static func run(_ request: RingKVCacheRequest) throws -> RingKVCacheResult {

27.18.3 Complete canonical listings

27.18.4 Sources/LeetLLMSolutions/P026RingKVCacheSolution.swift

```

1 import LeetLLMCore
2
3 public final class RingKVCache: KVCacheWritable {
4     public let configuration: KVCacheConfiguration
5     public var allocatedBytes: Int { configuration.allocatedFloat32Bytes }
6
7     private var keyStorage: [Float]
8     private var valueStorage: [Float]
9     private var positions: [Int]
10    private var counts: [Int]
11    private var nextSlots: [Int]
12    private var lastPositions: [Int?]
13
14    public init(configuration: KVCacheConfiguration) {
15        self.configuration = configuration
16        keyStorage = Array(repeating: 0, count: configuration.elementsPerTensor)
17        valueStorage = Array(repeating: 0, count: configuration.elementsPerTensor)
18        positions = Array(repeating: -1, count: configuration.layerCount * configuration.capacity)
19        counts = Array(repeating: 0, count: configuration.layerCount)
20        nextSlots = Array(repeating: 0, count: configuration.layerCount)
21        lastPositions = Array(repeating: nil, count: configuration.layerCount)
22    }
23
24    public func storageAddresses() -> (key: UInt, value: UInt) {
25        (
26            keyStorage.withUnsafeBufferPointer { UInt(bitPattern: $0.baseAddress!) },
27            valueStorage.withUnsafeBufferPointer { UInt(bitPattern: $0.baseAddress!) }
28        )
29    }
30
31    public func count(layer: Int) throws -> Int {
32        try configuration.validate(layer: layer)
33        return counts[layer]
34    }

```

```

35
36 public func logicalPositions(layer: Int) throws -> [Int] {
37     try configuration.validate(layer: layer)
38     let count = counts[layer]
39     let oldestSlot = count == configuration.capacity ? nextSlots[layer] : 0
40     return (0..

```

```

108     layer: request.layer,
109     logicalPosition: request.firstLogicalPosition + token,
110     key: FloatTensor(
111         Array(request.keys.storage[start..

```

Paged KV Allocation

28.1 Why this exists

Full contiguous reservation is simple but ties every sequence to its maximum capacity. Serving systems instead allocate smaller physical pages as sequences grow and return pages when requests finish. Logical sequence order then lives in a page table; adjacent logical pages may be far apart physically.

This batch-one systems lab uses a shared physical page pool across layers to make allocation, exhaustion, free/reuse, fragmentation, and gather behavior observable without pretending to implement a full batched serving scheduler.

28.2 Learning outcomes

You can:

- allocate sequence pages on page boundaries;
- translate logical token positions through a page table;
- free a layer's pages and reuse their physical IDs;
- gather across noncontiguous and out-of-order physical pages;
- report live tokens, free pages, internal fragmentation, and free runs; and
- feed paged reads into the shared cached-attention path.

28.3 Prerequisites

- Problem 022 for cache contracts and logical positions.
- Problem 023 for attention over `KVCacheReadable`.
- Problem 024 for separating logical order from physical layout.

28.4 Vocabulary

- **Page size P** : token slots represented by one physical page.
- **Physical page**: fixed storage block from the shared allocator.
- **Logical page ordinal**: `logicalSlot/P` within one layer's sequence.
- **Page table**: ordered physical page IDs for logical page ordinals.
- **Internal fragmentation**: allocated page slots not holding live tokens.
- **Free run**: consecutive physical page IDs currently available.

28.5 Math from first principles: paged addressing

For a layer whose first logical position is p_0 ,

$$s = p - p_0, \quad j = \lfloor s/P \rfloor, \quad r = s \bmod P.$$

Look up physical page $q = \text{pageTable}[\text{layer}][j]$. For token/head/feature,

$$o = (((qP + r)H_{kv} + h)d_h + d).$$

The judge uses three physical pages of two tokens. Layer zero receives physical page 0, layer one receives 1, and layer zero's second logical page receives 2. After freeing layer one, layer two reuses page 1. Final page tables are

```
1 layer 0: [0, 2]
2 layer 1: []
3 layer 2: [1]
```

Layer zero's logical positions 10...13 cross pages 0 and 2; assuming physical pages 0,1 would read layer two or stale data.

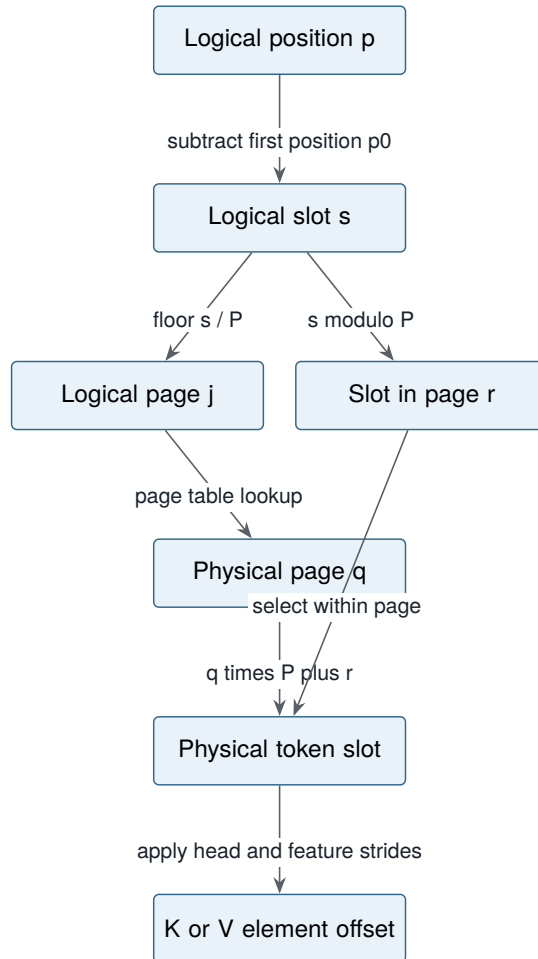


Figure 28.1: Page table translation.

28.6 Shape, layout, and dtype contract

The simplified page model stores all KV heads for P tokens in one sequence page. Physical K and V arrays are contiguous Float32 $[\text{physicalPage}, P, H_{kv}, d_h]$. The configuration remains batch one and fixes

layer count, per-layer logical capacity, heads, and width.

Append vectors are $[H_{kv}, d_h]$, logical positions are nonnegative and sequential per live layer, and append beyond logical capacity throws. A new logical page requires a free physical page or throws `pageUnavaillable`. Free resets that layer's count and position range and makes its pages unaddressable before reuse.

28.7 CPU reference path

Initialize a deterministic free list and fixed physical K/V arrays. On an append where `logicalSlot % P == 0`, pop one physical page and append its ID to the layer's page table. Translate through the page table and copy one K/V token.

On free, return every page ID, clear the table while retaining its array capacity, and reset logical metadata. Reads derive logical slot, page ordinal, slot in page, and physical offset. They never assume physical adjacency.

28.8 Independent correctness method

The judge runs the exact allocation/free sequence above. It requires page tables $[[0, 2], [], [1]]$, gathers distinct values across layer zero's page boundary and layer two's reused page, checks deterministic accounting and 96 allocated bytes, and compares attention over layer zero with a Double materialized oracle.

Focused tests exhaust a one-page pool, free it, and prove another layer reuses physical page zero. An implementation that constructs contiguous page IDs fails.

```
1 swift run leetllm check 027 --cpu
2 swift run leetllm check 027 --solution
```

28.9 Performance, bytes, and fragmentation model

The fixed physical pool uses

$$B = 2N_{pages}PH_{kv}d_h \cdot 4.$$

The judge's $N_{pages}=3, P=2, H_{kv}=1, d_h=2$ pool uses 96 bytes. With A allocated pages and N_{live} tokens,

$$fragmentSlots = AP - N_{live}.$$

Page-table reads add indirection. Smaller pages reduce worst-case internal fragmentation but enlarge page tables and increase allocation frequency. The largest contiguous free run is reported for deterministic allocator analysis; paged reads do not require such a run.

28.10 Metal mapping

This allocator/gather lesson is CPU-only. A paged Metal attention kernel would bind a page table and compute `pageOrdinal`, `physicalPage`, and `slotInPage` inside each K/V load. It would not first concatenate pages on CPU, because that would erase the behavior under study.

The current CPU gather feeding cached attention proves semantic integration. GPU page-table caching, vectorized gathers, and page size are later measured kernel choices, not claims made by this lab.

28.11 Implementation checkpoints

1. Allocate the first page on logical slot zero.
2. Reuse that page for slots inside one page boundary.
3. Allocate another page exactly at the boundary.
4. Exhaust a small pool and throw without partial mutation.
5. Free a layer and reuse its physical page.
6. Gather values through a noncontiguous page table.
7. Run cached attention over the paged reader.

28.12 Controlled experiments

28.12.1 Page-size sweep

Hold live token patterns fixed and sweep P . Prediction: smaller pages reduce internal fragment slots but increase page-table entries and allocation events.

28.12.2 Completion-order fragmentation

Free layers in different orders. Prediction: live logical values are unchanged, while physical page IDs and largest free run can differ under the LIFO allocator.

28.12.3 Gather versus materialize

Compare direct paged reads with copying pages to one contiguous temporary. Prediction: direct gather avoids copy allocation; materialization can repay its cost only if the contiguous result is reused enough times.

28.13 Engine integration

The page allocator is a simplified precursor to per-request paged KV in a serving engine. The attention boundary already depends on logical cache reads, so contiguous and paged stores are interchangeable semantically. A batched extension would give each request/layer its own page table while sharing the physical pool and scheduler.

28.14 Tradeoffs

- Smaller pages reduce fragmentation but increase metadata and allocation frequency.
- LIFO free lists are deterministic and simple; other policies change locality.
- Direct gather preserves paging; materialization trades a copy for contiguous reads.
- This model shares pages across layers but does not implement multi-request batching.

28.15 Hints

- Allocate a page before writing the first slot in that logical page.

- Use page-table order for chronology, never sort physical page IDs.
- Clear logical metadata on free even if physical bytes are left unchanged.
- Compute fragmentation from allocated slots minus live tokens.

28.16 Completion checklist

- ☐ Appends allocate exactly on page boundaries.
- ☐ Exhaustion throws before corrupting tables.
- ☐ Free pages are reused by another layer.
- ☐ Noncontiguous physical pages gather in logical order.
- ☐ Fragmentation, free-run, and byte reports are exact.
- ☐ Paged reads produce the materialized attention result.
- ☐ You ran a page-size, free-order, or gather experiment with a prediction.

28.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

28.17.1 Sources/LeetLLMExercises/P027PagedKVCacheExercise.swift

```

1 import LeetLLMCore
2
3 public enum P027PagedKVCacheExercise {
4     public static func run(_ request: PagedKVCacheRequest) throws -> PagedKVCacheResult {
5         let empty = try (0..

```

28.18 Worked solution

Solution strategy

Partition logical token sequence into pages of size P . Allocate physical pages on-demand from a shared pool. Page table maps logical page ordinal ($\text{floor}(\text{logicalSlot} / P)$) to physical page ID. Multiple layers share the allocator; freeing one layer returns its pages to the pool for reuse. Paged reads translate $\text{logicalPosition} \rightarrow \text{logicalSlot} \rightarrow \text{pageOrdinal} / \text{slotInPage} \rightarrow \text{physical page ID} \rightarrow \text{physical offset}$, gathering across non-contiguous pages transparently to attention.

Correctness invariant

$\text{logicalSlot} = \text{logicalPosition} - \text{firstPosition}$. Logical slots fill contiguously starting from 0. Each new logical page (when $\text{logicalSlot} \% P == 0$) allocates one physical page; allocation fails if no free page exists. Free resets a layer's page table and counts without deallocating the table array itself, preserving capacity for reuse. Physical pages never move; page table is the only address mapping.

Common failure mode

Assuming logical pages are contiguous in physical memory. Logical pages $[0, 1, 2]$ of one layer might use physical pages $[5, 12, 3]$ if the allocator reuses freed pages. Reading logical token at position 11 (in logical page 1, slot 1) must look up $\text{pageTable}[\text{layer}][1]$ to find physical page 12, then compute physical offset—direct table lookup is mandatory, not optional.

28.18.1 Implementation points

Paged addressing formula

$\text{logicalSlot} = p - p_0$, $\text{pageOrdinal} = \text{logicalSlot} / P$, $\text{slotInPage} = \text{logicalSlot} \% P$. $\text{physicalPage} = \text{pageTable}[\text{layer}][\text{pageOrdinal}]$. $\text{Element offset} = ((\text{physicalPage} * P + \text{slotInPage}) * H_{kv} + \text{head}) * d_h + \text{feature}$. Page table lookups are $O(1)$ array access; page IDs are deterministic once allocated.

Physical storage layout and free list

Physical K and V arrays are $[\text{physicalPages}, P, H_{kv}, d_h]$ allocated once at construction. `freePages` is a list of available page IDs, managed as a stack (pop to allocate, append to free). Free operation appends all pages of a layer to `freePages` and clears the page table, enabling reuse without shrinking arrays.

Allocator reporting and fragmentation

`allocatorReport()` computes `liveTokens` (sum of counts across layers), `internalFragmentation` ($\text{allocatedPages} * P - \text{liveTokens}$), and `largestContiguousFreeRun` (longest consecutive free page ID run). Internal fragmentation is unavoidable with partial pages; free runs reveal if the next allocation will succeed.

Exhaustion and error handling

pageUnavailable error thrown if a new logical page is required but freePages is empty. This is the point of paging: exhaustion is detectable before overflowing the pool. A properly integrated serving system calls free() when requests complete to return pages and avoid exhaustion.

28.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P027PagedKVCacheSolution.swift

- Line 3: public final class PagedKVCache: KVCacheWritable {
- Line 20: public init(
- Line 40: public func count(layer: Int) throws -> Int {
- Line 45: public func logicalPositions(layer: Int) throws -> [Int] {
- Line 51: public func physicalPages(layer: Int) throws -> [Int] {
- Line 56: public func append(
- Line 91: public func free(layer: Int) throws {
- Line 100: public func keyVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
- Line 104: public func valueVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
- Line 108: public func allocatorReport() -> PageAllocatorReport {
- Line 129: private func vector(
- Line 149: public enum P027PagedKVCacheSolution {
- Line 150: public static func run(_ request: PagedKVCacheRequest) throws -> PagedKVCacheResult {

28.18.3 Complete canonical listings

28.18.4 Sources/LeetLLMSolutions/P027PagedKVCacheSolution.swift

```

1 import LeetLLMCore
2
3 public final class PagedKVCache: KVCacheWritable {
4     public let configuration: KVCacheConfiguration
5     public let pageSize: Int
6     public let physicalPageCount: Int
7     public var allocatedBytes: Int {
8         physicalPageCount * pageSize * configuration.elementsPerToken * 2
9         * MemoryLayout<Float>.stride
10    }
11
12    private var keyStorage: [Float]
13    private var valueStorage: [Float]
14    private var pageTables: [[Int]]
15    private var freePages: [Int]
16    private var counts: [Int]
17    private var firstPositions: [Int?]
18    private var lastPositions: [Int?]
19
20    public init(
21        configuration: KVCacheConfiguration,
22        pageSize: Int,
23        physicalPageCount: Int
24    ) throws {
25        guard pageSize > 0 else { throw KVCacheError.invalidPageSize(pageSize) }
26        guard physicalPageCount > 0 else { throw KVCacheError.invalidPageCount(physicalPageCount) }
27        self.configuration = configuration
28        self.pageSize = pageSize
29        self.physicalPageCount = physicalPageCount

```

```

30     let elementCount = physicalPageCount * pageSize * configuration.elementsPerToken
31     keyStorage = Array(repeating: 0, count: elementCount)
32     valueStorage = Array(repeating: 0, count: elementCount)
33     pageTables = Array(repeating: [], count: configuration.layerCount)
34     freePages = Array((0..

```

```

103
104 public func valueVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
105     try vector(storage: valueStorage, layer: layer, logicalPosition: logicalPosition, head: head)
106 }
107
108 public func allocatorReport() -> PageAllocatorReport {
109     let allocated = physicalPageCount - freePages.count
110     let liveTokens = counts.reduce(0, +)
111     let sortedFree = freePages.sorted()
112     var largestRun = 0
113     var currentRun = 0
114     var previous: Int?
115     for page in sortedFree {
116         currentRun = previous.map { page == $0 + 1 ? currentRun + 1 : 1 } ?? 1
117         largestRun = max(largestRun, currentRun)
118         previous = page
119     }
120     return PageAllocatorReport(
121         physicalPageCount: physicalPageCount,
122         allocatedPageCount: allocated,
123         freePageCount: freePages.count,
124         liveTokenCount: liveTokens,
125         internalFragmentSlots: allocated * pageSize - liveTokens,
126         largestContiguousFreeRun: largestRun)
127 }
128
129 private func vector(
130     storage: [Float], layer: Int, logicalPosition: Int, head: Int
131 ) throws -> [Float] {
132     try configuration.validate(layer: layer)
133     try configuration.validate(head: head)
134     guard let first = firstPositions[layer] else {
135         throw KVCacheError.positionNotCached(layer: layer, logicalPosition: logicalPosition)
136     }
137     let logicalSlot = logicalPosition - first
138     guard logicalSlot >= 0, logicalSlot < counts[layer] else {
139         throw KVCacheError.positionNotCached(layer: layer, logicalPosition: logicalPosition)
140     }
141     let physicalPage = pageTables[layer][logicalSlot / pageSize]
142     let slotInPage = logicalSlot % pageSize
143     let start = ((physicalPage * pageSize + slotInPage) * configuration.keyValueHeadCount + head)
144         * configuration.headDimension
145     return Array(storage[start..

```

```
176     physicalPageTables: tables,  
177     allocatorReport: cache.allocatorReport(),  
178     attentionOutput: try P023CachedAttentionSolution.attend(  
179         query: request.query,  
180         cache: cache,  
181         layer: request.attentionLayer,  
182         queryLogicalPosition: request.queryLogicalPosition,  
183         queryHeadCount: request.queryHeadCount),  
184     allocatedBytes: cache.allocatedBytes)  
185 }  
186 }
```

Quantized KV Cache

29.1 Why this exists

Long-context decode repeatedly reads K/V and can spend more bandwidth on cache than on arithmetic. Storing INT8 instead of Float32 reduces element bytes, but requires a precise scale convention, metadata, dequantization work, and an explicit output-error budget. A smaller allocation alone is not evidence that the complete path is better.

This lesson implements symmetric INT8 per token/head vector for K and V, keeps their scales independent, reads dequantized values through the shared cache contract, and executes a real Metal kernel that dequantizes inside attention.

29.2 Learning outcomes

You can:

- quantize and dequantize one vector with a defined signed INT8 convention;
- store independent scale metadata for every K and V token/head vector;
- derive bytes including metadata rather than only packed elements;
- run cached attention over dequantized reads on CPU;
- dequantize K/V inside actual MSL attention loads; and
- compare output and storage error with Float cache before interpreting discrepancy.

29.3 Prerequisites

- Problem 011 for precision-boundary reasoning.
- Problem 023 for cached decode and its Float oracle.
- Problem 025 for KV-head-dependent cache bytes.

29.4 Vocabulary

- **Symmetric INT8**: signed integer range around zero with no zero-point.
- **Per-vector scale**: one Float32 scale for one token, one KV head, one K or V vector.
- **Quantized element**: stored Int8 value in $[-127, 127]$.
- **Dequantization**: $\text{Float}(q) * \text{scale}$ at read time.
- **Metadata**: scales required to interpret integer elements.
- **Convention check**: verifying layout, scale, range, and attention math before attributing error.

29.5 Quantization derivation and worked example

For vector x with maximum absolute value a ,

$$s = \begin{cases} 1 & a = 0 \\ a/127 & a > 0 \end{cases}, \quad q_i = \text{clamp}(\text{round}(x_i/s), -127, 127),$$

and

$$\hat{x}_i = q_i s.$$

For $[0.5, -1.0]$, $s=1/127$. The integers are approximately $[64, -127]$, and dequantized values are $[64/127, -1]$. The element error is bounded by about half a scale when inputs and conventions are valid. An all-zero vector uses scale 1, stores zeros, and remains finite.

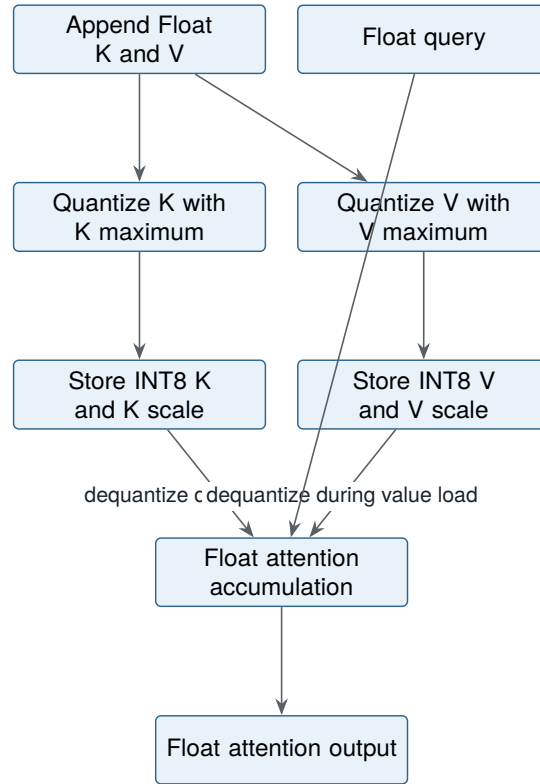


Figure 29.1: Fused quantized kv path.

29.6 Shape, layout, and dtype contract

Batch size is one. Logical append vectors are Float32 $[Hk_v, d_h]$. Quantized K and V element arrays use physical $[L, C, Hk_v, d_h]$ token-major order with Int8 elements. K scales and V scales are separate Float32 arrays $[L, C, Hk_v]$.

Logical positions are sequential as in Problem 022. Dequantized reads return Float32 head vectors. Query and output are Float32 $[H_q, d_h]$; score and online softmax accumulation are Float32 in learner/canonical paths. Invalid shapes, capacity, layers, heads, or positions throw before mutation.

29.7 CPU reference path

For each appended token/head, find max absolute K value and V value separately, store their scales, quantize each feature, then record the logical position. Readable cache methods locate the token/head vector and multiply each integer by its corresponding K or V scale.

The generic cached-attention function consumes those dequantized reads without knowing their physical dtype. The result report materializes dequantized K/V and computes maximum element error against original Float tensors.

29.8 Independent correctness method

The judge compares quantized attention with the independent Float materialized Double oracle using explicit absolute/relative tolerances. It checks K/V round-trip shapes and maximum errors, exact byte accounting, six K scales and six V scales for $T=3, H_{kv}=2$, and evidence that K/V metadata are independent.

The fixture uses different K and V ranges so sharing their scale arrays fails. A focused zero-vector test verifies finite metadata. A wrong implementation that returns exact Float outputs but reports Float storage bytes still fails.

```
1 swift run leetllm check 028 --cpu
2 swift run leetllm check 028 --metal
3 swift run leetllm check 028 --solution
```

29.9 Performance model: bytes, bandwidth, and error

For per-vector Float32 scales, total allocated bytes are

$$B = 2LCH_{kv}d_h \cdot 1 + 2LCH_{kv} \cdot 4.$$

Equivalently, each K/V vector costs d_h INT8 bytes plus one 4-byte scale. The judge's $L=1, C=3, H_{kv}=2, d_h=4$ cache uses 48 element bytes plus 48 scale bytes, or 96 total. At this deliberately tiny width, metadata erases much of the Float32-to-INT8 ratio; larger d_h amortizes scales better.

Read bandwidth falls only if dequantization is fused with the consumer. CPU materialization writes Float temporaries; the Metal path does not. Extra integer conversion and scale loads must be benchmarked against saved bytes.

An output discrepancy is not automatically “quantization noise.” First verify logical positions, head mapping, K/V scale separation, signed range, layout, softmax scaling, and accumulation dtype against the Float path.

29.10 Metal mapping

The grid is $[d_h, H_q]$, matching Problem 023's readable baseline. Each work item maps its query head by division, reads signed char K/V elements from raw cache storage, loads the token/head K and V scales, dequantizes during dot and weighted value accumulation, and writes one output feature.

No Float K/V cache or dequantized temporary buffer is created. The runtime compiles and dispatches the MSL source and checks command completion. Like 023, feature ownership repeats score work; that limitation is explicit rather than hidden by a CPU fallback.

See [P028QuantizedKVCache.metal](#).

29.11 Implementation checkpoints

1. Quantize a two-value vector by hand.
2. Handle an all-zero vector with finite scale and exact zeros.
3. Store K and V scales independently per token/head.
4. Dequantize readable vectors and bound element error.
5. Match Float cached attention on CPU within stated tolerance.
6. Include all scale bytes in the allocation report.
7. Bind INT8 arrays and scales to MSL and match the same judge.

29.12 Controlled experiments

29.12.1 Head-dimension sweep

Sweep dh while retaining one scale per vector. Prediction: metadata fraction $4/(dh+4)$ falls as width grows, improving the storage ratio.

29.12.2 Scale granularity

Compare one scale per vector with smaller feature blocks. Prediction: smaller blocks can lower element error but add metadata and scale loads.

29.12.3 Context sweep

Sweep cached token count with persistent buffers. Prediction: quantized bytes and Float bytes both grow linearly; fused dequantization has more opportunity to repay overhead as cache reads dominate.

29.12.4 Convention fault injection

Deliberately share K/V scales or use unsigned bytes. Prediction: errors become systematic and exceed the per-scale rounding bound, identifying a convention bug rather than an inherent format limit.

29.13 Engine integration

The quantized cache conforms to the same readable/writable boundary as Float, ring, and paged caches. A decoder can select it per model/layer policy while keeping logical positions and attention semantics unchanged. Production use would retain device buffers across steps and choose scale granularity from measured error, bandwidth, and latency.

29.14 Tradeoffs

- INT8 reduces element bytes but adds scale metadata and conversion work.
- Per-vector scales are simple; finer blocks can improve fidelity at metadata cost.
- Fused dequantization avoids Float temporaries; feature-owned baseline repeats scores.
- Float accumulation limits one error source but does not recover discarded precision.

29.15 Hints

- Use $[-127, 127]$ symmetrically; avoid the asymmetric -128 endpoint here.

- Quantize K and V with separate maxima and scale arrays.
- Treat all-zero vectors explicitly to avoid division by zero.
- Verify conventions and intermediate dequantized values before labeling output error.

29.16 Completion checklist

- ☐ Symmetric INT8 quantize/dequantize follows the explicit formula.
- ☐ K and V own independent per-token/head scales.
- ☐ Byte totals include all scale metadata.
- ☐ Dequantized element errors satisfy the stated bounds.
- ☐ CPU and actual Metal attention stay within the Float-oracle tolerance.
- ☐ Convention checks precede any interpretation of discrepancy.
- ☐ You ran a width, granularity, context, or fault experiment with a prediction.

29.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

29.17.1 Sources/LeetLLMExercises/P028QuantizedKVCacheExercise.swift

```

1 import LeetLLMCore
2
3 public enum P028QuantizedKVCacheExercise {
4     public static func run(_ request: CachedAttentionRequest) throws -> QuantizedKVCacheResult {
5         let zeroOutput = try FloatTensor(
6             Array(repeating: 0, count: request.query.elementCount), shape: request.query.shape)
7         return QuantizedKVCacheResult(
8             attentionOutput: zeroOutput,
9             dequantizedKeys: request.keys,
10            dequantizedValues: request.values,
11            keyScales: [],
12            valueScales: [],
13            allocatedBytes: request.cacheConfiguration.allocatedFloat32Bytes,
14            maximumKeyError: 0,
15            maximumValueError: 0)
16     }
17 }

```

29.17.2 Sources/LeetLLMExercises/P028QuantizedKVCacheMetal.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P028QuantizedKVCacheExercise {
5     public static func makeMetalPipeline() throws -> MetalQuantizedCachedAttentionPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P028QuantizedKVCache", withExtension: "metal", subdirectory: "Metal")
8         else { throw MetalNeuralOperatorError.kernelResourceMissing("P028QuantizedKVCache.metal") }
9         return try MetalQuantizedCachedAttentionPipeline(
10             source: String(contentsOf: url, encoding: .utf8))
11     }
12
13     public static func runMetal(
14         _ request: CachedAttentionRequest,
15         pipeline: MetalQuantizedCachedAttentionPipeline
16     ) throws -> QuantizedKVCacheResult {
17         let vectorCount = request.cacheConfiguration.layerCount
18             * request.cacheConfiguration.capacity
19             * request.cacheConfiguration.keyValueHeadCount
20         let output = try pipeline.apply(
21             query: request.query,
22             keyStorage: Array(repeating: 0, count: request.cacheConfiguration.elementsPerTensor),
23             valueStorage: Array(repeating: 0, count: request.cacheConfiguration.elementsPerTensor),
24             keyScales: Array(repeating: 1, count: vectorCount),
25             valueScales: Array(repeating: 1, count: vectorCount),
26             configuration: request.cacheConfiguration,
27             queryHeadCount: request.attentionConfiguration.queryHeadCount,
28             layer: request.layer,
29             tokenCount: request.tokenCount)
30         return QuantizedKVCacheResult(
31             attentionOutput: output,
32             dequantizedKeys: request.keys,
33             dequantizedValues: request.values,
34             keyScales: [],
35             valueScales: [],

```

```

36     allocatedBytes: request.cacheConfiguration.allocatedFloat32Bytes,
37     maximumKeyError: 0,
38     maximumValueError: 0)
39 }
40 }

```

29.17.3 Sources/LeetLLMExercises/Metal/P028QuantizedKVCache.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void quantized_cached_decode_attention(
5     device const float *query [[buffer(0)]],
6     device const char *keys [[buffer(1)]],
7     device const char *values [[buffer(2)]],
8     device const float *keyScales [[buffer(3)]],
9     device const float *valueScales [[buffer(4)]],
10    device float *output [[buffer(5)]],
11    constant uint4 &shape [[buffer(6)]],
12    constant uint4 &dimensions [[buffer(7)]],
13    uint2 index [[thread_position_in_grid]]) {
14    if (index.x >= dimensions.y || index.y >= shape.w) return;
15    output[index.y * dimensions.y + index.x] = 0.0f;
16 }

```

29.18 Worked solution

Solution strategy

Store K and V as INT8 with per-vector Float32 scales. For each appended K or V token/head vector, find its maximum absolute value, compute $\text{scale} = \max / 127$ (or 1 if $\max=0$), quantize each element $q_i = \text{clamp}(\text{round}(x_i / \text{scale}), -127, 127)$, and store both q_i and scale. On read, dequantize by multiplying $q_i * \text{scale}$, returning Float32. Metal kernel dequantizes on-the-fly during score and value accumulation, avoiding materialized dequantized storage.

Correctness invariant

K and V scales are independent (stored separately) to avoid coupling their dynamic ranges. Quantized elements are clamped to $[-127, 127]$. Dequantization returns Float32, seamlessly feeding into Float32 attention accumulation. Scale array has shape $[L, C, Hkv]$ with one entry per token/head vector (6 entries for $T=3, Hkv=2$ per K and per V).

Common failure mode

Sharing K and V scales (using one scale per token/head for both K and V). If K ranges $[-2, 2]$ ($\text{scale}=2/127$) and V ranges $[-10, 10]$ ($\text{scale}=10/127$), a shared scale over-quantizes one and under-quantizes the other, losing precision or saturating. Per-vector independence is mandatory to preserve both K and V fidelity.

29.18.1 Implementation points

Symmetric INT8 quantization formula

$\text{scale} = (\max(|x|) == 0) ? 1 : \max(|x|) / 127$. All-zero vectors use scale 1 and stay zero. Maximum element-wise error is bounded by $\sim 0.5 * \text{scale}$ due to rounding. For $x=[0.5, -1.0]$, $\text{scale}=1/127$, $q=[64, -127]$, $x_{\text{hat}}=[0.503, -1.0]$ —less than 1% error per element.

Byte accounting and metadata overhead

INT8 elements: $L * C * Hkv * dh$ bytes. K scales: $L * C * Hkv * 4$ bytes (Float32). V scales: $L * C * Hkv * 4$ bytes. Total INT8 cache $\approx 0.5x$ Float32 element bytes plus $2x$ scale metadata. For large dh , element savings dominate; for small dh , metadata overhead is significant.

Metal dequantization during attention

Metal kernel loads $\text{Int8 } k[d]$ and k_scale , computes $\text{Float32}(k[d]) * k_scale$ on-the-fly during score $= \text{sum}(\text{query}[d] * \text{dequantized_k}[d])$. No intermediate dequantized K/V buffer needed, amortizing scale multiplication across the dot product and saving GPU memory.

Error validation and convention checking

Result compares max element-wise error against original Float K/V. Expected error should be low ($< 1/127 * \text{max_range}$) if conventions are correct. Large errors often indicate a convention bug (e.g.,

wrong scale formula, norm convention mismatch) rather than inherent quantization loss; verify against Problem 011 before attributing to quantization.

29.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P028QuantizedKVCacheMetal.swift

- Line 4: extension P028QuantizedKVCacheSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalQuantizedCachedAttentionPipeline {

Sources/LeetLLMSolutions/P028QuantizedKVCacheSolution.swift

- Line 3: public final class QuantizedKVCache: KVCacheWritable {
- Line 20: public init(configuration: KVCacheConfiguration) {
- Line 33: public func count(layer: Int) throws -> Int {
- Line 38: public func logicalPositions(layer: Int) throws -> [Int] {
- Line 44: public func append(
- Line 80: public func keyVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
- Line 86: public func valueVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
- Line 92: public func rawKeyStorage() -> [Int8] { keyStorage }
- Line 93: public func rawValueStorage() -> [Int8] { valueStorage }
- Line 94: public func rawKeyScales() -> [Float] { keyScales }
- Line 95: public func rawValueScales() -> [Float] { valueScales }
- Line 97: private func quantize(
- Line 113: private func dequantize(
- Line 134: private func vectorOffset(layer: Int, slot: Int, head: Int) -> Int {
- Line 138: private func elementOffset(layer: Int, slot: Int, head: Int) -> Int {
- Line 157: public static func runMetal(
- Line 177: private static func populatedCache(_ request: CachedAttentionRequest) throws -> Quantized-KVCache {
- Line 198: private static func result(

Sources/LeetLLMSolutions/Metal/P028QuantizedKVCache.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

29.18.3 Complete canonical listings

29.18.4 Sources/LeetLLMSolutions/P028QuantizedKVCacheMetal.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P028QuantizedKVCacheSolution {
5     public static func makeMetalPipeline() throws -> MetalQuantizedCachedAttentionPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P028QuantizedKVCache", withExtension: "metal", subdirectory: "Metal")
8         else { throw MetalNeuralOperatorError.kernelResourceMissing("P028QuantizedKVCache.metal") }
9         return try MetalQuantizedCachedAttentionPipeline(
10             source: String(contentsOf: url, encoding: .utf8))
11     }
12 }

```


29.18.5 Sources/LeetLLMSolutions/P028QuantizedKVCacheSolution.swift

```

1 import LeetLLMCore
2
3 public final class QuantizedKVCache: KVCacheWritable {
4     public let configuration: KVCacheConfiguration
5     public var allocatedBytes: Int {
6         keyStorage.count * MemoryLayout<Int8>.stride
7         + valueStorage.count * MemoryLayout<Int8>.stride
8         + keyScales.count * MemoryLayout<Float>.stride
9         + valueScales.count * MemoryLayout<Float>.stride
10    }
11
12    private var keyStorage: [Int8]
13    private var valueStorage: [Int8]
14    private var keyScales: [Float]
15    private var valueScales: [Float]
16    private var positions: [Int]
17    private var counts: [Int]
18    private var firstPositions: [Int?]
19
20    public init(configuration: KVCacheConfiguration) {
21        self.configuration = configuration
22        keyStorage = Array(repeating: 0, count: configuration.elementsPerTensor)
23        valueStorage = Array(repeating: 0, count: configuration.elementsPerTensor)
24        let vectorCount = configuration.layerCount * configuration.capacity
25            * configuration.keyValueHeadCount
26        keyScales = Array(repeating: 1, count: vectorCount)
27        valueScales = Array(repeating: 1, count: vectorCount)
28        positions = Array(repeating: -1, count: configuration.layerCount * configuration.capacity)
29        counts = Array(repeating: 0, count: configuration.layerCount)
30        firstPositions = Array(repeating: nil, count: configuration.layerCount)
31    }
32
33    public func count(layer: Int) throws -> Int {
34        try configuration.validate(layer: layer)
35        return counts[layer]
36    }
37
38    public func logicalPositions(layer: Int) throws -> [Int] {
39        try configuration.validate(layer: layer)
40        let start = layer * configuration.capacity
41        return Array(positions[start..

```

```

71     offset: destination,
72     scales: &valueScales,
73     scaleIndex: scaleIndex)
74 }
75 positions[layer * configuration.capacity + slot] = logicalPosition
76 if firstPositions[layer] == nil { firstPositions[layer] = logicalPosition }
77 counts[layer] += 1
78 }
79
80 public func keyVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
81     try dequantize(
82         storage: keyStorage, scales: keyScales,
83         layer: layer, logicalPosition: logicalPosition, head: head)
84 }
85
86 public func valueVector(layer: Int, logicalPosition: Int, head: Int) throws -> [Float] {
87     try dequantize(
88         storage: valueStorage, scales: valueScales,
89         layer: layer, logicalPosition: logicalPosition, head: head)
90 }
91
92 public func rawKeyStorage() -> [Int8] { keyStorage }
93 public func rawValueStorage() -> [Int8] { valueStorage }
94 public func rawKeyScales() -> [Float] { keyScales }
95 public func rawValueScales() -> [Float] { valueScales }
96
97 private func quantize(
98     _ values: [Float],
99     into storage: inout [Int8],
100     offset: Int,
101     scales: inout [Float],
102     scaleIndex: Int
103 ) {
104     let maximum = values.map { abs($0) }.max() ?? 0
105     let scale = maximum == 0 ? 1 : maximum / 127
106     scales[scaleIndex] = scale
107     for index in values.indices {
108         let rounded = (values[index] / scale).rounded()
109         storage[offset + index] = Int8(max(-127, min(127, Int(rounded))))
110     }
111 }
112
113 private func dequantize(
114     storage: [Int8],
115     scales: [Float],
116     layer: Int,
117     logicalPosition: Int,
118     head: Int
119 ) throws -> [Float] {
120     try configuration.validate(layer: layer)
121     try configuration.validate(head: head)
122     guard let first = firstPositions[layer] else {
123         throw KVCacheError.positionNotCached(layer: layer, logicalPosition: logicalPosition)
124     }
125     let slot = logicalPosition - first
126     guard slot >= 0, slot < counts[layer],
127         positions[layer * configuration.capacity + slot] == logicalPosition
128     else { throw KVCacheError.positionNotCached(layer: layer, logicalPosition: logicalPosition) }
129     let start = elementOffset(layer: layer, slot: slot, head: head)
130     let scale = scales[vectorOffset(layer: layer, slot: slot, head: head)]
131     return (0..

```

```

144 public static func run(_ request: CachedAttentionRequest) throws -> QuantizedKVCacheResult {
145     let cache = try populatedCache(request)
146     return try result(
147         request: request,
148         cache: cache,
149         output: P023CachedAttentionSolution.attend(
150             query: request.query,
151             cache: cache,
152             layer: request.layer,
153             queryLogicalPosition: request.queryLogicalPosition,
154             queryHeadCount: request.attentionConfiguration.queryHeadCount))
155 }
156
157 public static func runMetal(
158     _ request: CachedAttentionRequest,
159     pipeline: MetalQuantizedCachedAttentionPipeline
160 ) throws -> QuantizedKVCacheResult {
161     let cache = try populatedCache(request)
162     return try result(
163         request: request,
164         cache: cache,
165         output: pipeline.apply(
166             query: request.query,
167             keyStorage: cache.rawKeyStorage(),
168             valueStorage: cache.rawValueStorage(),
169             keyScales: cache.rawKeyScales(),
170             valueScales: cache.rawValueScales(),
171             configuration: cache.configuration,
172             queryHeadCount: request.attentionConfiguration.queryHeadCount,
173             layer: request.layer,
174             tokenCount: request.tokenCount))
175 }
176
177 private static func populatedCache(_ request: CachedAttentionRequest) throws -> QuantizedKVCache {
178     let cache = QuantizedKVCache(configuration: request.cacheConfiguration)
179     let elements = request.cacheConfiguration.elementsPerToken
180     for token in 0..

```

```

217 }
218 }

```

29.18.6 Sources/LeetLLMSolutions/Metal/P028QuantizedKVCache.metal

```

1  #include <metal_stdlib>
2  using namespace metal;
3
4  kernel void quantized_cached_decode_attention(
5      device const float *query [[buffer(0)]],
6      device const char *keys [[buffer(1)]],
7      device const char *values [[buffer(2)]],
8      device const float *keyScales [[buffer(3)]],
9      device const float *valueScales [[buffer(4)]],
10     device float *output [[buffer(5)]],
11     constant uint4 &shape [[buffer(6)]],
12     constant uint4 &dimensions [[buffer(7)]],
13     uint2 index [[thread_position_in_grid]]) {
14     uint feature = index.x;
15     uint queryHead = index.y;
16     if (feature >= dimensions.y || queryHead >= shape.w) return;
17
18     uint kvHead = queryHead / dimensions.z;
19     float maximum = -INFINITY;
20     float denominator = 0.0f;
21     float weightedValue = 0.0f;
22     for (uint token = 0; token < shape.z; ++token) {
23         uint vectorIndex = ((shape.x * shape.y + token) * dimensions.x + kvHead);
24         uint elementBase = vectorIndex * dimensions.y;
25         float keyScale = keyScales[vectorIndex];
26         float valueScale = valueScales[vectorIndex];
27         float score = 0.0f;
28         for (uint d = 0; d < dimensions.y; ++d) {
29             score += query[queryHead * dimensions.y + d]
30                 * float(keys[elementBase + d]) * keyScale;
31         }
32         score *= rsqrt(float(dimensions.y));
33         float nextMaximum = max(maximum, score);
34         float alpha = isfinite(maximum) ? exp(maximum - nextMaximum) : 0.0f;
35         float beta = exp(score - nextMaximum);
36         denominator = denominator * alpha + beta;
37         weightedValue = weightedValue * alpha
38             + beta * float(values[elementBase + feature]) * valueScale;
39         maximum = nextMaximum;
40     }
41     output[queryHead * dimensions.y + feature] = weightedValue / denominator;
42 }

```

Part VI

Weight Quantization

Symmetric INT8 Quantization

30.1 Why this exists

Model weights are commonly stored with fewer than 32 bits per value, but an integer array is not a usable weight tensor by itself. The signed range, rounding rule, scale, zero handling, shape, and reconstruction error are all part of the representation. If any one differs between conversion and loading, the model computes with different weights.

This lesson starts weight quantization with one scale for an entire Float32 tensor. It is intentionally CPU-only: there is no useful GPU consumer yet, so a Metal dispatch would only copy or dequantize data without teaching a real engine path. Problem 033 will consume packed weights directly in MSL.

30.2 Learning outcomes

You can:

- state the exact symmetric INT8 range and explain why -128 is excluded;
- compute a finite scale for nonzero and all-zero tensors;
- apply the same deterministic tie rule used by later converters;
- reject NaN and infinity before deriving metadata;
- reconstruct Float32 values and calculate maximum absolute error and RMSE; and
- count integer payload bytes and Float32 scale bytes separately.

30.3 Prerequisites

- Problem 002 for contiguous tensor shape and storage contracts.
- Problem 006 for honest byte accounting.
- Problem 011 for reasoning about deliberate precision boundaries.
- Problem 028 for the distinction between a quantization convention and an unexplained output discrepancy.

30.4 Vocabulary

- **Quantization:** mapping a Float32 value to a member of a finite integer set.
- **Dequantization:** reconstructing an approximate Float32 value from an integer and scale.
- **Symmetric range:** integer endpoints with equal magnitude around zero.
- **Scale:** the Float32 step represented by one integer increment.
- **Saturation:** clamping a rounded value to the representable integer range.
- **RMSE:** root mean square element error, which weights all errors rather than only the largest.

30.5 Derivation and worked numbers

The format uses Int8 values in $[-127, 127]$. The bit pattern for -128 exists, but it is rejected so the positive and negative magnitudes share one denominator. For finite input values x_i ,

$$a = \max_i |x_i|, \quad s = \begin{cases} 1 & a = 0 \\ a/127 & a > 0 \end{cases}.$$

Quantization and reconstruction are

$$q_i = \text{clamp} \left(\text{round}_{\text{nearest, away on ties}}(x_i/s), -127, 127 \right), \quad \hat{x}_i = q_i s.$$

Swift spells the tie rule as `rounded(.toNearestOrAwayFromZero)`. Conversion happens on CPU in this module; later Metal kernels only decode stored integers, so they cannot silently choose a different rounding mode.

For $x = [-2, -1, 0, 1, 2]$, $s = 2/127$. The two halfway values are $-1/s = -63.5$ and $1/s = 63.5$, so ties move away from zero:

1	x	-2	-1	0	1	2
2	q	-127	-64	0	64	127
3	x_hat	-2.0	-128/127	0	128/127	2.0

The maximum error is $1/127$. For an all-zero tensor, $\text{scale} = 1$, every integer is zero, and reconstruction is exact. This special case avoids division by zero without introducing a scale that is NaN or zero.

The reported metrics are

$$E_{\max} = \max_i |x_i - \hat{x}_i|, \quad E_{\text{RMSE}} = \sqrt{\frac{1}{N} \sum_i (x_i - \hat{x}_i)^2}.$$

30.6 Shape, layout, and dtype contract

Input and dequantized output are contiguous row-major `FloatTensor` values with the same arbitrary-rank shape. Quantized storage is a flat `[Int8]` in that same logical order plus one `Float32` scale. Accumulation for judge metrics uses `Double`; stored metrics are `Float`.

`SymmetricInt8Tensor` validates shape element count, positive finite scale, and the $[-127, 127]$ payload range. The operation rejects each non-finite input with its flat index before computing a maximum.

30.7 CPU reference path

1. Validate every `Float32` element is finite.
2. Reduce the maximum absolute value.
3. Select 1 for an all-zero scale or divide the maximum by 127.
4. Divide, round to nearest with ties away from zero, saturate, and store `Int8`.
5. Multiply each stored integer by the scale into a new `Float` tensor.
6. Compare source and reconstruction using an independent `Double` metric pass.

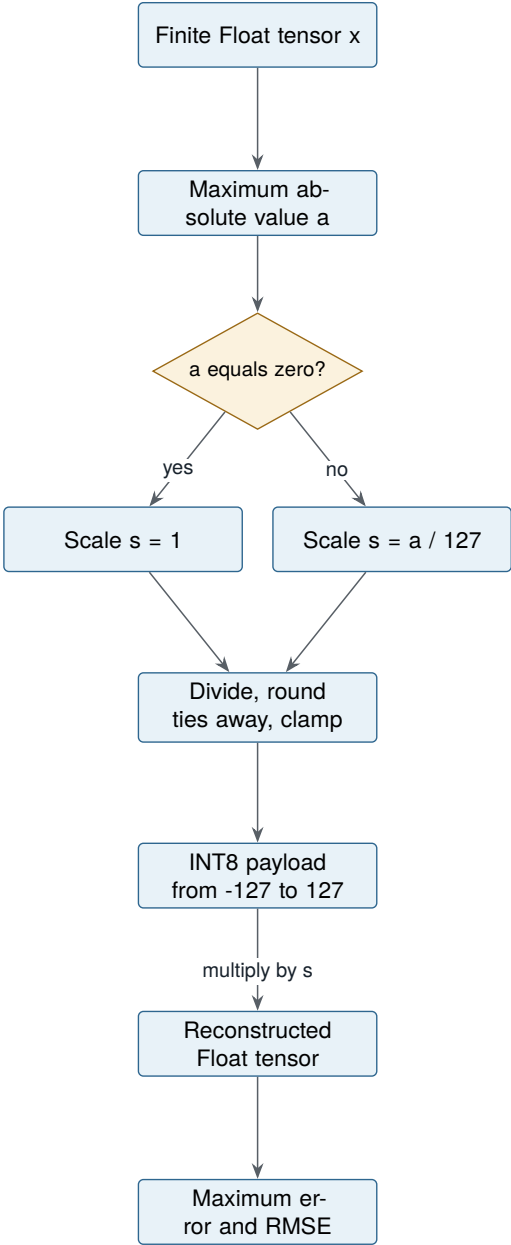


Figure 30.1: Symmetric int8 convention.

The starter performs all validation and returns correctly shaped storage, but its zero payload fails the nonzero fixture. That leaves a meaningful computation to implement while keeping setup errors out of the way.

30.8 Independent correctness

The judge derives expected bytes and reconstructed values independently. Its halfway fixture rejects truncation, banker's rounding, an asymmetric -128 endpoint, or a missing scale. A separate all-zero fixture requires exact zeros and finite scale 1; a non-finite fixture must throw. Tests also construct a format directly and verify that -128 is invalid.

```
1 swift run leetllm check 029 --cpu
2 swift run leetllm check 029 --solution
```

30.9 Performance model: bytes and arithmetic intensity

For N values, this format occupies

$$B_{\text{INT8}} = N + 4$$

bytes, compared with $B_{\text{FP32}} = 4N$. The scale is a real four-byte metadata cost. At $N=5$, the lesson fixture uses 9 bytes rather than 20; at large N , the ratio approaches one quarter.

Quantization reads $4N$ bytes and writes roughly $N+4$; dequantization reads $N+4$ and writes $4N$. Each pass does only a few operations per element, so it is traffic-heavy. Persisting quantized weights matters more than repeatedly converting them during inference.

30.10 Metal mapping

There is no Metal stage in 029. A standalone GPU dequantizer would allocate the Float32 tensor that quantization is intended to avoid. The first useful Metal mapping is 033, where unpacking and scaling happen inside GEMV accumulation.

30.11 Implementation checkpoints

1. Reject one NaN and one infinity.
2. Make all-zero input produce scale 1 and exact zeros.
3. Reproduce the -63.5 -> -64 and 63.5 -> 64 ties.
4. Reject a manually constructed `Int8.min` payload.
5. Match reconstruction shape and values.
6. Match independent maximum error and RMSE.

30.12 Controlled experiments

30.12.1 Range sweep

Keep tensor length fixed and increase one outlier. Prediction: the scale grows, so reconstruction error for small values grows even when their values do not.

30.12.2 Tie-rule injection

Replace the specified rounding with truncation. Prediction: positive and negative halfway values both fail the byte fixture, and the error has a directional bias rather than an unexplained random pattern.

30.12.3 Tensor-size sweep

Measure metadata fraction $4/(N+4)$. Prediction: the scale dominates tiny tensors and becomes negligible for full weight matrices.

30.13 Engine integration

This format is the conversion primitive used to establish range, rounding, finite-value, metric, and byte conventions. Problem 030 changes only scale granularity for [out,in] weights. Problems 031-033 change payload width and when reconstruction occurs, while retaining these validation principles.

30.14 Tradeoffs

- One scale minimizes metadata but lets one outlier set resolution everywhere.
- Excluding -128 simplifies a symmetric convention but leaves one bit pattern unused.
- Float32 scales are simple and portable; narrower scale types would save metadata with another error source.
- Preconversion saves runtime work; dynamic quantization can adapt to changing activations but is outside this weight lesson.

30.15 Hints

- Compute abs only after rejecting non-finite values.
- Handle `maximum == 0` before division.
- Clamp after rounding, even if the maximum-derived scale should already fit.
- Calculate metrics from reconstructed values, not from an assumed half-scale bound.

30.16 Completion checklist

- ☐ Stored integers are restricted to $[-127, 127]$.
- ☐ Rounding is nearest with ties away from zero.
- ☐ All-zero and non-finite inputs have explicit behavior.
- ☐ Reconstruction preserves shape and logical order.
- ☐ Maximum error and RMSE match an independent pass.
- ☐ Byte totals include the Float32 scale.
- ☐ An experiment prediction was written before measurement.

30.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

30.17.1 Sources/LeetLLMExercises/P029SymmetricInt8Exercise.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P029SymmetricInt8Exercise {
5     public static func quantize(
6         _ input: FloatTensor
7     ) throws -> SymmetricInt8QuantizationResult {
8         for (index, value) in input.storage.enumerated() where !value.isFinite {
9             throw WeightQuantizationError.nonFiniteValue(index: index, value: value)
10        }
11        let quantized = try SymmetricInt8Tensor(
12            values: Array(repeating: 0, count: input.elementCount),
13            shape: input.shape,
14            scale: 1)
15        let dequantized = try FloatTensor(
16            Array(repeating: 0, count: input.elementCount), shape: input.shape)
17        let maximum = input.storage.map(abs).max() ?? 0
18        let sumSquares = input.storage.reduce(0.0) { $0 + Double($1) * Double($1) }
19        return SymmetricInt8QuantizationResult(
20            quantized: quantized,
21            dequantized: dequantized,
22            error: QuantizationErrorMetrics(
23                maximumAbsoluteError: maximum,
24                rootMeanSquareError: input.storage.isEmpty
25                    ? 0 : Float(sqrt(sumSquares / Double(input.elementCount))))
26    }
27 }

```

30.18 Worked solution

Solution strategy

Quantize a Float32 tensor to INT8 using symmetric range $[-127, 127]$ by finding the maximum absolute value, deriving scale as $\max/127$ (or 1.0 for all-zero tensors to avoid NaN), applying deterministic rounding using nearest-with-ties-away-from-zero, clamping to range, then dequantizing by multiplying reconstructed integers by the scale. The implementation in `WeightQuantizationSolutionSupport.quantizeInt8` validates finite input, computes maximum absolute value, applies the rounding rule from `QuantizationRounding.nearestAwayFromZero`, and produces a `SymmetricInt8Tensor` storing values and scale separately.

Correctness invariant

For all-zero input, scale equals 1.0 and all quantized values are zero. For nonzero input, the maximum absolute value maps exactly to ± 127 . Reconstruction error is bounded by $\text{scale}/2$ for any value, and RMSE is computed over the full tensor using Double precision to avoid accumulation rounding.

Common failure mode

Excluding -128 from the range appears arbitrary but is essential: it preserves symmetric magnitude around zero so both positive and negative extrema map to the same denominator. Using nearest-to-even or truncation instead of away-from-zero introduces systematic bias. Omitting the all-zero case causes division by zero or NaN scales.

30.18.1 Implementation points

Symmetric range excludes -128 by design

The range $[-127, 127]$ has 255 representable values with equal magnitude above and below zero. Using -128 would create asymmetry: no +128 counterpart. This matters because $\text{scale} = \max / 127$ scales the entire range uniformly.

Finite checks precede quantization

`WeightQuantizationSolutionSupport.validateFinite` iterates the input array and rejects any NaN or infinity before computing the maximum. This prevents silent propagation of invalid inputs into scale derivation.

Ties always move away from zero

The rounding rule `nearestAwayFromZero` rounds $x.5$ toward $\pm\infty$, so $63.5 \rightarrow 64$ and $-63.5 \rightarrow -64$. This deterministic tie-breaking is baked into `QuantizationRounding` and must match both the converter and any downstream decoder.

Metrics use Double accumulation

Maximum absolute error is tracked as Double during comparison, then stored as Float. RMSE sums squared differences in Double before taking sqrt and converting to Float. This avoids single-precision rounding bias in error measurement.

All-zero tensors return scale 1.0

When maximum is zero, scale is explicitly set to 1 rather than 0/127. Every quantized value remains zero, and dequantization $0 * 1 = 0$ is exact. This special case avoids division by zero and allows correct reconstruction of sparse or zero-initialized tensors.

30.18.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P029SymmetricInt8Solution.swift

- Line 3: public enum P029SymmetricInt8Solution {
- Line 4: public static func quantize(

Sources/LeetLLMSolutions/WeightQuantizationSolutionSupport.swift

- Line 4: enum WeightQuantizationSolutionSupport {
- Line 5: static func validateFinite(_ values: [Float]) throws {
- Line 11: static func quantizeInt8(_ input: FloatTensor) throws -> SymmetricInt8Tensor {
- Line 22: static func dequantize(_ input: SymmetricInt8Tensor) throws -> FloatTensor {
- Line 26: static func metrics(
- Line 42: static func validateWeightMatrix(_ weights: FloatTensor, groupSize: Int) throws {
- Line 50: static func quantizeGroupwiseInt8(
- Line 86: static func dequantize(_ input: GroupwiseInt8WeightMatrix) throws -> FloatTensor {
- Line 99: static func quantizeGroupwiseQ4(
- Line 134: static func dequantize(_ input: GroupwiseQ4WeightMatrix) throws -> FloatTensor {
- Line 148: static func packQ4(

30.18.3 Complete canonical listings**30.18.4 Sources/LeetLLMSolutions/P029SymmetricInt8Solution.swift**

```

1 import LeetLLMCore
2
3 public enum P029SymmetricInt8Solution {
4     public static func quantize(
5         _ input: FloatTensor
6     ) throws -> SymmetricInt8QuantizationResult {
7         let quantized = try WeightQuantizationSolutionSupport.quantizeInt8(input)
8         let dequantized = try WeightQuantizationSolutionSupport.dequantize(quantized)
9         return SymmetricInt8QuantizationResult(
10             quantized: quantized,
11             dequantized: dequantized,
12             error: WeightQuantizationSolutionSupport.metrics(
13                 reference: input.storage, candidate: dequantized.storage))
14     }
15 }

```

30.18.5 Sources/LeetLLMSolutions/WeightQuantizationSolutionSupport.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 enum WeightQuantizationSolutionSupport {
5     static func validateFinite(_ values: [Float]) throws {
6         for (index, value) in values.enumerated() where !value.isFinite {
7             throw WeightQuantizationError.nonFiniteValue(index: index, value: value)
8         }
9     }
10
11     static func quantizeInt8(_ input: FloatTensor) throws -> SymmetricInt8Tensor {
12         try validateFinite(input.storage)
13         let maximum = input.storage.reduce(Float.zero) { max($0, abs($1)) }
14         let scale: Float = maximum == 0 ? 1 : maximum / 127
15         let values = input.storage.map { value -> Int8 in
16             let rounded = QuantizationRounding.nearestAwayFromZero(value / scale)
17             return Int8(max(-127, min(127, Int(rounded))))
18         }
19         return try SymmetricInt8Tensor(values: values, shape: input.shape, scale: scale)
20     }
21
22     static func dequantize(_ input: SymmetricInt8Tensor) throws -> FloatTensor {
23         try FloatTensor(input.values.map { Float($0) * input.scale }, shape: input.shape)
24     }
25
26     static func metrics(
27         reference: [Float],
28         candidate: [Float]
29     ) -> QuantizationErrorMetrics {
30         var maximum = 0.0
31         var sumSquares = 0.0
32         for (expected, actual) in zip(reference, candidate) {
33             let difference = Double(expected) - Double(actual)
34             maximum = max(maximum, abs(difference))
35             sumSquares += difference * difference
36         }
37         let rmse = reference.isEmpty ? 0 : sqrt(sumSquares / Double(reference.count))
38         return QuantizationErrorMetrics(
39             maximumAbsoluteError: Float(maximum), rootMeanSquareError: Float(rmse))
40     }
41
42     static func validateWeightMatrix(_ weights: FloatTensor, groupSize: Int) throws {
43         guard weights.rank == 2 else {
44             throw TensorError.rankMismatch(expected: 2, actual: weights.rank)
45         }
46         guard groupSize > 0 else { throw WeightQuantizationError.invalidGroupSize(groupSize) }
47         try validateFinite(weights.storage)
48     }
49
50     static func quantizeGroupwiseInt8(
51         _ weights: FloatTensor,
52         groupSize: Int
53     ) throws -> GroupwiseInt8WeightMatrix {
54         try validateWeightMatrix(weights, groupSize: groupSize)
55         let outputChannels = weights.shape[0]
56         let inputChannels = weights.shape[1]
57         let groups = inputChannels == 0 ? 0 : (inputChannels + groupSize - 1) / groupSize
58         var values = Array(repeating: Int8.zero, count: weights.elementCount)
59         var scales = Array(repeating: Float.zero, count: outputChannels * groups)
60         for output in 0..

```

```

71     let source = weights.storage[output * inputChannels + input]
72     let rounded = QuantizationRounding.nearestAwayFromZero(source / scale)
73     values[output * inputChannels + input] = Int8(
74         max(-127, min(127, Int(rounded))))
75     }
76 }
77 }
78 return try GroupwiseInt8WeightMatrix(
79     outputChannels: outputChannels,
80     inputChannels: inputChannels,
81     groupSize: groupSize,
82     values: values,
83     scales: scales)
84 }
85
86 static func dequantize(_ input: GroupwiseInt8WeightMatrix) throws -> FloatTensor {
87     var values = Array(repeating: Float.zero, count: input.values.count)
88     for output in 0..

```

```

144     }
145     return try FloatTensor(values, shape: input.shape)
146 }
147
148 static func packQ4(
149     _ values: [Int8],
150     outputChannels: Int,
151     inputChannels: Int,
152     groupSize: Int,
153     scales: [Float]
154 ) throws -> GroupwiseQ4WeightMatrix {
155     let (logicalCount, overflow) = outputChannels.multipliedReportingOverflow(by: inputChannels)
156     guard !overflow else { throw WeightQuantizationError.sizeOverflow }
157     guard values.count == logicalCount else {
158         throw WeightQuantizationError.storageCountMismatch(
159             name: "Q4 logical values", expected: logicalCount, actual: values.count)
160     }
161     for (index, value) in values.enumerated()
162     where value < GroupwiseQ4WeightMatrix.minimumStoredValue
163         || value > GroupwiseQ4WeightMatrix.maximumStoredValue
164     {
165         throw WeightQuantizationError.quantizedValueOutOfRange(
166             index: index,
167             value: Int(value),
168             minimum: GroupwiseQ4WeightMatrix.minimumStoredValue,
169             maximum: GroupwiseQ4WeightMatrix.maximumStoredValue)
170     }
171     var packed = Array(repeating: UInt8.zero, count: logicalCount / 2 + logicalCount % 2)
172     for (index, value) in values.enumerated() {
173         let nibble = UInt8(bitPattern: value) & 0x0f
174         if index.isMultiple(of: 2) {
175             packed[index / 2] = nibble
176         } else {
177             packed[index / 2] |= nibble << 4
178         }
179     }
180     return try GroupwiseQ4WeightMatrix(
181         outputChannels: outputChannels,
182         inputChannels: inputChannels,
183         groupSize: groupSize,
184         packedValues: packed,
185         scales: scales)
186 }
187 }

```


Per-Channel and Groupwise Scales

31.1 Why this exists

A transformer projection stores weights as $[out, in]$. One tensor-wide scale lets the largest value in any row determine resolution for every row. Per-output or smaller input groups preserve local ranges, but every extra scale consumes bytes and must be indexed exactly like the quantized values.

This lesson defines groupwise symmetric INT8 weights. $groupSize == in$ is one scale per output channel; smaller values split each row along its input axis. The final group may be shorter and still owns one scale.

31.2 Learning outcomes

You can:

- map a row-major weight coordinate to scale metadata $[out, group]$;
- derive the ceiling group count and handle a short tail group;
- quantize all-zero groups without invalid scales;
- compare reconstruction metrics against one per-tensor scale;
- count payload and metadata bytes for different group sizes; and
- explain why lower error is not a free storage or kernel win.

31.3 Prerequisites

- Problem 004 for the $[out, in]$ GEMV orientation.
- Problem 029 for symmetric INT8 range, finite checks, rounding, and metrics.
- Problem 006 for traffic and arithmetic-intensity models.

31.4 Vocabulary

- **Output channel**: one row of $[out, in]$, producing one GEMV output.
- **Per-channel scale**: one scale for an entire output row.
- **Groupwise scale**: one scale for a contiguous segment of a row.
- **Tail group**: the final segment when in is not divisible by group size.
- **Metadata shape**: the logical dimensions required to address scales.
- **Granularity**: how many values share one scale.

31.5 Derivation and worked numbers

For weights $W \in \mathbb{R}^{O \times I}$ and group size G ,

$$K = \left\lceil \frac{I}{G} \right\rceil, \quad g(c) = \left\lfloor \frac{c}{G} \right\rfloor,$$

and scale metadata has shape $[0, K]$. Scale index for $(\text{row}, \text{column})$ is

$$\text{scaleIndex}(r, c) = rK + g(c).$$

Each group independently applies Problem 029's $a/127$ rule and deterministic rounding. Consider $0=2$, $I=5$, $G=3$:

```
1 row 0: [-1.00, -0.50, 0.00 | 10.00, 5.00]
2 row 1: [ 0.10, -0.10, 0.05 | -0.02, 0.00]
```

There are $K = 2$ groups per row. Scales in row-major metadata order are

```
1 [1/127, 10/127, 0.1/127, 0.02/127]
```

and integer groups are approximately

```
1 [-127, -64, 0 | 127, 64]
2 [ 127, -127, 64 | -127, 0]
```

The two-value tails still own scales. With one tensor-wide scale $10/127$, the small second row is represented by integers near $[-1, 1]$; groupwise scales use the available range and lower this fixture's RMSE.

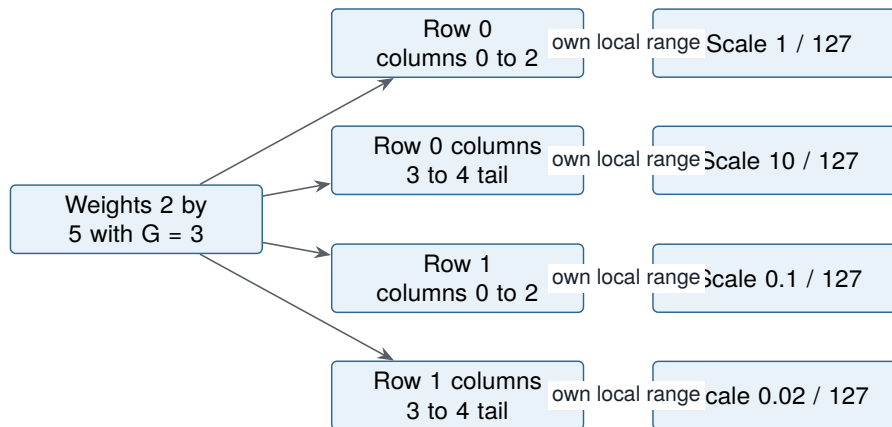


Figure 31.1: Groupwise scale granularity.

31.6 Shape, layout, and dtype contract

Input weights and both reconstructions are contiguous Float32 $[0, I]$ in row-major $[\text{out}, \text{in}]$ orientation. Groupwise payload is $\text{Int8}[0 \times I]$ in the same order. Float32 scales are logically $[0, \text{ceil}(I/G)]$ and physically flat row-major. G must be positive; 0 and I may be zero.

Every scale must be finite and positive. Every payload value is in $[-127, 127]$. Rank, finite inputs, value count, scale count, and overflow are validated before a format is accepted.

31.7 CPU reference path

The canonical implementation first computes the Problem 029 per-tensor result. It then walks output rows and groups. For each group it finds the maximum over $\text{start} \dots \min(\text{start} + G, I)$, assigns scale 1 to

an all-zero group, quantizes that slice, and writes metadata at $\text{row} \times K + \text{group}$. A separate pass reconstructs each value using column/G and reports both error sets.

The starter allocates valid payload and metadata shapes and enforces rank, group-size, and finite-value errors, but leaves integer values at zero.

31.8 Independent correctness

The judge calculates expected scale values and reconstructs payloads without calling the solution helper. It checks exact metadata count, the short tail, shape, 26 total bytes for the worked fixture, independent metrics, and the expected RMSE ordering for this controlled input. Error cases cover rank, nonpositive group size, and NaN.

```
1 swift run leetllm check 030 --cpu
2 swift run leetllm check 030 --solution
```

31.9 Performance model: bytes and arithmetic intensity

Payload and scale bytes are

$$B_{\text{values}} = OI, \quad B_{\text{scales}} = 4O \left\lceil \frac{I}{G} \right\rceil, \quad B_{\text{total}} = B_{\text{values}} + B_{\text{scales}}.$$

For the worked 2×5 , $G=3$ fixture, values use 10 bytes and four scales use 16, totaling 26. The tiny shape is metadata-heavy. For $I=4096$, $G=64$, scale metadata adds 256 bytes per row to 4096 payload bytes.

During a fused GEMV, each weight contributes about two floating-point operations. Ignoring input reuse and output writes, idealized intensity is approximately

$$I_{\text{Q8}} \approx \frac{2OI}{OI + 4O \lceil I/G \rceil}.$$

Smaller groups improve local resolution but lower this intensity through more scale traffic and indexing.

31.10 Metal mapping

Problem 030 remains CPU-only because it establishes conversion and metadata, not a GPU consumer. A standalone Metal round trip would materialize Float weights. Problem 033 uses the analogous `[out,group]` metadata in a real fused Q4 GEMV kernel.

31.11 Implementation checkpoints

1. Derive `groupsPerOutputChannel` with ceiling division.
2. Quantize one full group and one tail by hand.
3. Store scales at `[row,group]`, not one global group stream detached from rows.
4. Make an all-zero tail store scale 1.
5. Reconstruct using $\text{row} \times K + \text{column}/G$.
6. Report per-tensor and groupwise errors from actual values.
7. Include every Float32 scale in allocated bytes.

31.12 Controlled experiments

31.12.1 Group-size sweep

Sweep $G = I, 256, 64, 16$. Prediction: metadata rises monotonically; error usually falls but need not do so for every distribution.

31.12.2 Row-range imbalance

Multiply one row by 100 without changing others. Prediction: per-tensor error for untouched rows rises; per-channel error is mostly unchanged.

31.12.3 Tail placement

Try $I=63, 64, 65$ with $G=64$. Prediction: $I=65$ adds a full four-byte scale for one value, making metadata discontinuous at the group boundary.

31.13 Engine integration

The `[out,in]` orientation and `[out,group]` scale address become the weight contract for Q4 in 031. A checkpoint converter can choose a group size once, serialize payload plus scales, and let 032/033 consume exactly those dimensions.

31.14 Tradeoffs

- Per-tensor minimizes metadata but couples unrelated output ranges.
- Per-channel isolates rows while loading one scale for a long dot product.
- Small groups improve local fit but add scales and divisions/indexing.
- Float32 scales avoid another conversion convention at a nontrivial metadata cost.

31.15 Hints

- Use ceiling division only after handling $I == 0$.
- Reset the maximum for every row and group.
- Derive tail end with min, not an assumption that $I \% G == 0$.
- Compare errors from reconstructions; do not assert that one granularity always wins.

31.16 Completion checklist

- ☐ Weight orientation is explicitly `[out,in]`.
- ☐ Scale metadata is `[out,ceil(in/groupSize)]`.
- ☐ Tail and all-zero groups are handled.
- ☐ Per-tensor and groupwise reconstructions use actual payloads.
- ☐ Metrics come from an independent comparison pass.
- ☐ Byte totals include all Float32 scales.
- ☐ A granularity experiment has a written prediction.

31.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

31.17.1 Sources/LeetLLMExercises/P030GroupwiseInt8Exercise.swift

```

1 import LeetLLMCore
2
3 public enum P030GroupwiseInt8Exercise {
4     public static func compare(
5         _ weights: FloatTensor,
6         groupSize: Int
7     ) throws -> GroupwiseInt8Comparison {
8         guard weights.rank == 2 else {
9             throw TensorError.rankMismatch(expected: 2, actual: weights.rank)
10        }
11        guard groupSize > 0 else { throw WeightQuantizationError.invalidGroupSize(groupSize) }
12        for (index, value) in weights.storage.enumerated() where !value.isFinite {
13            throw WeightQuantizationError.nonFiniteValue(index: index, value: value)
14        }
15        let outputChannels = weights.shape[0]
16        let inputChannels = weights.shape[1]
17        let groups = inputChannels == 0 ? 0 : (inputChannels + groupSize - 1) / groupSize
18        let perTensor = try SymmetricInt8Tensor(
19            values: Array(repeating: 0, count: weights.elementCount),
20            shape: weights.shape,
21            scale: 1)
22        let groupwise = try GroupwiseInt8WeightMatrix(
23            outputChannels: outputChannels,
24            inputChannels: inputChannels,
25            groupSize: groupSize,
26            values: Array(repeating: 0, count: weights.elementCount),
27            scales: Array(repeating: 1, count: outputChannels * groups))
28        let zeros = try FloatTensor(
29            Array(repeating: 0, count: weights.elementCount), shape: weights.shape)
30        let error = QuantizationErrorMetrics(maximumAbsoluteError: 0, rootMeanSquareError: 0)
31        return GroupwiseInt8Comparison(
32            perTensor: perTensor,
33            perTensorDequantized: zeros,
34            groupwise: groupwise,
35            groupwiseDequantized: zeros,
36            perTensorError: error,
37            groupwiseError: error)
38    }
39 }

```

31.18 Worked solution

Solution strategy

Extend Problem 029's per-tensor INT8 quantization to per-group quantization by partitioning each row of $[out, in]$ weights into contiguous segments of size `groupSize` columns, deriving one scale per group from that group's maximum absolute value, and quantizing within each group independently. The scale metadata is logically shaped $[out, \text{ceil}(in/groupSize)]$ and indexed by $\text{scaleIndex}(row, column) = row * \text{groupsPerOutputChannel} + column / groupSize$. Tail groups (when `in` is not divisible by `groupSize`) are shorter but still own a scale. The implementation iterates output channels and groups, computes maximum in each group window, applies the ± 127 scale formula, quantizes within the window, and stores scales in row-major flat order.

Correctness invariant

Each weight value belongs to exactly one group. The scale for group (o, g) is derived only from values in $[o*in + g*G, o*in + \min((g+1)*G, in))$. All-zero groups have scale 1.0. The tail group for each row may be shorter than `groupSize` but is never omitted.

Common failure mode

Using $\text{floor}(in / groupSize)$ instead of ceiling for group count causes the final partial group to be lost. Incorrect scale indexing (e.g., $column \% groupSize$ instead of $column / groupSize$) misaligns quantized values and scales. Not handling the all-zero group case causes zero scale in metadata.

31.18.1 Implementation points

Group count uses ceiling division

$\text{groupsPerOutputChannel} = (\text{inputChannels} + \text{groupSize} - 1) / \text{groupSize}$. For `in=5`, `G=3`, this gives $(5+2)/3 = 2$ groups. Using integer division alone would yield 1 group, dropping the tail.

Scale indexing binds group to quantized window

$\text{scaleIndex}(row, col) = row * \text{groupsPerOutputChannel} + col / groupSize$. Column 4 in a 3-wide group belongs to group $4/3=1$, not group $4\%3=1$. This formula ensures each quantized value reads its group's scale.

Groupwise scales lower reconstruction error

Per-tensor scale is forced to accommodate the global maximum. Groupwise scales adapt to local ranges: a small group of values uses its own denominator, not the tensor-wide maximum. The fixture with `O=2`, `I=5`, `G=3` shows error reduction when row 1's small values use scale $0.1/127$ instead of $10/127$.

Metadata overhead grows with groups

Scales occupy $4 \cdot \text{out} \cdot K$ bytes where $K = \lceil \text{in} / G \rceil$. Smaller groups yield more scales; problem 006's roofline model quantifies whether error reduction justifies higher metadata traffic.

Tail groups are valid and necessary

When $\text{in} = 5$ and $G = 3$, the tail group spans columns 3-4 (2 values). It owns one scale derived from those 2 values only. Omitting it or reusing a neighboring scale breaks the quantization invariant.

31.18.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P030GroupwiseInt8Solution.swift

- Line 3: public enum P030GroupwiseInt8Solution {
- Line 4: public static func compare(

31.18.3 Complete canonical listings**31.18.4 Sources/LeetLLMSolutions/P030GroupwiseInt8Solution.swift**

```

1 import LeetLLMCore
2
3 public enum P030GroupwiseInt8Solution {
4     public static func compare(
5         _ weights: FloatTensor,
6         groupSize: Int
7     ) throws -> GroupwiseInt8Comparison {
8         let perTensor = try WeightQuantizationSolutionSupport.quantizeInt8(weights)
9         let perTensorDequantized = try WeightQuantizationSolutionSupport.dequantize(perTensor)
10        let groupwise = try WeightQuantizationSolutionSupport.quantizeGroupwiseInt8(
11            weights, groupSize: groupSize)
12        let groupwiseDequantized = try WeightQuantizationSolutionSupport.dequantize(groupwise)
13        return GroupwiseInt8Comparison(
14            perTensor: perTensor,
15            perTensorDequantized: perTensorDequantized,
16            groupwise: groupwise,
17            groupwiseDequantized: groupwiseDequantized,
18            perTensorError: WeightQuantizationSolutionSupport.metrics(
19                reference: weights.storage, candidate: perTensorDequantized.storage),
20            groupwiseError: WeightQuantizationSolutionSupport.metrics(
21                reference: weights.storage, candidate: groupwiseDequantized.storage))
22    }
23 }

```

Pack and Unpack INT4

32.1 Why this exists

Swift and Metal expose byte-addressable buffers, not a portable signed four-bit scalar array. A Q4 checkpoint therefore needs a byte contract shared by the converter, loader, CPU oracle, and kernel. “Two values per byte” is incomplete: nibble order, signed encoding, odd counts, row orientation, and scale alignment must all agree.

This lesson defines one canonical format in `GroupwiseQ4WeightMatrix`. Problem 032 reads it into `Float` weights; Problem 033 sends the same packed bytes and format code to MSL.

32.2 Learning outcomes

You can:

- state the signed Q4 range `[-8,7]`;
- encode a negative nibble with four-bit two’s-complement;
- pack the first logical value in the low nibble;
- define and validate zero padding for an odd value count;
- keep `[out,group]` scales aligned when a row ends mid-byte; and
- prove a representation with byte-exact fixtures rather than approximate floats.

32.3 Prerequisites

- Problem 002 for flat row-major indexing.
- Problem 030 for `[out,in]` weights and `[out,group]` scales.
- Familiarity with hexadecimal notation and bit masks.

32.4 Vocabulary

- **Nibble**: four bits, representing an unsigned pattern from `0x0` to `0xf`.
- **Two’s-complement**: signed encoding where patterns `0x8...0xf` mean `-8...-1`.
- **Low nibble first**: even logical index uses bits `0...3` of its byte.
- **Padding nibble**: unused high four bits when the logical count is odd.
- **Logical index**: `row*inputChannels+column` before packing.
- **Format code**: the enum value checked by the loader and passed to Metal.

32.5 Derivation and byte-exact example

Four-bit two’s-complement represents


```

1 decimal: -8 -7 ... -1  0  1 ... 7
2 nibble:  0x8 0x9 ... 0xf 0x0 0x1 ... 0x7

```

For logical value index i , byte index is $\lfloor i/2 \rfloor$. This format uses

$$\text{byte}[i/2] = \begin{cases} q_i \& 0xf & i \text{ even} \\ \text{byte}[i/2] | ((q_i \& 0xf) \ll 4) & i \text{ odd.} \end{cases}$$

Pack [-8, -7, -1, 0, 1, 7, 3]:

logical pair	nibble pair	stored byte
[-8, -7]	[0x8, 0x9]	0x98
[-1, 0]	[0xf, 0x0]	0x0f
[1, 7]	[0x1, 0x7]	0x71
[3, pad]	[0x3, 0x0]	0x03

The unused high nibble must be zero. A nonzero padding nibble is rejected so two serialized forms cannot claim to represent the same logical matrix.

Unpacking selects low nibble for even i , high nibble for odd i , then sign extends: if nibble ≥ 8 , the value is nibble - 16.

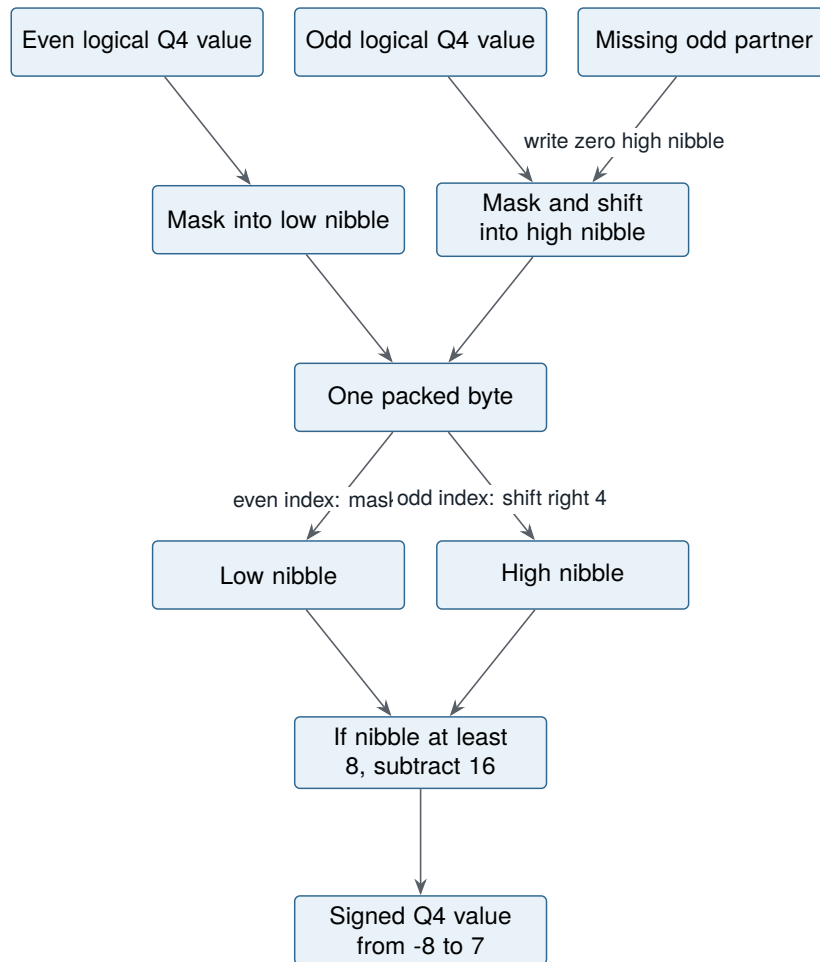


Figure 32.1: Int4 nibble packing.

32.6 Shape, layout, dtype, and format contract

Logical weights are signed integers $[0, I]$ in row-major $[out, in]$ order. Packing treats all OI values as one continuous stream; it does not restart at a row boundary. This matters when I is odd. Packed storage is `UInt8[ceil(0*I/2)]`.

Scale metadata remains `Float32 [0, ceil(I/6)]`. It is indexed from logical row and column, not from byte position. The only accepted `Q4WeightFormat` is `signedTwosComplementLowNibbleFirst` with raw format code 0.

The initializer validates dimensions, positive group size, exact packed byte count, exact scale count, finite positive scales, and zero odd padding.

32.7 CPU reference path

The canonical packer first validates logical count and every integer against $[-8, 7]$. It masks each `Int8` bit pattern with `0x0f`, writes even indices into low nibbles, ORs odd indices into high nibbles, and constructs the validated format. The shared format's `quantizedValue(row, column)` performs canonical unpacking and sign extension.

The starter performs all dimension, range, scale, and byte-count validation but returns zero bytes. It therefore fails exact fixtures without hiding the missing bit manipulation behind shape errors.

32.8 Independent correctness

The first judge fixture requires exactly `[0x98, 0x0f, 0x71, 0x03]` and verifies the round trip. The second uses shape `[2, 3]`, so row 0 ends in the low nibble of a byte and row 1 begins in its high nibble; expected bytes are `[0x78, 0xf1, 0xe2]`. It also checks scale index `(row=1, column=2) == 3`.

Error fixtures reject integer 8, missing scales, and nonzero odd padding. A test implements high-nibble-first packing and confirms the shared judge fails it.

```
1 swift run leetllm check 031 --cpu
2 swift run leetllm check 031 --solution
```

32.9 Performance model: bytes and arithmetic intensity

For $O \times I$ values and group size G ,

$$B_{Q4} = \left\lceil \frac{OI}{2} \right\rceil + 4O \left\lceil \frac{I}{G} \right\rceil.$$

The seven-value fixture uses four payload bytes and three `Float32` scales, or 16 total bytes. Padding occupies four unused bits but no additional byte beyond the ceiling formula.

Packing performs masks, shifts, and stores but is an offline conversion step. For inference, Q4 GEMV reads about half a payload byte per multiply plus amortized scale metadata. The useful arithmetic intensity appears only when a consumer dequantizes during accumulation, as in 033.

32.10 Metal mapping

There is no separate Metal stage in 031. A GPU pack/unpack round trip would not be an inference operator. The actual MSL unpack implementation is exercised in 033 against these exact bytes, including odd widths and group tails. Its format code and indexing are bound from `GroupwiseQ4WeightMatrix`.

32.11 Implementation checkpoints

1. Map every integer in $[-8, 7]$ to its nibble.
2. Reproduce $[-8, -7] \rightarrow 0 \times 98$.
3. Reproduce the complete seven-value fixture.
4. Reject -9 and 8.
5. Reject nonzero odd padding.
6. Pack $[2, 3]$ without restarting at row boundaries.
7. Verify scale indexing is independent from byte indexing.

32.12 Controlled experiments

32.12.1 Nibble-order fault

Swap low-first to high-first while leaving all other data fixed. Prediction: byte fixtures fail exactly and later GEMV differences are systematic; this is a format mismatch, not inherent Q4 error.

32.12.2 Odd-width matrix

Compare $I=64$ and $I=65$ for two rows. Prediction: the continuous stream saves the redundant per-row padding nibble a row-aligned format would require, but manual row-pointer arithmetic becomes less direct.

32.12.3 Group-size byte model

Keep payload fixed and sweep group size. Prediction: packed bytes do not change; only Float32 scale metadata changes at ceiling boundaries.

32.13 Engine integration

A model converter calls the canonical packer, serialized metadata reconstructs `GroupwiseQ4WeightMatrix`, and both CPU and Metal consumers receive that type. No consumer accepts an ad hoc byte array without dimensions, group size, scales, and format code. This is the interchange contract used by 032-034.

32.14 Tradeoffs

- Continuous packing minimizes bytes; row-aligned packing can simplify row starts.
- Two's-complement permits -8; a symmetric $[-7, 7]$ quantizer may leave that endpoint uncommon.
- Zero padding gives one canonical encoding; unspecified padding can leak stale bits.
- Smaller groups improve fit but metadata can dominate tiny matrices.

32.15 Hints

- Use `UInt8(bitPattern: value) & 0x0f` for the nibble.
- Do not shift the first value left.
- Compute byte index from the flat logical index, not `column/2` alone.
- Sign extension can be expressed as `nibble >= 8 ? nibble - 16 : nibble`.

32.16 Completion checklist

- ☐ Signed Q4 range is exactly `[-8, 7]`.
- ☐ Encoding is four-bit two's-complement.
- ☐ Even logical indices occupy low nibbles.
- ☐ Odd-count padding is zero and validated.
- ☐ Packing remains continuous across odd-width rows.
- ☐ Scale metadata stays `[out, group]`.
- ☐ Exact bytes, not only reconstructed floats, pass the judge.

32.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

32.17.1 Sources/LeetLLMExercises/P031PackQ4Exercise.swift

```

1 import LeetLLMCore
2
3 public enum P031PackQ4Exercise {
4     public static func pack(
5         _ values: [Int8],
6         outputChannels: Int,
7         inputChannels: Int,
8         groupSize: Int,
9         scales: [Float]
10    ) throws -> GroupwiseQ4WeightMatrix {
11        let (count, overflow) = outputChannels.multipliedReportingOverflow(by: inputChannels)
12        guard !overflow else { throw WeightQuantizationError.sizeOverflow }
13        guard values.count == count else {
14            throw WeightQuantizationError.storageCountMismatch(
15                name: "Q4 logical values", expected: count, actual: values.count)
16        }
17        for (index, value) in values.enumerated() where value < -8 || value > 7 {
18            throw WeightQuantizationError.quantizedValueOutOfRange(
19                index: index, value: Int(value), minimum: -8, maximum: 7)
20        }
21        return try GroupwiseQ4WeightMatrix(
22            outputChannels: outputChannels,
23            inputChannels: inputChannels,
24            groupSize: groupSize,
25            packedValues: Array(repeating: 0, count: count / 2 + count % 2),
26            scales: scales)
27    }
28 }

```

32.18 Worked solution

Solution strategy

Pack signed Q4 integers (range $[-8, 7]$) into UInt8 bytes, two values per byte, using low-nibble-first format. For logical value index i , the byte is computed as: $\text{byte}[i/2] = (\text{nibble}_i \& 0x0f) \mid ((\text{nibble}_{i+1} \& 0x0f) \ll 4)$ for even i . Unpacking reverses this: even indices read low nibble from $\text{byte} \& 0x0f$, odd indices read high nibble from $\text{byte} \gg 4$, then sign-extend: if $\text{nibble} \geq 8$, $\text{value} = \text{nibble} - 16$. The format requires zero padding in the unused high nibble when the logical count is odd. Implementation in `packQ4` iterates logical values, validates range $[-8, 7]$, packs by masking into nibbles, and validates zero padding in `GroupwiseQ4WeightMatrix` initialization.

Correctness invariant

Even logical indices occupy bits 0-3 of their byte; odd indices occupy bits 4-7. Sign extension rule is constant: nibble n maps to n if $n < 8$, else $n - 16$. For odd-length streams, the high nibble of the last byte is zero and must be validated to reject non-canonical encodings.

Common failure mode

Swapping nibble order (high-first instead of low-first) produces $\text{byte} = \text{nibble}_i \ll 4 \mid \text{nibble}_{i+1}$, making all existing packed data undecodable. Not zero-padding odd-length streams allows multiple byte sequences to decode to the same logical values, violating uniqueness. Forgetting sign extension (treating all nibbles as unsigned) produces incorrect negative values.

32.18.1 Implementation points

Two's-complement nibble encoding spans -8 to 7

Four bits represent decimal $-8..7$. Pattern $0x8=1000$ is -8 , $0xf=1111$ is -1 , $0x0=0000$ is 0 , $0x7=0111$ is 7 . Sign extension checks $\text{nibble} \geq 8$ and subtracts 16 if true.

Low nibble first packs logically contiguous values

Logical pairs $(i, i+1)$ pack into one byte as $(\text{low_nibble}, \text{high_nibble})$. Walking columns sequentially across rows produces one continuous logical stream. Odd I values create mid-byte boundaries when transitioning rows.

Padding nibble must be zero

When `logicalCount` is odd, the last byte's high nibble is unused padding. `GroupwiseQ4WeightMatrix` validation rejects `nonzeroPaddingNibble` to ensure only one byte encoding represents each logical matrix.

Format code identifies the decoding rule

`Q4WeightFormat = signedTwosComplementLowNibbleFirst` (raw value 0) is the only accepted format. Both CPU and Metal kernels check this enum to confirm they decode nibbles and sign-extend

identically.

Unpacking uses bit shifts, not lookups

Extracting nibble n from byte b uses: $n_low = b \& 0x0f$, $n_high = b \gg 4$. Sign extension is: $n \geq 8 ? n - 16 : n$. This bit-level operation runs identically in CPU and Metal without lookup tables.

32.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P031PackQ4Solution.swift

- Line 3: public enum P031PackQ4Solution {
- Line 4: public static func pack(

32.18.3 Complete canonical listings

32.18.4 Sources/LeetLLMSolutions/P031PackQ4Solution.swift

```

1 import LeetLLMCore
2
3 public enum P031PackQ4Solution {
4     public static func pack(
5         _ values: [Int8],
6         outputChannels: Int,
7         inputChannels: Int,
8         groupSize: Int,
9         scales: [Float]
10    ) throws -> GroupwiseQ4WeightMatrix {
11        try WeightQuantizationSolutionSupport.packQ4(
12            values,
13            outputChannels: outputChannels,
14            inputChannels: inputChannels,
15            groupSize: groupSize,
16            scales: scales)
17    }
18 }
```

Dequantize Then GEMV

33.1 Why this exists

Before fusing quantized arithmetic, the engine needs a baseline whose behavior is easy to inspect. The simplest correct Q4 projection reconstructs every Float32 weight, materializes an `[out, in]` tensor, and passes that tensor to the existing GEMV. It spends extra memory and traffic, but it separates format errors from fused-kernel errors.

This is not presented as an optimization. Its full Float32 temporary is part of the result contract and must be reported honestly.

33.2 Learning outcomes

You can:

- validate Q4 dimensions, metadata, input rank, and GEMV inner width;
- reconstruct `[out, in]` weights from low-first packed nibbles and group scales;
- call a known Float32 GEMV as a staged correctness baseline;
- calculate the exact temporary allocation and its traffic cost;
- compare staged output against an independent direct Q4 oracle; and
- explain why a slow baseline remains useful after fusion exists.

33.3 Prerequisites

- Problem 004 for Float32 GEMV and `[out, in]` orientation.
- Problem 030 for group-scale indexing.
- Problem 031 for canonical Q4 unpacking and padding.
- Problem 006 for separating algorithm bytes from runtime overhead.

33.4 Vocabulary

- **Staged path:** one operator writes an intermediate consumed by another.
- **Materialization:** allocating and filling all logical Float32 weights.
- **Temporary bytes:** storage live for the operation but absent from the compact checkpoint.
- **Parity baseline:** a clear implementation used to localize errors in an optimized path.
- **Independent oracle:** expected output computed without calling the staged implementation.

33.5 Derivation and worked projection

For packed Q4 weight $q_{r,c}$ and scale $s_{r,\lfloor c/G \rfloor}$,

$$\hat{W}_{r,c} = q_{r,c} s_{r, \lfloor c/G \rfloor}, \quad y_r = \sum_{c=0}^{I-1} \hat{W}_{r,c} x_c.$$

Use 0=2, I=5, G=3, logical Q4 values

```
1 row 0: [-8, -4, 0 | 3, 7] scales [0.25, 0.50]
2 row 1: [ 1, -1, 2 | -2, 0] scales [0.10, 0.20]
```

The packed bytes are [0xc8, 0x30, 0x17, 0x2f, 0x0e]. Continuous packing means row 1 starts in the high nibble of byte 2. Materialization produces

```
1 row 0: [-2.0, -1.0, 0.0, 1.5, 3.5]
2 row 1: [ 0.1, -0.1, 0.2, -0.4, 0.0]
```

For $x = [1, -2, 0.5, 1.5, -1]$:

$$y_0 = -2 + 2 + 0 + 2.25 - 3.5 = -1.25,$$

$$y_1 = 0.1 + 0.2 + 0.1 - 0.6 + 0 = -0.2.$$

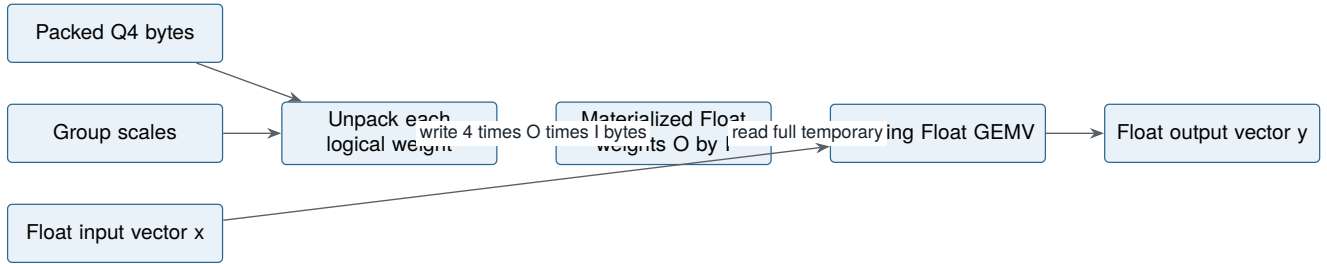


Figure 33.1: Staged dequantize gemv.

33.6 Shape, layout, and dtype contract

Weights are a validated `GroupwiseQ4WeightMatrix` with logical shape $[0, I]$, packed `UInt8` $\lceil 0 \cdot I / 2 \rceil$, `Float32` scales $[0, \lceil I / G \rceil]$, and canonical format code. Input is contiguous `Float32` $[I]$. Materialized weights are contiguous `Float32` $[0, I]$; output is `Float32` $[0]$.

Input rank and width are checked before indexing. The Q4 initializer has already validated byte count, padding, scale count, and scale finiteness. Non-finite input values are rejected. The reported temporary is exactly $4 \cdot 0 \cdot I$ bytes.

33.7 CPU reference path

1. Validate input rank, inner width, and finite values.
2. Allocate $0 \cdot I$ `Float32` elements.
3. For every logical (row, column), unpack through the shared Q4 format.
4. Load scale at $\text{row} \cdot \text{groupsPerRow} + \text{column} / \text{groupSize}$.
5. Write `Float(q) * scale` to the materialized tensor.
6. Call the canonical Problem 004 GEMV.
7. Return output, the materialized weights, and exact temporary bytes.

The starter validates everything and allocates the required shapes, but leaves the materialized tensor and output at zero.

33.8 Independent correctness

The judge independently unpacks every value and performs GEMV with `Double` accumulation. It compares both the intermediate matrix and final output, then requires exactly 40 temporary bytes for the 2x5 fixture. Rank-2 input and a short vector must throw. A focused wrong implementation that returns correct values but reports zero temporary bytes fails.

```
1 swift run leetllm check 032 --cpu
2 swift run leetllm check 032 --solution
```

33.9 Performance model: bytes and arithmetic intensity

The compact persistent representation occupies

$$B_Q = \left\lceil \frac{OI}{2} \right\rceil + 4O \left\lceil \frac{I}{G} \right\rceil.$$

The staged operation additionally writes 40I materialized bytes, then GEMV reads those 40I bytes. Ignoring cache effects and small output terms, traffic is

$$B_{\text{staged}} \approx B_Q + 8OI + 4I + 4O.$$

GEMV performs about 20I model FLOPs. Dequantization adds one multiply per weight but does not increase the model's useful projection work. The extra Float32 write/read lowers effective arithmetic intensity and can erase the bandwidth benefit of compact storage.

33.10 Metal mapping

There is no Metal stage for 032. A staged GPU implementation would need a Q4 dequantization dispatch, a full device Float32 buffer, and a second GEMV dispatch. That path is valid in a production fallback but adds no new semantic lesson here. Problem 033 implements the genuinely distinct fused device path.

33.11 Implementation checkpoints

1. Decode the five-value first row from packed bytes.
2. Decode row 1 across the shared byte boundary.
3. Select scales from `[row, column/G]`.
4. Reproduce the worked Float matrix.
5. Reproduce `[-1.25, -0.2]` within tolerance.
6. Report `4*0*I`, not packed bytes, as temporary weight bytes.
7. Reject rank and width mismatches before allocation.

33.12 Controlled experiments

33.12.1 Width sweep

Measure fixed 0 while doubling I. Prediction: both materialization traffic and GEMV traffic grow linearly, so staged cost becomes increasingly visible.

33.12.2 Reuse sweep

Materialize once and run several vectors versus materialize for every vector. Prediction: reuse can amortize conversion for batch/prefill, while single-token decode receives little amortization.

33.12.3 Format fault localization

Reverse nibble order and inspect the materialized matrix before GEMV. Prediction: the first wrong weight identifies a converter/loader convention issue without needing to reason backward from final output.

33.13 Engine integration

This operation is the parity and fallback implementation for quantized linear layers. A model loader can validate a Q4 checkpoint by comparing selected staged projections with Float32 or fused paths. Problem 033 must match it while removing the full-weight temporary.

33.14 Tradeoffs

- Materialization is readable and debuggable but expensive per decode token.
- Reusing a materialized matrix can help batches but gives up compact working-set benefits.
- Calling existing GEMV reduces implementation risk while adding a dispatch boundary on GPU.
- Keeping the intermediate in reports aids diagnosis but should not become a production API requirement.

33.15 Hints

- Iterate logical values; let the format type own nibble extraction.
- Do not derive scale index from byte index.
- Preserve [out, in]; a transpose can produce plausible but wrong output shapes.
- Count the Float32 temporary separately from persistent Q4 bytes.

33.16 Completion checklist

- ☐ Q4 metadata and input dimensions are validated.
- ☐ Materialized weights exactly match the format contract.
- ☐ Existing Float32 GEMV consumes [out, in] weights.
- ☐ Output matches the independent Double oracle.
- ☐ Full temporary bytes are reported as $4 \times 0 \times I$.
- ☐ No GPU work is claimed for this CPU-only baseline.
- ☐ A staged-cost prediction was written before measurement.

33.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

33.17.1 Sources/LeetLLMExercises/P032DequantizeThenGEMVExercise.swift

```

1 import LeetLLMCore
2
3 public enum P032DequantizeThenGEMVExercise {
4     public static func multiply(
5         _ weights: GroupwiseQ4WeightMatrix,
6         _ input: FloatTensor
7     ) throws -> DequantizeThenGEMVResult {
8         guard input.rank == 1 else {
9             throw TensorError.rankMismatch(expected: 1, actual: input.rank)
10        }
11        guard input.shape[0] == weights.inputChannels else {
12            throw DenseLinearAlgebraError.innerDimensionMismatch(
13                operation: "dequantize-then-GEMV", lhs: weights.inputChannels, rhs: input.shape[0])
14        }
15        for (index, value) in input.storage.enumerated() where !value.isFinite {
16            throw WeightQuantizationError.nonFiniteValue(index: index, value: value)
17        }
18        let materialized = try FloatTensor(
19            Array(repeating: 0, count: weights.logicalValueCount), shape: weights.shape)
20        return DequantizeThenGEMVResult(
21            output: try FloatTensor(
22                Array(repeating: 0, count: weights.outputChannels), shape: [weights.outputChannels]),
23            materializedWeights: materialized,
24            temporaryWeightBytes: weights.logicalValueCount * MemoryLayout<Float>.stride)
25    }
26 }

```

33.18 Worked solution

Solution strategy

Validate input rank and inner width, allocate $O \cdot I$ Float32 elements for materialized weights, iterate logical (row, column) pairs, unpack each Q4 value through `GroupwiseQ4WeightMatrix.quantizedValue` (which reads the packed byte, extracts the correct nibble, and sign-extends), load the group scale via `scaleIndex`, multiply quantized * scale, store into the float array, then call the existing P004 GEMV on the materialized tensor. Return the output, materialized weights, and `temporaryWeightBytes = 4 * O * I` (exact byte count). This staged path does not fuse; every weight is reconstructed to Float before accumulation. The CPU oracle in problem 033 bypasses this step to verify fused output against the staged baseline.

Correctness invariant

Materialized weights follow $W_{rc} = \text{Float}(q_{rc}) * s_{\{r, \text{floor}(c/G)\}}$. Each output row is the Float32 sum of I products $W_{rc} * x_c$; all O rows together consume $O \cdot I$ logical weights. The canonical fused CPU path uses the same left-to-right product and column accumulation order, allowing direct comparison with this staged baseline without claiming that floating-point multiplication is associative.

Common failure mode

Mixing logical column index with packed byte index during unpacking. Not validating input rank or inner width, allowing out-of-bounds reads in GEMV. Forgetting to multiply by the group scale, leaving quantized values as integers. Not accounting for the $4 * O * I$ temporary bytes in reporting.

33.18.1 Implementation points

Rank and width validation precedes indexing

Input rank must be 1. Input width must equal `weights.inputChannels`. These checks run before any array access, preventing rank mismatches and dimension errors from propagating into GEMV.

Materialization materializes all logical values

The loop walks (`row=0..O-1`, `column=0..I-1`), unpacks via `quantizedValue`, dequantizes by scale, and stores into a flat Float32 $[O \cdot I]$ array. The temporary exists for the lifetime of the GEMV call only; it is not cached.

quantizedValue handles packing transparently

Calling `weights.quantizedValue(row, col)` abstracts the byte lookup and sign extension. The caller does not manipulate packed bytes; it reads signed Q4 integers directly.

Temporary allocation is explicit

DequantizeThenGEMVResult reports `temporaryWeightBytes = weights.logicalValueCount * 4`. For a 64x1024 matrix, this is 262,144 bytes, or 256 KiB, of Float32 storage. Materialization writes that temporary and the following GEMV reads it; the reported field is an allocation count rather than a hardware traffic measurement.

Staged output baseline catches format errors early

If the packed bytes or scale indexing is wrong, the materialized weights will obviously be incorrect. Running the plain GEMV on that tensor isolates format errors from fused-kernel errors, making debugging deterministic.

33.18.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P032DequantizeThenGEMVSolution.swift

- Line 3: `public enum P032DequantizeThenGEMVSolution {`
- Line 4: `public static func multiply(`

33.18.3 Complete canonical listings**33.18.4 Sources/LeetLLMSolutions/P032DequantizeThenGEMVSolution.swift**

```

1 import LeetLLMCore
2
3 public enum P032DequantizeThenGEMVSolution {
4     public static func multiply(
5         _ weights: GroupwiseQ4WeightMatrix,
6         _ input: FloatTensor
7     ) throws -> DequantizeThenGEMVResult {
8         guard input.rank == 1 else {
9             throw TensorError.rankMismatch(expected: 1, actual: input.rank)
10        }
11        guard input.shape[0] == weights.inputChannels else {
12            throw DenseLinearAlgebraError.innerDimensionMismatch(
13                operation: "dequantize-then-GEMV",
14                lhs: weights.inputChannels,
15                rhs: input.shape[0])
16        }
17        try WeightQuantizationSolutionSupport.validateFinite(input.storage)
18        let materialized = try WeightQuantizationSolutionSupport.dequantize(weights)
19        return DequantizeThenGEMVResult(
20            output: try P004GEMVSolution.multiply(matrix: materialized, vector: input),
21            materializedWeights: materialized,
22            temporaryWeightBytes: weights.logicalValueCount * MemoryLayout<Float>.stride)
23    }
24 }

```

Fused Q4 GEMV

34.1 Why this exists

Single-token decoder projections are GEMV operations that stream large weight matrices with little reuse. Q4 can reduce those weight bytes only if the consumer reads packed values directly. Reconstructing a full Float32 matrix first reintroduces the traffic Problem 032 measured.

This lesson implements a CPU fused reference and a real Metal kernel. Both read the same `GroupwiseQ4WeightMatrix`, unpack each nibble, load its group scale, and accumulate without allocating `Float[0, I]`.

34.2 Learning outcomes

You can:

- fuse Q4 unpacking and group-scale dequantization into GEMV accumulation;
- prove that no full Float32 weight temporary is allocated;
- map one output row to a 256-thread Metal reduction;
- handle odd logical counts, tail groups, widths over 256, and zero rows;
- compare persistent logical bytes and end-to-end timing honestly; and
- distinguish a format mismatch from expected precision loss.

34.3 Prerequisites

- Problem 001 for a barrier-safe Metal reduction.
- Problem 004 for row-wise GEMV.
- Problem 006 for roofline and benchmark boundaries.
- Problems 030-032 for group metadata, Q4 bytes, and the staged baseline.

34.4 Vocabulary

- **Fused dequantization**: reconstructing a weight only in the register that consumes it.
- **Logical weight bytes**: packed payload plus required scale metadata.
- **Full-weight temporary**: an `0*I` Float32 allocation; required count here is zero.
- **Lane-strided loop**: lane `t` handles columns `t`, `t+256`,
- **End-to-end Metal timing**: host buffer creation, copies, submission, execution, wait, and readback.

34.5 Derivation and worked projection

The fused equation substitutes Q4 reconstruction directly into GEMV:

$$y_r = \sum_{c=0}^{I-1} q_{r,c} s_{r,[c/G]} x_c.$$

No mathematical output changes from 032. With bytes `[0xc8,0x30,0x17,0x2f,0x0e]`, scales `[0.25,0.5,0.1,0.2]`, and input `[1,-2,0.5,1.5,-1]`, the fused loop still produces `[-1.25,-0.2]`. The difference is lifetime: each Float weight exists only while its multiply-add is evaluated.

Quantization itself uses Problem 029's nearest-with-ties-away rule. The Metal kernel does not quantize or round weights; it decodes exact stored nibbles and multiplies by the serialized Float32 scale. Swift and MSL therefore agree at the format boundary rather than depending on two runtime rounding functions.

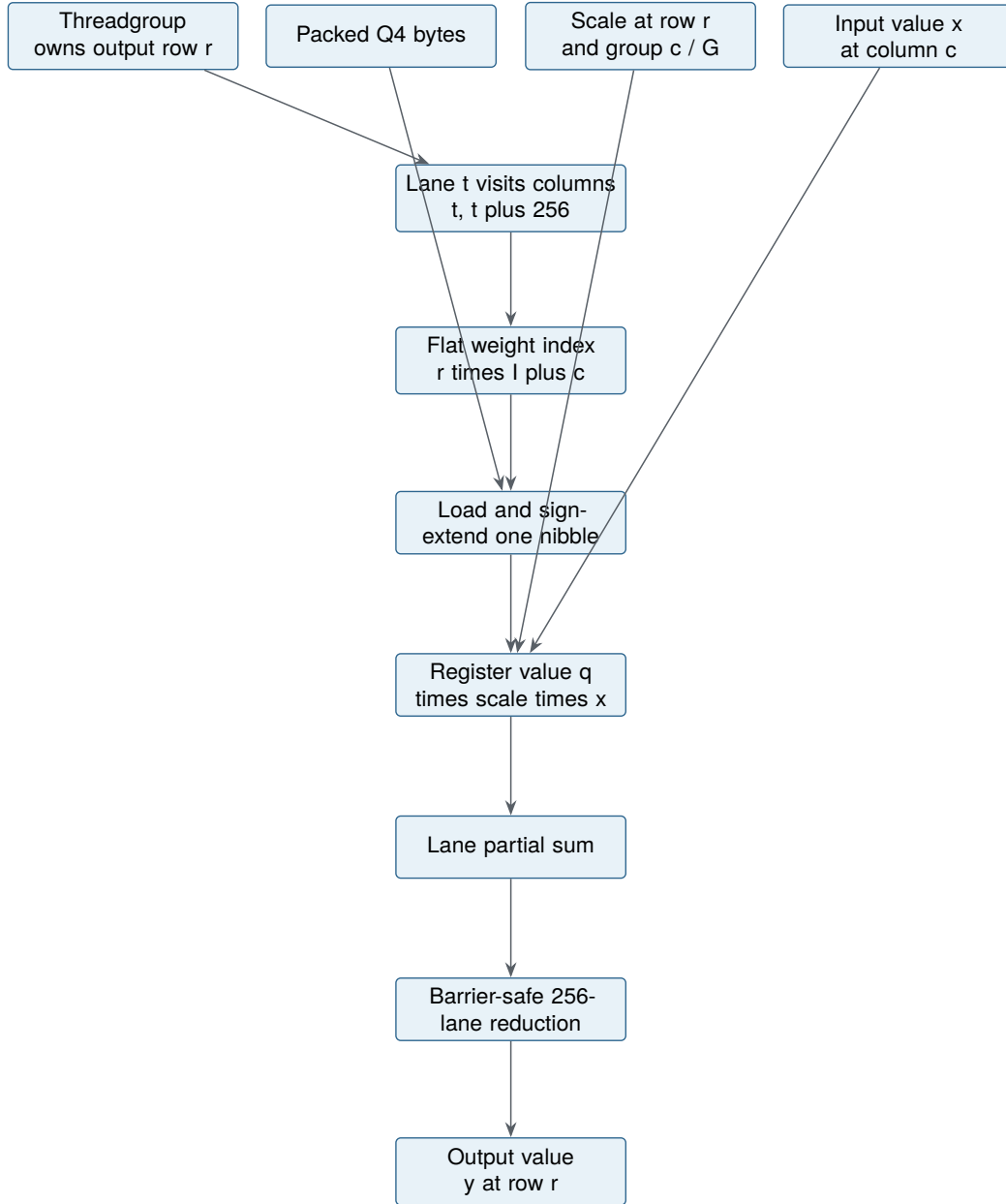


Figure 34.1: Fused q4 register path.

34.6 Shape, layout, dtype, and format contract

Logical weights are $[0, I]$ row-major. Packed bytes are a continuous $\text{UInt8}[\text{ceil}(O \cdot I/2)]$ low-nibble-first two's-complement stream. Scales are $\text{Float32}[0, \text{ceil}(I/G)]$. Input is $\text{Float32}[I]$; output is $\text{Float32}[O]$.

CPU and Metal accumulation are Float32 . The independent judge accumulates in Double and allows reduction-order tolerance. Format code 0 means `signedTwosComplementLowNibbleFirst`; the host passes that enum raw value to MSL. Input rank, width, finite values, and UInt32 -sized dispatch dimensions are validated before encoding.

34.7 CPU fused reference

For each output row, the CPU implementation initializes one Float32 sum. It walks columns, asks the shared format for the signed nibble, loads $\text{scale}[\text{row} \cdot K + \text{column}/G]$, multiplies $\text{Float}(q) \cdot \text{scale} \cdot \text{input}[\text{column}]$, and accumulates. It returns compact logical bytes and `temporaryWeightBytes = 0`.

This reference is separate from 032: it never calls the Q4 materializer and its API does not return Float weights.

34.8 Independent correctness

The judge directly decodes nibbles and accumulates expected outputs in Double . Fixtures include $I=5, G=3$, a 3×259 matrix with $G=257$, and zero output rows. The 259-column case crosses one 256-lane stride and has a two-value tail group. Rank and width errors must throw.

Every successful result must report exactly `packedValueBytes + scaleBytes` and zero full-weight temporary bytes. A wrong path that calls 032 can match output but fails the allocation invariant. Metal runs the identical judge after actual runtime MSL compilation and dispatch.

```
1 swift run leetllm check 033 --cpu
2 swift run leetllm check 033 --metal
3 swift run leetllm check 033 --solution
```

34.9 Bytes and arithmetic intensity

Persistent logical weight bytes are

$$B_Q = \left\lceil \frac{OI}{2} \right\rceil + 4O \left\lceil \frac{I}{G} \right\rceil.$$

An ideal fused call additionally reads about $4I$ input bytes and writes $4O$ output bytes:

$$B_{\text{fused}} \approx B_Q + 4I + 4O.$$

Using the conventional $2OI$ GEMV FLOP count,

$$\mathcal{J}_{\text{fused}} = \frac{2OI}{B_Q + 4I + 4O}.$$

Nibble extraction, integer conversion, and scaling add instructions but do not change the model’s useful FLOP count. Compared with 032, the fused path removes approximately 80I Float32 intermediate write/read bytes and one allocation.

34.10 Metal grid, unpacking, and reduction

The grid has 0 threadgroups. Each group has exactly 256 threads and owns one output row. Lane t processes columns t , $t+256$, ..., so all groups have the same barrier behavior even when $I < 256$ or I is not divisible by 256.

For logical index $\text{row} \cdot I + \text{column}$, MSL loads byte $\text{index}/2$, selects low nibble when index is even and high nibble otherwise, then computes $\text{nibble} \gg 8 \ ? \ \text{nibble} - 16 : \text{nibble}$. Scale index is $\text{row} \cdot \text{ceil}(I/G) + \text{column}/G$. Each lane stores a partial in threadgroup memory; a binary reduction reaches a barrier at every stride. Lane zero writes one Float.

Only four device buffers are bound: packed Q4 bytes, Float32 scales, Float32 input, and Float32 output. There is no host dequantization and no device Float weight buffer. The host-side CPU oracle is independent validation, not data fed to the kernel.

See [P033FusedQ4GEMV.metal](#).

34.11 Benchmark contract

Run:

```
1 swift run -c release leetllm benchmark 033 \
2   --out 1024 --in 1024 --group-size 64 --iterations 20
```

Weights are quantized once before timing. The report prints Float32 logical bytes, packed bytes, scale bytes, total Q4 bytes, and the staged temporary. CPU staged and CPU fused measurements include their function work. “Metal end-to-end” includes shared-buffer allocation, host copies, command submission, synchronization, and readback. It is not labeled kernel-only.

The current pipeline reallocates buffers each call. That makes it a truthful course baseline, not a production persistent-buffer benchmark.

34.12 Implementation checkpoints

1. Match 032 on the 2x5 hand fixture without materialization.
2. Report exact packed and scale bytes.
3. Keep full-weight temporary bytes at zero.
4. Unpack an odd logical index correctly in Swift and MSL.
5. Cross 256 columns with all lanes reaching every barrier.
6. Select a short tail group’s scale.
7. Pass the real Metal judge and benchmark smoke command.

34.13 Controlled experiments

34.13.1 Width sweep

Sweep $I=64, 255, 256, 257, 1024$. Prediction: tiny widths underutilize the fixed group; 257 adds a second lane iteration for one column while preserving output.

34.13.2 Group-size sweep

Hold packed values fixed and vary G . Prediction: smaller groups add scale traffic and may improve reconstruction; timing and accuracy must both be measured.

34.13.3 Staged versus fused

Predict before running whether saved 80I temporary traffic exceeds unpack and scale overhead on the current machine. Record release timings and the explicit end-to-end boundary.

34.13.4 Nibble-order injection

Reverse nibble selection in one implementation. Prediction: byte-level and output errors become structured immediately. Verify the convention before using the phrase “quantization noise.”

34.14 Engine integration

The kernel is the one-token decode projection path for compact attention and MLP weights. A loader validates `GroupwiseQ4WeightMatrix` once, retains packed buffers, and dispatches fused projections for Q/K/V and feed-forward rows. Problem 034 then checks how representation errors propagate across operators.

34.15 Tradeoffs

- Smaller Q4 traffic competes with unpack, conversion, and scale-load instructions.
- One threadgroup per row is readable; SIMD-group or vectorized kernels may map hardware better.
- Float32 accumulation protects reduction precision but costs registers and bandwidth relative to narrower accumulation.
- Persistent buffers are required for production latency; per-call allocation keeps this baseline inspectable.

34.16 Hints

- Derive nibble selection from the flat logical index, not column parity.
- Do not return early for lanes beyond I ; they still participate in barriers.
- Compute groups per row with ceiling division.
- Keep quantization outside benchmark timing unless dynamic quantization is the experiment.

34.17 Completion checklist

- ☐ CPU and MSL consume the same validated Q4 format.
- ☐ Output matches the independent Double oracle.
- ☐ Odd counts, tail groups, and widths over 256 pass.
- ☐ No full Float32 weight temporary is allocated or reported.
- ☐ Actual MSL compiles and dispatches in the Metal check.
- ☐ Benchmark output separates bytes and labels end-to-end timing.
- ☐ A performance prediction precedes release measurement.

34.18 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

34.18.1 Sources/LeetLLMExercises/P033FusedQ4GEMVExercise.swift

```
1 import LeetLLMCore
2
3 public enum P033FusedQ4GEMVExercise {
4     public static func multiply(
5         _ weights: GroupwiseQ4WeightMatrix,
6         _ input: FloatTensor
7     ) throws -> FusedQ4GEMVResult {
8         guard input.rank == 1 else {
9             throw TensorError.rankMismatch(expected: 1, actual: input.rank)
10        }
11        guard input.shape[0] == weights.inputChannels else {
12            throw DenseLinearAlgebraError.innerDimensionMismatch(
13                operation: "fused Q4 GEMV", lhs: weights.inputChannels, rhs: input.shape[0])
14        }
15        for (index, value) in input.storage.enumerated() where !value.isFinite {
16            throw WeightQuantizationError.nonFiniteValue(index: index, value: value)
17        }
18        return FusedQ4GEMVResult(
19            output: try FloatTensor(
20                Array(repeating: 0, count: weights.outputChannels), shape: [weights.outputChannels]),
21            logicalWeightBytes: weights.allocatedBytes,
22            temporaryWeightBytes: 0)
23    }
24 }
```

34.18.2 Sources/LeetLLMExercises/P033FusedQ4GEMVMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P033FusedQ4GEMVExercise {
5     public static func makeMetalPipeline() throws -> MetalFusedQ4GEMVPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P033FusedQ4GEMV", withExtension: "metal", subdirectory: "Metal")
8         else { throw MetalNeuralOperatorError.kernelResourceMissing("P033FusedQ4GEMV.metal") }
9         return try MetalFusedQ4GEMVPipeline(source: String(contentsOf: url, encoding: .utf8))
10    }
11 }
```

34.18.3 Sources/LeetLLMExercises/Metal/P033FusedQ4GEMV.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 kernel void fused_q4_gemv(
5     device const uchar *packedWeights [[buffer(0)]],
6     device const float *scales [[buffer(1)]],
7     device const float *input [[buffer(2)]],
8     device float *output [[buffer(3)]],
9     constant uint4 &shape [[buffer(4)]],
10    uint lane [[thread_index_in_threadgroup]],
11    uint row [[threadgroup_position_in_grid]] {
```

```
12  if (lane == 0 && row < shape.x) output[row] = 0.0f;  
13  }
```

34.19 Worked solution

Solution strategy

Fuse Q4 unpacking, scale loading, and GEMV accumulation into one operator. The CPU reference iterates output rows and columns, loads one Q4 nibble via `quantizedValue`, multiplies by the group scale (`scaleIndex` maps column to the correct scale), multiplies by `input[column]`, and accumulates all products into one output value. The Metal kernel assigns one output row to each threadgroup, distributes columns across lanes using lane-strided loop (`column = lane, lane+256, ...`), extracts nibbles directly from packed bytes using bit operations, dequantizes in registers, and reduces using a 256-element threadgroup barrier. The format code (`signedTwosComplementLowNibbleFirst = 0`) is validated and passed to MSL. Both return `temporaryWeightBytes = 0` because no Float [O, I] temporary is allocated.

Correctness invariant

Output $y_r = \sum_{c=0}^{I-1} \text{quantizedValue}(r, c) * \text{scale}[\text{scaleIndex}(r, c)] * \text{input}[c]$. The accumulation is Float32 in both paths. No intermediate weight tensor is stored or read from memory. Each quantized weight is fetched and dequantized exactly once per GEMV call.

Common failure mode

Allocating a temporary Float weight matrix and claiming zero temporary bytes. Using the wrong nibble (swapping low/high). Missing the threadgroup barrier in the Metal reduction, causing races on the scratch array. Forgetting to sign-extend nibbles ≥ 8 . Validating format as an integer instead of checking the enum value.

34.19.1 Implementation points

Lane-strided loop distributes work across 256 threads

Lane `t` processes columns `t, t+256, t+512, ...`. This spreads `I` columns evenly across lanes. Each lane accumulates its columns independently, then the threadgroup reduction combines partial sums.

Quantized values are decoded in the register that consumes them

The Metal kernel does not allocate storage for `q_rc`. It reads the packed byte, extracts the nibble, sign-extends, multiplies by scale and input, and accumulates the product—all in one lane's local registers.

Threadgroup barrier synchronizes before reduction starts

Each lane writes its partial sum into threadgroup scratch at index `[lane]`. The barrier ensures all writes are visible before any lane reads `scratch[lane + stride]`. Without the barrier, reads of partially-written data race and produce wrong sums.

Sign extension uses bitwise OR for speed

Metal: if (nibble >= 8) quantized = int(nibble) - 16; CPU: Int8(bitPattern: nibble >= 8 ? nibble | 0xf0 : nibble). The bitwise form sets the sign bit without arithmetic.

Zero temporary weight bytes is the goal of fusion

FusedQ4GEMVResult.temporaryWeightBytes = 0 records that no [O, I] Float matrix was allocated. logicalWeightBytes counts the persistent packed values and scales: $\text{ceil}(O \cdot I / 2) + 4 \cdot O \cdot \text{ceil}(I / G)$. It does not claim that these are the only runtime bytes moved; the operator also reads the input and scales, writes output, and the Metal reduction uses threadgroup scratch.

34.19.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P033FusedQ4GEMVMetal.swift

- Line 4: extension P033FusedQ4GEMVSolution {
- Line 5: public static func makeMetalPipeline() throws -> MetalFusedQ4GEMVPipeline {

Sources/LeetLLMSolutions/P033FusedQ4GEMVSolution.swift

- Line 3: public enum P033FusedQ4GEMVSolution {
- Line 4: public static func quantize(
- Line 12: public static func multiply(
- Line 34: static func validate(

Sources/LeetLLMSolutions/Metal/P033FusedQ4GEMV.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

34.19.3 Complete canonical listings**34.19.4 Sources/LeetLLMSolutions/P033FusedQ4GEMVMetal.swift**

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P033FusedQ4GEMVSolution {
5     public static func makeMetalPipeline() throws -> MetalFusedQ4GEMVPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P033FusedQ4GEMV", withExtension: "metal", subdirectory: "Metal")
8         else { throw MetalNeuralOperatorError.kernelResourceMissing("P033FusedQ4GEMV.metal") }
9         return try MetalFusedQ4GEMVPipeline(source: String(contentsOf: url, encoding: .utf8))
10     }
11 }

```

34.19.5 Sources/LeetLLMSolutions/P033FusedQ4GEMVSolution.swift

```

1 import LeetLLMCore
2
3 public enum P033FusedQ4GEMVSolution {
4     public static func quantize(
5         _ weights: FloatTensor,
6         groupSize: Int
7     ) throws -> GroupwiseQ4WeightMatrix {
8         try WeightQuantizationSolutionSupport.quantizeGroupwiseQ4(
9             weights, groupSize: groupSize)

```

```

10 }
11
12 public static func multiply(
13     _ weights: GroupwiseQ4WeightMatrix,
14     _ input: FloatTensor
15 ) throws -> FusedQ4GEMVResult {
16     try validate(weights: weights, input: input)
17     var output = Array(repeating: Float.zero, count: weights.outputChannels)
18     for row in 0..

```

34.19.6 Sources/LeetLLMSolutions/Metal/P033FusedQ4GEMV.metal

```

1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint q4ThreadgroupWidth = 256;
5 constant uint signedTwosComplementLowNibbleFirst = 0;
6
7 kernel void fused_q4_gemv(
8     device const uchar *packedWeights [[buffer(0)]],
9     device const float *scales [[buffer(1)]],
10    device const float *input [[buffer(2)]],
11    device float *output [[buffer(3)]],
12    constant uint4 &shape [[buffer(4)]],
13    uint lane [[thread_index_in_threadgroup]],
14    uint row [[threadgroup_position_in_grid]] {
15    threadgroup float partials[q4ThreadgroupWidth];
16    float sum = 0.0f;
17    uint groupsPerRow = (shape.y + shape.z - 1) / shape.z;
18    if (shape.w == signedTwosComplementLowNibbleFirst) {
19        for (uint column = lane; column < shape.y; column += q4ThreadgroupWidth) {
20            uint logicalIndex = row * shape.y + column;
21            uchar byte = packedWeights[logicalIndex / 2];
22            uchar nibble = (logicalIndex & 1) == 0 ? byte & 0x0f : byte >> 4;
23            int quantized = nibble >= 8 ? int(nibble) - 16 : int(nibble);
24            float scale = scales[row * groupsPerRow + column / shape.z];
25            sum += float(quantized) * scale * input[column];
26        }
27    }
28 }

```



```
28 partials[lane] = sum;
29 threadgroup_barrier(mem_flags::mem_threadgroup);
30 for (uint stride = q4ThreadgroupWidth / 2; stride > 0; stride >> 1) {
31     if (lane < stride) partials[lane] += partials[lane + stride];
32     threadgroup_barrier(mem_flags::mem_threadgroup);
33 }
34 if (lane == 0) output[row] = partials[0];
35 }
```

Quantization Error Propagation

35.1 Why this exists

A final-token difference does not identify its cause. Weight approximation can accumulate across layers, but wrong nibble order, scale association, matrix orientation, or normalization convention can create much larger systematic differences. Calling every discrepancy “quantization noise” hides conversion bugs behind a precision label.

This CPU investigation runs the same deterministic operator pipeline with Float32, groupwise INT8, groupwise Q4, and one deliberately mismatched Q4 decoder. It captures every layer before classifying evidence.

35.2 Learning outcomes

You can:

- capture corresponding intermediate outputs across four execution variants;
- calculate cosine similarity, maximum absolute error, RMSE, and argmax agreement;
- locate the first layer whose argmax diverges;
- inject one known format fault while holding topology and scales fixed;
- classify a controlled convention mismatch from structural evidence rather than one threshold; and
- describe what evidence remains before making a model-quality claim.

35.3 Prerequisites

- Problem 010 for normalization-convention discipline.
- Problem 011 for precision-boundary experiments.
- Problems 029-031 for INT8/Q4 conversion and byte conventions.
- Problems 032-033 for staged and fused projection parity.

35.4 Vocabulary

- **Layer capture:** the complete output vector recorded at an operator boundary.
- **Cosine similarity:** directional agreement normalized by vector magnitudes.
- **Argmax agreement:** whether two vectors select the same maximum index.
- **First divergent layer:** earliest captured layer with argmax disagreement.
- **Controlled intervention:** changing one known convention while holding other variables fixed.
- **Convention mismatch:** producer and consumer interpret the same bytes or parameters differently.
- **Inconclusive:** evidence does not isolate a cause.

35.5 Pipeline and worked metric example

The deterministic investigation uses width-five states and three square weight matrices. Each layer computes

$$h^{(l+1)} = \tanh(W^{(l)}h^{(l)}).$$

Four states advance independently:

1. Float32 weights.
2. Groupwise INT8 weights with the requested group size.
3. Canonical low-nibble-first groupwise Q4 weights.
4. The same Q4 bytes and scales decoded high-nibble-first.

Suppose one Float capture is $a=[1,2]$ and a candidate is $b=[1,1]$.

$$\cos(a, b) = \frac{a \cdot b}{\|a\|_2 \|b\|_2} = \frac{3}{\sqrt{5}\sqrt{2}} \approx 0.9487,$$

$$E_{\max} = 1, \quad E_{\text{RMSE}} = \sqrt{(0^2 + 1^2)/2} \approx 0.7071.$$

Both vectors have argmax index 1, so argmax agreement is true despite nonzero error. No single metric tells the whole story.

35.6 Shape, layout, and dtype contract

Initial state is finite contiguous Float32 [D]. Every layer weight is finite contiguous Float32 [D,D] in [out,in] orientation. Group size is positive. The request initializer rejects rank, shape, finite-value, and group errors.

All four state streams and captures are Float32 [D]. Weight quantization uses Problem 029's deterministic rounding. Metric reductions use Double and are stored as Float. Layer indices are zero-based. First divergence is derived from argmax disagreement and may be nil; absence of argmax divergence does not mean the vectors are identical.

35.7 CPU investigation path

For each layer, the canonical implementation quantizes the Float weights to groupwise INT8 and Q4. It computes four GEMVs from the four incoming states, applies tanh, stores all output tensors, then calculates three metric sets against the Float capture. It does not reset quantized states to the Float state, so earlier differences can propagate.

For the mismatch stream, only nibble selection changes. Packed bytes, scales, group size, matrix orientation, input state topology, activation, and accumulation dtype remain fixed. The report counts decoded logical values that change under this intervention.

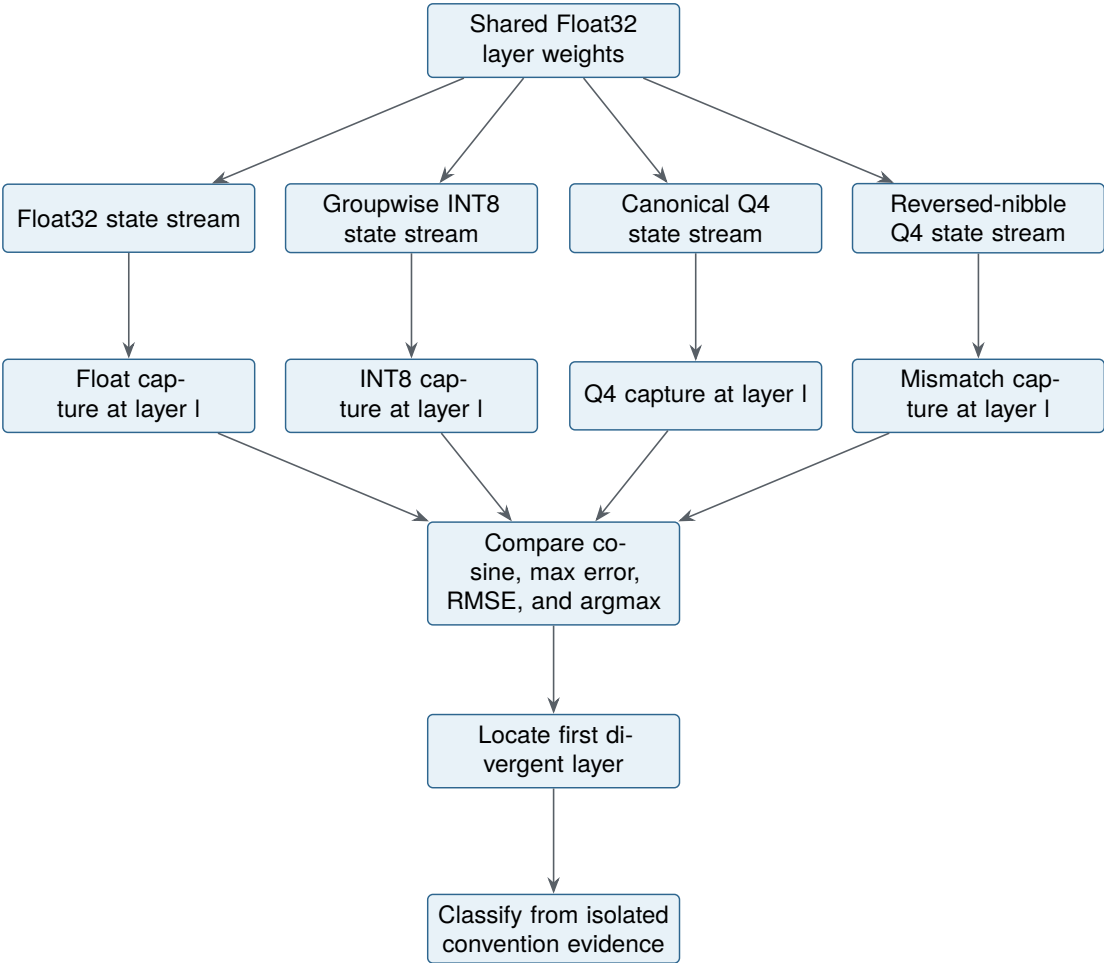


Figure 35.1: Propagation diagnostic.

35.8 Independent correctness

The judge owns a separate three-layer fixture and oracle. It independently quantizes weights, reconstructs INT8/Q4 values, runs Double-accumulated GEMV, applies tanh, calculates all metrics, and derives divergence indices. It compares every captured vector rather than accepting only summary numbers.

A test returns correct numerical captures but labels the known intervention inconclusive; the judge rejects it. This prevents a hardcoded metric threshold from replacing the causal evidence in the controlled experiment.

```
1 swift run leetllm check 034 --cpu
2 swift run leetllm check 034 --solution
```

35.9 Convention-mismatch diagnostic

Canonical Q4 stores even logical values in low nibbles. The deliberate fault reads even values from high nibbles and odd values from low nibbles. For an odd logical count, the last value reads the required zero padding nibble.

Classification is `conventionMismatch` because two pieces of structural evidence are available:

- the intervention is known and isolated to nibble order; and
- at least one decoded integer changed.

RMSE, cosine, and argmax results show consequences, but no fixed threshold makes the classification. If the intervention changed no decoded value, the report would be inconclusive regardless of a chosen numerical cutoff.

In a real conversion investigation, replace the deliberate intervention with verified producer/consumer evidence: inspect upstream format code, capture pre-normalization states, and compare layer by layer before blaming precision.

35.10 Bytes and arithmetic-intensity model

For L square layers of width D , persistent Float32 weights use

$$B_{32} = 4LD^2.$$

Groupwise INT8 and Q4 use

$$B_8 = LD^2 + 4LD \left\lceil \frac{D}{G} \right\rceil,$$

$$B_4 = L \left\lceil \frac{D^2}{2} \right\rceil + 4LD \left\lceil \frac{D}{G} \right\rceil.$$

The investigation executes all variants and stores layer captures, so it is not a performance benchmark. Each variant still performs about $2LD^2$ useful GEMV FLOPs. Capture bytes are evidence overhead accepted for diagnosis and should not be confused with production inference traffic.

35.11 Metal mapping

Problem 034 is CPU-only because its purpose is synchronized capture and causal comparison, not a new kernel. Problem 033 already validates actual MSL Q4 unpacking. A production investigation can instrument both CPU and Metal at the same named boundaries, but adding an uninstrumented GPU path here would make the evidence harder to inspect.

35.12 Implementation checkpoints

1. Run one Float layer and record its complete output.
2. Quantize the same weights to INT8 and Q4 with identical grouping.
3. Carry each variant's own state into the next layer.
4. Calculate all four requested metrics from captures.
5. Derive first argmax divergence without inventing a tolerance.
6. Reverse only nibble order and count changed decodes.
7. Classify from intervention evidence, then inspect numerical consequences.

35.13 Controlled experiments

35.13.1 Depth sweep

Repeat a deterministic layer family for increasing depth. Prediction: RMSE can grow, shrink, or saturate through tanh; it is not guaranteed to increase monotonically.

35.13.2 Group-size sweep

Compare $G=D$, $G=64$, and smaller groups. Prediction: metadata rises and local weight error often falls, but argmax can remain unchanged even when RMSE moves.

35.13.3 Convention intervention

Inject nibble order, scale-row association, transpose, or RMSNorm gamma convention one at a time. Prediction: the first affected capture and directional error pattern localize the boundary better than final output alone.

35.13.4 Near-tie logits

Construct a final vector with two nearly equal maxima. Prediction: argmax can flip while cosine remains high; report both facts without declaring the entire representation invalid from one sample.

35.14 Engine integration

Use this report when qualifying converted checkpoints or new quantized kernels. Capture residual or projection states at stable layer boundaries, compare them with a trusted path, and route the first divergence to format, normalization, layout, or arithmetic checks. Only after conventions agree should aggregate precision metrics inform model-quality and deployment decisions.

35.15 Tradeoffs

- Full captures localize faults but consume memory and synchronization time.
- Cosine is scale-insensitive; maximum error is outlier-sensitive; RMSE summarizes energy; argmax reflects one decision.
- Deterministic fixtures are reproducible but do not replace representative model evaluation.
- A controlled mismatch proves diagnostic machinery, not that every large error is a convention bug.

35.16 Hints

- Keep variant states separate after layer zero.
- Define one stable tie behavior for argmax by using the first maximum index.
- Handle zero-norm cosine explicitly.
- Do not infer the cause from a metric threshold when intermediate evidence is available.
- Verify format, scale, orientation, normalization, and downcast boundaries in that order.

35.17 Completion checklist

- ☐ Float32, INT8, Q4, and mismatched states are captured per layer.
- ☐ Cosine, maximum absolute error, RMSE, and argmax agreement are all reported.
- ☐ First divergent layer is derived from captures.
- ☐ The deliberate mismatch changes only nibble interpretation.
- ☐ Classification uses controlled structural evidence, not one threshold.
- ☐ No arbitrary output difference is named quantization noise.
- ☐ An experiment records predictions and retains intermediate evidence.

35.18 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

35.18.1 Sources/LeetLLMExercises/P034QuantizationPropagationExercise.swift

```

1 import LeetLLMCore
2
3 public enum P034QuantizationPropagationExercise {
4     public static func investigate(
5         _ request: QuantizationPropagationRequest
6     ) throws -> QuantizationPropagationReport {
7         let captures = try request.weights.enumerated().map { layer, weights in
8             let zeros = try FloatTensor(Array(repeating: 0, count: weights.shape[0]), shape: [weights.shape[0]])
9             let metrics = VectorComparisonMetrics(
10                 cosineSimilarity: 1,
11                 maximumAbsoluteError: 0,
12                 rootMeanSquareError: 0,
13                 argmaxAgreement: true)
14             return QuantizationLayerCapture(
15                 layer: layer,
16                 float32Output: zeros,
17                 int8Output: zeros,
18                 q40Output: zeros,
19                 mismatchedQ40Output: zeros,
20                 int8Metrics: metrics,
21                 q4Metrics: metrics,
22                 mismatchMetrics: metrics)
23         }
24         return QuantizationPropagationReport(
25             layers: captures,
26             firstInt8DivergentLayer: nil,
27             firstQ4DivergentLayer: nil,
28             mismatchDiagnostic: ConventionMismatchDiagnostic(
29                 injectedFault: .highNibbleFirstInsteadOfLowNibbleFirst,
30                 changedDecodedValueCount: 0,
31                 firstDivergentLayer: nil,
32                 classification: .inconclusive))
33     }
34 }

```


35.19 Worked solution

Solution strategy

Run four independent execution streams through the same pipeline: (1) Float32 weights, (2) Groupwise INT8 weights with requested groupSize, (3) Groupwise Q4 weights with requested groupSize, (4) The same Q4 bytes and scales but decoded with reversed nibble order. Each stream starts with the initial Float state, quantizes the Float weights once (not reset), applies a GEMV (float, int8, Q4-canonical, or Q4-reversed), applies tanh activation, and captures the output. For each layer, compute metrics between the Float output and each candidate: cosine similarity, maximum absolute error, RMSE (all using Double during reduction), and argmax agreement (check if argmax indices match). Record the first layer where INT8, Q4, or reversed-Q4 streams show argmax disagreement. For the mismatch stream, count how many logical Q4 values differ under low-nibble-first versus high-nibble-first decoding. This deterministic investigation isolates convention bugs from inherent quantization noise.

Correctness invariant

Streams do not reset to Float state between layers, so earlier approximation errors propagate to later layers. Cosine similarity is computed as $(a \cdot b) / (\|a\| \|b\|)$ using Double. Argmax agreement is true iff both vectors select the same maximum index. The reversed-nibble stream proves whether a nibble-order error exists by counting changed values under the deliberate fault.

Common failure mode

Resetting quantized states to Float after each layer breaks error accumulation and makes layers appear independent. Using only cosine similarity without argmax agreement misses cases where vectors are directionally similar but the peak value shifts. Not injecting a controlled format fault leaves the diagnosis inconclusive if reversal count is zero. Blaming quantization noise when the first divergence is at layer 0 (likely a scale or convention mismatch, not noise).

35.19.1 Implementation points

Layer capture preserves intermediate state without reset

Each layer's output becomes the input to the next layer. States are not reset to Float, so accumulated errors from layer 0 propagate forward. This reveals whether errors diverge at layer 1 or accumulate silently until a later layer flips argmax.

Metrics must include both direction and magnitude

Cosine similarity $\in [0, 1]$ measures directional agreement. Maximum error and RMSE measure magnitude. Argmax agreement is a discrete binary: two vectors with $\cos(a, b) = 0.95$ may still have different argmax indices if peaks are near-ties.

Double precision accumulation avoids metric bias

Cosine similarity and RMSE are computed in Double to avoid single-precision rounding bias in the metric itself. A difference between streams may be sub-Float-epsilon but still meaningful; Double accumulation makes it visible.

Controlled nibble-order fault isolates convention mismatches

reversedNibbleValue decodes the same packed bytes with high-nibble-first (the opposite of the canonical format). If changedDecodedValueCount > 0, then many Q4 values change meaning under this reversal. If streams agree despite the reversal, the divergence is not nibble order.

First divergence layer narrows the search space

If INT8 and Float agree through layer 2 but diverge at layer 3, the error originated in layer 3's weights or computation, not earlier. Similarly, if Q4 follows Float exactly until layer 5, quantization is behaving correctly through layer 4.

35.19.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P034QuantizationPropagationSolution.swift

- Line 4: public enum P034QuantizationPropagationSolution {
- Line 5: public static func investigate(
- Line 54: private static func floatGEMV(_ weights: [Float], input: [Float], width: Int) -> [Float] {
- Line 64: private static func int8GEMV(
- Line 80: private static func q4GEMV(
- Line 100: private static func reversedNibbleValue(
- Line 109: private static func changedValueCount(_ weights: GroupwiseQ4WeightMatrix) -> Int {
- Line 118: private static func activate(_ values: [Float]) -> [Float] {
- Line 122: private static func metrics(
- Line 151: private static func argmax(_ values: [Float]) -> Int? {

35.19.3 Complete canonical listings**35.19.4 Sources/LeetLLMSolutions/P034QuantizationPropagationSolution.swift**

```

1 import Darwin
2 import LeetLLMCore
3
4 public enum P034QuantizationPropagationSolution {
5     public static func investigate(
6         _ request: QuantizationPropagationRequest
7     ) throws -> QuantizationPropagationReport {
8         var floatState = request.initial.storage
9         var int8State = request.initial.storage
10        var q4State = request.initial.storage
11        var mismatchState = request.initial.storage
12        var captures: [QuantizationLayerCapture] = []
13        var changedDecodedValues = 0
14
15        for (layer, floatWeights) in request.weights.enumerated() {

```

```

16     let int8Weights = try WeightQuantizationSolutionSupport.quantizeGroupwiseInt8(
17         floatWeights, groupSize: request.groupSize)
18     let q4Weights = try WeightQuantizationSolutionSupport.quantizeGroupwiseQ4(
19         floatWeights, groupSize: request.groupSize)
20     changedDecodedValues += changedValueCount(q4Weights)
21
22     floatState = activate(floatGEMV(floatWeights.storage, input: floatState, width: floatWeights.shape[0]))
23     int8State = activate(int8GEMV(int8Weights, input: int8State))
24     q4State = activate(q4GEMV(q4Weights, input: q4State, reverseNibbleOrder: false))
25     mismatchState = activate(q4GEMV(
26         q4Weights, input: mismatchState, reverseNibbleOrder: true))
27
28     let floatOutput = try FloatTensor(floatState, shape: [floatWeights.shape[0]])
29     let int8Output = try FloatTensor(int8State, shape: [floatWeights.shape[0]])
30     let q4Output = try FloatTensor(q4State, shape: [floatWeights.shape[0]])
31     let mismatchOutput = try FloatTensor(mismatchState, shape: [floatWeights.shape[0]])
32     captures.append(QuantizationLayerCapture(
33         layer: layer,
34         float32Output: floatOutput,
35         int8Output: int8Output,
36         q4Output: q4Output,
37         mismatchedQ4Output: mismatchOutput,
38         int8Metrics: metrics(reference: floatState, candidate: int8State),
39         q4Metrics: metrics(reference: floatState, candidate: q4State),
40         mismatchMetrics: metrics(reference: floatState, candidate: mismatchState)))
41 }
42
43 return QuantizationPropagationReport(
44     layers: captures,
45     firstInt8DivergentLayer: captures.first { !$0.int8Metrics.argmaxAgreement }?.layer,
46     firstQ4DivergentLayer: captures.first { !$0.q4Metrics.argmaxAgreement }?.layer,
47     mismatchDiagnostic: ConventionMismatchDiagnostic(
48         injectedFault: .highNibbleFirstInsteadOfLowNibbleFirst,
49         changedDecodedValueCount: changedDecodedValues,
50         firstDivergentLayer: captures.first { !$0.mismatchMetrics.argmaxAgreement }?.layer,
51         classification: changedDecodedValues > 0 ? .conventionMismatch : .inconclusive))
52 }
53
54 private static func floatGEMV(_ weights: [Float], input: [Float], width: Int) -> [Float] {
55     (0..

```

```

89     let quantized = reverseNibbleOrder
90     ? reversedNibbleValue(weights, logicalIndex: logicalIndex)
91     : weights.quantizedValue(outputChannel: row, inputChannel: column)
92     sum += Float(quantized)
93     * weights.scales[weights.scaleIndex(outputChannel: row, inputChannel: column)]
94     * input[column]
95 }
96 return sum
97 }
98 }
99
100 private static func reversedNibbleValue(
101     _ weights: GroupwiseQ4WeightMatrix,
102     logicalIndex: Int
103 ) -> Int8 {
104     let byte = weights.packedValues[logicalIndex / 2]
105     let nibble = logicalIndex.isMultiple(of: 2) ? byte » 4 : byte & 0x0f
106     return Int8(bitPattern: nibble >= 8 ? nibble | 0xf0 : nibble)
107 }
108
109 private static func changedValueCount(_ weights: GroupwiseQ4WeightMatrix) -> Int {
110     (0..

```

Part VII

Assemble Inference

One Decoder Transformer Block

36.1 Why this exists

A decoder layer is not a bag of operators. Its order is part of the checkpoint contract. Normalizing the wrong residual, adding a branch too early, omitting RoPE, or mapping a query head to the wrong KV head can produce finite, plausible-looking output while implementing a different model.

This lesson assembles the earlier operators into one readable pre-norm block. It deliberately captures every semantic boundary. Problem 039 will choose prefill dispatches for this block, Problem 040 will replace full-sequence attention with cache-aware one-token attention, and Problem 043 will decide which intermediate tensors evidence justifies fusing.

36.2 Learning outcomes

You can:

- state and implement the exact order of a pre-norm decoder block;
- derive every Q/K/V, head, attention, and MLP shape from one configuration;
- distinguish P014's [input,output] teaching matrix from this checkpoint-facing [output,input] convention;
- apply adjacent-pair RoPE at an explicit absolute position offset;
- map H_q query heads onto H_{kv} shared KV heads;
- identify both residual sources and both normalization inputs; and
- use captured intermediates to localize an order or convention defect.

36.3 Prerequisites

- Problems 002, 004, and 005 for contiguous tensors and dense projections.
- Problems 008 and 010 for direct-gamma SwiGLU and RMSNorm.
- Problems 014-018 for head shapes, RoPE, causal attention, and GQA.
- Problems 022-028 for the cache contracts that Problem 040 will connect.
- Problems 029-034 for checkpoint-oriented [out,in] weight storage and convention diagnosis.

36.4 Vocabulary

- **Residual stream:** the shared $[S,D]$ state updated by attention and MLP branches.
- **Pre-norm:** normalize a residual before computing its branch.
- **Attention branch:** Q/K/V projection, RoPE, causal GQA, head concatenation, and output projection.
- **MLP branch:** gate/up projection, SiLU gate, elementwise product, and down projection.
- **Position offset:** absolute position of sequence row zero.

- **Capture:** a named intermediate retained so correctness can be checked before the final residual hides the first divergence.

36.5 Math and exact order

Let x be the incoming residual $[S, D]$. RMSNorm uses the direct gamma convention from Problem 010:

$$N(x)_j = \frac{x_j}{\sqrt{D^{-1} \sum_i x_i^2 + \epsilon}} \gamma_j.$$

The attention branch is

$$A_0 = N_{\text{attn}}(X), \quad Q = A_0 W_Q^T, \quad K = A_0 W_K^T, \quad V = A_0 W_V^T,$$

$$(\tilde{Q}, \tilde{K}) = \text{RoPE}(Q, K, p_0),$$

$$C = \text{CausalGQA}(\tilde{Q}, \tilde{K}, V), \quad R_1 = X + C W_O^T.$$

For query row i , query head h , and visible key row $j \leq i$,

$$s_{i,h,j} = \frac{\tilde{q}_{i,h} \cdot \tilde{k}_{j, \lfloor h/(H_q/H_{kv}) \rfloor}}{\sqrt{d_h}}.$$

Softmax is applied only over visible keys. The MLP branch is

$$M_0 = N_{\text{mlp}}(R_1), \quad G = M_0 W_G^T, \quad U = M_0 W_U^T,$$

$$Z = \text{SiLU}(G) \odot U, \quad R_2 = R_1 + Z W_D^T.$$

R_2 is the block output. Neither norm is applied after its residual add.

36.5.1 Worked values

For one row $x=[3, 4]$, $\gamma=[1, 1]$, and negligible positive epsilon,

$$\text{rms}(x) = \sqrt{(9 + 16)/2} = \sqrt{12.5} \approx 3.5355,$$

so the normalized row is approximately $[0.8485, 1.1314]$. With one token, causal attention has one visible key, so its softmax weight is exactly 1 and the head output equals $v[0]$. The output projection is still required: a one-token sequence simplifies the mask, not the learned projection or either residual add.

36.6 Shape, layout, and dtype contract

Batch size is exactly one and is omitted from shapes. Sequence length $S \geq 1$. All tensors are contiguous row-major Float32 and all projection accumulations in the learner path are Float32.

Value	Shape	Stored order
residual	$[S, D]$	token, model feature
attention/MLP gamma	$[D]$	model feature
W_q	$[H_q * d_h, D] = [D, D]$	output, input
W_k, W_v	$[H_{kv} * d_h, D]$	output, input
Q	$[S, H_q, d_h]$	token, head, feature
K, V	$[S, H_{kv}, d_h]$	token, KV head, feature
attention heads	$[S, H_q, d_h]$	token, query head, feature
concatenated heads	$[S, D]$	query heads contiguous within token
W_o	$[D, H_q * d_h]$	output, concatenated input
W_{gate}, W_{up}	$[F, D]$	MLP output, model input
gate/up/hidden	$[S, F]$	token, MLP feature
W_{down}	$[D, F]$	model output, MLP input

The configuration requires $D = H_q * d_h$, $H_q \% H_{kv} == 0$, positive dimensions, positive even rotaryDimension $\leq d_h$, finite positive epsilon, and finite ropeBase > 1 . The state validates rank, width, finite input, non-negative position offset, and position arithmetic overflow. Every weight shape and every weight value is checked before computation.

This lesson intentionally differs from P014's $[D, \text{projection}]$ matrix because the model-loader and quantized GEMV boundary use $[\text{out}, \text{in}]$. Orientation is never inferred from equal dimensions.

36.7 CPU reference path

Implement these checkpoints in order:

1. RMS-normalize each incoming residual row with attentionNormGamma.
2. Project Q, K , and V with $[\text{out}, \text{in}]$ row dot products.
3. Reinterpret contiguous projection outputs as head tensors.
4. Rotate Q and K adjacent feature pairs at positionOffset + token.
5. Compute scaled causal GQA with $kvHead = queryHead / (H_q / H_{kv})$.
6. Concatenate heads without changing their token-local order.
7. Apply W_o , then add the original incoming residual to form R_1 .
8. RMS-normalize R_1 with mlpNormGamma.
9. Project gate and up, compute $\text{SiLU}(\text{gate}) * \text{up}$, and project down.
10. Add the down projection to R_1 to form R_2 .

The canonical implementation favors named tensors and loops over reuse. The result retains normalized states, projected and rotated $Q/K, V$, attention heads, both residual boundaries, all MLP projections, the activated gate, and the gated hidden state.

36.8 Independent correctness

The judge owns non-symmetric deterministic weights and a separate Double oracle. It compares all captured stages with absolute tolerance $4e-5$ plus relative tolerance $8e-5$. One fixture uses GQA and a nonzero position offset; another uses three causal rows.

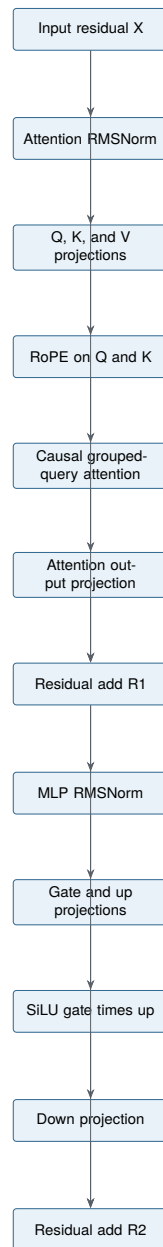


Figure 36.1: Prenorm decoder order.

Tests deliberately replace rotated captures with unrotated Q/K and replace the MLP norm input with the first norm. Both are rejected even if other output is copied from the canonical result. Error cases cover empty sequences, residual width, negative offsets, wrong weight shapes, non-finite state, invalid head division, and invalid rotary dimensions.

```
1 swift run leetllm check 035 --cpu
2 swift run leetllm check 035 --solution
```

36.9 Performance model: work, bytes, and allocation

Ignoring lower-order elementwise work, projections cost approximately

$$4SD^2 + 4SD(H_{kv}d_h) + 6SDF$$

FLOPs: Q and output are two $D \times D$ projections, K/V are two $H_{kv}d_h \times D$ projections, and gate/up/down are three MLP projections. Full causal attention adds approximately

$$2H_qd_hS(S+1)$$

FLOPs for query-key dots and weighted values.

Float32 weight bytes are

$$4(2D + 2D^2 + 2D(H_{kv}d_h) + 3DF).$$

The teaching implementation materializes every capture, so its transient bytes are intentionally much larger than a production layer. Record those allocations as diagnostic cost, not an engine memory target.

36.10 Metal mapping

Problem 035 is CPU-only. It establishes semantics and testable order; it does not pretend that invoking the CPU implementation is a Metal path. Earlier lessons already provide Metal kernels for RMSNorm, RoPE, attention, and selected projections. A literal GPU composition would dispatch each stage and write each capture to device memory. That is useful for parity but not automatically fast.

Problem 039 will choose GEMM-oriented prefill dispatches, Problem 040 will use cached single-token attention and GEMV-oriented decode, and Problem 043 will use profiles to decide which norm/projection/gate boundaries merit fusion. Any future Metal path must preserve the same captures in a diagnostic mode.

36.11 Implementation checkpoints

1. Make configuration and weight validation pass before arithmetic.
2. Match the first RMSNorm capture.
3. Match Q/K/V values and head shapes before RoPE.
4. Match rotated Q/K at offset zero and a nonzero offset.
5. Match one-token attention, then multi-token causal GQA.
6. Match R1 and verify it adds the original x.
7. Match MLP norm, gate, up, SiLU, product, and down captures.
8. Match R2 and all error cases.

36.12 Controlled experiments

36.12.1 Position-offset intervention

Run identical rows at offsets 0 and 32. Prediction: V is unchanged, Q/K before RoPE are unchanged, rotated Q/K change, and attention may change.

36.12.2 GQA head-count sweep

Hold H_q and d_h fixed while comparing $H_{kv}=H_q$, $H_q/2$, and 1 with compatible weights. Prediction: K/V projection and future cache bytes decrease; query and output projection work do not.

36.12.3 Sequence-length sweep

Compare $S=1, 8, 64$. Prediction: projection work grows linearly, while this full causal attention path approaches quadratic work. Do not use this result to claim decode is quadratic after Problem 040 introduces caching.

36.12.4 Capture removal

Measure peak allocations with all captures and with selected captures disabled in a private experiment. Prediction: elapsed time and memory improve, but fault localization becomes coarser. Keep the canonical diagnostic path intact.

36.13 Engine integration

`DecoderConfiguration`, `DecoderBlockWeights`, `DecoderBlockState`, and `DecoderBlockResult` are the shared API for Problems 036 and 039-047. Problem 036 loads the configuration and exact named weights. Problem 039 feeds a prompt sequence and uses the returned residual. Problem 040 will adapt the attention stage to append/read KV state while reusing the same projection, norm, MLP, and sampling boundaries.

36.14 Tradeoffs

- `[out,in]` aligns GEMV and checkpoint storage; some GEMM libraries prefer a transposed view or packed representation.
- Full captures are excellent for parity and expensive for production.
- Shared KV heads reduce projection/cache cost but are fixed by checkpoint architecture.
- Float32 accumulation is readable and deterministic enough for this oracle; production mixed precision needs new parity evidence.
- Separate kernels maximize inspectability; fusion can remove traffic only at the cost of a larger correctness surface.

36.15 Hints

- Write every expected weight shape beside its validation.
- Preserve projected output order when reshaping; a reshape is not a transpose.
- Apply RoPE to Q and K, never V.
- Mask before softmax by omitting future keys.
- Save X for the first residual and R1 for the second.
- Direct gamma means multiply by gamma, not $1 + \text{gamma}$.

36.16 Completion checklist

- ☐ Every configuration, state, and weight error is rejected before computation.
- ☐ All 16 intermediate captures match the independent oracle.
- ☐ Nonzero-offset RoPE changes Q/K at the documented boundary.
- ☐ Causal GQA uses the documented query-to-KV head mapping.
- ☐ The first residual adds x ; the second adds $R1$.
- ☐ Weight orientation and direct-gamma semantics are explicit.
- ☐ Experiment predictions are recorded before measurements.
- ☐ No CPU result is presented as a Metal implementation.

36.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

36.17.1 Sources/LeetLLMExercises/P035DecoderBlockExercise.swift

```

1 import LeetLLMCore
2
3 public enum P035DecoderBlockExercise {
4     public static func apply(
5         state: DecoderBlockState,
6         weights: DecoderBlockWeights,
7         configuration: DecoderConfiguration
8     ) throws -> DecoderBlockResult {
9         try P035DecoderBlockContract.validate(
10             state: state, weights: weights, configuration: configuration)
11
12         let sequence = state.residual.shape[0]
13         let model = configuration.modelDimension
14         let hidden = configuration.hiddenDimension
15         let queryShape = [sequence, configuration.queryHeadCount, configuration.headDimension]
16         let keyValueShape = [sequence, configuration.keyValueHeadCount, configuration.headDimension]
17         let modelShape = [sequence, model]
18         let hiddenShape = [sequence, hidden]
19         let zeroModel = try FloatTensor(Array(repeating: 0, count: sequence * model), shape: modelShape)
20         let zeroQuery = try FloatTensor(
21             Array(repeating: 0, count: sequence * configuration.queryProjectionDimension),
22             shape: queryShape)
23         let zeroKeyValue = try FloatTensor(
24             Array(repeating: 0, count: sequence * configuration.keyValueProjectionDimension),
25             shape: keyValueShape)
26         let zeroHidden = try FloatTensor(
27             Array(repeating: 0, count: sequence * hidden), shape: hiddenShape)
28
29         // TODO: implement the ordered pre-norm attention and SwiGLU block.
30         return DecoderBlockResult(
31             state: state,
32             intermediates: DecoderBlockIntermediates(
33                 attentionNormalized: zeroModel,
34                 queries: zeroQuery,
35                 keys: zeroKeyValue,
36                 values: zeroKeyValue,
37                 rotatedQueries: zeroQuery,
38                 rotatedKeys: zeroKeyValue,
39                 attentionHeads: zeroQuery,
40                 concatenatedAttention: zeroModel,
41                 attentionProjection: zeroModel,
42                 postAttentionResidual: state.residual,
43                 mlpNormalized: zeroModel,
44                 gateProjection: zeroHidden,
45                 upProjection: zeroHidden,
46                 activatedGate: zeroHidden,
47                 gatedHidden: zeroHidden,
48                 downProjection: zeroModel))
49     }
50 }

```

36.18 Worked solution

Solution strategy

Assemble one complete pre-norm decoder block that sequences attention and MLP as independent branches off a shared residual stream. Normalize the input residual before Q/K/V projection, apply RoPE at explicit absolute position offsets after projection, compute causal grouped-query attention by mapping query heads to KV heads via integer division, concatenate attention heads back to model dimension, add the attention output to the original residual (first residual boundary), normalize post-attention residual before gate and up projections, apply SiLU activation to the gate and element-wise multiply with the up projection output, project down, and add to post-attention residual (second residual boundary). Capture intermediates at every semantic boundary to localize convention errors.

Correctness invariant

Both residual addition points (post-attention and post-MLP) must accumulate exactly once. RMSNorm must be applied before every projection using direct-gamma convention: $\text{output}[i] = \text{input}[i] * (\text{gamma}[i] / \sqrt{\text{mean}(\text{input}^2) + \text{epsilon}})$. RoPE must be applied to projected Q and K after reshaping to heads but before attention, not before projection. SwiGLU output is element-wise product of $\text{SiLU}(\text{gate})$ and up, never $\text{SiLU}(\text{gate} + \text{up})$ or other formulations.

Common failure mode

Applying RoPE before reshaping to head dimensions or after cache appending produces plausible finite logits but implements a different model. Normalizing the wrong residual input (e.g., applying MLP norm to the pre-attention residual instead of post-attention) breaks the pre-norm structure and diverges at layer 0. Failing to map query heads to KV heads during attention causes shape mismatches or silent correctness loss in grouped-query configurations.

36.18.1 Implementation points

RMSNorm direct-gamma formula

For input vector x and learned scale gamma , compute $\text{rms_scale} = 1 / \sqrt{((1/D) * \sum(x_i^2) + \text{epsilon})}$, then $\text{output}[i] = x[i] * \text{rms_scale} * \text{gamma}[i]$. The denominator is summed across the hidden dimension D , not across sequence positions. This is the checkpoint-facing convention used in Llama and similar models.

Head shape derivation from projections

Q projection output $[S, D] \times [D, D]$ produces $[S, D]$, reshaped to $[S, H_q, d_h]$ where H_q is query head count and d_h is head dimension. K/V projections output $[S, H_{kv} * d_h]$, reshaped to $[S, H_{kv}, d_h]$. The reshaping is a view operation with no data copying; integer division H_q/H_{kv} must be exact or the configuration is invalid.

GQA head mapping via integer division

For query head h and configuration with H_q query heads and H_{kv} KV heads, map to KV head $\text{floor}(h * H_{kv} / H_q)$. This is equivalent to $\text{floor}(h / (H_q/H_{kv}))$. All H_q/H_{kv} consecutive query heads attend to the same KV head. The `groupSize` parameter is H_q/H_{kv} .

SwiGLU activation and gating

Gate and up projections both receive the same normalized residual input. Compute `activated_gate = SiLU(gate_projection) = gate / (1 + exp(-gate))` and `gated_output = activated_gate` element-wise product `up_projection`. The down projection then processes this gated representation. Never add gate and up before activation.

36.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P035DecoderBlockSolution.swift

- Line 4: `public enum P035DecoderBlockSolution {`
- Line 5: `public static func apply(`
- Line 14: `func normalize(_ input: FloatTensor, gamma: FloatTensor) throws -> FloatTensor {`
- Line 32: `func project(_ input: FloatTensor, weights: FloatTensor) throws -> FloatTensor {`
- Line 49: `func reshapeHeads(_ projection: FloatTensor, headCount: Int) throws -> FloatTensor {`
- Line 55: `func rotate(_ input: FloatTensor) throws -> FloatTensor {`
- Line 78: `func causalAttention(`
- Line 121: `func add(_ lhs: FloatTensor, _ rhs: FloatTensor) throws -> FloatTensor {`

36.18.3 Complete canonical listings

36.18.4 Sources/LeetLLMSolutions/P035DecoderBlockSolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P035DecoderBlockSolution {
5     public static func apply(
6         state: DecoderBlockState,
7         weights: DecoderBlockWeights,
8         configuration: DecoderConfiguration
9     ) throws -> DecoderBlockResult {
10         try P035DecoderBlockContract.validate(
11             state: state, weights: weights, configuration: configuration)
12         let sequenceLength = state.residual.shape[0]
13
14         func normalize(_ input: FloatTensor, gamma: FloatTensor) throws -> FloatTensor {
15             let width = input.shape[1]
16             var output = Array(repeating: Float.zero, count: input.elementCount)
17             for row in 0..

```

```

26         input.storage[row * width + column] * inverseRMS * gamma.storage[column]
27     }
28 }
29 return try FloatTensor(output, shape: input.shape)
30 }
31
32 func project(_ input: FloatTensor, weights: FloatTensor) throws -> FloatTensor {
33     let inputWidth = input.shape[1]
34     let outputWidth = weights.shape[0]
35     var output = Array(repeating: Float.zero, count: input.shape[0] * outputWidth)
36     for row in 0..

```



```

99         }
100         scores[key] = dot * scale
101     }
102     let maximum = scores.max()!
103     let exponentials = scores.map { exp($0 - maximum) }
104     let denominator = exponentials.reduce(0, +)
105     for feature in 0..

```

```
172     gatedHidden: gatedHidden,  
173     downProjection: downProjection))  
174 }  
175 }
```

Parse a Model Weight Format

37.1 Why this exists

A weight file controls allocation sizes, tensor shapes, byte ranges, model configuration, and every numerical value passed to the decoder. Treating its metadata as trusted can turn one corrupt length or offset into an out-of-bounds read, overlapping tensors, a silent transpose, or a model built with incompatible head counts.

This lesson defines `LeetWeight`, a small educational container used by later mini-model fixtures. It resembles the broad idea of a metadata header followed by raw tensor bytes, but it is not `SafeTensors`, `GGUF`, or compatible with either. The narrow format makes every trust decision visible.

37.2 Learning outcomes

You can:

- decode fixed-width integers explicitly as little-endian bytes;
- validate a length-delimited UTF-8 JSON header before parsing its schema;
- derive a tensor byte count from dtype and shape with checked arithmetic;
- reject duplicate names, unaligned or overlapping ranges, and payload escapes;
- decode `Float32` without unsafe or unaligned typed loads;
- require named tensors before constructing a decoder block; and
- separate a convenient fixture encoder from an adversarial parser.

37.3 Prerequisites

- Problem 002 for shape products, contiguous storage, and overflow checks.
- Problems 029-031 for the rule that producer and consumer byte conventions must agree.
- Problem 035 for the configuration and nine block-weight shapes loaded here.
- Basic UTF-8, JSON objects/arrays, and fixed-width integer notation.

37.4 Vocabulary

- **Preamble:** fixed 20-byte prefix containing magic, version, and header length.
- **Magic:** eight identifying bytes, ASCII `LLMWGT01`.
- **Descriptor:** JSON record naming one tensor and declaring dtype, shape, payload-relative offset, and byte count.
- **Payload:** raw tensor bytes after the aligned header.
- **Relative offset:** byte position measured from payload start, not file start.
- **Alignment:** requirement that each `Float32` tensor offset is divisible by four.

- **Overlap:** two nonempty descriptor ranges claim at least one common payload byte.
- **Trust boundary:** the point at which external bytes become typed engine data.

37.5 Math and binary contract with worked bytes

The file layout is:

File bytes	Field	Encoding
0.. ₈	magic	ASCII LLMWGT01
8.. ₁₂	version	UInt32 little-endian, currently 1
12.. ₂₀	JSON header length H	UInt64 little-endian
20.. _(20+H)	header	exactly H UTF-8 JSON bytes
through next multiple of 8	padding	zero bytes only
remaining bytes	payload	descriptor offsets are relative to this byte

For version 1, bytes 01 00 00 00 decode to UInt32 1. Float32 1.0 has bits 0x3f800000, so its payload bytes are 00 00 80 3f. Float32 -2.5 has bits 0xc0200000, so its bytes are 00 00 20 c0. Reading these four-byte groups as native pointers would make alignment and host byte order implicit; the canonical parser shifts bytes into a UInt32 and then uses `Float(bitPattern:)`.

37.6 JSON metadata contract

The header has one model object and an array of tensor descriptors:

```

1 {
2   "model": {
3     "modelDimension": 4,
4     "hiddenDimension": 6,
5     "queryHeadCount": 2,
6     "keyValueHeadCount": 1,
7     "headDimension": 2,
8     "rotaryDimension": 2,
9     "rmsNormEpsilon": 0.00001,
10    "ropeBase": 100
11  },
12  "tensors": [
13    {
14      "name": "block.attention_norm.weight",
15      "dtype": "f32-le",
16      "shape": [4],
17      "offset": 0,
18      "byteCount": 16
19    }
20  ]
21 }
```

Only dtype string `f32-le` is supported. Shape dimensions are nonnegative and their product is checked against `Int.max`. Scalars use shape `[]` and one element; a dimension containing zero produces a zero-byte tensor. For Float32,

$$B_{tensor} = 4 \prod_i d_i.$$

The declared `byteCount` must equal this value exactly.

37.7 Decoder configuration and required names

Model fields construct the throwing `DecoderConfiguration` from Problem 035. That enforces positive model/hidden/head dimensions, $D=Hq*dh$, divisibility of query heads by KV heads, rotary bounds, finite positive epsilon, and `ropeBase>1`.

The decoder loader requires exactly these semantic names before shape validation:

```
1 block.attention_norm.weight
2 block.attention.query.weight
3 block.attention.key.weight
4 block.attention.value.weight
5 block.attention.output.weight
6 block.mlp_norm.weight
7 block.mlp.gate.weight
8 block.mlp.up.weight
9 block.mlp.down.weight
```

Additional tensors may be present. Required-name lookup is explicit; array position is not a semantic name. After lookup, the Problem 035 contract validates all nine [out,in] shapes and finite values.

37.8 CPU parser path

The canonical parser performs these steps before returning typed tensors:

1. Require at least 20 bytes and compare all eight magic bytes.
2. Decode version `UInt32` and header-length `UInt64` with shifts.
3. Check `UInt64`-to-`Int` conversion, addition, aligned header end, and file bounds.
4. Require every alignment padding byte to be zero.
5. Validate header UTF-8, then decode the JSON schema with `JSONDecoder`.
6. Construct and validate `DecoderConfiguration`.
7. Reject empty or duplicate names and unsupported dtype strings.
8. Check every dimension product, exact byte count, nonnegative aligned offset, and payload-relative end with reporting-overflow arithmetic.
9. Sort nonempty ranges by offset and reject adjacent overlap.
10. Decode each `Float32` from four explicit little-endian bytes and reject non-finite values.
11. Enforce caller-supplied required names and return named tensors.

No descriptor is used to index payload bytes until all descriptor ranges have passed bounds and overlap validation.

37.9 Independent correctness

The judge builds one valid nine-tensor decoder archive, parses it, invokes the Problem 035 loader, and compares the complete configuration and weights. It then independently constructs or mutates files for 12 corruptions:

- wrong magic and unsupported version;
- overflowing header length and invalid UTF-8;
- duplicate names and unsupported dtype;
- shape/byte-count mismatch and unaligned offset;
- overlapping ranges and payload escape;
- a missing required name; and
- an IEEE infinity payload.

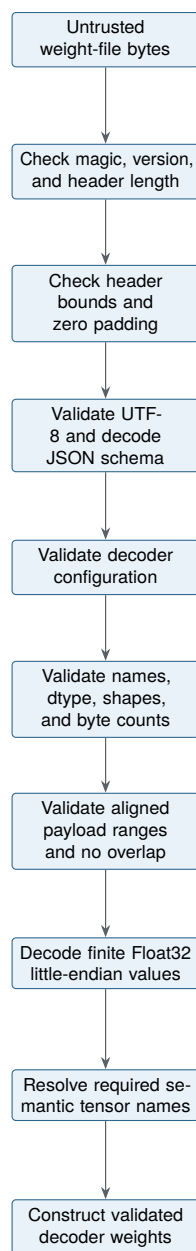


Figure 37.1: Weight file trust path.

Tests also round-trip 1, -2.5, and 0.125, exercise truncated input, and show that a structurally valid archive can still be rejected by the model loader for a wrong query-weight shape.

```
1 swift run leetllm check 036 --cpu
2 swift run leetllm check 036 --solution
```

37.10 Performance model: bytes, work, and allocation

Let F be file bytes, H header bytes, T tensor count, and N decoded Float32 elements. The parser reads the preamble in constant work, validates and decodes H bytes of JSON, sorts T ranges in $O(T \log T)$, and reads $4N$ payload bytes. Overall work is

$$O(H + T \log T + N),$$

and every payload byte is read once during Float32 conversion. The teaching API accepts `[UInt8]`, JSON decoding allocates schema values, and each tensor allocates a Swift `[Float]`; therefore it is not zero-copy. The fixture encoder similarly materializes header and payload arrays. These costs are explicit baselines for a later mapped-file or packed-weight design.

37.11 Metal mapping

Problem 036 is CPU-only. File I/O, UTF-8/JSON validation, name lookup, and model construction are host control work. A GPU must not be asked to interpret untrusted descriptors. Metal kernels consume already validated buffers and explicit shape/stride constants.

A production loader may upload or memory-map validated payload ranges, but that does not change this trust order. No CPU parse result is labeled as a Metal path.

37.12 Implementation checkpoints

1. Decode and test only magic, version, and header length.
2. Add aligned header-bound and zero-padding checks.
3. Distinguish invalid UTF-8 from schema-incompatible JSON.
4. Validate names, dtype, dimensions, and exact byte counts.
5. Validate all ranges and overlaps before decoding one value.
6. Decode known Float32 bit patterns explicitly little-endian.
7. Enforce required names.
8. Construct the Problem 035 block and reject model-shape mismatches.

37.13 Controlled experiments

37.13.1 One-field corruption matrix

Starting from a valid fixture, change one field at a time: version, H , dtype, dimension, offset, or byte count. Prediction: each file fails before any claimed range is decoded, and the error identifies the violated layer of the contract.

37.13.2 Descriptor-order permutation

Shuffle descriptor order without changing names or offsets. Prediction: parsed tensors and loaded weights are identical because names and ranges, not array position, define identity.

37.13.3 Header-size sweep

Add unrelated valid tensor descriptors while holding payload bytes fixed. Prediction: JSON/range-validation time grows with header and tensor count, while Float32 decode time remains tied to decoded payload elements.

37.13.4 Truncation sweep

Truncate at every byte around the preamble, header end, alignment padding, and final tensor. Prediction: every truncation fails deterministically; none traps or returns a partially trusted model.

37.14 Engine integration

Later mini-model fixtures use `P036WeightFixtureEncoder` to produce deterministic course assets. Runtime code calls the learner/parser implementation with required names, then `P036DecoderBlockLoader.load`. Problem 039 receives the resulting `LoadedDecoderBlock`; it does not duplicate configuration fields or hardcode weight orientation.

The encoder is a convenience, not evidence that the parser is safe. Corruption tests construct bytes the encoder would never emit.

37.15 Tradeoffs and limitations

- JSON is inspectable and larger/slower than a compact binary metadata table.
- Named tensors are robust to descriptor order and cost string storage/lookups.
- Copying to `[Float]` is simple and safe; memory mapping can reduce copies after validation.
- Rejecting non-finite weights catches fixture corruption but excludes deliberate non-finite research inputs.
- Version 1 supports only finite `f32-le`. It does not yet encode Q4 scales, tokenizer data, multiple layers, endianness variants, compression, or checksums.
- This deterministic mini-container is not a production checkpoint and must not be presented as compatible with an established model format.

37.16 Hints

- Treat every multiplication and addition involving file values as overflow-prone.
- Convert offsets to absolute indices only after validating them against payload size.
- Sort ranges for overlap checks; descriptor order is arbitrary.
- Zero-byte ranges do not overlap other zero-byte or nonempty ranges.
- Decode bits, not pointers.
- Validate semantic model shapes after generic container safety.

37.17 Completion checklist

- ☐ Fixed fields are decoded explicitly little-endian.

- ☐ Header length, alignment padding, UTF-8, and JSON schema are validated.
- ☐ Names, dimensions, dtype, offsets, byte counts, ranges, and overlap are checked.
- ☐ Required tensors and model configuration fail explicitly.
- ☐ Float32 decoding uses no unsafe or unaligned typed loads.
- ☐ All corruption judge cases pass.
- ☐ The Problem 035 loader constructs the exact reusable configuration and weights.
- ☐ The format is described as educational and Float32-only, not as a production standard.

37.18 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

37.18.1 Sources/LeetLLMExercises/P036WeightContainerExercise.swift

```
1 import LeetLLMCore
2
3 public enum P036WeightContainerExercise {
4     public static func parse(
5         bytes: [UInt8],
6         requiredTensorNames: [String]
7     ) throws -> ParsedWeightContainer {
8         _ = try P036WeightContainerContract.validatePreamble(bytes)
9         _ = requiredTensorNames
10        // TODO: decode and validate the JSON descriptors before reading any payload bytes.
11        throw WeightContainerError.invalidJSONHeader("TODO: implement the LeetWeight parser")
12    }
13 }
```

37.19 Worked solution

Solution strategy

Parse a binary weight container with a 20-byte preamble (magic, version, header length), length-delimited UTF-8 JSON header, alignment padding to 8-byte boundary, and aligned payload. Validate the magic bytes (ASCII 'LLMWGT01'), decode little-endian integers for version and header byte count, parse JSON to extract model configuration and tensor descriptors, validate every descriptor's dtype, shape, offset alignment, byte count, and payload bounds, detect overlapping ranges, read Float32 values as little-endian byte sequences converted via bitpattern, and require all named tensors before returning the parsed container.

Correctness invariant

Every tensor has a unique non-empty name. No two nonempty tensors share bytes in the payload (no overlapping ranges after sorted by start offset). Every Float32 tensor offset is divisible by 4. Byte count equals 4 times the product of shape dimensions. All float values are finite. The model configuration object must supply exactly the 9 required decoder parameters (modelDimension, hiddenDimension, queryHeadCount, keyValueHeadCount, headDimension, rotaryDimension, rmsNormEpsilon, ropeBase, and implicit model fingerprint computation).

Common failure mode

Trusting external metadata (length, offset) without validation can cause out-of-bounds reads, integer overflow on shape products, or silent tensor aliasing if overlaps are not detected. Reading Float32 bytes using native pointer casting makes alignment and host byte-order implicit; bitpattern conversion is required for reproducibility. Failing to validate that descriptor byte counts match shape sizes allows misaligned or truncated tensors to pass silently.

37.19.1 Implementation points

Preamble and header layout

Bytes 0-7: magic string 'LLMWGT01' in ASCII. Bytes 8-11: version as UInt32 little-endian (currently 1). Bytes 12-19: header byte count as UInt64 little-endian. Bytes 20 to 20+count: UTF-8 JSON header (no embedded nulls). Bytes 20+count to next multiple of 8: zero padding. Payload offsets are relative to payload start, not file start.

Float32 bitpattern reading without unsafe casting

Read 4 bytes at offset i : $\text{bits} = \text{bytes}[i] \mid (\text{bytes}[i+1] \ll 8) \mid (\text{bytes}[i+2] \ll 16) \mid (\text{bytes}[i+3] \ll 24)$. Convert UInt32 to Float via `Float(bitPattern: bits)`. This preserves exact IEEE 754 values (including subnormals and negative zero) and works identically across host byte orders after decoding little-endian bytes.

Overlap detection algorithm

Filter nonempty ranges ($\text{start} < \text{end}$), sort by (start ascending, end ascending), then iterate: for each $\text{range}[i]$, check that $\text{range}[i].\text{start} \geq \text{range}[i-1].\text{end}$. If sorted $\text{start} < \text{prior end}$, ranges overlap. This $O(n \log n)$ approach catches all pairs without $O(n^2)$ comparisons.

Shape-to-byte-count validation

Compute `elementCount` by multiplying shape dimensions with overflow check. `expectedByteCount = elementCount * 4` (for `Float32`). If `descriptor.byteCount != expectedByteCount`, reject the descriptor. A shape containing 0 is valid (zero elements, zero bytes).

37.19.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P036WeightContainerSolution.swift

- Line 4: `public enum P036WeightContainerSolution {`
- Line 5: `public static func parse(`
- Line 114: `private struct PayloadRange {`
- Line 120: `private static func checkedElementCount(shape: [Int], tensor: String) throws -> Int {`
- Line 132: `private static func checkedMultiply(_ lhs: Int, _ rhs: Int, tensor: String) throws -> Int {`
- Line 138: `private static func readUInt32LittleEndian(_ bytes: [UInt8], at offset: Int) -> UInt32 {`

37.19.3 Complete canonical listings**37.19.4 Sources/LeetLLMSolutions/P036WeightContainerSolution.swift**

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P036WeightContainerSolution {
5     public static func parse(
6         bytes: [UInt8],
7         requiredTensorNames: [String]
8     ) throws -> ParsedWeightContainer {
9         let preamble = try P036WeightContainerContract.validatePreamble(bytes)
10        let headerBytes = Array(
11            bytes[P036WeightContainerContract.preambleByteCount..

```

```

30     }
31     guard names.insert(descriptor.name).inserted else {
32         throw WeightContainerError.duplicateTensorName(descriptor.name)
33     }
34     guard descriptor.dtype == P036WeightContainerContract.float32DType else {
35         throw WeightContainerError.unsupportedDType(
36             tensor: descriptor.name, dtype: descriptor.dtype)
37     }
38     guard descriptor.offset >= 0 else {
39         throw WeightContainerError.negativeOffset(
40             tensor: descriptor.name, value: descriptor.offset)
41     }
42     guard descriptor.offset.isMultiple(
43         of: P036WeightContainerContract.float32Alignment)
44     else {
45         throw WeightContainerError.misalignedOffset(
46             tensor: descriptor.name,
47             offset: descriptor.offset,
48             alignment: P036WeightContainerContract.float32Alignment)
49     }
50     guard descriptor.byteCount >= 0 else {
51         throw WeightContainerError.negativeByteCount(
52             tensor: descriptor.name, value: descriptor.byteCount)
53     }
54     let elementCount = try checkedElementCount(
55         shape: descriptor.shape, tensor: descriptor.name)
56     let expectedBytes = try checkedMultiply(
57         elementCount, MemoryLayout<Float>.size, tensor: descriptor.name)
58     guard descriptor.byteCount == expectedBytes else {
59         throw WeightContainerError.byteCountMismatch(
60             tensor: descriptor.name,
61             expected: expectedBytes,
62             actual: descriptor.byteCount)
63     }
64     let (end, overflow) = descriptor.offset.addingReportingOverflow(descriptor.byteCount)
65     guard !overflow, end <= payloadSize else {
66         throw WeightContainerError.payloadOutOfBounds(
67             tensor: descriptor.name,
68             end: overflow ? Int.max : end,
69             payloadSize: payloadSize)
70     }
71     ranges.append(PayloadRange(name: descriptor.name, start: descriptor.offset, end: end))
72     metadata[descriptor.name] = descriptor
73 }
74
75 let nonemptyRanges = ranges.filter { $0.start < $0.end }.sorted {
76     $0.start == $1.start ? $0.name < $1.name : $0.start < $1.start
77 }
78 if nonemptyRanges.count > 1 {
79     for index in 1..

```

```

103     }
104     for name in requiredTensorNames where tensors[name] == nil {
105         throw WeightContainerError.missingTensor(name)
106     }
107     return ParsedWeightContainer(
108         configuration: configuration,
109         tensorMetadata: metadata,
110         tensors: tensors,
111         payloadByteCount: payloadSize)
112 }
113
114 private struct PayloadRange {
115     let name: String
116     let start: Int
117     let end: Int
118 }
119
120 private static func checkedElementCount(shape: [Int], tensor: String) throws -> Int {
121     var count = 1
122     for (axis, dimension) in shape.enumerated() {
123         guard dimension >= 0 else {
124             throw WeightContainerError.invalidDimension(
125                 tensor: tensor, axis: axis, value: dimension)
126         }
127         count = try checkedMultiply(count, dimension, tensor: tensor)
128     }
129     return count
130 }
131
132 private static func checkedMultiply(_ lhs: Int, _ rhs: Int, tensor: String) throws -> Int {
133     let (result, overflow) = lhs.multipliedReportingOverflow(by: rhs)
134     guard !overflow else { throw WeightContainerError.tensorSizeOverflow(tensor) }
135     return result
136 }
137
138 private static func readUInt32LittleEndian(_ bytes: [UInt8], at offset: Int) -> UInt32 {
139     UInt32(bytes[offset])
140     | UInt32(bytes[offset + 1]) << 8
141     | UInt32(bytes[offset + 2]) << 16
142     | UInt32(bytes[offset + 3]) << 24
143 }
144 }

```

Byte-Level BPE Tokenization and Detokenization

38.1 Why this exists

The model consumes token IDs, not words or Swift Character values. The checkpoint's vocabulary, merge ranks, and BOS/EOS policy define which IDs a prompt becomes. A mathematically correct transformer paired with a whitespace splitter is running a different input protocol and cannot match reference inference.

This lesson implements a real byte-level byte-pair encoder. It starts from the UTF-8 bytes of text, repeatedly applies ranked adjacent merges, and preserves a raw-byte decode API even when the resulting bytes are not valid UTF-8. The fixture is deterministic and structurally faithful, but deliberately tiny compared with a production model vocabulary.

38.2 Learning outcomes

You can:

- distinguish Unicode scalar/character boundaries from UTF-8 byte boundaries;
- define vocabulary entries as token IDs owning exact byte sequences;
- apply ranked adjacent merges with deterministic leftmost tie behavior;
- add BOS/EOS only when an explicit encoding policy requests them;
- detokenize losslessly to bytes and conditionally to valid UTF-8 text;
- reject invalid IDs and make missing-byte behavior explicit; and
- explain why tokenizer data is part of checkpoint compatibility.

38.3 Prerequisites

- Problem 002 for indexed contiguous storage and bounds errors.
- Problem 013 for the fact that token IDs index embedding rows.
- Problem 036 for checkpoint compatibility and explicit format conventions.
- Familiarity with UTF-8 as a variable-width byte encoding.

38.4 Vocabulary

- **Vocabulary token:** (`id`, `bytes`) mapping owned by the tokenizer.
- **Base byte token:** a vocabulary entry containing exactly one byte.
- **Merge pair:** adjacent token IDs (`left`, `right`) eligible to become one result ID.
- **Merge rank:** nonnegative priority; lower rank is applied first.

- **Leftmost tie:** if the same best-ranked pair appears more than once, merge its lowest sequence index first.
- **BOS/EOS:** beginning/end special IDs added by policy, not inferred from text.
- **Raw decode:** concatenate token bytes without requiring valid UTF-8.
- **Text decode:** raw decode followed by strict UTF-8 validation.

38.5 Math and algorithm from first principles

Given UTF-8 bytes $b[0..<B]$, map each byte to a one-byte token ID. At each iteration, inspect every adjacent token pair. Among pairs with a merge rule, choose the smallest ($\text{rank}, \text{leftIndex}$), replace the two IDs with the rule's result ID, and repeat until no pair has a rule.

If sequence t contains pair $t[i], t[i+1]$, a valid merge requires

$$\text{bytes}(\text{result}) = \text{bytes}(t_i) \parallel \text{bytes}(t_{i+1}),$$

where $\parallel$ means byte concatenation. The tokenizer initializer verifies this, so merging changes segmentation but never changes represented bytes.

38.6 Worked merge sequence

The course fixture uses base-byte IDs equal to byte values. It also defines:

ID	Bytes	Merge rank and pair
258	th	rank 0: (116, 104)
259	the	rank 1: (258, 101)
260	the	rank 2: (32, 259)
261	UTF-8 é	rank 3: (195, 169)
262	ll	rank 4: (108, 108)
263	ell	rank 5: (101, 262)
264	hell	rank 6: (104, 263)
265	hello	rank 7: (264, 111)

For text `the the`, initial IDs are

```
1 [116, 104, 101, 32, 116, 104, 101]
```

The left rank-0 `th` merges first, then the second rank-0 pair:

```
1 [258, 101, 32, 116, 104, 101]
2 [258, 101, 32, 258, 101]
```

Rank 1 forms both the tokens, and rank 2 joins the space to the second:

```
1 [259, 32, 258, 101]
2 [259, 32, 259]
3 [259, 260]
```

For `café`, UTF-8 starts as `[99, 97, 102, 195, 169]`; rank 3 produces `[99, 97, 102, 261]`. The algorithm never assumes `é` is one byte.

38.7 Vocabulary and API contract

ByteBPETokenizer validates:

- nonnegative unique token IDs;
- nonempty unique byte sequences;
- distinct BOS and EOS IDs present in the vocabulary;
- nonnegative unique merge ranks and unique merge pairs;
- existence of left, right, and result tokens;
- exact result-byte concatenation; and
- existence of a configured unknown-byte token, if that policy is selected.

The fixture contains all 256 one-byte tokens, BOS 256, EOS 257, and merge tokens 258...265. Its unknown policy is `.error`, but all input UTF-8 bytes are covered. A sparse vocabulary can choose `.error` or one explicit fallback token; fallback replacement is not lossless and is never silently enabled.

`encode` accepts a Swift String and BPEEncodingOptions. Empty input returns `[]`, `[BOS]`, `[EOS]`, or `[BOS,EOS]` according to those booleans. `decodeBytes` accepts a `skipSpecialTokens` policy. `decodeText` performs the same concatenation and throws if strict UTF-8 construction fails.

38.8 CPU reference path

1. Materialize `Array(text.utf8)`.
2. Map every byte through `initialTokenID(for:at:)`.
3. Scan adjacent pairs for the lowest merge rank; scan order provides the leftmost tie break.
4. Replace exactly that pair and restart the scan.
5. Add requested special IDs after all text-byte merges.
6. Decode by looking up each ID and concatenating its bytes, optionally skipping BOS/EOS.
7. For the text API only, require `String(bytes:encoding:.utf8)` to succeed.

The learner starter completes steps 1, 2, 5, 6, and 7. It is a compiling, lossless byte tokenizer but intentionally does not perform BPE merges.

38.9 Independent correctness

The judge uses exact expected token sequences rather than calling the canonical encoder. It checks the worked ranked-merge sequence and repeated-pair tie, Unicode byte merging, empty input with BOS/EOS, and a round trip containing an emoji. It separately checks that raw decode preserves `[0xff]`, text decode rejects that byte, invalid token IDs throw, and a sparse `.error` vocabulary rejects an uncovered byte.

Tests pass a byte-only implementation to the shared judge and require failure, proving whitespace or byte splitting alone is not accepted as BPE.

```
1 swift run leetllm check 037 --cpu
2 swift run leetllm check 037 --solution
```

38.10 Performance model: work, bytes, and allocation

Let B be UTF-8 input bytes and M the number of successful merges. The readable algorithm rescans up to the current token count and shifts an array on each replacement. Worst-case work is quadratic:

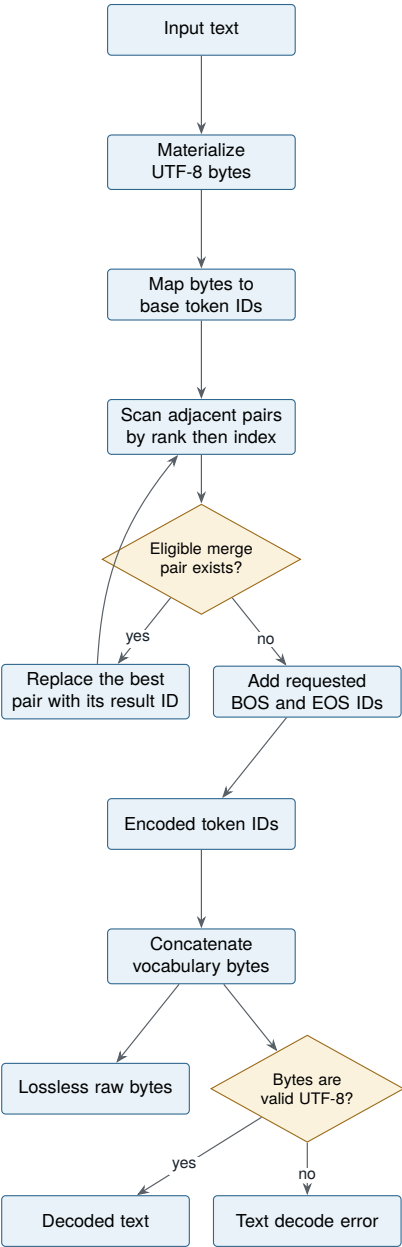


Figure 38.1: Byte bpe round trip.

$$O(B^2),$$

with $O(B)$ token storage. Decode reads the sum of selected token byte lengths and appends that many bytes, so it is linear in output bytes.

Production tokenizers commonly use linked structures, priority queues, or pretokenized regions to avoid repeated full scans. This lesson keeps the selection rule visible; optimize only after exact token parity is preserved.

38.11 Metal mapping

Problem 037 is CPU-only. Tokenization operates on short, irregular byte sequences with dictionary lookups, changing adjacency, and serial rank choices. It is host preprocessing before embedding lookup. Detokenization is likewise a small control-path operation around generated IDs.

A GPU implementation would add dispatch and synchronization complexity without teaching an inference kernel boundary. No CPU tokenizer output is presented as a Metal result.

38.12 Implementation checkpoints

1. Validate vocabulary IDs, bytes, and special tokens.
2. Validate merge references, rank uniqueness, and byte concatenation.
3. Encode ASCII as base byte IDs.
4. Apply one merge and verify represented bytes are unchanged.
5. Handle repeated best pairs with leftmost tie behavior.
6. Encode multi-byte Unicode and empty input policies.
7. Decode arbitrary valid IDs to bytes.
8. Add strict text decoding and explicit error paths.

38.13 Controlled experiments

38.13.1 Merge-density sweep

Encode long repeated `th`, `the`, and `hello` strings. Prediction: token count falls as more rules match, while the naive rescanning implementation takes more than linear time as input grows.

38.13.2 Rank intervention

Swap two dependent merge ranks in a private fixture. Prediction: final segmentation can change even though decoded bytes remain identical. This shows why rank order is checkpoint data, not an optimization hint.

38.13.3 Unicode boundary set

Test ASCII, composed `é`, decomposed `e` plus combining accent, CJK, and emoji. Prediction: visually similar strings can have different UTF-8 bytes and token sequences; each valid sequence round-trips its own bytes exactly.

38.13.4 Special-token policy matrix

Encode empty and nonempty input under all four BOS/EOS combinations. Prediction: only requested IDs appear, always outside the merged text sequence.

38.14 Engine integration

Problem 039 uses this tokenizer to turn prompt text into IDs before embedding lookup. Problem 040 feeds each sampled token ID back into the model and uses raw decode to accumulate output bytes; a UI may emit text only when the accumulated prefix is valid UTF-8. Vocabulary and merge data must match the loaded model.

The course fixture gives later tests a deterministic prompt protocol. It is not a substitute for a real checkpoint's tokenizer files.

38.15 Tradeoffs and limitations

- Byte-level bases make arbitrary UTF-8 representable but start with more tokens than character-aware schemes before merges.
- Strict UTF-8 text decode reports invalid output; replacement decoding is more permissive and loses evidence.
- Global merging is easy to inspect; production tokenizers may apply regex pretokenization before BPE, changing valid merge boundaries.
- The fixture has 266 entries and eight merges. It has no normalization, pretokenization regex, added-token matching, or production vocabulary.
- Tokenizer compatibility includes byte sequences, IDs, ranks, and BOS/EOS policy together. Matching only token strings is insufficient.

38.16 Hints

- Convert the input to UTF-8 once; do not iterate Swift characters.
- Compare candidate merges by rank, then by adjacent index.
- Apply one merge per iteration so new neighboring pairs can become eligible.
- Add special IDs after merging text bytes.
- Keep byte and text decode as separate APIs.
- Test exact IDs, not only decoded round trips; many segmentations decode to the same bytes.

38.17 Completion checklist

- ☐ Vocabulary and merge invariants fail explicitly.
- ☐ Encoding begins from UTF-8 bytes, not words or characters.
- ☐ Lowest rank and leftmost tie behavior match exact expected IDs.
- ☐ Empty input and BOS/EOS policies are explicit.
- ☐ Raw-byte decode preserves invalid UTF-8 bytes.
- ☐ Text decode rejects invalid UTF-8 and invalid IDs.
- ☐ Unicode and round-trip tests pass.
- ☐ The fixture is described as educational, not checkpoint-compatible by default.

38.18 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

38.18.1 Sources/LeetLLMExercises/P037ByteBPPExercise.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P037ByteBPPExercise {
5     public static func encode(
6         tokenizer: ByteBPETokenizer,
7         text: String,
8         options: BPEEncodingOptions
9     ) throws -> [Int] {
10         var tokenIDs = try Array(text.utf8).enumerated().map {
11             try tokenizer.initialTokenID(for: $0.element, at: $0.offset)
12         }
13         // TODO: repeatedly apply the lowest-rank adjacent merge, breaking ties to the left.
14         if options.addBeginningOfSequence {
15             tokenIDs.insert(tokenizer.beginningOfSequenceTokenID, at: 0)
16         }
17         if options.addEndOfSequence {
18             tokenIDs.append(tokenizer.endOfSequenceTokenID)
19         }
20         return tokenIDs
21     }
22
23     public static func decodeBytes(
24         tokenizer: ByteBPETokenizer,
25         tokenIDs: [Int],
26         skipSpecialTokens: Bool
27     ) throws -> [UInt8] {
28         var bytes: [UInt8] = []
29         for tokenID in tokenIDs {
30             if skipSpecialTokens,
31                 tokenID == tokenizer.beginningOfSequenceTokenID
32                 || tokenID == tokenizer.endOfSequenceTokenID
33             {
34                 continue
35             }
36             bytes.append(contentsOf: try tokenizer.tokenBytes(for: tokenID))
37         }
38         return bytes
39     }
40
41     public static func decodeText(
42         tokenizer: ByteBPETokenizer,
43         tokenIDs: [Int],
44         skipSpecialTokens: Bool
45     ) throws -> String {
46         let bytes = try decodeBytes(
47             tokenizer: tokenizer,
48             tokenIDs: tokenIDs,
49             skipSpecialTokens: skipSpecialTokens)
50         guard let text = String(bytes: bytes, encoding: .utf8) else {
51             throw ByteBPError.invalidUTF8
52         }
53         return text
54     }
55 }

```

38.19 Worked solution

Solution strategy

Implement byte-level byte-pair encoding by starting with UTF-8 bytes of text mapped to single-byte token IDs (0-255), then iteratively finding and applying the best-ranked merge pair. At each step, inspect all adjacent token pairs, select the pair with the lowest merge rank (and lowest sequence index for ties), replace the two IDs with the rule's result ID, and repeat until no pair has a merge rule. After merging completes, optionally prepend BOS or append EOS token IDs based on encoding policy flags. Decode by reversing the process: concatenate token bytes losslessly, or validate UTF-8 for text decode.

Correctness invariant

Every merge rule has result bytes equal to the concatenation of left-token bytes and right-token bytes. The encoded sequence length never exceeds the prompt text byte length (merges only reduce length). Base vocabulary IDs 0-255 map to single bytes. BOS/EOS tokens are added only when explicitly requested via encoding options, never inferred from text content. Detokenization is lossless: encoding then decoding always recovers the original bytes.

Common failure mode

Applying merges out of rank order or using highest-ranked pair instead of lowest-ranked produces a different tokenization and breaks checkpoint compatibility. Merging a non-minimum-rank pair even once causes subsequent merges to operate on a different sequence. Adding BOS/EOS unconditionally instead of checking encoding options causes token count mismatches with reference tokenizers.

38.19.1 Implementation points

Base byte vocabulary and merge result IDs

Token IDs 0-255 own exactly one byte each: `token_bytes[0] = [0x00]`, `token_bytes[1] = [0x01]`, ..., `token_bytes[255] = [0xFF]`. Merge rules create higher IDs (starting above 256) whose bytes equal the concatenation of their two source tokens. A merge pair (`left_id=116`, `right_id=104`) with `result=258` means `bytes[258] = bytes[116] || bytes[104] = [0x74] || [0x68] = [0x74, 0x68]`.

Leftmost tie-breaking in merge selection

If the same best-ranked pair appears at multiple sequence indices, always merge the occurrence at the lowest index first. For example, if (101, 102) has rank 5 and appears at indices [3, 7], merge index 3, then re-scan for the next best pair. This ensures deterministic output independent of pair duplication.

Merge invariant preservation

Before merging pair (left, right) to result, verify that `bytes[result] == bytes[left] || bytes[right]` where `||` is byte concatenation. The tokenizer initializer checks this invariant. After any merge operation, the byte sequence represented by the token list never changes; only the segmentation (number of tokens) changes.

BOS/EOS policy vs. content inference

The encoding API accepts `BPEEncodingOptions` with `addBeginningOfSequence` and `addEndOfSequence` flags. Only insert `tokenizer.beginningOfSequenceTokenID` at index 0 if `addBeginningOfSequence` is true, and append `tokenizer.endOfSequenceTokenID` if `addEndOfSequence` is true. Never infer these tokens from text; they are explicit policy, not automatic.

38.19.2 Code map**How to read the listing**

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P037ByteBPESolution.swift

- Line 4: `public enum P037ByteBPESolution {`
- Line 5: `public static func encode(`
- Line 36: `public static func decodeBytes(`
- Line 54: `public static func decodeText(`

38.19.3 Complete canonical listings**38.19.4 Sources/LeetLLMSolutions/P037ByteBPESolution.swift**

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P037ByteBPESolution {
5     public static func encode(
6         tokenizer: ByteBPETokenizer,
7         text: String,
8         options: BPEEncodingOptions
9     ) throws -> [Int] {
10         var tokenIDs = try Array(text.utf8).enumerated().map {
11             try tokenizer.initialTokenID(for: $0.element, at: $0.offset)
12         }
13         while tokenIDs.count >= 2 {
14             var bestIndex: Int?
15             var bestRule: BPEMergeRule?
16             for index in 0.. $(tokenIDs.count - 1)$  {
17                 let pair = BPEPair(left: tokenIDs[index], right: tokenIDs[index + 1])
18                 guard let rule = tokenizer.mergeRules[pair] else { continue }
19                 if bestRule == nil || rule.rank < bestRule!.rank {
20                     bestIndex = index
21                     bestRule = rule
22                 }
23             }
24             guard let index = bestIndex, let rule = bestRule else { break }
25             tokenIDs.replaceSubrange(index.. $(index + 1)$ , with: [rule.result])
26         }
27         if options.addBeginningOfSequence {
28             tokenIDs.insert(tokenizer.beginningOfSequenceTokenID, at: 0)
29         }
30         if options.addEndOfSequence {
31             tokenIDs.append(tokenizer.endOfSequenceTokenID)
32         }
33         return tokenIDs
34     }
35
36     public static func decodeBytes(
37         tokenizer: ByteBPETokenizer,
38         tokenIDs: [Int],
39         skipSpecialTokens: Bool

```

```
40 ) throws -> [UInt8] {
41   var bytes: [UInt8] = []
42   for tokenID in tokenIDs {
43     if skipSpecialTokens,
44       tokenID == tokenizer.beginningOfSequenceTokenID
45       || tokenID == tokenizer.endOfSequenceTokenID
46     {
47       continue
48     }
49     bytes.append(contentsOf: try tokenizer.tokenBytes(for: tokenID))
50   }
51   return bytes
52 }
53
54 public static func decodeText(
55   tokenizer: ByteBPETokenizer,
56   tokenIDs: [Int],
57   skipSpecialTokens: Bool
58 ) throws -> String {
59   let bytes = try decodeBytes(
60     tokenizer: tokenizer,
61     tokenIDs: tokenIDs,
62     skipSpecialTokens: skipSpecialTokens)
63   guard let text = String(bytes: bytes, encoding: .utf8) else {
64     throw ByteBPEError.invalidUTF8
65   }
66   return text
67 }
68 }
```

Logits and Sampling

39.1 Why this exists

The decoder produces one finite logit per vocabulary token. Turning that vector into the next token is a policy boundary: greedy choice, temperature, top-k, top-p, and randomness change generation behavior without changing model weights. If filtering order, tie behavior, or random state is implicit, two otherwise identical engines can diverge on the first generated token.

This lesson implements deterministic greedy and seeded stochastic sampling with a complete trace. Problem 040 can reuse the API directly in its decode loop.

39.2 Learning outcomes

You can:

- compute stable softmax over extreme finite logits;
- explain how positive temperature changes relative probabilities;
- apply top-k and nucleus top-p with explicit ordering and boundary rules;
- renormalize the retained distribution before drawing;
- implement deterministic logit ties and seeded random draws;
- distinguish greedy mode from invalid temperature-zero division; and
- inspect a trace containing retained candidates, probabilities, and draw.

39.3 Prerequisites

- Problem 009 for max-subtracted stable softmax.
- Problem 013 for unembedding from hidden state to vocabulary logits.
- Problem 034 for near-tie argmax behavior and reproducible diagnostics.
- Problem 037 for interpreting the selected integer as a tokenizer ID.

39.4 Vocabulary

- **Logit**: unnormalized finite score for one token ID.
- **Greedy**: choose the maximum logit without a random draw.
- **Temperature**: positive divisor applied to logits before softmax.
- **Top-k**: retain the k highest-ranked candidates.
- **Top-p / nucleus**: retain the shortest ranked prefix whose cumulative probability reaches or exceeds p.
- **Boundary-crossing token**: candidate that first makes cumulative probability meet/exceed p; it remains included.
- **Renormalization**: divide retained probabilities by their retained sum.

- **Sampling trace:** selected ID, retained candidates in deterministic order, final probabilities, and optional random draw.

39.5 Math and probability derivation

For finite logits $z[0 \dots V]$ and positive temperature T , stable probabilities are

$$m = \max_i z_i / T, \quad p_i = \frac{\exp(z_i / T - m)}{\sum_j \exp(z_j / T - m)}.$$

Subtracting m does not change ratios because the same factor cancels from numerator and denominator. Smaller positive T sharpens differences; larger T flattens them. $T=0$ is undefined. This API uses `.greedy` for the temperature-zero policy and rejects stochastic configurations with $T \leq 0$.

39.6 Worked filtering example

For logits $[2, 1, 0]$ at $T=1$, max subtraction gives exponentials $[1, e^{-1}, e^{-2}]$, approximately $[1, 0.3679, 0.1353]$. Full probabilities are approximately $[0.6652, 0.2447, 0.0900]$.

With $\text{topK}=2$, discard token 2 and normalize over tokens 0, 1:

```
1 token 0: 0.7311
2 token 1: 0.2689
```

Then $\text{topP}=0.8$ keeps both: $0.7311 < 0.8$, and adding token 1 crosses the boundary. For $\text{topP}=0.7$, only token 0 remains and is renormalized to 1.0.

The implemented composition order is fixed:

```
1 rank by (logit descending, token ID ascending)
2 -> top-k prefix
3 -> stable softmax over that prefix
4 -> shortest top-p prefix including crossing token
5 -> renormalize
6 -> seeded draw
```

Changing this order defines a different policy.

39.7 Shape, dtype, and error contract

Input is a nonempty Swift `[Float]` representing contiguous Float32 logits with shape $[V]$. Every value must be finite. Token ID is the zero-based array index. Equal logits are ordered by smaller token ID.

Stochastic configuration requires:

- finite temperature > 0 ;
- optional topK in $1 \dots V$; and
- optional finite topP in $(0, 1]$.

Probability reduction uses Double in the canonical sampler and stores trace probabilities as Float. Greedy returns one candidate with probability 1 and `randomDraw=nil`; it does not advance the PRNG. Stochastic mode advances exactly once per call, even when filtering leaves one candidate.

39.8 Deterministic PRNG contract

SeededGenerator uses SplitMix64 state transitions. Each stochastic call gets a UInt64, discards the low 11 bits, and divides the remaining 53 bits by 2^{53} :

$$u = \frac{\text{next()} \gg 11}{2^{53}}, \quad 0 \leq u < 1.$$

Candidates are traversed in ranked order while accumulating final probabilities; the first candidate with $u < \text{cumulative}$ is selected. The last retained token is a rounding fallback.

SeededLogitsSampler owns this generator. Two samplers initialized with the same seed and called with the same sequence of stochastic requests produce identical traces and next states. A caller that changes call count changes future draws.

39.9 CPU reference path

1. Validate logits and strategy before reading a maximum.
2. Build (tokenID, logit) candidates and sort by descending logit, then ID.
3. For greedy, return the first candidate without touching the generator.
4. For stochastic mode, truncate to top-k if configured.
5. Divide by temperature in Double and compute max-subtracted softmax.
6. If top-p is configured, keep at least one candidate and include the crossing token.
7. Renormalize retained probabilities.
8. Generate one u and select from cumulative mass.
9. Return the selected ID, ranked retained candidates, final probabilities, and draw.

The learner starter validates all inputs and implements correct deterministic greedy selection. Its stochastic path intentionally returns the greedy result, so probability, trace, draw, and PRNG-state cases remain red.

39.10 Independent correctness

The judge owns a separate Double reference implementation. It compares selected token, ordered candidate IDs/logits, probabilities within $2e-6$, exact random draw, and final generator state. Cases cover:

- equal and near-equal logits with lowest-ID tie order;
- logits 10000, 0, and -10000 without overflow;
- composed top-k then top-p;
- a top-p boundary that must retain the second tied token;
- equal-seed reproducibility;
- empty and non-finite logits;
- temperature zero, $k > v$, and $p > 1$.

A test passes an always-greedy implementation to the same judge and requires it to fail.

```
1 swift run leetllm check 038 --cpu
2 swift run leetllm check 038 --solution
```

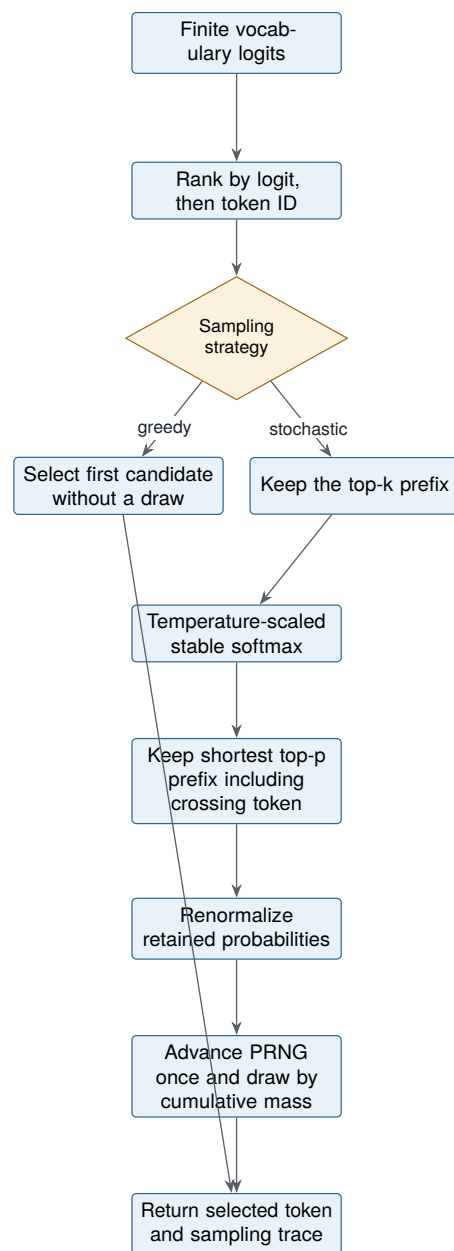


Figure 39.1: Logits sampling policy.

39.11 Performance model: work, bytes, and allocation

The readable implementation sorts all V candidates, so work is

$$O(V \log V)$$

and transient storage is $O(V)$ candidates and probabilities. It reads $4V$ logit bytes before allocation overhead. Softmax/filter passes are linear in the retained top- k prefix.

Greedy selection alone can be an $O(V)$ argmax with constant auxiliary storage. Large-vocabulary production top- k can use selection algorithms or hierarchical GPU reductions instead of a full CPU sort. This lesson keeps complete ordering and trace evidence visible.

39.12 Metal mapping

Problem 038 is CPU-only. The policy and seeded draw are host-side in this course. For a real large vocabulary, transferring all logits to the CPU can dominate; a Metal path could reduce argmax or top- k on device and transfer only candidates. That future kernel would need deterministic tie rules and parity for retained probabilities and must define where PRNG state lives.

No such kernel is implemented here, and the CPU sampler is not presented as a GPU check. Optimization belongs after Problem 040 exposes actual decode timing.

39.13 Implementation checkpoints

1. Validate empty/non-finite logits and all stochastic fields.
2. Implement greedy with smallest-ID tie order and no PRNG advance.
3. Match stable full-softmax probabilities for extreme logits.
4. Add top- k truncation and verify $k=1$ and $k=V$.
5. Add top- p prefix selection with the crossing token included.
6. Renormalize and verify retained probabilities sum to one.
7. Draw once from SeededGenerator and return the complete trace.
8. Verify repeated sampler calls and equal-seed reproducibility.

39.14 Controlled experiments

39.14.1 Temperature sweep

For fixed logits $[2, 1, 0]$, compare $T=0.25, 1, 4$. Prediction: candidate order is unchanged; entropy rises with temperature and the largest-token probability falls.

39.14.2 Top- k boundary sweep

Sweep $k=1 \dots V$ at fixed seed with $\text{topP}=\text{nil}$. Prediction: $k=1$ always selects the greedy token but still records a stochastic draw; larger k can change the selected token and retained distribution.

39.14.3 Top- p boundary sweep

Use equal logits and vary p just below, at, and above cumulative fractions. Prediction: the retained count changes only when the shortest prefix no longer reaches p ; the crossing candidate is never dropped.

39.14.4 Seed and call-order experiment

Run two samplers with equal seeds, then insert one extra stochastic call into only one stream. Prediction: traces agree before the insertion and can diverge after it because PRNG state ownership includes call order.

39.14.5 Near-tie intervention

Perturb two logits around equality. Prediction: exact equality chooses the smaller ID; a representable positive difference changes rank. Seeded stochastic selection may still choose either retained token.

39.15 Engine integration

Problem 040 obtains vocabulary logits from the final normalized hidden state and unembedding weights, then calls `SeededLogitsSampler.sample`. The selected token is appended to the token sequence, fed through embedding and the decoder on the next step, and passed to Problem 037 detokenization. The trace belongs in decode diagnostics so policy divergence is distinguishable from model-logit divergence.

39.16 Tradeoffs and limitations

- Full sorting is simple and deterministic; partial selection is faster for large vocabularies.
- CPU sampling is inspectable; device-side filtering can avoid a large logits transfer.
- Float trace probabilities are compact; Double computation reduces avoidable normalization error.
- One fixed PRNG makes tests reproducible but is not a cryptographic generator.
- Top-k-before-top-p is one explicit policy. Frameworks using another order will not match token-for-token even with equal logits and seeds.
- The lesson does not implement penalties, logit bias, repetition controls, beam search, typical sampling, or batched request-specific generators.

39.17 Hints

- Positive temperature preserves ranking, so one deterministic sort is sufficient.
- Always subtract the maximum scaled logit before exp.
- Top-p retains the token that crosses the threshold.
- Renormalize after every filtering decision before drawing.
- Compare $\text{draw} < \text{cumulative}$; keep the last candidate as a rounding fallback.
- Never emulate temperature zero by dividing or clamping; call greedy mode.

39.18 Completion checklist

- ☐ Empty/non-finite logits and every invalid stochastic field throw.
- ☐ Greedy ties choose the lowest token ID and do not advance PRNG state.
- ☐ Stable probabilities remain finite for extreme logits.
- ☐ Top-k then top-p ordering and crossing-token behavior match the judge.
- ☐ Retained probabilities sum to one within tolerance.
- ☐ Equal seeds and call sequences produce equal traces and states.
- ☐ Trace contains selected ID, ordered candidates, final probabilities, and draw.
- ☐ Temperature zero is represented only by explicit greedy mode.

39.19 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

39.19.1 Sources/LeetLLMExercises/P038LogitsSamplingExercise.swift

```
1 import LeetLLMCore
2
3 public enum P038LogitsSamplingExercise {
4     public static func sample(
5         logits: [Float],
6         strategy: SamplingStrategy,
7         generator: inout SeededGenerator
8     ) throws -> SamplingTrace {
9         try P038LogitsSamplingContract.validate(logits: logits, strategy: strategy)
10        _ = generator
11        let winner = logits.indices.min {
12            logits[$0] == logits[$1] ? $0 < $1 : logits[$0] > logits[$1]
13        }!
14        // TODO: implement temperature scaling, top-k, top-p, renormalization, and seeded draw.
15        return SamplingTrace(
16            selectedToken: winner,
17            retainedCandidates: [SamplingCandidate(
18                tokenID: winner, logit: logits[winner], probability: 1)],
19            randomDraw: nil)
20    }
21 }
```

39.20 Worked solution

Solution strategy

Sort logits in descending order (breaking ties by token ID ascending), optionally retain only the top-k highest logits, compute stable softmax over retained logits by subtracting the maximum to avoid overflow, optionally apply nucleus (top-p) filtering by accumulating probabilities until the cumulative sum reaches or exceeds p (including the boundary-crossing token), renormalize the retained probabilities by their sum, then either return the argmax as the greedy selection or draw a sample from the renormalized distribution using a seeded PRNG. Compose these operations in fixed order: rank, top-k, softmax, top-p, draw.

Correctness invariant

Stable softmax max-subtraction (subtracting $\max(\text{logits}/T)$ before computing exponentials) does not change probability ratios. Renormalization of retained probabilities to sum to 1.0 preserves the ranking of candidates. Temperature > 0 is required for stochastic sampling; temperature $= 0$ is undefined and must use greedy enum instead. Top-p filtering includes the boundary-crossing token (the one that first makes cumulative probability $\geq p$).

Common failure mode

Applying top-p before softmax or before top-k, or reversing either filter order, defines a mathematically different sampling policy. Temperature $= 0$ causes division by zero in softmax and must be rejected. Deterministic logit tie-breaking (lower token ID wins) is required; breaking ties inconsistently across runs produces different results. Using non-seeded or incorrectly-advanced PRNG state causes unreproducible multi-token generations.

39.20.1 Implementation points

Stable softmax with temperature scaling

For finite logits and positive temperature T : $\text{scaled} = \text{logits}[i] / T$, $\text{max_subtract} = \max(\text{scaled})$, $\text{exponentials}[i] = \exp(\text{scaled}[i] - \text{max_subtract})$, $\text{probabilities}[i] = \text{exponentials}[i] / \sum(\text{exponentials})$. The max subtraction prevents overflow and does not change the probability ratios (the max factors out and cancels). Smaller T sharpens the distribution; larger T flattens it.

Composition order: rank $\rightarrow$ top-k $\rightarrow$ softmax $\rightarrow$ top-p $\rightarrow$ draw

Rank all logits descending (ties broken by token ID ascending). Retain only the top-k entries (if specified). Compute softmax over the retained logits. Apply top-p filtering: accumulate probabilities until reaching p, keeping the boundary-crossing token. Renormalize retained probabilities. Sample or return greedy. Changing this order produces a different policy.

Top-p (nucleus) sampling with boundary inclusion

Sort candidates by descending probability. Initialize cumulative = 0. For each candidate in order: cumulative += probability. If cumulative $\geq p$, include this candidate and stop. The candidate that

crosses the boundary (cumulative becomes $\geq p$) must be included, even if it makes cumulative $> p$. Renormalize the retained candidates.

Seeded PRNG state advancement

Draw a uniform random number from $[0, 1)$ using seeded PRNG and advance the generator state. Accumulate probabilities until cumulative $>$ draw; the first candidate to exceed draw is selected. The generator state must advance deterministically with each call, enabling reproducible multi-token sequences from a fixed seed.

39.20.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P038LogitsSamplingSolution.swift

- Line 4: public enum P038LogitsSamplingSolution {
- Line 5: public static func sample(
- Line 69: private struct RankedLogit {

39.20.3 Complete canonical listings

39.20.4 Sources/LeetLLMSolutions/P038LogitsSamplingSolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P038LogitsSamplingSolution {
5     public static func sample(
6         logits: [Float],
7         strategy: SamplingStrategy,
8         generator: inout SeededGenerator
9     ) throws -> SamplingTrace {
10         try P038LogitsSamplingContract.validate(logits: logits, strategy: strategy)
11         var ranked = logits.indices.map { RankedLogit(tokenID: $0, logit: logits[$0]) }
12         ranked.sort {
13             $0.logit == $1.logit ? $0.tokenID < $1.tokenID : $0.logit > $1.logit
14         }
15         guard case .stochastic(let configuration) = strategy else {
16             let winner = ranked[0]
17             return SamplingTrace(
18                 selectedToken: winner.tokenID,
19                 retainedCandidates: [SamplingCandidate(
20                     tokenID: winner.tokenID, logit: winner.logit, probability: 1)],
21                 randomDraw: nil)
22         }
23
24         if let topK = configuration.topK {
25             ranked = Array(ranked.prefix(topK))
26         }
27         let scaledLogits = ranked.map {
28             Double($0.logit) / Double(configuration.temperature)
29         }
30         let maximum = scaledLogits.max()!
31         let exponentials = scaledLogits.map { exp($0 - maximum) }
32         let denominator = exponentials.reduce(0, +)
33         var probabilities = exponentials.map { $0 / denominator }
34
35         if let topP = configuration.topP {

```

```

36     var cumulative = 0.0
37     var retainedCount = 0
38     repeat {
39         cumulative += probabilities[retainedCount]
40         retainedCount += 1
41     } while retainedCount < probabilities.count && cumulative < Double(topP)
42     ranked = Array(ranked.prefix(retainedCount))
43     probabilities = Array(probabilities.prefix(retainedCount))
44 }
45
46 let retainedSum = probabilities.reduce(0, +)
47 probabilities = probabilities.map { $0 / retainedSum }
48 let draw = generator.nextUnitInterval()
49 var cumulative = 0.0
50 var selectedToken = ranked.last!.tokenID
51 for index in ranked.indices {
52     cumulative += probabilities[index]
53     if draw < cumulative {
54         selectedToken = ranked[index].tokenID
55         break
56     }
57 }
58 return SamplingTrace(
59     selectedToken: selectedToken,
60     retainedCandidates: ranked.indices.map {
61         SamplingCandidate(
62             tokenID: ranked[$0].tokenID,
63             logit: ranked[$0].logit,
64             probability: Float(probabilities[$0]))
65     },
66     randomDraw: draw)
67 }
68
69 private struct RankedLogit {
70     let tokenID: Int
71     let logit: Float
72 }
73 }

```

Prompt Prefill

40.1 Why this exists

A decoder cannot begin generation from prompt token IDs alone. It must gather their embeddings, run every decoder block in checkpoint order, retain each layer's rotated keys and values, normalize the final residual, and project the last prompt position to vocabulary logits. A path that produces plausible logits but omits cache writes has only delayed the same work until decode.

This lesson introduces one complete **educational mini-model** and its multi-token execution path. It is deterministic test data, not a pretrained or production model. Problems 040-042 reuse the same model, positions, cache, and capture names rather than inventing parallel conventions.

40.2 Learning outcomes

You can:

- validate a complete decoder model rather than one isolated block;
- gather $[S,D]$ embeddings from token IDs and a $[V,D]$ table;
- execute multiple Problem 035 blocks in order;
- append each layer's rotated K and unrotated V at absolute positions;
- select the final prompt row before unembedding to $[V]$ logits;
- explain why prefill projections are GEMM-shaped; and
- account for projection FLOPs, causal attention work, weight reads, and cache writes.

40.3 Prerequisites

- Problem 013 for embedding and tied-output semantics.
- Problems 018, 022, and 023 for integer-division GQA and contiguous KV caches.
- Problem 035 for the complete pre-norm decoder block and its captures.
- Problem 036 for checkpoint-facing $[out, in]$ weights and validated tensors.
- Problem 038 for the next-token policy used after these logits.

40.4 Vocabulary

- **Prefill**: process all prompt tokens before serial one-token generation.
- **Model-level contract**: vocabulary, embedding, layer stack, final norm, and output policy.
- **Tied output**: reuse the $[V,D]$ embedding table as unembedding weights.
- **Layer cache**: K/V records belonging to one decoder layer only.
- **Absolute position**: $positionOffset + tokenIndex$, retained in cache metadata.
- **Work model**: timing-independent FLOP, byte, and projected-token counts.

40.5 Math, model, and equations

The shared `MiniDecoderModel` contains:

```
1 vocabularySize V
2 DecoderConfiguration(D, F, Hq, Hkv, dh, rotaryDimension, epsilon, ropeBase)
3 tokenEmbedding [V,D]
4 blocks[L]
5 finalNormGamma [D]
6 outputProjection = tiedEmbedding | independent [V,D]
```

For prompt IDs $t_0, \dots, t_{S-1}$, gather

$$X_{i,:}^{(0)} = E_{t_i,:}$$

Run the Problem 035 block in order:

$$X^{(\ell+1)} = \text{DecoderBlock}_\ell(X^{(\ell)}, p_0).$$

During each layer, append

$$\text{cacheK}[\ell, p_0 + i] = \widetilde{K}_i^{(\ell)}, \quad \text{cacheV}[\ell, p_0 + i] = V_i^{(\ell)}.$$

After the final layer,

$$Y = \text{RMSNorm}(X^{(L)}, \gamma_f), \quad h = Y_{S-1,:}, \quad z = W_{out}h.$$

Only $h=Y[S-1]$ is unembedded. Selecting row zero or averaging rows is a different model output.

40.6 Worked prompt trace

The fixture uses $V=7, D=4$, two layers, $Hq=2, Hkv=1, dh=2$, and tied output weights. For prompt IDs $[1, 4, 2]$ at `positionOffset=5`:

1. Gather embedding rows 1, 4, and 2 into $[3, 4]$.
2. Layer 0 normalizes all three rows, projects Q/K/V, and rotates at positions 5, 6, 7.
3. Layer 0 appends three $[1, 2]$ K records and three $[1, 2]$ V records.
4. Layer 1 consumes layer 0's residual output and repeats with its own weights and cache region.
5. Final RMSNorm returns $[3, 4]$; row 2 becomes hidden $[4]$.
6. Tied unembedding computes seven dot products and returns logits $[7]$.

Both cache layers must report positions $[5, 6, 7]$. Layer 1 must not consume the original embeddings; its residual input is layer 0's output.

40.7 Shape, state, and error contract

Batch size is one and omitted. All tensors are contiguous row-major Float32. Projection weights remain `[out,in]`, RMSNorm uses direct gamma, RoPE rotates adjacent pairs, and GQA maps `kvHead = queryHead / (Hq/Hkv)`.

Value	Shape
token IDs	[S] Swift integers, $S \geq 1$
embedding table	[V,D]
residual per layer	[S,D]
Q	[S,Hq,dh]
K, V	[S,Hkv,dh]
final normalized residual	[S,D]
final hidden	[D]
output weights	[V,D]
logits	[V]

Model construction validates positive v , at least one layer, every embedding, block, final-norm, and output shape, and every Float value. Tied output is an explicit enum case. Independent output weights must be [V,D].

The cache must match L, Hkv, and dh, have enough capacity, and be empty at prefill entry. Token IDs must lie in $0..<v$; positions must be nonnegative and must not overflow. Every layer writes exactly S consecutive records.

Problem 036 intentionally remains a minimal single-block archive. It validates and loads one DecoderBlockWeights; this lesson constructs the complete multi-layer educational fixture in code. It does not silently claim that the P036 file format already represents embeddings, a layer array, or final output weights.

40.8 CPU reference algorithm

1. Validate the model, prompt, position range, and empty compatible cache.
2. Copy each embedding row into a contiguous [S,D] residual.
3. For $\text{layer}=0..<L$, call the complete Problem 035 block with `positionOffset`.
4. Slice each token's rotatedKeys and values as [Hkv,dh] and append them.
5. Pass the returned residual to the next layer.
6. Apply direct-gamma final RMSNorm to all rows.
7. Copy the last row to [D] and multiply by [V,D] output weights.
8. Return captures, cache counts/positions, final tensors, and the work model.

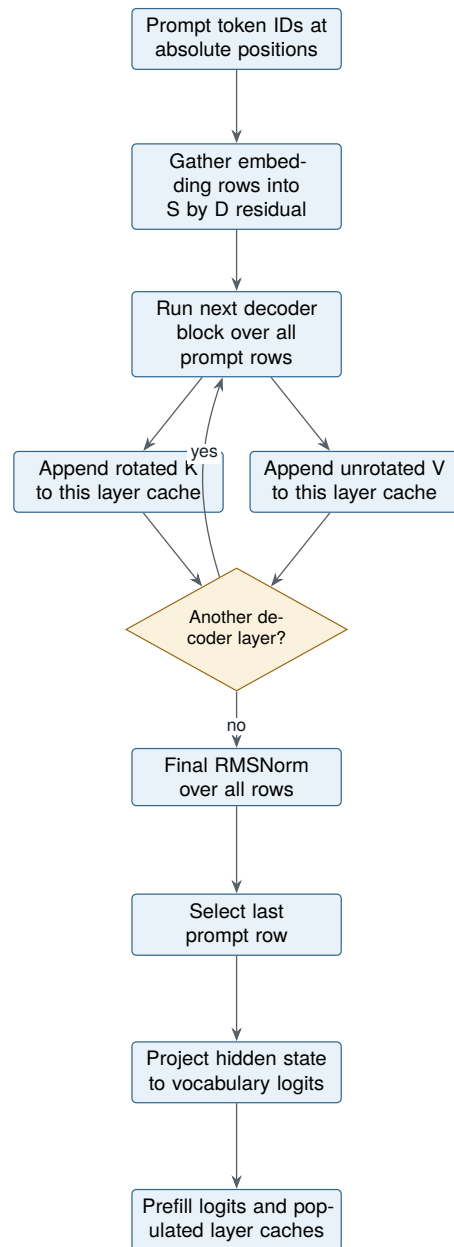
The canonical path uses Float32 arithmetic and reuses the Problem 035 solution. The judge uses a separately implemented Double-accumulation block oracle.

40.9 Independent correctness

The judge compares every captured boundary in every layer against its Double oracle with absolute tolerance $6e-5$ plus relative tolerance $1.2e-4$. It also materializes the cache supplied to the implementation and compares it with the oracle's rotated keys, unrotated values, and absolute positions. This catches an implementation that computes correct logits using a private cache.

The judge also checks the exact work model and rejects an out-of-vocabulary ID, a nonempty prefill cache, and an incompatible layer count. Focused tests reject missing cache writes, wrong model tensor shapes, non-finite model values, and unstable fingerprints.

```
1 swift run leetllm check 039 --cpu
2 swift run leetllm check 039 --solution
```

**Figure 40.1:** Prompt prefill dataflow.

40.10 Performance model: FLOPs, bytes, cache, and allocation

For one layer, projection multiply-add work is approximately

$$2S(D^2 + 2D(H_{kv}d_h) + D^2 + 3DF).$$

Every projection consumes an $[S, in]$ activation matrix and $[out, in]$ weights; the result is $[S, out]$. That is GEMM-shaped even though the readable code uses loops. Unlike one-token GEMV, many prompt rows can reuse each loaded weight tile.

Causal score and value work is approximately

$$4LH_qd_h \frac{S(S+1)}{2}.$$

Float32 cache writes are exact at

$$B_{cache} = 4 \cdot 2LSH_{kv}d_h.$$

The trace reports projection FLOPs, attention FLOPs, estimated weight bytes, cache bytes written, $L \cdot S$ K/V projection input tokens, and zero prior tokens reprojected. Captures deliberately allocate more memory than a production engine; Problem 041 plans reusable ranges without claiming these Swift tensors already use an arena.

40.11 Honest Metal mapping

Problem 039 owns control and state correctness and has no Metal check. It does not label CPU execution as GPU prefill. Earlier lessons provide Metal kernels for GEMM, RMSNorm, RoPE, and attention. A future GPU path would dispatch GEMM-shaped Q/K/V and MLP projections over S rows, then append rotated K/V to layer-specific device cache storage.

Diagnostic capture mode must preserve the same names and positions. Production fusion may remove writes only after a parity path can still expose them. Grid, threadgroup, SIMD-group, barrier, and bounds decisions belong to those earlier kernel lessons; this lesson's new responsibility is ordered orchestration.

40.12 Implementation checkpoints

1. Validate every model tensor and token before allocation.
2. Match the $[S, D]$ embedding capture.
3. Match layer 0 captures and cache positions at offset zero.
4. Repeat at a nonzero offset and verify rotated K changes.
5. Feed layer 0 output into layer 1 and match every capture.
6. Match final RMSNorm for all rows.
7. Select only the last row and match $[V]$ logits.
8. Match work counts and all invalid-state cases.

40.13 Controlled experiments

40.13.1 Prompt-length sweep

Compare $s=1, 8, 64$. Prediction: projection work and cache bytes grow linearly; full causal attention grows quadratically. Projection arithmetic intensity should improve as rows reuse weights.

40.13.2 Position-offset intervention

Run equal IDs at offsets 0 and 32. Prediction: embeddings, norms, projected Q/K/V, and values remain equal; rotated Q/K and downstream attention can change.

40.13.3 Layer-order intervention

Swap two nonidentical fixture blocks. Prediction: shape validation still passes, but the first changed capture is layer 0 output and all later states diverge.

40.13.4 Tied versus independent output

Copy the embedding to an explicit independent matrix, then perturb one row. Prediction: all block and final-norm captures remain equal; only that token's logit changes before argmax consequences.

Write each prediction before running the code.

40.14 Engine integration

Problem 040 receives this populated cache and treats the token sampled from prefill logits as the current token at absolute position `positionOffset+s`. It appends one new record per layer without recomputing these prompt K/V values. Problem 041 derives buffer schedules from the same dimensions. Problem 042 serializes these named captures with the model fingerprint.

40.15 Tradeoffs

- Returning only final-position logits saves output projection work versus all-position logits.
- Full captures improve diagnosis and increase transient memory.
- Tied output reduces model bytes but is a checkpoint policy, not a universal assumption.
- A Float32 scalar path is readable; optimized prefill wants tiled GEMM and fused attention.
- Caching during prefill adds writes now to avoid repeated projections during decode.

40.16 Hints

- Cache `rotatedKeys`, not pre-RoPE keys, and cache V without RoPE.
- Append to the layer whose weights produced the tensors.
- Keep the same `positionOffset` through every layer.
- A reshape from projection width to heads is not a transpose.
- Select $s-1$ only after final RMSNorm.

40.17 Completion checklist

- ☐ Complete model shapes, values, output policy, and cache compatibility validate.

- ☐ Embeddings and all layer captures match the independent oracle.
- ☐ Every layer cache contains rotated K, V, and exact absolute positions.
- ☐ The next layer consumes the previous layer's output.
- ☐ Final logits come from only the last normalized prompt row.
- ☐ Work and cache-byte fields match the documented formulas.
- ☐ Experiment predictions are recorded before results.
- ☐ The fixture is described only as an educational mini-model.
- ☐ No CPU path is claimed to be Metal.

40.18 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

40.18.1 Sources/LeetLLMExercises/P039PromptPrefillExercise.swift

```

1 import LeetLLMCore
2
3 public enum P039PromptPrefillExercise {
4     public static func run(
5         _ request: PromptPrefillRequest,
6         cache: ContiguousKVCache
7     ) throws -> PromptPrefillResult {
8         try P039PromptPrefillContract.validate(request: request, cache: cache)
9         let sequence = request.tokenIDs.count
10        let dimension = request.model.configuration.modelDimension
11        let residual = try FloatTensor(
12            Array(repeating: 0, count: sequence * dimension),
13            shape: [sequence, dimension])
14        let hidden = try FloatTensor(Array(repeating: 0, count: dimension), shape: [dimension])
15        let logits = try FloatTensor(
16            Array(repeating: 0, count: request.model.vocabularySize),
17            shape: [request.model.vocabularySize])
18
19        // TODO: embed all prompt tokens, run every block in order, append each layer's
20        // rotated K and V to the cache, then normalize and unembed the final position.
21        return PromptPrefillResult(
22            promptTokenCount: sequence,
23            finalResidual: residual,
24            finalNormalized: residual,
25            finalHidden: hidden,
26            logits: logits,
27            layers: [],
28            cacheCounts: Array(repeating: 0, count: request.model.layerCount),
29            cachePositions: Array(repeating: [], count: request.model.layerCount),
30            work: MiniDecoderWorkModel(
31                projectionFLOPs: 0,
32                attentionFLOPs: 0,
33                estimatedWeightBytesRead: 0,
34                cacheBytesWritten: 0,
35                keyValueProjectionInputTokens: 0,
36                priorKeyValueTokensReprojected: 0))
37    }
38 }

```

40.19 Worked solution

Solution strategy

Load a prompt as a sequence of token IDs $[t_0, t_1, \dots, t_{S-1}]$. Gather embeddings by copying rows from the vocabulary table: $\text{residual}[i] = \text{embedding}[t_i]$. Execute each decoder block in checkpoint order, passing the residual through RoPE at absolute position offsets $[\text{positionOffset}, \text{positionOffset}+1, \dots, \text{positionOffset}+S-1]$. After each block, append the layer's rotated K and unrotated V values at each position to a persistent cache. After all layers, apply final RMSNorm to the residual, select the last row (hidden state at row $S-1$), project to vocabulary logits via tied or independent output weights. Capture work counts for FLOPs, bytes, and token-dimension products.

Correctness invariant

Cache appends must be done in checkpoint order (layer 0 before layer 1, etc.) and must preserve all prior K/V records (no overwrites). RoPE positions are absolute, not relative to prompt start. Final hidden unembedding uses only the last sequence row (index $S-1$), not a reduction over all rows or row 0. All K/V appends at a layer must occur before that layer completes, so decode can read a contiguous cache. Prefill returns the sampled next token from final logits and the complete cache for later decode calls.

Common failure mode

Applying RoPE before caching (rotating K after cache append) or using relative position offsets breaks cache semantics and causes decode position collision. Selecting row 0 or averaging all rows instead of row $S-1$ produces reference logits that do not match the true next-token probability distribution. Interleaving cache appends from different layers (layer 0 partial, layer 1, back to layer 0) causes cache coherency loss.

40.19.1 Implementation points

Embedding gather and residual initialization

For each prompt token ID t_i , copy embedding table row t_i to residual row i : $\text{residual}[i] = \text{embedding}[t_i]$. This produces a $[S, D]$ matrix where S is prompt length and D is model dimension. The embeddings represent the token's input semantic content and position in the prompt sequence.

Absolute logical positions and cache metadata

Each token at sequence index i receives absolute logical position $= \text{positionOffset} + i$. RoPE is applied at this absolute position. Cache append stores metadata binding each (layer, position) pair to K and V slices. Decode later reads cache by position range $[0, \text{current_position}]$ to determine attention visibility.

Rotated K vs. unrotated V in cache

Apply RoPE to Q and K projections before attention, then append the rotated K to cache. Append the unrotated V to cache (rotation is not applied to V). This separation allows decode to apply RoPE at the current position to new K values while reading unrotated V from prior cache entries directly.

Final row selection and projection shapes

Prefill attention is a full $[S, S]$ matrix computation ($O(S^2)$ causal pairs). Projection shapes are all $[S, D]$ inputs (batch dimension). After all layers, final norm is applied to all S rows of the residual. Final hidden is extracted as `residual[S-1]`, a $[D]$ vector. Unembedding projects $[D]$ to $[V]$ logits for the vocabulary.

40.19.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P039PromptPrefillSolution.swift

- Line 3: `public enum P039PromptPrefillSolution {`
- Line 4: `public static func run(`

Sources/LeetLLMSolutions/MiniDecoderCPUEngine.swift

- Line 4: `public enum MiniDecoderCPUEngine {`
- Line 5: `public static func prefill(`
- Line 71: `public static func decode(`
- Line 209: `static func normalize(`
- Line 233: `static func project(_ input: FloatTensor, weights: FloatTensor) throws -> FloatTensor {`
- Line 251: `static func projectVector(_ input: FloatTensor, weights: FloatTensor) throws -> FloatTensor {`
- Line 257: `static func rotate(`
- Line 282: `static func tokenSlice(`
- Line 295: `static func workModel(`
- Line 332: `static func add(_ lhs: FloatTensor, _ rhs: FloatTensor) throws -> FloatTensor {`

40.19.3 Complete canonical listings

40.19.4 Sources/LeetLLMSolutions/P039PromptPrefillSolution.swift

```
1 import LeetLLMCore
2
3 public enum P039PromptPrefillSolution {
4     public static func run(
5         _ request: PromptPrefillRequest,
6         cache: ContiguousKVCache
7     ) throws -> PromptPrefillResult {
8         try MiniDecoderCPUEngine.prefill(request, cache: cache)
9     }
10 }
```

40.19.5 Sources/LeetLLMSolutions/MiniDecoderCPUEngine.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 public enum MiniDecoderCPUEngine {
5     public static func prefill(
6         _ request: PromptPrefillRequest,
7         cache: ContiguousKVCache
8     ) throws -> PromptPrefillResult {
9         try P039PromptPrefillContract.validate(request: request, cache: cache)
10         let model = request.model
11         let sequence = request.tokenIDs.count
```

```

12 let dimension = model.configuration.modelDimension
13 var embedded: [Float] = []
14 embedded.reserveCapacity(sequence * dimension)
15 for tokenID in request.tokenIDs {
16     let start = tokenID * dimension
17     embedded.append(contentsOf: model.tokenEmbedding.storage[start..

```

```

85     var traces: [MiniDecoderLayerTrace] = []
86
87     for (layer, weights) in model.blocks.enumerated() {
88         let residualInput = residual
89         let attentionNormalized = try normalize(
90             residual, gamma: weights.attentionNormGamma, epsilon: configuration.rmsNormEpsilon)
91         let queryProjection = try project(attentionNormalized, weights: weights.queryWeights)
92         let keyProjection = try project(attentionNormalized, weights: weights.keyWeights)
93         let valueProjection = try project(attentionNormalized, weights: weights.valueWeights)
94         let queries = try FloatTensor(
95             queryProjection.storage,
96             shape: [1, configuration.queryHeadCount, configuration.headDimension])
97         let keys = try FloatTensor(
98             keyProjection.storage,
99             shape: [1, configuration.keyValueHeadCount, configuration.headDimension])
100        let values = try FloatTensor(
101            valueProjection.storage,
102            shape: [1, configuration.keyValueHeadCount, configuration.headDimension])
103        let rotatedQueries = try rotate(
104            queries, positionOffset: request.logicalPosition, configuration: configuration)
105        let rotatedKeys = try rotate(
106            keys, positionOffset: request.logicalPosition, configuration: configuration)
107        let queryVector = try tokenSlice(
108            rotatedQueries,
109            token: 0,
110            headCount: configuration.queryHeadCount,
111            headDimension: configuration.headDimension)
112        try cache.append(
113            layer: layer,
114            logicalPosition: request.logicalPosition,
115            key: try tokenSlice(
116                rotatedKeys,
117                token: 0,
118                headCount: configuration.keyValueHeadCount,
119                headDimension: configuration.headDimension),
120            value: try tokenSlice(
121                values,
122                token: 0,
123                headCount: configuration.keyValueHeadCount,
124                headDimension: configuration.headDimension))
125        let attended = try P023CachedAttentionSolution.attend(
126            query: queryVector,
127            cache: cache,
128            layer: layer,
129            queryLogicalPosition: request.logicalPosition,
130            queryHeadCount: configuration.queryHeadCount)
131        let attentionHeads = try FloatTensor(
132            attended.storage,
133            shape: [1, configuration.queryHeadCount, configuration.headDimension])
134        let concatenatedAttention = try FloatTensor(attended.storage, shape: [1, dimension])
135        let attentionProjection = try project(
136            concatenatedAttention, weights: weights.attentionOutputWeights)
137        let postAttentionResidual = try add(residual, attentionProjection)
138        let mlpNormalized = try normalize(
139            postAttentionResidual,
140            gamma: weights.mlpNormGamma,
141            epsilon: configuration.rmsNormEpsilon)
142        let gateProjection = try project(mlpNormalized, weights: weights.gateWeights)
143        let upProjection = try project(mlpNormalized, weights: weights.upWeights)
144        let activatedGate = try FloatTensor(
145            gateProjection.storage.map { $0 / (1 + exp(-$0)) },
146            shape: gateProjection.shape)
147        let gatedHidden = try FloatTensor(
148            zip(activatedGate.storage, upProjection.storage).map(*),
149            shape: gateProjection.shape)
150        let downProjection = try project(gatedHidden, weights: weights.downWeights)
151        residual = try add(postAttentionResidual, downProjection)
152        let result = DecoderBlockResult(
153            state: DecoderBlockState(
154                residual: residual, positionOffset: request.logicalPosition),
155            intermediates: DecoderBlockIntermediates(
156                attentionNormalized: attentionNormalized,
157                queries: queries,

```

```

158     keys: keys,
159     values: values,
160     rotatedQueries: rotatedQueries,
161     rotatedKeys: rotatedKeys,
162     attentionHeads: attentionHeads,
163     concatenatedAttention: concatenatedAttention,
164     attentionProjection: attentionProjection,
165     postAttentionResidual: postAttentionResidual,
166     mlpNormalized: mlpNormalized,
167     gateProjection: gateProjection,
168     upProjection: upProjection,
169     activatedGate: activatedGate,
170     gatedHidden: gatedHidden,
171     downProjection: downProjection))
172   traces.append(MiniDecoderLayerTrace(
173     layerIndex: layer,
174     residualInput: residualInput,
175     block: result,
176     cachePositions: try cache.logicalPositions(layer: layer)))
177 }
178 let finalHiddenMatrix = try normalize(
179   residual, gamma: model.finalNormGamma, epsilon: configuration.rmsNormEpsilon)
180 let finalHidden = try FloatTensor(finalHiddenMatrix.storage, shape: [dimension])
181 let logits = try projectVector(finalHidden, weights: model.outputWeights)
182 let sampling = try P038LogitsSamplingSolution.sample(
183   logits: logits.storage,
184   strategy: request.samplingStrategy,
185   generator: &generator)
186 let countsAfter = try (0..

```

```

231 }
232
233 static func project(_ input: FloatTensor, weights: FloatTensor) throws -> FloatTensor {
234     let rows = input.shape[0]
235     let inputWidth = input.shape[1]
236     let outputWidth = weights.shape[0]
237     var output = Array(repeating: Float.zero, count: rows * outputWidth)
238     for row in 0..

```



```

304 let hidden = configuration.hiddenDimension
305 let layerTerms = dimension * dimension + 2 * dimension * keyValueWidth
306 + dimension * dimension + 3 * dimension * hidden
307 let projectionFLOPs = 2 * tokenCount * model.layerCount * layerTerms
308 + 2 * model.vocabularySize * dimension
309 let visiblePairs = decode
310 ? (attentionTokenCount ?? tokenCount)
311 : tokenCount * (tokenCount + 1) / 2
312 let attentionFLOPs = 4 * visiblePairs * configuration.queryHeadCount
313 * configuration.headDimension * model.layerCount
314 let weightElements = model.blocks.reduce(model.outputWeights.elementCount) { partial, block in
315   partial + block.queryWeights.elementCount + block.keyWeights.elementCount
316   + block.valueWeights.elementCount + block.attentionOutputWeights.elementCount
317   + block.gateWeights.elementCount + block.upWeights.elementCount
318   + block.downWeights.elementCount + block.attentionNormGamma.elementCount
319   + block.mlpNormGamma.elementCount
320 } + model.finalNormGamma.elementCount
321 return MiniDecoderWorkModel(
322   projectionFLOPs: projectionFLOPs,
323   attentionFLOPs: attentionFLOPs,
324   estimatedWeightBytesRead: weightElements * MemoryLayout<Float>.stride,
325   cacheBytesWritten: tokenCount * model.layerCount * 2
326   * configuration.keyValueHeadCount * configuration.headDimension
327   * MemoryLayout<Float>.stride,
328   keyValueProjectionInputTokens: tokenCount * model.layerCount,
329   priorKeyValueTokensReprojected: 0)
330 }
331
332 static func add(_ lhs: FloatTensor, _ rhs: FloatTensor) throws -> FloatTensor {
333   try FloatTensor(zip(lhs.storage, rhs.storage).map(+), shape: lhs.shape)
334 }
335 }

```

Autoregressive Decode

41.1 Why this exists

Generation is serial: the token selected now becomes the input token for the next model step. Re-running the full prefix would repeatedly project every old token's K and V. A real decode loop instead projects one current token, appends one K/V record per layer, attends over existing records, and advances sampling state exactly once.

This lesson reuses Problem 039's complete educational mini-model and cache. It owns control flow and mutable state correctness. Earlier lessons own the individual kernels.

41.2 Learning outcomes

You can:

- execute a complete decoder for one current token;
- apply RoPE at the supplied absolute logical position;
- append current rotated K/V once before cached GQA attention;
- preserve every prior cache byte and avoid prior-token K/V projections;
- compare cached output with full-sequence recomputation;
- advance a seeded sampler deterministically across repeated calls; and
- implement EOS and maximum-token stopping in a generation session.

41.3 Prerequisites

- Problem 023 for append-then-attend cached attention.
- Problem 035 for norm, projection, residual, and SwiGLU order.
- Problems 038 and 039 for sampling and the populated prompt cache.

41.4 Vocabulary

- **Current token:** token ID embedded by this decode call.
- **Next token:** token selected from this call's output logits.
- **Logical position:** absolute position assigned to the current token.
- **Cache prefix:** previously appended records that must remain unchanged.
- **Full-recomputation oracle:** run all tokens as a prompt and inspect its last row.
- **Seed progression:** deterministic sampler state after an exact call sequence.
- **Generation session:** owner of model, cache, next position, strategy, and PRNG.

41.5 One-token equations

For current token t_p at position p , gather one row

$$x^{(0)} = E_{t_p, :}$$

At layer ℓ , compute direct-gamma norm and projections only for this row:

$$q = W_q N(x), \quad k = W_k N(x), \quad v = W_v N(x).$$

Apply adjacent-pair RoPE at absolute position p and append before attention:

$$\text{cacheK}[\ell, p] = \text{RoPE}(k, p), \quad \text{cacheV}[\ell, p] = v.$$

For query head h , integer-division GQA selects

$$h_{kv} = \left\lfloor \frac{h}{H_q/H_{kv}} \right\rfloor.$$

Cached attention reads all visible positions through p , then output projection, the first residual, MLP norm, SwiGLU, down projection, and the second residual match Problem 035. Final norm and $[V, D]$ unembedding produce logits; Problem 038 selects the next token.

41.6 Worked state transition

Suppose prefill cached prompt $[1, 4]$ at positions $[3, 4]$. Decode current token 2 at position 5:

```
1 before each layer: cache count 2, positions [3,4]
2 projected this call: one Q row, one K row, one V row
3 append: rotated K(5), V(5)
4 attention visibility: [3,4,5]
5 after each layer: cache count 3, positions [3,4,5]
6 prior K/V projection input tokens: 0
```

The selected token from these logits is input to position 6. It is not added to cache until that next decode call computes its layer-specific K/V.

41.7 Shape and state contract

Batch size and sequence axis are both one in computation.

Value	Shape
current token ID	scalar integer in $0 \dots V$
embedded residual	$[1, D]$
Q	$[1, H_q, d_h]$
K, V	$[1, H_{kv}, d_h]$
cache record	$[H_{kv}, d_h]$ per tensor per layer
cached attention output	$[H_q, d_h]$, captured as $[1, H_q, d_h]$
final hidden	$[D]$
logits	$[V]$

The cache configuration must match model L , H_{kv} , and dh . All layer position lists must be equal. If nonempty, their last position plus one must be the request's logical position. Capacity must remain. The token, sampling configuration, finite model tensors, and position are validated before mutation.

On success every layer count changes by exactly one. On contract failure no append occurs. The canonical cache is preallocated and has stable storage.

41.8 CPU reference algorithm

For each layer:

1. RMS-normalize the $[1,D]$ residual with attention gamma.
2. Use GEMV-shaped $[out,in]$ Q/K/V projections.
3. Reshape, then rotate Q and K at the exact logical position.
4. Append one rotated K and one V record for this layer.
5. Call cached online GQA attention over visible cache positions.
6. Concatenate heads, output-project, and add the incoming residual.
7. Apply MLP norm, gate/up projections, SiLU product, down projection, and residual.

After all layers, final-normalize, unembed, and call the Problem 038 sampler. Return all captures, counts before/after, positions, sampling trace, and work fields.

41.9 Stateful generation helper

MiniDecoderGenerationSession owns one model, contiguous cache, sampling strategy, SeededGenerator, optional EOS ID, and next logical position.

`generate(prompt,maxNewTokens)` first prefills. If the limit is positive, it samples the first generated token from prefill logits. Each later iteration decodes the latest generated token and appends the selected next token. It stops when EOS is selected or the generated count reaches the limit. `maxNewTokens=0` returns after prefill without advancing the sampler.

Equal seeds, prompt, strategy, and limits produce equal tokens, traces, and final PRNG state. Greedy calls do not advance the generator; stochastic calls advance once per selected token.

41.10 Independent correctness

The judge prefills two tokens into a cache, then performs three sequential decode calls. After each call it appends the current token to an independent token list and runs the complete Double prefill oracle over that full sequence. It compares the cached path with the oracle's last row at every layer boundary, final hidden, logits, and sampled decision.

It separately checks that:

- prior cache positions and values are byte-for-byte preserved;
- each layer grows by exactly one;
- `keyValueProjectionInputTokens == L` for each call;
- `priorKeyValueTokensReprojected == 0`; and
- candidate IDs, draw, and generator state are exact while candidate floats use tolerance.

Position gaps and invalid token IDs are rejected. Tests also inject a false reprojection counter and require rejection.

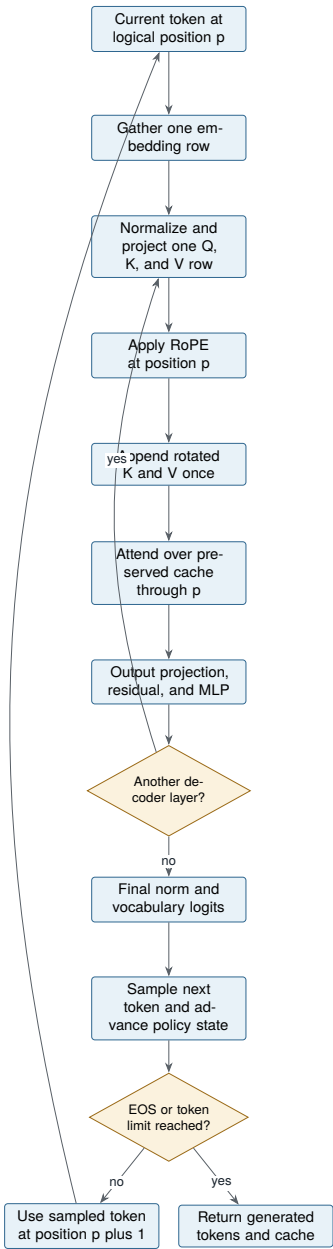


Figure 41.1: Autoregressive cache loop.

```
1 swift run leetllm check 040 --cpu
2 swift run leetllm check 040 --solution
```

41.11 Performance model: FLOPs, bytes, and cache

Per layer, projection work is the S=1 form:

$$2(D^2 + 2D(H_{kv}d_h) + D^2 + 3DF).$$

At τ visible cached tokens, attention dot and weighted-value work is roughly

$$4H_qd_hT$$

per layer. Weight bytes are reread for every serial token in this scalar CPU path. Cache writes per call are fixed:

$$B_{write} = 4 \cdot 2LH_{kv}d_h.$$

Cache reads grow with context. Crucially, K/V projection work for old tokens is zero. The trace exposes that invariant directly rather than inferring it from elapsed time.

41.12 Honest Metal mapping

Problem 040 is canonical CPU only. No fake Metal wrapper is provided. Decode projections are GEMV-shaped and tend to stream weights with little activation reuse. Problem 004 owns GEMV mapping, Problem 012 norm/projection fusion, Problem 023 cached attention, and Problems 032-033 quantized projection choices.

A future GPU decode loop should keep weights and cache device-resident, avoid a CPU round trip between layers, and preserve append ordering. Fusing kernels is not allowed to alter absolute positions, cache ownership, GQA division, direct gamma, or seeded sampling call count.

41.13 Implementation checkpoints

1. Reject token, cache, position, capacity, and sampler errors before mutation.
2. Match one-token attention norm and Q/K/V at an empty position-zero cache.
3. Match RoPE and one append per layer at a nonzero position.
4. Match cached attention with a two-token prefix.
5. Match both residuals and all MLP captures.
6. Match final norm, logits, selected token, draw, and PRNG state.
7. Repeat across several positions and preserve every prior cache element.
8. Implement EOS and maximum-token stopping.

41.14 Controlled experiments

41.14.1 Context-length sweep

Hold model and current token fixed; compare $\tau=1, 16, 256$. Prediction: projection work and cache writes stay constant, while attention reads/work grow linearly.

41.14.2 Recompute intervention

Implement a private full-prefix path for comparison. Prediction: outputs match within tolerance, but K/V projection input tokens grow with τ instead of remaining exactly L .

41.14.3 GQA sweep

Compare compatible fixtures with $H_{kv}=H_q$, $H_q/2$, and 1. Prediction: query and output projections are unchanged; K/V weight and cache bytes fall with H_{kv} .

41.14.4 Seed call-order intervention

Insert one extra stochastic sample in only one session. Prediction: model logits remain equal at the insertion point; later selected tokens can diverge because PRNG ownership includes call count.

41.15 Engine integration

The same model types and cache object flow from Problem 039 into this lesson. Problem 041 plans the one-token intermediates separately from prefill. Problem 042 captures the complete prefill path and applies the same convention checks that protect decode. Later profiling may choose different kernels without changing this state machine.

41.16 Tradeoffs

- A contiguous cache is simple; paged or ring storage changes allocation, not attention math.
- Online cached attention removes score storage; it still reads every visible K/V.
- Host sampling is inspectable; device filtering can reduce logits transfer.
- Full traces aid parity and add allocations that production mode may disable.
- Stateful sessions make ordering explicit but require one owner per request.

41.17 Hints

- Append current K/V before attention so the token can attend to itself.
- Do not run RoPE over cached keys again.
- Use the request's absolute position, not the cache count, as the RoPE position.
- Preserve [out, in] projection orientation.
- Advance the stochastic generator once per sampled token, never once per candidate.

41.18 Completion checklist

- ☐ Invalid token, position, sampler, cache shape, and capacity fail before append.
- ☐ One current token is projected per layer.
- ☐ RoPE uses the exact absolute logical position.
- ☐ Each layer appends once and preserves its prior cache prefix.
- ☐ Cached captures and logits match full recomputation at several positions.
- ☐ Prior K/V projection count remains zero.
- ☐ Seed progression and sampling trace are deterministic.
- ☐ EOS and maximum-token stop policies are tested.
- ☐ CPU orchestration is not presented as a Metal kernel.

41.19 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

41.19.1 Sources/LeetLLMExercises/P040AutoregressiveDecodeExercise.swift

```

1 import LeetLLMCore
2
3 public enum P040AutoregressiveDecodeExercise {
4     public static func run(
5         _ request: AutoregressiveDecodeRequest,
6         cache: ContiguousKVCache,
7         generator: inout SeededGenerator
8     ) throws -> AutoregressiveDecodeResult {
9         try P040AutoregressiveDecodeContract.validate(request: request, cache: cache)
10        let dimension = request.model.configuration.modelDimension
11        let hidden = try FloatTensor(Array(repeating: 0, count: dimension), shape: [dimension])
12        let logits = try FloatTensor(
13            Array(repeating: 0, count: request.model.vocabularySize),
14            shape: [request.model.vocabularySize])
15        let sampling = SamplingTrace(
16            selectedToken: 0,
17            retainedCandidates: [SamplingCandidate(tokenID: 0, logit: 0, probability: 1)],
18            randomDraw: nil)
19
20        // TODO: project only the current token, rotate at logicalPosition, append one
21        // K/V record per layer, attend over the existing cache, and sample the logits.
22        return AutoregressiveDecodeResult(
23            inputTokenID: request.tokenID,
24            logicalPosition: request.logicalPosition,
25            selectedNextTokenID: sampling.selectedToken,
26            sampling: sampling,
27            finalResidual: hidden,
28            finalHidden: hidden,
29            logits: logits,
30            layers: [],
31            cacheCountsBefore: try (0..

```


41.20 Worked solution

Solution strategy

Embed a single current token to produce $[1,D]$ residual. Run each decoder block in order, applying RoPE at the supplied logical position before cached attention. Append the current token's rotated K and unrotated V to cache at this position before calling cached attention (so the current token is visible to itself and future tokens). Perform cached attention by reading all cache entries at positions $[0, \text{logicalPosition}]$ inclusive. Apply final RMSNorm to the $[1,D]$ residual, extract final hidden (the only row), project to $[V]$ logits, sample the next token, increment the logical position counter, return the sampled token as input to the next decode call.

Correctness invariant

Cache prefix (all prior K/V records) must remain unchanged. Logical position must be the true absolute position, incremented by 1 on each decode call. Cached attention reads all visible positions $[0, \text{currentPosition}]$ inclusive; re-running full-sequence attention instead defeats cache reuse (every prior token would be re-projected). RoPE must be applied before cache append, not after. The sampled token from this call becomes the current token for the next decode call.

Common failure mode

Using full-sequence attention instead of cached single-token attention re-projects all prior tokens, eliminating the memory and compute savings of cached inference. Applying RoPE after cache append or using the previous position instead of current position breaks position semantics. Not incrementing logical position causes new cache appends to collide with prior positions. Advancing the sampler state inconsistently across calls breaks reproducibility of multi-token generation.

41.20.1 Implementation points

Logical position and cache visibility

Decode assigns one new logical position per call. Cached attention attends over positions $[0, \text{currentPosition}]$ inclusive (causal mask over cache depth). Prior cache entries are never re-computed; decode only projects the current token, appends its K/V, and attends over all cached records.

Single-token projection and head reshaping

Current token embedding is $[1,D]$. Q/K/V projections produce $[1,D]$ outputs. Reshape to $[1,H_q,d_h]$ and $[1,H_k,d_h]$. RoPE is applied at current logicalPosition, rotating both query and key within their $[1,H_q,d_h]$ and $[1,H_k,d_h]$ shapes.

Cache append ordering: before attention, with rotated K

Append rotated K and unrotated V to cache before calling cached attention. This ensures the current token's K and V are available during its own attention computation (enabling self-attention if needed by the architecture). Do not project or cache prior tokens again.

State advancement and session invariants

MiniDecoderGenerationSession maintains nextLogicalPosition, cache, generator (PRNG state), and model. Prefill initializes nextLogicalPosition = positionOffset + len(promptTokenIDs). Each decode call increments nextLogicalPosition by 1 and advances generator state deterministically. Stopping conditions are EOS token match or maximum token count reached.

41.20.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P040AutoregressiveDecodeSolution.swift

- Line 3: public enum P040AutoregressiveDecodeSolution {
- Line 4: public static func run(
- Line 13: public final class MiniDecoderGenerationSession {
- Line 22: public init(
- Line 42: public func prefill(
- Line 56: public func decode(tokenID: Int) throws -> AutoregressiveDecodeResult {
- Line 74: public func generate(

41.20.3 Complete canonical listings

41.20.4 Sources/LeetLLMSolutions/P040AutoregressiveDecodeSolution.swift

```

1 import LeetLLMCore
2
3 public enum P040AutoregressiveDecodeSolution {
4     public static func run(
5         _ request: AutoregressiveDecodeRequest,
6         cache: ContiguousKVCache,
7         generator: inout SeededGenerator
8     ) throws -> AutoregressiveDecodeResult {
9         try MiniDecoderCPUEngine.decode(request, cache: cache, generator: &generator)
10    }
11 }
12
13 public final class MiniDecoderGenerationSession {
14     public let model: MiniDecoderModel
15     public let cache: ContiguousKVCache
16     public let samplingStrategy: SamplingStrategy
17     public let endOfSequenceTokenID: Int?
18     public private(set) var generator: SeededGenerator
19
20     private var nextLogicalPosition: Int?
21
22     public init(
23         model: MiniDecoderModel,
24         cacheCapacity: Int,
25         samplingStrategy: SamplingStrategy,
26         seed: UInt64,
27         endOfSequenceTokenID: Int? = nil
28     ) throws {
29         if let endOfSequenceTokenID,
30            !(0..

```

```

37     self.samplingStrategy = samplingStrategy
38     self.endOfSequenceTokenID = endOfSequenceTokenID
39     generator = SeededGenerator(seed: seed)
40 }
41
42 public func prefill(
43     tokenIDs: [Int],
44     positionOffset: Int = 0
45 ) throws -> PromptPrefillResult {
46     let result = try MiniDecoderCPUEngine.prefill(
47         PromptPrefillRequest(
48             model: model, tokenIDs: tokenIDs, positionOffset: positionOffset),
49             cache: cache)
50     let (nextPosition, overflow) = positionOffset.addingReportingOverflow(tokenIDs.count)
51     guard !overflow else { throw DecoderBlockError.positionOverflow }
52     nextLogicalPosition = nextPosition
53     return result
54 }
55
56 public func decode(tokenID: Int) throws -> AutoregressiveDecodeResult {
57     guard let logicalPosition = nextLogicalPosition else {
58         throw MiniDecoderError.emptyPrompt
59     }
60     let result = try MiniDecoderCPUEngine.decode(
61         AutoregressiveDecodeRequest(
62             model: model,
63             tokenID: tokenID,
64             logicalPosition: logicalPosition,
65             samplingStrategy: samplingStrategy),
66             cache: cache,
67             generator: &generator)
68     let (nextPosition, overflow) = logicalPosition.addingReportingOverflow(1)
69     guard !overflow else { throw DecoderBlockError.positionOverflow }
70     nextLogicalPosition = nextPosition
71     return result
72 }
73
74 public func generate(
75     promptTokenIDs: [Int],
76     maxNewTokens: Int,
77     positionOffset: Int = 0
78 ) throws -> MiniDecoderGenerationResult {
79     guard maxNewTokens >= 0 else {
80         throw MiniDecoderError.invalidGenerationLimit(maxNewTokens)
81     }
82     let prefillResult = try prefill(
83         tokenIDs: promptTokenIDs, positionOffset: positionOffset)
84     guard maxNewTokens > 0 else {
85         return MiniDecoderGenerationResult(
86             promptTokenIDs: promptTokenIDs,
87             generatedTokenIDs: [],
88             stopReason: .maximumTokenCount,
89             prefill: prefillResult,
90             decodeSteps: [])
91     }
92     let firstSampling = try P038LogitsSamplingSolution.sample(
93         logits: prefillResult.logits.storage,
94         strategy: samplingStrategy,
95         generator: &generator)
96     var generated = [firstSampling.selectedToken]
97     if generated.last == endOfSequenceTokenID {
98         return MiniDecoderGenerationResult(
99             promptTokenIDs: promptTokenIDs,
100             generatedTokenIDs: generated,
101             stopReason: .endOfSequence,
102             prefill: prefillResult,
103             decodeSteps: [])
104     }
105     var steps: [AutoregressiveDecodeResult] = []
106     while generated.count < maxNewTokens {
107         let step = try decode(tokenID: generated.last!)
108         steps.append(step)
109         generated.append(step.selectedNextTokenID)

```

```
110     if generated.last == endOfSequenceTokenID {
111         return MiniDecoderGenerationResult(
112             promptTokenIDs: promptTokenIDs,
113             generatedTokenIDs: generated,
114             stopReason: .endOfSequence,
115             prefill: prefillResult,
116             decodeSteps: steps)
117     }
118 }
119 return MiniDecoderGenerationResult(
120     promptTokenIDs: promptTokenIDs,
121     generatedTokenIDs: generated,
122     stopReason: .maximumTokenCount,
123     prefill: prefillResult,
124     decodeSteps: steps)
125 }
126 }
```

Buffer Reuse and Memory Planning

42.1 Why this exists

An inference graph can allocate far more temporary storage than is live at one time. Q is dead after attention; an MLP buffer created later may safely reuse part of its physical range. Repeated heap allocation in a hot path adds latency, fragmentation, and unstable memory behavior. Reuse without a precise lifetime model is worse: one operation can overwrite a value that another still reads.

This lesson turns prefill and decode intermediates into validated lifetimes and assigns aligned offsets in one arena. The result is an executable plan, not just a peak-memory estimate. The current Swift tensor engine does **not** allocate from this arena; integrating those offsets is future work and is not implied by a passing planner.

42.2 Learning outcomes

You can:

- express tensor liveness as inclusive operation intervals;
- prove when two buffers may share physical bytes;
- account for alignment holes separately from requested bytes;
- implement deterministic first-fit and best-fit allocation;
- validate every placement against simultaneous liveness;
- derive prefill and one-token decode schedules from model dimensions; and
- distinguish a memory plan from an allocator-backed execution engine.

42.3 Prerequisites

- Problem 002 for shape-to-storage and byte-address reasoning.
- Problems 039 and 040 for prefill and decode intermediate shapes.
- Problem 022 for the distinction between persistent cache capacity and transient work.

42.4 Vocabulary

- **Lifetime**: inclusive interval `[firstOperation, lastOperation]` during which bytes are needed.
- **Arena**: one contiguous physical byte range containing all placements.
- **Placement**: (name, offset, size, alignment, lifetime) assignment.
- **Physical overlap**: two half-open byte ranges share at least one byte.
- **First-fit**: choose the lowest-offset aligned gap that fits.
- **Best-fit**: choose the fitting gap with least remainder, then lowest offset.
- **Naive bytes**: sum of requested sizes with no reuse or alignment padding.

- **Peak live bytes:** maximum sum of requested sizes live at an operation.

42.5 Lifetime and aliasing math

For buffer i , let its inclusive lifetime be $[f_i, l_i]$ and its physical range be $[o_i, o_i + b_i)$. Two lifetimes overlap when

$$f_i \leq l_j \quad \wedge \quad f_j \leq l_i.$$

Two physical ranges overlap when

$$o_i < o_j + b_j \quad \wedge \quad o_j < o_i + b_i.$$

A valid plan requires that no pair satisfies both predicates. Because operation endpoints are inclusive, $[0, 2]$ and $[2, 4]$ are simultaneously live at operation 2 and cannot reuse overlapping bytes. $[0, 2]$ and $[3, 4]$ may reuse.

For power-of-two alignment a , the next aligned offset is

$$\text{align}(x, a) = (x + a - 1) \& \sim (a - 1).$$

Every addition, multiplication, and aligned-size calculation is checked for integer overflow.

42.6 Worked placement example

The judge owns these lifetimes:

Name	Lifetime	Bytes	Alignment
a	0...2	24	8
b	1...1	8	16
c	2...4	16	8
d	3...3	20	4

Deterministic first-fit produces:

```
1 a -> offset 0
2 b -> offset 32
3 c -> offset 24
4 d -> offset 0, reusing bytes formerly used by a
```

At operation 2, a and c are both live, so c begins at 24. At operation 3, a is dead and d can use offset 0. The arena is 40 bytes. Peak live requested storage is 40 bytes and naive requested storage is 68 bytes.

Alignment means these quantities do not have a universal ordering beyond nonnegativity. An arena can exceed naiveByteCount for a sparse, highly aligned fixture even when every placement is valid. Reuse is established by actual overlapping physical ranges assigned to disjoint lifetimes, not by assuming arenaByteCount < naiveByteCount for every input.

42.7 Input and output contract

Each `BufferLifetime` requires:

- a unique nonempty name;
- $0 \leq \text{firstOperation} \leq \text{lastOperation}$;
- positive `byteSize`; and
- positive power-of-two alignment.

The planner returns one placement per input plus strategy, arena size, peak live bytes, naive bytes, and explicit reuseAssignments. Input order must not change the result. Lifetimes are sorted by first operation, then last operation, then name. Placement output follows this deterministic planning order.

Plan validation checks exact lifetime metadata, nonnegative aligned offsets, $\text{offset} + \text{size} \leq \text{arenaByteCount}$, and every pair of simultaneously live buffers. The validator does not trust summary fields as proof of safety; it checks the placements themselves.

42.8 Deterministic planning algorithm

For each sorted lifetime:

1. Keep prior placements whose `lastOperation` $\geq$ `current.firstOperation`.
2. Convert them to physical ranges and merge touching/overlapping ranges.
3. Enumerate aligned gaps from offset zero through the current arena end.
4. For first-fit, choose the candidate with lowest offset.
5. For best-fit, choose least unused remainder, then lowest offset.
6. If no existing gap fits, align the current arena end and grow it.
7. Check $\text{offset} + \text{size}$ for overflow and record the placement.

After all placements, compute naive bytes, sample inclusive liveness at every distinct first/last operation, and report previous disjoint-lifetime placements whose physical ranges overlap each new placement.

The planner is offline and deterministic. It does not allocate memory, run a tensor operation, or mutate the decoder.

42.9 Decoder-derived schedules

`MiniDecoderBufferSchedules` derives byte sizes from the shared model. Prefill uses `S` rows; decode uses one row. Both include residual, attention norm, Q/K/V, attention output, post-attention state, MLP norm, gate, up, gated, and down buffers at 64-byte alignment.

The schedule names operations `0...14` and keeps residual/post-attention live across their consumers. Prefill bytes scale with `S`; decode transient bytes do not. `cachedTokenCount` must be positive but does not change the current decode schedule because persistent KV cache and attention scratch are not placed in this arena. The cache remains separately preallocated by Problems 022/039/040.

Run the report:

```
1 swift run leetllm benchmark 041 --tokens 16 --cached-tokens 16
2 swift run leetllm benchmark 041 --tokens 128 --cached-tokens 128
```

Despite the shared benchmark command family, Problem 041 reports deterministic plans and does not claim elapsed-time performance. It prints both placements and the fact that current Swift tensors are not

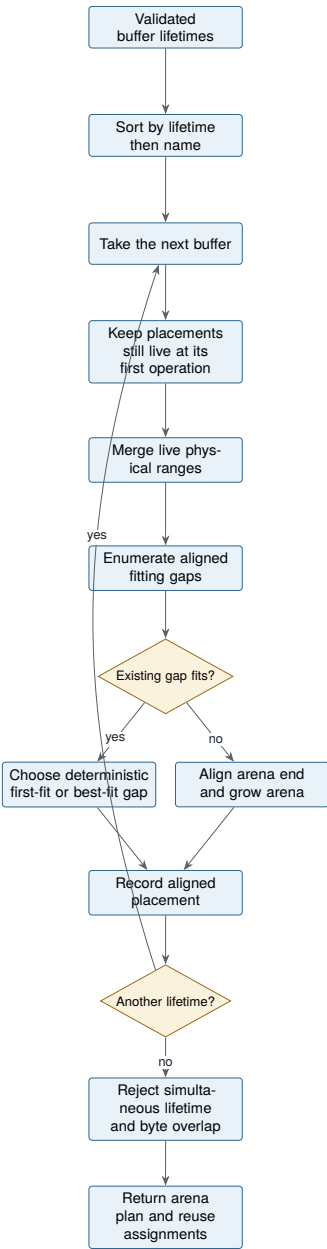


Figure 42.1: Lifetime arena planning.

allocated from them.

42.10 Independent correctness

The judge validates canonical first-fit offsets and statistics, deterministic best-fit output for reversed input, non-aliasing of simultaneous lifetimes, and reuse in model-derived prefill/decode plans. It also rejects duplicate names, non-power-of-two alignment, reversed operation ranges, and aligned-size overflow.

The contract independently checks returned placements. A malicious or mistaken planner cannot pass merely by reporting the expected arena size.

```
1 swift run leetllm check 041 --cpu
2 swift run leetllm check 041 --solution
```

42.11 Bytes, bounds, and complexity

For lifetimes $i = 1 \dots n$,

$$B_{naive} = \sum_i b_i, \quad B_{peak} = \max_t \sum_{i: f_i \leq t \leq l_i} b_i.$$

B_{peak} is a lower bound without alignment and fragmentation. The produced arena may be larger. The straightforward planner filters, sorts, and merges prior live ranges for each buffer, so worst-case work is $O(n^2 \log n)$ and plan metadata is $O(n)$. Decoder schedules are small enough that predictability and auditable tie-breaking matter more than asymptotic optimization.

Persistent model weights and KV cache are not included in `naiveByteCount` or `arenaByteCount`. These values describe the listed transient lifetimes only.

42.12 Honest Metal mapping

Problem 041 is host-side planning and has no Metal kernel. In a GPU engine, the arena could be one `MTLBuffer`; each placement becomes an offset used to create views or passed into kernels. Alignment must satisfy both tensor representation and Metal API requirements. GPU command ordering, not Swift lexical scope, defines liveness.

Aliasing is unsafe across concurrently executing command buffers unless the lifetime model includes those overlaps. Heaps and hazard tracking introduce additional constraints. None of that is implemented here, and the report is not evidence that a Metal engine already uses the arena.

42.13 Implementation checkpoints

1. Validate names, intervals, sizes, alignment, and overflow.
2. Sort independently of input ordering.
3. Match the four-buffer first-fit fixture exactly.
4. Prevent reuse when lifetime endpoints touch.
5. Implement best-fit with deterministic tie-breaking.
6. Compute summaries with checked arithmetic.
7. Derive prefill and decode schedules from model dimensions.
8. Validate the final plan independently before returning it.

42.14 Controlled experiments

42.14.1 Sequence-length sweep

Run the report at $S=1, 16, 128$. Prediction: prefill tensor sizes and arena grow linearly. The one-row decode arena remains unchanged because persistent cache is outside this schedule.

42.14.2 Alignment sweep

Clone a fixture with alignments 4, 16, 64, 256. Prediction: requested/peak bytes stay fixed while holes and arena size can increase.

42.14.3 Lifetime extension

Extend `attention_norm` through the MLP. Prediction: at least one reuse assignment disappears and arena size may grow because a prior range remains live.

42.14.4 Strategy comparison

Construct fragmented gaps and compare first-fit with best-fit. Prediction: both plans validate and are deterministic; best-fit may reduce one fixture's arena but is not guaranteed globally optimal.

42.15 Engine integration

Problem 039 and 040 provide the dimensions and stage order used by these schedules. The next integration step would replace selected `[Float]` allocations in `MiniDecoderCPUEngine` with typed views into one aligned arena, then rerun all numerical tests and inspect actual allocation counts. Until that change exists, planner correctness and decoder allocation behavior are separate claims.

42.16 Tradeoffs and limitations

- Inclusive intervals are conservative and easy to audit; finer scheduling can expose more reuse.
- First-fit is predictable; best-fit can reduce local fragmentation but is not globally optimal.
- One arena removes allocation churn; it reserves peak capacity up front.
- Explicit offsets ease debugging; in-place operators could reduce memory further with stronger contracts.
- The schedule excludes weights, KV storage, attention score scratch, and concurrent requests.
- Arena planning is implemented; arena-backed tensor execution is not.

42.17 Hints

- Free a prior placement only when `lastOperation < firstOperation`.
- Merge live physical ranges before searching gaps.
- Align the candidate start, not only the final arena size.
- Keep best-fit's remainder and offset tie-break explicit.
- Validate a produced plan with a separate pairwise overlap pass.

42.18 Completion checklist

- ☐ Every lifetime field and arithmetic boundary validates.
- ☐ Input ordering cannot change either strategy's output.
- ☐ Simultaneously live buffers never overlap physically.
- ☐ Disjoint lifetimes can reuse all or part of a prior range.
- ☐ Alignment holes are reflected in actual offsets and arena size.
- ☐ Prefill and decode reports derive from the shared model.
- ☐ Persistent weights/cache are excluded explicitly.
- ☐ Experiments distinguish requested bytes, peak liveness, and arena bytes.
- ☐ No planner report is presented as an arena-backed decoder measurement.

42.19 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

42.19.1 Sources/LeetLLMExercises/P041BufferPlanningExercise.swift

```

1 import LeetLLMCore
2
3 public enum P041BufferPlanningExercise {
4     public static func plan(
5         lifetimes: [BufferLifetime],
6         strategy: ArenaAllocationStrategy
7     ) throws -> ArenaPlan {
8         try P041BufferPlanningContract.validate(lifetimes)
9         var offset = 0
10        var placements: [ArenaPlacement] = []
11        for lifetime in lifetimes.sorted(by: { $0.name < $1.name }) {
12            let remainder = offset % lifetime.alignment
13            if remainder != 0 { offset += lifetime.alignment - remainder }
14            placements.append(ArenaPlacement(lifetime: lifetime, offset: offset))
15            offset += lifetime.byteSize
16        }
17
18        // TODO: release ranges when lifetimes end and choose a deterministic aligned
19        // first-fit or best-fit gap instead of allocating every buffer sequentially.
20        return ArenaPlan(
21            strategy: strategy,
22            placements: placements,
23            arenaByteCount: offset,
24            peakLiveBytes: lifetimes.map(\.byteSize).reduce(0, +),
25            naiveByteCount: lifetimes.map(\.byteSize).reduce(0, +),
26            reuseAssignments: placements.map {
27                ArenaReuseAssignment(buffer: $0.name, reusesStorageFrom: [])
28            })
29    }
30 }

```

42.20 Worked solution

Solution strategy

Define lifetime intervals $[firstOperation, lastOperation]$ (inclusive-inclusive) for each intermediate buffer. Sort buffers by operation boundaries. For each buffer in order, identify all simultaneously-live buffers (those whose lifetimes overlap), extract their physical ranges, find gaps in the live ranges, and place the current buffer at the lowest aligned offset in a gap (first-fit) or in the gap with smallest remainder (best-fit). Compute peak live bytes as the maximum sum of requested sizes live at any single operation. Validate that no two overlapping-lifetime buffers share physical bytes.

Correctness invariant

Lifetime overlap is inclusive-inclusive: two lifetimes $[f_i, l_i]$ and $[f_j, l_j]$ overlap if $f_i \leq l_j$ and $f_j \leq l_i$. Physical overlap is half-open: ranges $[o_i, o_i + b_i)$ and $[o_j, o_j + b_j)$ overlap if $o_i < o_j + b_j$ and $o_j < o_i + b_i$. A valid plan requires: for every pair of buffers, if their lifetimes do not overlap, their physical ranges may overlap; if their lifetimes overlap, their physical ranges must not overlap. All alignments are power-of-two.

Common failure mode

Confusing lifetime overlap (temporal, at operation l_i) with physical overlap (spatial, byte ranges) leads to aliasing errors. Using exclusive operation endpoints $[f, l)$ instead of inclusive $[f, l]$ misses simultaneous liveness at operation l . Not accounting for alignment padding overstates reuse opportunity and produces invalid plans. Confusing first-fit and best-fit strategies or applying them inconsistently violates determinism.

42.20.1 Implementation points

Lifetime overlap and simultaneous liveness

Two buffers' lifetimes $[f_i, l_i]$ and $[f_j, l_j]$ overlap (are simultaneously live) if $f_i \leq l_j$ AND $f_j \leq l_i$. Inclusive endpoints mean operation l is live for a buffer with lifetime ending at l . Use this predicate to determine which buffers are live during the placement of each new buffer.

First-fit allocation: lowest-offset aligned gap

Scan the list of live ranges in sorted order. Find the first gap (either within the merged live ranges or beyond them) that fits the buffer when aligned. Place the buffer at the aligned offset. If no gap fits, extend the arena.

Best-fit allocation: smallest remainder

Among all gaps (merged live ranges and beyond) that can fit the aligned buffer, select the gap with the least remainder (available space minus buffer size). If multiple gaps have the same remainder, choose the lowest offset. Best-fit often reduces arena size for heterogeneous buffer sizes.

Alignment arithmetic and overflow checking

Next aligned offset from value x with power-of-two alignment a is $(x + a - 1) \& \sim(a - 1)$. Check for integer overflow when adding alignment padding $(a-1)$ to x . Peak live bytes is computed by iterating all unique operation points and summing the sizes of buffers live at each point.

42.20.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P041BufferPlanningSolution.swift

- Line 3: public enum P041BufferPlanningSolution {
- Line 4: public static func plan(
- Line 112: public static func compareDecoderPlans(
- Line 129: private static func merge(_ ranges: [(Int, Int)]) -> [(Int, Int)] {
- Line 141: private static func align(_ value: Int, to alignment: Int, name: String) throws -> Int {

42.20.3 Complete canonical listings

42.20.4 Sources/LeetLLMSolutions/P041BufferPlanningSolution.swift

```

1 import LeetLLMCore
2
3 public enum P041BufferPlanningSolution {
4     public static func plan(
5         lifetimes: [BufferLifetime],
6         strategy: ArenaAllocationStrategy
7     ) throws -> ArenaPlan {
8         try P041BufferPlanningContract.validate(lifetimes)
9         let ordered = lifetimes.sorted {
10             if $0.firstOperation != $1.firstOperation {
11                 return $0.firstOperation < $1.firstOperation
12             }
13             if $0.lastOperation != $1.lastOperation {
14                 return $0.lastOperation < $1.lastOperation
15             }
16             return $0.name < $1.name
17         }
18         var placements: [ArenaPlacement] = []
19         var arenaByteCount = 0
20
21         for lifetime in ordered {
22             let livePlacements = placements.filter {
23                 $0.lastOperation >= lifetime.firstOperation
24             }
25             var liveRanges: [(Int, Int)] = livePlacements.map { placement in
26                 (placement.offset, placement.offset + placement.byteSize)
27             }
28             liveRanges.sort { lhs, rhs in
29                 lhs.0 == rhs.0 ? lhs.1 < rhs.1 : lhs.0 < rhs.0
30             }
31             let merged = merge(liveRanges)
32             var candidates: [(offset: Int, available: Int)] = []
33             var cursor = 0
34             for range in merged {
35                 let aligned = try align(cursor, to: lifetime.alignment, name: lifetime.name)
36                 if aligned <= range.0, range.0 - aligned >= lifetime.byteSize {
37                     candidates.append((aligned, range.0 - aligned))
38                 }
39                 cursor = max(cursor, range.1)
40             }

```

```

41     let alignedCursor = try align(cursor, to: lifetime.alignment, name: lifetime.name)
42     if alignedCursor <= arenaByteCount,
43         arenaByteCount - alignedCursor >= lifetime.byteSize
44     {
45         candidates.append((alignedCursor, arenaByteCount - alignedCursor))
46     }
47     let offset: Int
48     if candidates.isEmpty {
49         offset = try align(arenaByteCount, to: lifetime.alignment, name: lifetime.name)
50     } else {
51         switch strategy {
52         case .firstFit:
53             offset = candidates.min { $0.offset < $1.offset }!.offset
54         case .bestFit:
55             offset = candidates.min {
56                 let leftRemainder = $0.available - lifetime.byteSize
57                 let rightRemainder = $1.available - lifetime.byteSize
58                 return leftRemainder == rightRemainder
59                     ? $0.offset < $1.offset
60                     : leftRemainder < rightRemainder
61             }!.offset
62         }
63     }
64     let (end, overflow) = offset.addingReportingOverflow(lifetime.byteSize)
65     guard !overflow else {
66         throw ArenaPlanningError.arithmeticOverflow(context: "placement of \(${lifetime.name}")
67     }
68     placements.append(ArenaPlacement(lifetime: lifetime, offset: offset))
69     arenaByteCount = max(arenaByteCount, end)
70 }
71
72 let naiveByteCount = try lifetimes.reduce(0) { partial, lifetime in
73     let (next, overflow) = partial.addingReportingOverflow(lifetime.byteSize)
74     guard !overflow else {
75         throw ArenaPlanningError.arithmeticOverflow(context: "naive byte count")
76     }
77     return next
78 }
79 let operations = Set(lifetimes.flatMap { [$0.firstOperation, $0.lastOperation] })
80 var peakLiveBytes = 0
81 for operation in operations {
82     let liveBytes = try lifetimes
83         .filter { $0.firstOperation <= operation && operation <= $0.lastOperation }
84         .reduce(0) { partial, lifetime in
85             let (next, overflow) = partial.addingReportingOverflow(lifetime.byteSize)
86             guard !overflow else {
87                 throw ArenaPlanningError.arithmeticOverflow(context: "peak live bytes")
88             }
89             return next
90         }
91     peakLiveBytes = max(peakLiveBytes, liveBytes)
92 }
93 let reuseAssignments = placements.enumerated().map { index, placement in
94     let reused = placements[..<index].filter { prior in
95         !prior.lifetime.overlapsLifetime(of: placement.lifetime)
96         && P041BufferPlanningContract.rangesOverlap(
97             prior.offset, prior.byteSize, placement.offset, placement.byteSize)
98     }.map(\.name).sorted()
99     return ArenaReuseAssignment(buffer: placement.name, reusesStorageFrom: reused)
100 }
101 let plan = ArenaPlan(
102     strategy: strategy,
103     placements: placements,
104     arenaByteCount: arenaByteCount,
105     peakLiveBytes: peakLiveBytes,
106     naiveByteCount: naiveByteCount,
107     reuseAssignments: reuseAssignments)
108 try P041BufferPlanningContract.validate(plan: plan, for: lifetimes)
109 return plan
110 }
111
112 public static func compareDecoderPlans(
113     model: MiniDecoderModel,

```

```

114     prefillTokenCount: Int,
115     cachedTokenCount: Int,
116     strategy: ArenaAllocationStrategy = .firstFit
117 ) throws -> DecoderArenaComparison {
118     DecoderArenaComparison(
119         prefill: try plan(
120             lifetimes: MiniDecoderBufferSchedules.prefill(
121                 model: model, tokenCount: prefillTokenCount),
122                 strategy: strategy),
123         decode: try plan(
124             lifetimes: MiniDecoderBufferSchedules.decode(
125                 model: model, cachedTokenCount: cachedTokenCount),
126                 strategy: strategy))
127     }
128
129     private static func merge(_ ranges: [(Int, Int)]) -> [(Int, Int)] {
130         var merged: [(Int, Int)] = []
131         for range in ranges {
132             guard let last = merged.last, range.0 <= last.1 else {
133                 merged.append(range)
134                 continue
135             }
136             merged[merged.count - 1].1 = max(last.1, range.1)
137         }
138         return merged
139     }
140
141     private static func align(_ value: Int, to alignment: Int, name: String) throws -> Int {
142         let mask = alignment - 1
143         let (sum, overflow) = value.addingReportingOverflow(mask)
144         guard !overflow else {
145             throw ArenaPlanningError.arithmeticOverflow(context: "alignment of \$(name)")
146         }
147         return sum & ~mask
148     }
149 }

```

Checkpoint Parity and First Divergence

43.1 Why this exists

Final logits are a poor first debugging boundary. A wrong RMSNorm convention or RoPE position can propagate through every later layer while still producing finite logits and a plausible token. Comparing named intermediate tensors in semantic order answers the useful question: where did two implementations first stop computing the same model?

This lesson serializes captures from an independent Double-accumulation oracle for the deterministic educational mini-model, runs the Float32 candidate engine, and reports the first failing capture. The bundled artifact generator is not a production checkpoint or third-party runtime. It establishes that the parity machinery and fault localization work on known data; it does not establish real-model parity.

43.2 Learning outcomes

You can:

- define stable names and shapes at model semantic boundaries;
- bind a reference artifact to exact model, prompt, and position identities;
- compute maximum absolute error, RMSE, cosine similarity, and argmax agreement;
- apply combined absolute/relative elementwise tolerance;
- find the first divergence in reference capture order;
- localize deliberate RoPE and RMSNorm convention faults; and
- replace the educational artifact with captures from a trusted external implementation.

43.3 Prerequisites

- Problem 034 for metric interpretation and convention-before-quantization diagnosis.
- Problems 035 and 039 for complete block/prefill capture boundaries.
- Problem 036 for the difference between validated bytes and model-level truth.

43.4 Vocabulary

- **Reference artifact:** versioned JSON containing provenance, identity, tensors, and selected token.
- **Candidate:** implementation being checked against the artifact.
- **Provenance:** human-readable description of who generated reference values.
- **Fingerprint:** deterministic identity of the exact educational model configuration and weights.
- **Capture order:** serialized semantic order used to define first divergence.
- **First divergence:** first capture whose shape or values fail tolerance.

- **Convention fault:** coherent but wrong rule such as shifted RoPE positions or additive RMSNorm gamma.

43.5 Capture schema and order

Artifact format version 1 contains:

```
1 formatVersion
2 provenance
3 modelFingerprint
4 tokenIDs
5 positionOffset
6 captures[] = { name, shape, values }
7 selectedTokenID
```

JSON keys are encoded in sorted order for deterministic bytes. Capture names must be unique and nonempty. Dimensions must be nonnegative, their checked product must equal `values.count`, and every Float value must be finite.

Capture order is:

```
1 embeddings
2 layer.0.residual_input
3 layer.0.attention_norm
4 layer.0.query
5 layer.0.key
6 layer.0.value
7 layer.0.rope.query
8 layer.0.rope.key
9 layer.0.attention
10 layer.0.attention_output
11 layer.0.post_attention
12 layer.0.mlp_norm
13 layer.0.mlp.gate
14 layer.0.mlp.up
15 layer.0.mlp.gated
16 layer.0.output
17 ...same names for each later layer...
18 final_norm
19 logits
20 selected_token
```

This order is part of diagnosis. A dictionary can locate candidate tensors, but the report iterates reference captures in artifact order.

43.6 Math: error metrics and tolerance

For reference r_i and candidate c_i with equal shapes:

$$E_{max} = \max_i |c_i - r_i|,$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_i (c_i - r_i)^2},$$

$$\cos(r, c) = \frac{r \cdot c}{\|r\|_2 \|c\|_2}.$$

Both zero vectors have cosine 1; one zero and one nonzero vector have cosine 0. Element i passes when

$$|c_i - r_i| \leq \epsilon_{abs} + \epsilon_{rel}|r_i|.$$

Default tolerances are 8e-5 absolute and 1.5e-4 relative. Both must be finite and nonnegative. A shape mismatch fails immediately and reports numerical metrics as absent. Argmax agreement is reported for logits and selected_token; token agreement also appears at report level. Metrics describe differences; the elementwise tolerance determines pass/fail.

43.7 Worked fault localization

The judge generates a reference for the default prompt at nonzero offset 4. Three candidates are run:

1. Correct Float32 candidate: every capture passes and selected token matches.
2. RoPE candidate using positionOffset+1: embeddings, norms, and projected Q/K/V pass; first divergence is `layer.0.rope.query`.
3. Candidate using (1+gamma) RMSNorm instead of direct gamma: embeddings and residual input pass; first divergence is `layer.0.attention_norm`.

Both faults eventually change later states. Reporting only logits would lose the evidence that distinguishes a positional convention from normalization.

43.8 Identity and rejection contract

Before candidate execution, the request validates:

- the complete model and token IDs;
- nonnegative position offset;
- finite nonnegative tolerances;
- supported artifact version;
- artifact tensor names, shapes, counts, and finite values;
- exact model fingerprint; and
- exact token IDs and position offset.

A stale artifact for another weight set, a capture from another prompt, or a different absolute position is rejected rather than compared. Missing candidate captures and unexpected candidate names are errors, not silently skipped rows.

The fingerprint is an identity guard, not a cryptographic signature or a proof of provenance. Provenance is retained in the report but is not automatically trusted merely because it is a string.

43.9 Correctness method: independent educational reference

`P042EducationalReference` runs `MiniDecoderReference.prefill`, the separately implemented Double-accumulation oracle from Problem 039. It greedily selects a token and converts all named tensors to Float artifact values. Its provenance is explicitly:

```
1 educational-mini-model fixture generated by LeetLLM's independent Double CPU oracle
```

The candidate path independently executes Float32 block operations. The two paths share model data and contracts but do not call the same block implementation. Deterministic JSON round-trip is tested byte for byte.

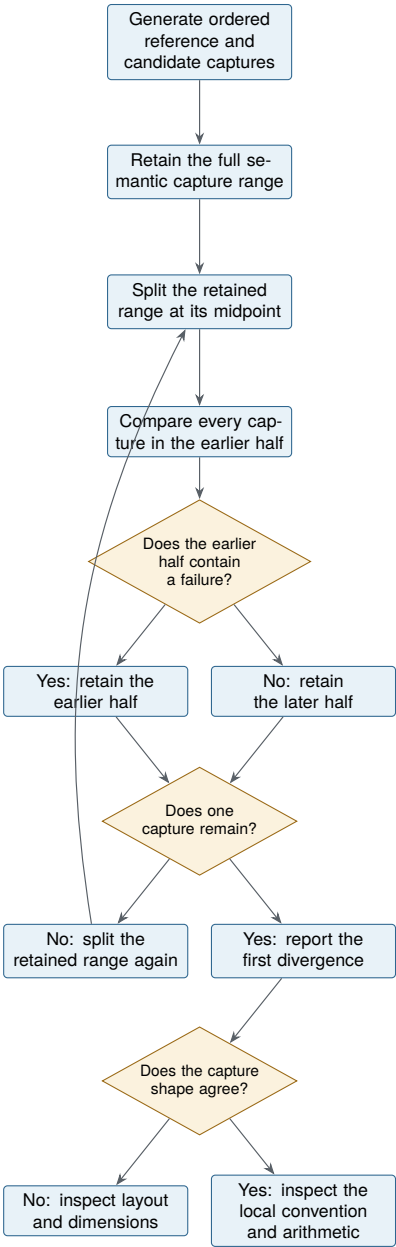


Figure 43.1: Checkpoint bisection.

```
1 swift run leetllm check 042 --cpu
2 swift run leetllm check 042 --solution
```

43.10 External reference replacement protocol

To turn this machinery into external parity evidence, replace only artifact generation, not comparison:

1. **Freeze identity.** Export the exact model configuration and tensor values consumed by MiniDecoder-Model. Record its modelFingerprint. For an actual production checkpoint, first extend the loader/model contract beyond P036's one-block educational archive; do not reuse a fingerprint from different data.
2. **Freeze input.** Record exact integer token IDs and absolute position offset. Tokenizer text alone is insufficient evidence.
3. **Implement hooks in a trusted runtime.** Capture the tensors at the names and semantic boundaries above, after matching [out,in] orientation, direct-gamma RMSNorm, adjacent-pair RoPE, causal mask, and integer-division GQA.
4. **Normalize layout.** Export each tensor as logical row-major Float values with its explicit shape. Transpose framework-native views before serialization when necessary; never rely on equal square dimensions to hide orientation.
5. **Emit format version 1 JSON.** Include nonempty provenance naming the runtime, version/commit, model source, conversion tool, and capture command. Include all captures exactly once and in semantic order.
6. **Bind and validate.** Decode with expectedModelFingerprint, then compare with identical token IDs and offset. Archive the artifact alongside the exact conversion/runtime metadata needed to regenerate it.
7. **Investigate the first failure.** Check shape/layout first, then the local convention at that boundary. Do not increase tolerance until the discrepancy has a measured numerical explanation.

The current repository does not ship such an external artifact, framework adapter, production checkpoint, or license for model weights. Until those steps are performed, say “educational independent-oracle parity,” not “real-model parity.”

43.11 Performance model: work, bytes, and diagnostic cost

For every captured Float tensor of N_k elements, artifact payload is at least

$$4 \sum_k N_k$$

raw value bytes before JSON number expansion, names, shapes, and structure. Comparison is linear in total captured elements and stores the complete candidate captures plus per-capture summaries. JSON is intentionally inspectable but substantially larger and slower than a binary tensor container.

Capture mode can dominate memory relative to normal inference because values that would die after one consumer remain retained. This is diagnostic cost. It must not be mixed into production latency or peak-memory claims.

43.12 Honest Metal mapping

Problem 042 is CPU-only and has no Metal check. A Metal candidate can use this artifact format, but diagnostic mode must copy named device tensors back only after the producing command completes. Buffer aliasing from Problem 041 means a capture must occur before later operations overwrite its range.

GPU mixed precision and reduction order may require evidence-based tolerances by capture class. The same first-divergence procedure still applies. No CPU result is labeled a GPU parity result, and no tolerance is presented as universal.

43.13 Implementation checkpoints

1. Validate version, fingerprint, prompt identity, names, shapes, values, and tolerances.
2. Produce candidate captures with exact stable names.
3. Reject missing and unexpected captures.
4. Fail shape mismatches before elementwise metrics.
5. Compute metrics in Double and elementwise combined tolerance.
6. Preserve artifact order and return its first failing name.
7. Match selected token separately from floating metrics.
8. Prove the two deliberate faults localize at their originating boundaries.

43.14 Controlled experiments

43.14.1 Tolerance sweep

Compare exact, default, and much looser tolerances. Prediction: exact comparison may expose Float32/Double accumulation differences; default passes the correct candidate; loose thresholds can hide deliberate faults and are therefore not a fix.

43.14.2 Position identity intervention

Keep values and token IDs but change only request offset. Prediction: the artifact is rejected before comparison, preventing misleading RoPE metrics.

43.14.3 Capture removal

Remove `layer.0.rope.query` from a candidate. Prediction: comparison throws a missing-capture error; it does not relabel the next mismatch as first divergence.

43.14.4 Near-tie logits

Perturb logits within tolerance around two close maxima. Prediction: floating metrics can pass while argmax/token agreement changes, demonstrating why both numerical and discrete fields are retained.

43.15 Engine integration

The capture vocabulary follows the same engine from embeddings through every Problem 035 block, final norm, logits, and token selection. Problem 039 supplies the independent oracle and complete model trace. A future optimized prefill or decode path should run against the same artifact before and after fusion, quantization, or arena integration. First divergence then identifies the optimization boundary that changed semantics.

43.16 Tradeoffs and limitations

- Dense captures localize faults well and cost substantial memory and I/O.

- JSON is portable and reviewable; binary tensor formats are more compact.
- One global tolerance is simple; real mixed-precision models may need justified per-capture policies.
- Fingerprints prevent accidental model mismatch; they do not authenticate provenance.
- Greedy token agreement catches discrete changes; stochastic parity also requires sampler state and policy.
- The bundled evidence covers only the deterministic educational mini-model.

43.17 Hints

- Compare in semantic order, not alphabetic order.
- Treat a shape mismatch as a contract failure, not a large numerical error.
- Keep Float storage but accumulate metrics in Double.
- Check the earliest failing boundary before inspecting later logits.
- Preserve exact token IDs and absolute positions with every artifact.

43.18 Completion checklist

- ☐ Artifact bytes round-trip deterministically and validate every tensor.
- ☐ Model fingerprint, token IDs, and offset bind reference to request.
- ☐ Candidate names/shapes exactly match the ordered artifact.
- ☐ Max error, RMSE, cosine, tolerance, and argmax fields are correct.
- ☐ Correct Float32 execution passes the independent Double artifact.
- ☐ Shifted RoPE first diverges at `layer.0.rope.query`.
- ☐ Additive-gamma RMSNorm first diverges at `layer.0.attention_norm`.
- ☐ External-reference provenance and conversion steps are recorded.
- ☐ Educational parity is never described as production real-model parity.

43.19 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

43.19.1 Sources/LeetLLMExercises/P042CheckpointParityExercise.swift

```
1 import LeetLLMCore
2
3 public enum P042CheckpointParityExercise {
4     public static func compare(
5         _ request: CheckpointParityRequest
6     ) throws -> CheckpointParityReport {
7         let artifact = try P042CheckpointParityContract.validate(request)
8
9         // TODO: execute the complete mini-model with named captures, compare every
10        // capture in artifact order, and report the first boundary outside tolerance.
11        return CheckpointParityReport(
12            modelFingerprint: request.model.fingerprint,
13            artifactProvenance: artifact.provenance,
14            comparisons: [],
15            firstDivergentCapture: nil,
16            referenceSelectedTokenID: artifact.selectedTokenID,
17            candidateSelectedTokenID: artifact.selectedTokenID,
18            selectedTokenMatches: true,
19            isPassing: false)
20    }
21 }
```


43.20 Worked solution

Solution strategy

Create or load a reference artifact containing model fingerprint, token IDs, position offset, and a deterministically-ordered sequence of capture tensors at semantic boundaries (embeddings, per-layer residual inputs, norms, projections, attention outputs, FFN branches, residuals). Run the candidate implementation and extract the same capture tensors in the same order. Compare shapes (must match exactly) and values (using absolute and relative error tolerances combined). Report the first capture whose shape or values fail tolerance. Provide optional fault injection (RoPE position offset ± 1 , additive RMSNorm gamma) to localize convention errors.

Correctness invariant

Artifact capture order is deterministic: embeddings, layer.0.residual_input, layer.0.attention_norm, layer.0.query, ..., layer.L.output, final_norm, logits, selected_token. Candidate must produce all reference captures with matching shapes. Comparison metrics (max absolute error, RMSE, cosine similarity, argmax agreement) are applied in the order they appear in captures. First divergence halts the comparison chain; later divergences may be cascades. Fingerprint and selected token must match exactly.

Common failure mode

Comparing only final logits masks layer-level convention errors (e.g., RoPE shifted by 1 position, additive gamma instead of multiplicative). Using a single loose tolerance instead of combined absolute/relative tolerance misses subtle errors in near-zero regions. Not capturing layer intermediates makes first-divergence attribution impossible (divergence appears only at final logits). Injecting faults globally instead of isolating them to one block prevents root-cause diagnosis.

43.20.1 Implementation points

Artifact format and capture schema

JSON with formatVersion, provenance (human-readable description), modelFingerprint (deterministic hash of configuration), tokenIDs, positionOffset, captures (array of {name, shape, values}), selectedTokenID. Capture names are unique and nonempty. All float values must be finite. Shape dimensions are nonnegative; product checked against Int.max.

Capture order for first-divergence attribution

Order is: embeddings, then for each layer (residual_input, attention_norm, query, key, value, rope.query, rope.key, attention, attention_output, post_attention, mlp_norm, mlp.gate, mlp.up, mlp.gated, output), then final_norm, logits, selected_token. This order enables layer-by-layer fault localization.

Combined absolute and relative error tolerance

For reference r_i and candidate c_i : $\text{max_abs_error} = \max(|c_i - r_i|)$, passes if $\text{max_abs_error} \leq \text{abs_tol}$ OR $\text{max_abs_error} \leq \text{rel_tol} * |r_i|$ (per-element). This handles both near-zero values

(where relative tolerance is ineffective) and far-from-zero values (where absolute tolerance alone is loose).

Fault injection for convention diagnosis

ropePositionOffset fault: apply RoPE at position + 1 instead of position (test whether positions are off-by-one). additiveRMSNormGamma fault: use scale = gamma + 1 instead of gamma in norm formula (test whether gamma is multiplicative or additive). Inject one fault per block to isolate its effect.

43.20.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P042CheckpointParitySolution.swift

- Line 4: public enum P042CheckpointParitySolution {
- Line 5: public static func compare(
- Line 43: private static func candidateCaptures(
- Line 117: private static func faultedBlock(
- Line 193: private static func groupedCausalAttention(
- Line 233: private static func compare(

43.20.3 Complete canonical listings

43.20.4 Sources/LeetLLMSolutions/P042CheckpointParitySolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P042CheckpointParitySolution {
5     public static func compare(
6         _ request: CheckpointParityRequest
7     ) throws -> CheckpointParityReport {
8         let artifact = try P042CheckpointParityContract.validate(request)
9         let candidate = try candidateCaptures(request)
10        let candidateByName = Dictionary(
11            uniqueKeysWithValues: candidate.captures.map { ($0.name, $0.tensor) })
12        let referenceNames = Set(artifact.captures.map(\.name))
13        if let unexpected = candidate.captures.first(where: {
14            !referenceNames.contains($0.name)
15        }) {
16            throw CheckpointParityError.unexpectedCapture(unexpected.name)
17        }
18
19        var comparisons: [CaptureComparison] = []
20        for reference in artifact.captures {
21            guard let tensor = candidateByName[reference.name] else {
22                throw CheckpointParityError.missingCapture(reference.name)
23            }
24            comparisons.append(compare(
25                reference: reference,
26                candidate: tensor,
27                absoluteTolerance: request.absoluteTolerance,
28                relativeTolerance: request.relativeTolerance))
29        }
30        let firstDivergence = comparisons.first(where: { !$0.passesTolerance })?.name
31        let selectedTokenMatches = artifact.selectedTokenID == candidate.selectedTokenID

```

```

32     return CheckpointParityReport(
33         modelFingerprint: request.model.fingerprint,
34         artifactProvenance: artifact.provenance,
35         comparisons: comparisons,
36         firstDivergentCapture: firstDivergence,
37         referenceSelectedTokenID: artifact.selectedTokenID,
38         candidateSelectedTokenID: candidate.selectedTokenID,
39         selectedTokenMatches: selectedTokenMatches,
40         isPassing: firstDivergence == nil && selectedTokenMatches)
41     }
42
43     private static func candidateCaptures(
44         _ request: CheckpointParityRequest
45     ) throws -> MiniDecoderCaptureSet {
46         let model = request.model
47         let sequence = request.tokenIDs.count
48         let dimension = model.configuration.modelDimension
49         var embedded: [Float] = []
50         for tokenID in request.tokenIDs {
51             let start = tokenID * dimension
52             embedded.append(contentsOf: model.tokenEmbedding.storage[start..

```

```

105     repeating: Array(request.positionOffset..<(request.positionOffset + sequence)),
106     count: model.layerCount),
107     work: try MiniDecoderCPUEngine.workModel(
108       model: model, tokenCount: sequence, decode: false))
109   return try MiniDecoderCaptureSet.fromPrefill(
110     result,
111     model: model,
112     tokenIDs: request.tokenIDs,
113     positionOffset: request.positionOffset,
114     selectedTokenID: selected)
115 }
116
117 private static func faultedBlock(
118   residual: FloatTensor,
119   positionOffset: Int,
120   weights: DecoderBlockWeights,
121   configuration: DecoderConfiguration,
122   fault: MiniDecoderParityFault
123 ) throws -> DecoderBlockResult {
124   let sequence = residual.shape[0]
125   let dimension = configuration.modelDimension
126   let attentionNormalized = try MiniDecoderCPUEngine.normalize(
127     residual,
128     gamma: weights.attentionNormGamma,
129     epsilon: configuration.rmsNormEpsilon,
130     additiveGamma: fault == .additiveRMSNormGamma)
131   let queryProjection = try MiniDecoderCPUEngine.project(
132     attentionNormalized, weights: weights.queryWeights)
133   let keyProjection = try MiniDecoderCPUEngine.project(
134     attentionNormalized, weights: weights.keyWeights)
135   let valueProjection = try MiniDecoderCPUEngine.project(
136     attentionNormalized, weights: weights.valueWeights)
137   let queries = try FloatTensor(
138     queryProjection.storage,
139     shape: [sequence, configuration.queryHeadCount, configuration.headDimension])
140   let keys = try FloatTensor(
141     keyProjection.storage,
142     shape: [sequence, configuration.keyValueHeadCount, configuration.headDimension])
143   let values = try FloatTensor(
144     valueProjection.storage,
145     shape: [sequence, configuration.keyValueHeadCount, configuration.headDimension])
146   let ropeOffset = positionOffset + (fault == .ropePositionOffset ? 1 : 0)
147   let rotatedQueries = try MiniDecoderCPUEngine.rotate(
148     queries, positionOffset: ropeOffset, configuration: configuration)
149   let rotatedKeys = try MiniDecoderCPUEngine.rotate(
150     keys, positionOffset: ropeOffset, configuration: configuration)
151   let attentionHeads = try groupedCausalAttention(
152     queries: rotatedQueries,
153     keys: rotatedKeys,
154     values: values,
155     configuration: configuration)
156   let concatenated = try FloatTensor(attentionHeads.storage, shape: [sequence, dimension])
157   let attentionProjection = try MiniDecoderCPUEngine.project(
158     concatenated, weights: weights.attentionOutputWeights)
159   let postAttention = try MiniDecoderCPUEngine.add(residual, attentionProjection)
160   let mlpNormalized = try MiniDecoderCPUEngine.normalize(
161     postAttention,
162     gamma: weights.mlpNormGamma,
163     epsilon: configuration.rmsNormEpsilon,
164     additiveGamma: fault == .additiveRMSNormGamma)
165   let gate = try MiniDecoderCPUEngine.project(mlpNormalized, weights: weights.gateWeights)
166   let up = try MiniDecoderCPUEngine.project(mlpNormalized, weights: weights.upWeights)
167   let activated = try FloatTensor(
168     gate.storage.map { $0 / (1 + exp(-$0)) }, shape: gate.shape)
169   let gated = try FloatTensor(zip(activated.storage, up.storage).map(*), shape: gate.shape)
170   let down = try MiniDecoderCPUEngine.project(gated, weights: weights.downWeights)
171   let output = try MiniDecoderCPUEngine.add(postAttention, down)
172   return DecoderBlockResult(
173     state: DecoderBlockState(residual: output, positionOffset: positionOffset),
174     intermediates: DecoderBlockIntermediates(
175       attentionNormalized: attentionNormalized,
176       queries: queries,
177       keys: keys,

```

```

178     values: values,
179     rotatedQueries: rotatedQueries,
180     rotatedKeys: rotatedKeys,
181     attentionHeads: attentionHeads,
182     concatenatedAttention: concatenated,
183     attentionProjection: attentionProjection,
184     postAttentionResidual: postAttention,
185     mlpNormalized: mlpNormalized,
186     gateProjection: gate,
187     upProjection: up,
188     activatedGate: activated,
189     gatedHidden: gated,
190     downProjection: down))
191 }
192
193 private static func groupedCausalAttention(
194     queries: FloatTensor,
195     keys: FloatTensor,
196     values: FloatTensor,
197     configuration: DecoderConfiguration
198 ) throws -> FloatTensor {
199     let sequence = queries.shape[0]
200     let headDimension = configuration.headDimension
201     let groupSize = configuration.queryHeadCount / configuration.keyValueHeadCount
202     let scale = 1 / sqrt(Float(headDimension))
203     var output = Array(repeating: Float.zero, count: queries.elementCount)
204     for query in 0..

```

```

251 var squareErrorSum = 0.0
252 var dot = 0.0
253 var referenceNorm = 0.0
254 var candidateNorm = 0.0
255 var passes = true
256 for index in reference.values.indices {
257     let expected = Double(reference.values[index])
258     let actual = Double(candidate.storage[index])
259     let error = abs(actual - expected)
260     maximumError = max(maximumError, error)
261     squareErrorSum += error * error
262     dot += expected * actual
263     referenceNorm += expected * expected
264     candidateNorm += actual * actual
265     let allowed = Double(absoluteTolerance) + Double(relativeTolerance) * abs(expected)
266     if error > allowed { passes = false }
267 }
268 let count = max(reference.values.count, 1)
269 let cosine: Double
270 if referenceNorm == 0, candidateNorm == 0 {
271     cosine = 1
272 } else if referenceNorm == 0 || candidateNorm == 0 {
273     cosine = 0
274 } else {
275     cosine = dot / sqrt(referenceNorm * candidateNorm)
276 }
277 let shouldCompareArgmax = reference.name == "logits" || reference.name == "selected_token"
278 let referenceArgmax = reference.values.indices.dropFirst().reduce(0) {
279     reference.values[$1] > reference.values[$0] ? $1 : $0
280 }
281 let candidateArgmax = candidate.storage.indices.dropFirst().reduce(0) {
282     candidate.storage[$1] > candidate.storage[$0] ? $1 : $0
283 }
284 return CaptureComparison(
285     name: reference.name,
286     referenceShape: reference.shape,
287     candidateShape: candidate.shape,
288     maximumAbsoluteError: maximumError,
289     rootMeanSquareError: sqrt(squareErrorSum / Double(count)),
290     cosineSimilarity: cosine,
291     argmaxMatches: shouldCompareArgmax ? referenceArgmax == candidateArgmax : nil,
292     passesTolerance: passes)
293 }
294 }

```

Part VIII

Tradeoffs from Evidence

Fuse RMSNorm and Q/K/V Projections

44.1 Why this exists

Every decoder attention block normalizes one residual tensor and projects it into queries, keys, and values. A literal implementation launches RMSNorm, writes the normalized tensor, then launches three projection kernels that read it again. The arithmetic is valid, but the boundary creates an intermediate allocation, four dispatches, and extra memory traffic.

This problem fuses that subgraph. The CPU path remains readable; the Metal path performs one RMS reduction and all three projections in one dispatch. The benchmark reports measured wall-clock time separately from modeled logical traffic.

44.2 Learning outcomes

You can:

- derive fused RMSNorm and Q/K/V without changing model semantics;
- state the [output, input] projection convention used by the decoder;
- identify the intermediate and dispatches removed by fusion;
- map one token to one Metal threadgroup with a shared reduction;
- account for allocation, host-to-device, device-to-host, command-buffer, and wait costs; and
- reject a fusion claim when parity or measured evidence does not support it.

44.3 Prerequisites

- Problem 010: direct-gamma RMSNorm and epsilon placement.
- Problem 012: fusion across normalization and projection.
- Problem 014: query-head and KV-head output shapes.
- Problem 035: the decoder block's [out, in] weights and pre-norm ordering.

44.4 Vocabulary

- **Subgraph fusion:** executing several connected operators without materializing every logical boundary.
- **Intermediate:** the normalized [S,D] tensor written by the separate path.
- **Logical traffic:** bytes implied by tensor reads and writes, independent of cache behavior.
- **Dispatch:** one encoded kernel invocation.
- **End-to-end Metal time:** host time including buffer work, submission, and synchronization.
- **GQA projection width:** query width $H_q * d_h$ versus key/value width $H_{kv} * d_h$.

44.5 Math from first principles

For token row $x_t \in \mathbb{R}^D$ and learned scale γ ,

$$r_t = \sqrt{\frac{1}{D} \sum_{j=0}^{D-1} x_{t,j}^2 + \epsilon}, \quad n_{t,j} = \frac{x_{t,j}}{r_t} \gamma_j.$$

With row-major weights $W_Q \in \mathbb{R}^{Q \times D}$ and $W_K, W_V \in \mathbb{R}^{K \times D}$,

$$q_{t,o} = \sum_j W_{Q,o,j} n_{t,j}, \quad k_{t,o} = \sum_j W_{K,o,j} n_{t,j}, \quad v_{t,o} = \sum_j W_{V,o,j} n_{t,j}.$$

Fusion substitutes the definition of n inside each projection. It does not change gamma to 1+gamma, transpose weights, or merge the three learned matrices semantically.

For $x = [3, 4]$, $\gamma = [1, 0.5]$, and $\epsilon = 0$,

$$r = \sqrt{(9 + 16)/2} = \sqrt{12.5} \approx 3.5355,$$

so $n \approx [0.8485, 0.5657]$. If one query row is $[2, -1]$, its output is approximately $2(0.8485) - 0.5657 = 1.1313$. The fused path must produce the same value without storing n .

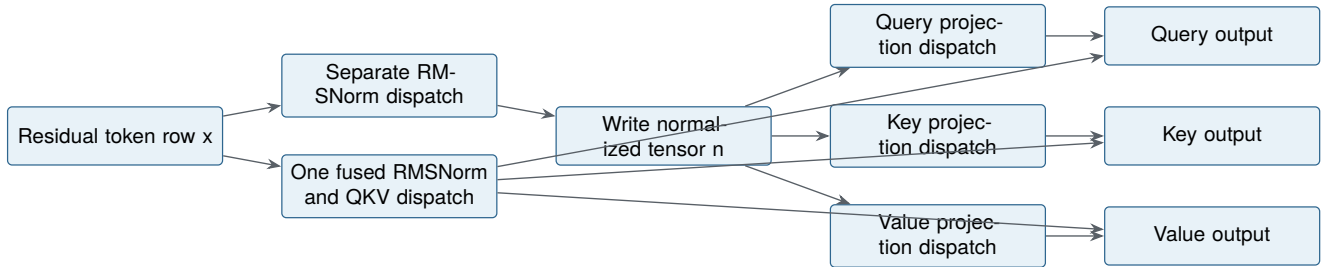


Figure 44.1: Fused qkv dataflow.

44.6 Shape, layout, and dtype contract

- Input: contiguous row-major Float32 $[S, D]$, with $1 \leq S \leq 64$ and $D \leq 256$.
- Gamma: contiguous Float32 $[D]$.
- Query weights: contiguous Float32 $[H_q \times d_h, D]$.
- Key/value weights: contiguous Float32 $[H_{kv} \times d_h, D]$.
- Outputs: query $[S, H_q, d_h]$; key and value $[S, H_{kv}, d_h]$.
- Combined projection width must not exceed 768.
- Epsilon must be finite and positive; every input and weight must be finite.
- Float32 accumulation is the learner and Metal contract. The judge uses an independent Double oracle and combined tolerance.

44.7 CPU reference path

`P043FusedQKVSolution.separate` makes the normalized tensor explicit, then projects it three times. `fused` computes normalized values while accumulating projection outputs and never allocates $[S, D]$ normaliza-

tion storage. Both are useful: the separate path explains semantics; the fused path exposes the removed boundary.

```
1 swift run leetllm check 043 --cpu --solution
```

44.8 Correctness method

The judge independently computes RMSNorm and each projection in Double. It covers GQA shapes, the maximum sequence and projection boundaries, nonuniform gamma, invalid ranks and shapes, zero epsilon, empty sequences, and non-finite values.

An element passes when its absolute error is bounded by $5e-5 + 1e-4 * \max(|actual|, |expected|)$. CPU and Metal implementations use the same judge.

```
1 swift run leetllm check 043 --solution
```

44.9 Performance model

Let $P = H_q d_h + 2H_{kv} d_h$. Projection work dominates at approximately

$$2SDP \text{ FLOPs}$$

when a multiply-add counts as two operations, plus the $O(SD)$ normalization reduction and scaling.

The bundled cost model counts four separate dispatches versus one fused dispatch. The separate path allocates $4 * S * D$ bytes for normalized Float32 values. Its logical traffic includes writing that tensor once and reading it for three projections; the fused path removes those four normalized-tensor transfers.

This is a traffic model, not a cache-counter measurement. Run:

```
1 swift run -c release leetllm benchmark 043 --tokens 32 --iterations 20
```

The Metal timing is end-to-end and includes shared-buffer allocation, copies, submission, and a host wait.

44.10 Metal mapping

The canonical kernel launches one 256-thread threadgroup per token. Each thread accumulates a strided subset of squared features into threadgroup memory. A tree reduction computes the row sum of squares; every thread reaches every barrier. Threads then stride over the combined Q/K/V output channels.

- Grid: 5 threadgroups.
- Threadgroup width: 256.
- Threadgroup memory: 256 Float32 partial sums.
- Device memory: input, gamma, three weight matrices, and three outputs.
- Dispatches: one.
- Bounds: host validation caps dimensions; feature and channel loops are strided.
- Synchronization: reduction barriers inside a threadgroup, then one host wait after the command buffer.

The starter kernel reaches the reduction but deliberately returns incorrect projection values until completed.

44.11 Implementation checkpoints

1. Validate every shape, finite value, and limit before dispatch.
2. Implement the separate Float32 path and pass the oracle.
3. Remove the normalized allocation in the CPU fused path.
4. Map the RMS reduction to one threadgroup per token.
5. Route combined channels to the correct Q, K, or V weight base.
6. Reshape flat outputs into head views without copying.
7. Match the independent oracle and inspect resource accounting.
8. Benchmark only after writing a prediction.

44.12 Controlled experiments

44.12.1 Sequence sweep

Run $S=1, 2, 8, 32, 64$ with fixed dimensions. Predict where launch and transfer overhead dominate and where removed traffic becomes visible.

44.12.2 Head-sharing sweep

Compare MHA and GQA configurations with the same query width but fewer KV heads. Predict how P , output bytes, and weight bytes change.

44.12.3 Boundary intervention

Compare CPU separate, CPU fused, and Metal end-to-end. Do not assume the CPU fused path is faster: recomputation and Swift loop structure can outweigh one removed allocation.

Record build configuration, machine, shape, median iterations, and whether each number is modeled or measured.

44.13 Engine integration

Problem 047 invokes the fused QKV Metal kernel on the real layer-0 residual captured from capstone prefill, then applies RoPE and compares named values with CPU. Generation remains on the shared CPU engine; this kernel is a verified subgraph, not a complete GPU decoder.

44.14 Tradeoffs and limitations

- Fusion saves traffic but couples operators and complicates debugging.
- One threadgroup per token is educational, not a universal production mapping.
- Large dimensions require a different reduction or tiling design.
- The kernel reads normalized input values again for each output channel; matrix tiling could improve reuse.
- End-to-end shared-buffer timing is not kernel-only GPU time.
- A modeled byte reduction does not prove a wall-clock speedup.

44.15 Hints

- Compute inverse RMS once per token in Metal.

- Keep all threads participating in reduction barriers.
- Flatten Q, K, and V into one channel interval, then subtract the appropriate offset.
- Use decoder weights as [output, input]; square shapes do not make orientation irrelevant.
- Compare the first failing tensor before changing tolerance.

44.16 Completion checklist

- ☐ Separate and fused CPU values match the independent oracle.
- ☐ Metal Q, K, and V pass the same tolerance.
- ☐ Invalid shape and non-finite cases are rejected.
- ☐ The normalized intermediate and removed dispatches are derived in bytes.
- ☐ A release benchmark records shape, machine, and timing boundary.
- ☐ The result distinguishes modeled traffic from measured wall time.
- ☐ The learner can explain why this is one verified subgraph, not a full Metal engine.

44.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

44.17.1 Sources/LeetLLMExercises/P043FusedQKVExercise.swift

```
1 import LeetLLMCore
2
3 public enum P043FusedQKVExercise {
4     public static func fused(_ request: FusedQKVRequest) throws -> FusedQKVResult {
5         try P043FusedQKVContract.validate(request)
6         let sequence = request.input.shape[0]
7         let headDimension = request.configuration.headDimension
8         // TODO: compute RMSNorm and all three projections without allocating a normalized tensor.
9         return FusedQKVResult(
10             queries: try FloatTensor(
11                 Array(repeating: 0, count: sequence * request.configuration.queryProjectionDimension),
12                 shape: [sequence, request.configuration.queryHeadCount, headDimension]),
13             keys: try FloatTensor(
14                 Array(repeating: 0, count: sequence * request.configuration.keyValueProjectionDimension),
15                 shape: [sequence, request.configuration.keyValueHeadCount, headDimension]),
16             values: try FloatTensor(
17                 Array(repeating: 0, count: sequence * request.configuration.keyValueProjectionDimension),
18                 shape: [sequence, request.configuration.keyValueHeadCount, headDimension]))
19     }
20 }
```

44.17.2 Sources/LeetLLMExercises/P043FusedQKVMetal.swift

```
1 import Foundation
2 import LeetLLMCore
3
4 extension P043FusedQKVExercise {
5     public static func makeMetalPipeline() throws -> MetalFusedQKVPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P043FusedQKV", withExtension: "metal", subdirectory: "Metal")
8         else { throw MetalNeuralOperatorError.kernelResourceMissing("P043FusedQKV.metal") }
9         return try MetalFusedQKVPipeline(source: String(contentsOf: url, encoding: .utf8))
10     }
11 }
```

44.17.3 Sources/LeetLLMExercises/Metal/P043FusedQKV.metal

```
1 #include <metal_stdlib>
2 using namespace metal;
3
4 constant uint fusedThreadgroupWidth = 256;
5
6 kernel void fused_rmsnorm_qkv(
7     device const float *input [[buffer(0)]],
8     device const float *gamma [[buffer(1)]],
9     device const float *queryWeights [[buffer(2)]],
10    device const float *keyWeights [[buffer(3)]],
11    device const float *valueWeights [[buffer(4)]],
12    device float *queries [[buffer(5)]],
13    device float *keys [[buffer(6)]],
14    device float *values [[buffer(7)]],
15    constant uint4 &shape [[buffer(8)]],
```

```

16     constant float &epsilon [[buffer(9)]],
17     uint localIndex [[thread_index_in_threadgroup]],
18     uint token [[threadgroup_position_in_grid]]
19 ) {
20     threadgroup float scratch[fusedThreadgroupWidth];
21     float localSumSquares = 0.0f;
22     for (uint feature = localIndex; feature < shape.y; feature += fusedThreadgroupWidth) {
23         float value = input[token * shape.y + feature];
24         localSumSquares += value * value;
25     }
26     scratch[localIndex] = localSumSquares;
27     threadgroup_barrier(mem_flags::mem_threadgroup);
28     for (uint stride = fusedThreadgroupWidth / 2; stride > 0; stride >>= 1) {
29         if (localIndex < stride) {
30             scratch[localIndex] += scratch[localIndex + stride];
31         }
32         threadgroup_barrier(mem_flags::mem_threadgroup);
33     }
34     float inverseRMS = rsqrt(scratch[0] / float(shape.y) + epsilon);
35
36     // TODO: use the normalized row to evaluate Q, K, and V channel dot products.
37     uint combinedWidth = shape.z + 2 * shape.w;
38     for (uint channel = localIndex; channel < combinedWidth; channel += fusedThreadgroupWidth) {
39         if (channel < shape.z) {
40             queries[token * shape.z + channel] = 0.0f * inverseRMS;
41         } else if (channel < shape.z + shape.w) {
42             keys[token * shape.w + channel - shape.z] = 0.0f * inverseRMS;
43         } else {
44             values[token * shape.w + channel - shape.z - shape.w] = 0.0f * inverseRMS;
45         }
46     }
47 }

```

44.18 Worked solution

Solution strategy

Fuses RMSNorm and Q/K/V projections into a single operation to eliminate the intermediate normalized tensor and its four transfers. The separate path computes normalization, writes $[S, D]$ to memory, then launches three projection kernels that read it again. The fused path computes normalized values on-the-fly during projection accumulation. The Metal kernel launches one 256-thread threadgroup per token. Each thread accumulates a strided subset of squared features into threadgroup scratch memory, then participates in a tree reduction to compute the row sum of squares. All threads use the shared inverse RMS to scale normalized values while routing combined output channels to the correct Q, K, or V weight base and output buffer.

Correctness invariant

The fused and separate CPU paths produce identical Float32 outputs (absolute error bounded by $5e-5 + 1e-4 \cdot \max(|\text{actual}|, |\text{expected}|)$). Each output element equals the inner product of the normalized token row with the corresponding weight row, without materializing or storing the intermediate tensor. The Metal dispatch computes the same values with one RMS reduction and one combined channel loop per token.

Common failure mode

Attempting fusion without careful channel offset routing leads to writing Q outputs into K space or vice versa. The code routes channels $[0, Hq \cdot dh)$ to query, $[Hq \cdot dh, Hq \cdot dh + Hkv \cdot dh)$ to key, and $[Hq \cdot dh + Hkv \cdot dh, \dots)$ to value by subtracting the appropriate offset. Mixing up these boundaries silently corrupts attention head assignments.

44.18.1 Implementation points

Intermediate Removal and Traffic Model

Let $\text{normalizedBytes} = 4 \cdot S \cdot D$ for the Float32 $[S, D]$ tensor. The separate cost model counts one write by RMSNorm and three reads by the Q/K/V projections, for $4 \cdot \text{normalizedBytes} = 16 \cdot S \cdot D$ logical bytes, plus normalizedBytes of live intermediate storage. The fused model removes those four transfers and reports zero intermediate bytes. Metal end-to-end timing still includes shared-buffer allocation, host-to-device copy, command submission, and synchronous wait.

Threadgroup Reduction Barrier Discipline

Metal kernel strides features across 256 threads, accumulating partial sums into scratch memory. After initialization, threads participate in a barrier. Tree reduction runs in $\log(256)$ steps, with barriers after each stride to ensure all threads see updated values. Threads that do not participate in the final stride steps (e.g., when $\text{stride} < \text{thread index}$) must still reach the barrier to avoid deadlock. All threads use $\text{scratch}[0]$ as the final sum, then proceed to projection.

Combined Channel Indexing and Weight Routing

The Metal kernel flattens all Q/K/V outputs into one interval $[0, H_q * d_h + 2 * H_{kv} * d_h)$. For channel c , the code selects: if $c < H_q * d_h$, use `queryWeights` and `queries` buffer; else if $c < H_q * d_h + H_{kv} * d_h$, use `keyWeights` and `keys` with offset $c - H_q * d_h$; else use `valueWeights` and `values` with offset $c - H_q * d_h - H_{kv} * d_h$. This avoids three separate channel loops and keeps projection data access coalesced.

CPU Fused Path Recomputes Normalization Per Token

The fused CPU path does not allocate the normalized intermediate. Instead, for each token, it recomputes the sum of squares and inverse RMS inline within the projection loop. This trades memory allocation for arithmetic recomputation. The CPU path remains readable and demonstrates the mathematical equivalence; the Metal path trades the same recomputation for better parallelism, as threads share one reduction.

44.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P043FusedQKVMetal.swift

- Line 4: extension `P043FusedQKVSolution` {
- Line 5: public static func `makeMetalPipeline()` throws -> `MetalFusedQKVPipeline` {

Sources/LeetLLMSolutions/P043FusedQKVSolution.swift

- Line 4: public enum `P043FusedQKVSolution` {
- Line 5: public static func `separate(_ request: FusedQKVRequest)` throws -> `FusedQKVResult` {
- Line 24: func `project(_ weights: FloatTensor, heads: Int)` throws -> `FloatTensor` {
- Line 50: public static func `fused(_ request: FusedQKVRequest)` throws -> `FusedQKVResult` {
- Line 55: func `project(_ weights: FloatTensor, heads: Int)` throws -> `FloatTensor` {

Sources/LeetLLMSolutions/Metal/P043FusedQKV.metal

The Metal file defines the chapter's complete kernel entry points and synchronization schedule.

44.18.3 Complete canonical listings

44.18.4 Sources/LeetLLMSolutions/P043FusedQKVMetal.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 extension P043FusedQKVSolution {
5     public static func makeMetalPipeline() throws -> MetalFusedQKVPipeline {
6         guard let url = Bundle.module.url(
7             forResource: "P043FusedQKV", withExtension: "metal", subdirectory: "Metal")
8         else { throw MetalNeuralOperatorError.kernelResourceMissing("P043FusedQKV.metal") }
9         return try MetalFusedQKVPipeline(source: String(contentsOf: url, encoding: .utf8))
10     }
11 }

```


44.18.5 Sources/LeetLLMSolutions/P043FusedQKVSolution.swift

```

1 import Foundation
2 import LeetLLMCore
3
4 public enum P043FusedQKVSolution {
5     public static func separate(_ request: FusedQKVRequest) throws -> FusedQKVResult {
6         try P043FusedQKVContract.validate(request)
7         let sequence = request.input.shape[0]
8         let model = request.configuration.modelDimension
9         var normalized = Array(repeating: Float.zero, count: request.input.elementCount)
10        for token in 0..

```

```

71     }
72     output[token * outputWidth + channel] = sum
73 }
74 }
75 return try FloatTensor(
76     output, shape: [sequence, heads, request.configuration.headDimension])
77 }
78
79 return FusedQKVResult(
80     queries: try project(
81         request.queryWeights, heads: request.configuration.queryHeadCount),
82     keys: try project(
83         request.keyWeights, heads: request.configuration.keyValueHeadCount),
84     values: try project(
85         request.valueWeights, heads: request.configuration.keyValueHeadCount))
86 }
87 }

```

44.18.6 Sources/LeetLLMSolutions/Metal/P043FusedQKV.metal

```

1  #include <metal_stdlib>
2  using namespace metal;
3
4  constant uint fusedThreadgroupWidth = 256;
5
6  kernel void fused_rmsnorm_qkv(
7      device const float *input [[buffer(0)]],
8      device const float *gamma [[buffer(1)]],
9      device const float *queryWeights [[buffer(2)]],
10     device const float *keyWeights [[buffer(3)]],
11     device const float *valueWeights [[buffer(4)]],
12     device float *queries [[buffer(5)]],
13     device float *keys [[buffer(6)]],
14     device float *values [[buffer(7)]],
15     constant uint4 &shape [[buffer(8)]],
16     constant float &epsilon [[buffer(9)]],
17     uint localIndex [[thread_index_in_threadgroup]],
18     uint token [[threadgroup_position_in_grid]]
19 ) {
20     threadgroup float scratch[fusedThreadgroupWidth];
21     float localSumSquares = 0.0f;
22     for (uint feature = localIndex; feature < shape.y; feature += fusedThreadgroupWidth) {
23         float value = input[token * shape.y + feature];
24         localSumSquares += value * value;
25     }
26     scratch[localIndex] = localSumSquares;
27     threadgroup_barrier(mem_flags::mem_threadgroup);
28     for (uint stride = fusedThreadgroupWidth / 2; stride > 0; stride >= 1) {
29         if (localIndex < stride) {
30             scratch[localIndex] += scratch[localIndex + stride];
31         }
32         threadgroup_barrier(mem_flags::mem_threadgroup);
33     }
34
35     float inverseRMS = rsqrt(scratch[0] / float(shape.y) + epsilon);
36     uint combinedWidth = shape.z + 2 * shape.w;
37     for (uint channel = localIndex; channel < combinedWidth; channel += fusedThreadgroupWidth) {
38         device const float *weights;
39         device float *output;
40         uint outputWidth;
41         uint outputChannel;
42         if (channel < shape.z) {
43             weights = queryWeights;
44             output = queries;
45             outputWidth = shape.z;
46             outputChannel = channel;
47         } else if (channel < shape.z + shape.w) {
48             weights = keyWeights;
49             output = keys;
50             outputWidth = shape.w;

```

```
51     outputChannel = channel - shape.z;  
52 } else {  
53     weights = valueWeights;  
54     output = values;  
55     outputWidth = shape.w;  
56     outputChannel = channel - shape.z - shape.w;  
57 }  
58  
59     float sum = 0.0f;  
60     for (uint feature = 0; feature < shape.y; ++feature) {  
61         float normalized = input[token * shape.y + feature] * inverseRMS * gamma[feature];  
62         sum += weights[outputChannel * shape.y + feature] * normalized;  
63     }  
64     output[token * outputWidth + outputChannel] = sum;  
65 }  
66 }
```

Profile Prefill and Decode Separately

45.1 Why this exists

A single tokens-per-second number combines two different workloads. Prefill processes a prompt with multi-token matrix shapes and creates the KV cache. Decode processes one new token at a time, rereads weights, and scans a growing cache. Averaging them hides whether startup latency, prompt throughput, or steady-state decode is the actual problem.

This lab times the shared CPU reference engine with a monotonic clock. It reports fresh-cache prefill trials separately from per-token decode trials at several initial context lengths.

45.2 Learning outcomes

You can:

- define time-to-first-token, prompt throughput, and decode throughput boundaries;
- collect warmups and repeated measurements without timing prefill inside decode;
- compute median and nearest-rank percentiles;
- explain why decode context length changes attention work;
- label backend, clock, allocations, sampling, and synchronization included in a number; and
- choose an optimization target from stage-specific evidence.

45.3 Prerequisites

- Problem 006 for prediction, measurement, and roofline discipline.
- Problem 039 for fresh-cache prompt prefill.
- Problem 040 for serial cached decode.
- Problem 041 for allocation and intermediate-lifetime costs.

45.4 Vocabulary

- **Time to first token (TTFT)**: prompt prefill plus selection of the first generated token under a stated boundary.
- **Inter-token latency**: elapsed time for one serial decode engine call.
- **Prompt throughput**: prompt token count divided by prefill time.
- **Decode throughput**: generated token count divided by serial decode time.
- **Warmup**: unreported execution used to reach a representative runtime state.
- **Nearest-rank percentile**: sorted sample at index $\text{ceil}(p*N) - 1$.
- **Context length**: number of prior tokens visible to the next decode token.

45.5 Math from first principles

For sorted latencies $x_{(1)} \leq \dots \leq x_{(N)}$, the nearest-rank percentile is

$$P_p = x_{(\lceil pN \rceil)}.$$

For an even number of samples, median is the mean of the two middle values. For $[10, 20, 30, 40]$ ns, median is 25 ns and nearest-rank P_{75} is 30 ns.

If prefill handles S prompt tokens in median time t_p seconds,

$$R_p = \frac{S}{t_p}.$$

If one decode step has median latency t_d ,

$$R_d = \frac{1}{t_d}.$$

These rates are not interchangeable. Prefill rate measures many prompt tokens in one engine call; decode rate measures a serial dependency chain.

45.6 Shape, layout, and dtype contract

The request contains a validated `MiniDecoderModel`, nonempty in-vocabulary prompt IDs, positive context lengths, nonnegative warmups, positive measured trials, positive decode steps, and percentile in $(0, 1]$.

Latency samples are `UInt64` nanoseconds from `DispatchTime uptimeNanoseconds`. Reported medians and rates use `Double`. Work estimates are integer FLOPs and bytes from the educational engine model; they are not hardware performance counters.

Each decode context is prepared by repeating the supplied prompt IDs to exactly τ tokens. A fresh contiguous `Float32` KV cache is populated before the measured serial steps.

45.7 CPU reference path

The canonical profiler performs:

1. Unreported fresh-cache prefill warmups.
2. One fresh cache allocation and prefill call per measured trial.
3. Separate cache population for every decode context and trial.
4. Per-token timing around only `MiniDecoderCPUEngine.decode`.
5. Median, percentile, and rate computation after all trials.

```
1 swift run -c release leetllm profile 044 \
2   --prompt-tokens 16 --trials 7 --warmup 2 --decode-steps 4
```

`benchmark 044` is an alias with the same options.

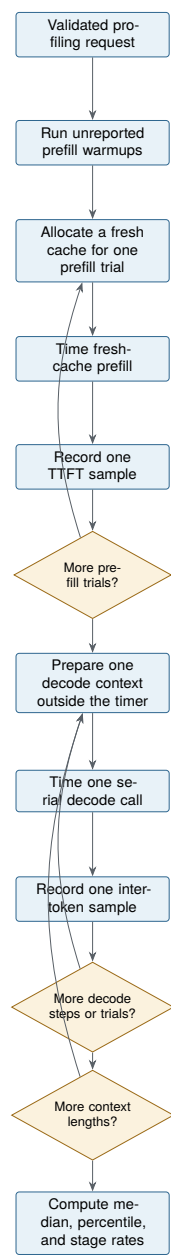


Figure 45.1: Prefill decode timing boundaries.

45.8 Correctness method

Pure statistics tests establish odd/even medians and nearest-rank percentiles independently of wall-clock noise. The profiling judge then checks:

- exact sample counts;
- positive durations and rates;
- stage names `prefill.ttft` and `decode.context.T`;
- context order;
- backend label; and
- invalid percentile, warmup, trial, decode-step, and context inputs.

The judge does not require one context to be faster than another because real timing is noisy. It verifies measurement structure, not a fabricated performance ordering.

45.9 Performance model

For the educational decoder, projection work and estimated weight bytes come from `MiniDecoderWorkModel`. Prefill's multi-token projections can reuse weights across S rows. Decode reads a similar weight set for one row and attention scans cached K/V. As context T grows, attention work grows approximately linearly per new token even when projection work stays fixed.

The report labels:

- real monotonic wall-clock samples;
- modeled FLOPs and estimated bytes;
- fresh-cache allocation included in prefill; and
- tensor allocation, cache append, sampling, and host execution included in decode.

No hardware counter is inferred from elapsed time.

45.10 Metal mapping

This implemented profiler runs the CPU reference backend only. There is no Metal check for Problem 044.

To profile a future Metal engine honestly, place timestamp samples around command-buffer execution or use Instruments, preserve prefill/decode boundaries, report whether command submission and waits are included, and avoid a host timer that stops before asynchronous GPU work completes. Counter availability must be observed from the actual tool and device; this lesson does not invent GPU occupancy, bandwidth, or cache-hit values.

45.11 Implementation checkpoints

1. Implement median and percentile statistics with known arrays.
2. Validate all counts and context lengths.
3. Warm up prefill without recording samples.
4. Allocate a fresh cache inside each measured prefill trial.
5. Populate decode context outside the per-token timer.
6. Record exactly `trials*steps` decode samples per context.
7. Convert nanoseconds to rates with explicit units.

8. Print backend, clock, and timing boundary beside results.

45.12 Controlled experiments

45.12.1 Context-length sweep

Use initial contexts T , $4T$, and $16T$. Predict how attention scanning changes decode latency while weight-read estimates remain similar.

45.12.2 Prompt-length sweep

Hold trial count fixed and vary prompt length. Predict whether prompt tokens per second improves as fixed overhead is amortized.

45.12.3 Allocation boundary

Move cache allocation outside prefill timing in a private experiment. The numerical result should not change, but TTFT should. This demonstrates why boundary labels are part of the result.

45.12.4 Distribution stability

Compare one trial with at least seven. Predict how median and p95 change when transient system activity appears.

45.13 Engine integration

The profile identifies whether the next engine change should target prompt execution, serial decode, cache traffic, or host allocation. Problem 047 reuses the same CPU engine and reports prefill and decode separately; it does not replace this multi-context profiling sweep.

45.14 Tradeoffs and limitations

- Median is robust but hides tail latency; p95 is unstable with very few samples.
- More warmup improves runtime stability but increases experiment cost.
- Fresh-cache prefill reflects TTFT but includes allocation policy.
- CPU reference timings teach boundaries; they do not predict a future Metal backend.
- The educational model is too small for production throughput conclusions.
- Background system work can move wall-clock samples without a code change.

45.15 Hints

- Use a monotonic clock, never wall-calendar time.
- Keep warmup samples out of the reported array.
- Rebuild the cache before each independent trial.
- Time one decode step at a time; do not divide a mixed prefill/decode session.
- Write the timing boundary before interpreting the number.

45.16 Completion checklist

- ☐ Statistics pass hand-computed fixtures.
- ☐ Prefill and decode are measured in separate boundaries.
- ☐ Warmup and measured sample counts are explicit.
- ☐ Context lengths are reported beside per-token latency.
- ☐ Backend, clock, and included allocations are labeled.
- ☐ Modeled work is not presented as a hardware counter.
- ☐ A release profile and written prediction are recorded.

45.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

45.17.1 Sources/LeetLLMExercises/P044PrefillDecodeProfilingExercise.swift

```

1 import LeetLLMCore
2
3 public enum P044PrefillDecodeProfilingExercise {
4     public static func profile(
5         _ request: PrefillDecodeProfilingRequest
6     ) throws -> PrefillDecodeProfileReport {
7         try P044ProfilingContract.validate(request)
8         let zeroLatency = LatencyStatistics(
9             samplesNanoseconds: Array(repeating: 0, count: request.measuredTrials),
10             medianNanoseconds: 0,
11             percentile: request.percentile,
12             percentileNanoseconds: 0,
13             minimumNanoseconds: 0,
14             maximumNanoseconds: 0)
15         // TODO: collect fresh-cache prefill trials and serial decode trials independently.
16         return PrefillDecodeProfileReport(
17             backend: P044ProfilingContract.cpuReferenceBackend,
18             clock: "not implemented",
19             timingBoundary: "not implemented",
20             warmupIterations: request.warmupIterations,
21             measuredTrials: request.measuredTrials,
22             prefill: PrefillProfile(
23                 stageName: "prefill.ttft",
24                 promptTokenCount: request.promptTokenIDs.count,
25                 latency: zeroLatency,
26                 promptTokensPerSecond: 0,
27                 work: ProfilingWorkEstimate(
28                     floatingPointOperations: 0, estimatedWeightBytesRead: 0, cacheBytesWritten: 0)),
29             decode: [])
30     }
31 }

```

45.18 Worked solution

Solution strategy

Separates prefill and decode profiling into independent measurement boundaries using a monotonic clock (`DispatchTime uptimeNanoseconds`). Prefill runs unreported warmup trials, then times fresh-cache allocations and engine calls to produce time-to-first-token (TTFT). Every warmup or measured decode trial independently repeats the prompt to the target context length, allocates a fresh cache, and prefills it outside the per-token timer; the trial then measures serial decode engine calls. Multiple context lengths reveal how attention work scales. The profiler derives medians and nearest-rank percentiles from sorted samples, converts them to throughput rates, and reports modeled work separately from measured wall-clock time.

Correctness invariant

Warmup samples are discarded and never included in reported medians. Prefill allocates a fresh cache inside each measured trial. Decode context is prepared outside the timer; only the engine call is timed. Cache is repopulated for each trial and context. Percentiles are computed as the nearest-rank statistic $x_{(\text{ceil}(p*N))}$, and medians account for even/odd sample count correctly. Modeled work (FLOPs, weight bytes) is derived from the educational model, not inferred from elapsed time.

Common failure mode

Measuring prefill inside decode timing, mixing cache allocation costs into the decode latency, or conflating modeled work with hardware performance counters. Reusing a cache across prefill trials hides allocation overhead in the first trial.

45.18.1 Implementation points

Fresh-Cache Prefill Per Trial

Each measured prefill trial allocates a new `ContiguousKVCache` with capacity equal to prompt length. The allocation time is included in the measured wall-clock sample. This simulates TTFT for a cold-start prompt but means that the first trial may be slower due to system allocation overhead. Comparing multiple trials with fresh caches reveals whether a median latency is stable or affected by transient allocation costs.

Decode Context Prepared Outside Timer

Each decode trial creates its own repeated prompt and fresh cache, then runs prefill and selects the initial token before any per-step timer starts. Only each `MiniDecoderCPUEngine.decode` call is timed. Rebuilding the context for every warmup and measured trial prevents cache state from leaking between trials while keeping cache construction and prefill outside the reported decode samples.

Nearest-Rank Percentile Computation

Sorted samples are indexed using $P_p = x_{(\text{ceil}(p*N) - 1)}$. For nine samples and $p=0.75$, $\text{ceil}(0.75*9) = 7$, so $P_{.75}$ is the 7th element (0-indexed at [6]). Medians use the mean of the two middle elements for even counts. This avoids interpolation and remains robust to small sample sizes used in educational profiling.

Work Model Labels in Report

The report records prefill and decode separately, each with a stageName such as 'prefill.ttft' or 'decode.context.16'. It stores backend 'CPU reference (Swift)' and clock 'DispatchTime.uptimeNanoseconds (monotonic)' as separate fields. The timing-boundary text states that prefill includes fresh cache allocation and the engine call, while decode excludes prefill but includes per-step tensor allocations, cache append, sampling, and host execution. Modeled FLOPs and weight bytes remain separate from wall-clock samples.

45.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P044PrefillDecodeProfilingSolution.swift

- Line 4: public enum P044PrefillDecodeProfilingSolution {
- Line 5: public static func profile(
- Line 93: private struct DecodeTrialResult {
- Line 98: private static func runFreshPrefill(
- Line 110: private static func runDecodeTrial(
- Line 152: private static func positiveElapsed(since start: UInt64) -> UInt64 {

45.18.3 Complete canonical listings

45.18.4 Sources/LeetLLMSolutions/P044PrefillDecodeProfilingSolution.swift

```

1 import Dispatch
2 import LeetLLMCore
3
4 public enum P044PrefillDecodeProfilingSolution {
5     public static func profile(
6         _ request: PrefillDecodeProfilingRequest
7     ) throws -> PrefillDecodeProfileReport {
8         try P044ProfilingContract.validate(request)
9
10        for _ in 0..

```

```

24     capacity: request.promptTokenIDs.count)
25     prefillSamples.append(positiveElapsed(since: start))
26     prefillWork = result.work
27 }
28 let prefillLatency = try P044LatencyStatistics.summarize(
29     prefillSamples, percentile: request.percentile)
30 let prefillRate = Double(request.promptTokenIDs.count) * 1_000_000_000
31     / prefillLatency.medianNanoseconds
32
33 var decodeProfiles: [DecodeContextProfile] = []
34 for contextLength in request.decodeContextLengths {
35     for warmup in 0..

```

```

96 }
97
98 private static func runFreshPrefill(
99     model: MiniDecoderModel,
100     tokenIDs: [Int],
101     capacity: Int
102 ) throws -> PromptPrefillResult {
103     let cache = ContiguousKVCache(
104         configuration: try model.cacheConfiguration(capacity: capacity))
105     return try MiniDecoderCPUEngine.prefill(
106         PromptPrefillRequest(model: model, tokenIDs: tokenIDs),
107         cache: cache)
108 }
109
110 private static func runDecodeTrial(
111     request: PrefillDecodeProfilingRequest,
112     contextLength: Int,
113     trialSeed: UInt64,
114     recordSamples: Bool
115 ) throws -> DecodeTrialResult {
116     let prompt = (0..

```

Static and Continuous Batching

46.1 Why this exists

Single-request decode leaves accelerator capacity unused when one sequence finishes before another or when requests arrive over time. Batching can improve total throughput, but scheduling policy changes queueing delay and per-request latency. A policy can improve makespan while making an already-running request finish later.

This systems lab uses a deterministic discrete-event simulator. Time is explicitly labeled as modeled scheduler units; it is not wall-clock GPU evidence. Token state remains independent for every request while static and continuous policies share the same semantic executor.

46.2 Learning outcomes

You can:

- distinguish static groups from continuous slot refill;
- represent arrivals, prefill stalls, decode iterations, and completions as events;
- calculate makespan, request latency, throughput, and slot utilization;
- explain a throughput improvement that worsens one request's latency;
- preserve per-request token and cache ownership; and
- identify which modeled costs require real engine measurements before deployment.

46.3 Prerequisites

- Problem 039 for prompt prefill as a distinct stage.
- Problem 040 for one serial token step per active request.
- Problems 022 and 027 for sequence-owned KV state and allocation.
- Problem 044 for separating measured latency from modeled quantities.

46.4 Vocabulary

- **Static batching:** admit a group up to slot capacity and run it to completion before admitting another group.
- **Continuous batching:** refill slots as requests complete while other requests remain active.
- **Slot:** capacity for one active sequence in this model.
- **Iteration:** one decode token for each active request.
- **Makespan:** finish time of the final request.
- **Request latency:** finish time minus arrival time.
- **Slot utilization:** occupied slot-time divided by available slot-time.

- **Head-of-line effect:** one request delays work that could otherwise proceed.

46.5 Math from first principles

For request i with arrival a_i and finish f_i ,

$$L_i = f_i - a_i.$$

If the run processes N prompt and generated tokens over makespan M modeled units,

$$R = \frac{N}{M}.$$

For C slots and event e with duration Δt_e and occupancy o_e ,

$$U = \frac{\sum_e o_e \Delta t_e}{CM}.$$

The fixture uses two slots. Requests A and B arrive at 0; C arrives at 1. Decode lengths are 5, 1, and 5. Prefill costs one unit per maximum prompt token in the admitted group; every decode iteration costs one unit.

Static batching finishes A at 6, B at 2, and C at 16. Continuous batching admits C after B frees a slot. Its prefill pauses A, so A finishes at 11 rather than 6, while total makespan improves to 12. The fixture deliberately proves a tradeoff, not universal dominance.

46.6 Shape, layout, and dtype contract

This problem has request records rather than tensors:

- nonempty unique string ID;
- nonnegative integer arrival time;
- nonempty integer prompt-token array;
- positive integer decode-token count;
- positive slot count; and
- nonnegative integer cost coefficients that yield positive stage durations.

Events form a contiguous integer timeline. Generated token IDs are deterministic integers from a request-local semantic function. Metrics use integer modeled units and Double rates. No duration is labeled milliseconds or nanoseconds.

46.7 CPU reference path

The simulator stores runtime state by request ID. Static batching admits up to `slotCount`, emits one explicit prefill event, and advances that group until empty. Continuous batching refills free slots, emits a prefill event for newly admitted requests, then resumes decode for all active requests.

Each decode event records the exact token emitted for each participant. The semantic executor depends on request ID, prompt, prior generated tokens, and step so cross-request contamination is detectable.

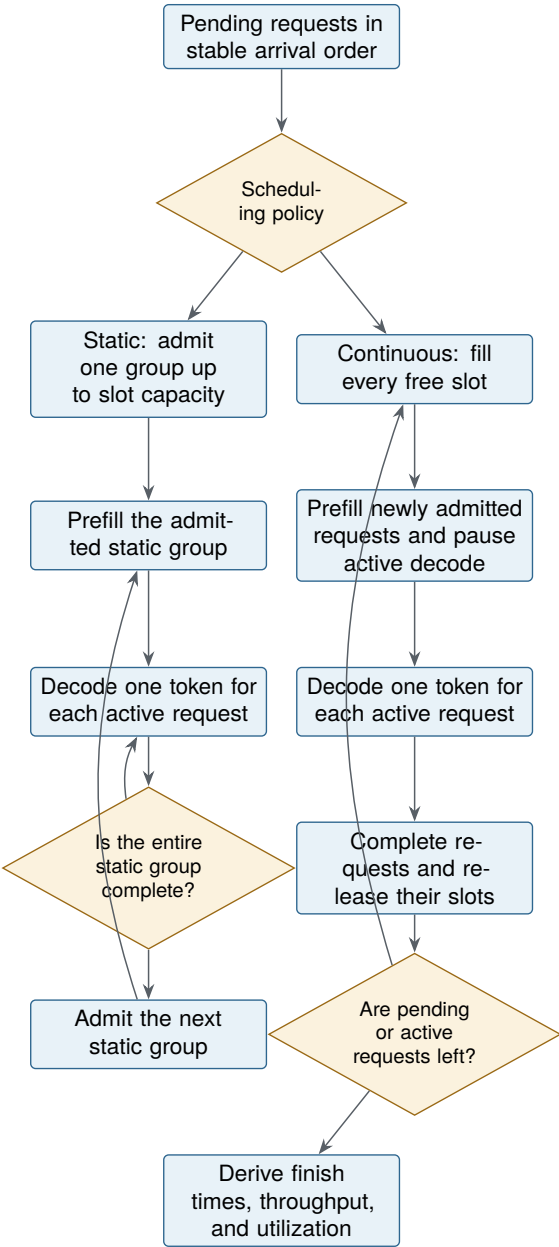


Figure 46.1: Static continuous scheduler.

```
1 swift run leetllm check 045 --cpu --solution
```

46.8 Correctness method

The judge checks the exact static and continuous timelines through finish times and makespan. It also requires:

- continuous throughput greater than static throughput in the fixture;
- A's continuous latency greater than its static latency;
- request-local generated token sequences under both policies;
- total-token and throughput identities;
- utilization in $[0, 1]$;
- contiguous bounded events; and
- rejection of duplicate IDs and zero slots.

The oracle computes each request's tokens independently of scheduling order.

46.9 Performance model

Prefill duration is

$$C_p = c_{p,0} + c_{p,t} \max_i S_i$$

for newly admitted requests. Decode iteration duration is

$$C_d = c_{d,0} + c_{d,r} A,$$

where A is active request count. These equations are intentionally replaceable. They let the lab isolate scheduling mechanics before introducing hardware variability.

The simulator reports modeled makespan, tokens per modeled unit, and slot utilization. It does not claim that padding to the longest prompt, linear active-request cost, or a prefill pause matches a particular production server.

46.10 Metal mapping

There is no Metal kernel in Problem 045 because scheduling is a host-side policy. A real GPU implementation would construct batched tensor descriptors, sequence-to-slot maps, cache page tables, and command-buffer work for each iteration. It would also decide whether newly arrived prefill can interleave with decode.

Before replacing modeled units, measure prefill shapes, decode batch sizes, cache gather cost, and synchronization using the actual backend. Keep scheduler decisions on the host unless evidence supports device-side orchestration.

46.11 Implementation checkpoints

1. Validate request identity, arrival, prompt, decode length, slots, and costs.

2. Implement a contiguous event append operation.
3. Advance idle time to the next arrival.
4. Admit requests in stable arrival order.
5. Preserve one runtime state per request ID.
6. Implement static groups to completion.
7. Implement continuous refill with explicit prefill events.
8. Derive metrics from the completed timeline and validate them.

46.12 Controlled experiments

46.12.1 Slot-count sweep

Try one, two, and four slots. Predict which workloads gain utilization and where prefill stalls dominate.

46.12.2 Arrival burst

Move all arrivals to zero, then spread them apart. Predict when continuous refill becomes indistinguishable from static grouping.

46.12.3 Length variance

Hold total generated tokens fixed while increasing variance among request lengths. Predict how static slot waste changes.

46.12.4 Prefill cost

Increase the prompt length of late request C. Predict both makespan and the latency penalty imposed on already-running A.

Write predictions in modeled units. Do not translate them to milliseconds without a measured calibration.

46.13 Engine integration

A production version would give each admitted request a tokenizer result, sampler state, stop policy, and KV-cache allocation, then gather active rows into the next batched decoder call. Problem 047 remains batch size one; this lab defines the scheduler boundary required to extend it without mixing sequence state.

46.14 Tradeoffs and limitations

- Continuous batching can improve throughput while increasing latency for active requests.
- Static batches are simple and reproducible but waste slots after early completion.
- Prefill/decode interleaving policy can dominate tail latency.
- Padding and cache indirection costs are absent from the semantic executor.
- Modeled units establish causal scheduling behavior, not hardware performance.
- Fairness, priorities, cancellation, and memory admission are outside this fixture.

46.15 Hints

- Keep pending, active, and completed states distinct.

- Use stable arrival order for equal timestamps.
- Record participants before removing completed requests.
- Set finish time after the decode event advances time.
- Recompute semantic tokens by request ID in the judge.

46.16 Completion checklist

- ☐ Static and continuous fixture finish times match exactly.
- ☐ Every timeline event is contiguous and slot-bounded.
- ☐ Per-request generated tokens are schedule-independent.
- ☐ Throughput and utilization are derived from the timeline.
- ☐ The latency regression for A is explained.
- ☐ All timing quantities are labeled modeled scheduler units.
- ☐ A real-backend measurement plan is stated before hardware claims.

46.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

46.17.1 Sources/LeetLLMExercises/P045BatchSchedulingExercise.swift

```
1 import LeetLLMCore
2
3 public enum P045BatchSchedulingExercise {
4     public static func simulate(
5         _ request: SchedulingSimulationRequest
6     ) throws -> BatchSimulationReport {
7         try P045BatchSchedulingContract.validate(request)
8         // TODO: model static groups and continuous slot refill with explicit prefill events.
9         return BatchSimulationReport(
10             policy: request.policy,
11             timingUnitLabel: P045BatchSchedulingContract.timingUnitLabel,
12             timeline: [],
13             requests: [],
14             makespan: 1,
15             totalTokens: 0,
16             throughputTokensPerUnit: 0,
17             slotUtilization: 0)
18     }
19 }
```

46.18 Worked solution

Solution strategy

A discrete-event simulator with two policies: static batching admits up to 'slotCount' requests, runs them to completion (prefill once, then decode iterations until all finish), then admits the next group; continuous batching fills free slots as requests complete while other requests remain active. Both policies emit the same semantic tokens per request (deterministic via P045SemanticExecutor, which takes request ID, prompt, generated tokens, and step). The simulator maintains a timeline of events (idle, prefill, decodeIteration), tracks request state (startTime, finishTime, generatedTokens), and computes makespan, per-request latency, throughput (total tokens / makespan), and slot utilization (occupied slot-time / total available slot-time).

Correctness invariant

Generated tokens are deterministic per request ID and independent of scheduling order. Throughput under continuous batching is greater than or equal to static (in the fixture, strictly greater). Request latency (finishTime - arrivalTime) may increase for early requests under continuous batching due to prefill pauses for newly admitted requests. Slot utilization is bounded in $[0, 1]$; total tokens and makespan are consistent across both policies.

Common failure mode

Allowing generated tokens to depend on schedule order (e.g., sampling from the wrong request's prefix or step), continuing decode after a request is complete, or incorrectly computing slot occupancy when requests finish at different times.

46.18.1 Implementation points

Semantic Oracle and Request ID Isolation

The P045SemanticExecutor.nextToken function takes request ID, prompt, generated tokens so far, and step count, and returns a deterministic token. The oracle recomputes each request's full token sequence by request ID, independent of the timeline, to detect cross-request contamination. If the simulator allows tokens to bleed from one request's distribution to another, the oracle catches it.

Static Batching Admits and Runs to Completion

Static batching issues one prefill event for the admitted group, then alternates decode iteration events until no request in that group has remaining tokens. The loop checks 'active.removeAll { remainingDecodeTokens == 0 }' after each iteration. No new requests are admitted until the entire group finishes, so slot waste occurs if one request finishes before others in the same static batch.

Continuous Refill and Prefill Pause

Continuous batching admits new requests into free slots up to 'slotCount - active.count'. When new requests are admitted, a prefill event is issued for only the newly admitted set, and the occupancy for that event includes both the newly admitted and previously active requests. This prefill 'pauses' active decode, explaining why early requests may have longer latency than under static batching.

Timeline and Utilization Metrics

Each event records `startTime`, `endTime`, `occupiedSlotCount`, and `participants`. Occupancy is only counted during prefill and decode events (idle events have zero occupancy). Slot utilization is `occupiedSlotUnits` (sum of occupancy * duration for all events) divided by `slotCount * makespan`. The fixture proves that continuous improves throughput while increasing latency for request A, demonstrating the tradeoff explicitly in modeled units without hardware measurements.

46.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P045BatchSchedulingSolution.swift

- Line 3: public enum P045BatchSchedulingSolution {
- Line 4: public static func simulate(
- Line 16: private struct RuntimeRequest {
- Line 26: mutating func advance() -> ScheduledToken {
- Line 38: private struct SimulationState {
- Line 45: init(requests: [ScheduledRequest]) {
- Line 57: mutating func appendEvent(
- Line 75: mutating func advanceToNextArrival() {
- Line 82: mutating func admit(upTo count: Int) -> [String] {
- Line 98: func report(for request: SchedulingSimulationRequest) throws -> BatchingSimulationReport {
- Line 126: private static func simulateStatic(
- Line 165: private static func simulateContinuous(

46.18.3 Complete canonical listings

46.18.4 Sources/LeetLLMSolutions/P045BatchSchedulingSolution.swift

```

1 import LeetLLMCore
2
3 public enum P045BatchSchedulingSolution {
4     public static func simulate(
5         _ request: SchedulingSimulationRequest
6     ) throws -> BatchingSimulationReport {
7         try P045BatchSchedulingContract.validate(request)
8         switch request.policy {
9             case .staticBatching:
10                 return try simulateStatic(request)
11             case .continuousBatching:
12                 return try simulateContinuous(request)
13         }
14     }
15
16     private struct RuntimeRequest {
17         let request: ScheduledRequest
18         var startTime: Int?
19         var finishTime: Int?
20         var generatedTokenIDs: [Int] = []
21
22         var remainingDecodeTokens: Int {
23             request.decodeTokenCount - generatedTokenIDs.count
24         }
25
26         mutating func advance() -> ScheduledToken {
27             let step = generatedTokenIDs.count
28             let token = P045SemanticExecutor.nextToken(

```

```

29     requestID: request.id,
30     promptTokenIDs: request.promptTokenIDs,
31     generatedTokenIDs: generatedTokenIDs,
32     step: step)
33     generatedTokenIDs.append(token)
34     return ScheduledToken(requestID: request.id, tokenID: token, sequenceIndex: step)
35 }
36 }
37
38 private struct SimulationState {
39     var time = 0
40     var timeline: [SchedulingEvent] = []
41     var occupiedSlotUnits = 0
42     var runtimes: [String: RuntimeRequest]
43     var pendingIDs: [String]
44
45     init(requests: [ScheduledRequest]) {
46         runtimes = Dictionary(uniqueKeysWithValues: requests.map {
47             ($0.id, RuntimeRequest(request: $0))
48         })
49         pendingIDs = requests.enumerated().sorted {
50             if $0.element.arrivalTime == $1.element.arrivalTime {
51                 return $0.offset < $1.offset
52             }
53             return $0.element.arrivalTime < $1.element.arrivalTime
54         }.map(\.element.id)
55     }
56
57     mutating func appendEvent(
58         kind: SchedulingEventKind,
59         duration: Int,
60         requestIDs: [String],
61         occupiedSlotCount: Int,
62         generatedTokens: [ScheduledToken] = []
63     ) {
64         timeline.append(SchedulingEvent(
65             kind: kind,
66             startTime: time,
67             endTime: time + duration,
68             requestIDs: requestIDs,
69             occupiedSlotCount: occupiedSlotCount,
70             generatedTokens: generatedTokens))
71         occupiedSlotUnits += duration * occupiedSlotCount
72         time += duration
73     }
74
75     mutating func advanceToNextArrival() {
76         guard let id = pendingIDs.first, let next = runtimes[id]?.request.arrivalTime,
77             next > time
78         else { return }
79         appendEvent(kind: .idle, duration: next - time, requestIDs: [], occupiedSlotCount: 0)
80     }
81
82     mutating func admit(upTo count: Int) -> [String] {
83         var admitted: [String] = []
84         while admitted.count < count,
85             let id = pendingIDs.first,
86             let runtime = runtimes[id],
87             runtime.request.arrivalTime <= time
88         {
89             pendingIDs.removeFirst()
90             var updated = runtime
91             updated.startTime = time
92             runtimes[id] = updated
93             admitted.append(id)
94         }
95         return admitted
96     }
97
98     func report(for request: SchedulingSimulationRequest) throws -> BatchingSimulationReport {
99         let metrics = request.requests.map { item -> RequestSchedulingMetrics in
100             let runtime = runtimes[item.id]!
101             return RequestSchedulingMetrics(

```



```

102     requestID: item.id,
103     arrivalTime: item.arrivalTime,
104     startTime: runtime.startTime!,
105     finishTime: runtime.finishTime!,
106     latency: runtime.finishTime! - item.arrivalTime,
107     generatedTokenIDs: runtime.generatedTokenIDs)
108 }
109 let totalTokens = request.requests.reduce(0) {
110     $0 + $1.promptTokenIDs.count + $1.decodeTokenCount
111 }
112 let result = BatchingSimulationReport(
113     policy: request.policy,
114     timingUnitLabel: P045BatchSchedulingContract.timingUnitLabel,
115     timeline: timeline,
116     requests: metrics,
117     makespan: time,
118     totalTokens: totalTokens,
119     throughputTokensPerUnit: Double(totalTokens) / Double(time),
120     slotUtilization: Double(occupiedSlotUnits) / Double(request.slotCount * time))
121 try P045BatchSchedulingContract.validate(result, for: request)
122 return result
123 }
124 }
125
126 private static func simulateStatic(
127     _ request: SchedulingSimulationRequest
128 ) throws -> BatchingSimulationReport {
129     var state = SimulationState(requests: request.requests)
130     while !state.pendingIDs.isEmpty {
131         state.advanceToNextArrival()
132         var active = state.admit(upTo: request.slotCount)
133         let admittedRequests = active.map { state.runtimes[$0]!.request }
134         state.appendEvent(
135             kind: .prefill,
136             duration: request.costModel.prefillCost(for: admittedRequests),
137             requestIDs: active,
138             occupiedSlotCount: active.count)
139         while !active.isEmpty {
140             var tokens: [ScheduledToken] = []
141             for id in active {
142                 var runtime = state.runtimes[id]!
143                 tokens.append(runtime.advance())
144                 state.runtimes[id] = runtime
145             }
146             let participants = active
147             state.appendEvent(
148                 kind: .decodeIteration,
149                 duration: request.costModel.decodeIterationCost(activeRequestCount: participants.count),
150                 requestIDs: participants,
151                 occupiedSlotCount: participants.count,
152                 generatedTokens: tokens)
153             active.removeAll { id in
154                 var runtime = state.runtimes[id]!
155                 guard runtime.remainingDecodeTokens == 0 else { return false }
156                 runtime.finishTime = state.time
157                 state.runtimes[id] = runtime
158                 return true
159             }
160         }
161     }
162     return try state.report(for: request)
163 }
164
165 private static func simulateContinuous(
166     _ request: SchedulingSimulationRequest
167 ) throws -> BatchingSimulationReport {
168     var state = SimulationState(requests: request.requests)
169     var active: [String] = []
170     while !state.pendingIDs.isEmpty || !active.isEmpty {
171         if active.isEmpty {
172             state.advanceToNextArrival()
173         }
174         let admitted = state.admit(upTo: request.slotCount - active.count)

```

```

175     if !admitted.isEmpty {
176         active.append(contentsOf: admitted)
177         let admittedRequests = admitted.map { state.runtimes[$0]!.request }
178         state.appendEvent(
179             kind: .prefill,
180             duration: request.costModel.prefillCost(for: admittedRequests),
181             requestIDs: admitted,
182             occupiedSlotCount: active.count)
183     }
184     guard !active.isEmpty else { continue }
185
186     var tokens: [ScheduledToken] = []
187     for id in active {
188         var runtime = state.runtimes[id]!
189         tokens.append(runtime.advance())
190         state.runtimes[id] = runtime
191     }
192     let participants = active
193     state.appendEvent(
194         kind: .decodeIteration,
195         duration: request.costModel.decodeIterationCost(activeRequestCount: participants.count),
196         requestIDs: participants,
197         occupiedSlotCount: participants.count,
198         generatedTokens: tokens)
199     active.removeAll { id in
200         var runtime = state.runtimes[id]!
201         guard runtime.remainingDecodeTokens == 0 else { return false }
202         runtime.finishTime = state.time
203         state.runtimes[id] = runtime
204         return true
205     }
206 }
207 return try state.report(for: request)
208 }
209 }

```

Speculative Decoding

47.1 Why this exists

Autoregressive target-model decoding has a serial dependency: token $t + 1$ cannot be selected until token t is known. Speculative decoding asks a cheaper draft model to propose several tokens, then verifies those proposals with the target distribution. When proposals are often accepted and target verification handles a block efficiently, one serial target step can emit several tokens.

A proposal is not accepted merely because the target also considers it likely. The accept/reject rule and rejection correction are required to preserve the target distribution.

47.2 Learning outcomes

You can:

- generate a draft block with prefix-dependent distributions;
- compute the acceptance ratio $\min(1, p/q)$;
- sample the rejection correction proportional to $\max(p - q, 0)$;
- emit a target bonus token when all proposals pass;
- prove the one-step output distribution equals the target; and
- derive when draft and verification costs can break even.

47.3 Prerequisites

- Problem 009 for stable normalization.
- Problem 038 for deterministic categorical sampling.
- Problem 040 for the serial target-only baseline.
- Problem 044 for separating modeled speedup from measured latency.

47.4 Vocabulary

- **Draft model:** cheaper provider with distribution q .
- **Target model:** distribution p whose output law must be preserved.
- **Proposal block:** up to K tokens sampled autoregressively from the draft.
- **Acceptance ratio:** $\alpha(y) = \min(1, p(y)/q(y))$ for sampled proposal y .
- **Correction distribution:** normalized positive part $[p - q]_+$ after rejection.
- **Bonus token:** target sample after all K proposals are accepted.
- **Acceptance rate:** empirical or modeled probability that a proposal passes.

47.5 Math from first principles

For proposal $y \sim q$, accept with

$$\alpha(y) = \min\left(1, \frac{p(y)}{q(y)}\right).$$

Accepted probability mass for token y is

$$q(y)\alpha(y) = \min(q(y), p(y)).$$

If rejected, sample replacement from

$$r(y) = \frac{\max(p(y) - q(y), 0)}{\sum_z \max(p(z) - q(z), 0)}.$$

The total rejection probability is

$$1 - \sum_y \min(p(y), q(y)) = \sum_y \max(p(y) - q(y), 0).$$

Therefore accepted mass plus rejected-and-corrected mass is

$$\min(p(y), q(y)) + \max(p(y) - q(y), 0) = p(y).$$

For $p=[0.25, 0.75]$ and $q=[0.75, 0.25]$, accepted mass is $[0.25, 0.25]$; rejection mass is 0.5 ; correction is $[0, 1]$; final output mass is $[0.25, 0.75]$.

47.6 Shape, layout, and dtype contract

A distribution provider receives the complete integer prefix and returns either logits or probabilities of exactly vocabulary size $v > 0$.

- Logits: finite Double values, normalized by max-subtracted softmax.
- Probabilities: finite nonnegative Double values with positive total mass, then normalized.
- Draft length: positive integer κ .
- Prefix: any integer history meaningful to the provider.
- Random draws: deterministic Double values from `SeededGenerator`.
- A sampled draft token must have positive q probability.

Traces retain every proposal prefix, distribution, probability, draw, acceptance decision, correction, and optional bonus.

47.7 CPU reference path

The canonical block first samples all K draft proposals autoregressively. It then verifies them in order using target distributions at each proposal prefix.

- On acceptance: emit proposal and continue.

- On rejection: sample one corrected replacement, emit it, and stop the block.
- If all pass: sample and emit one target bonus token from the prefix containing all proposals.

A separate target-only function emits the same requested number of tokens without drafting.

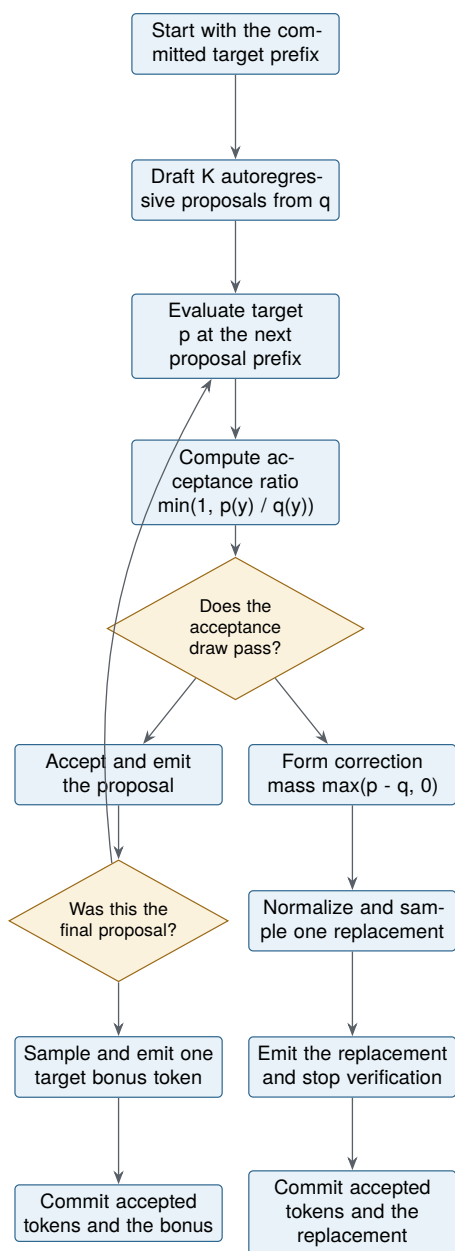


Figure 47.1: Speculative verification.

```
1 swift run leetllm check 046 --cpu --solution
```

47.8 Correctness method

The judge uses exactly enumerable two-token distributions:

- accept-all proposals followed by a target bonus;
- certain rejection with correction $[0, 1]$;

- algebraic one-step reconstruction of the target distribution;
- a high-acceptance cost case that crosses modeled break-even; and
- invalid $\kappa=0$ and vocabulary mismatch inputs.

The trace is part of correctness. It catches using the wrong prefix, skipping the bonus, continuing after rejection, or sampling from target rather than the correction distribution.

47.9 Performance model

If each proposal is independently accepted with probability a , a block of maximum length K emits an expected

$$E[N] = \sum_{i=0}^K a^i$$

tokens, including the possible bonus. The bundled modeled cost is

$$C_s = KC_d + C_v,$$

where C_d is draft cost per token and C_v is one target block-verification cost. Target-only cost for the same expected output is

$$C_t = E[N]C_t^{(1)}.$$

Modeled speedup is C_t/C_s . This assumes target verification is block-efficient. The current executable algorithm calls the target provider per proposal to expose probability semantics; it is not a measured parallel target kernel.

47.10 Metal mapping

Problem 046 is CPU-only. A useful Metal implementation would run draft decoding, then evaluate target logits for the proposed block with prefill-like matrix shapes and a causal verification mask. It must commit KV state only through accepted proposals plus the replacement or bonus token.

GPU implementation must separately measure draft execution, target block verification, rollback or cache-copy cost, command submission, and synchronization. This lesson does not claim speedup from the CPU trace implementation.

47.11 Implementation checkpoints

1. Normalize logits and probabilities with strict validation.
2. Sample exactly K draft proposals with evolving prefixes.
3. Re-evaluate target distribution at each proposal prefix.
4. Compute and trace $\min(1, p/q)$.
5. Stop at first rejection and sample normalized $[p-q]^+$.
6. Emit a bonus target token after accept-all.
7. Implement a target-only baseline.
8. Verify distribution identity before evaluating cost.

47.12 Controlled experiments

47.12.1 Acceptance sweep

For fixed $K=4$, sweep modeled acceptance from 0 to 1. Predict expected emitted tokens and break-even.

47.12.2 Draft-length sweep

Hold acceptance and costs fixed while changing K . Predict when extra draft work exceeds additional accepted output.

47.12.3 Distribution mismatch

Move q progressively away from p . Inspect acceptance ratios and correction mass, not only emitted tokens.

47.12.4 Target verification cost

Compare a hypothetical block verification cost with K independent target calls. Speculation only addresses serial target steps when block verification has a favorable shape or implementation.

47.13 Engine integration

The providers can be backed by two decoder models sharing token IDs and stop rules. Integration requires separate draft and target KV state, accepted-prefix commit logic, and exact sampler semantics. Problem 047 does not claim speculative speedup; it supplies the target-style serial baseline and report boundary a future integration would compare against.

47.14 Tradeoffs and limitations

- A poor draft lowers acceptance and can add work.
- A large K increases possible output per verification but wastes more rejected proposals.
- Draft and target must share vocabulary and tokenization semantics.
- Cache rollback and memory pressure can erase arithmetic gains.
- Greedy agreement is not a substitute for probability-correct stochastic verification.
- The cost model is conditional and modeled; no wall-clock speedup is bundled.

47.15 Hints

- Save the draft distribution for every proposal; rejection needs its q .
- Verify proposal i against the prefix before that proposal.
- Consume a fresh acceptance draw even when ratio is one; deterministic traces depend on draw order.
- Stop verification immediately after correction sampling.
- Sample the bonus from target at the full accepted draft prefix.

47.16 Completion checklist

- ☐ Accept-all emits K proposals plus one target bonus.
- ☐ First rejection emits one corrected token and stops.

- ☐ One-step output mass reconstructs the target distribution.
- ☐ Prefixes, draws, probabilities, and counts are traceable.
- ☐ Invalid dimensions and distributions are rejected.
- ☐ Break-even is derived from explicit draft and verification costs.
- ☐ Modeled speedup is never presented as measured acceleration.

47.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

47.17.1 Sources/LeetLLMExercises/P046SpeculativeDecodingExercise.swift

```

1 import LeetLLMCore
2
3 public enum P046SpeculativeDecodingExercise {
4     public static func decodeBlock(
5         prefix: [Int],
6         maximumDraftTokens: Int,
7         vocabularySize: Int,
8         draft: TokenDistributionProvider,
9         target: TokenDistributionProvider,
10        generator: inout SeededGenerator
11    ) throws -> SpeculativeBlockResult {
12        try P046SpeculativeDecodingContract.validate(
13            maximumDraftTokens: maximumDraftTokens,
14            vocabularySize: vocabularySize)
15        _ = try P046DistributionMath.normalized(
16            draft(prefix), vocabularySize: vocabularySize, modelName: "draft")
17        _ = try P046DistributionMath.normalized(
18            target(prefix), vocabularySize: vocabularySize, modelName: "target")
19        // TODO: draft K tokens, verify with min(1, p/q), correct rejection, and emit a bonus.
20        return SpeculativeBlockResult(
21            emittedTokenIDs: [],
22            proposals: [],
23            verifications: [],
24            bonus: nil,
25            rejectedAtProposalIndex: nil,
26            draftEvaluationCount: 0,
27            targetEvaluationCount: 0)
28    }
29 }

```

47.18 Worked solution

Solution strategy

Samples up to K tokens autoregressively from a draft model, then verifies each proposal against target probabilities using an acceptance ratio $\min(1, p(y)/q(y))$. On acceptance, emit the proposal and advance the prefix; on rejection, sample one corrected replacement from normalized $\max(p(y)-q(y), 0)$ and stop. If all K proposals pass, emit one bonus token from the target at the full accepted prefix. Traces record every proposal prefix, draft/target probabilities, acceptance decisions, and random draws, enabling deterministic reconstruction. The output distribution equals the target exactly: accepted mass is $\min(q,p)$, rejected mass is $q*(1-\alpha)$, and corrected mass is $\max(p-q,0)$, summing to p .

Correctness invariant

One-step output distribution equals target distribution. For every vocabulary token y , emitted probability = $\min(p(y), q(y)) + \max(p(y)-q(y), 0) = p(y)$. Traces must record drafts, targets, acceptance draws, and corrections; sampling the same draft must yield the same proposal (same draw used). Rejection stops the block immediately; bonus is sampled only after all proposals are accepted.

Common failure mode

Sampling from target instead of correction distribution after rejection, continuing verification after rejection, verifying proposal i against the wrong prefix (must use prefix before proposal i , not after), or omitting the bonus token when all proposals pass.

47.18.1 Implementation points

Acceptance Ratio and Correctness Preservation

For each proposal y sampled from q , the acceptance ratio $\min(1, p(y)/q(y))$ preserves the target distribution. Accepted mass is $q(y) * \alpha(y) = \min(q(y), p(y))$. If rejection occurs, the correction distribution is $(\max(p(y)-q(y), 0)) / \sum_z(\max(p(z)-q(z), 0))$, which restores $p(y)$ after accounting for both accepted and rejected paths. The sum of all emitted masses equals the target marginal for each token.

Rejection Correction Sampling

On rejection of proposal i , the code computes the correction distribution $(p-q)+$ normalized and samples one replacement token using the rejection distribution. This replacement is emitted, and the block terminates. The replacement must come from the correction distribution, not from the target directly, to maintain the mathematical law. Tracing the rejection distribution is necessary to verify the replacement was correctly sampled.

Evolving Proposal Prefix and Per-Proposal Verification

Draft proposals are built autoregressively: proposal 0 is sampled from prefix; proposal 1 from prefix + [proposal_0]; etc. Target verification occurs at each proposal prefix (before appending that proposal). The target distribution for proposal i is computed at prefix + [proposals_0..._{i-1}], not at the full prefix including proposal i. This ordering is critical: verify proposal i against the prefix state before it was generated.

Bonus Token After Accept-All and Cost Break-Even

If all K proposals pass verification, the code samples one additional 'bonus' token from the target given the prefix [original_prefix] + [all_K_proposals], then emits it. Expected emitted tokens $E[N] = 1 + K \cdot a + a^K$ (accept all K plus bonus with prob a^K). Modeled speedup is $\text{targetOnly}(E[N]) / (K \cdot \text{draftCost} + \text{verifyK})$, which breaks even when draft cost is low enough to amortize verification. Modeled speedup is never presented as wall-clock acceleration.

47.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P046SpeculativeDecodingSolution.swift

- Line 3: public enum P046SpeculativeDecodingSolution {
- Line 4: public static func decodeBlock(
- Line 112: public static func sampleTargetOnly(
- Line 144: private static func sample(

47.18.3 Complete canonical listings

47.18.4 Sources/LeetLLMSolutions/P046SpeculativeDecodingSolution.swift

```

1 import LeetLLMCore
2
3 public enum P046SpeculativeDecodingSolution {
4     public static func decodeBlock(
5         prefix: [Int],
6         maximumDraftTokens: Int,
7         vocabularySize: Int,
8         draft: TokenDistributionProvider,
9         target: TokenDistributionProvider,
10        generator: inout SeededGenerator
11    ) throws -> SpeculativeBlockResult {
12        try P046SpeculativeDecodingContract.validate(
13            maximumDraftTokens: maximumDraftTokens,
14            vocabularySize: vocabularySize)
15
16        var proposalPrefix = prefix
17        var proposals: [DraftProposalTrace] = []
18        for index in 0..

```

```

27     }
28     proposals.append(DraftProposalTrace(
29         index: index,
30         prefix: proposalPrefix,
31         tokenID: sampled.tokenID,
32         draftDistribution: distribution,
33         draftProbability: probability,
34         samplingDraw: sampled.draw))
35     proposalPrefix.append(sampled.tokenID)
36 }
37
38 var emitted: [Int] = []
39 var verifications: [VerificationTrace] = []
40 for proposal in proposals {
41     let targetDistribution = try P046DistributionMath.normalized(
42         target(proposal.prefix),
43         vocabularySize: vocabularySize,
44         modelName: "target")
45     let targetProbability = targetDistribution[proposal.tokenID]
46     let ratio = min(1, targetProbability / proposal.draftProbability)
47     let acceptanceDraw = generator.nextUnitInterval()
48     if acceptanceDraw < ratio {
49         emitted.append(proposal.tokenID)
50         verifications.append(VerificationTrace(
51             proposalIndex: proposal.index,
52             tokenID: proposal.tokenID,
53             targetDistribution: targetDistribution,
54             targetProbability: targetProbability,
55             draftProbability: proposal.draftProbability,
56             acceptanceRatio: ratio,
57             acceptanceDraw: acceptanceDraw,
58             accepted: true,
59             rejectionDistribution: nil,
60             replacementTokenID: nil,
61             replacementSamplingDraw: nil))
62     } else {
63         let correction = try P046DistributionMath.correction(
64             target: targetDistribution,
65             draft: proposal.draftDistribution)
66         let replacement = sample(correction, generator: &generator)
67         emitted.append(replacement.tokenID)
68         verifications.append(VerificationTrace(
69             proposalIndex: proposal.index,
70             tokenID: proposal.tokenID,
71             targetDistribution: targetDistribution,
72             targetProbability: targetProbability,
73             draftProbability: proposal.draftProbability,
74             acceptanceRatio: ratio,
75             acceptanceDraw: acceptanceDraw,
76             accepted: false,
77             rejectionDistribution: correction,
78             replacementTokenID: replacement.tokenID,
79             replacementSamplingDraw: replacement.draw))
80         return SpeculativeBlockResult(
81             emittedTokenIDs: emitted,
82             proposals: proposals,
83             verifications: verifications,
84             bonus: nil,
85             rejectedAtProposalIndex: proposal.index,
86             draftEvaluationCount: proposals.count,
87             targetEvaluationCount: verifications.count)
88     }
89 }
90
91 let bonusPrefix = prefix + proposals.map(\.tokenID)
92 let bonusDistribution = try P046DistributionMath.normalized(
93     target(bonusPrefix),
94     vocabularySize: vocabularySize,
95     modelName: "target")
96 let bonusSample = sample(bonusDistribution, generator: &generator)
97 emitted.append(bonusSample.tokenID)
98 return SpeculativeBlockResult(
99     emittedTokenIDs: emitted,

```

```

100     proposals: proposals,
101     verifications: verifications,
102     bonus: BonusTokenTrace(
103         prefix: bonusPrefix,
104         targetDistribution: bonusDistribution,
105         tokenID: bonusSample.tokenID,
106         samplingDraw: bonusSample.draw),
107     rejectedAtProposalIndex: nil,
108     draftEvaluationCount: proposals.count,
109     targetEvaluationCount: verifications.count + 1)
110 }
111
112 public static func sampleTargetOnly(
113     prefix: [Int],
114     tokenCount: Int,
115     vocabularySize: Int,
116     target: TokenDistributionProvider,
117     generator: inout SeededGenerator
118 ) throws -> TargetOnlySamplingResult {
119     guard tokenCount >= 0 else {
120         throw SpeculativeDecodingError.invalidTargetOnlyTokenCount(tokenCount)
121     }
122     guard vocabularySize > 0 else {
123         throw SpeculativeDecodingError.invalidVocabularySize(vocabularySize)
124     }
125     var current = prefix
126     var tokens: [Int] = []
127     var draws: [Double] = []
128     for _ in 0..

```

Capstone Inference Engine

48.1 Why this exists

The earlier problems are useful only if their contracts compose. This capstone takes prompt text through byte tokenization, multi-token prefill, a persistent per-layer KV cache, serial sampling and decode, byte rendering, memory planning, profiling, and named Metal parity captures.

The bundled model is deliberately small and deterministic. It has seven token IDs and synthetic Float32 weights. It proves that the course engine is connected and independently judged; it is not a pretrained language model or a production Metal inference backend.

48.2 Learning outcomes

You can:

- enforce tokenizer/model vocabulary compatibility before inference;
- run text-to-token-to-text generation through one shared model;
- explain why the cache grows for decode inputs but not the just-selected final token;
- compare cached generation with independent full-prefix recomputation;
- report measured timings and modeled memory with explicit boundaries;
- verify a real Metal subgraph without relabeling CPU generation; and
- adopt and reject optimizations from evidence rather than feature lists.

48.3 Prerequisites

- Problems 035-040 for model, tokenizer, sampling, prefill, and decode.
- Problem 041 for reusable arena plans.
- Problem 042 for named parity boundaries.
- Problems 043-044 for fused Metal verification and stage timing.
- Problems 045-046 for clearly scoped extensions not yet integrated into generation.

48.4 Vocabulary

- **Compatible vocabulary:** tokenizer IDs exactly match model logits and embedding rows.
- **Generation backend:** implementation that actually computes the complete token sequence.
- **Verification slice:** limited backend execution used for parity, not generation.
- **Full-prefix oracle:** recomputes all prior tokens without using the candidate KV cache.
- **Stop reason:** EOS selection or maximum token count.
- **Arena plan:** modeled reusable intermediate address space from Problem 041.
- **Engineering report:** outputs, timings, resources, formats, evidence, and limitations together.

48.5 Math from first principles

The capstone tokenizer maps exactly:

```
1 0 EOS 1 BOS 2 a 3 b 4 c 5 space 6 period
```

For prompt `ab c.`, encoding with BOS is `[1,2,3,5,4,6]`. Every ID is in the model vocabulary `0..<7`.

Prefill computes logits after the final prompt position and populates every layer cache with S entries. Sampling selects the first generated token y_0 . That selected token is not in the cache until it becomes the input to the next decode call. After G generated tokens,

$$T_{cache} = S + \max(0, G - 1).$$

Each subsequent step computes

$$y_i \sim \text{Sample}(\text{Decoder}(y_{i-1}, K, V), \text{seed state}).$$

The full-prefix oracle instead recomputes logits for `[prompt,y0,...,y(i-1)]` from scratch using the independent Double-accumulation reference. Equal sampled IDs demonstrate cache semantics, not merely repeated determinism.

48.6 Shape, layout, and dtype contract

- Prompt bytes are restricted to ASCII `a, b, c, space`, and `period`. Other bytes fail rather than map to an unknown model ID.
- Tokenizer IDs are exactly `0..6`; BOS is 1 and EOS is 0.
- Model tensors are finite Float32 row-major with decoder projections `[output,input]`.
- KV cache is Float32 contiguous `[layer,position,KV head,feature]` storage.
- `maxNewTokens` is nonnegative; seed is UInt64.
- Generated IDs must remain in vocabulary and bytes must decode from those exact IDs.
- CPU and Metal capture comparison uses combined absolute/relative Float32 tolerance.

48.7 CPU reference path

The canonical execution validates compatibility, encodes prompt bytes, allocates one cache, and runs `MiniDecoderCPUEngine.prefill`. It samples the first token from prefill logits, then repeatedly passes the previous selected token into `MiniDecoderCPUEngine.decode` with monotonically increasing absolute position.

Generated IDs are decoded while skipping BOS/EOS. Valid UTF-8 is printed as text; otherwise the report uses hexadecimal bytes. The report includes each stage time, final cache counts, weight bytes, allocated cache bytes, prefill/decode arena plans, formats, and limitations.

```
1 swift run -c release leetllm capstone --prompt "ab c." --max-tokens 4 --seed 47
```

48.8 Correctness method

The capstone judge checks six independent concerns:

1. Exact restricted-byte tokenization and deterministic generation.
2. Cached Float32 decode against Double full-prefix recomputation on every step.
3. Cache growth and complete report invariants.
4. Explicit CPU backend and educational-model limitations.
5. Rejection of unsupported prompt bytes.
6. Rejection of incompatible tokenizer vocabularies.

Focused tests also run the actual Metal verification slice and require five named parity captures and three dispatches.

```
1 swift run leetllm check 047 --cpu --solution
2 swift test --filter P047CapstoneMetalVerificationTests
```

48.9 Performance model

Real monotonic measurements include:

- prefill engine time;
- first-token sampling in TTFT; and
- each serial decode engine call.

Modeled memory includes exact Float32 model tensor bytes, allocated KV capacity, and first-fit arena sizes. The optimization comparison uses Problem 043's logical dispatch and tensor-byte model for layer-0 RMSNorm plus Q/K/V.

These categories remain separate. Arena bytes are not measured resident memory because Swift tensors do not allocate from that modeled arena. Metal resource bytes are actual buffer sizes requested by the verification pipeline, but they are not process peak RSS.

48.10 Metal mapping

Generation runs on the CPU reference backend. When Metal is available, the capstone takes the real layer-0 residual from prefill and executes:

1. one fused RMSNorm/QKV dispatch;
2. one RoPE query dispatch; and
3. one RoPE key dispatch.

It compares these captures:

```
1 layer.0.fused_qkv.query
2 layer.0.fused_qkv.key
3 layer.0.fused_qkv.value
4 layer.0.rope.query
5 layer.0.rope.key
```

The report records three command buffers, three synchronous host waits, per-call allocated shared buffers, and transfer bytes. Attention, output projection, MLP, cache updates, logits, and sampling remain on CPU. The report therefore says “Metal verification slice,” never “Metal inference backend.”

48.11 Implementation checkpoints

1. Define an explicit tokenizer whose IDs exactly cover model vocabulary.

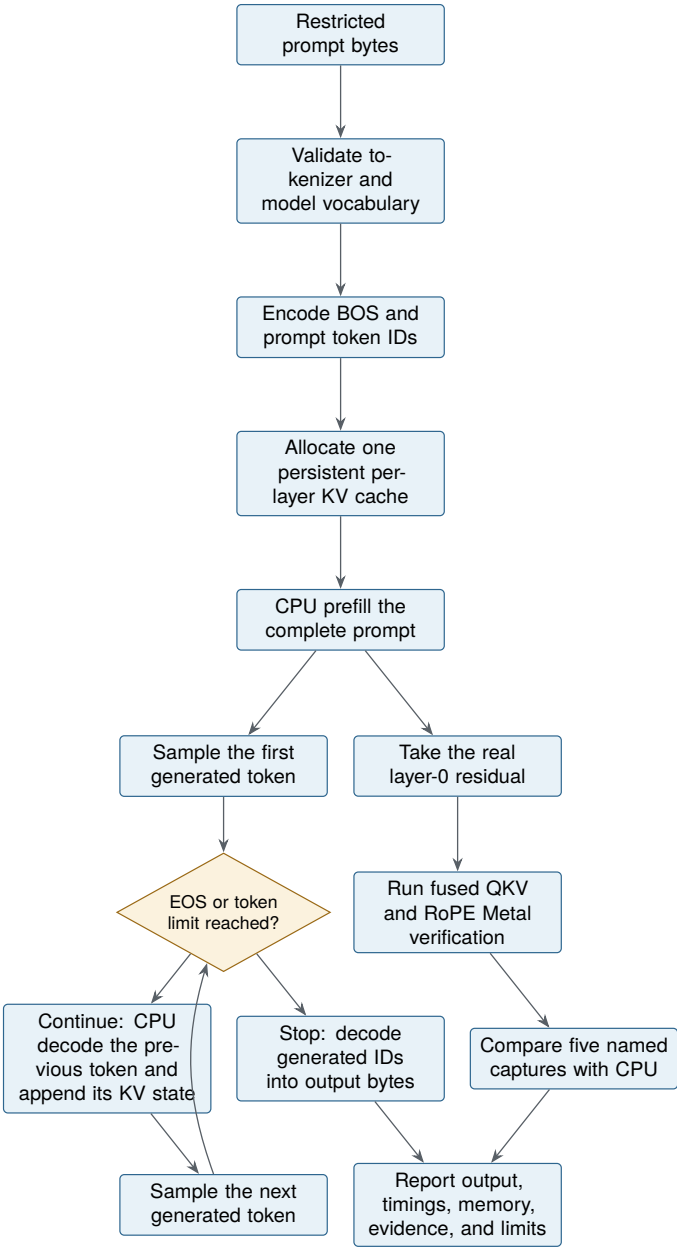


Figure 48.1: Inference engine flow.

2. Round-trip prompt bytes before model execution.
3. Prefill one persistent cache and sample the first token.
4. Decode serially with correct token and logical-position ordering.
5. Render generated bytes without assuming UTF-8.
6. Match a full-prefix independent oracle.
7. Derive cache counts and memory fields.
8. Execute and label the Metal verification slice.
9. Record one adopted and one rejected optimization with evidence.
10. Print limitations beside results.

48.12 Controlled experiments

48.12.1 Prompt and seed repeatability

Run the same prompt and seed twice. IDs and bytes must match; wall-clock samples need not.

48.12.2 Decode-length sweep

Use max-tokens 1, 2, 4, and 8. Predict cache counts from $S + G - 1$ before running.

48.12.3 Metal boundary

Run once normally and once with `--no-metal`. Generated IDs must match. Only verification captures and resource evidence should disappear.

48.12.4 Arena comparison

Inspect prefill and decode arena bytes versus naive temporary bytes in code. State why this is modeled allocation evidence rather than current process memory reduction.

48.12.5 Unsupported input

Try a byte outside the five-byte prompt alphabet. Predict the exact boundary at which execution fails: tokenizer compatibility before model execution.

48.13 Engine integration

This is the cumulative educational engine. It integrates model data, tokenization, sampling, prefill, cached decode, memory planning, profiling, and a verified Metal subgraph. Problems 045 and 046 remain separately judged systems prototypes. Integrating them next would require batched model execution and separate draft/target model state, not merely calling their simulators from the capstone.

A real-model extension also requires a compatible production checkpoint loader, complete tokenizer vocabulary and merges, model-specific conventions, external reference captures, and rights to distribute the weights.

48.14 Tradeoffs and limitations

- The seven-token fixture is connected but not linguistically meaningful.
- Restricted bytes make vocabulary compatibility explicit at the cost of general prompts.

- Float32 CPU execution is readable but not a throughput implementation.
- The Metal slice proves local numerical parity, not full-device generation.
- Per-call Metal allocation and waits are intentionally reported and currently rejected as a backend design.
- Wall-clock results are machine- and run-specific.
- Modeled arena and logical traffic values are not resident-memory or hardware-counter observations.

48.15 Hints

- Sample the first token from prefill logits before any decode call.
- Append a generated token to cache only when it becomes the next decode input.
- Keep one seeded generator across the complete session.
- Decode bytes from token IDs, not from debug token labels.
- Make backend labels describe execution, not aspiration.
- Compare the earliest Metal capture before inspecting later values.

48.16 Completion checklist

- ☐ Prompt bytes round-trip through a model-compatible tokenizer.
- ☐ Cached generation matches the full-prefix independent oracle.
- ☐ Cache growth matches $S + \max(0, G-1)$ at every layer.
- ☐ Generated IDs, bytes, rendering, seed, and stop reason are reported.
- ☐ Prefill, TTFT, and serial decode boundaries are explicit.
- ☐ Weight, cache, and arena bytes are labeled by evidence type.
- ☐ Five Metal captures pass when a device is available.
- ☐ CPU generation is never labeled Metal inference.
- ☐ One optimization is adopted and one rejected with evidence.
- ☐ Educational-model and real-model limitations are stated.

48.17 Exercise source files

Exercise

These are the complete chapter-owned starter files. Work in the repository copies so the shared judge, SwiftPM build, and Metal host pipeline remain authoritative. Placeholder bodies and deliberately incomplete kernels are part of the exercise.

48.17.1 Sources/LeetLLMExercises/P047CapstoneExercise.swift

```

1 import LeetLLMCore
2
3 public enum P047CapstoneExercise {
4     public static func run(_ request: CapstoneRequest) throws -> CapstoneReport {
5         try P047CapstoneContract.validate(request)
6         // TODO: connect tokenizer, prefill, KV cache, serial decode, sampling, rendering, and report.
7         return CapstoneReport(
8             prompt: request.prompt,
9             promptTokenIDs: [],
10            generatedTokenIDs: [],
11            generatedBytes: [],
12            rendering: .text(""),
13            stopReason: .maximumTokenCount,
14            timings: [],
15            timeToFirstTokenNanoseconds: nil,
16            decodeTokensPerSecond: nil,
17            finalCacheCounts: [],
18            modelWeightBytes: 0,
19            allocatedKVCacheBytes: 0,
20            prefillArenaBytes: 0,
21            decodeArenaBytes: 0,
22            generationBackend: P047CapstoneContract.generationBackend,
23            weightFormat: "Float32 row-major",
24            keyValueFormat: "Float32 contiguous KV cache",
25            metalVerification: CapstoneMetalVerification(
26                label: P047CapstoneContract.metalVerificationLabel,
27                status: .notRequested,
28                captures: [],
29                resources: nil),
30            optimizationComparison: CapstoneOptimizationComparison(
31                name: "not implemented",
32                baselineDispatchCount: 1,
33                optimizedDispatchCount: 1,
34                baselineLogicalBytes: 0,
35                optimizedLogicalBytes: 0,
36                basis: "not implemented"),
37            rejectedOptimization: CapstoneRejectedOptimization(
38                name: "not implemented", evidence: "not implemented"),
39            limitations: [])
40     }
41 }

```

48.18 Worked solution

Solution strategy

Integrates tokenization, prefill, persistent KV cache, and serial decode with optional Metal verification. Restricted-ASCII tokenizer encodes prompt bytes (a, b, c, space, period) to token IDs 1-6 (BOS=1, EOS=0). Tokenizer and model vocabulary are validated for exact compatibility. Prefill processes all prompt tokens in one batch, populating every layer's cache with S entries. Decode processes one token at a time: the previously selected token is embedded, passed through all layers (appending its KV state to cache when selected), sampled, and checked against EOS. Cache grows by G-1 tokens (each selected token appends KV state only when it becomes the next input). A full-prefix independent oracle (Double accumulation) recomputes all prior tokens to verify cache semantics. Metal verification slice executes layer-0 fused-QKV and RoPE on the real prefill residual, producing five named captures (query, key, value after fused QKV; query, key after RoPE), and compares with CPU using parity tolerance. Report separates CPU generation from Metal verification, labels all boundaries and resource costs, and records one adopted (fusion cost model) and one rejected (labeling verification as a backend) optimization.

Correctness invariant

Cached generation matches full-prefix independent oracle on every decode step. Cache count at layer L after G generated tokens is $\text{promptLength} + \max(0, G-1)$. Generated token IDs remain in vocabulary 0..6. Bytes decode from those IDs round-trip through tokenizer. Metal captures pass parity tolerance: $\text{abs}(\text{metalValue} - \text{cpuValue}) \leq 5e-5 + 1e-4 * \max(|\text{metal}|, |\text{cpu}|)$. Metal is labeling only; it does not own attention, MLP, sampling, or full generation.

Common failure mode

Appending a selected token's KV state to cache before it becomes an input (over-incrementing cache count). Labeling the Metal verification slice as a complete backend. Assuming modeled arena bytes are current process resident memory. Mixing tokenizer vocabulary (0..6) with unsupported input bytes.

48.18.1 Implementation points

Tokenizer-Model Vocabulary Compatibility Check

Prompt bytes must be in {a, b, c, space, period} (ASCII 97, 98, 99, 32, 46); other bytes throw `tokenizerRoundTripMismatch`. BOS is token 1, EOS is 0, and content tokens map to 2-6. The tokenizer and model must agree exactly: any mismatch in ID-to-byte mapping fails before model execution. This explicit restriction exposes vocabulary compatibility as a contract requirement, not a silent mismatch.

Cache Growth and Selected Token Appending

Prefill appends S entries (one per prompt token, all sampled in one batch). Decode appends one entry per step, but only when that token is selected and becomes the next input. After G generated tokens, cache size is $S + \max(0, G-1)$, not $S + G$. If `maxNewTokens` is 0, cache remains at size S. This invariant is verified by computing `finalCacheCounts` and comparing with $S + \max(0, |\text{generated}|-1)$

per layer.

Full-Prefix Independent Oracle and Parity

The PromptPrefillResult includes a full-prefix oracle trace at each layer. On every decode step, the capstone compares the cached KV state (read from cache) against a fresh computation of all prior tokens using Double-precision accumulation. If they differ, the cache semantic is broken. Metal parity is checked similarly: captured Metal values against CPU values using combined tolerance, element-wise. Parity passes imply the kernel logic is correct for this input; it does not imply production deployment.

Arena Modeled vs. Measured and Metal Resource Labeling

Prefill and decode arena bytes are modeled first-fit allocations for reusable intermediates (e.g., attention scores, MLP hidden states), derived from P041BufferPlanningSolution.compareDecoderPlans. These are not current process RSS. Metal verification resources include allocatedBufferBytes (shared buffers per dispatch), hostToDeviceBytes, deviceToHostBytes, and dispatch count. The report rejects the optimization 'label verification as backend' because only CPU executes attention, MLP, sampling, and cache management; Metal is a parity slice only.

48.18.2 Code map

How to read the listing

Use this map to connect declarations to the strategy above before reading each full listing. Line numbers refer to the generated listing's source file.

Sources/LeetLLMSolutions/P047CapstoneSolution.swift

- Line 5: public enum P047CapstoneSolution {
- Line 6: public static func run(_ request: CapstoneRequest) throws -> CapstoneReport {
- Line 154: private static func verifyMetalIfRequested()
- Line 215: private static func parity()
- Line 231: private static func modelWeightBytes(_ model: MiniDecoderModel) -> Int {
- Line 245: private static func positiveElapsed(since start: UInt64) -> UInt64 {

48.18.3 Complete canonical listings

48.18.4 Sources/LeetLLMSolutions/P047CapstoneSolution.swift

```

1 import Dispatch
2 import LeetLLMCore
3 import Metal
4
5 public enum P047CapstoneSolution {
6     public static func run(_ request: CapstoneRequest) throws -> CapstoneReport {
7         try P047CapstoneContract.validate(request)
8         let promptTokenIDs = try P037ByteBPESolution.encode(
9             tokenizer: request.tokenizer,
10            text: request.prompt,
11            options: BPEEncodingOptions(addBeginningOfSequence: true))
12         try request.model.validate(tokenIDs: promptTokenIDs)
13         let roundTrip = try P037ByteBPESolution.decodeBytes(
14             tokenizer: request.tokenizer,
15             tokenIDs: promptTokenIDs,
16             skipSpecialTokens: true)

```

```

17 guard roundTrip == Array(request.prompt.utf8) else {
18     throw CapstoneError.tokenizerRoundTripMismatch
19 }
20
21 let capacity = promptTokenIDs.count + max(1, request.maxNewTokens)
22 let cache = ContiguousKVCache(
23     configuration: try request.model.cacheConfiguration(capacity: capacity))
24 var generator = SeededGenerator(seed: request.seed)
25 let prefillStart = DispatchTime.now().uptimeNanoseconds
26 let prefill = try MiniDecoderCPUEngine.prefill(
27     PromptPrefillRequest(model: request.model, tokenIDs: promptTokenIDs),
28     cache: cache)
29 let prefillNanoseconds = positiveElapsed(since: prefillStart)
30 var timings = [CapstoneStageTiming(name: "prefill.engine", nanoseconds: prefillNanoseconds)]
31 var generated: [Int] = []
32 var stopReason = GenerationStopReason.maximumTokenCount
33 var timeToFirstToken: UInt64?
34
35 if request.maxNewTokens > 0 {
36     let firstStart = DispatchTime.now().uptimeNanoseconds
37     let first = try P038LogitsSamplingSolution.sample(
38         logits: prefill.logits.storage,
39         strategy: request.samplingStrategy,
40         generator: &generator)
41     let firstSamplingNanoseconds = positiveElapsed(since: firstStart)
42     generated.append(first.selectedToken)
43     timeToFirstToken = prefillNanoseconds + firstSamplingNanoseconds
44     if first.selectedToken == request.tokenizer.endOfSequenceTokenID {
45         stopReason = .endOfSequence
46     }
47 }
48
49 while generated.count < request.maxNewTokens, stopReason != .endOfSequence {
50     let stepIndex = generated.count
51     let start = DispatchTime.now().uptimeNanoseconds
52     let result = try MiniDecoderCPUEngine.decode(
53         AutoregressiveDecodeRequest(
54             model: request.model,
55             tokenID: generated.last!,
56             logicalPosition: promptTokenIDs.count + stepIndex - 1,
57             samplingStrategy: request.samplingStrategy),
58         cache: cache,
59         generator: &generator)
60     let elapsed = positiveElapsed(since: start)
61     timings.append(CapstoneStageTiming(
62         name: "decode.token.\(stepIndex)", nanoseconds: elapsed))
63     generated.append(result.selectedNextTokenID)
64     if result.selectedNextTokenID == request.tokenizer.endOfSequenceTokenID {
65         stopReason = .endOfSequence
66     }
67 }
68
69 let generatedBytes = try P037ByteBPESolution.decodeBytes(
70     tokenizer: request.tokenizer,
71     tokenIDs: generated,
72     skipSpecialTokens: true)
73 let rendering: GeneratedRendering
74 if let text = String(bytes: generatedBytes, encoding: .utf8) {
75     rendering = .text(text)
76 } else {
77     rendering = .hexadecimal(generatedBytes.map { String(format: "%02x", $0) }.joined())
78 }
79
80 let finalCacheCounts = try (0..

```

```

90     input: layerZeroInput,
91     gamma: weights.attentionNormGamma,
92     queryWeights: weights.queryWeights,
93     keyWeights: weights.keyWeights,
94     valueWeights: weights.valueWeights,
95     epsilon: request.model.configuration.rmsNormEpsilon,
96     configuration: request.model.configuration)
97 let fusionCost = try P043FusedQKVCostModel.compare(fusedRequest)
98 let metalVerification = try verifyMetalIfRequested(
99     request: request,
100    fusedRequest: fusedRequest)
101 let decodeDurations = timings.filter { $0.name.hasPrefix("decode.token.") }
102 let decodeRate: Double? = decodeDurations.isEmpty ? nil :
103     Double(decodeDurations.count) * 1_000_000_000
104     / Double(decodeDurations.reduce(0) { $0 + $1.nanoseconds })
105 let rejectionEvidence: String
106 if let resources = metalVerification.resources {
107     rejectionEvidence =
108         "The verification slice performs \(resources.commandBufferCount) command buffers,
109         ↪ \(resources.allocatedBufferBytes) bytes of per-call buffer allocation, \(resources.hostToDeviceBytes)
110         ↪ host-to-device bytes, \(resources.deviceToHostBytes) device-to-host bytes, and \(resources.hostWaitCount)
111         ↪ synchronous waits; attention, MLP, sampling, and cache execution remain on CPU."
112 } else {
113     rejectionEvidence =
114         "Only the CPU engine executes attention, MLP, sampling, and cache updates; no completed Metal verification was
115         ↪ available to support labeling this a Metal inference backend."
116 }
117
118 let report = CapstoneReport(
119     prompt: request.prompt,
120     promptTokenIDs: promptTokenIDs,
121     generatedTokenIDs: generated,
122     generatedBytes: generatedBytes,
123     rendering: rendering,
124     stopReason: stopReason,
125     timings: timings,
126     timeToFirstTokenNanoseconds: timeToFirstToken,
127     decodeTokensPerSecond: decodeRate,
128     finalCacheCounts: finalCacheCounts,
129     modelWeightBytes: modelWeightBytes(request.model),
130     allocatedKVCacheBytes: cache.allocatedBytes,
131     prefillArenaBytes: arena.prefill.arenaByteCount,
132     decodeArenaBytes: arena.decode.arenaByteCount,
133     generationBackend: P047CapstoneContract.generationBackend,
134     weightFormat: "Float32 row-major",
135     keyValueFormat: "Float32 contiguous KV cache",
136     metalVerification: metalVerification,
137     optimizationComparison: CapstoneOptimizationComparison(
138         name: "RMSNorm + Q/K/V fusion",
139         baselineDispatchCount: fusionCost.separate.dispatchCount,
140         optimizedDispatchCount: fusionCost.fused.dispatchCount,
141         baselineLogicalBytes: fusionCost.separate.logicalTensorBytes,
142         optimizedLogicalBytes: fusionCost.fused.logicalTensorBytes,
143         basis: "Modeled logical tensor traffic and dispatches for layer 0 over this prompt; not a wall-clock speedup
144         ↪ claim."),
145         rejectedOptimization: CapstoneRejectedOptimization(
146             name: "Label the verification slice as a Metal inference backend",
147             evidence: rejectionEvidence),
148         limitations: [
149             "The seven-token tokenizer is restricted to the bytes a, b, c, space, and period plus BOS/EOS.",
150             "The deterministic educational fixture is not pretrained and its output is not a language-quality
151             ↪ demonstration.",
152             "Generation is batch size one on the CPU reference backend.",
153             "Metal executes only fused QKV and RoPE verification captures, not the full decoder.",
154             "Wall-clock timings are machine- and run-specific and include Swift tensor allocation overhead.",
155         ])
156     try P047CapstoneContract.validate(report, for: request)
157     return report
158 }
159
160 private static func verifyMetalIfRequested(
161     request: CapstoneRequest,

```



```

156     fusedRequest: FusedQKVRequest
157 ) throws -> CapstoneMetalVerification {
158     guard request.includeMetalVerification else {
159         return CapstoneMetalVerification(
160             label: P047CapstoneContract.metalVerificationLabel,
161             status: .notRequested,
162             captures: [],
163             resources: nil)
164     }
165     guard MTLCreateSystemDefaultDevice() != nil else {
166         return CapstoneMetalVerification(
167             label: P047CapstoneContract.metalVerificationLabel,
168             status: .unavailable("No Metal device is available."),
169             captures: [],
170             resources: nil)
171     }
172
173     let cpuFused = try P043FusedQKVSolution.fused(fusedRequest)
174     let fusedPipeline = try P043FusedQKVSolution.makeMetalPipeline()
175     let metalFused = try fusedPipeline.run(fusedRequest)
176     let configuration = request.model.configuration
177     let cpuRoPE = try P015RoPESolution.apply(
178         queries: cpuFused.queries,
179         keys: cpuFused.keys,
180         rotaryDimension: configuration.rotaryDimension,
181         base: configuration.ropeBase,
182         queryPositionOffset: 0,
183         keyPositionOffset: 0)
184     let ropePipeline = try P015RoPESolution.makeMetalPipeline()
185     let metalRoPE = try ropePipeline.apply(
186         metalFused.result.queries,
187         metalFused.result.keys,
188         rotaryDimension: configuration.rotaryDimension,
189         base: configuration.ropeBase,
190         queryPositionOffset: 0,
191         keyPositionOffset: 0)
192     let captures = [
193         parity(name: "layer.0.fused_qkv.query", metalFused.result.queries, cpuFused.queries),
194         parity(name: "layer.0.fused_qkv.key", metalFused.result.keys, cpuFused.keys),
195         parity(name: "layer.0.fused_qkv.value", metalFused.result.values, cpuFused.values),
196         parity(name: "layer.0.rope.query", metalRoPE.queries, cpuRoPE.queries),
197         parity(name: "layer.0.rope.key", metalRoPE.keys, cpuRoPE.keys),
198     ]
199     let queryBytes = cpuFused.queries.elementCount * MemoryLayout<Float>.stride
200     let keyBytes = cpuFused.keys.elementCount * MemoryLayout<Float>.stride
201     let ropeAllocated = 2 * queryBytes + 2 * keyBytes
202     return CapstoneMetalVerification(
203         label: P047CapstoneContract.metalVerificationLabel,
204         status: .completed,
205         captures: captures,
206         resources: MetalVerificationResources(
207             allocatedBufferBytes: metalFused.allocatedBufferBytes + ropeAllocated,
208             hostToDeviceBytes: metalFused.hostToDeviceBytes + ropeAllocated,
209             deviceToHostBytes: metalFused.deviceToHostBytes + queryBytes + keyBytes,
210             dispatchCount: metalFused.dispatchCount + 2,
211             commandBufferCount: metalFused.commandBufferCount + 2,
212             hostWaitCount: metalFused.hostWaitCount + 2))
213 }
214
215 private static func parity(
216     name: String,
217     _ actual: FloatTensor,
218     _ expected: FloatTensor
219 ) -> CapstoneParityCapture {
220     let maximum = zip(actual.storage, expected.storage)
221         .map { abs(Double($0) - Double($1)) }
222         .max() ?? 0
223     let passes = actual.shape == expected.shape
224         && zip(actual.storage, expected.storage).allSatisfy { actual, expected in
225             abs(actual - expected) <= 5e-5 + 1e-4 * max(abs(actual), abs(expected))
226         }
227     return CapstoneParityCapture(
228         name: name, maximumAbsoluteError: maximum, passes: passes)

```

```
229 }
230
231 private static func modelWeightBytes(_ model: MiniDecoderModel) -> Int {
232     let elements = model.tokenEmbedding.elementCount
233     + model.finalNormGamma.elementCount
234     + model.blocks.reduce(0) { partial, block in
235         partial + block.attentionNormGamma.elementCount + block.queryWeights.elementCount
236         + block.keyWeights.elementCount + block.valueWeights.elementCount
237         + block.attentionOutputWeights.elementCount + block.mlpNormGamma.elementCount
238         + block.gateWeights.elementCount + block.upWeights.elementCount
239         + block.downWeights.elementCount
240     }
241     + (model.outputProjection == .tiedEmbedding ? 0 : model.outputWeights.elementCount)
242     return elements * MemoryLayout<Float>.stride
243 }
244
245 private static func positiveElapsed(since start: UInt64) -> UInt64 {
246     max(1, DispatchTime.now().uptimeNanoseconds - start)
247 }
248 }
```

Part IX

Reference

Anatomy of One Generated Token

This reading names the entire machine before the course builds each part. The goal is to make every later problem locatable in one execution trace.

A large language model receives **tokens**, integer IDs representing pieces of text or special markers, and predicts one next token at a time. **Inference** is the act of running the model's already learned weights to make those predictions. For a slower introduction to this vocabulary and the intended learning workflow, begin with [Start Here](#).

A.1 Symbols

Symbol	Meaning
S	Prompt sequence length
T	Tokens currently held in the KV cache
D	Model or residual-stream width
L	Number of transformer layers
H_q	Number of query heads
H_{kv}	Number of key/value heads
d_h	Width of one attention head
D_{ff}	Hidden width of the feed-forward network
V	Vocabulary size

Scope: every example below uses batch size one: one generation request at a time. Batching is introduced in Problem 045, after the single-request memory and execution behavior is visible.

A.2 Inference has two distinct stages

Prefill processes the complete input prompt together. **Decode** then processes one newly generated token per step, because each step depends on the token selected by the preceding step. Prefill exposes parallel work across prompt positions; decode is serial across generated tokens and repeatedly reuses model weights and saved context. Their different shapes lead to different performance bottlenecks and often different kernels.

A.2.1 Prefill

Prefill processes the prompt tokens. Its important shapes include a sequence axis:

1 token IDs	[S]
2 embeddings	[S, D]
3 Q	[S, H _q , d _h]
4 K, V	[S, H _{kv} , d _h]
5 MLP intermediate	[S, D _{ff}]

An **attention head** is one learned subspace in which positions compare and exchange information. Each head receives three learned projections of the current token representations: a **query (Q)** describes what a position seeks, a **key (K)** describes how a context position can match, and a **value (V)** contains the information that matching position contributes. **Attention** scores query-key matches, normalizes those scores into weights, and uses the weights to mix values.

Linear layers can use matrix-matrix multiplication (**GEMM**, general matrix- matrix multiply) because many token rows are available together. This creates reuse and parallel work. A **causal mask** prevents a prompt position from attending to later positions, which would reveal tokens that are still in its future.

Prefill produces two outputs:

1. The final hidden state used to choose the first generated token.
2. A K and V entry for every prompt position at every layer, stored in the **KV cache**. Decode reads these saved entries instead of recomputing prior positions at every generation step.

A.2.2 Decode

Decode processes one newly selected token, then repeats serially:

```

1 new token ID          [1]
2 embedding              [D]
3 Q                     [H_q, d_h]
4 new K, V               [H_kv, d_h]
5 cached K, V            [T, H_kv, d_h]
```

The large projections are now matrix-vector operations (**GEMV**, general matrix-vector multiply). The current query must read keys and values from prior positions. Weights are reused across generated tokens, but if they do not remain in a sufficiently fast hardware cache, substantial weight data is read again for each token. The KV cache grows with context and is also read repeatedly.

That is why “inference speed” is not one property. Prefill and decode have different shapes, reuse, bottlenecks, latency measures, and useful kernels.

A.3 One decoder layer

A common pre-normalized decoder layer follows this dataflow. Exact ordering, biases, normalization conventions, head counts, and activation details are model-specific and must be read from the chosen architecture.

```

1 residual input x
2 |
3 +--> RMSNorm --> Q/K/V projections --> RoPE --> attention
4 |                                     --> output projection
5 |
6 +-----+-----+ residual add
7 |                                     |
8 |                                     v
9 |                                     residual y
10 |
11 +--> RMSNorm --> gate/up projections --> SwiGLU --> down projection
12 |
13 +-----+-----+ residual add
14 |                                     |
15 |                                     v
16 |                                     layer output
```

This is repeated L times. The **residual stream** is the main token representation that flows through the model. Each attention or feed-forward branch computes a change and adds it back to that stream, preserving earlier information while each layer refines it.

A.4 Step-by-step decode trace

A.4.1 1. Token embedding

An **embedding table** stores one learned vector for every vocabulary item. The latest token ID selects one row from its shape $[V, D]$, yielding the initial residual vector $\mathbf{x} \in \mathbb{R}^D$.

A.4.2 2. Attention normalization

RMSNorm, or root-mean-square normalization, computes a scale from the D features and uses it to keep the vector's magnitude controlled. This contains a reduction followed by an elementwise pass.

A.4.3 3. Q/K/V projection

The normalized vector is multiplied by learned weight matrices to produce Q, K, and V. The query represents what the current position is looking for; each key represents how a position can be matched; and each value holds the content retrieved when that position receives weight. These names describe roles in the attention calculation, not literal text questions or database records. During one-token decode the projections are GEMV-shaped operations. Their outputs are viewed as heads rather than one flat feature axis.

A.4.4 4. Position encoding with RoPE

RoPE, or rotary position embeddings, rotates feature pairs in the new query and key according to the current token position. The resulting query-key dot products carry relative-position information. The value is not rotated.

A.4.5 5. KV-cache append

The new key and value are written to this layer's cache. Prior K/V entries are not recomputed. Cache layout determines whether later attention reads are contiguous and whether appends require copying or reallocation.

A.4.6 6. Scaled dot-product attention

Each query head computes dot products against allowed cached keys and divides the scores by the square root of the head width. This complete operation is called **scaled dot-product attention**. A **stable softmax** subtracts the largest score before exponentiation, avoiding numerical overflow, and normalizes the scores into nonnegative weights that sum to one. Those weights combine the cached values.

With **multi-head attention (MHA)**, each query head has distinct K/V heads. **Multi-query attention (MQA)** shares one K/V head across query heads. **Grouped-query attention (GQA)** shares each K/V head across a group of query heads. Sharing lowers KV storage and read traffic, but it is an architectural choice established by the model's trained weights, not a runtime switch that preserves an arbitrary model exactly.

A basic implementation materializes scores. An online implementation updates the softmax statistics and weighted value as keys arrive. A tiled, Flash-style implementation keeps score tiles and partial reductions in fast memory to avoid writing the full score matrix. These approaches target the same mathematical result with different memory behavior.

A.4.7 7. Attention output and residual

Head outputs are joined, projected back to width D , and added to the incoming residual stream.

A.4.8 8. MLP normalization and SwiGLU

A second normalization feeds gate and up projections. SiLU is applied to the gate, multiplied elementwise by the up projection, then projected from D_{ff} back to D . The result is added to the residual stream.

A.4.9 9. Repeat layers

Steps 2 through 8 run for every decoder layer. Each layer owns learned weights and its own section of the KV cache.

A.4.10 10. Final normalization and logits

After the last layer, a final normalization is followed by an output projection to $[V]$ **logits**: unnormalized scores, one for every vocabulary item. Some architectures tie this projection to the embedding table; others do not.

A.4.11 11. Sampling

The runtime transforms or filters logits according to the selected sampling policy, such as greedy selection, temperature, top-k, or top-p, and chooses the next token ID. Sampling changes selection behavior; it does not alter the transformer computation that produced the logits.

A.4.12 12. Detokenization and repeat

The chosen token is converted back to bytes or text as permitted by the tokenizer. Its ID becomes the input to the next decode step, and the KV cache now has one more position.

A.5 The two dominant memory budgets

A.5.1 Weights

Ignoring format metadata, a rough lower bound is

$$\text{weight bytes} \approx \text{parameter count} \times \frac{\text{bits per stored weight}}{8}.$$

Quantized formats also store scales and sometimes zero points. Runtime buffers, alignment, mappings, and dequantized copies can increase resident memory.

A.5.2 KV cache

For one sequence, a basic cache budget is

$$\text{KV bytes} = 2LTH_{kv}d_hb.$$

The factor 2 is for K and V. This formula makes several tradeoffs concrete: longer context increases memory linearly; MQA/GQA reduce H_{kv} ; lower-precision cache storage reduces b ; sliding windows bound effective T .

A.6 Questions to carry through the course

For every operation, ask:

1. What are the exact input and output shapes in prefill and decode?
2. Which values are reused, and where can they remain resident?
3. What is the arithmetic intensity: how many useful floating-point operations (FLOPs) are performed per byte moved?
4. Which intermediates are materialized only because operators are separate?
5. Which reduction order and dtype define acceptable numerical behavior?
6. Is a proposed optimization reducing computation, memory traffic, allocation, launch overhead, or synchronization?
7. Does the measurement isolate that cost?

The rest of LeetLLM turns each question into an executable problem.

Math Primer

This is a just-in-time reference, not an entrance exam. Read the section named by the current problem and work its small example before coding.

B.1 Notation and shapes

A scalar is one number, such as $x \in \mathbb{R}$. A vector is an ordered list of numbers, such as $\mathbf{x} \in \mathbb{R}^D$. A matrix has rows and columns, such as $W \in \mathbb{R}^{M \times K}$. A tensor generalizes this to more axes.

In this course, brackets describe shapes:

```
1 x: [D]
2 W: [M, D]
3 tokens: [S, D]
4 Q: [S, H_q, d_h]
5 K, V: [S, H_kv, d_h]
```

The shape is logical. A layout additionally says how indices map to adjacent memory. A dtype says how each element is represented and how many bytes it occupies. Kernel correctness depends on all three.

B.2 Dot product

For equal-length vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^N$,

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=0}^{N-1} x_i y_i.$$

For $\mathbf{x} = [1, 2, 3]$ and $\mathbf{y} = [4, -1, 2]$, the result is $4 - 2 + 6 = 8$.

There are N multiplications and approximately N additions. In common performance accounting this is about $2N$ floating-point operations, or FLOPs.

B.3 Matrix-vector multiplication

For $W \in \mathbb{R}^{M \times K}$ and $\mathbf{x} \in \mathbb{R}^K$,

$$\mathbf{y} = W\mathbf{x}, \quad y_m = \sum_{k=0}^{K-1} W_{m,k} x_k.$$

Each output is a dot product between one row of W and $\mathbf{x}$. A single-token projection in decode commonly has this shape.

B.4 Matrix-matrix multiplication

For $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$,

$$C = AB, \quad C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

The inner dimensions must match. Unlike GEMV, a well-tiled GEMM can reuse values from both inputs across many output elements. Prompt prefill creates these larger matrix shapes.

B.5 Linear layers

A dense neural-network layer is

$$\mathbf{y} = W\mathbf{x} + \mathbf{b}.$$

Some transformer projections omit the bias. The model architecture, not a generic formula, determines whether it exists and how weights are laid out.

B.6 Elementwise activations and gates

ReLU is

$$\text{ReLU}(x) = \max(0, x).$$

The sigmoid and SiLU functions are

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \text{SiLU}(x) = x\sigma(x).$$

A common SwiGLU feed-forward form is

$$\text{MLP}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}W_g) \odot (\mathbf{x}W_u)) W_d,$$

where $\odot$ is elementwise multiplication. Exact orientation and bias rules come from the model being implemented.

B.7 Stable softmax

Softmax turns logits z_i into positive values that sum to one:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Direct exponentiation can overflow. Let $m = \max_j z_j$. Then

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}.$$

Subtracting the same constant from every logit does not change the ratio, and the largest exponent is now $e^0 = 1$.

B.8 RMS normalization

One common RMSNorm form is

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{D} \sum_{i=0}^{D-1} x_i^2 + \epsilon}} \odot \gamma.$$

The reduction is often accumulated in wider precision than the stored input. The exact scale convention must be taken from the model definition and weight conversion path; it must not be guessed from the operator name.

B.9 Rotary position embeddings

RoPE treats neighboring features as 2D pairs. At position p , pair j is rotated by angle $\theta_{p,j}$:

$$\begin{aligned} x'_{2j} &= x_{2j} \cos \theta_{p,j} - x_{2j+1} \sin \theta_{p,j}, \\ x'_{2j+1} &= x_{2j} \sin \theta_{p,j} + x_{2j+1} \cos \theta_{p,j}. \end{aligned}$$

The model specifies how frequencies produce $\theta_{p,j}$ and which dimensions are rotated. RoPE is applied to queries and keys, not values.

B.10 Scaled dot-product attention

For query, key, and value matrices with head width d_h ,

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_h}} + M \right) V.$$

M is a mask. In causal attention, locations that would look into the future receive a value that makes their post-softmax probability zero.

This equation describes the result, not a required implementation. A baseline may materialize the score matrix. An online or tiled algorithm can produce the same result while retaining only a small part of it at once.

B.11 Quantization

For a simple symmetric signed quantizer with maximum integer magnitude q_{max} , choose

$$s = \frac{\max_i |w_i|}{q_{max}}, \quad q_i = \text{clip}(\text{round}(w_i/s), -q_{max}, q_{max}).$$

Approximate reconstruction is $\hat{w}_i = sq_i$. A scale can cover one tensor, one output channel, or a smaller group. Smaller groups usually track local ranges better but require more metadata and indexing work.

Low-bit formats are contracts. Signedness, packing order, zero points, group size, scale dtype, and weight orientation must match between converter, loader, and kernel.

B.12 FLOPs, bytes, and arithmetic intensity

Arithmetic intensity is

$$I = \frac{\text{FLOPs performed}}{\text{bytes moved from the limiting memory}}.$$

For a large FP32 dot product, reading two vectors costs about $8N$ bytes and the work is about $2N$ FLOPs, so

$$I \approx \frac{2N}{8N} = 0.25 \text{ FLOP/byte}.$$

The roofline estimate for attainable compute rate is

$$P \leq \min(P_{\text{peak}}, B_{\text{memory}}I).$$

This is a model, not a benchmark result. Small operations may instead be dominated by launch, allocation, cache, or synchronization overhead.

B.13 Latency and throughput

Latency is time for one request or stage. Throughput is work completed per unit time. They are related but not interchangeable.

- Time to first token is strongly affected by prompt prefill.
- Decode tokens per second measures a serial stream of generated tokens.
- Batching can improve total throughput while making an individual request wait.

Always state which quantity and execution stage a number describes.

B.14 Floating-point reductions

Real-number addition is associative; floating-point addition is not:

$$(a + b) + c \neq a + (b + c)$$

for some representable values. Parallel reductions change grouping, so a CPU loop and Metal tree reduction can both be correct without matching bit for bit. Judges therefore use a stated tolerance. A large or directional discrepancy is still a defect to investigate, not something to dismiss automatically as rounding or quantization noise.